Mã sạch

A Handbook of Agile Software Craftsmanship

The Object Mentors:

Robert C. Martin
Michael C. Feathers Timothy R. Ottinger
Jeffrey J. Langr Brett L. Schuchert
James W. Grenning Kevin Dean Wampler
Object Mentor Inc.

Viết mã sạch là điều bạn phải làm để có thể gọi mình là người làm nghề chuyên nghiệp. Không có lý do nào để làm bất cứ điều gì kém hơn mức tốt nhất của mình.

Bản dịch tiếng Việt

Pearson Education

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The authors and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

The publisher offers excellent discounts on this book when ordered in quantity for bulk purchases or special sales, which may include electronic versions and/or custom covers and content particular to your business, training goals, marketing focus, and branding interests. For more information, please contact:

U.S. Corporate and Government Sales
(800) 382-3419
corpsales@pearsontechgroup.com

For sales outside the United States please contact:

International Sales
international@pearsoned.com

Includes bibliographical references and index.
ISBN-13: 978-0-13-235088-4
ISBN-10: 0-13-235088-2

Copyright © 2009 Pearson Education, Inc.

All rights reserved. Printed in the United States of America. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, write to:

Pearson Education, Inc
Rights and Contracts Department
501 Boylston Street, Suite 900
Boston, MA 02116
Fax: (617) 671-3447

ISBN-13: 978-0-13-235088-4
ISBN-10: 0-13-235088-2

Text printed in the United States on recycled paper at Courier in Stoughton, Massachusetts.

First printing, July 2008

Bản quyền bản gốc thuộc Pearson Education. Bản dịch này chỉ dùng cho mục đích học tập cá nhân.

Mục lục

Lời tựa xiii

Lời giới thiệu xviii

Lời cảm ơn xxi

Hình minh họa ..... **xxi**
Về trang bìa xxii

Chương 1: Mã sạch **1**
Sẽ vẫn có code **2**
Mã xấu **2**
Tổng chi phí sở hữu một mớ hỗn độn **4**
Cuộc đại tái thiết kế trên trời 5
Thái độ 5
Nghịch lý căn bản 6
Nghệ thuật của mã sạch? 6
Mã sạch là gì? 7
Các trường phái **13**
Chúng ta là tác giả **14**
Quy tắc Hướng đạo sinh **15**
Tiền truyện và các nguyên lý **16**
Kết luận **16**
Tài liệu tham khảo **16**
Chương 2: Tên có ý nghĩa **17**
Giới thiệu **18**
Dùng tên thể hiện ý định **18**
Tránh tên gây hiểu lầm **19**
Tạo ra những khác biệt có nghĩa **20**
Dùng tên đọc được **21**
Dùng tên tìm kiếm được **22**

Tránh mã hóa trong tên **23**
Ký pháp Hungary 23
Tiền tố cho biến thành viên 24
Giao diện và lớp hiện thực 24
Tránh ánh xạ trong đầu **25**
Tên lớp **25**
Tên phương thức **25**
Đừng tỏ ra dí dỏm **26**
Mỗi khái niệm chọn một từ **26**
Đừng chơi chữ **27**
Dùng tên thuộc miền giải pháp **27**
Dùng tên thuộc miền bài toán **27**
Thêm bối cảnh có nghĩa **28**
Đừng thêm bối cảnh vô cớ **30**
Lời cuối **30**
Chương 3: Hàm **31**
Nhỏ! **34**
Khối lệnh và thụt lề 35
Làm một việc **35**
Các mục trong hàm 36
Mỗi hàm một mức trừu tượng **36**
Đọc mã từ trên xuống: Quy tắc bậc thang 37
Câu lệnh switch **37**
Dùng tên mô tả **39**
Đối số của hàm **40**
Các dạng một đối số phổ biến 41
Đối số cờ 41
Hàm hai đối số 41
Hàm ba đối số 42
Đối tượng đối số 42
Danh sách đối số 43
Động từ và từ khóa 43
Đừng gây tác dụng phụ **43**
Đối số đầu ra 44
Tách lệnh khỏi truy vấn **45**
Ưu tiên ngoại lệ hơn mã lỗi **45**
Tách khối try/catch 46
Xử lý lỗi là một việc 47
Error.java, nam châm hút phụ thuộc 47
Đừng lặp lại chính mình **48**

Lập trình có cấu trúc **48**
Làm thế nào để viết được những hàm như thế này? **49**
Kết luận **49**
SetupTeardownIncluder **50**
Tài liệu tham khảo **52**
Chương 4: Chú thích **53**
Chú thích không bù đắp được cho mã tồi **55**
Tự diễn đạt bằng mã **55**
Chú thích tốt **55**
Chú thích pháp lý 55
Chú thích cung cấp thông tin 56
Giải thích ý định 56
Làm rõ 57
Cảnh báo hậu quả 58
Chú thích TODO 59
Khuếch đại 59
Javadoc trong API công khai 59
Chú thích xấu **60**
Nói lảm nhảm 60
Chú thích dư thừa 60
Chú thích gây hiểu nhầm 63
Chú thích bị bắt buộc 63
Chú thích nhật ký 64
Chú thích ồn ào 64
Tiếng ồn đáng sợ 66
Đừng dùng chú thích khi có thể dùng hàm hay biến 67
Điểm đánh dấu vị trí 67
Chú thích trên ngoặc đóng 67
Ghi công và dòng tác giả 68
Mã bị chú thích hóa 69
Chú thích HTML 69
Thông tin phi cục bộ 70
Quá nhiều thông tin 70
Liên hệ không hiển nhiên 71
Đầu hàm 71
Javadoc trong mã không công khai 71
Ví dụ 71
Tài liệu tham khảo **75**
Chương 5: Định dạng **76**
Mục đích của định dạng **77**

Định dạng theo chiều dọc ... **77**
Ẩn dụ tờ báo ... 78
Độ thoáng dọc giữa các khái niệm ... 79
Độ đặc theo chiều dọc ... 80
Khoảng cách dọc ... 81
Thứ tự dọc ... 85
Định dạng theo chiều ngang ... **86**
Độ thoáng và độ đặc theo chiều ngang ... 87
Căn lề ngang ... 87
Thụt lề ... 89
Thân rỗng ... 90
Quy tắc của nhóm ... **91**
Quy tắc định dạng của Uncle Bob ... **91**

Chương 6: Đối tượng và cấu trúc dữ liệu ... **94**
Trừu tượng hóa dữ liệu ... **95**
Đối tượng và cấu trúc dữ liệu: tính phản đối xứng ... **96**
Định luật Demeter ... **99**
Tàu hỏa trật bánh ... 99
Cấu trúc lai ... 100
Che giấu cấu trúc ... 100
Đối tượng truyền dữ liệu ... **101**
Active Record ... 102
Kết luận ... **103**
Tài liệu tham khảo ... **103**

Chương 7: Xử lý lỗi ... **104**
Dùng ngoại lệ thay cho mã lỗi ... **105**
Viết câu lệnh try-catch-finally trước ... **106**
Dùng ngoại lệ không kiểm tra ... **108**
Cung cấp ngữ cảnh qua ngoại lệ ... **108**
Định nghĩa lớp ngoại lệ theo nhu cầu của bên gọi ... **109**
Định nghĩa luồng bình thường ... **110**
Đừng trả về null ... **111**
Đừng truyền null ... **113**
Kết luận ... **114**
Tài liệu tham khảo ... **114**

Chương 8: Ranh giới ... **115**
Dùng mã của bên thứ ba ... **116**
Khám phá và tìm hiểu ranh giới ... **118**
Tìm hiểu log4j ... **118**

Kiểm thử tìm hiểu có lợi hơn miễn phí **120**
Dùng mã chưa tồn tại **121**
Ranh giới sạch **122**
Tài liệu tham khảo **122**

Chương 9: Kiểm thử đơn vị 123

Ba quy luật của TDD **124**
Giữ cho kiểm thử sạch **125**
Kiểm thử tạo điều kiện cho mọi "-ility" 126
Kiểm thử sạch **126**
Ngôn ngữ kiểm thử chuyên biệt theo miền 129
Hai tiêu chuẩn 129
Mỗi bài kiểm thử một assert **131**
Mỗi bài kiểm thử một khái niệm 132
F.I.R.S.T. **134**
Kết luận **134**
Tài liệu tham khảo **135**

Chương 10: Lớp 136

Tổ chức lớp **137**
Đóng gói 137
Lớp phải nhỏ! **137**
Nguyên tắc trách nhiệm đơn nhất 139
Độ gắn kết 141
Duy trì độ gắn kết tạo ra nhiều lớp nhỏ 142
Tổ chức để thay đổi **147**
Cách ly khỏi thay đổi 150
Tài liệu tham khảo **151**

Chương 11: Hệ thống 152

Bạn sẽ xây dựng một thành phố như thế nào? **153**
Tách việc xây dựng hệ thống khỏi việc sử dụng hệ thống **153**
Tách biệt hàm main 154
Factory 155
Tiêm phụ thuộc 156
Mở rộng quy mô **156**
Mối quan tâm cắt ngang 159
Proxy Java **160**
Framework AOP thuần Java **162**
Khía cạnh AspectJ **165**
Phát triển kiến trúc hệ thống theo hướng kiểm thử **165**
Tối ưu việc ra quyết định **166**

Dùng tiêu chuẩn một cách khôn ngoan, khi chúng đem lại giá trị rõ ràng 167
Hệ thống cần ngôn ngữ chuyên biệt cho miền 167
Kết luận 168
Tài liệu tham khảo 168

Chương 12: Thiết kế nổi lên 169
Thiết kế sạch nhờ thiết kế nổi lên 169
Quy tắc Thiết kế Đơn giản 1: Chạy qua tất cả các kiểm thử 170
Quy tắc Thiết kế Đơn giản 2–4: Tái cấu trúc 170
Không trùng lặp 171
Biểu đạt 173
Tối thiểu hóa số lớp và phương thức 174
Kết luận 174
Tài liệu tham khảo 174

Chương 13: Xử lý đồng thời 175
Vì sao cần xử lý đồng thời? 176
Những lầm tưởng và quan niệm sai 177
Thách thức 177
Các nguyên tắc phòng vệ khi xử lý đồng thời 178
Nguyên tắc trách nhiệm đơn 179
Hệ quả: Giới hạn phạm vi dữ liệu 179
Hệ quả: Dùng bản sao dữ liệu 179
Hệ quả: Các luồng nên độc lập tối đa 180
Hiểu rõ thư viện của bạn 180
Collection an toàn luồng 180
Hiểu rõ các mô hình thực thi 181
Nhà sản xuất-Người tiêu thụ 181
Người đọc-Người ghi 181
Các triết gia ăn tối 182
Cẩn thận với phụ thuộc giữa các phương thức đồng bộ 182
Giữ các vùng đồng bộ nhỏ 183
Viết mã tắt hệ thống đúng cách rất khó 183
Kiểm thử mã đa luồng 184
Xem các lỗi chập chờn là dấu hiệu có thể liên quan đến luồng 184
Trước hết, hãy làm cho mã không đa luồng hoạt động đúng 184
Làm cho mã đa luồng có thể thay thế linh hoạt 185
Làm cho mã đa luồng có thể điều chỉnh 185
Chạy với số luồng nhiều hơn số bộ xử lý 185
Chạy trên các nền tảng khác nhau 185
Gắn thiết bị đo đạc vào mã để cố ép lỗi xuất hiện 186

Viết thủ công 186
Tự động hóa 187
Kết luận 188
Tài liệu tham khảo 189
Chương 14: Tinh chỉnh từng bước 190
Cài đặt Args 191
Tôi đã làm việc này như thế nào? 197
Args: bản nháp đầu tiên 198
Thế là tôi dừng lại 208
Từng bước một 209
Đối số kiểu chuỗi 211
Kết luận 247
Chương 15: Bên trong JUnit 248
Framework JUnit 249
Kết luận 262
Chương 16: Tái cấu trúc SerialDate 263
Trước hết, làm cho nó hoạt động 264
Sau đó, làm cho nó đúng đắn 266
Kết luận 279
Tài liệu tham khảo 279
Chương 17: Mùi code và nguyên tắc kinh nghiệm 280
Chú thích 281
C1: Thông tin không phù hợp 281
C2: Chú thích lỗi thời 281
C3: Chú thích thừa 281
C4: Chú thích viết kém 282
C5: Mã bị chú thích vô hiệu hóa 282
Môi trường 282
E1: Build cần nhiều hơn một bước 282
E2: Kiểm thử cần nhiều hơn một bước 282
Hàm 283
F1: Quá nhiều đối số 283
F2: Đối số đầu ra 283
F3: Đối số cờ 283
F4: Hàm chết 283
Tổng quát 283
G1: Nhiều ngôn ngữ trong cùng một tệp mã nguồn 283
G2: Hành vi hiển nhiên chưa được triển khai 283
G3: Hành vi sai ở các ranh giới 284

G4: Vô hiệu hóa các cơ chế an toàn 284
G5: Trùng lặp 284
G6: Mã ở sai mức trừu tượng 285
G7: Lớp cơ sở phụ thuộc vào lớp dẫn xuất 286
G8: Quá nhiều thông tin 287
G9: Mã chết 287
G10: Tách biệt theo chiều dọc 287
G11: Không nhất quán 288
G12: Rác 288
G13: Tạo sự phụ thuộc nhân tạo 288
G14: Hàm mê dữ liệu của lớp khác 288
G15: Đối số chọn hành vi 289
G16: Ý định bị che khuất 290
G17: Trách nhiệm đặt sai chỗ 291
G18: Dùng static không phù hợp 291
G19: Dùng biến diễn giải 292
G20: Tên hàm phải nói rõ hàm làm gì 292
G21: Hiểu thuật toán 292
G22: Biến sự phụ thuộc logic thành phụ thuộc cụ thể 293
G23: Ưu tiên đa hình thay cho if/else hoặc switch/case 295
G24: Tuân theo quy ước chuẩn 295
G25: Thay số ma thuật bằng hằng có tên 295
G26: Chính xác 296
G27: Cấu trúc thay cho quy ước 297
G28: Đóng gói điều kiện 297
G29: Tránh điều kiện phủ định 297
G30: Mỗi hàm chỉ nên làm một việc 297
G31: Sự phụ thuộc thời điểm bị che giấu 298
G32: Đừng tùy tiện 299
G33: Đóng gói điều kiện biên 299
G34: Hàm chỉ nên đi xuống một mức trừu tượng 300
G35: Giữ dữ liệu cấu hình ở mức cao 301
G36: Tránh điều hướng bắc cầu 302
Java 302
J1: Tránh danh sách import dài bằng cách dùng ký tự đại diện 302
J2: Đừng kế thừa hằng 303
J3: Hằng hay enum 304
Tên 305
N1: Chọn tên có tính mô tả 305
N2: Chọn tên phù hợp với mức trừu tượng 306
N3: Dùng thuật ngữ chuẩn khi có thể 307

N4: Tên không mơ hồ 307
N5: Dùng tên dài cho phạm vi dài 308
N6: Tránh mã hóa tên 308
N7: Tên nên mô tả tác dụng phụ 308
Kiểm thử 309
T1: Kiểm thử chưa đầy đủ 309
T2: Dùng công cụ đo độ bao phủ! 309
T3: Đừng bỏ qua kiểm thử đơn giản 309
T4: Kiểm thử bị bỏ qua là câu hỏi về một điểm mơ hồ 309
T5: Kiểm thử điều kiện ranh giới 309
T6: Kiểm thử kỹ quanh lỗi 310
T7: Mẫu lỗi tiết lộ nhiều điều 310
T8: Mẫu độ bao phủ kiểm thử có thể tiết lộ nhiều điều 310
T9: Kiểm thử phải nhanh 310
Kết luận 310
Tài liệu tham khảo 311
Phụ lục A: Xử lý đồng thời II 312
Ví dụ client/server 312
Server 312
Thêm luồng 314
Nhận xét về server 314
Kết luận 316
Các đường thực thi khả dĩ 316
Số lượng đường thực thi 317
Tìm hiểu sâu hơn 318
Kết luận 320
Hiểu thư viện của bạn 320
Framework Executor 321
Giải pháp không chặn 321
Các lớp không an toàn với luồng 323
Phụ thuộc giữa các phương thức có thể làm hỏng mã đồng thời 324
Chấp nhận lỗi 325
Khóa phía máy khách 325
Khóa phía máy chủ 326
Tăng thông lượng 327
Tính thông lượng khi tính toán bằng một luồng 329
Tính thông lượng khi dùng nhiều luồng 329
Bế tắc 330
Loại trừ tương hỗ 331
Khóa và chờ 331

Không thể thu hồi 331
Chờ vòng tròn 331
Phá vỡ điều kiện loại trừ tương hỗ 331
Phá vỡ điều kiện khóa và chờ 332
Phá vỡ điều kiện không thể thu hồi 332
Phá vỡ điều kiện chờ vòng tròn 333
Kiểm thử mã đa luồng 333
Công cụ hỗ trợ kiểm thử mã dựa trên luồng 336
Kết luận 336
Hướng dẫn: Ví dụ mã đầy đủ 337
Máy khách/máy chủ không dùng luồng 337
Máy khách/máy chủ dùng luồng 340
Phụ lục B: org.jfree.date.SerialDate 342
Phụ lục C: Tham chiếu chéo các nguyên tắc 410
Lời kết 413
Về bản dịch 414

Lời tựa

Tác giả: James O. Coplien, Mørdrup, Đan Mạch

Ở Đan Mạch, một trong những viên kẹo chúng tôi thích nhất là Ga-Jol. Mùi cam thảo nồng của nó hợp một cách hoàn hảo với tiết trời ẩm ướt và thường lạnh lẽo nơi đây. Với người Đan Mạch chúng tôi, một phần duyên dáng của Ga-Jol nằm ở những câu danh ngôn thông thái hoặc dí dỏm in trên nắp mỗi hộp. Sáng nay tôi mua một gói hai viên và thấy trên đó in câu ngạn ngữ Đan Mạch cổ này:

Ærlighed i små ting er ikke nogen lille ting.

("Trung thực trong những việc nhỏ không phải là việc nhỏ.")

Đó là điềm lành, hợp với điều tôi vốn muốn nói ở đây: những việc nhỏ có ý nghĩa. Cuốn sách này nói về những mối quan tâm khiêm nhường nhưng giá trị chẳng hề nhỏ.

"Thượng đế nằm trong chi tiết", kiến trúc sư Ludwig Mies van der Rohe từng nói. Câu nói ấy gợi lại những tranh luận hiện nay về vai trò của kiến trúc trong phát triển phần mềm, nhất là trong thế giới Agile. Bob và tôi đôi khi say sưa tham gia cuộc đối thoại này. Mies van der Rohe quả thật coi trọng công năng và những hình thức xây dựng vượt thời gian, nền tảng của kiến trúc lớn. Ông còn tự tay chọn từng tay nắm cửa cho mọi ngôi nhà mình thiết kế. Vì sao? Vì những việc nhỏ có ý nghĩa.

Trong cuộc "tranh luận" vẫn tiếp diễn về TDD, Bob và tôi nhận ra rằng cả hai đều đồng ý: kiến trúc phần mềm có vị trí quan trọng trong quá trình phát triển, dù có lẽ chúng tôi hình dung khác nhau về ý nghĩa cụ thể của nó. Những bất đồng như vậy không mấy quan trọng, bởi ta có thể mặc nhiên cho rằng người làm nghề có trách nhiệm sẽ dành thời gian suy nghĩ và lập kế hoạch từ đầu dự án. Quan niệm cuối thập niên 1990, rằng thiết kế chỉ do kiểm thử và mã dẫn dắt, đã qua lâu rồi. Dẫu vậy, chú tâm đến chi tiết còn là nền tảng quan trọng hơn mọi tầm nhìn lớn lao của tính chuyên nghiệp. Trước hết, nhờ thực hành những việc nhỏ, người làm nghề mới có được sự thành thạo và niềm tin để làm việc lớn. Rồi chỉ một chút cẩu thả trong thi công, cánh cửa khép không kín, viên gạch lát hơi lệch, thậm chí chiếc bàn làm việc bừa bộn, cũng đủ làm mất vẻ duyên dáng của toàn thể. Đó là điều mã sạch hướng tới.

Tuy vậy, kiến trúc chỉ là một ẩn dụ cho phát triển phần mềm, nhất là cho phần tạo ra sản phẩm ban đầu, như kiến trúc sư tạo nên một tòa nhà tinh khôi. Trong thời đại Scrum và Agile, người ta chú trọng đưa sản phẩm ra thị trường thật nhanh. Ta muốn nhà máy chạy

hết tốc lực để sản xuất phần mềm. Nhưng đây là những nhà máy của con người: những người viết mã biết suy nghĩ, biết cảm nhận, làm việc từ product backlog hoặc user story để tạo ra sản phẩm. Ẩn dụ sản xuất vẫn in đậm trong lối nghĩ ấy. Scrum chịu ảnh hưởng nhiều từ khía cạnh sản xuất của ngành ô tô Nhật Bản, từ thế giới của dây chuyền lắp ráp.

Thế nhưng ngay trong ngành ô tô, phần lớn công việc là bảo trì, hoặc tìm cách tránh phải bảo trì, chứ không phải sản xuất. Trong phần mềm, 80% công việc chúng ta làm, thậm chí hơn, vẫn được gọi theo cách hơi cổ là "bảo trì": sửa chữa. Thay vì theo lối nghĩ phương Tây điển hình, tập trung sản xuất phần mềm tốt, có lẽ chúng ta nên nghĩ như những người thợ sửa nhà trong ngành xây dựng hay thợ sửa ô tô. Các nhà quản lý Nhật Bản nói gì về chuyện đó?

Khoảng năm 1951, tại Nhật Bản xuất hiện một phương pháp chất lượng mang tên Total Productive Maintenance (TPM). Phương pháp này tập trung vào bảo trì thay vì sản xuất. Một trong những trụ cột chính của TPM là bộ nguyên tắc gọi là 5S. Đó là một tập hợp kỷ luật, và ở đây tôi dùng từ "kỷ luật" với ý nghĩa giáo dục. Những nguyên tắc 5S này nằm ngay nền tảng của Lean, một từ thời thượng khác ở phương Tây và ngày càng phổ biến trong giới phần mềm. Những nguyên tắc ấy không phải thứ tùy chọn. Như Uncle Bob kể ở phần đầu sách, thực hành phần mềm tốt đòi hỏi kỷ luật: tập trung, tỉnh táo và suy nghĩ. Công việc không phải lúc nào cũng chỉ là làm, hay thúc thiết bị nhà máy chạy ở tốc độ tối ưu. Triết lý 5S gồm những khái niệm sau:

- Seiri, tức sắp xếp (liên tưởng đến từ "sort" trong tiếng Anh). Cần biết mọi thứ ở đâu, chẳng hạn nhờ đặt tên phù hợp. Bạn nghĩ tên định danh không quan trọng ư? Cứ đọc tiếp những chương sau.
- Seiton, tức ngăn nắp (liên tưởng đến "systematize" trong tiếng Anh). Người Mỹ có câu: "Mỗi thứ một nơi, mỗi nơi một thứ." Một đoạn mã nên nằm đúng chỗ bạn chờ đợi tìm thấy nó. Nếu không, hãy tái cấu trúc để đưa nó về đó.
- Seiso, tức dọn dẹp (liên tưởng đến "shine" trong tiếng Anh): giữ nơi làm việc không vướng dây dợ, dầu mỡ, mảnh vụn và rác. Các tác giả cuốn này nói gì về những chú thích và dòng mã bị vô hiệu hóa bằng chú thích, được để lại nhằm lưu giữ lịch sử hoặc gửi gắm mong muốn cho tương lai? Hãy xóa chúng đi.
- Seiketsu, tức tiêu chuẩn hóa: cả nhóm thống nhất cách giữ nơi làm việc sạch sẽ. Bạn nghĩ cuốn sách này có nói đến phong cách viết mã và các thực hành nhất quán trong nhóm không? Những tiêu chuẩn ấy từ đâu mà có? Cứ đọc tiếp.
- Shutsuke, tức kỷ luật (tự kỷ luật): tự giác tuân theo các thực hành, thường xuyên nhìn lại công việc và sẵn lòng thay đổi.

Nếu nhận lời thách thức, đúng vậy, thách thức, đọc và áp dụng cuốn sách này, bạn sẽ hiểu và trân trọng nguyên tắc cuối cùng. Đến đây, chúng ta chạm tới gốc rễ của tính chuyên nghiệp có trách nhiệm trong một nghề cần quan tâm đến vòng đời sản phẩm. Theo TPM, khi bảo trì ô tô và các máy móc khác, sửa chữa sau khi hỏng, tức chờ lỗi xuất hiện, là chuyện ngoại lệ. Thay vào đó, ta nâng thêm một bậc: kiểm tra máy móc hằng ngày, sửa những bộ phận đang mòn trước khi chúng gãy, hoặc làm việc tương tự như thay dầu sau mỗi 10.000 dặm để ngăn hao mòn. Với mã, hãy tái cấu trúc không thương tiếc. Ta còn có thể tiến thêm một

bậc nữa, như phong trào TPM đã đổi mới hơn 50 năm trước: ngay từ đầu, hãy chế tạo máy móc dễ bảo trì hơn. Làm cho mã dễ đọc quan trọng ngang với làm cho mã chạy được. Thực hành cao nhất, được giới TPM khởi xướng khoảng năm 1960, là tập trung đưa vào máy móc hoàn toàn mới hoặc thay máy cũ. Fred Brooks nhắc chúng ta rằng có lẽ cứ khoảng bảy năm nên làm lại từ đầu những phần mềm lớn để quét sạch đống rác tích tụ dần. Có lẽ nên đổi hằng số thời gian của Brooks từ năm sang tuần, ngày hoặc giờ. Chi tiết nằm ở đó.

Có sức mạnh lớn trong chi tiết, nhưng cách sống ấy cũng có nét khiêm nhường và sâu sắc, đúng như định kiến thường gán cho bất kỳ lối tiếp cận nào nhận mình có gốc gác Nhật Bản. Song đây không chỉ là cách nhìn phương Đông. Kho tàng dân gian Anh và Mỹ cũng đầy những lời răn như thế. Câu nói về Seiton ở trên do một mục sư ở Ohio viết; ông thật sự xem sự ngăn nắp "là phương thuốc cho mọi mức độ của cái ác". Còn Seiso thì sao? Sạch sẽ đứng gần sự thánh thiện. Ngôi nhà có đẹp đến đâu, chiếc bàn bừa bộn vẫn làm mất vẻ huy hoàng của nó. Còn Shutsuke trong những việc nhỏ? Ai trung tín trong việc nhỏ thì cũng trung tín trong việc lớn. Còn chuyện hăng hái tái cấu trúc đúng lúc, để có vị thế tốt hơn cho những quyết định "lớn" sau này, thay vì trì hoãn? Một mũi khâu kịp lúc đỡ chín mũi khâu về sau. Chim dậy sớm bắt được sâu. Việc gì làm được hôm nay chớ để đến mai. (Đó là nghĩa ban đầu của cụm "thời điểm chịu trách nhiệm cuối cùng" trong Lean, trước khi rơi vào tay các nhà tư vấn phần mềm.) Còn chuyện đặt những nỗ lực nhỏ, riêng lẻ vào đúng vị trí trong tổng thể lớn? Cây sồi vĩ đại mọc từ những hạt sồi bé nhỏ. Hay đưa những việc phòng ngừa đơn giản vào sinh hoạt hằng ngày? Phòng bệnh hơn chữa bệnh. Mỗi ngày một quả táo, bác sĩ ở xa. Mã sạch trân trọng những cội rễ trí tuệ sâu xa trong nền văn hóa rộng lớn của chúng ta, hoặc nền văn hóa từng có, nên có và có thể có, nếu ta chú tâm đến chi tiết.

Ngay cả trong các tác phẩm lớn về kiến trúc, ta cũng gặp những câu ngạn ngữ nhắc đến những chi tiết tưởng như vụn vặt ấy. Hãy nghĩ đến tay nắm cửa của Mies van der Rohe. Đó là seiri, là chú tâm đến từng tên biến. Hãy đặt tên biến cẩn trọng như khi đặt tên cho đứa con đầu lòng.

Như mọi chủ nhà đều biết, sự chăm chút và trau chuốt ấy chẳng bao giờ kết thúc. Kiến trúc sư Christopher Alexander, cha đẻ của các pattern và pattern language, xem mỗi hành vi thiết kế là một lần sửa chữa nhỏ, mang tính cục bộ. Ông cho rằng tay nghề tạo nên kết cấu tinh xảo thuộc riêng về kiến trúc sư; những hình thức lớn hơn có thể được định hình bằng các pattern và cách cư dân áp dụng chúng. Thiết kế luôn tiếp diễn: ta không chỉ thêm phòng mới mà còn sơn sửa, thay thảm đã mòn hay nâng cấp bồn rửa trong bếp. Nhiều loại hình nghệ thuật cũng có những quan niệm tương tự. Trong lúc tìm những người cùng cho rằng Chúa ngự trong chi tiết, chúng ta gặp nhà văn Pháp thế kỷ 19 Gustav Flaubert. Nhà thơ Pháp Paul Valery khuyên rằng bài thơ không bao giờ hoàn tất, luôn cần được sửa lại; ngừng làm việc trên nó là bỏ rơi nó. Mối bận tâm đến chi tiết có mặt trong mọi nỗ lực hướng tới sự xuất sắc. Có lẽ chẳng có gì mới ở đây, nhưng đọc cuốn sách này, bạn sẽ được thách thức quay lại với những kỷ luật tốt đẹp mà từ lâu mình đã buông xuôi, vì thờ ơ hoặc vì muốn được tự phát và chỉ "phản ứng với thay đổi".

Đáng tiếc, ta thường không xem những mối quan tâm ấy là nền tảng của nghệ thuật lập trình. Ta bỏ mã của mình quá sớm, không phải vì nó đã xong, mà vì hệ giá trị chú trọng vẻ ngoài hơn thực chất của thứ ta bàn giao. Sự lơ là ấy cuối cùng cũng khiến ta phải trả giá:

đồng tiền xấu rồi cũng lộ mặt. Cả nghiên cứu trong ngành lẫn trong học thuật đều không hạ mình xuống việc giữ mã sạch, vốn bị xem là thấp kém. Khi còn làm ở tổ chức Software Production Research của Bell Labs (đúng là nghiên cứu về sản xuất!), chúng tôi có vài kết quả tính nhẩm cho thấy phong cách thụt lề nhất quán là một trong những chỉ báo có ý nghĩa thống kê mạnh nhất về mật độ lỗi thấp. Ta muốn tin rằng chất lượng đến từ kiến trúc, ngôn ngữ lập trình hay một ý niệm cao siêu nào khác. Tự cho rằng tính chuyên nghiệp của mình dựa vào sự thành thạo công cụ và phương pháp thiết kế cao siêu, ta thấy bị xúc phạm khi những người viết mã dưới sàn xưởng góp phần nâng cao chất lượng chỉ bằng cách áp dụng nhất quán một phong cách thụt lề. Nhắc lại cuốn sách tôi viết 17 năm trước, phong cách ấy phân biệt sự xuất sắc với mức đủ dùng. Thế giới quan Nhật Bản hiểu giá trị cốt yếu của người lao động thường ngày, và hơn thế, của những hệ thống phát triển nhờ hành động giản dị hằng ngày của họ. Chất lượng là kết quả của hàng triệu hành động chăm chút vô tư, chứ không chỉ nhờ một phương pháp vĩ đại nào từ trên trời rơi xuống. Những hành động ấy giản dị không có nghĩa là đơn giản hóa quá mức, càng không có nghĩa là dễ dàng. Dẫu vậy, chúng làm nên sự vĩ đại, và cả cái đẹp, trong mọi nỗ lực của con người. Bỏ qua chúng là chưa sống trọn vẹn như một con người.

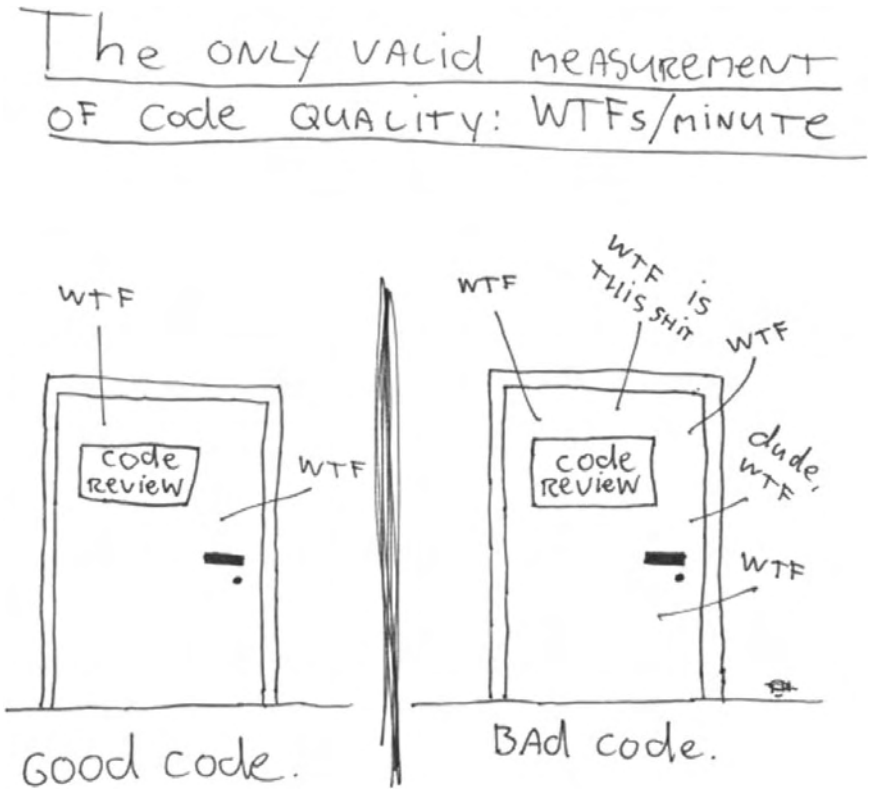
Dĩ nhiên, tôi vẫn ủng hộ việc suy nghĩ ở phạm vi rộng hơn, nhất là những cách tiếp cận kiến trúc bắt nguồn từ hiểu biết sâu về miền nghiệp vụ và khả năng sử dụng phần mềm. Cuốn sách này không bàn về những điều đó, hoặc ít nhất là không nói rõ. Thông điệp của sách tinh tế hơn và không nên xem nhẹ chiều sâu của nó. Thông điệp ấy gần với câu cửa miệng của những người thật sự gắn bó với code như Peter Sommerlad, Kevlin Henney và Giovanni Asproni: “Code là thiết kế” và “Code đơn giản”. Chúng ta cần nhớ giao diện chính là chương trình và cấu trúc của nó nói lên nhiều điều về cấu trúc chương trình. Quan trọng là phải luôn khiêm nhường, thừa nhận thiết kế nằm trong code. Nếu làm lại trong ẩn dụ sản xuất dẫn đến chi phí, thì làm lại trong thiết kế tạo ra giá trị. Hãy xem code như cách diễn đạt đẹp đẽ của những nỗ lực thiết kế cao quý, trong đó thiết kế là một quá trình chứ không phải điểm kết thúc tĩnh tại. Các thước đo kiến trúc về sự phụ thuộc và độ gắn kết thể hiện ngay trong code. Nghe Larry Constantine nói về sự phụ thuộc và độ gắn kết, bạn sẽ thấy ông dùng ngôn ngữ của code, chứ không phải những khái niệm trừu tượng cao siêu trong UML. Trong tiểu luận “Abstraction Descant”, Richard Gabriel khuyên rằng trừu tượng hóa là điều xấu. Code chống lại cái xấu, và mã sạch có lẽ là thần thánh.

Trở lại hộp kẹo Ga-Jol nhỏ của tôi, cần nhớ rằng trí tuệ Đan Mạch khuyên ta không chỉ chú ý đến việc nhỏ mà còn phải trung thực trong việc nhỏ. Hãy trung thực với code, với đồng nghiệp về tình trạng code, và nhất là với chính mình. Ta đã “Làm Hết Sức Mình” để “rời khu cắm trại sạch hơn lúc đến” chưa? Ta đã tái cấu trúc code trước khi check in chưa? Đây là những mối quan tâm nằm ngay trung tâm các giá trị Agile. Scrum khuyến nghị đưa tái cấu trúc vào khái niệm “Done”. Cả kiến trúc lẫn mã sạch đều không đòi hỏi hoàn hảo, chỉ đòi hỏi sự trung thực và nỗ lực hết sức. Sai lầm là chuyện con người; tha thứ là chuyện thần thánh. Trong Scrum, ta làm mọi thứ hiển lộ, phơi bày cả những điều chưa ổn. Ta thành thật về tình trạng code, vì code chẳng bao giờ hoàn hảo. Nhờ vậy, ta sống trọn vẹn hơn với tư cách con người, xứng đáng với điều thần thánh hơn và gần hơn với sự vĩ đại trong chi tiết.

Trong nghề này, chúng ta cần mọi sự trợ giúp có thể có. Nếu sàn xưởng sạch giúp giảm tai nạn và dụng cụ được sắp xếp ngăn nắp giúp tăng năng suất, tôi hoàn toàn ủng hộ. Còn cuốn sách này, đây là ứng dụng thực tiễn nhất các nguyên tắc Lean vào phần mềm mà tôi từng thấy được in thành sách. Tôi không mong đợi gì ít hơn ở nhóm nhỏ những người thực tế, biết suy nghĩ này. Suốt nhiều năm, họ cùng nhau nỗ lực để tiến bộ và trao lại kiến thức cho ngành qua những tác phẩm như cuốn sách bạn đang cầm trên tay. Cuốn sách khiến thế giới tốt đẹp hơn một chút so với lúc Uncle Bob gửi tôi bản thảo.

Sau khi hoàn thành bài tập về những tuệ giác cao siêu này, tôi đi dọn bàn làm việc đây.

Lời giới thiệu



Hình minh họa: (c) 2008 Focus Shift. Được sử dụng lại với sự cho phép của Thom Holwerda. `*http://www.osnews.com/story/19266/WTFs_m*`

Cánh cửa nào đại diện cho code của bạn? Cánh cửa nào đại diện cho nhóm hay công ty của bạn? Vì sao chúng ta ở trong căn phòng đó? Đây chỉ là buổi review mã bình thường, hay chúng ta vừa phát hiện hàng loạt vấn đề nghiêm trọng ngay sau khi hệ thống đi vào hoạt động? Có phải chúng ta đang cuống cuồng gỡ lỗi, chăm chú nhìn đoạn mã tưởng rằng

đã chạy được? Khách hàng đang lũ lượt bỏ đi, còn quản lý thì thở sát gáy chúng ta chăng? Làm sao biết khi mọi chuyện trở nên khó khăn, chúng ta sẽ đứng sau cánh cửa đúng? Câu trả lời là: tay nghề.

Học tay nghề gồm hai phần: kiến thức và thực hành. Bạn cần nắm những nguyên tắc, pattern, thực hành và nguyên tắc kinh nghiệm mà một người thợ lành nghề biết. Rồi còn phải mài giũa kiến thức ấy trong ngón tay, đôi mắt và ruột gan bằng cách làm việc chăm chỉ, luyện tập.

Tôi có thể dạy bạn vật lý của việc đi xe đạp. Thật vậy, toán học cổ điển đằng sau nó khá dễ hiểu. Trọng lực, ma sát, mô men động lượng, khối tâm, vân vân, đều có thể giải thích bằng chưa đầy một trang phương trình. Dựa vào những công thức ấy, tôi có thể chứng minh việc đi xe đạp là khả thi và cung cấp cho bạn mọi kiến thức cần thiết. Vậy mà lần đầu trèo lên xe, bạn vẫn sẽ ngã.

Viết code cũng vậy. Chúng tôi có thể ghi ra mọi nguyên tắc "cảm thấy dễ chịu" của mã sạch rồi tin rằng bạn sẽ tự lo phần còn lại, tức là để bạn ngã khi trèo lên xe. Nếu thế, chúng tôi là thầy kiểu gì, còn bạn là trò kiểu gì?

Không. Cuốn sách này không làm theo cách đó.

Học viết mã sạch là việc khó. Biết các nguyên tắc và pattern thôi chưa đủ. Bạn phải đổ mồ hôi vì nó, tự thực hành và nhìn mình thất bại. Bạn phải xem người khác thực hành rồi thất bại, thấy họ vấp ngã và lần lại từng bước. Bạn cũng phải thấy họ dằn vặt trước những quyết định, rồi chứng kiến cái giá họ trả khi chọn sai.

Hãy chuẩn bị làm việc chăm chỉ khi đọc cuốn sách này. Đây không phải cuốn sách "cảm thấy dễ chịu" để bạn đọc trên máy bay rồi đọc xong trước khi hạ cánh. Cuốn sách sẽ bắt bạn làm việc, và làm việc vất vả. Bạn sẽ làm gì? Đọc code, thật nhiều code. Bạn sẽ phải suy nghĩ xem đoạn code ấy đúng ở đâu, sai ở đâu. Bạn cũng sẽ theo dõi khi chúng tôi tháo rời các module rồi lắp lại. Việc này tốn thời gian và công sức, nhưng chúng tôi nghĩ xứng đáng.

Chúng tôi chia cuốn sách thành ba phần. Vài chương đầu trình bày các nguyên tắc, pattern và thực hành để viết mã sạch. Những chương này có khá nhiều code và đọc sẽ không dễ. Chúng chuẩn bị cho bạn bước vào phần hai. Nếu đọc xong phần một mà bạn gấp sách lại thì chúc may mắn!

Phần hai là phần khó nhọc hơn. Phần này gồm vài nghiên cứu tình huống có độ phức tạp tăng dần. Mỗi nghiên cứu là một bài tập dọn dẹp code, biến đoạn code còn nhiều vấn đề thành đoạn code ít vấn đề hơn. Phần này rất chi tiết. Bạn sẽ phải lật qua lật lại giữa lời kể và các đoạn mã, phân tích và hiểu code chúng tôi đang xử lý, rồi theo dõi lý do đằng sau từng thay đổi. Hãy dành thời gian, vì việc này có thể kéo dài vài ngày.

Phần ba là phần đền đáp: một chương tập hợp các nguyên tắc kinh nghiệm và mùi code rút ra trong quá trình thực hiện các nghiên cứu tình huống. Khi lần lượt dọn code trong những nghiên cứu ấy, chúng tôi ghi lại mọi lý do cho hành động của mình dưới dạng nguyên tắc kinh nghiệm hoặc mùi code. Chúng tôi cố hiểu phản ứng của chính mình khi đọc và sửa code, rồi tìm cách ghi lại vì sao mình cảm thấy và hành động như vậy. Kết quả là một cơ sở tri thức mô tả cách chúng tôi nghĩ khi viết, đọc và dọn code.

Cơ sở tri thức này chỉ có giá trị hạn chế nếu bạn không đọc kỹ các nghiên cứu tình

huống ở phần hai. Trong đó, chúng tôi cẩn thận chú thích từng thay đổi bằng tham chiếu đến các nguyên tắc kinh nghiệm sẽ trình bày sau. Những tham chiếu này nằm trong ngoặc vuông, như [H22]. Nhờ vậy, bạn thấy được bối cảnh các nguyên tắc ấy được áp dụng và hình thành. Giá trị nằm ở mối liên hệ giữa các nguyên tắc và những quyết định cụ thể chúng tôi đưa ra khi dọn code trong các nghiên cứu tình huống, chứ không nằm ở bản thân các nguyên tắc.

Để giúp bạn hiểu những mối liên hệ đó, cuối sách có bảng tham chiếu chéo ghi số trang của mọi tham chiếu. Bạn có thể tra bảng để tìm từng nơi một nguyên tắc được áp dụng.

Nếu chỉ đọc phần một và phần ba rồi bỏ qua các nghiên cứu tình huống, bạn sẽ lại đọc thêm một cuốn sách "cảm thấy dễ chịu" về cách viết phần mềm tốt. Nhưng nếu dành thời gian lần theo từng bước nhỏ, từng quyết định nhỏ trong các nghiên cứu, đặt mình vào vị trí của chúng tôi và buộc mình suy nghĩ theo đúng những con đường chúng tôi đã đi, bạn sẽ hiểu sâu hơn nhiều về các nguyên tắc, pattern, thực hành và nguyên tắc kinh nghiệm ấy. Chúng sẽ không còn là thứ kiến thức "cảm thấy dễ chịu" nữa. Chúng sẽ ngấm vào ruột gan, ngón tay và trái tim bạn, trở thành một phần con người bạn, như chiếc xe đạp trở thành phần nối dài ý chí khi bạn đã thành thạo cách đi.

Lời cảm ơn

Hình minh họa

Cảm ơn hai họa sĩ của tôi, Jennifer Kohnke và Angela Brooks. Jennifer chịu trách nhiệm về những bức tranh tuyệt đẹp và giàu sáng tạo ở đầu mỗi chương, cùng các bức chân dung Kent Beck, Ward Cunningham, Bjarne Stroustrup, Ron Jeffries, Grady Booch, Dave Thomas, Michael Feathers và chính tôi.

Angela vẽ những bức tranh dí dỏm trang trí bên trong mỗi chương. Suốt nhiều năm, cô đã vẽ cho tôi khá nhiều tranh, trong đó có nhiều bức bên trong cuốn Agile Software Development: Principles, Patterns, and Practices. Cô cũng là đứa con đầu lòng của tôi, đứa con mà tôi vô cùng hài lòng.

Về trang bìa

Hình trên bìa là M104, Thiên hà Sombrero. M104 nằm trong chòm Xử Nữ (Virgo), cách chúng ta chưa đầy 30 triệu năm ánh sáng. Ở tâm thiên hà là một lỗ đen siêu khối lượng, có khối lượng khoảng một tỷ lần khối lượng Mặt Trời.

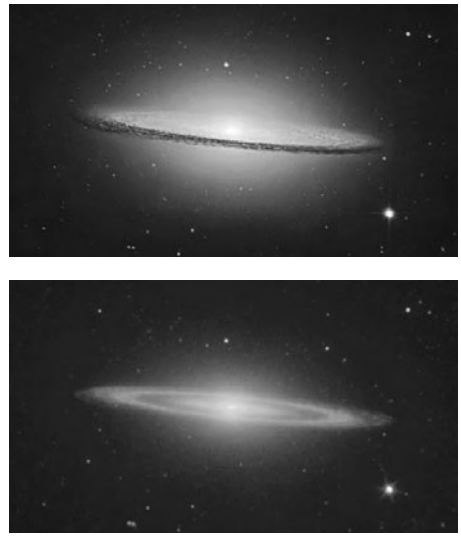
Hình ảnh có gợi bạn nhớ đến vụ nổ của mặt trăng năng lượng Praxis của người Klingon không? Tôi nhớ rất rõ cảnh trong Star Trek VI, khi một vành đai mảnh vỡ bay ra từ vụ nổ ấy ở vùng xích đạo. Từ đó, vành đai xích đạo trở thành hình ảnh quen thuộc trong các vụ nổ phim khoa học viễn tưởng. Chi tiết ấy thậm chí còn được thêm vào vụ nổ Alderaan trong những bản phát hành sau này của bộ phim Star Wars đầu tiên.

Điều gì tạo nên vành đai quanh M104? Vì sao thiên hà có phần phình trung tâm lớn đến vậy nhưng nhân lại nhỏ và sáng đến thế? Với tôi, trông như thể lỗ đen trung tâm mất bình tĩnh rồi thổi thủng một lỗ rộng 30.000 năm ánh sáng giữa thiên hà. Khốn khổ cho bất kỳ nền văn minh nào có thể từng nằm trên đường đi của cơn biến động vũ trụ ấy.

Lỗ đen siêu khối lượng nuốt trọn cả những ngôi sao vào bữa trưa, chuyển một phần đáng kể khối lượng của chúng thành năng lượng. E = MC2 đã đủ tạo ra sức mạnh lớn, nhưng khi M là khối lượng một ngôi sao thì hãy coi chừng! Bao nhiêu ngôi sao đã lao đầu vào cái mõm ấy trước khi con quái vật no nê? Kích thước khoảng trống ở tâm thiên hà có thể là một gợi ý chăng?

Hình M104 trên bìa là sự kết hợp giữa bức ảnh ánh sáng khả kiến nổi tiếng của Hubble (bên phải) và bức ảnh hồng ngoại gần đây của đài quan sát quỹ đạo Spitzer (dưới, bên phải). Chính bức ảnh hồng ngoại cho thấy rõ thiên hà có cấu trúc vành. Trong ánh sáng khả kiến, ta chỉ thấy mép trước của vành in thành bóng đen. Phần phình trung tâm che khuất phần còn lại của vành.

Nhưng trong ảnh hồng ngoại, các hạt nóng trong vành xuyên qua phần phình trung tâm. Ghép hai bức ảnh lại, ta có góc nhìn chưa từng thấy trước đây, và có thể suy ra rằng từ rất lâu rồi nơi này từng là một lò lửa hoạt động dữ dội.



Ảnh bìa: © Spitzer Space Telescope

1

Mã sạch



Bạn đọc cuốn sách này vì hai lý do. Thứ nhất, bạn là lập trình viên. Thứ hai, bạn muốn trở thành lập trình viên giỏi hơn. Tốt. Chúng ta cần những lập trình viên giỏi hơn.

Đây là cuốn sách về lập trình tốt. Sách đầy code. Chúng ta sẽ nhìn code từ mọi hướng: từ trên xuống, từ dưới lên, rồi xuyên qua nó từ trong ra ngoài. Đọc xong, chúng ta sẽ hiểu khá nhiều về code. Hơn nữa, chúng ta sẽ phân biệt được code tốt với code xấu, biết cách viết code tốt và biến code xấu thành code tốt.

Sẽ vẫn có code

Có người cho rằng một cuốn sách về code đã lỗi thời, rằng code không còn là vấn đề và chúng ta nên quan tâm đến mô hình cùng yêu cầu thay vì code. Quả thật, có người nói chúng ta sắp bước vào thời kỳ không còn code. Chẳng bao lâu nữa, mọi code sẽ được tạo ra thay vì viết. Lập trình viên cũng sẽ không còn cần thiết, vì người làm kinh doanh có thể tạo chương trình từ đặc tả.

Vô lý! Chúng ta sẽ không bao giờ thoát khỏi code, vì code thể hiện những chi tiết của yêu cầu. Ở một mức nào đó, không thể bỏ qua hay trừu tượng hóa các chi tiết ấy; chúng phải được đặc tả. Đặc tả yêu cầu chi tiết đến mức máy có thể thực thi chính là lập trình. Một bản đặc tả như vậy là code.

Tôi cho rằng mức trừu tượng của các ngôn ngữ chúng ta dùng sẽ tiếp tục tăng. Số lượng ngôn ngữ chuyên miền (domain-specific language) cũng sẽ tăng. Đó là điều tốt, nhưng sẽ không xóa bỏ code. Thật vậy, mọi đặc tả viết bằng những ngôn ngữ bậc cao hơn và ngôn ngữ chuyên miền ấy đều là code! Chúng vẫn phải chặt chẽ, chính xác và trang trọng, chi tiết đến mức máy có thể hiểu rồi thực thi.

Những người nghĩ rằng một ngày nào đó code sẽ biến mất cũng giống các nhà toán học hy vọng tìm ra một nền toán học không cần hình thức hóa. Họ mong một ngày chúng ta sẽ tạo được những cỗ máy làm điều mình muốn thay vì điều mình nói. Những cỗ máy ấy phải hiểu chúng ta đủ rõ để chuyển các nhu cầu được đặc tả mơ hồ thành những chương trình chạy hoàn hảo, đáp ứng chính xác các nhu cầu đó.

Điều ấy sẽ không bao giờ xảy ra. Ngay cả con người, với tất cả trực giác và óc sáng tạo, cũng chưa tạo được những hệ thống thành công chỉ từ cảm nhận mơ hồ của khách hàng. Thật vậy, nếu kỷ luật đặc tả yêu cầu đã dạy chúng ta điều gì, thì đó là yêu cầu được đặc tả tốt cũng trang trọng như code và có thể làm bài kiểm thử thực thi được cho chính code ấy!

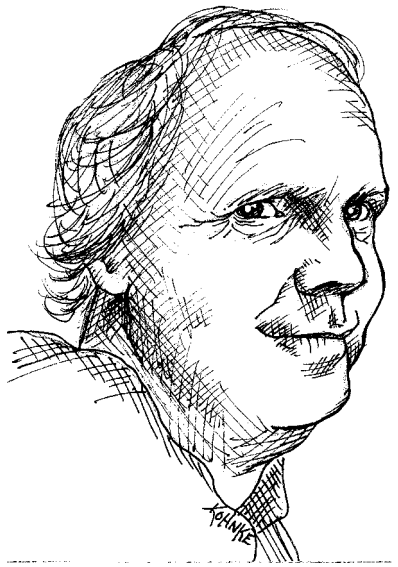
Hãy nhớ rằng code chính là ngôn ngữ cuối cùng chúng ta dùng để diễn đạt yêu cầu. Ta có thể tạo ra những ngôn ngữ gần với yêu cầu hơn, cùng những công cụ giúp phân tích và ghép các yêu cầu thành cấu trúc trang trọng. Nhưng độ chính xác cần thiết sẽ không bao giờ mất đi, nên code sẽ luôn tồn tại.

Mã xấu

Gần đây tôi đọc lời tựa cuốn *Implementation Patterns* của Kent Beck.[1] Ông viết: "… cuốn sách này dựa trên một tiền đề khá mong manh: rằng code tốt có ý nghĩa …" Một tiền đề mong manh ư? Tôi không đồng ý! Theo tôi, đó là một trong những tiền đề vững chắc, được

1. [Beck07].

ủng hộ rộng rãi và bị lạm dụng nhiều nhất trong nghề của chúng ta (và tôi nghĩ Kent biết điều đó). Chúng ta biết code tốt có ý nghĩa vì đã quá lâu phải sống chung với sự thiếu vắng nó.



Tôi biết một công ty vào cuối thập niên 80 đã viết ra một ứng dụng cực kỳ ăn khách. Sản phẩm rất phổ biến, được nhiều chuyên gia mua và sử dụng. Rồi các chu kỳ phát hành bắt đầu kéo dài. Lỗi không được sửa từ bản phát hành này sang bản tiếp theo. Thời gian tải tăng lên, số lần sụp cũng nhiều hơn. Tôi nhớ ngày mình tắt sản phẩm trong cơn bực bội và không bao giờ dùng lại. Không lâu sau, công ty phá sản.

Hai thập kỷ sau, tôi gặp một trong những nhân viên đầu tiên của công ty ấy và hỏi chuyện gì đã xảy ra. Câu trả lời xác nhận nỗi lo của tôi. Họ vội đưa sản phẩm ra thị trường và để lại một mớ hỗn độn khổng lồ trong code. Càng thêm tính năng, code càng tệ, cho đến khi họ không còn quản lý nổi. Chính code xấu đã kéo công ty xuống.

Bạn đã bao giờ bị code xấu cản trở đáng kể chưa? Nếu đã có chút kinh nghiệm lập trình, hẳn bạn từng gặp chuyện đó nhiều lần. Thật vậy, chúng ta có một từ để gọi nó: “lội”. Chúng ta lội qua code xấu, lê bước trong một đầm lầy đầy bụi gai rối rắm và cạm bẫy ẩn giấu. Ta cố tìm đường, mong bắt gặp một gợi ý, một manh mối về chuyện đang diễn ra. Nhưng thứ duy nhất hiện ra lại là thêm nhiều code vô nghĩa.

Dĩ nhiên bạn từng bị code xấu cản trở. Vậy tại sao bạn lại viết ra nó?

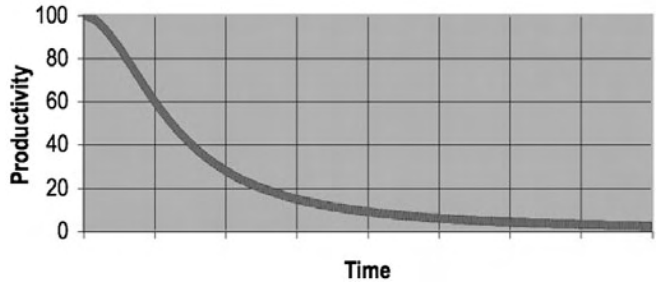
Bạn muốn làm nhanh? Bạn đang vội? Có lẽ vậy. Có thể bạn nghĩ mình không có thời gian làm cho tử tế, hoặc sếp sẽ nổi giận nếu bạn dành thời gian dọn dẹp code. Có thể bạn đã mệt với chương trình này và chỉ muốn kết thúc. Hoặc nhìn vào đống việc khác đã hứa sẽ làm, bạn nhận ra mình phải ghép vội module này để còn chuyển sang việc tiếp theo. Ai trong chúng ta cũng từng như vậy.

Ai trong chúng ta cũng từng nhìn mớ hỗn độn vừa tạo ra rồi để dành việc dọn dẹp cho hôm khác. Ta từng nhẹ nhõm khi thấy chương trình bừa bộn chạy được và nghĩ rằng một mớ hỗn độn còn chạy được vẫn hơn là chẳng có gì. Ta cũng từng hẹn sẽ quay lại dọn sau. Hồi ấy, dĩ nhiên, chúng ta chưa biết định luật LeBlanc: Sau này nghĩa là không bao giờ.

Tổng chi phí sở hữu một mớ hỗn độn

Nếu đã làm lập trình viên hơn hai ba năm, có lẽ bạn từng bị code bừa bộn của người khác làm chậm đi đáng kể. Làm lâu hơn nữa, hẳn bạn cũng từng bị code bừa bộn làm chậm. Mức độ chậm lại có thể rất lớn. Trong một hai năm, những nhóm khởi đầu dự án với tốc độ rất nhanh có thể thấy mình bò chậm như sên. Mỗi thay đổi lại làm hỏng hai ba chỗ khác trong code. Không thay đổi nào là nhỏ. Mỗi lần thêm hay sửa hệ thống, họ đều phải "hiểu" những mớ rối, chỗ xoắn và nút thắt trước khi có thể thêm vào đó những mớ rối, chỗ xoắn và nút thắt mới. Dần dà, mớ hỗn độn lớn đến mức không thể dọn nổi. Hoàn toàn không còn cách nào.

Mớ hỗn độn càng lớn, năng suất của nhóm càng giảm, dần tiến về không. Khi năng suất đi xuống, ban quản lý chỉ còn biết thêm người vào dự án, hy vọng năng suất sẽ tăng. Nhưng người mới không nắm thiết kế hệ thống. Họ không biết phân biệt thay đổi phù hợp với ý đồ thiết kế và thay đổi phá vỡ ý đồ ấy. Hơn nữa, tất cả mọi người trong nhóm đều chịu áp lực khủng khiếp phải tăng năng suất. Thế là họ tạo ra ngày càng nhiều mớ hỗn độn, khiến năng suất tiếp tục tụt về không. (Xem Hình 1-1.)



Hình 1-1
Năng suất theo thời gian

Cuộc đại tái thiết kế trên trời

Cuối cùng, nhóm nổi loạn. Họ báo với ban quản lý rằng không thể tiếp tục phát triển trên code base đáng ghét này và đòi thiết kế lại. Ban quản lý không muốn bỏ nguồn lực vào việc thiết kế lại toàn bộ dự án, nhưng cũng không thể phủ nhận năng suất đang rất tệ. Cuối cùng, họ nhượng bộ trước yêu cầu của các nhà phát triển và phê duyệt cuộc đại tái thiết kế trên trời.

Một nhóm hổ mới được chọn. Ai cũng muốn vào nhóm này vì đây là dự án bắt đầu từ đầu. Họ được làm lại và tạo ra một thứ thật sự đẹp. Nhưng chỉ những người giỏi nhất và sáng giá nhất được chọn. Những người còn lại phải tiếp tục bảo trì hệ thống hiện tại.

Giờ hai nhóm chạy đua. Nhóm hổ phải xây dựng hệ thống mới có đủ mọi chức năng của hệ thống cũ. Họ còn phải theo kịp những thay đổi liên tục được đưa vào hệ thống cũ. Ban quản lý sẽ không thay hệ thống cũ cho đến khi hệ thống mới làm được mọi thứ mà hệ thống cũ làm.

Cuộc đua ấy có thể kéo dài rất lâu. Tôi từng thấy nó mất 10 năm. Đến khi hoàn tất, những thành viên ban đầu của nhóm hổ đã rời đi từ lâu, còn các thành viên hiện tại lại đòi thiết kế lại hệ thống mới vì nó quá bừa bộn.

Nếu từng trải qua dù chỉ một phần nhỏ của câu chuyện này, bạn hẳn đã hiểu rằng dành thời gian giữ code sạch không chỉ có lợi về chi phí, mà còn là chuyện sống còn trong nghề.

Thái độ

Bạn đã bao giờ lội qua một mớ hỗn độn nghiêm trọng đến mức phải mất hàng tuần để làm việc lẽ ra chỉ mất vài giờ chưa? Bạn đã từng thấy một thay đổi lẽ ra chỉ cần một dòng lại phải thực hiện ở hàng trăm module khác nhau chưa? Những chuyện này xảy ra quá thường xuyên.

Vì sao code lại thành ra như vậy? Vì sao code tốt nhanh chóng mục ruỗng thành code xấu? Chúng ta có đủ kiểu giải thích. Ta than phiền rằng yêu cầu thay đổi theo cách phá vỡ thiết kế ban đầu, than tiến độ quá gấp khiến không thể làm cho đúng. Ta lải nhải về những ông sếp ngốc nghếch, khách hàng thiếu bao dung, dân marketing vô dụng và những kẻ làm sạch điện thoại. Nhưng lỗi, Dilbert thân mến, không nằm ở các vì sao, mà ở chính chúng ta. Chúng ta thiếu tính chuyên nghiệp.

Đây có thể là viên thuốc đắng khó nuốt. Sao mớ hỗn độn này lại là lỗi của chúng ta? Còn yêu cầu, tiến độ thì sao? Những ông sếp ngốc nghếch và dân marketing vô dụng chẳng phải cũng phải chịu một phần trách nhiệm ư?

Không. Người quản lý và dân marketing trông cậy vào chúng ta để có thông tin họ cần, từ đó đưa ra lời hứa và cam kết. Ngay cả khi họ không hỏi, chúng ta cũng không nên ngại nói rõ suy nghĩ của mình. Người dùng trông vào chúng ta để xác nhận yêu cầu sẽ khớp với hệ thống ra sao. Quản lý dự án trông vào chúng ta để cùng lập tiến độ. Chúng ta tham gia sâu vào việc lập kế hoạch và cùng chịu phần lớn trách nhiệm cho mọi thất bại, nhất là những thất bại do code xấu!

“Nhưng khoan!” bạn nói. “Nếu tôi không làm điều sếp bảo, tôi sẽ bị sa thải.” Có lẽ không đâu. Hầu hết người quản lý muốn nghe sự thật, kể cả khi họ không thể hiện như vậy.

Họ muốn code tốt, kể cả khi đang ám ảnh về tiến độ. Họ có thể hết lòng bảo vệ tiến độ và yêu cầu; đó là việc của họ. Còn việc của bạn là hết lòng bảo vệ code.

Để thấy rõ điều này, hãy tưởng tượng bạn là bác sĩ, có một bệnh nhân yêu cầu bỏ qua việc rửa tay tưởng như vô nghĩa trước ca mổ vì mất quá nhiều thời gian.[2] Bệnh nhân rõ ràng là ông chủ, vậy mà bác sĩ phải kiên quyết từ chối. Vì sao? Bác sĩ hiểu nguy cơ bệnh tật và nhiễm trùng hơn bệnh nhân. Làm theo yêu cầu ấy sẽ là thiếu chuyên nghiệp, chưa kể còn phạm pháp.

Lập trình viên cũng vậy. Uốn theo ý muốn của những người quản lý không hiểu rủi ro khi tạo ra mớ hỗn độn là thiếu chuyên nghiệp.

Nghịch lý căn bản

Lập trình viên đối mặt với một nghịch lý về những giá trị cơ bản. Nhà phát triển nào có vài năm kinh nghiệm cũng biết những mớ hỗn độn trước đây làm mình chậm lại. Vậy mà ai cũng cảm thấy áp lực phải tạo thêm mớ hỗn độn để kịp hạn chót. Nói ngắn gọn, họ không dành thời gian để đi nhanh!

Người chuyên nghiệp thật sự biết vế thứ hai của nghịch lý là sai. Tạo ra mớ hỗn độn không giúp bạn kịp hạn. Nó lập tức làm bạn chậm lại và khiến bạn trễ hạn. Cách duy nhất để kịp hạn, cũng là cách duy nhất để đi nhanh, là luôn giữ code sạch hết mức có thể.

Nghệ thuật của mã sạch?

Giả sử bạn tin rằng code bừa bộn là trở ngại đáng kể, và chấp nhận rằng cách duy nhất để đi nhanh là giữ code sạch. Khi ấy, bạn phải tự hỏi: "Làm sao để viết code sạch?" Cố viết code sạch mà không biết code sạch nghĩa là gì thì chẳng ích gì!

Tin xấu là viết code sạch cũng giống như vẽ tranh. Hầu hết chúng ta đều nhận ra một bức tranh đẹp hay dở, nhưng biết phân biệt nghệ thuật hay với dở không có nghĩa là biết vẽ. Tương tự, nhận ra code sạch hay bẩn không có nghĩa là biết viết code sạch!

Viết code sạch đòi hỏi kỷ luật áp dụng vô số kỹ thuật nhỏ, cùng một "cảm giác sạch sẽ" phải rèn luyện rất lâu mới có. "Cảm giác code" ấy là điều cốt yếu. Một số người sinh ra đã có nó, số khác phải vất vả mới có được. Nó giúp ta nhận ra code tốt hay xấu, đồng thời chỉ cho ta cách dùng kỷ luật để biến code xấu thành code sạch.

Lập trình viên không có "cảm giác code" có thể nhìn module bừa bộn và nhận ra mớ hỗn độn, nhưng không biết phải làm gì. Người có "cảm giác code" sẽ nhìn thấy nhiều lựa chọn và cách biến đổi. Cảm giác ấy giúp họ chọn phương án tốt nhất, rồi vạch ra một chuỗi thay đổi giữ nguyên hành vi để đi từ đây đến đó.

Nói ngắn gọn, lập trình viên viết code sạch là một nghệ sĩ có thể đưa màn hình trắng qua một chuỗi biến đổi cho đến khi thành một hệ thống được lập trình thanh lịch.

---

2. Khi Ignaz Semmelweis lần đầu khuyến nghị bác sĩ rửa tay vào năm 1847, đề xuất này bị bác bỏ với lý do bác sĩ quá bận, không có thời gian rửa tay giữa các lần thăm bệnh nhân.

Mã sạch là gì?

Có lẽ mỗi lập trình viên có một định nghĩa riêng. Vì vậy, tôi đã hỏi một số lập trình viên nổi tiếng, giàu kinh nghiệm xem họ nghĩ gì.



Bjarne Stroustrup, người phát minh ra C++ và tác giả cuốn *The C++ Programming Language*

> *Tôi muốn code của mình vừa thanh lịch vừa hiệu quả. Logic phải thẳng thắn để lỗi khó ẩn nấp, phụ thuộc phải tối thiểu để dễ bảo trì, xử lý lỗi phải đầy đủ theo một chiến lược rõ ràng, hiệu năng phải gần mức tối ưu để người ta không bị cám dỗ làm code bừa bộn bằng những tối ưu hóa thiếu nguyên tắc. Mã sạch làm tốt một việc.*

Bjarne dùng từ "thanh lịch". Quả là một từ đáng chú ý! Từ điển trên chiếc MacBook® của tôi định nghĩa nó là duyên dáng, phong cách trong diện mạo hoặc cử chỉ; tài tình và giản dị. Hãy để ý chữ "dễ chịu" trong các định nghĩa ấy. Có lẽ Bjarne cho rằng đọc code sạch cũng dễ chịu. Đọc nó phải khiến bạn mỉm cười, như khi nghe một chiếc hộp nhạc được chế tác khéo hay ngắm một chiếc xe thiết kế đẹp.

Bjarne cũng nhắc đến hiệu quả, đến hai lần. Có lẽ điều đó không lạ khi ông là người phát minh ra C++, nhưng theo tôi, ý ông không chỉ là mong muốn tốc độ. Chu kỳ máy bị lãng phí thì thiếu thanh lịch, chẳng dễ chịu gì. Hãy để ý từ Bjarne dùng để nói về hệ quả ấy: "cám dỗ". Đây là một sự thật sâu sắc: code xấu khiến mớ hỗn độn ngày càng lớn! Khi sửa code xấu, người ta thường làm nó tệ hơn.

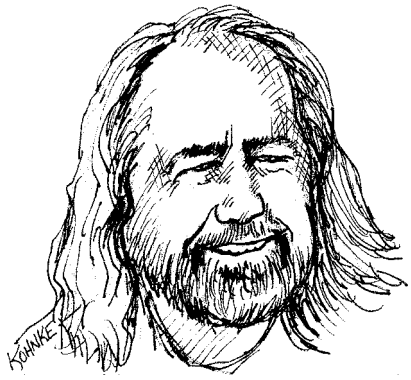
Pragmatic Dave Thomas và Andy Hunt diễn đạt ý ấy theo cách khác bằng ẩn dụ cửa sổ vỡ.[3] Một tòa nhà có cửa sổ vỡ trông như chẳng ai quan tâm. Người khác cũng thôi quan tâm, để thêm nhiều cửa sổ vỡ. Cuối cùng, họ chủ động đập vỡ cửa sổ, bôi bẩn mặt tiền bằng hình vẽ nguệch ngoạc và để rác chất đống. Một cửa sổ vỡ khởi đầu quá trình mục ruỗng.

---

3. `http://www.pragmaticprogrammer.com/booksellers/2004-12.html`

Bjarne còn nói xử lý lỗi phải đầy đủ. Điều đó đòi hỏi kỷ luật chú ý đến chi tiết. Xử lý lỗi qua loa là một cách lập trình viên bỏ qua chi tiết. Rò rỉ bộ nhớ là một cách khác, điều kiện tranh chấp cũng vậy. Đặt tên thiếu nhất quán cũng thế. Tóm lại, code sạch cho thấy sự chú ý sát sao đến chi tiết.

Bjarne kết luận rằng code sạch làm tốt một việc. Không phải ngẫu nhiên mà nhiều nguyên lý thiết kế phần mềm có thể gói gọn trong lời khuyên giản dị ấy. Nhiều tác giả đã cố truyền đạt cùng một ý. Code xấu cố làm quá nhiều việc, khiến ý định lẫn lộn và mục đích mơ hồ. Mã sạch tập trung. Mỗi hàm, lớp hay module đều thể hiện một mục đích duy nhất, không bị những chi tiết xung quanh làm phân tâm hay vấy bẩn.



Grady Booch, tác giả cuốn *Object Oriented Analysis and Design with Applications*

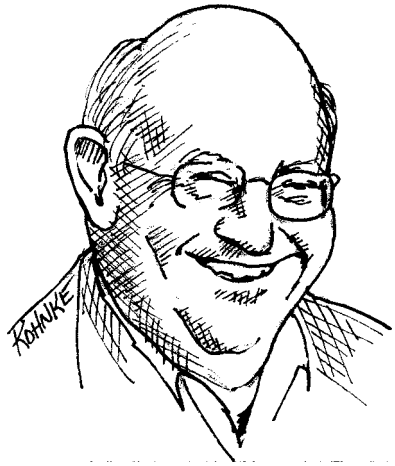
> *Mã sạch đơn giản và trực tiếp. Mã sạch đọc như văn xuôi được viết khéo. Mã sạch không bao giờ che khuất ý đồ của người thiết kế, mà đầy những trừu tượng sắc nét và những dòng điều khiển thẳng thắn.*

Grady nêu một số ý giống Bjarne, nhưng nhìn từ góc độ khả năng đọc. Tôi đặc biệt thích ý của ông rằng code sạch phải đọc như văn xuôi được viết khéo. Hãy nhớ lại một cuốn sách thật hay bạn từng đọc. Bạn còn nhớ những con chữ dường như biến mất, nhường chỗ cho hình ảnh không? Cứ như đang xem phim vậy, phải không? Còn hơn thế: bạn thấy nhân vật, nghe âm thanh, cảm nhận nỗi bi thương và sự hài hước.

Đọc code sạch sẽ không bao giờ giống hẳn đọc *Lord of the Rings*. Dù vậy, ẩn dụ văn chương ấy không tồi. Như một cuốn tiểu thuyết hay, code sạch phải làm lộ rõ những căng thẳng trong bài toán cần giải. Nó phải đẩy những căng thẳng ấy lên cao trào, rồi để người đọc thốt lên "À! Dĩ nhiên rồi!" khi vấn đề được giải quyết và một lời giải hiển nhiên hiện ra.

Tôi thấy cách Grady dùng cụm "trừu tượng sắc nét" (crisp abstraction) thật thú vị, gần như một nghịch hợp! Suy cho cùng, "crisp" gần nghĩa với "concrete". Từ điển trên MacBook của tôi định nghĩa "crisp" là quyết đoán nhanh gọn, thực tế, không do dự hay thêm chi tiết thừa. Hai nghĩa tưởng như trái ngược ấy lại truyền tải một thông điệp mạnh

mẽ. Code phải thực tế, không suy đoán. Nó chỉ nên chứa những gì cần thiết. Người đọc phải thấy rằng chúng ta đã quyết đoán.



“Big” Dave Thomas, người sáng lập OTI, cha đỡ đầu của chiến lược Eclipse

> *Mã sạch có thể được một nhà phát triển khác, không phải tác giả ban đầu, đọc và mở rộng. Nó có kiểm thử đơn vị và kiểm thử chấp nhận. Tên gọi có ý nghĩa. Nó đưa ra một cách làm thay vì nhiều cách cho cùng một việc. Nó có ít phụ thuộc, được định nghĩa rõ ràng, và một API rõ ràng, tối giản. Code phải có tính văn học, vì tùy ngôn ngữ mà không phải thông tin cần thiết nào cũng có thể diễn đạt rõ chỉ bằng code.*

Big Dave cũng mong code dễ đọc như Grady, nhưng ông nhấn mạnh thêm một điểm: mã sạch giúp người khác dễ mở rộng. Điều này nghe có vẻ hiển nhiên, nhưng vẫn cần nhắc kỹ. Dù sao, code dễ đọc và code dễ thay đổi là hai chuyện khác nhau.

Dave gắn sự sạch sẽ với kiểm thử! Mười năm trước, nhiều người hẳn sẽ ngạc nhiên. Nhưng Phát triển hướng kiểm thử (Test Driven Development) đã tác động sâu sắc đến ngành và trở thành một trong những kỷ luật nền tảng nhất. Dave nói đúng. Code không có kiểm thử thì không sạch. Dù thanh lịch, dễ đọc và dễ tiếp cận đến đâu, thiếu kiểm thử thì code vẫn bẩn.

Dave dùng từ “tối giản” hai lần. Có vẻ ông coi trọng code nhỏ hơn code lớn. Thật vậy, đây là điệp khúc quen thuộc trong văn liệu phần mềm từ thuở ban đầu. Nhỏ hơn thì tốt hơn.

Dave cũng nói code phải có tính văn học. Đây là cách nhắc nhẹ đến lập trình văn học của Knuth.[4] Nói cách khác, code phải được viết sao cho con người có thể đọc.

---

4. [Knuth92].

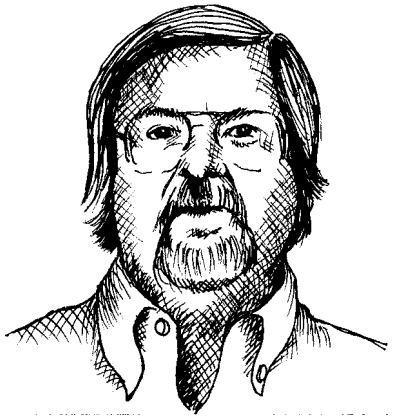


Michael Feathers, tác giả cuốn *Working Effectively with Legacy Code*

> *Tôi có thể kể ra mọi phẩm chất mình thấy ở mã sạch, nhưng có một phẩm chất bao trùm dẫn đến tất cả những phẩm chất kia. Mã sạch luôn trông như được viết bởi một người quan tâm. Không có điều gì hiển nhiên bạn có thể làm để khiến nó tốt hơn. Tác giả code đã nghĩ đến tất cả những điều đó. Nếu bạn thử hình dung cách cải thiện, cuối cùng bạn lại trở về chỗ cũ, ngồi trân trọng đoạn code người khác để lại, đoạn code của một người thực sự quan tâm đến tay nghề.*

Một từ: quan tâm. Đó chính là chủ đề của cuốn sách này. Có lẽ phụ đề phù hợp là *Cách quan tâm đến code*.

Michael nói trúng phóc. Mã sạch là code đã được chăm chút. Có người dành thời gian giữ nó đơn giản, ngăn nắp và chú ý đúng mức đến từng chi tiết. Họ đã quan tâm.



Ron Jeffries, tác giả cuốn *Extreme Programming Installed* và *Extreme Programming Adventures in C#*

Ron bắt đầu sự nghiệp lập trình bằng Fortran tại Strategic Air Command. Ông đã viết code bằng gần như mọi ngôn ngữ, trên gần như mọi loại máy. Lời ông rất đáng để cân nhắc.

> *Những năm gần đây, tôi thường bắt đầu, và gần như kết thúc, bằng các quy tắc về code đơn giản của Beck. Theo thứ tự ưu tiên, code đơn giản:*
>
> - *Chạy được mọi bài kiểm thử;*
> - *Không chứa trùng lặp;*
> - *Thể hiện mọi ý tưởng thiết kế có trong hệ thống;*
> - *Giảm số lượng thực thể như lớp, phương thức, hàm và những thứ tương tự.*
>
> *Trong số đó, tôi tập trung nhiều nhất vào trùng lặp. Khi cùng một việc cứ lặp đi lặp lại, có thể trong đầu chúng ta đang có một ý tưởng chưa được thể hiện tốt trong code. Tôi cố tìm ra ý tưởng ấy, rồi diễn đạt nó rõ hơn.*
>
> *Với tôi, diễn đạt tốt bao gồm cả tên gọi có ý nghĩa. Tôi thường đổi tên vài lần mới chốt được. Các công cụ lập trình hiện đại như Eclipse giúp việc đổi tên khá dễ dàng, nên tôi không ngại thay đổi. Nhưng khả năng diễn đạt không chỉ nằm ở tên gọi. Tôi cũng xem một đối tượng hay phương thức có đang làm nhiều hơn một việc không. Nếu là đối tượng, có lẽ nên tách thành hai hay nhiều đối tượng. Nếu là phương thức, tôi luôn dùng tái cấu trúc Extract Method, để tạo ra một phương thức nói rõ hơn nó làm gì và vài phương thức con mô tả cách thực hiện.*
>
> *Chỉ riêng việc loại bỏ trùng lặp và diễn đạt rõ hơn đã giúp tôi tiến rất xa trên con đường đến với mã sạch. Chỉnh code bẩn chỉ với hai điều này trong đầu cũng có thể tạo ra khác biệt lớn. Nhưng còn một điều nữa tôi biết mình vẫn làm, dù khó giải thích hơn.*
>
> *Sau nhiều năm làm việc này, tôi thấy chương trình nào cũng được tạo nên từ những thành phần khá giống nhau. Chẳng hạn, "tìm thứ gì đó trong một tập hợp".*

Dù là cơ sở dữ liệu hồ sơ nhân viên, hash map gồm khóa và giá trị, hay một mảng phần tử nào đó, ta thường cần lấy một phần tử cụ thể trong tập hợp. Khi gặp trường hợp ấy, tôi thường bọc cách hiện thực cụ thể trong một phương thức hay lớp trừu tượng hơn. Cách này đem lại cho tôi vài lợi ích thú vị.

Tôi có thể hiện thực chức năng ngay bằng thứ đơn giản, chẳng hạn hash map. Nhưng vì mọi chỗ tham chiếu đến thao tác tìm kiếm đều được lớp trừu tượng nhỏ của tôi bao bọc, tôi có thể đổi cách hiện thực bất cứ lúc nào. Nhờ vậy, tôi tiến nhanh mà vẫn giữ được khả năng thay đổi về sau.

Hơn nữa, lớp trừu tượng cho tập hợp thường khiến tôi chú ý đến điều "thật sự" đang diễn ra, giúp tôi không lao vào hiện thực những hành vi tùy tiện cho tập hợp khi điều mình cần chỉ là vài cách đơn giản để tìm thứ mong muốn.

Giảm trùng lặp, diễn đạt rõ ràng và sớm xây dựng những trừu tượng đơn giản. Với tôi, đó là những gì tạo nên mã sạch.

Chỉ trong vài đoạn ngắn, Ron đã tóm tắt nội dung cuốn sách này: không trùng lặp, mỗi thứ làm một việc, diễn đạt rõ ràng, trừu tượng nhỏ. Đủ cả.



Ward Cunningham, người phát minh ra Wiki và Fit, đồng phát minh eXtreme Programming. Động lực đằng sau *Design Patterns*. Người dẫn dắt tư tưởng Smalltalk và OO. Cha đỡ đầu của tất cả những ai quan tâm đến code.

Bạn biết mình đang làm việc với mã sạch khi mỗi thủ tục bạn đọc đều đúng như mong đợi. Có thể gọi code là đẹp khi nó còn khiến ngôn ngữ trông như được tạo ra cho bài toán.

Những câu như vậy rất đặc trưng của Ward. Bạn đọc, gật đầu rồi chuyển sang chủ đề khác. Nghe hợp lý và hiển nhiên đến mức khó nhận ra chiều sâu của nó. Có thể bạn nghĩ rằng điều ấy đúng như mong đợi. Nhưng hãy nhìn kỹ hơn.

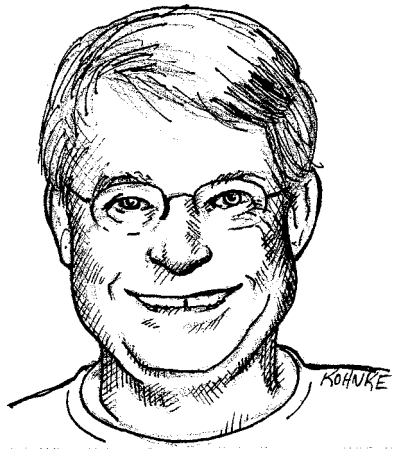
"… đúng như mong đợi." Lần cuối bạn thấy một module đúng như mong đợi là khi

nào? Có phải những module bạn gặp thường khó hiểu, phức tạp, rối rắm hơn không? Chẳng phải chúng thường dẫn bạn đi sai hướng? Bạn đã quen với việc vùng vẫy để nắm lấy những mạch lập luận tuôn ra từ toàn hệ thống rồi đan vào module mình đang đọc, phải không? Lần cuối bạn đọc code và gật đầu như khi đọc câu của Ward là khi nào?

Ward cho rằng khi đọc code sạch, bạn sẽ không ngạc nhiên chút nào, thậm chí chẳng cần tốn nhiều công sức. Bạn đọc nó và thấy mọi thứ đúng như mong đợi: hiển nhiên, đơn giản, thuyết phục. Mỗi module mở đường cho module tiếp theo, đồng thời cho bạn biết module sau sẽ được viết thế nào. Chương trình sạch được viết tốt đến mức bạn khó nhận ra điều đó. Người thiết kế khiến nó trông đơn giản đến buồn cười, như mọi thiết kế xuất sắc.

Còn quan niệm về cái đẹp của Ward thì sao? Chúng ta vẫn phàn nàn rằng ngôn ngữ không được thiết kế cho bài toán của mình. Nhưng Ward đặt trách nhiệm trở lại lên chúng ta. Ông nói code đẹp khiến ngôn ngữ trông như được tạo ra cho bài toán. Vậy chúng ta phải làm cho ngôn ngữ trông đơn giản! Những kẻ cuồng ngôn ngữ khắp nơi, hãy coi chừng! Chính lập trình viên, chứ không phải ngôn ngữ, khiến chương trình trông đơn giản.

Các trường phái



Còn tôi (Uncle Bob) nghĩ mã sạch là gì? Cuốn sách này sẽ trình bày thật chi tiết quan niệm của tôi và các đồng nghiệp về mã sạch. Chúng tôi sẽ nói điều gì tạo nên tên biến, hàm, lớp sạch và những thứ tương tự. Chúng tôi sẽ nêu ý kiến như những chân lý tuyệt đối, không xin lỗi vì giọng điệu gay gắt. Ở thời điểm này trong sự nghiệp, với chúng tôi, chúng là chân lý. Đó là trường phái của chúng tôi về mã sạch.

Những người tập võ không phải ai cũng đồng ý môn võ nào tốt nhất hay kỹ thuật nào tốt nhất trong một môn. Các bậc thầy võ thuật thường lập trường phái riêng và tập hợp học trò đến học. Có Gracie Jiu Jitsu do gia đình Gracie sáng lập và truyền dạy ở Brazil; Hakkoryu Jiu Jitsu do Okuyama Ryuho sáng lập và truyền dạy tại Tokyo; Jeet Kune Do do

Bruce Lee sáng lập và truyền dạy ở Hoa Kỳ.

Học trò của các trường phái này đắm mình trong giáo huấn của người sáng lập. Họ dốc sức học theo vị thầy ấy, thường gạt bỏ giáo huấn của những người khác. Sau này, khi tiến bộ trong môn võ, họ có thể theo học một vị thầy khác để mở rộng kiến thức và cách luyện tập. Một số người tiếp tục trau dồi kỹ năng, khám phá kỹ thuật mới rồi lập trường phái riêng.

Không trường phái nào đúng tuyệt đối. Thế nhưng trong một trường phái cụ thể, chúng ta vẫn hành xử như thể giáo huấn và kỹ thuật của trường phái ấy là đúng. Suy cho cùng, vẫn có cách đúng để luyện Hakkoryu Jiu Jitsu hay Jeet Kune Do. Nhưng cái đúng trong một trường phái không khiến giáo huấn của trường phái khác trở thành sai.

Hãy xem cuốn sách này như lời giới thiệu về Trường phái Mã sạch Object Mentor. Những kỹ thuật và giáo huấn trong sách là cách chúng tôi thực hành nghề của mình. Chúng tôi sẵn sàng khẳng định rằng nếu làm theo, bạn sẽ nhận được những lợi ích chúng tôi từng có và học được cách viết code sạch, chuyên nghiệp. Nhưng đừng nghĩ chúng tôi "đúng" theo nghĩa tuyệt đối. Có những trường phái và bậc thầy khác cũng có quyền tuyên bố mình chuyên nghiệp như chúng tôi. Bạn cũng nên học hỏi từ họ.

Quả thật, nhiều khuyến nghị trong sách này gây tranh cãi. Có lẽ bạn sẽ không đồng ý với tất cả, thậm chí có thể phản đối gay gắt một số điều. Không sao. Chúng tôi không thể nhận mình là thẩm quyền cuối cùng. Dù vậy, những khuyến nghị này đều là điều chúng tôi đã suy nghĩ lâu dài, kỹ càng và học được qua nhiều thập kỷ kinh nghiệm cùng những lần thử sai lặp đi lặp lại. Dù đồng ý hay không, bạn cũng nên đọc và tôn trọng quan điểm của chúng tôi.

Chúng ta là tác giả

Trường `@author` trong Javadoc cho biết chúng ta là ai. Chúng ta là tác giả. Tác giả luôn có người đọc và có trách nhiệm giao tiếp tốt với họ. Lần tới khi viết một dòng code, hãy nhớ rằng bạn là tác giả, đang viết cho những người đọc sẽ đánh giá công sức của mình.

Bạn có thể hỏi: Code thật sự được đọc bao nhiêu? Chẳng phải phần lớn công sức là viết code sao?

Bạn đã bao giờ phát lại một phiên chỉnh sửa chưa? Thập niên 80 và 90, chúng ta có những trình soạn thảo như Emacs ghi lại từng lần gõ phím. Bạn có thể làm việc một giờ rồi phát lại cả phiên như xem một bộ phim tua nhanh. Khi tôi làm vậy, kết quả thật hấp dẫn.

Phần lớn thời gian phát lại là cuộn màn hình và chuyển đến các module khác!

Bob bước vào module.

Anh cuộn xuống hàm cần sửa.

Anh dừng lại, cân nhắc các lựa chọn.

Ồ, anh cuộn lên đầu module để kiểm tra việc khởi tạo một biến.

Giờ anh cuộn trở xuống rồi bắt đầu gõ.

Ối, anh xóa phần vừa gõ!

Anh gõ lại.

Anh lại xóa!

Anh gõ được nửa đoạn khác rồi cũng xóa luôn!

Anh cuộn xuống một hàm khác đang gọi hàm mình sửa để xem nó được gọi thế nào.

Anh cuộn lên và gõ lại đúng đoạn code vừa xóa.

Anh dừng lại.

Anh lại xóa đoạn code đó!

Anh mở cửa sổ khác và xem một lớp con. Hàm đó có bị ghi đè không?

…

Bạn hiểu ý rồi đấy. Thật vậy, thời gian đọc nhiều hơn thời gian viết gấp hơn 10 lần. Chúng ta liên tục đọc code cũ trong lúc viết code mới.

Vì tỷ lệ này cao như vậy, chúng ta muốn code dễ đọc, kể cả khi điều đó khiến việc viết khó hơn. Dĩ nhiên, không thể viết code mà không đọc nó; vì thế, làm code dễ đọc cũng khiến việc viết dễ hơn.

Không thể thoát khỏi lẽ đó. Bạn không thể viết code nếu không đọc được code xung quanh. Việc viết đoạn code hôm nay dễ hay khó tùy vào việc code xung quanh dễ hay khó đọc. Muốn đi nhanh, hoàn thành sớm và viết code dễ hơn, hãy làm cho code dễ đọc.

Quy tắc Hướng đạo sinh

Viết code tốt thôi chưa đủ. Code phải luôn sạch theo thời gian. Ai cũng từng thấy code mục ruỗng, xuống cấp theo năm tháng. Vì vậy, chúng ta phải chủ động ngăn sự xuống cấp ấy.

Hướng đạo sinh Hoa Kỳ có một quy tắc đơn giản mà chúng ta có thể áp dụng vào nghề mình:

Hãy để khu cắm trại sạch hơn lúc bạn đến.[5]

Nếu ai cũng check-in code sạch hơn một chút so với lúc check-out, code sẽ không thể mục ruỗng. Việc dọn dẹp không cần lớn lao: đổi một tên biến cho hay hơn, tách một hàm hơi quá lớn, bỏ một đoạn trùng lặp nhỏ, dọn một câu lệnh if phức hợp.

Bạn có tưởng tượng được cảnh làm việc trong một dự án mà code ngày càng tốt hơn không? Bạn có nghĩ lựa chọn nào khác cũng chuyên nghiệp không? Chẳng phải cải tiến liên tục vốn là một phần của tính chuyên nghiệp sao?

---

5. Câu này phỏng theo lời từ biệt của Robert Stephenson Smyth Baden-Powell gửi các Hướng đạo sinh: “Hãy cố gắng để lại thế giới này tốt đẹp hơn một chút so với lúc bạn đến …”

Tiền truyện và các nguyên lý

Theo nhiều cách, cuốn sách này là "tiền truyện" của cuốn sách tôi viết năm 2002, *Agile Software Development: Principles, Patterns, and Practices* (PPP). PPP bàn về các nguyên lý thiết kế hướng đối tượng và nhiều thực hành của các nhà phát triển chuyên nghiệp. Nếu chưa đọc PPP, có thể bạn sẽ thấy cuốn sách ấy tiếp nối câu chuyện trong cuốn này. Nếu đã đọc, bạn sẽ nhận ra nhiều tư tưởng của cuốn trước được nhắc lại ở đây, nhưng ở cấp độ code.

Trong sách này, bạn sẽ gặp những chỗ nhắc rải rác đến các nguyên lý thiết kế như Nguyên lý Trách nhiệm Đơn nhất (Single Responsibility Principle, SRP), Nguyên lý Mở Đóng (Open Closed Principle, OCP) và Nguyên lý Đảo ngược Phụ thuộc (Dependency Inversion Principle, DIP). PPP trình bày kỹ những nguyên lý ấy.

Kết luận

Sách về nghệ thuật không hứa biến bạn thành nghệ sĩ. Chúng chỉ có thể trao cho bạn một số công cụ, kỹ thuật và cách tư duy mà nghệ sĩ khác từng dùng. Cuốn sách này cũng không thể hứa biến bạn thành lập trình viên giỏi hay trao cho bạn "cảm giác code". Nó chỉ có thể cho bạn thấy cách các lập trình viên giỏi tư duy, cùng những mẹo, kỹ thuật và công cụ họ dùng.

Cũng như sách về nghệ thuật, cuốn này có rất nhiều chi tiết và code. Bạn sẽ thấy code tốt lẫn code xấu, thấy code xấu được biến thành code tốt. Bạn sẽ gặp những danh sách nguyên tắc kinh nghiệm, kỷ luật và kỹ thuật, hết ví dụ này đến ví dụ khác. Sau đó, tùy bạn.

Hãy nhớ câu chuyện cười cũ về một nghệ sĩ violin hòa tấu bị lạc đường đến buổi biểu diễn. Ông dừng một cụ già ở góc phố và hỏi đường đến Carnegie Hall. Cụ già nhìn người nghệ sĩ cùng cây đàn kẹp dưới cánh tay rồi nói: "Luyện tập đi, con trai. Luyện tập!"

Tài liệu tham khảo

[Beck07]: *Implementation Patterns*, Kent Beck, Addison-Wesley, 2007.

[Knuth92]: *Literate Programming*, Donald E. Knuth, Center for the Study of Language and Information, Leland Stanford Junior University, 1992.

2

Tên có ý nghĩa

Tác giả: Tim Ottinger



Giới thiệu

Tên xuất hiện khắp nơi trong phần mềm. Ta đặt tên cho biến, hàm, đối số, lớp và package. Tệp mã nguồn và thư mục chứa chúng cũng cần tên. Rồi còn tệp jar, war, ear. Ta cứ đặt tên mãi. Đã làm việc này nhiều đến thế, ta nên làm cho tốt. Dưới đây là vài quy tắc đơn giản để đặt tên hay.

Dùng tên thể hiện ý định

Ai cũng có thể nói tên phải thể hiện ý định. Nhưng chúng tôi muốn bạn hiểu rằng chúng tôi thật sự nghiêm túc về chuyện này. Chọn tên tốt tốn thời gian, nhưng thời gian tiết kiệm được còn nhiều hơn. Vì vậy, hãy cân nhắc kỹ tên mình đặt và đổi tên khi tìm được tên hay hơn. Làm vậy, bất cứ ai đọc mã của bạn, kể cả chính bạn, cũng sẽ thấy dễ chịu hơn.

Tên của một biến, một hàm hay một lớp phải trả lời được những câu hỏi lớn: nó tồn tại để làm gì, nó làm gì và được dùng ra sao. Nếu phải viết chú thích mới hiểu được tên, thì cái tên ấy chưa thể hiện được ý định.

```
int d; // thời gian trôi qua tính bằng ngày
```

Tên d chẳng nói lên điều gì. Nó không gợi đến thời gian đã trôi qua hay đến đơn vị ngày. Ta nên chọn tên cho biết đang đo điều gì và đo bằng đơn vị nào:

```
int elapsedTimeInDays;
int daysSinceCreation;
int daysSinceModification;
int fileAgeInDays;
```

Tên thể hiện ý định giúp ta hiểu và sửa mã dễ hơn nhiều. Đoạn mã này dùng để làm gì?

```
public List<int[]> getThem() {
    List<int[]> list1 = new ArrayList<int[]>();
    for (int[] x : theList)
        if (x[0] == 4)
            list1.add(x);
    return list1;
}
```

Vì sao khó nhận ra đoạn mã đang làm gì? Không có biểu thức phức tạp. Khoảng trắng và thụt lề đều hợp lý. Đoạn mã chỉ nhắc đến ba biến và hai hằng. Cũng chẳng có lớp cầu kỳ hay phương thức đa hình nào, chỉ có một danh sách các mảng, có vẻ vậy.

Vấn đề không nằm ở sự đơn giản của mã, mà ở tính hàm ẩn của nó, nếu cho phép chúng tôi dùng từ này: mức độ mà bối cảnh không được thể hiện rõ ngay trong mã. Đoạn mã ngầm yêu cầu ta biết câu trả lời cho những câu hỏi như:

1. theList chứa những thứ gì?
2. Chỉ số 0 của một phần tử trong theList có ý nghĩa gì?

3. Giá trị 4 có ý nghĩa gì?
4. Tôi sẽ dùng danh sách được trả về như thế nào?

Đoạn mã không trả lời những câu hỏi ấy, dù hoàn toàn có thể làm vậy. Giả sử ta đang làm một trò chơi dò mìn. Ta nhận ra bàn chơi là một danh sách các ô, hiện được gọi là theList. Hãy đổi tên nó thành gameBoard.

Mỗi ô trên bàn chơi được biểu diễn bằng một mảng đơn giản. Ta cũng nhận ra chỉ số 0 là vị trí chứa giá trị trạng thái, còn trạng thái 4 nghĩa là "đã cắm cờ". Chỉ cần đặt tên cho những khái niệm đó, đoạn mã đã rõ ràng hơn nhiều:

```
public List<int[]> getFlaggedCells() {
    List<int[]> flaggedCells = new ArrayList<int[]>();
    for (int[] cell : gameBoard)
        if (cell[STATUS_VALUE] == FLAGGED)
            flaggedCells.add(cell);
    return flaggedCells;
}
```

Độ đơn giản của mã không đổi. Số toán tử, số hằng và số mức lồng nhau đều giữ nguyên. Nhưng giờ mã đã tường minh hơn nhiều.

Ta còn có thể thay mảng int bằng một lớp đơn giản biểu diễn ô. Lớp đó có thể cung cấp một hàm thể hiện ý định, chẳng hạn isFlagged, để che đi các con số ma thuật. Khi ấy hàm sẽ thành:

```
public List<Cell> getFlaggedCells() {
    List<Cell> flaggedCells = new ArrayList<Cell>();
    for (Cell cell : gameBoard)
        if (cell.isFlagged())
            flaggedCells.add(cell);
    return flaggedCells;
}
```

Chỉ đổi vài cái tên, ta đã dễ hiểu chuyện gì đang diễn ra hơn hẳn. Đó là sức mạnh của việc chọn tên tốt.

Tránh tên gây hiểu lầm

Lập trình viên phải tránh để lại những manh mối sai khiến mã khó hiểu. Đừng dùng những từ mà nghĩa quen thuộc của chúng khác với điều ta muốn diễn đạt. Chẳng hạn, hp, aix và sco là tên biến tồi vì chúng cũng là tên các nền tảng hoặc biến thể Unix. Ngay cả khi viết mã tính cạnh huyền và thấy hp là cách viết tắt hợp lý, tên đó vẫn có thể gây hiểu lầm.

Đừng gọi một nhóm account là accountList trừ khi nó thực sự là một List. Với lập trình viên, từ list có nghĩa cụ thể. Nếu vật chứa các account không phải List, tên ấy có thể khiến người đọc hiểu sai.[1] Dùng accountGroup, bunchOfAccounts hoặc đơn giản là accounts sẽ hợp lý hơn.

---

1. Như sẽ thấy ở phần sau, kể cả khi vật chứa là một List, có lẽ vẫn không nên mã hóa kiểu vật chứa vào tên.

Cẩn thận với những cái tên chỉ khác nhau chút ít. Bạn mất bao lâu mới nhận ra khác biệt tinh vi giữa XYZControllerForEfficientHandlingOfStrings trong một module và XYZControllerForEfficientStorageOfStrings ở một chỗ khác? Hai cụm từ trông giống nhau đến đáng sợ.

Viết các khái niệm tương tự theo cùng một cách sẽ giúp người đọc có thêm thông tin. Viết không nhất quán lại khiến họ hiểu sai. Các môi trường Java hiện đại có tính năng tự động hoàn thành mã. Ta gõ vài ký tự của tên rồi nhấn một tổ hợp phím tắt nào đó, đôi khi chẳng cần nhấn gì, và nhận được danh sách các tên có thể hoàn thành. Danh sách sẽ hữu ích hơn nếu tên của những thứ gần giống nhau nằm cạnh nhau theo thứ tự chữ cái, còn điểm khác biệt thì dễ thấy. Nhà phát triển thường chọn đối tượng theo tên mà không đọc những chú thích dài của bạn, thậm chí không xem danh sách phương thức của lớp.

Một ví dụ tệ hại về tên gây hiểu lầm là dùng chữ l thường hoặc chữ O hoa làm tên biến, nhất là khi dùng cả hai. Chúng gần như không khác gì các hằng số 1 và 0.

```
int a = l;
if ( O == l )
    a = O1;
else
    l = 01;
```

Có thể bạn nghĩ đây là ví dụ bịa, nhưng chúng tôi từng thấy những đoạn mã đầy những cái tên như vậy. Có lần, tác giả đề nghị đổi sang phông chữ khác để các ký tự trông khác nhau hơn. Giải pháp ấy sẽ phải được truyền lại cho mọi nhà phát triển về sau, bằng lời hoặc bằng văn bản. Đổi tên là đủ để xử lý dứt điểm vấn đề, không cần tạo thêm tài liệu hay sản phẩm nào.



Tạo ra những khác biệt có nghĩa

Lập trình viên tự gây rắc rối khi viết mã chỉ để làm vừa lòng trình biên dịch hoặc trình thông dịch. Chẳng hạn, vì không thể dùng cùng một tên cho hai thứ khác nhau trong cùng phạm vi, ta có thể tùy tiện sửa một cái tên. Đôi khi người ta còn viết sai chính tả tên đó, dẫn đến tình huống trớ trêu: sửa lỗi chính tả xong thì mã không biên dịch được nữa.[2]

Thêm dãy số hay từ nhiễu tuy làm trình biên dịch hài lòng, nhưng như thế chưa đủ.

2. Hãy nghĩ đến thói quen thật kinh khủng là tạo biến tên klass chỉ vì tên class đã được dùng cho thứ khác.

Nếu tên phải khác nhau thì ý nghĩa của chúng cũng phải khác nhau.

Đặt tên theo dãy số (a1, a2, .. aN) đi ngược lại việc đặt tên có chủ đích. Những tên như vậy không gây hiểu lầm, nhưng cũng chẳng cung cấp thông tin hay manh mối nào về ý định của tác giả. Xem ví dụ:

```
public static void copyChars(char a1[], char a2[]) {
    for (int i = 0; i < a1.length; i++) {
        a2[i] = a1[i];
    }
}
```

Hàm này dễ đọc hơn nhiều nếu dùng source và destination làm tên đối số.

Từ nhiễu cũng tạo ra khác biệt vô nghĩa. Giả sử bạn có một lớp Product, rồi thêm lớp ProductInfo hoặc ProductData. Bạn đã làm tên khác nhau nhưng không làm ý nghĩa của chúng khác đi. Info và Data là những từ nhiễu mơ hồ, giống như a, an và the.

Dùng tiền tố như a và the không có gì sai nếu chúng tạo ra khác biệt có ý nghĩa. Chẳng hạn, bạn có thể dùng a cho mọi biến cục bộ và the cho mọi đối số của hàm.[3] Vấn đề xuất hiện khi bạn gọi một biến là theZork chỉ vì đã có biến khác tên zork.

Từ nhiễu là phần thừa. Từ variable không nên xuất hiện trong tên biến, cũng như table không nên xuất hiện trong tên bảng. NameString hơn Name ở điểm nào? Name có khi nào lại là một số dấu phẩy động không? Nếu có thì cái tên ấy vi phạm quy tắc về tên gây hiểu lầm đã nói ở trên. Hãy tưởng tượng bạn gặp một lớp tên Customer và một lớp tên CustomerObject. Bạn sẽ hiểu hai lớp khác nhau ở điểm nào? Lớp nào giúp bạn tìm đến lịch sử thanh toán của khách hàng?

Có một ứng dụng chúng tôi biết minh họa rõ điều này. Chúng tôi đã đổi tên để không nêu đích danh người có lỗi, nhưng dạng lỗi thì vẫn nguyên như sau:

```
getActiveAccount();
getActiveAccounts();
getActiveAccountInfo();
```

Làm sao lập trình viên trong dự án biết nên gọi hàm nào?

Nếu không có quy ước cụ thể, moneyAmount chẳng khác gì money, customerInfo chẳng khác gì customer, accountData chẳng khác gì account, còn theMessage chẳng khác gì message. Hãy đặt tên sao cho người đọc hiểu được sự khác biệt giữa chúng có ý nghĩa gì.

Dùng tên đọc được

Con người giỏi xử lý từ ngữ. Một phần đáng kể bộ não dành cho khái niệm từ ngữ, mà từ ngữ thì có thể phát âm. Thật phí nếu không tận dụng phần lớn bộ não đã tiến hóa để xử lý ngôn ngữ nói. Vì vậy, hãy đặt những cái tên có thể phát âm được.

3. Uncle Bob từng làm vậy trong C++ nhưng đã bỏ thói quen này vì các IDE hiện đại khiến nó không còn cần thiết.

Tên khó phát âm khiến bạn khó bàn luận về nó mà không nghe như kẻ ngốc. "À, ở đằng này, trên cái bee cee arr three cee enn tee, chúng ta có một pee ess zee kyew int, thấy chưa?" Điều này quan trọng vì lập trình là hoạt động xã hội.

Có một công ty tôi biết dùng tên genymdhms (generation date, year, month, day, hour, minute, and second: ngày giờ sinh, năm, tháng, ngày, giờ, phút, giây), nên mọi người đi quanh đọc nó là "gen why emm dee aich emm ess". Tôi có thói quen khó chịu là đọc mọi thứ đúng như cách viết, nên bắt đầu gọi nó là "gen-yah-mudda-hims". Sau đó, nhiều nhà thiết kế và chuyên viên phân tích cũng gọi theo tôi. Nghe vẫn ngớ ngẩn, nhưng chúng tôi cùng hiểu câu đùa nên thấy vui. Dù vui hay không, chúng tôi vẫn đang chấp nhận một cách đặt tên tồi. Lập trình viên mới phải được giải thích các biến, rồi họ gọi chúng bằng những từ bịa ngớ ngẩn thay vì dùng thuật ngữ tiếng Anh đúng nghĩa. Hãy so sánh:

```
class DtaRcrd102 {
    private Date genymdhms;
    private Date modymdhms;
    private final String pszqint = "102";
    /* ... */
};
```

với

```
class Customer {
    private Date generationTimestamp;
    private Date modificationTimestamp;;
    private final String recordId = "102";
    /* ... */
};
```

Giờ ta có thể nói chuyện cho ra lẽ: "Này Mikey, xem bản ghi này đi! Dấu thời gian sinh lại được đặt vào ngày mai! Sao có thể thế được?"

Dùng tên tìm kiếm được

Tên một chữ cái và hằng số có một nhược điểm: khó tìm trong một khối văn bản.

Ta có thể dễ dàng grep MAX_CLASSES_PER_STUDENT, nhưng tìm số 7 thì rắc rối hơn. Kết quả có thể cho ra chữ số đó trong tên tệp, định nghĩa các hằng khác, và đủ kiểu biểu thức dùng giá trị ấy với ý nghĩa khác. Còn tệ hơn nếu hằng là một số dài, ai đó đảo vị trí các chữ số, tạo ra lỗi mà công cụ tìm kiếm của lập trình viên không phát hiện được.

Tương tự, e là tên tồi cho một biến mà lập trình viên có thể cần tìm. Đây là chữ cái xuất hiện nhiều nhất trong tiếng Anh và có thể có mặt ở bất kỳ đoạn văn bản nào trong bất kỳ chương trình nào. Vì vậy, tên dài thường dễ tìm hơn tên ngắn, và tên nào tìm được cũng tốt hơn một hằng số viết trực tiếp trong mã.

Theo sở thích của tôi, tên một chữ cái CHỈ nên dùng cho biến cục bộ trong các phương thức ngắn. Độ dài tên nên tương ứng với kích thước phạm vi của nó [N5]. Nếu biến hay hằng có thể được thấy hoặc dùng ở nhiều chỗ trong khối mã, nhất thiết phải đặt cho nó một cái tên dễ tìm. Một lần nữa, hãy so sánh:

```
for (int j=0; j<34; j++) {
    s += (t[j]*4)/5;
}
```

với

```
int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;
for (int j=0; j < NUMBER_OF_TASKS; j++) {
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
    int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);
    sum += realTaskWeeks;
}
```

Lưu ý, sum không phải cái tên hữu ích lắm, nhưng ít nhất có thể tìm được. Đoạn mã đặt tên có chủ đích dài hơn. Tuy nhiên, hãy nghĩ xem tìm WORK_DAYS_PER_WEEK dễ hơn bao nhiêu so với tìm mọi chỗ dùng số 5 rồi lọc ra những chỗ dùng nó đúng với ý nghĩa cần tìm.

Tránh mã hóa trong tên

Ta đã phải xử lý đủ thứ mã hóa rồi, đừng tự tạo thêm gánh nặng. Mã hóa kiểu hoặc phạm vi vào tên chỉ khiến người đọc phải tốn công giải mã. Bắt mỗi nhân viên mới học thêm một "ngôn ngữ" mã hóa bên cạnh khối mã, thường khá lớn, mà họ phải làm việc cùng thì thật không hợp lý. Đó là gánh nặng tinh thần không cần thiết khi giải quyết vấn đề. Tên đã mã hóa thường khó phát âm và dễ gõ sai.

Ký pháp Hungary

Ngày trước, khi làm việc với những ngôn ngữ giới hạn độ dài tên, ta đành vi phạm quy tắc này vì bắt buộc, và chẳng vui vẻ gì. Fortran buộc ta mã hóa bằng cách dùng chữ cái đầu làm ký hiệu kiểu dữ liệu. Các phiên bản BASIC đầu tiên chỉ cho phép một chữ cái cộng với một chữ số. Ký pháp Hungary (Hungarian Notation, HN) đẩy cách làm này đi xa hơn nhiều.

Thời Windows C API, HN được xem là khá quan trọng vì lúc ấy mọi thứ đều là handle kiểu integer, con trỏ long, con trỏ void, hoặc một trong vài cách hiện thực "string" với công dụng và thuộc tính khác nhau. Trình biên dịch khi đó không kiểm tra kiểu, nên lập trình viên cần một cái nạng để nhớ kiểu dữ liệu.

Ngôn ngữ hiện đại có hệ thống kiểu phong phú hơn nhiều, trình biên dịch ghi nhớ và kiểm tra kiểu. Lớp cũng có xu hướng nhỏ hơn, hàm ngắn hơn, nhờ vậy ta thường nhìn thấy chỗ khai báo của từng biến đang dùng.

Lập trình viên Java không cần mã hóa kiểu. Đối tượng có kiểu mạnh, còn môi trường soạn thảo phát hiện lỗi kiểu từ lâu trước khi bạn chạy biên dịch! Ngày nay, HN và các kiểu mã hóa dữ liệu khác chỉ cản trở công việc. Chúng khiến việc đổi tên hoặc đổi kiểu của biến,

hàm hay lớp khó hơn, làm mã khó đọc hơn, và có thể khiến hệ thống mã hóa đánh lừa người đọc.

```
PhoneNumber phoneString;
// tên không đổi khi kiểu đã đổi!
```

Tiền tố cho biến thành viên

Bạn cũng không cần thêm tiền tố m_ vào biến thành viên nữa. Lớp và hàm nên đủ nhỏ để không cần tiền tố đó. Hãy dùng môi trường soạn thảo có thể làm nổi bật hoặc tô màu thành viên để phân biệt chúng.

```
public class Part {
    private String m_dsc; // phần mô tả dạng văn bản
    void setName(String name) {
        m_dsc = name;
    }
}
```

```
public class Part {
    String description;
    void setDescription(String description) {
        this.description = description;
    }
}
```

Hơn nữa, người ta nhanh chóng học cách bỏ qua tiền tố (hoặc hậu tố) để nhìn vào phần có nghĩa của tên. Càng đọc mã, ta càng ít để ý đến tiền tố. Rồi chúng thành đống lộn xộn vô hình và là dấu hiệu của mã cũ.

Giao diện và lớp hiện thực

Đôi khi giao diện và lớp hiện thực cũng là một trường hợp mã hóa. Chẳng hạn, giả sử bạn đang xây dựng một ABSTRACT FACTORY để tạo hình. Factory này sẽ là giao diện, còn một lớp cụ thể sẽ hiện thực nó. Bạn nên đặt tên thế nào? IShapeFactory và ShapeFactory? Tôi thích để giao diện không có tiền tố trang trí. Chữ I ở đầu tên, vốn rất phổ biến trong những khối mã cũ ngày nay, gây xao nhãng trong trường hợp tốt nhất và thừa thông tin trong trường hợp xấu nhất. Tôi không muốn người dùng biết mình đang cung cấp cho họ một giao diện. Tôi chỉ muốn họ biết đó là ShapeFactory. Nếu buộc phải mã hóa tên giao diện hoặc lớp hiện thực, tôi sẽ chọn lớp hiện thực. Gọi nó là ShapeFactoryImp, hay thậm chí CShapeFactory đáng ghét, vẫn hơn là mã hóa giao diện.

Tránh ánh xạ trong đầu

Người đọc không nên phải tự dịch tên bạn đặt sang những tên khác mà họ đã biết. Vấn đề thường xuất hiện khi ta chọn từ ngữ không thuộc miền bài toán lẫn miền giải pháp.

Tên biến một chữ cái là một ví dụ. Biến đếm vòng lặp có thể tên i, j hoặc k (nhưng đừng bao giờ dùng l!) nếu phạm vi của nó rất nhỏ và không xung đột với tên nào khác. Các tên một chữ cái này đã thành quy ước cho biến đếm vòng lặp. Tuy nhiên, ở hầu hết bối cảnh khác, tên một chữ cái là lựa chọn tồi: nó chỉ là chỗ trống để người đọc tự liên hệ với khái niệm thật trong đầu. Không có lý do nào tệ hơn để đặt tên c ngoài chuyện a và b đã bị dùng mất.

Nói chung, lập trình viên khá thông minh. Người thông minh đôi khi thích khoe khả năng xoay xở trong đầu. Nếu bạn có thể nhớ chắc rằng r là phiên bản viết thường của url sau khi bỏ host và scheme, hẳn bạn rất thông minh.

Lập trình viên chuyên nghiệp khác lập trình viên thông minh ở chỗ họ hiểu rằng sự rõ ràng quan trọng hơn cả. Họ dùng năng lực của mình vào việc có ích và viết mã để người khác hiểu được.

Tên lớp

Lớp và đối tượng nên có tên là danh từ hoặc cụm danh từ, chẳng hạn Customer, WikiPage, Account và AddressParser. Tránh những từ như Manager, Processor, Data hay Info trong tên lớp. Tên lớp không nên là động từ.

Tên phương thức

Phương thức nên có tên là động từ hoặc cụm động từ, chẳng hạn postPayment, deletePage hay save. Accessor, mutator và predicate nên được đặt tên theo giá trị của chúng, với tiền tố get, set và is theo chuẩn javabean.[4]

```
string name = employee.getName();
customer.setName("mike");
if (paycheck.isPosted())...
```

Khi nạp chồng hàm khởi tạo, hãy dùng các phương thức factory tĩnh có tên mô tả đối số. Ví dụ:

```
Complex fulcrumPoint = Complex.FromRealNumber(23.0);
```

thường tốt hơn:

```
Complex fulcrumPoint = new Complex(23.0);
```

4. http://java.sun.com/products/javabeans/docs/spec.html

Hãy cân nhắc buộc người dùng gọi các phương thức ấy bằng cách đặt những hàm khởi tạo tương ứng ở mức private.

Đừng tỏ ra dí dỏm



Tên quá khéo léo chỉ dễ nhớ với những người có cùng khiếu hài hước với tác giả, và chỉ khi họ còn nhớ câu đùa. Họ có biết hàm tên HolyHandGrenade dùng để làm gì không? Đúng là nghe vui, nhưng ở đây DeleteItems có lẽ sẽ là cái tên tốt hơn. Hãy chọn sự rõ ràng thay vì tính giải trí.

Tên dí dỏm trong mã thường là tiếng lóng hoặc cách nói thông tục. Chẳng hạn, đừng dùng whack() với nghĩa kill(). Cũng đừng đưa vào những câu đùa nhỏ phụ thuộc vào văn hóa như eatMyShorts() với nghĩa abort().

Nói đúng ý mình. Giữ đúng lời mình nói.

Mỗi khái niệm chọn một từ

Hãy chọn một từ cho mỗi khái niệm trừu tượng rồi dùng nhất quán. Chẳng hạn, thật khó hiểu nếu các lớp khác nhau có những phương thức tương đương tên fetch, retrieve và get. Làm sao bạn nhớ tên nào đi với lớp nào? Tiếc là thường phải nhớ công ty, nhóm hay cá nhân nào viết thư viện hoặc lớp đó mới biết họ dùng thuật ngữ nào. Nếu không, bạn sẽ mất nhiều thời gian xem các tệp header và những ví dụ mã cũ.

Các môi trường soạn thảo hiện đại như Eclipse và IntelliJ cung cấp gợi ý theo ngữ cảnh, chẳng hạn danh sách phương thức có thể gọi trên một đối tượng. Nhưng danh sách ấy thường không hiển thị chú thích bạn viết quanh tên hàm và danh sách tham số. May ra nó cho thấy tên tham số trong khai báo hàm. Tên hàm phải tự đứng vững và nhất quán để bạn chọn đúng phương thức mà không cần tìm hiểu thêm.

Tương tự, thật khó hiểu khi cùng một code base có controller, manager và driver. Khác biệt cốt yếu giữa DeviceManager và ProtocolController là gì? Sao cả hai không cùng là controller hoặc cùng là manager? Thật ra chúng có phải đều là Driver không? Tên gọi khiến bạn nghĩ hai đối tượng khác xa nhau về kiểu lẫn lớp.

Từ vựng nhất quán là món quà lớn dành cho những lập trình viên phải dùng mã của

bạn.

Đừng chơi chữ

Tránh dùng cùng một từ cho hai mục đích. Dùng một thuật ngữ cho hai ý tưởng khác nhau về bản chất chính là chơi chữ.

Nếu theo quy tắc "mỗi khái niệm một từ", bạn có thể có nhiều lớp cùng có phương thức add. Chừng nào danh sách tham số và giá trị trả về của các phương thức add còn tương đương về ngữ nghĩa thì không có vấn đề gì.

Tuy nhiên, đôi khi người ta dùng từ add để tạo "tính nhất quán" dù thực chất không thêm theo cùng một nghĩa. Giả sử ta có nhiều lớp trong đó add tạo ra giá trị mới bằng cách cộng hoặc nối hai giá trị có sẵn. Giờ ta viết một lớp mới có phương thức đưa đối số duy nhất của nó vào một collection. Có nên gọi phương thức ấy là add không? Nghe có vẻ nhất quán vì đã có nhiều phương thức add, nhưng ngữ nghĩa ở đây khác. Ta nên dùng tên như insert hoặc append. Gọi phương thức mới là add sẽ thành chơi chữ.

Với tư cách tác giả, mục tiêu của chúng ta là làm cho mã dễ hiểu hết mức có thể. Ta muốn người đọc lướt qua là nắm được, không phải nghiền ngẫm. Ta nên theo cách viết sách phổ thông: tác giả có trách nhiệm diễn đạt rõ ràng, chứ không theo lối học thuật, nơi học giả phải đào bới ý nghĩa từ trang giấy.

Dùng tên thuộc miền giải pháp

Hãy nhớ người đọc mã của bạn là lập trình viên. Vì vậy, cứ dùng thuật ngữ khoa học máy tính (CS), tên thuật toán, tên pattern, thuật ngữ toán học, vân vân. Không nên lấy mọi tên từ miền bài toán, vì ta không muốn đồng nghiệp cứ phải chạy đi hỏi khách hàng mỗi cái tên nghĩa là gì trong khi họ đã biết khái niệm ấy dưới một tên khác.

Tên AccountVisitor gợi nhiều ý nghĩa cho lập trình viên biết pattern VISITOR. Lập trình viên nào lại không biết JobQueue là gì? Lập trình viên phải làm nhiều việc mang tính kỹ thuật. Thường nên chọn tên kỹ thuật cho những việc đó.

Dùng tên thuộc miền bài toán

Khi không có thuật ngữ lập trình phù hợp với việc đang làm, hãy dùng tên trong miền bài toán. Ít ra, lập trình viên bảo trì mã của bạn có thể hỏi chuyên gia nghiệp vụ để biết nó nghĩa là gì.

Phân biệt khái niệm thuộc miền giải pháp với khái niệm thuộc miền bài toán là một phần công việc của lập trình viên và nhà thiết kế giỏi. Đoạn mã nào gắn nhiều hơn với khái niệm miền bài toán thì nên dùng tên lấy từ miền đó.

Thêm bối cảnh có nghĩa

Một số tên tự thân đã có nghĩa, nhưng phần lớn thì không. Hãy đặt tên vào bối cảnh để người đọc hiểu chúng, chẳng hạn bằng cách đưa chúng vào những lớp, hàm hoặc namespace có tên rõ ràng. Nếu mọi cách khác đều không hiệu quả, thêm tiền tố cho tên có thể là giải pháp cuối cùng.

Hãy tưởng tượng bạn có các biến firstName, lastName, street, houseNumber, city, state và zipcode. Đặt cạnh nhau, chúng rõ ràng tạo thành một địa chỉ. Nhưng nếu chỉ thấy biến state được dùng riêng trong một phương thức thì sao? Bạn có tự hiểu nó là một phần của địa chỉ không?

Có thể thêm bối cảnh bằng tiền tố: addrFirstName, addrLastName, addrState, vân vân. Như vậy, ít nhất người đọc biết các biến thuộc về một cấu trúc lớn hơn. Tốt hơn nữa là tạo lớp Address. Khi ấy, ngay cả trình biên dịch cũng biết các biến thuộc về một khái niệm rộng hơn.

Xem phương thức trong Đoạn mã 2-1. Các biến có cần bối cảnh rõ hơn không? Tên hàm chỉ cho biết một phần bối cảnh, phần còn lại nằm trong thuật toán. Đọc hết hàm, bạn sẽ thấy number, verb và pluralModifier là ba biến thuộc về thông điệp "thống kê phỏng đoán". Tiếc là người đọc phải tự suy ra bối cảnh ấy. Mới nhìn vào phương thức, ý nghĩa các biến còn rất mờ nhạt.

Đoạn mã 2-1
Các biến có bối cảnh không rõ ràng.

```
private void printGuessStatistics(char candidate, int count) {
    String number;
    String verb;
    String pluralModifier;
    if (count == 0) {
        number = "no";
        verb = "are";
        pluralModifier = "s";
    } else if (count == 1) {
        number = "1";
        verb = "is";
        pluralModifier = "";
    } else {
        number = Integer.toString(count);
        verb = "are";
        pluralModifier = "s";
    }
    String guessMessage = String.format(
        "There %s %s %s%s", verb, number, candidate, pluralModifier
    );
    print(guessMessage);
}
```

Hàm hơi dài, còn các biến được dùng xuyên suốt. Muốn tách hàm thành những phần nhỏ hơn, ta cần tạo lớp GuessStatisticsMessage rồi chuyển ba biến thành các trường của lớp. Nhờ đó, ba biến có bối cảnh rõ ràng: chúng chắc chắn thuộc về GuessStatisticsMessage.

Bối cảnh rõ hơn cũng giúp ta chia thuật toán thành nhiều hàm nhỏ và làm nó gọn gàng hơn. (Xem Đoạn mã 2-2.)

Đoạn mã 2-2
Các biến có bối cảnh.

```
public class GuessStatisticsMessage {
    private String number;
    private String verb;
    private String pluralModifier;

    public String make(char candidate, int count) {
        createPluralDependentMessageParts(count);
        return String.format(
            "There %s %s %s%s",
            verb, number, candidate, pluralModifier );
    }

    private void createPluralDependentMessageParts(int count) {
        if (count == 0) {
            thereAreNoLetters();
        } else if (count == 1) {
            thereIsOneLetter();
        } else {
            thereAreManyLetters(count);
        }
    }

    private void thereAreManyLetters(int count) {
        number = Integer.toString(count);
        verb = "are";
        pluralModifier = "s";
    }

    private void thereIsOneLetter() {
        number = "1";
        verb = "is";
        pluralModifier = "";
    }

    private void thereAreNoLetters() {
        number = "no";
        verb = "are";
        pluralModifier = "s";
    }
}
```

Đừng thêm bối cảnh vô cớ

Trong ứng dụng tưởng tượng tên "Gas Station Deluxe", thêm tiền tố GSD vào tên mọi lớp là một ý tồi. Nói thẳng ra, làm thế là chống lại chính công cụ của bạn. Gõ G rồi nhấn phím hoàn thành, bạn sẽ nhận một danh sách dài dằng dặc gồm mọi lớp trong hệ thống. Có khôn ngoan không? Sao lại khiến IDE khó hỗ trợ bạn hơn?

Tương tự, giả sử bạn tạo lớp MailingAddress trong module kế toán của GSD và gọi nó là GSDAccountAddress. Sau này, ứng dụng liên hệ khách hàng cần một địa chỉ gửi thư. Bạn có dùng GSDAccountAddress không? Tên đó nghe có đúng không? Mười trong số mười bảy ký tự là thừa hoặc không liên quan.

Nhìn chung, tên ngắn tốt hơn tên dài nếu vẫn rõ nghĩa. Đừng thêm bối cảnh nhiều hơn mức cần thiết.

accountAddress và customerAddress là những tên hợp lý cho các đối tượng thuộc lớp Address, nhưng có thể là tên tồi cho lớp. Address là tên lớp phù hợp. Nếu cần phân biệt địa chỉ MAC, địa chỉ cổng và địa chỉ Web, tôi có thể chọn PostalAddress, MAC và URI. Các tên này chính xác hơn, đó cũng là mục đích của việc đặt tên.

Lời cuối

Điều khó nhất khi chọn tên hay là cần có khả năng mô tả tốt và nền tảng văn hóa chung. Đây là vấn đề dạy và học, không phải vấn đề kỹ thuật, kinh doanh hay quản lý. Vì vậy, nhiều người trong ngành không học được cách làm việc này cho tốt.

Người ta cũng ngại đổi tên vì sợ nhà phát triển khác phản đối. Chúng tôi không ngại như vậy, và thật sự biết ơn mỗi khi tên được đổi theo hướng tốt hơn. Phần lớn thời gian, ta không cần ghi nhớ tên lớp và phương thức. Công cụ hiện đại lo những chi tiết ấy, để ta tập trung xem mã có đọc như đoạn văn, câu văn, hoặc ít nhất như bảng và cấu trúc dữ liệu hay không. (Câu văn không phải lúc nào cũng là cách tốt nhất để trình bày dữ liệu.) Khi đổi tên, có lẽ bạn sẽ làm ai đó ngạc nhiên, cũng như khi thực hiện bất kỳ cải tiến mã nào khác. Đừng để chuyện đó cản bước.

Hãy thử áp dụng một vài quy tắc này để xem khả năng đọc mã có cải thiện không. Nếu đang bảo trì mã của người khác, hãy dùng công cụ tái cấu trúc để xử lý những vấn đề ấy. Công sức bỏ ra sẽ có ích trước mắt và cả về sau.

3

Hàm



Thuở sơ khai của lập trình, chúng ta dựng hệ thống từ các routine và subroutine. Rồi đến thời Fortran và PL/1, chúng ta dựng hệ thống từ program, subprogram và function. Ngày nay, trong những thứ thuở ấy chỉ còn function sống sót. Hàm là cấp tổ chức đầu tiên trong mọi chương trình. Viết hàm cho tốt là chủ đề của chương này.

Hãy xem đoạn mã trong Đoạn mã 3-1. Rất khó tìm một hàm dài trong FitNesse[1], nhưng

1. Một công cụ kiểm thử mã nguồn mở. www.fitnese.org

sau một hồi lục lọi tôi cũng vấp phải hàm này. Nó không chỉ dài mà còn có mã trùng lặp, đầy những chuỗi kỳ quặc, cùng nhiều kiểu dữ liệu và API lạ lùng, khó đoán. Bạn thử xem trong ba phút tới mình hiểu được bao nhiêu.

Đoạn mã 3-1
HtmlUtil.java (FitNesse 20070619)

```
public static String testableHtml(
  PageData pageData,
  boolean includeSuiteSetup
) throws Exception {
  WikiPage wikiPage = pageData.getWikiPage();
  StringBuffer buffer = new StringBuffer();
  if (pageData.hasAttribute("Test")) {
    if (includeSuiteSetup) {
      WikiPage suiteSetup =
        PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_SETUP_NAME, wikiPage
        );
      if (suiteSetup != null) {
        WikiPagePath pagePath =
          suiteSetup.getPageCrawler().getFullPath(suiteSetup);
        String pagePathName = PathParser.render(pagePath);
        buffer.append("!include -setup .")
              .append(pagePathName)
              .append("\n");
      }
    }
    WikiPage setup =
      PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
    if (setup != null) {
      WikiPagePath setupPath =
        wikiPage.getPageCrawler().getFullPath(setup);
      String setupPathName = PathParser.render(setupPath);
      buffer.append("!include -setup .")
            .append(setupPathName)
            .append("\n");
    }
  }
  buffer.append(pageData.getContent());
  if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
      PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
      WikiPagePath tearDownPath =
        wikiPage.getPageCrawler().getFullPath(teardown);
      String tearDownPathName = PathParser.render(tearDownPath);
      buffer.append("\n")
            .append("!include -teardown .")
            .append(tearDownPathName)
            .append("\n");
    }
```

Đoạn mã 3-1 (tiếp theo)
HtmlUtil.java (FitNesse 20070619)

```
    if (includeSuiteSetup) {
      WikiPage suiteTeardown =
        PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_TEARDOWN_NAME,
                wikiPage
        );
      if (suiteTeardown != null) {
        WikiPagePath pagePath =
          suiteTeardown.getPageCrawler().getFullPath (suiteTeardown);
        String pagePathName = PathParser.render(pagePath);
        buffer.append("!include -teardown .")
              .append(pagePathName)
              .append("\n");
      }
    }
  }
  pageData.setContent(buffer.toString());
  return pageData.getHtml();
}
```

Ba phút nghiên cứu xong, bạn đã hiểu hàm này chưa? Chắc là chưa. Trong đó có quá nhiều thứ đang diễn ra, ở quá nhiều mức trừu tượng khác nhau. Có những chuỗi kỳ quặc và những lời gọi hàm lạ lùng chen giữa các câu lệnh if lồng đôi được điều khiển bằng cờ.

Ấy vậy mà chỉ với vài phép tách phương thức đơn giản, vài lần đổi tên và một chút sắp xếp lại, tôi đã nắm được ý định của hàm trong chín dòng của Đoạn mã 3-2. Bạn xem trong ba phút tới mình có hiểu được đoạn này không.

Đoạn mã 3-2
HtmlUtil.java (đã tái cấu trúc)

```
public static String renderPageWithSetupsAndTeardowns(
  PageData pageData, boolean isSuite
) throws Exception {
  boolean isTestPage = pageData.hasAttribute("Test");
  if (isTestPage) {
    WikiPage testPage = pageData.getWikiPage();
    StringBuffer newPageContent = new StringBuffer();
    includeSetupPages(testPage, newPageContent, isSuite);
    newPageContent.append(pageData.getContent());
    includeTeardownPages(testPage, newPageContent, isSuite);
    pageData.setContent(newPageContent.toString());
  }

  return pageData.getHtml();
}
```

Nếu không phải dân trong nghề FitNesse, bạn có lẽ không hiểu hết mọi chi tiết. Nhưng chắc bạn hiểu được rằng hàm này chèn một số trang thiết lập và dọn dẹp vào một trang kiểm thử

rồi kết xuất trang đó ra HTML. Nếu đã quen với JUnit[2], bạn có lẽ nhận ra hàm này thuộc về một kiểu framework kiểm thử chạy trên Web nào đó. Và dĩ nhiên, đúng là như vậy. Đoán ra thông tin ấy từ Đoạn mã 3-2 khá dễ, nhưng Đoạn mã 3-1 thì che nó đi khá kỹ.

Vậy điều gì khiến một hàm như trong Đoạn mã 3-2 trở nên dễ đọc, dễ hiểu? Làm sao để một hàm truyền đạt được ý định của mình? Ta có thể trao cho hàm những phẩm chất gì để người đọc thông thường cũng linh cảm được kiểu chương trình mà hàm ấy đang sống trong đó?

Nhỏ!

Quy tắc đầu tiên của hàm là chúng phải nhỏ. Quy tắc thứ hai của hàm là chúng phải nhỏ hơn thế. Đây không phải điều tôi có thể chứng minh. Tôi không có nghiên cứu nào cho thấy hàm rất nhỏ thì tốt hơn. Điều tôi có thể nói là gần bốn thập kỷ qua tôi đã viết hàm đủ mọi cỡ. Tôi đã viết vài con quái vật 3.000 dòng kinh khủng. Tôi đã viết hàng đống hàm dài 100 đến 300 dòng. Và tôi cũng đã viết những hàm chỉ 20 đến 30 dòng. Kinh nghiệm ấy, qua bao lần thử sai, dạy tôi rằng hàm nên rất nhỏ.

Thập niên tám mươi, chúng ta thường nói rằng một hàm không nên lớn hơn một màn hình. Dĩ nhiên, chúng ta nói vậy vào thời màn hình VT100 chỉ có 24 dòng 80 cột, và trình soạn thảo chiếm mất 4 dòng để hiển thị thông tin quản lý. Ngày nay, với cỡ chữ thu nhỏ và một màn hình to đẹp, bạn có thể nhét 150 ký tự vào một dòng và hơn 100 dòng vào một màn hình. Dòng thì không nên dài 150 ký tự. Hàm thì không nên dài 100 dòng. Hàm gần như không bao giờ nên dài 20 dòng.

Vậy hàm nên ngắn đến mức nào? Năm 1999, tôi đến thăm Kent Beck tại nhà anh ở Oregon. Chúng tôi ngồi xuống cùng lập trình. Có lúc anh cho tôi xem một chương trình Java/Swing nhỏ xinh mà anh gọi là Sparkle. Nó tạo ra hiệu ứng hình ảnh trên màn hình rất giống cây đũa thần của bà tiên đỡ đầu trong phim Cinderella. Khi bạn di chuột, những tia sáng lấp lánh sẽ nhỏ giọt từ con trỏ với một thứ ánh sáng đã mắt, rồi rơi xuống đáy cửa sổ qua một trường hấp dẫn mô phỏng. Khi Kent cho tôi xem mã, tôi sững người vì mọi hàm đều nhỏ đến thế. Tôi vốn quen với những hàm trong chương trình Swing chiếm hàng dặm chiều dọc. Ở chương trình này, mỗi hàm chỉ dài hai, ba hoặc bốn dòng. Hàm nào cũng trong suốt, hiển nhiên. Hàm nào cũng kể một câu chuyện. Và mỗi hàm dẫn bạn sang hàm tiếp theo trong một trình tự cuốn hút. Hàm của bạn nên ngắn đến mức ấy![3]

Hàm của bạn nên ngắn đến mức nào? Thường thì nên ngắn hơn Đoạn mã 3-2! Thật vậy, Đoạn mã 3-2 nên được rút gọn thành Đoạn mã 3-3.

Đoạn mã 3-3
HtmlUtil.java (tái cấu trúc lần nữa)

```
public static String renderPageWithSetupsAndTeardowns(
```

2. Một công cụ kiểm thử đơn vị mã nguồn mở cho Java. www.junit.org

3. Tôi hỏi Kent xem anh còn giữ một bản không, nhưng anh không tìm được. Tôi cũng lục tung các máy tính cũ của mình, vô ích. Giờ chỉ còn ký ức của tôi về chương trình đó.

```
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
      includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
  }
```

Khối lệnh và thụt lề

Điều này ngụ ý rằng khối lệnh bên trong các câu lệnh if, else, while và tương tự nên chỉ dài một dòng. Có lẽ dòng đó nên là một lời gọi hàm. Như vậy không những giữ cho hàm bao quanh nhỏ, mà còn thêm giá trị tài liệu, bởi hàm được gọi trong khối có thể mang một cái tên mô tả đẹp đẽ.

Điều này cũng ngụ ý rằng hàm không nên lớn đến mức chứa được các cấu trúc lồng nhau. Vì thế, mức thụt lề trong một hàm không nên vượt quá một hoặc hai. Dĩ nhiên, điều đó khiến hàm dễ đọc và dễ hiểu hơn.

Làm một việc



Phải thấy rõ rằng Đoạn mã 3-1 đang làm nhiều hơn một việc. Nó tạo bộ đệm, lấy trang, tìm các trang kế thừa, kết xuất đường dẫn, nối những chuỗi bí hiểm và sinh HTML, cùng nhiều việc khác. Đoạn mã 3-1 rất bận rộn với hàng loạt việc khác nhau. Ngược lại, Đoạn mã 3-3 chỉ làm một việc đơn giản. Nó chèn phần thiết lập và dọn dẹp vào các trang kiểm thử.

Lời khuyên sau đây đã xuất hiện dưới hình thức này hay hình thức khác suốt 30 năm hoặc hơn.

HÀM NÊN LÀM MỘT VIỆC. VÀ LÀM THẬT TỐT. CHỈ VIỆC ĐÓ MÀ THÔI.

Vấn đề với câu nói ấy là rất khó biết "một việc" nghĩa là gì. Đoạn mã 3-3 có làm một việc không? Dễ lập luận rằng nó đang làm ba việc:

1. Xác định xem trang có phải trang kiểm thử không.
2. Nếu phải, chèn phần thiết lập và dọn dẹp.

3. Kết xuất trang ra HTML.

Vậy rốt cuộc là sao? Hàm làm một việc hay ba việc? Hãy để ý rằng ba bước của hàm nằm ở mức trừu tượng thấp hơn một bậc so với cái tên đã đặt cho hàm. Ta có thể mô tả hàm bằng một đoạn TO[4] ngắn gọn:

> *TO RenderPageWithSetupsAndTeardowns, chúng ta kiểm tra xem trang có phải trang kiểm thử không; nếu phải, chúng ta chèn phần thiết lập và dọn dẹp. Dù thế nào, chúng ta cũng kết xuất trang ra HTML.*

Nếu một hàm chỉ làm những bước nằm dưới tên của nó đúng một mức, thì hàm ấy đang làm một việc. Suy cho cùng, lý do ta viết hàm là để phân rã một khái niệm lớn hơn (nói cách khác, tên của hàm) thành một tập bước ở mức trừu tượng kế tiếp.

Phải thấy rõ rằng Đoạn mã 3-1 chứa các bước ở rất nhiều mức trừu tượng khác nhau. Vậy nên rõ ràng nó làm nhiều hơn một việc. Ngay cả Đoạn mã 3-2 cũng có hai mức trừu tượng, bằng chứng là ta có thể thu nhỏ nó lại. Nhưng sẽ rất khó thu nhỏ Đoạn mã 3-3 một cách có ý nghĩa. Ta có thể tách câu lệnh if thành một hàm tên là includeSetupsAndTeardownsIfTestPage, nhưng làm vậy chỉ diễn đạt lại đoạn mã mà không đổi mức trừu tượng.

Vậy một cách khác để biết hàm đang làm nhiều hơn "một việc" là xem bạn có tách được khỏi nó một hàm khác với cái tên không chỉ là diễn đạt lại phần hiện thực hay không [G34].

Các mục trong hàm

Xem Đoạn mã 4-7 ở trang 71. Bạn sẽ thấy hàm generatePrimes được chia thành các mục như khai báo, khởi tạo và sàng. Đây là triệu chứng rõ ràng của việc làm nhiều hơn một việc. Những hàm chỉ làm một việc không thể bị chia thành các mục một cách hợp lý.

Mỗi hàm một mức trừu tượng

Để chắc chắn các hàm của mình đang làm "một việc", ta cần bảo đảm mọi câu lệnh trong hàm đều nằm ở cùng một mức trừu tượng. Dễ thấy Đoạn mã 3-1 vi phạm quy tắc này. Trong đó có những khái niệm ở mức trừu tượng rất cao, như getHtml(); có những khái niệm ở mức trung gian, như: `String pagePathName = PathParser.render(pagePath);` và lại có những thứ ở mức thấp đáng kinh ngạc, như: `.append("\n")`.

Trộn lẫn các mức trừu tượng trong một hàm thì luôn gây bối rối. Người đọc có thể không phân biệt nổi một biểu thức cụ thể là khái niệm thiết yếu hay chỉ là chi tiết. Tệ hơn, giống như những ô cửa sổ vỡ, một khi chi tiết đã trộn vào khái niệm thiết yếu thì trong hàm sẽ ngày càng tích tụ thêm chi tiết.

---

4. Ngôn ngữ LOGO dùng từ khóa "TO" theo đúng cách Ruby và Python dùng "def". Vì thế mọi hàm đều bắt đầu bằng chữ "TO". Điều này có ảnh hưởng thú vị đến cách các hàm được thiết kế.

Đọc mã từ trên xuống: Quy tắc bậc thang

Ta muốn mã đọc như một câu chuyện kể từ trên xuống[5]. Ta muốn mỗi hàm được theo sau bởi những hàm ở mức trừu tượng kế tiếp, để có thể đọc chương trình, mỗi lúc hạ xuống một mức trừu tượng khi đọc dần danh sách hàm. Tôi gọi đó là Quy tắc bậc thang.

Nói cách khác, ta muốn có thể đọc chương trình như thể nó là một tập các đoạn TO, mỗi đoạn mô tả mức trừu tượng hiện tại và tham chiếu tới những đoạn TO tiếp theo ở mức thấp hơn.

> *Để chèn phần thiết lập và dọn dẹp, chúng ta chèn phần thiết lập, rồi chèn nội dung trang kiểm thử, rồi chèn phần dọn dẹp.*
>
> *Để chèn phần thiết lập, chúng ta chèn thiết lập của suite nếu đây là một suite, rồi chèn thiết lập thường.*
>
> *Để chèn thiết lập của suite, chúng ta tìm trang "SuiteSetUp" trong hệ thống phân cấp cha và thêm một câu lệnh include kèm đường dẫn của trang đó.*
>
> *Để tìm trang cha. . .*

Suy cho cùng, việc học theo quy tắc này và viết được những hàm chỉ giữ một mức trừu tượng là rất khó đối với lập trình viên. Nhưng học được mẹo này cũng rất quan trọng. Đó là chìa khóa để giữ hàm ngắn và bảo đảm chúng làm "một việc". Làm cho mã đọc như một tập đoạn TO từ trên xuống là kỹ thuật hiệu quả để giữ mức trừu tượng nhất quán.

Hãy xem Đoạn mã 3-7 ở cuối chương này. Nó cho thấy toàn bộ hàm testableHtml đã được tái cấu trúc theo những nguyên tắc mô tả ở đây. Hãy để ý mỗi hàm giới thiệu hàm kế tiếp, và hàm nào cũng giữ một mức trừu tượng nhất quán.

Câu lệnh switch

Rất khó làm một câu lệnh switch nhỏ[6]. Ngay cả một câu lệnh switch chỉ có hai nhánh cũng lớn hơn mức tôi muốn ở một khối hay một hàm. Cũng rất khó làm một câu lệnh switch chỉ làm một việc. Bản chất của chúng là luôn làm N việc. Đáng tiếc, không phải lúc nào ta cũng tránh được switch, nhưng ta có thể bảo đảm mỗi câu lệnh switch được chôn trong một lớp mức thấp và không bao giờ bị lặp lại. Dĩ nhiên, ta làm điều đó bằng đa hình.

Xét Đoạn mã 3-4. Nó chỉ cho thấy một trong những thao tác có thể phụ thuộc vào loại nhân viên.

Đoạn mã 3-4
Payroll.java

```
public Money calculatePay(Employee e)
throws InvalidEmployeeType {
```

5. [KP78], tr. 37.

6. Và dĩ nhiên, tôi gộp cả những chuỗi if/else vào đây.

```
    switch (e.type) {
      case COMMISSIONED:
        return calculateCommissionedPay(e);
      case HOURLY:
        return calculateHourlyPay(e);
      case SALARIED:
        return calculateSalariedPay(e);
      default:
        throw new InvalidEmployeeType(e.type);
    }
  }
```

Hàm này có vài vấn đề. Thứ nhất, nó lớn, và khi có thêm loại nhân viên mới, nó sẽ phình ra. Thứ hai, rõ ràng nó làm nhiều hơn một việc. Thứ ba, nó vi phạm Nguyên lý Trách nhiệm Đơn (SRP)[7] vì có nhiều hơn một lý do để nó thay đổi. Thứ tư, nó vi phạm Nguyên lý Mở/Đóng (OCP)[8] vì nó buộc phải thay đổi mỗi khi thêm loại mới. Nhưng có lẽ vấn đề tệ nhất của hàm này là có vô số hàm khác sẽ có cùng cấu trúc. Chẳng hạn, ta có thể có `isPayday(Employee e, Date date)`, hoặc `deliverPay(Employee e, Money pay)`, hoặc cả một loạt hàm khác. Tất cả đều sẽ có cùng cấu trúc tai hại ấy.

Giải pháp cho vấn đề này (xem Đoạn mã 3-5) là chôn câu lệnh switch dưới tầng hầm của một ABSTRACT FACTORY[9], và không để ai nhìn thấy nó nữa. Nhà máy sẽ dùng câu lệnh switch để tạo các thực thể phù hợp của những lớp dẫn xuất từ Employee, còn các hàm khác nhau như calculatePay, isPayday và deliverPay sẽ được điều phối đa hình qua giao diện Employee.

Quy tắc chung của tôi với câu lệnh switch là có thể chấp nhận chúng nếu chúng chỉ xuất hiện một lần, được dùng để tạo các đối tượng đa hình, và được giấu sau một quan hệ kế thừa để phần còn lại của hệ thống không nhìn thấy chúng [G23]. Dĩ nhiên, mỗi hoàn cảnh đều riêng biệt, và có những lúc tôi vi phạm một hoặc vài phần của quy tắc ấy.

Đoạn mã 3-5
Employee and Factory

```
public abstract class Employee {
  public abstract boolean isPayday();
  public abstract Money calculatePay();
  public abstract void deliverPay(Money pay);
}
-----------------
public interface EmployeeFactory {
  public Employee makeEmployee(EmployeeRecord r) throws InvalidEmployeeType;
}
-----------------
public class EmployeeFactoryImpl implements EmployeeFactory {
```

7. a. http://en.wikipedia.org/wiki/Single_responsibility_principle
   b. http://www.objectmentor.com/resources/articles/srp.pdf
8. a. http://en.wikipedia.org/wiki/Open/closed_principle
   b. http://www.objectmentor.com/resources/articles/ocp.pdf
9. [GOF].

```
    public Employee makeEmployee(EmployeeRecord r) throws InvalidEmployeeType {
      switch (r.type) {
        case COMMISSIONED:
          return new CommissionedEmployee(r) ;
        case HOURLY:
          return new HourlyEmployee(r);
        case SALARIED:
          return new SalariedEmploye(r);
        default:
          throw new InvalidEmployeeType(r.type);
      }
    }
  }
```

Dùng tên mô tả

Trong Đoạn mã 3-7, tôi đổi tên hàm ví dụ từ testableHtml thành SetupTeardownIncluder.render. Đây là cái tên tốt hơn nhiều vì nó mô tả đúng hơn việc hàm làm. Tôi cũng đặt cho mỗi phương thức riêng một cái tên mô tả tương xứng, chẳng hạn isTestable hoặc includeSetupAndTeardownPages. Thật khó mà đánh giá quá cao giá trị của những cái tên hay. Hãy nhớ nguyên lý của Ward: "Bạn biết mình đang làm việc với mã sạch khi mỗi thủ tục hóa ra đúng như bạn mong đợi." Một nửa trận chiến để đạt được nguyên lý ấy là chọn tên hay cho những hàm nhỏ chỉ làm một việc. Hàm càng nhỏ và càng tập trung bao nhiêu, càng dễ chọn cho nó một cái tên mô tả bấy nhiêu.

Đừng ngại đặt tên dài. Một cái tên dài mà mô tả vẫn hơn một cái tên ngắn mà bí hiểm. Một cái tên dài mà mô tả cũng hơn một chú thích dài mà mô tả. Hãy dùng một quy ước đặt tên cho phép đọc dễ dàng nhiều từ trong tên hàm, rồi tận dụng những từ ấy để đặt cho hàm một cái tên nói đúng việc nó làm.

Đừng ngại dành thời gian chọn tên. Thật vậy, bạn nên thử vài cái tên khác nhau và đọc đoạn mã với từng cái. Những IDE hiện đại như Eclipse hay IntelliJ khiến việc đổi tên trở nên dễ như trở bàn tay. Hãy dùng một trong những IDE ấy và thử nghiệm các tên khác nhau cho tới khi tìm được cái tên mô tả nhất mà bạn có thể nghĩ ra.

Chọn những cái tên mô tả sẽ làm thiết kế của module rõ ràng hơn trong đầu bạn và giúp bạn cải thiện nó. Không hiếm chuyện săn tìm một cái tên hay lại dẫn đến một lần tái cấu trúc có lợi cho mã.

Hãy nhất quán trong cách đặt tên. Dùng cùng những cụm từ, danh từ và động từ trong tên các hàm bạn chọn cho module của mình. Chẳng hạn, hãy xét các tên includeSetupAndTeardownPages, includeSetupPages, includeSuiteSetupPage và includeSetupPage. Cách dùng từ tương tự trong những cái tên ấy cho phép cả chuỗi ấy kể nên một câu chuyện. Thật vậy, nếu tôi chỉ đưa bạn xem chuỗi trên, bạn sẽ tự hỏi: "Thế includeTeardownPages, includeSuiteTeardownPage và includeTeardownPage đâu rồi?" Thấy chưa, đúng là ". . . hóa ra đúng như bạn mong đợi."



Đối số của hàm

Số đối số lý tưởng của một hàm là không (niladic). Tiếp đến là một (monadic), rồi sát ngay sau là hai (dyadic). Ba đối số (triadic) nên tránh khi có thể. Nhiều hơn ba (polyadic) đòi hỏi lý do rất đặc biệt, và dù có lý do thì cũng không nên dùng.

Đối số là thứ khó chơi. Chúng ngốn rất nhiều sức khái quát. Đó là lý do tôi loại bỏ gần như toàn bộ đối số khỏi ví dụ. Chẳng hạn, hãy xét StringBuffer trong ví dụ. Đáng lẽ ta có thể truyền nó qua lại như một đối số thay vì biến nó thành biến thực thể, nhưng khi ấy người đọc sẽ phải diễn giải nó mỗi lần nhìn thấy. Khi bạn đọc câu chuyện mà module kể, includeSetupPage() dễ hiểu hơn includeSetupPageInto(newPageContent). Đối số nằm ở mức trừu tượng khác với tên hàm và buộc bạn phải biết một chi tiết (nói cách khác, StringBuffer) không quan trọng lắm ở điểm đó.

Đối số còn khó hơn nữa dưới góc nhìn kiểm thử. Thử tưởng tượng nỗi vất vả khi phải viết mọi ca kiểm thử để bảo đảm mọi tổ hợp đối số khác nhau đều chạy đúng. Không có đối số thì chuyện này tầm thường. Có một đối số thì không khó lắm. Với hai đối số, vấn đề bắt đầu thử thách hơn một chút. Với hơn hai đối số, kiểm thử mọi tổ hợp giá trị phù hợp có thể là một cơn ác mộng.

Đối số đầu ra khó hiểu hơn đối số đầu vào. Khi đọc một hàm, ta quen với ý tưởng thông tin đi vào hàm qua đối số và đi ra qua giá trị trả về. Ta thường không chờ đợi thông tin đi ra qua đối số. Vì thế đối số đầu ra hay khiến ta phải nhìn lại lần hai.

Một đối số đầu vào là lựa chọn tốt nhất ngay sau không có đối số nào. SetupTeardown-Includer.render(pageData) khá dễ hiểu. Rõ ràng ta sắp kết xuất dữ liệu trong đối tượng pageData.

Các dạng một đối số phổ biến

Có hai lý do rất phổ biến để truyền một đối số vào hàm. Có thể bạn đang hỏi một câu về đối số đó, như trong `boolean fileExists("MyFile")`. Hoặc bạn đang thao tác trên đối số đó, biến nó thành thứ khác rồi trả về. Chẳng hạn, `InputStream fileOpen("MyFile")` biến một chuỗi tên tệp thành giá trị trả về InputStream. Hai cách dùng này là điều người đọc trông đợi khi thấy một hàm. Bạn nên chọn những cái tên làm nổi bật sự phân biệt ấy, và luôn dùng hai dạng này trong một bối cảnh nhất quán. (Xem phần Tách lệnh khỏi truy vấn bên dưới.)

Một dạng ít phổ biến hơn nhưng vẫn rất hữu ích của hàm một đối số là sự kiện. Ở dạng này có một đối số đầu vào nhưng không có đối số đầu ra. Toàn bộ chương trình được hiểu là hãy diễn giải lời gọi hàm như một sự kiện và dùng đối số để thay đổi trạng thái của hệ thống, chẳng hạn `void passwordAttemptFailedNtimes(int attempts)`. Hãy dùng dạng này cẩn thận. Phải để người đọc thấy rõ đây là một sự kiện. Chọn tên và bối cảnh cho cẩn thận.

Hãy cố tránh mọi hàm một đối số không theo các dạng ấy, chẳng hạn `void includeSetupPageInto(StringBuffer pageText)`. Dùng đối số đầu ra thay cho giá trị trả về trong một phép biến đổi là chuyện gây bối rối. Nếu hàm sắp biến đổi đối số đầu vào của nó, phép biến đổi nên xuất hiện dưới dạng giá trị trả về. Thật vậy, `StringBuffer transform(StringBuffer in)` tốt hơn `void transform(StringBuffer out)`, kể cả khi phần hiện thực của trường hợp đầu chỉ đơn giản trả về đối số đầu vào. Ít nhất nó vẫn tuân theo dạng của một phép biến đổi.

Đối số cờ

Đối số cờ thật xấu xí. Truyền một boolean vào hàm là một thói quen thật sự tệ hại. Nó lập tức làm phức tạp chữ ký của phương thức, và lớn tiếng tuyên bố rằng hàm này làm nhiều hơn một việc. Nó làm một việc nếu cờ là true và làm việc khác nếu cờ là false!

Trong Đoạn mã 3-7, chúng tôi không có lựa chọn nào khác vì những nơi gọi đã truyền cờ ấy vào từ trước, và tôi muốn giới hạn phạm vi tái cấu trúc trong hàm và những thứ bên dưới nó. Dẫu vậy, lời gọi render(true) vẫn cứ gây bối rối cho một người đọc tội nghiệp. Rà chuột lên lời gọi và thấy render(boolean isSuite) cũng đỡ được chút ít, nhưng không đáng kể. Đáng lẽ chúng tôi nên tách hàm thành hai: renderForSuite() và renderForSingleTest().

Hàm hai đối số

Hàm hai đối số khó hiểu hơn hàm một đối số. Chẳng hạn, writeField(name) dễ hiểu hơn `writeField(outputStream, name)`[10]. Dù ý nghĩa của cả hai đều rõ, cái đầu trôi qua mắt và dễ dàng đọng lại ý nghĩa. Cái thứ hai đòi một khoảng dừng ngắn cho tới khi ta học cách phớt lờ tham số đầu tiên. Và dĩ nhiên, điều đó cuối cùng dẫn đến rắc rối, vì ta không bao giờ nên phớt lờ bất kỳ phần nào của mã. Những phần ta phớt lờ chính là nơi lũ bug trú

10. Tôi vừa tái cấu trúc xong một module dùng dạng hai đối số. Tôi đã biến được outputStream thành trường của lớp và chuyển mọi lời gọi writeField sang dạng một đối số. Kết quả sạch hơn nhiều.

ngụ.

Có những lúc, dĩ nhiên, hai đối số là hợp lý. Chẳng hạn, `Point p = new Point(0,0);` hoàn toàn hợp lý. Điểm Descartes vốn dĩ nhận hai đối số. Thật vậy, ta sẽ rất ngạc nhiên nếu thấy `new Point(0)`. Tuy nhiên, hai đối số trong trường hợp này là các thành phần có thứ tự của một giá trị duy nhất! Trong khi đó, outputStream và name không có sự gắn kết tự nhiên, cũng chẳng có thứ tự tự nhiên.

Ngay cả những hàm hai đối số hiển nhiên như `assertEquals(expected, actual)` cũng có vấn đề. Bao nhiêu lần bạn đã đặt actual vào chỗ đáng lẽ là expected? Hai đối số không có thứ tự tự nhiên. Thứ tự expected, actual là một quy ước cần luyện tập mới nhớ.

Hàm hai đối số không phải thứ xấu xa, và chắc chắn bạn sẽ phải viết chúng. Nhưng bạn nên biết rằng chúng phải trả giá, và nên tận dụng mọi cơ chế sẵn có để chuyển chúng thành hàm một đối số. Chẳng hạn, bạn có thể đưa phương thức writeField vào outputStream như một thành viên, để có thể viết outputStream.writeField(name). Hoặc biến outputStream thành biến thành viên của lớp hiện tại để khỏi phải truyền nó. Hoặc tách ra một lớp mới như FieldWriter, nhận outputStream trong hàm dựng và có một phương thức write.

Hàm ba đối số

Hàm nhận ba đối số khó hiểu hơn hẳn hàm hai đối số. Các vấn đề về thứ tự, khoảng dừng và sự phớt lờ tăng lên hơn gấp đôi. Tôi khuyên bạn nghĩ thật kỹ trước khi tạo một hàm ba đối số.

Chẳng hạn, hãy xét bản nạp chồng phổ biến của assertEquals nhận ba đối số: `assertEquals(message, expected, actual)`. Bao nhiêu lần bạn đọc message rồi tưởng đó là expected? Tôi đã vấp và dừng lại vì cái bộ ba ấy không biết bao nhiêu lần. Thật vậy, mỗi lần thấy nó, tôi đều phải nhìn lại lần hai, rồi sau đó học cách phớt lờ message.

Mặt khác, đây là một bộ ba không quá hiểm: `assertEquals(1.0, amount, .001)`. Dù vẫn phải nhìn lại lần hai, lần nhìn lại này đáng giá. Luôn tốt khi được nhắc rằng sự bằng nhau của các giá trị số thực dấu phẩy động là chuyện tương đối.

Đối tượng đối số

Khi một hàm có vẻ cần nhiều hơn hai hoặc ba đối số, rất có thể một số đối số ấy nên được gói vào một lớp riêng. Chẳng hạn, hãy xét sự khác biệt giữa hai khai báo sau:

```
Circle makeCircle(double x, double y, double radius);
Circle makeCircle(Point center, double radius);
```

Giảm số đối số bằng cách tạo đối tượng từ chúng có vẻ như ăn gian, nhưng không phải vậy. Khi những nhóm biến được truyền cùng nhau, theo cách x và y được truyền trong ví dụ trên, chúng rất có thể là một phần của một khái niệm xứng đáng có tên riêng.

Danh sách đối số

Đôi khi ta muốn truyền một số lượng đối số thay đổi vào hàm. Chẳng hạn, hãy xét phương thức String.format:

```
String.format("%s worked %.2f hours.", name, hours);
```

Nếu mọi đối số thay đổi đều được đối xử như nhau, như trong ví dụ trên, thì chúng tương đương với một đối số duy nhất kiểu List. Lập luận như vậy thì String.format thực ra là hàm hai đối số. Thật vậy, khai báo của String.format như dưới đây rõ ràng là hai đối số.

```
public String format(String format, Object... args)
```

Vậy mọi quy tắc cũ vẫn áp dụng. Hàm nhận đối số thay đổi có thể là một đối số, hai đối số, thậm chí ba đối số. Nhưng cho chúng nhiều đối số hơn thế sẽ là sai lầm.

```
void monad(Integer... args);
void dyad(String name, Integer... args);
void triad(String name, int count, Integer... args);
```

Động từ và từ khóa

Chọn tên hay cho hàm có thể đi được một quãng dài trên đường giải thích ý định của hàm cùng thứ tự và ý định của các đối số. Với hàm một đối số, hàm và đối số nên tạo thành một cặp động từ/danh từ thật đẹp. Chẳng hạn, write(name) gợi ra rất nhiều điều. Thứ “name” này, dù nó là gì, đang được “write”. Một cái tên còn hay hơn có lẽ là writeField(name), cho ta biết thứ “name” là một “field”.

Cái sau là ví dụ về dạng từ khóa của tên hàm. Dùng dạng này, ta mã hóa tên của các đối số vào tên hàm. Chẳng hạn, assertEquals có lẽ nên được viết là assertExpectedEquals-Actual(expected, actual). Cách này giảm mạnh vấn đề phải nhớ thứ tự của các đối số.

Đừng gây tác dụng phụ

Tác dụng phụ là những lời dối trá. Hàm của bạn hứa làm một việc, nhưng nó còn làm những việc ẩn giấu khác. Đôi khi nó tạo ra những thay đổi bất ngờ đối với các biến của chính lớp chứa nó. Đôi khi nó tạo ra những thay đổi ấy đối với các tham số được truyền vào hàm hoặc đối với biến toàn cục của hệ thống. Trong cả hai trường hợp, chúng đều là những lời dối trá xảo quyệt và tai hại, thường dẫn đến những mối ghép thời gian và những phụ thuộc thứ tự kỳ quặc.

Chẳng hạn, hãy xét hàm có vẻ vô hại trong Đoạn mã 3-6. Hàm này dùng một thuật toán chuẩn để ghép userName với mật khẩu. Nó trả về true nếu hai thứ khớp nhau và false nếu có gì sai. Nhưng nó còn có một tác dụng phụ. Bạn có nhận ra không?

Đoạn mã 3-6
UserValidator.java

```
public class UserValidator {
  private Cryptographer cryptographer;

  public boolean checkPassword(String userName, String password) {
    User user = UserGateway.findByName(userName);
    if (user != User.NULL) {
      String codedPhrase = user.getPhraseEncodedByPassword();
      String phrase = cryptographer.decrypt(codedPhrase, password);
      if ("Valid Password".equals(phrase)) {
        Session.initialize();
        return true;
      }
    }
    return false;
  }
}
```

Tác dụng phụ ấy là lời gọi Session.initialize(), dĩ nhiên rồi. Hàm checkPassword, qua cái tên của mình, nói rằng nó kiểm tra mật khẩu. Cái tên không hề ngụ ý rằng nó khởi tạo phiên làm việc. Vì thế, một người gọi tin vào điều tên hàm nói sẽ có nguy cơ xóa sạch dữ liệu phiên hiện có khi quyết định kiểm tra tính hợp lệ của người dùng.

Tác dụng phụ này tạo ra một mối ghép thời gian. Nghĩa là checkPassword chỉ có thể được gọi vào những thời điểm nhất định (nói cách khác, khi khởi tạo phiên là an toàn). Nếu nó bị gọi sai thứ tự, dữ liệu phiên có thể bị mất ngoài ý muốn. Những mối ghép thời gian gây bối rối, nhất là khi bị giấu dưới dạng tác dụng phụ. Nếu buộc phải có một mối ghép thời gian, bạn nên làm rõ nó trong tên của hàm. Trong trường hợp này, có lẽ ta đổi tên hàm thành checkPasswordAndInitializeSession, dù làm vậy chắc chắn vi phạm “Làm một việc”.

Đối số đầu ra

Đối số tự nhiên được hiểu là đầu vào của hàm. Nếu bạn đã lập trình hơn vài năm, tôi chắc bạn đã từng phải nhìn lại lần hai trước một đối số hóa ra là đầu ra chứ không phải đầu vào. Chẳng hạn:

```
appendFooter(s);
```

Hàm này nối s vào làm chân trang cho một thứ gì đó ư? Hay nó nối một chân trang nào đó vào s? s là đầu vào hay đầu ra? Không mất nhiều thời gian để nhìn chữ ký hàm và thấy:

```
public void appendFooter(StringBuffer report)
```

Điều này làm rõ vấn đề, nhưng chỉ với cái giá là phải kiểm tra khai báo của hàm. Bất cứ thứ gì buộc bạn phải kiểm tra chữ ký hàm đều tương đương với việc nhìn lại lần hai. Đó là một sự gián đoạn nhận thức và nên tránh.

Vào thời trước lập trình hướng đối tượng, đôi khi người ta buộc phải có đối số đầu ra. Tuy nhiên, phần lớn nhu cầu về đối số đầu ra biến mất trong các ngôn ngữ hướng đối tượng, vì `this` được dùng như một đối số đầu ra. Nói cách khác, sẽ tốt hơn nếu appendFooter được gọi như sau:

```
report.appendFooter();
```

Nhìn chung, nên tránh đối số đầu ra. Nếu hàm của bạn buộc phải thay đổi trạng thái của một thứ gì đó, hãy để nó thay đổi trạng thái của đối tượng sở hữu nó.

Tách lệnh khỏi truy vấn

Hàm nên hoặc làm việc gì đó, hoặc trả lời việc gì đó, chứ không nên làm cả hai. Hoặc hàm của bạn nên thay đổi trạng thái của một đối tượng, hoặc nó nên trả về một thông tin nào đó về đối tượng ấy. Làm cả hai thường dẫn đến bối rối. Chẳng hạn, hãy xét hàm sau:

```
public boolean set(String attribute, String value);
```

Hàm này đặt giá trị cho một thuộc tính có tên và trả về true nếu thành công, false nếu không có thuộc tính ấy. Điều này dẫn đến những câu lệnh kỳ quặc như:

```
if (set("username", "unclebob"))...
```

Hãy tưởng tượng điều này từ góc nhìn người đọc. Nó nghĩa là gì? Nó đang hỏi xem thuộc tính “username” trước đó có được đặt thành “unclebob” không? Hay nó đang hỏi xem thuộc tính “username” có được đặt thành công thành “unclebob” không? Thật khó suy ra ý nghĩa từ lời gọi, vì không rõ từ “set” là động từ hay tính từ.

Tác giả định dùng set như một động từ, nhưng trong bối cảnh câu lệnh if, nó có cảm giác như một tính từ. Vậy nên câu lệnh đọc thành “Nếu thuộc tính username trước đó đã được đặt thành unclebob”, chứ không phải “đặt thuộc tính username thành unclebob và nếu việc đó thành công thì . . .”. Ta có thể thử giải quyết bằng cách đổi tên hàm set thành setAndCheckIfExists, nhưng cách ấy không giúp ích mấy cho khả năng đọc của câu lệnh if. Giải pháp thật sự là tách lệnh khỏi truy vấn để sự mơ hồ không thể xảy ra.

```
if (attributeExists("username")) {
  setAttribute("username", "unclebob");
  ...
}
```

Ưu tiên ngoại lệ hơn mã lỗi

Trả về mã lỗi từ các hàm lệnh là một sự vi phạm tinh vi đối với nguyên tắc tách lệnh khỏi truy vấn. Nó khuyến khích việc dùng lệnh làm biểu thức trong các vị từ của câu lệnh if.

```
if (deletePage(page) == E_OK)
```

Cách này không mắc chứng nhầm lẫn động từ/tính từ nhưng lại dẫn đến những cấu trúc lồng rất sâu. Khi trả về một mã lỗi, bạn tạo ra vấn đề là người gọi buộc phải xử lý lỗi ngay lập tức.

```
if (deletePage(page) == E_OK) {
  if (registry.deleteReference(page.name) == E_OK) {
     if (configKeys.deleteKey(page.name.makeKey()) == E_OK){
       logger.log("page deleted");
     } else {
       logger.log("configKey not deleted");
     }
  } else {
     logger.log("deleteReference from registry failed");
  }
} else {
  logger.log("delete failed");
  return E_ERROR;
}
```

Mặt khác, nếu dùng ngoại lệ thay cho mã lỗi trả về, thì phần mã xử lý lỗi có thể được tách khỏi phần mã của luồng thuận lợi và có thể được đơn giản hóa:

```
try {
  deletePage(page);
  registry.deleteReference(page.name);
  configKeys.deleteKey(page.name.makeKey());
}
catch (Exception e) {
  logger.log(e.getMessage());
}
```

Tách khối try/catch

Bản thân các khối try/catch đã xấu xí. Chúng làm rối cấu trúc của mã và trộn việc xử lý lỗi với việc xử lý bình thường. Vì thế, tốt hơn là tách phần thân của khối try và khối catch ra thành những hàm riêng.

```
public void delete(Page page) {
  try {
    deletePageAndAllReferences(page);
  }
  catch (Exception e) {
    logError(e);
  }
}

private void deletePageAndAllReferences(Page page) throws Exception {
  deletePage(page);
  registry.deleteReference(page.name);
  configKeys.deleteKey(page.name.makeKey());
```

```
  }
  private void logError(Exception e) {
    logger.log(e.getMessage());
  }
```

Ở đoạn trên, hàm delete chỉ toàn là chuyện xử lý lỗi. Nó dễ hiểu và dễ bỏ qua. Hàm deletePageAndAllReferences chỉ toàn là chuyện xóa hoàn toàn một trang. Phần xử lý lỗi có thể bỏ qua. Điều này tạo ra một sự tách bạch đẹp đẽ, khiến mã dễ hiểu và dễ sửa hơn.

Xử lý lỗi là một việc

Hàm nên làm một việc. Xử lý lỗi là một việc. Vì thế, một hàm xử lý lỗi không nên làm gì khác. Điều này ngụ ý (như trong ví dụ trên) rằng nếu từ khóa try xuất hiện trong một hàm, nó phải là từ đầu tiên trong hàm và không được có gì sau các khối catch/finally.

Error.java, nam châm hút phụ thuộc

Trả về mã lỗi thường ngụ ý có một lớp hay enum nào đó định nghĩa mọi mã lỗi.

```
  public enum Error {
    OK,
    INVALID,
    NO_SUCH,
    LOCKED,
    OUT_OF_RESOURCES,
    WAITING_FOR_EVENT;
  }
```

Những lớp như thế là một nam châm hút phụ thuộc; rất nhiều lớp khác buộc phải import và dùng chúng. Vì vậy, khi enum Error thay đổi, tất cả những lớp kia đều phải biên dịch lại và triển khai lại[11]. Điều này tạo áp lực tiêu cực lên lớp Error. Lập trình viên không muốn thêm lỗi mới vì khi ấy họ phải build lại và triển khai lại mọi thứ. Thế là họ tái sử dụng những mã lỗi cũ thay vì thêm mã mới.

Khi dùng ngoại lệ thay cho mã lỗi, những ngoại lệ mới là lớp dẫn xuất của lớp ngoại lệ. Có thể thêm chúng mà không buộc phải biên dịch lại hay triển khai lại bất cứ thứ gì[12].

---

11. Những người tưởng có thể thoát khỏi việc biên dịch lại và triển khai lại đều đã bị tìm ra và xử lý.

12. Đây là một ví dụ của Nguyên lý Mở/Đóng (OCP) [PPP02].



Đừng lặp lại chính mình

Hãy nhìn lại Đoạn mã 3-1 thật kỹ[13], bạn sẽ thấy có một thuật toán bị lặp lại bốn lần, mỗi lần ứng với một trong các trường hợp SetUp, SuiteSetUp, TearDown và SuiteTearDown. Không dễ phát hiện sự trùng lặp này vì bốn thực thể ấy chen lẫn với mã khác và không được lặp lại một cách đồng nhất. Dẫu vậy, trùng lặp vẫn là vấn đề vì nó làm mã phình ra và sẽ đòi hỏi phải sửa bốn chỗ nếu thuật toán ấy có ngày phải thay đổi. Đó cũng là bốn cơ hội để mắc lỗi bỏ sót.

Sự trùng lặp này đã được khắc phục bằng phương thức include trong Đoạn mã 3-7. Hãy đọc lại đoạn mã ấy và để ý khả năng đọc của cả module được nâng lên thế nào nhờ giảm bớt sự trùng lặp.

Trùng lặp có thể là gốc rễ của mọi điều xấu trong phần mềm. Nhiều nguyên lý và thực hành đã ra đời nhằm mục đích kiểm soát hoặc loại bỏ nó. Chẳng hạn, hãy xét việc mọi dạng chuẩn hóa cơ sở dữ liệu của Codd đều nhằm loại bỏ trùng lặp trong dữ liệu. Cũng hãy xét việc lập trình hướng đối tượng nhằm dồn mã vào các lớp cơ sở, những thứ nếu không thì sẽ dư thừa. Lập trình có cấu trúc, Lập trình Hướng khía cạnh, Lập trình Hướng thành phần, tất cả đều, ở một phần nào đó, là những chiến lược loại bỏ trùng lặp. Có vẻ như kể từ khi phát minh ra subroutine, những đổi mới trong phát triển phần mềm đã là một nỗ lực không ngừng nhằm loại bỏ trùng lặp khỏi mã nguồn của chúng ta.

Lập trình có cấu trúc

Một số lập trình viên theo các quy tắc lập trình có cấu trúc[14] của Edsger Dijkstra. Dijkstra nói rằng mọi hàm, và mọi khối trong một hàm, nên có một điểm vào và một điểm ra. Theo những quy tắc này, trong một hàm chỉ nên có một câu lệnh return, không có câu lệnh break hay continue trong vòng lặp, và tuyệt đối không bao giờ có câu lệnh goto.

Dù chúng tôi đồng cảm với mục tiêu và kỷ luật của lập trình có cấu trúc, những quy

13. Nguyên lý DRY. [PRAG].

14. [SP72].

tắc ấy chẳng mang lại bao nhiêu lợi ích khi hàm rất nhỏ. Chỉ trong những hàm lớn hơn, các quy tắc như vậy mới mang lại lợi ích đáng kể.

Vậy nên nếu bạn giữ hàm của mình nhỏ, thì việc thỉnh thoảng có nhiều câu lệnh return, break hay continue không gây hại, và đôi khi còn có thể biểu cảm hơn quy tắc một điểm vào, một điểm ra. Ngược lại, goto chỉ có ý nghĩa trong những hàm lớn, nên cần tránh.

Làm thế nào để viết được những hàm như thế này?

Viết phần mềm cũng giống như bất kỳ kiểu viết nào khác. Khi viết một bài báo hay một bài viết, bạn ghi ý tưởng ra trước, rồi nhào nặn nó cho tới khi đọc xuôi. Bản nháp đầu có thể vụng về và lộn xộn, nên bạn trau chuốt từng chữ, sắp xếp lại, tinh chỉnh cho tới khi nó đọc đúng như bạn muốn.

Khi tôi viết hàm, chúng thường ra đời dài và phức tạp. Chúng có nhiều thụt lề và vòng lặp lồng nhau. Chúng có những danh sách đối số dài. Tên gọi thì tùy tiện, và mã thì trùng lặp. Nhưng tôi cũng có một bộ kiểm thử đơn vị bao phủ từng dòng mã vụng về ấy.

Rồi tôi nhào nặn và tinh chỉnh đoạn mã đó, tách hàm ra, đổi tên, loại bỏ trùng lặp. Tôi thu nhỏ các phương thức và sắp xếp lại chúng. Có khi tôi tách hẳn những lớp ra, và trong suốt quá trình ấy vẫn giữ cho các bài kiểm thử chạy qua.

Cuối cùng, tôi có được những hàm tuân theo các quy tắc tôi đã trình bày trong chương này. Tôi không viết chúng theo cách ấy ngay từ đầu. Tôi không nghĩ có ai làm được.

Kết luận

Mọi hệ thống đều được dựng nên từ một ngôn ngữ chuyên miền (domain-specific language) do chính các lập trình viên thiết kế để mô tả hệ thống ấy. Hàm là động từ của ngôn ngữ đó, còn lớp là danh từ. Đây không phải sự quay lại với quan niệm cũ kỹ ghê sợ rằng những danh từ và động từ trong một tài liệu yêu cầu chính là phỏng đoán đầu tiên về các lớp và hàm của một hệ thống. Đúng hơn, đây là một chân lý cổ xưa hơn nhiều. Nghệ thuật lập trình là, và luôn luôn là, nghệ thuật thiết kế ngôn ngữ.

Những lập trình viên bậc thầy nghĩ về hệ thống như những câu chuyện cần được kể, chứ không phải những chương trình cần được viết. Họ dùng các phương tiện của ngôn ngữ lập trình mình chọn để dựng nên một ngôn ngữ phong phú và biểu cảm hơn nhiều, có thể dùng để kể câu chuyện ấy. Một phần của ngôn ngữ chuyên miền đó là hệ thống phân cấp các hàm mô tả mọi hành động diễn ra bên trong hệ thống. Trong một hành vi đệ quy đầy nghệ thuật, những hành động ấy được viết bằng chính ngôn ngữ chuyên miền mà chúng định nghĩa, để kể phần câu chuyện nhỏ bé của riêng chúng.

Chương này nói về cơ chế viết hàm cho tốt. Nếu bạn theo các quy tắc trong đây, hàm của bạn sẽ ngắn, được đặt tên hay và được tổ chức gọn gàng. Nhưng đừng bao giờ quên rằng mục tiêu thật sự của bạn là kể câu chuyện của hệ thống, và những hàm bạn viết cần khớp vào nhau một cách sạch sẽ để tạo thành một ngôn ngữ rõ ràng, chính xác, giúp bạn kể câu chuyện đó.

SetupTeardownIncluder

Đoạn mã 3-7
SetupTeardownIncluder.java

```
package fitnesse.html;

import fitnesse.responders.run.SuiteResponder;
import fitnesse.wiki.*;
public class SetupTeardownIncluder {
  private PageData pageData;
  private boolean isSuite;
  private WikiPage testPage;
  private StringBuffer newPageContent;
  private PageCrawler pageCrawler;

  public static String render(PageData pageData) throws Exception {
    return render(pageData, false);
  }

  public static String render(PageData pageData, boolean isSuite)
    throws Exception {
    return new SetupTeardownIncluder(pageData).render(isSuite);
  }

  private SetupTeardownIncluder(PageData pageData) {
    this.pageData = pageData;
    testPage = pageData.getWikiPage();
    pageCrawler = testPage.getPageCrawler();
    newPageContent = new StringBuffer();
  }
  private String render(boolean isSuite) throws Exception {
    this.isSuite = isSuite;
    if (isTestPage())
      includeSetupAndTeardownPages();
    return pageData.getHtml();
  }

  private boolean isTestPage() throws Exception {
    return pageData.hasAttribute("Test");
  }

  private void includeSetupAndTeardownPages() throws Exception {
    includeSetupPages();
    includePageContent();
    includeTeardownPages();
    updatePageContent();
  }
```

Đoạn mã 3-7 (tiếp theo)
SetupTeardownIncluder.java

```
  private void includeSetupPages() throws Exception {
```

```
    if (isSuite)
      includeSuiteSetupPage();
    includeSetupPage();
  }
  private void includeSuiteSetupPage() throws Exception {
    include(SuiteResponder.SUITE_SETUP_NAME, "-setup");
  }
  private void includeSetupPage() throws Exception {
    include("SetUp", "-setup");
  }

  private void includePageContent() throws Exception {
    newPageContent.append(pageData.getContent());
  }
  private void includeTeardownPages() throws Exception {
    includeTeardownPage();
    if (isSuite)
      includeSuiteTeardownPage();
  }

  private void includeTeardownPage() throws Exception {
    include("TearDown", "-teardown");
  }

  private void includeSuiteTeardownPage() throws Exception {
    include(SuiteResponder.SUITE_TEARDOWN_NAME, "-teardown");
  }

  private void updatePageContent() throws Exception {
    pageData.setContent(newPageContent.toString());
  }
  private void include(String pageName, String arg) throws Exception {
    WikiPage inheritedPage = findInheritedPage(pageName);
    if (inheritedPage != null) {
      String pagePathName = getPathNameForPage(inheritedPage);
      buildIncludeDirective(pagePathName, arg);
    }
  }

  private WikiPage findInheritedPage(String pageName) throws Exception {
    return PageCrawlerImpl.getInheritedPage(pageName, testPage);
  }

  private String getPathNameForPage(WikiPage page) throws Exception {
    WikiPagePath pagePath = pageCrawler.getFullPath(page);
    return PathParser.render(pagePath);
  }

  private void buildIncludeDirective(String pagePathName, String arg) {
    newPageContent
      .append("\n!include ")
```

Đoạn mã 3-7 (tiếp theo)
SetupTeardownIncluder.java

```
      .append(arg)
```

```
            .append(" .")
            .append(pagePathName)
            .append("\n");
        }
    }
```

Tài liệu tham khảo

[KP78]: Kernighan and Plaugher, *The Elements of Programming Style*, 2d. ed., McGraw-Hill, 1978.

[PPP02]: Robert C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2002.

[GOF]: *Design Patterns: Elements of Reusable Object Oriented Software*, Gamma et al., Addison-Wesley, 1996.

[PRAG]: *The Pragmatic Programmer*, Andrew Hunt, Dave Thomas, Addison-Wesley, 2000.

[SP72]: *Structured Programming*, O.-J. Dahl, E. W. Dijkstra, C. A. R. Hoare, Academic Press, London, 1972.

4

Chú thích



"Đừng chú thích mã xấu, hãy viết lại nó."
—Brian W. Kernighan và P. J. Plaugher[1]

Không gì hữu ích bằng một chú thích đặt đúng chỗ. Không gì làm rối một module bằng những chú thích tầm phào và giáo điều. Không gì tai hại bằng một chú thích cũ kỹ, nát bấy, đang lan truyền dối trá và thông tin sai lạc.

Chú thích không giống *Schindler's List*. Chúng không phải là "điều thiện thuần khiết". Thật vậy, trong trường hợp tốt nhất, chú thích chỉ là một thứ xấu nhưng cần thiết. Nếu ngôn ngữ lập trình của chúng ta đủ biểu cảm, hoặc nếu chúng ta đủ tài để vận dụng tinh tế những ngôn ngữ ấy mà diễn đạt ý định của mình, thì chúng ta sẽ không cần chú thích nhiều, có lẽ

1. [KP78], tr. 144.

chẳng cần chút nào.

Cách dùng đúng đắn của chú thích là bù đắp cho thất bại của chúng ta trong việc diễn đạt bản thân bằng mã. Bạn để ý tôi đã dùng từ thất bại. Tôi nói thật đấy. Chú thích luôn là những thất bại. Chúng ta phải có chúng vì không phải lúc nào cũng tìm ra cách diễn đạt bản thân mà không cần đến chúng, nhưng dùng chúng không phải là chuyện đáng ăn mừng.

Vậy nên khi rơi vào tình huống cần viết một chú thích, hãy nghĩ cho kỹ và xem có cách nào xoay ngược tình thế mà diễn đạt ý mình bằng mã không. Mỗi lần diễn đạt bản thân bằng mã, bạn nên tự khen mình. Mỗi lần viết chú thích, bạn nên nhăn mặt và cảm thấy thất bại trong khả năng diễn đạt.

Vì sao tôi lại chê chú thích đến thế? Vì chúng nói dối. Không phải lúc nào cũng vậy, và không phải cố ý, nhưng quá thường xuyên. Chú thích càng cũ, càng xa đoạn mã nó mô tả, thì càng dễ sai hẳn. Lý do đơn giản. Lập trình viên không thể duy trì chúng một cách thực tế.

Mã thay đổi và tiến hóa. Từng khối mã chuyển từ chỗ này sang chỗ khác. Những khối ấy phân nhánh, sinh sôi rồi lại hợp nhất thành những con quái vật lai. Đáng tiếc là chú thích không phải lúc nào cũng đi theo chúng, không thể lúc nào cũng đi theo. Và rất thường xuyên, chú thích bị tách khỏi đoạn mã nó mô tả, trở thành những mẩu ghi chú mồ côi ngày càng kém chính xác. Chẳng hạn, hãy xem chuyện gì đã xảy ra với chú thích này và dòng mã mà nó định mô tả:

```
MockRequest request;
private final String HTTP_DATE_REGEXP =
  "[SMTWF][a-z]{2}\\,\\s[0-9]{2}\\s[JFMASOND][a-z]{2}\\s"+
  "[0-9]{4}\\s[0-9]{2}\\:[0-9]{2}\\:[0-9]{2}\\sGMT";
private Response response;
private FitNesseContext context;
private FileResponder responder;
private Locale saveLocale;
// Ví dụ: "Tue, 02 Apr 2003 22:18:49 GMT"
```

Những biến thực thể khác, có lẽ được thêm vào sau, đã chen vào giữa hằng HTTP_DATE_REGEXP và chú thích giải thích nó.

Có thể lập luận rằng lập trình viên nên đủ kỷ luật để giữ chú thích ở trạng thái được sửa chữa tốt, vẫn liên quan và chính xác. Tôi đồng ý, họ nên như vậy. Nhưng tôi thà rằng năng lượng ấy được dồn vào việc làm cho mã rõ ràng và biểu cảm đến mức không cần đến chú thích ngay từ đầu.

Chú thích không chính xác còn tệ hơn hẳn không có chú thích. Chúng đánh lừa và dẫn sai. Chúng đặt ra những kỳ vọng sẽ không bao giờ được đáp ứng. Chúng đặt ra những quy tắc cũ không cần, hoặc không nên, tuân theo nữa.

Sự thật chỉ có thể tìm thấy ở một nơi: mã. Chỉ mã mới thật sự cho bạn biết nó làm gì. Đó là nguồn thông tin chính xác duy nhất. Vì vậy, dù chú thích đôi khi là cần thiết, chúng ta sẽ dồn năng lượng đáng kể để giảm thiểu chúng.

Chú thích không bù đắp được cho mã tồi

Một trong những động cơ phổ biến để viết chú thích là mã tồi. Chúng ta viết một module và biết nó khó hiểu, lộn xộn. Chúng ta biết nó là một mớ hỗn độn. Thế là tự nhủ: "Ồ, tốt hơn mình nên chú thích chỗ đó!" Không! Tốt hơn bạn nên dọn nó đi!

Mã rõ ràng, biểu cảm với ít chú thích vượt xa mã rối rắm, phức tạp với nhiều chú thích. Thay vì dành thời gian viết những chú thích giải thích mớ hỗn độn mình vừa tạo ra, hãy dành thời gian dọn mớ hỗn độn ấy.

Tự diễn đạt bằng mã

Chắc chắn có những lúc mã là phương tiện giải thích kém cỏi. Đáng tiếc, nhiều lập trình viên lại suy ra rằng mã hiếm khi, nếu có, là phương tiện giải thích tốt. Điều này hoàn toàn sai. Bạn thích thấy cái nào hơn? Cái này:

```
// Kiểm tra xem nhân viên có đủ điều kiện hưởng phúc lợi đầy đủ không
if ((employee.flags & HOURLY_FLAG) &&
    (employee.age > 65))
```

Hay cái này?

```
if (employee.isEligibleForFullBenefits())
```

Chỉ cần vài giây suy nghĩ là có thể giải thích phần lớn ý định của mình bằng mã. Trong nhiều trường hợp, đó đơn giản là chuyện tạo ra một hàm nói đúng điều bạn muốn viết trong chú thích.

Chú thích tốt

Một số chú thích là cần thiết hoặc có ích. Chúng ta sẽ xem vài chú thích mà tôi cho là xứng đáng với số bit chúng chiếm. Tuy vậy, hãy nhớ rằng chú thích thật sự tốt duy nhất là chú thích bạn tìm được cách để không phải viết.

Chú thích pháp lý

Đôi khi chuẩn lập trình của công ty buộc chúng ta phải viết một số chú thích vì lý do pháp lý. Chẳng hạn, các tuyên bố về bản quyền và tác quyền là thứ cần thiết và hợp lý để đặt trong chú thích ở đầu mỗi tệp nguồn.

Đây là ví dụ về phần đầu chú thích chuẩn mà chúng tôi đặt ở đầu mọi tệp nguồn trong FitNesse. Tôi vui mừng nói rằng IDE của chúng tôi tự động thu gọn chú thích này để nó không làm rối mắt.

```
// Bản quyền (C) 2003,2004,2005 thuộc Object Mentor, Inc. Bảo lưu mọi quyền.
// Phát hành theo các điều khoản của Giấy phép Công cộng GNU phiên bản 2 hoặc
    mới hơn.
```

Những chú thích như thế không nên là hợp đồng hay những cuốn sách luật đồ sộ. Khi có thể, hãy tham chiếu đến một giấy phép chuẩn hoặc tài liệu bên ngoài thay vì đưa toàn bộ điều khoản và điều kiện vào chú thích.

Chú thích cung cấp thông tin

Đôi khi việc cung cấp thông tin cơ bản bằng một chú thích là hữu ích. Chẳng hạn, hãy xét chú thích giải thích giá trị trả về của một phương thức trừu tượng:

```
// Trả về một thực thể Responder đang được kiểm thử.
protected abstract Responder responderInstance();
```

Một chú thích như thế đôi khi có ích, nhưng tốt hơn là dùng tên hàm để truyền đạt thông tin khi có thể. Chẳng hạn, trong trường hợp này, có thể làm chú thích thành dư thừa bằng cách đổi tên hàm: responderBeingTested.

Đây là một trường hợp khá hơn:

```
// định dạng khớp kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher = Pattern.compile(
  "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

Trong trường hợp này, chú thích cho ta biết biểu thức chính quy định khớp một thời gian và ngày tháng được định dạng bằng hàm SimpleDateFormat.format với chuỗi định dạng đã chỉ định. Dù vậy, sẽ tốt hơn và rõ hơn nếu đoạn mã này được chuyển vào một lớp chuyên biệt chuyển đổi định dạng ngày và giờ. Khi đó chú thích có lẽ sẽ thừa.

Giải thích ý định

Đôi khi một chú thích vượt ra ngoài thông tin hữu ích về cách hiện thực và cho biết ý định đằng sau một quyết định. Trong trường hợp sau, ta thấy một quyết định thú vị được ghi lại bằng chú thích. Khi so sánh hai đối tượng, tác giả quyết định rằng anh ta muốn xếp các đối tượng thuộc lớp của mình cao hơn đối tượng thuộc bất kỳ lớp nào khác.

```
public int compareTo(Object o)
{
  if(o instanceof WikiPagePath)
  {
     WikiPagePath p = (WikiPagePath) o;
     String compressedName = StringUtil.join(names, "");
     String compressedArgumentName = StringUtil.join(p.names, "");
     return compressedName.compareTo(compressedArgumentName);
  }
  return 1; // chúng ta lớn hơn vì chúng ta đúng kiểu.
}
```

Đây là một ví dụ còn tốt hơn. Có thể bạn không đồng ý với cách lập trình viên giải quyết vấn đề, nhưng ít nhất bạn biết anh ta đang cố làm gì.

```
public void testConcurrentAddWidgets() throws Exception {
  WidgetBuilder widgetBuilder =
     new WidgetBuilder(new Class[]{BoldWidget.class});
  String text = "'''bold text'''";
  ParentWidget parent =
     new BoldWidget(new MockWidgetRoot(), "'''bold text'''");
  AtomicBoolean failFlag = new AtomicBoolean();
  failFlag.set(false);
  //Đây là nỗ lực tốt nhất của chúng ta để tạo ra một điều kiện tranh chấp
  //bằng cách tạo ra số lượng lớn luồng.
  for (int i = 0; i < 25000; i++) {
     WidgetBuilderThread widgetBuilderThread =
        new WidgetBuilderThread(widgetBuilder, text, parent, failFlag);
     Thread thread = new Thread(widgetBuilderThread);
     thread.start();
  }
  assertEquals(false, failFlag.get());
}
```

Làm rõ

Đôi khi chỉ cần dịch nghĩa của một đối số hoặc giá trị trả về khó hiểu thành thứ dễ đọc. Nói chung, tốt hơn là tìm cách làm cho đối số hay giá trị trả về ấy tự nó rõ ràng; nhưng khi nó thuộc thư viện chuẩn, hoặc nằm trong mã bạn không thể sửa, thì một chú thích làm rõ hữu ích có thể có giá trị.

```
public void testCompareTo() throws Exception
{
  WikiPagePath a = PathParser.parse("PageA");
  WikiPagePath ab = PathParser.parse("PageA.PageB");
  WikiPagePath b = PathParser.parse("PageB");
  WikiPagePath aa = PathParser.parse("PageA.PageA");
  WikiPagePath bb = PathParser.parse("PageB.PageB");
  WikiPagePath ba = PathParser.parse("PageB.PageA");
    assertTrue(a.compareTo(a) == 0);    // a == a
    assertTrue(a.compareTo(b) != 0);    // a != b
    assertTrue(ab.compareTo(ab) == 0); // ab == ab
    assertTrue(a.compareTo(b) == -1); // a < b
    assertTrue(aa.compareTo(ab) == -1); // aa < ab
    assertTrue(ba.compareTo(bb) == -1); // ba < bb
    assertTrue(b.compareTo(a) == 1);    // b > a
    assertTrue(ab.compareTo(aa) == 1); // ab > aa
    assertTrue(bb.compareTo(ba) == 1); // bb > ba
}
```

Tất nhiên, có một rủi ro đáng kể là chú thích làm rõ lại sai. Hãy xem lại ví dụ trước và thấy việc kiểm chứng chúng đúng là khó đến mức nào. Điều này giải thích cả vì sao cần làm rõ lẫn vì sao nó rủi ro. Vậy nên trước khi viết những chú thích như thế, hãy cẩn thận xem có cách nào tốt hơn không, rồi cẩn thận hơn nữa để chúng chính xác.

Cảnh báo hậu quả



Đôi khi hữu ích khi cảnh báo các lập trình viên khác về những hậu quả nhất định. Chẳng hạn, đây là chú thích giải thích vì sao một ca kiểm thử cụ thể bị tắt:

```
// Đừng chạy trừ khi bạn
// có thời gian để giết.
public void _testWithReallyBigFile()
{
  writeLinesToFile(10000000);

    response.setBody(testFile);
    response.readyToSend(this);
    String responseString = output.toString();
    assertSubString("Content-Length: 1000000000", responseString);
    assertTrue(bytesSent > 10000000000);
}
```

Tất nhiên, ngày nay chúng ta sẽ tắt ca kiểm thử bằng annotation @Ignore kèm một chuỗi giải thích thích hợp, chẳng hạn `@Ignore("Takes too long to run")`. Nhưng quay lại thời trước JUnit 4, đặt dấu gạch dưới trước tên phương thức là một quy ước phổ biến. Chú thích này, dù cợt nhả, truyền đạt ý khá tốt.

Đây là một ví dụ khác, sâu sắc hơn:

```
public static SimpleDateFormat makeStandardHttpDateFormat()
{
  //SimpleDateFormat không an toàn với luồng,
  //nên chúng ta cần tạo từng thực thể một cách độc lập.
  SimpleDateFormat df = new SimpleDateFormat("EEE, dd MMM yyyy HH:mm:ss z");
  df.setTimeZone(TimeZone.getTimeZone("GMT"));
  return df;
}
```

Có thể bạn phàn nàn rằng có những cách tốt hơn để giải quyết vấn đề này. Tôi có thể đồng ý với bạn. Nhưng chú thích như đưa ra ở đây là hoàn toàn hợp lý. Nó sẽ ngăn một lập trình

viên quá hăng hái dùng khởi tạo tĩnh nhân danh hiệu quả.

Chú thích TODO

Đôi khi để lại ghi chú "Việc cần làm" dưới dạng chú thích `//TODO` là hợp lý. Trong trường hợp sau, chú thích TODO giải thích vì sao hàm có một cách hiện thực thoái hóa và tương lai của hàm ấy nên ra sao.

```
//TODO-MdM những thứ này không cần thiết
// Chúng tôi kỳ vọng thứ này sẽ biến mất khi chúng tôi làm mô hình checkout
protected VersionInfo makeVersion() throws Exception
{
  return null;
}
```

TODO là những việc mà lập trình viên nghĩ nên làm, nhưng vì lý do nào đó chưa thể làm ngay. Nó có thể là lời nhắc xóa một tính năng đã bị đánh dấu lỗi thời, hoặc lời cầu xin ai đó xem giúp một vấn đề. Nó có thể là yêu cầu ai đó nghĩ ra cái tên hay hơn, hoặc lời nhắc thực hiện một thay đổi phụ thuộc vào một sự kiện đã lên kế hoạch. Dù TODO có thể là gì đi nữa, nó không phải cái cớ để mã tồi nằm lại trong hệ thống.

Ngày nay, hầu hết IDE tốt cung cấp thao tác và tính năng đặc biệt để tìm mọi chú thích TODO, nên chúng khó có thể bị thất lạc. Dù vậy, bạn không muốn mã của mình bị rải rác đầy TODO. Vì thế hãy quét qua chúng thường xuyên và loại bỏ những cái có thể loại bỏ.

Khuếch đại

Một chú thích có thể được dùng để khuếch đại tầm quan trọng của một thứ mà nếu không thì có vẻ chẳng đáng kể.

```
String listItemContent = match.group(3).trim();
// việc trim thật sự quan trọng. Nó loại bỏ những dấu
// cách ở đầu có thể khiến mục này bị nhận nhầm
// là một danh sách khác.
new ListItemWidget(this, listItemContent, this.level + 1);
return buildList(text.substring(match.end()));
```

Javadoc trong API công khai

Không gì hữu ích và thỏa mãn bằng một API công khai được mô tả tốt. Javadoc cho thư viện Java chuẩn là một ví dụ điển hình. Khó có thể viết chương trình Java mà không có chúng.

Nếu bạn đang viết một API công khai thì chắc chắn bạn nên viết javadoc tử tế cho nó. Nhưng hãy nhớ phần còn lại của lời khuyên trong chương này. Javadoc có thể gây hiểu nhầm, phi cục bộ và thiếu trung thực y như bất kỳ loại chú thích nào khác.

Chú thích xấu

Hầu hết chú thích đều rơi vào loại này. Thường chúng là nạng hoặc cái cớ cho mã tồi, hay lời biện hộ cho những quyết định thiếu thỏa đáng, chẳng khác gì lập trình viên tự nói chuyện với chính mình.

Nói lảm nhảm

Đặt đại một chú thích chỉ vì bạn thấy nên làm thế hoặc vì quy trình đòi hỏi là một mánh tạm bợ. Nếu đã quyết định viết chú thích, hãy dành đủ thời gian để chắc rằng đó là chú thích tốt nhất bạn có thể viết.

Đây là ví dụ về một trường hợp tôi tìm thấy trong FitNesse, nơi một chú thích đáng lẽ có thể thật sự hữu ích. Nhưng tác giả đang vội hoặc chỉ không để ý lắm. Sự lảm nhảm của anh ta để lại một điều bí ẩn:

```
public void loadProperties()
{
  try
  {
    String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
    FileInputStream propertiesStream = new FileInputStream(propertiesPath);
    loadedProperties.load(propertiesStream);
  }
  catch(IOException e)
  {
     // Không có tệp properties nghĩa là mọi giá trị mặc định đã được nạp
  }
}
```

Chú thích trong khối catch ấy nghĩa là gì? Rõ ràng nó có ý nghĩa với tác giả, nhưng ý nghĩa không truyền đạt được rõ lắm. Có vẻ như nếu ta gặp IOException thì nghĩa là không có tệp properties; và trong trường hợp đó mọi giá trị mặc định đã được nạp. Nhưng ai nạp mọi giá trị mặc định? Chúng được nạp trước lời gọi loadProperties.load ư? Hay loadProperties. load bắt ngoại lệ, nạp các giá trị mặc định, rồi chuyển ngoại lệ cho chúng ta bỏ qua? Hay loadProperties.load nạp mọi giá trị mặc định trước khi thử nạp tệp? Tác giả đang cố tự an ủi về việc mình để khối catch trống ư? Hay, và đây mới là khả năng đáng sợ, tác giả đang tự nhủ phải quay lại đây sau và viết đoạn mã nạp các giá trị mặc định?

Cách duy nhất của chúng ta là xem mã ở những phần khác của hệ thống để tìm ra chuyện gì đang diễn ra. Bất kỳ chú thích nào buộc bạn phải tìm sang module khác để hiểu nghĩa của chính nó là đã thất bại trong việc truyền đạt, và không đáng với số bit nó chiếm.

Chú thích dư thừa

Đoạn mã 4-1 cho thấy một hàm đơn giản với chú thích đầu hàm hoàn toàn dư thừa. Đọc chú thích có lẽ còn lâu hơn đọc chính đoạn mã.

Đoạn mã 4-1
waitForClose

```
// Phương thức tiện ích trả về khi this.closed là true. Ném ngoại lệ
// nếu hết thời gian chờ.
public synchronized void waitForClose(final long timeoutMillis)
throws Exception
{
   if(!closed)
   {
     wait(timeoutMillis);
     if(!closed)
        throw new Exception("MockResponseSender could not be closed");
   }
}
```

Chú thích này phục vụ mục đích gì? Chắc chắn nó không cung cấp nhiều thông tin hơn đoạn mã. Nó không biện minh cho đoạn mã, cũng không cho biết ý định hay lý do. Nó không dễ đọc hơn đoạn mã. Thật vậy, nó kém chính xác hơn đoạn mã và dụ người đọc chấp nhận sự thiếu chính xác ấy thay cho hiểu biết thật sự. Nó giống như một anh bán xe cũ khéo mồm trấn an bạn rằng bạn không cần nhìn dưới nắp ca-pô.

Giờ hãy xét cả một loạt javadoc vô dụng và dư thừa trong Đoạn mã 4-2, lấy từ Tomcat. Những chú thích này chỉ làm rối và che khuất đoạn mã. Chúng không phục vụ mục đích ghi chép tài liệu nào cả. Tệ hơn nữa, tôi chỉ cho bạn thấy vài cái đầu tiên. Còn nhiều cái nữa trong module này.

Đoạn mã 4-2
ContainerBase.java (Tomcat)

```
public abstract class ContainerBase
  implements Container, Lifecycle, Pipeline,
  MBeanRegistration, Serializable {
   /**
    * Độ trễ xử lý của thành phần này.
    */
   protected int backgroundProcessorDelay = -1;

   /**
    * Hỗ trợ sự kiện vòng đời của thành phần này.
    */
   protected LifecycleSupport lifecycle =
     new LifecycleSupport(this);

   /**
    * Các listener sự kiện container của Container này.
    */
   protected ArrayList listeners = new ArrayList();

   /**
    * Cách hiện thực Loader gắn với
    * Container này.
    */
```

```
    protected Loader loader = null;

    /**
     * Cách hiện thực Logger gắn với
     * Container này.
     */
    protected Log logger = null;

    /**
     * Tên logger liên quan.
     */
    protected String logName = null;
```

Đoạn mã 4-2 (tiếp theo)
ContainerBase.java (Tomcat)

```
/**
 * Cách hiện thực Manager gắn với
 * Container này.
 */
protected Manager manager = null;

/**
 * Cluster gắn với Container này.
 */
protected Cluster cluster = null;

/**
 * Tên dễ đọc của Container này.
 */
protected String name = null;

/**
 * Container cha mà Container này là con.
 */
protected Container parent = null;

/**
 * Class loader cha được cấu hình khi chúng ta cài một
 * Loader.
 */
protected ClassLoader parentClassLoader = null;

/**
 * Đối tượng Pipeline gắn với
 * Container này.
 */
protected Pipeline pipeline = new StandardPipeline(this);

/**
 * Realm gắn với Container này.
 */
protected Realm realm = null;


/**
```

```
     * Đối tượng DirContext chứa tài nguyên gắn với
     * Container này.
     */
    protected DirContext resources = null;
```

Chú thích gây hiểu nhầm

Đôi khi, với tất cả thiện ý, một lập trình viên viết trong chú thích của mình một câu không đủ chính xác để đúng. Hãy nghĩ thêm một chút về chú thích vừa dư thừa vừa tinh vi gây hiểu nhầm mà ta đã thấy ở Đoạn mã 4-1.

Bạn có phát hiện ra chú thích ấy gây hiểu nhầm ở đâu không? Phương thức không trả về khi this.closed trở thành true. Nó trả về nếu this.closed đang là true; nếu không, nó chờ một khoảng thời gian mù và sau đó ném ngoại lệ nếu this.closed vẫn chưa true.

Mẩu thông tin sai lạc tinh vi này, được bọc trong một chú thích khó đọc hơn cả thân đoạn mã, có thể khiến một lập trình viên khác vô tư gọi hàm này với kỳ vọng nó sẽ trả về ngay khi this.closed trở thành true. Anh lập trình viên tội nghiệp ấy rồi sẽ thấy mình trong một phiên gỡ lỗi, cố tìm ra vì sao mã của mình chạy chậm đến thế.

Chú thích bị bắt buộc

Thật nực cười khi có một quy tắc nói rằng mọi hàm phải có javadoc, hoặc mọi biến phải có chú thích. Những chú thích như thế chỉ làm rối mã, lan truyền dối trá và góp phần gây nhầm lẫn, mất trật tự nói chung.

Chẳng hạn, javadoc bắt buộc cho mọi hàm dẫn đến những thứ quái dị như Đoạn mã 4-3. Mớ rối rắm này chẳng thêm gì và chỉ làm mờ đoạn mã, tạo ra khả năng cho dối trá và dẫn sai.

Đoạn mã 4-3

```
/**
  *
  * @param title Tiêu đề của CD
  * @param author Tác giả của CD
  * @param tracks Số bài hát trên CD
  * @param durationInMinutes Thời lượng của CD tính bằng phút
  */
public void addCD(String title, String author,
                  int tracks, int durationInMinutes) {
    CD cd = new CD();
    cd.title = title;
    cd.author = author;
    cd.tracks = tracks;
    cd.duration = duration;
    cdList.add(cd);
}
```

Chú thích nhật ký

Đôi khi người ta thêm một chú thích vào đầu module mỗi lần sửa nó. Những chú thích này tích tụ thành một loại nhật ký, hay sổ ghi, ghi lại mọi thay đổi từng được thực hiện. Tôi đã từng thấy những module với hàng chục trang ghi chép nhật ký nối dài như thế.

```
     * Thay đổi (từ 11-Oct-2001)
     * -------------------------
     * 11-Oct-2001 : Tổ chức lại lớp và chuyển nó sang package mới
     *               com.jrefinery.date (DG);
     * 05-Nov-2001 : Thêm phương thức getDescription(), và loại bỏ lớp
       NotableDate
     *               (DG);
     * 12-Nov-2001 : IBD yêu cầu phương thức setDescription(), vì lớp
       NotableDate
     *               đã bị bỏ (DG); Sửa getPreviousDayOfWeek(),
     *               getFollowingDayOfWeek() và getNearestDayOfWeek() để khắc
       phục
     *               lỗi (DG);
     * 05-Dec-2001 : Sửa lỗi trong lớp SpreadsheetDate (DG);
     * 29-May-2002 : Chuyển các hằng tháng vào một giao diện riêng
     *               (MonthConstants) (DG);
     * 27-Aug-2002 : Sửa lỗi trong phương thức addMonths(), cảm ơn N???levka
       Petr (DG);
     * 03-Oct-2002 : Sửa các lỗi do Checkstyle báo (DG);
     * 13-Mar-2003 : Hiện thực Serializable (DG);
     * 29-May-2003 : Sửa lỗi trong phương thức addMonths (DG);
     * 04-Sep-2003 : Hiện thực Comparable. Cập nhật javadoc của isInRange (DG);
     * 05-Jan-2005 : Sửa lỗi trong phương thức addYears() (1096282) (DG);
```

Xưa kia có lý do chính đáng để tạo và duy trì những ghi chép nhật ký này ở đầu mỗi module. Chúng ta chưa có hệ thống quản lý mã nguồn làm việc đó thay mình. Ngày nay, những cuốn nhật ký dài này chỉ là thêm một mớ rối rắm làm mờ module. Chúng nên được xóa hoàn toàn.

Chú thích ồn ào

Đôi khi bạn thấy những chú thích chẳng là gì ngoài tiếng ồn. Chúng nhắc lại điều hiển nhiên và không cung cấp thông tin mới.

```
/**
 * Hàm khởi tạo mặc định.
 */
protected AnnualDateRule() {
}
```

Không, thật sao? Hay thế này:

```
/** Ngày trong tháng. */
    private int dayOfMonth;
```

Và đây là tấm gương mẫu mực của sự dư thừa:

```
/**
 * Trả về ngày trong tháng.
 *
 * @return ngày trong tháng.
 */
public int getDayOfMonth() {
  return dayOfMonth;
}
```

Những chú thích này ồn ào đến mức chúng ta học cách phớt lờ chúng. Khi đọc mã, mắt ta chỉ lướt qua chúng. Cuối cùng chú thích bắt đầu nói dối khi đoạn mã xung quanh thay đổi.

Chú thích đầu tiên trong Đoạn mã 4-4 có vẻ hợp lý.[2] Nó giải thích vì sao khối catch bị bỏ qua. Nhưng chú thích thứ hai là tiếng ồn thuần túy. Có vẻ lập trình viên đã quá bực mình vì phải viết các khối try/catch trong hàm này đến mức cần xả bực.

Đoạn mã 4-4
startSending

```
private void startSending()
{
  try
  {
     doSending();
  }
  catch(SocketException e)
  {
     // bình thường. ai đó đã dừng yêu cầu.
  }
  catch(Exception e)
  {
     try
     {
       response.add(ErrorResponder.makeExceptionString(e));
       response.closeAll();
     }
     catch(Exception e1)
     {
       //Cho tôi xin!
     }
  }
}
```

Thay vì xả bực trong một chú thích vô giá trị và ồn ào, lập trình viên nên nhận ra rằng có thể giải tỏa sự bực bội của mình bằng cách cải thiện cấu trúc mã. Anh ta nên dồn năng lượng vào việc tách khối try/catch cuối cùng thành một hàm riêng, như trong Đoạn mã 4-5.

2. Xu hướng hiện nay là các IDE kiểm tra chính tả trong chú thích sẽ là một liều thuốc an ủi cho những người đọc nhiều mã như chúng ta.

Đoạn mã 4-5
startSending (đã tái cấu trúc)

```
    private void startSending()
    {
      try
      {
         doSending();
      }
```

Đoạn mã 4-5 (tiếp theo)
startSending (đã tái cấu trúc)

```
      catch(SocketException e)
      {
        // bình thường. ai đó đã dừng yêu cầu.
      }
      catch(Exception e)
      {
        addExceptionAndCloseResponse(e);
      }
  }
  private void addExceptionAndCloseResponse(Exception e)
  {
    try
    {
       response.add(ErrorResponder.makeExceptionString(e));
       response.closeAll();
    }
    catch(Exception e1)
    {
    }
  }
```

Hãy thay cám dỗ tạo tiếng ồn bằng quyết tâm làm sạch mã của bạn. Bạn sẽ thấy nó khiến bạn thành một lập trình viên giỏi hơn và vui hơn.

Tiếng ồn đáng sợ

Javadoc cũng có thể ồn ào. Những javadoc sau (từ một thư viện mã nguồn mở nổi tiếng) phục vụ mục đích gì? Trả lời: không gì cả. Chúng chỉ là những chú thích dư thừa, ồn ào, được viết ra từ một mong muốn đặt nhầm chỗ là cung cấp tài liệu.

```
/** Tên. */
private String name;
/** Phiên bản. */
private String version;

/** licenceName. */
private String licenceName;
```

```
/** Phiên bản. */
private String info;
```

Hãy đọc lại những chú thích này kỹ hơn. Bạn có thấy lỗi cắt dán không? Nếu tác giả không để ý khi viết (hoặc dán) chú thích, thì tại sao người đọc lại được kỳ vọng thu được lợi ích từ chúng?

Đừng dùng chú thích khi có thể dùng hàm hay biến

Hãy xét đoạn mã sau:

```
// module trong danh sách toàn cục <mod> có phụ thuộc vào
// subsystem mà chúng ta là một phần không?
if (smodule.getDependSubsystems().contains(subSysMod.getSubSystem()))
```

Có thể viết lại mà không cần chú thích như sau:

```
ArrayList moduleDependees = smodule.getDependSubsystems();
String ourSubSystem = subSysMod.getSubSystem();
if (moduleDependees.contains(ourSubSystem))
```

Tác giả đoạn mã gốc có lẽ đã viết chú thích trước (khó có vậy) rồi viết mã để thỏa mãn chú thích. Tuy nhiên, sau đó tác giả nên tái cấu trúc mã như tôi đã làm, để có thể xóa chú thích.

Điểm đánh dấu vị trí

Đôi khi lập trình viên thích đánh dấu một vị trí cụ thể trong tệp nguồn. Chẳng hạn, gần đây tôi thấy dòng này trong một chương trình đang xem qua:

```
// Hành động ////////////////////////////////
```

Có những lúc hiếm hoi mà việc gom một số hàm lại dưới một tấm băng rôn như thế là hợp lý. Nhưng nói chung chúng là mớ rối rắm nên được loại bỏ, nhất là chuỗi dấu gạch chéo ồn ào ở cuối.

Hãy nghĩ thế này. Một tấm băng rôn gây giật mình và hiển nhiên nếu bạn không thường thấy băng rôn. Vì thế hãy dùng chúng thật tiết kiệm, và chỉ khi lợi ích đáng kể. Nếu dùng băng rôn quá nhiều, chúng sẽ chìm vào tiếng ồn nền và bị phớt lờ.

Chú thích trên ngoặc đóng

Đôi khi lập trình viên đặt những chú thích đặc biệt trên dấu ngoặc đóng, như trong Đoạn mã 4-6. Dù điều này có thể hợp lý với những hàm dài, cấu trúc lồng sâu, nó chỉ làm rối những hàm nhỏ và được đóng gói mà chúng ta ưa thích. Vì vậy nếu bạn thấy mình muốn đánh dấu ngoặc đóng, hãy thử rút ngắn hàm của mình.

Đoạn mã 4-6
wc.java

```
public class wc {
  public static void main(String[] args) {
    BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
    String line;
    int lineCount = 0;
    int charCount = 0;
    int wordCount = 0;
    try {
      while ((line = in.readLine()) != null) {
        lineCount++;
        charCount += line.length();
        String words[] = line.split("\\W");
        wordCount += words.length;
      } //while
      System.out.println("wordCount = " + wordCount);
      System.out.println("lineCount = " + lineCount);
      System.out.println("charCount = " + charCount);
    } // try
    catch (IOException e) {
      System.err.println("Error:" + e.getMessage());
    } //catch
  } //main
}
```

Ghi công và dòng tác giả

```
/* Thêm bởi Rick */
```

Hệ thống quản lý mã nguồn rất giỏi ghi nhớ ai đã thêm cái gì, khi nào. Không cần làm ô nhiễm mã bằng những dòng tác giả nhỏ xíu. Có thể bạn nghĩ những chú thích như thế hữu ích để giúp người khác biết nên trao đổi với ai về đoạn mã. Nhưng thực tế là chúng có xu hướng nán lại hàng năm trời, ngày càng kém chính xác và kém liên quan.

Một lần nữa, hệ thống quản lý mã nguồn là nơi tốt hơn cho loại thông tin này.

Mã bị chú thích hóa

Ít thực hành nào đáng ghét bằng việc chú thích hóa mã. Đừng làm thế!

```
InputStreamResponse response = new InputStreamResponse();
response.setBody(formatter.getResultStream(), formatter.getByteCount());
//    InputStream resultsStream = formatter.getResultStream();
//    StreamReader reader = new StreamReader(resultsStream);
//    response.setContent(reader.read(formatter.getByteCount()));
```

Người khác thấy đoạn mã bị chú thích hóa ấy sẽ không đủ can đảm xóa nó. Họ nghĩ nó nằm đó vì một lý do nào đó và quá quan trọng để xóa. Thế là mã bị chú thích hóa đọng lại như cặn dưới đáy một chai rượu vang tồi.

Hãy xét đoạn này từ apache commons:

```
this.bytePos = writeBytes(pngIdBytes, 0);
//hdrPos = bytePos;
writeHeader();
writeResolution();
//dataPos = bytePos;
if (writeImageData()) {
     writeEnd();
     this.pngBytes = resizeByteArray(this.pngBytes, this.maxPos);
}
else {
     this.pngBytes = null;
}
return this.pngBytes;
```

Vì sao hai dòng mã ấy bị chú thích hóa? Chúng có quan trọng không? Chúng được để lại làm lời nhắc cho một thay đổi sắp tới ư? Hay chúng chỉ là rác mà ai đó chú thích hóa từ nhiều năm trước và đơn giản là không buồn dọn?

Đã có thời, vào những năm sáu mươi, chú thích hóa mã có thể hữu ích. Nhưng chúng ta đã có hệ thống quản lý mã nguồn tốt từ rất lâu rồi. Những hệ thống ấy sẽ nhớ mã giúp chúng ta. Chúng ta không phải chú thích hóa nó nữa. Cứ xóa mã đi. Chúng ta sẽ không mất nó đâu. Hứa đấy.

Chú thích HTML

HTML trong chú thích mã nguồn là một điều ghê tởm, như bạn có thể thấy khi đọc đoạn mã dưới đây. Nó khiến chú thích khó đọc ngay tại nơi duy nhất mà chúng phải dễ đọc: trình soạn thảo/IDE. Nếu chú thích sẽ được một công cụ nào đó (như Javadoc) trích xuất để xuất hiện trên một trang Web, thì công cụ ấy, chứ không phải lập trình viên, nên có trách nhiệm trang trí chú thích bằng HTML thích hợp.

```
/**
 * Tác vụ chạy các kiểm thử fit.
 * Tác vụ này chạy các kiểm thử fitnesse và công bố kết quả.
 * <p/>
 * <pre>
```

```
* Cách dùng:
* &lt;taskdef name=&quot;execute-fitnesse-tests&quot;
*      classname=&quot;fitnesse.ant.ExecuteFitnesseTestsTask&quot;
*      classpathref=&quot;classpath&quot; /&gt;
* HOẶC
* &lt;taskdef classpathref=&quot;classpath&quot;
*              resource=&quot;tasks.properties&quot; /&gt;
* <p/>
* &lt;execute-fitnesse-tests
*      suitepage=&quot;FitNesse.SuiteAcceptanceTests&quot;
*      fitnesseport=&quot;8082&quot;
*      resultsdir=&quot;${results.dir}&quot;
*      resultshtmlpage=&quot;fit-results.html&quot;
*      classpathref=&quot;classpath&quot; /&gt;
* </pre>
*/
```

Thông tin phi cục bộ

Nếu buộc phải viết chú thích, hãy chắc rằng nó mô tả đoạn mã nằm gần nó. Đừng đưa thông tin toàn hệ thống vào ngữ cảnh của một chú thích cục bộ. Chẳng hạn, xét chú thích javadoc dưới đây. Ngoài việc dư thừa kinh khủng, nó còn cung cấp thông tin về cổng mặc định. Vậy mà hàm hoàn toàn không kiểm soát được giá trị mặc định đó. Chú thích không mô tả hàm mà mô tả một phần khác, xa hơn nhiều, của hệ thống. Tất nhiên không có gì bảo đảm chú thích này sẽ được sửa khi đoạn mã chứa giá trị mặc định thay đổi.

```
/**
  * Cổng mà fitnesse sẽ chạy trên đó. Mặc định là <b>8082</b>.
  *
  * @param fitnessePort
  */
public void setFitnessePort(int fitnessePort)
{
    this.fitnessePort = fitnessePort;
}
```

Quá nhiều thông tin

Đừng đưa những thảo luận lịch sử thú vị hay mô tả chi tiết không liên quan vào chú thích. Chú thích dưới đây được trích từ một module kiểm tra rằng một hàm có thể mã hóa và giải mã base64. Ngoài số RFC, người đọc đoạn mã này không cần thông tin uyên bác chứa trong chú thích.

```
/*
      RFC 2045 - Multipurpose Internet Mail Extensions (MIME)
      Part One: Format of Internet Message Bodies
      section 6.8. Base64 Content-Transfer-Encoding
      Quá trình mã hóa biểu diễn các nhóm 24 bit đầu vào thành chuỗi
      đầu ra gồm 4 ký tự được mã hóa. Đi từ trái sang phải, một
      nhóm đầu vào 24 bit được tạo bằng cách ghép 3 nhóm đầu vào 8 bit.
      24 bit này sau đó được xử lý như 4 nhóm 6 bit ghép lại, mỗi
      nhóm được dịch thành một chữ số trong bảng chữ cái base64.
```

```
        Khi mã hóa một dòng bit bằng bảng mã base64, dòng bit
        phải được giả định là có thứ tự bit có nghĩa nhất đứng trước.
        Nghĩa là bit đầu tiên trong dòng sẽ là bit bậc cao trong
        byte 8 bit đầu tiên, và bit thứ tám sẽ là bit bậc thấp trong
        byte 8 bit đầu tiên, cứ thế tiếp tục.
    */
```

Liên hệ không hiển nhiên

Mối liên hệ giữa một chú thích và đoạn mã nó mô tả phải hiển nhiên. Nếu bạn đã bỏ công viết chú thích, ít nhất bạn cũng muốn người đọc có thể nhìn chú thích và đoạn mã mà hiểu chú thích đang nói về cái gì.

Chẳng hạn, hãy xét chú thích này lấy từ apache commons:

```
/*
 * bắt đầu với một mảng đủ lớn để chứa toàn bộ pixel
 * (cộng thêm byte bộ lọc), và thêm 200 byte cho thông tin header
 */
this.pngBytes = new byte[((this.width + 1) * this.height * 3) + 200];
```

Byte bộ lọc là gì? Nó liên quan đến +1 ư? Hay đến *3? Cả hai? Một pixel có phải một byte không? Vì sao 200? Mục đích của chú thích là giải thích đoạn mã không tự giải thích được. Thật đáng tiếc khi một chú thích lại cần chú thích cho chính nó.

Đầu hàm

Hàm ngắn không cần nhiều mô tả. Một cái tên chọn kỹ cho hàm nhỏ chỉ làm một việc thường tốt hơn một chú thích đầu hàm.

Javadoc trong mã không công khai

Javadoc hữu ích cho API công khai bao nhiêu thì lại là thứ đáng ghét với mã không dành cho công chúng bấy nhiêu. Sinh trang javadoc cho các lớp và hàm bên trong một hệ thống nói chung không hữu ích, còn sự trang trọng thừa thãi của các chú thích javadoc chẳng qua chỉ là rác rưởi và thứ gây phân tâm.

Ví dụ

Tôi viết module trong Đoạn mã 4-7 cho XP Immersion lần đầu. Nó nhằm làm ví dụ về phong cách viết mã và chú thích tồi. Kent Beck sau đó tái cấu trúc đoạn mã này thành dạng dễ chịu hơn nhiều trước mặt vài chục sinh viên nhiệt tình. Về sau tôi chuyển thể ví dụ cho cuốn *Agile Software Development, Principles, Patterns, and Practices* và bài đầu tiên trong loạt bài Craftsman của tôi đăng trên tạp chí *Software Development*.

Điều tôi thấy hấp dẫn ở module này là đã có thời nhiều người trong chúng ta sẽ coi nó là “được ghi chép tài liệu tốt”. Giờ chúng ta thấy nó là một mớ hỗn độn nhỏ. Hãy xem bạn tìm được bao nhiêu vấn đề chú thích khác nhau.

Đoạn mã 4-7
GeneratePrimes.java

```
/**
 * Lớp này sinh các số nguyên tố tới một giới hạn tối đa
 * do người dùng chỉ định. Thuật toán dùng là Sàng Eratosthenes.
 * <p>
 * Eratosthenes xứ Cyrene, sinh khoảng 276 TCN, tại Cyrene, Libya,
 * mất khoảng 194, tại Alexandria. Người đầu tiên tính được
 * chu vi Trái Đất. Cũng nổi tiếng nhờ làm việc trên
 * lịch với năm nhuận và điều hành thư viện ở Alexandria.
 * <p>
 * Thuật toán khá đơn giản. Cho một mảng số nguyên
 * bắt đầu từ 2. Gạch bỏ mọi bội số của 2. Tìm số nguyên
 * chưa bị gạch kế tiếp, và gạch bỏ mọi bội số của nó.
 * Lặp lại cho đến khi bạn đã vượt qua căn bậc hai của giá trị
 * tối đa.
 *
 * @author Alphonse
 * @version 13 Feb 2002 atp
 */
import java.util.*;

public class GeneratePrimes
{
  /**
    * @param maxValue là giới hạn sinh số.
    */
  public static int[] generatePrimes(int maxValue)
  {
     if (maxValue >= 2) // trường hợp hợp lệ duy nhất
     {
       // khai báo
       int s = maxValue + 1; // kích thước mảng
       boolean[] f = new boolean[s];
       int i;

       // khởi tạo mảng thành true.
       for (i = 0; i < s; i++)
         f[i] = true;

       // loại bỏ các số đã biết không phải số nguyên tố
       f[0] = f[1] = false;

       // sàng
       int j;
       for (i = 2; i < Math.sqrt(s) + 1; i++)
       {
         if (f[i]) // nếu i chưa bị gạch, gạch bỏ các bội số của nó.
         {
           for (j = 2 * i; j < s; j += i)
              f[j] = false; // bội số không phải số nguyên tố
         }
       }
       // có bao nhiêu số nguyên tố?
       int count = 0;
       for (i = 0; i < s; i++)
```

```
        {
          if (f[i])
            count++; // tăng biến đếm.
        }
        int[] primes = new int[count];
        // chuyển các số nguyên tố vào kết quả
        for (i = 0, j = 0; i < s; i++)
        {
          if (f[i])             // nếu là số nguyên tố
            primes[j++] = i;
        }

        return primes; // trả về các số nguyên tố
      }
      else // maxValue < 2
        return new int[0]; // trả về mảng null nếu đầu vào không hợp lệ.
    }
  }
```

Trong Đoạn mã 4-8, bạn thấy một phiên bản đã tái cấu trúc của cùng module. Hãy để ý rằng việc dùng chú thích được tiết chế đáng kể. Cả module chỉ có hai chú thích. Cả hai đều mang tính giải thích.

Đoạn mã 4-8
PrimeGenerator.java (đã tái cấu trúc)

```
/**
 * Lớp này sinh các số nguyên tố tới một giới hạn tối đa
 * do người dùng chỉ định. Thuật toán dùng là Sàng Eratosthenes.
 * Cho một mảng số nguyên bắt đầu từ 2:
 * Tìm số nguyên chưa bị gạch đầu tiên, và gạch bỏ mọi
```

Đoạn mã 4-8 (tiếp theo)
PrimeGenerator.java (đã tái cấu trúc)

```
 * bội số của nó. Lặp lại cho đến khi không còn bội số
 * nào trong mảng.
 */

public class PrimeGenerator
{
  private static boolean[] crossedOut;
  private static int[] result;

  public static int[] generatePrimes(int maxValue)
  {
    if (maxValue < 2)
      return new int[0];
    else
    {
      uncrossIntegersUpTo(maxValue);
      crossOutMultiples();
      putUncrossedIntegersIntoResult();
```

```
    return result;
  }
}

private static void uncrossIntegersUpTo(int maxValue)
{
  crossedOut = new boolean[maxValue + 1];
  for (int i = 2; i < crossedOut.length; i++)
    crossedOut[i] = false;
}

private static void crossOutMultiples()
{
  int limit = determineIterationLimit();
  for (int i = 2; i <= limit; i++)
    if (notCrossed(i))
      crossOutMultiplesOf(i);
}
private static int determineIterationLimit()
{
  // Mọi bội số trong mảng đều có một ước nguyên tố
  // nhỏ hơn hoặc bằng căn bậc hai của kích thước mảng,
  // nên ta không cần gạch bỏ bội số của những số
  // lớn hơn căn bậc hai đó.
  double iterationLimit = Math.sqrt(crossedOut.length);
  return (int) iterationLimit;
}
private static void crossOutMultiplesOf(int i)
{
  for (int multiple = 2*i;
       multiple < crossedOut.length;
       multiple += i)
    crossedOut[multiple] = true;
}
```

Đoạn mã 4-8 (tiếp theo)
PrimeGenerator.java (đã tái cấu trúc)

```
private static boolean notCrossed(int i)
{
  return crossedOut[i] == false;
}

private static void putUncrossedIntegersIntoResult()
{
  result = new int[numberOfUncrossedIntegers()];
  for (int j = 0, i = 2; i < crossedOut.length; i++)
    if (notCrossed(i))
      result[j++] = i;
}

private static int numberOfUncrossedIntegers()
{
  int count = 0;
  for (int i = 2; i < crossedOut.length; i++)
    if (notCrossed(i))
```

```
            count++;

        return count;
    }
}
```

Có thể dễ dàng lập luận rằng chú thích đầu là dư thừa vì nó đọc rất giống chính hàm generatePrimes. Tuy vậy, tôi nghĩ chú thích ấy giúp người đọc làm quen với thuật toán, nên tôi nghiêng về việc giữ lại.

Lập luận thứ hai gần như chắc chắn là cần thiết. Nó giải thích lý do dùng căn bậc hai làm giới hạn vòng lặp. Tôi không tìm được cái tên biến đơn giản nào, cũng không tìm được cấu trúc mã nào khác làm rõ điểm này. Mặt khác, việc dùng căn bậc hai có thể chỉ là thói tự phụ. Tôi có thật sự tiết kiệm được nhiều thời gian đến thế khi giới hạn vòng lặp ở căn bậc hai không? Việc tính căn bậc hai có thể tốn nhiều thời gian hơn phần tôi tiết kiệm được không?

Điều này đáng để nghĩ. Dùng căn bậc hai làm giới hạn vòng lặp làm hài lòng con người hacker C và hợp ngữ trong tôi, nhưng tôi không tin nó đáng với thời gian và công sức mà mọi người khác sẽ bỏ ra để hiểu nó.

Tài liệu tham khảo

[KP78]: Kernighan and Plaugher, *The Elements of Programming Style*, 2d. ed., McGraw-Hill, 1978.

5

Định dạng



Khi người ta nhìn vào bên trong, chúng ta muốn họ ấn tượng với sự gọn gàng, nhất quán và chăm chút đến từng chi tiết mà họ nhìn thấy. Chúng ta muốn họ sững sờ vì trật tự ngăn nắp. Chúng ta muốn họ nhướng mày khi cuộn qua các module. Chúng ta muốn họ cảm nhận rằng những người chuyên nghiệp đã làm việc ở đây. Còn nếu thay vào đó họ thấy một mớ mã lộn xộn như thể được viết bởi một lũ thủy thủ say rượu, họ sẽ kết luận rằng sự cẩu thả ấy cũng lan ra mọi khía cạnh khác của dự án.

Bạn nên chăm chút để mã của mình được định dạng đẹp. Bạn nên chọn một bộ quy tắc đơn giản chi phối định dạng mã, rồi áp dụng chúng một cách nhất quán. Nếu làm việc

trong một nhóm, cả nhóm nên thống nhất một bộ quy tắc định dạng duy nhất và mọi thành viên đều tuân theo. Có một công cụ tự động áp dụng những quy tắc đó cho bạn sẽ rất hữu ích.

Mục đích của định dạng

Trước hết, hãy nói cho rõ. Định dạng mã là chuyện quan trọng. Nó quan trọng đến mức không thể bỏ qua, và cũng quan trọng đến mức không nên đối xử với nó như một tôn giáo. Định dạng mã là chuyện giao tiếp, mà giao tiếp là ưu tiên hàng đầu trong công việc của một nhà phát triển chuyên nghiệp.

Có lẽ bạn từng nghĩ rằng "làm cho nó chạy được" mới là ưu tiên hàng đầu của một nhà phát triển chuyên nghiệp. Tôi hy vọng đến giờ cuốn sách này đã giúp bạn từ bỏ ý nghĩ đó. Chức năng bạn viết hôm nay rất có thể sẽ thay đổi trong bản phát hành tới, nhưng khả năng đọc của mã sẽ ảnh hưởng sâu sắc đến mọi thay đổi về sau. Phong cách lập trình và khả năng đọc tạo ra những tiền lệ còn tiếp tục ảnh hưởng đến khả năng bảo trì và khả năng mở rộng rất lâu sau khi mã gốc đã bị sửa đến mức không còn nhận ra. Phong cách và kỷ luật của bạn còn lại, dù mã của bạn thì không.

Vậy những vấn đề định dạng nào giúp chúng ta giao tiếp tốt nhất?

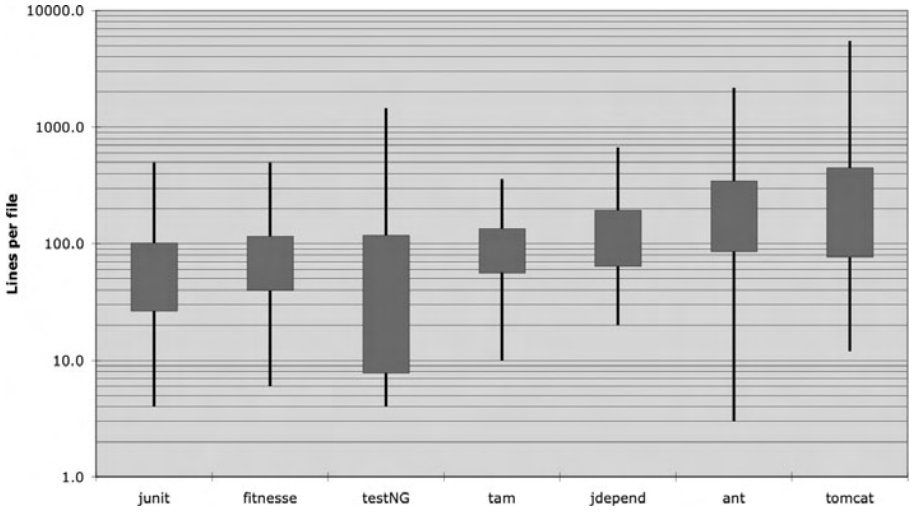
Định dạng theo chiều dọc

Bắt đầu với kích thước theo chiều dọc. Một tệp nguồn nên lớn đến mức nào? Trong Java, kích thước tệp gắn chặt với kích thước lớp. Chúng ta sẽ bàn về kích thước lớp khi bàn về lớp. Còn lúc này, hãy chỉ xét kích thước tệp.

Hầu hết tệp nguồn Java lớn cỡ nào? Hóa ra kích thước trải rất rộng, và phong cách cũng khác nhau đáng kể. Hình 5-1 cho thấy một vài khác biệt đó.

Hình vẽ mô tả bảy dự án khác nhau: Junit, FitNesse, testNG, Time and Money, JDepend, Ant và Tomcat. Các đường thẳng đứng xuyên qua mỗi hộp cho biết độ dài tệp nhỏ nhất và lớn nhất trong dự án. Chiếc hộp thể hiện khoảng một phần ba số tệp (một độ lệch chuẩn[1]). Điểm giữa của hộp là giá trị trung bình. Như vậy, kích thước tệp trung bình trong dự án FitNesse vào khoảng 65 dòng, và khoảng một phần ba số tệp nằm trong khoảng 40 đến hơn 100 dòng. Tệp lớn nhất của FitNesse dài khoảng 400 dòng, tệp nhỏ nhất 6 dòng. Lưu ý đây là thang log, nên chênh lệch nhỏ về vị trí theo chiều dọc ứng với chênh lệch rất lớn về kích thước tuyệt đối.

1. Chiếc hộp thể hiện sigma/2 ở trên và dưới giá trị trung bình. Vâng, tôi biết phân bố độ dài tệp không phải phân bố chuẩn, nên độ lệch chuẩn không chính xác về mặt toán học. Nhưng ở đây chúng ta không cố đạt độ chính xác. Chúng ta chỉ muốn có một cảm nhận.



Hình 5-1

Phân bố độ dài tệp theo thang LOG (chiều cao hộp = sigma)

Junit, FitNesse và Time and Money được tạo nên từ những tệp tương đối nhỏ. Không tệp nào vượt quá 500 dòng và phần lớn dưới 200 dòng. Ngược lại, Tomcat và Ant có những tệp dài hàng nghìn dòng, gần một nửa trong số đó vượt 200 dòng.

Điều đó nói lên gì? Có vẻ vẫn có thể xây những hệ thống đáng kể (FitNesse gần 50. 000 dòng) từ những tệp thường dài 200 dòng, với giới hạn trên là 500. Đây không nên là quy tắc cứng nhắc, nhưng là điều rất đáng mong muốn. Tệp nhỏ thường dễ hiểu hơn tệp lớn.

Ẩn dụ tờ báo

Hãy nghĩ đến một bài báo được viết tốt. Bạn đọc nó từ trên xuống. Ở đầu bài, bạn chờ đợi một dòng tít cho biết câu chuyện nói về điều gì, giúp bạn quyết định có muốn đọc hay không. Đoạn đầu tiên tóm tắt toàn bộ câu chuyện, giấu đi mọi chi tiết và chỉ đưa ra những khái niệm khái quát. Càng xuống dưới, chi tiết càng nhiều, cho đến khi bạn có đủ ngày tháng, tên người, trích dẫn, tuyên bố và những chi tiết vụn vặt khác.

Chúng ta muốn tệp nguồn giống như một bài báo. Cái tên phải đơn giản nhưng có tính giải thích. Chỉ riêng cái tên đã đủ cho biết ta đang ở đúng module hay không. Phần trên cùng của tệp nguồn nên trình bày những khái niệm và thuật toán mức cao. Càng xuống dưới, chi tiết càng tăng, cho đến cuối tệp là những hàm và chi tiết mức thấp nhất.

Một tờ báo gồm nhiều bài; phần lớn rất ngắn. Một số bài dài hơn chút. Rất ít bài chứa đủ chữ cho cả một trang. Chính điều đó làm tờ báo dễ dùng. Nếu tờ báo chỉ là một câu chuyện dài gồm một mớ lộn xộn các sự kiện, ngày tháng và tên riêng, hẳn chúng ta sẽ chẳng buồn đọc.

Độ thoáng dọc giữa các khái niệm

Gần như mọi đoạn mã đều được đọc từ trái sang phải, từ trên xuống dưới. Mỗi dòng là một biểu thức hoặc một mệnh đề, mỗi nhóm dòng là một ý trọn vẹn. Các ý đó nên được ngăn cách bằng dòng trống.

Xét Đoạn mã 5-1. Những dòng trống ngăn cách khai báo package, các câu lệnh import và từng hàm. Quy tắc cực kỳ đơn giản này có ảnh hưởng sâu sắc đến bố cục thị giác của mã. Mỗi dòng trống là một tín hiệu thị giác đánh dấu một khái niệm mới, tách biệt. Khi lướt xuống đoạn mã, mắt bạn bị hút về dòng đầu tiên sau mỗi dòng trống.

Đoạn mã 5-1
BoldWidget.java

```
package fitnesse.wikitext.widgets;

import java.util.regex.*;

public class BoldWidget extends ParentWidget {
  public static final String REGEXP = "'''.+?'''";
  private static final Pattern pattern = Pattern.compile("'''(.+?)'''",
    Pattern.MULTILINE + Pattern.DOTALL
  );

  public BoldWidget(ParentWidget parent, String text) throws Exception {
    super(parent);
    Matcher match = pattern.matcher(text);
    match.find();
    addChildWidgets(match.group(1));
  }

  public String render() throws Exception {
    StringBuffer html = new StringBuffer("<b>");
    html.append(childHtml()).append("</b>");
    return html.toString();
  }
}
```

Bỏ những dòng trống đó đi, như trong Đoạn mã 5-2, lại có tác dụng che khuất khả năng đọc của mã một cách đáng kinh ngạc.

Đoạn mã 5-2
BoldWidget.java

```
package fitnesse.wikitext.widgets;
import java.util.regex.*;
public class BoldWidget extends ParentWidget {
  public static final String REGEXP = "'''.+?'''";
  private static final Pattern pattern = Pattern.compile("'''(.+?)'''",
    Pattern.MULTILINE + Pattern.DOTALL);
  public BoldWidget(ParentWidget parent, String text) throws Exception {
    super(parent);
    Matcher match = pattern.matcher(text);
```

```
      match.find();
      addChildWidgets(match.group(1));}
    public String render() throws Exception {
      StringBuffer html = new StringBuffer("<b>");
      html.append(childHtml()).append("</b>");
      return html.toString();
    }
  }
```

Hiệu ứng này còn rõ hơn khi bạn làm mờ mắt đi. Ở ví dụ thứ nhất, những nhóm dòng khác nhau nổi bật lên; còn ví dụ thứ hai trông như một mớ lẫn lộn. Khác biệt giữa hai đoạn mã chỉ là một chút độ thoáng theo chiều dọc.

Độ đặc theo chiều dọc

Nếu độ thoáng phân tách các khái niệm thì độ đặc theo chiều dọc ngụ ý sự gắn bó chặt chẽ. Vì thế những dòng mã liên quan chặt với nhau nên xuất hiện sát nhau theo chiều dọc. Hãy để ý những chú thích vô ích trong Đoạn mã 5-3 đã phá vỡ sự gắn bó giữa hai biến thực thể như thế nào.

Đoạn mã 5-3

```
public class ReporterConfig {

  /**
   * Tên lớp của reporter listener
   */
  private String m_className;

  /**
   * Các thuộc tính của reporter listener
   */
  private List<Property> m_properties = new ArrayList<Property>();

  public void addProperty(Property property) {
    m_properties.add(property);
  }
```

Đoạn mã 5-4 dễ đọc hơn nhiều. Nó vừa một “tầm mắt”, ít nhất là với tôi. Tôi có thể nhìn vào và thấy ngay đây là một lớp có hai biến và một phương thức, mà không phải xoay đầu hay đảo mắt nhiều. Đoạn mã trước buộc tôi phải cử động mắt và đầu nhiều hơn hẳn mới hiểu được ngang ấy.

Đoạn mã 5-4

```
public class ReporterConfig {
  private String m_className;
  private List<Property> m_properties = new ArrayList<Property>();

  public void addProperty(Property property) {
    m_properties.add(property);
```

```
    }
```

Khoảng cách dọc

Bạn đã bao giờ quanh quẩn trong một lớp, nhảy từ hàm này sang hàm kia, cuộn lên cuộn xuống tệp nguồn, cố đoán xem các hàm liên hệ và vận hành ra sao, rồi cuối cùng lạc trong một mớ rối ren? Bạn đã bao giờ lần ngược chuỗi kế thừa để tìm định nghĩa của một biến hay một hàm? Điều đó thật bực bội, vì bạn đang cố hiểu hệ thống làm gì, nhưng lại tiêu tốn thời gian và sức lực tinh thần vào việc tìm và nhớ xem các mảnh nằm ở đâu.

Những khái niệm liên quan chặt chẽ nên được đặt gần nhau theo chiều dọc [G10]. Rõ ràng quy tắc này không áp dụng cho những khái niệm thuộc về các tệp khác nhau. Nhưng khi ấy, những khái niệm liên quan chặt chẽ cũng không nên bị tách sang các tệp khác nhau, trừ khi bạn có lý do rất chính đáng. Đây cũng là một trong những lý do nên tránh biến protected.

Với những khái niệm liên quan chặt đến mức cùng thuộc một tệp nguồn, khoảng cách dọc giữa chúng nên phản ánh mức độ quan trọng của khái niệm này đối với việc hiểu khái niệm kia. Chúng ta muốn tránh buộc người đọc phải nhảy nhót qua các tệp nguồn và các lớp.

Khai báo biến

Biến nên được khai báo càng gần nơi sử dụng càng tốt. Vì các hàm của chúng ta rất ngắn, biến cục bộ nên xuất hiện ở đầu mỗi hàm, như trong hàm hơi dài này từ Junit4.3.1.

```
private static void readPreferences() {
  InputStream is= null;
  try {
    is= new FileInputStream(getPreferencesFile());
    setPreferences(new Properties(getPreferences()));
    getPreferences().load(is);
  } catch (IOException e) {
    try {
      if (is != null)
        is.close();
    } catch (IOException e1) {
    }
  }
}
```

Biến điều khiển của vòng lặp thường nên được khai báo ngay trong câu lệnh lặp, như trong hàm nhỏ xinh này cũng từ nguồn đó.

```
public int countTestCases() {
  int count= 0;
  for (Test each : tests)
    count += each.countTestCases();
  return count;
}
```

Trong vài trường hợp hiếm, biến có thể được khai báo ở đầu một khối hoặc ngay trước một vòng lặp trong một hàm khá dài. Bạn có thể thấy một biến như vậy trong đoạn trích giữa một hàm rất dài của TestNG.

```
...
for (XmlTest test : m_suite.getTests()) {
  TestRunner tr = m_runnerFactory.newTestRunner(this, test);
  tr.addListener(m_textReporter);
  m_testRunners.add(tr);

  invoker = tr.getInvoker();

  for (ITestNGMethod m : tr.getBeforeSuiteMethods()) {
    beforeSuiteMethods.put(m.getMethod(), m);
  }
  for (ITestNGMethod m : tr.getAfterSuiteMethods()) {
    afterSuiteMethods.put(m.getMethod(), m);
  }
}
...
```

Ngược lại, biến thực thể nên được khai báo ở đầu lớp. Điều này không làm tăng khoảng cách dọc của những biến ấy, vì trong một lớp được thiết kế tốt, chúng được nhiều phương thức của lớp dùng đến, nếu không nói là tất cả.

Đã có nhiều tranh luận về chỗ đặt biến thực thể. Trong C++, chúng tôi thường áp dụng cái gọi là quy tắc cái kéo, đặt toàn bộ biến thực thể xuống cuối. Tuy vậy, quy ước phổ biến trong Java là đặt tất cả ở đầu lớp. Tôi không thấy lý do gì để theo một quy ước khác. Điều quan trọng là biến thực thể phải được khai báo ở một chỗ ai cũng biết. Mọi người đều phải biết đi đâu để xem các khai báo.

Xét trường hợp kỳ lạ của lớp TestSuite trong JUnit 4.3.1. Tôi đã lược bớt rất nhiều trong lớp này để làm rõ ý. Nếu nhìn khoảng giữa đoạn mã, bạn sẽ thấy hai biến thực thể được khai báo ở đó. Khó mà giấu chúng ở một chỗ tốt hơn. Người đọc mã này chỉ có thể tình cờ vấp phải các khai báo ấy (như tôi đã từng).

```
public class TestSuite implements Test {
  static public Test createTest(Class<? extends TestCase> theClass,
                                String name) {
    ...
  }

  public static Constructor<? extends TestCase>
  getTestConstructor(Class<? extends TestCase> theClass)
  throws NoSuchMethodException {
    ...
  }

  public static Test warning(final String message) {
    ...
  }

  private static String exceptionToString(Throwable t) {
    ...
```

```
    }
    private String fName;

    private Vector<Test> fTests= new Vector<Test>(10);

    public TestSuite() {
    }

    public TestSuite(final Class<? extends TestCase> theClass) {
      ...
    }

    public TestSuite(Class<? extends TestCase> theClass, String name) {
      ...
    }
    ... ... ... ... ...
  }
```

Các hàm phụ thuộc

Nếu một hàm gọi hàm khác, hai hàm nên ở gần nhau theo chiều dọc, và nếu có thể thì hàm gọi nằm trên hàm được gọi. Điều này cho chương trình một dòng chảy tự nhiên. Nếu quy ước được tuân thủ đáng tin cậy, người đọc có thể yên tâm rằng định nghĩa của hàm sẽ xuất hiện ngay sau chỗ nó được dùng. Xét đoạn trích từ FitNesse trong Đoạn mã 5-5. Hãy để ý hàm trên cùng gọi những hàm bên dưới nó, và những hàm đó lại gọi những hàm dưới chúng. Điều này giúp dễ tìm các hàm được gọi và làm tăng mạnh khả năng đọc của cả module.

Đoạn mã 5-5
WikiPageResponder.java

```
public class WikiPageResponder implements SecureResponder {
  protected WikiPage page;
  protected PageData pageData;
  protected String pageTitle;
  protected Request request;
  protected PageCrawler crawler;

  public Response makeResponse(FitNesseContext context, Request request)
    throws Exception {
    String pageName = getPageNameOrDefault(request, "FrontPage");
```

Đoạn mã 5-5 (tiếp theo)
WikiPageResponder.java

```
    loadPage(pageName, context);
    if (page == null)
      return notFoundResponse(context, request);
    else
      return makePageResponse(context);
  }
```

```
    private String getPageNameOrDefault(Request request, String
      defaultPageName)
    {
      String pageName = request.getResource();
      if (StringUtil.isBlank(pageName))
        pageName = defaultPageName;

      return pageName;
    }

    protected void loadPage(String resource, FitNesseContext context)
      throws Exception {
      WikiPagePath path = PathParser.parse(resource);
      crawler = context.root.getPageCrawler();
      crawler.setDeadEndStrategy(new VirtualEnabledPageCrawler());
      page = crawler.getPage(context.root, path);
      if (page != null)
        pageData = page.getData();
    }

    private Response notFoundResponse(FitNesseContext context, Request request)
      throws Exception {
      return new NotFoundResponder().makeResponse(context, request);
    }

    private SimpleResponse makePageResponse(FitNesseContext context)
      throws Exception {
      pageTitle = PathParser.render(crawler.getFullPath(page));
      String html = makeHtml(context);

      SimpleResponse response = new SimpleResponse();
      response.setMaxAge(0);
      response.setContent(html);
      return response;
    }
  ...
```

Nhân đây, đoạn trích này là một ví dụ hay về việc giữ hằng ở đúng mức [G35]. Hằng "FrontPage" lẽ ra có thể bị chôn trong hàm getPageNameOrDefault, nhưng làm vậy sẽ giấu một hằng ai cũng biết và ai cũng chờ đợi vào một hàm có mức thấp không thích hợp. Tốt hơn là truyền hằng đó từ nơi hợp lý để biết nó xuống nơi thực sự dùng nó.

Gần gũi về khái niệm

Một số đoạn mã muốn ở gần những đoạn khác. Giữa chúng có một sự gần gũi nhất định về khái niệm. Sự gần gũi càng mạnh thì khoảng cách dọc giữa chúng càng nên nhỏ.

Như đã thấy, sự gần gũi này có thể dựa trên phụ thuộc trực tiếp, chẳng hạn hàm này gọi hàm kia, hoặc một hàm dùng một biến. Nhưng còn những nguyên nhân khác. Sự gần gũi cũng có thể đến từ việc một nhóm hàm cùng thực hiện một thao tác tương tự. Xét đoạn mã sau từ Junit 4.3.1:

```
public class Assert {
  static public void assertTrue(String message, boolean condition) {
    if (!condition)
      fail(message);
  }

  static public void assertTrue(boolean condition) {
    assertTrue(null, condition);
  }

  static public void assertFalse(String message, boolean condition) {
    assertTrue(message, !condition);
  }

  static public void assertFalse(boolean condition) {
    assertFalse(null, condition);
  }
...
```

Những hàm này có sự gần gũi mạnh về khái niệm vì chúng dùng chung một cách đặt tên và thực hiện những biến thể của cùng một nhiệm vụ cơ bản. Việc chúng gọi nhau chỉ là chuyện phụ. Kể cả khi không gọi nhau, chúng vẫn muốn ở gần nhau.



Thứ tự dọc

Nói chung, chúng ta muốn các phụ thuộc gọi hàm hướng xuống dưới. Nghĩa là hàm được gọi nên nằm dưới hàm gọi nó[2]. Điều này tạo ra một dòng chảy đẹp từ mức cao xuống mức thấp trong module mã nguồn.

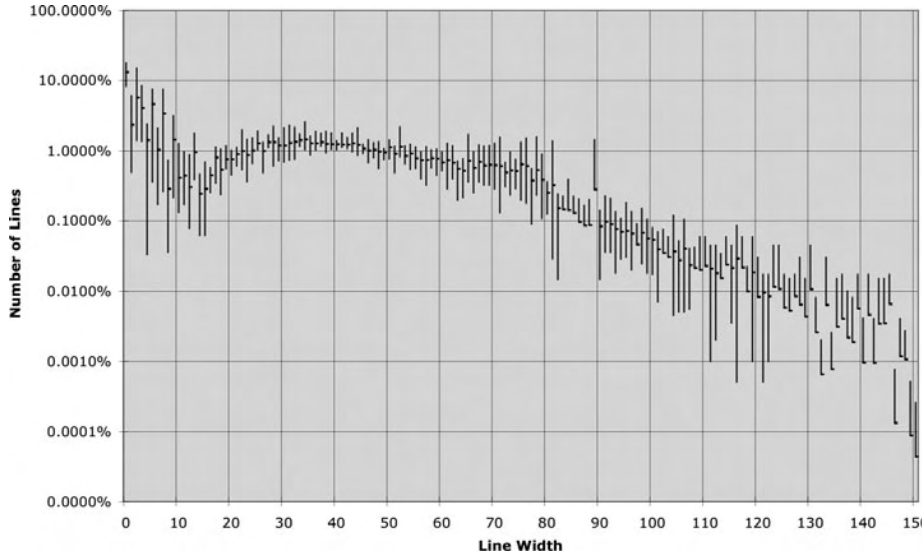
Như trong các bài báo, chúng ta chờ đợi những khái niệm quan trọng nhất xuất hiện trước, và được diễn đạt với ít chi tiết gây nhiễu nhất. Chúng ta chờ đợi những chi tiết mức thấp xuất hiện sau cùng. Nhờ vậy, ta có thể lướt qua tệp nguồn, nắm được ý chính từ vài

2. Điều này hoàn toàn ngược với những ngôn ngữ như Pascal, C và C++, vốn bắt buộc hàm phải được định nghĩa, hoặc ít nhất là khai báo, trước khi dùng.

hàm đầu mà không phải đắm mình vào chi tiết. Đoạn mã 5-5 được tổ chức theo cách này. Có lẽ còn những ví dụ hay hơn ở Đoạn mã 15-5 trang 263 và Đoạn mã 3-7 trang 50.

Định dạng theo chiều ngang

Một dòng nên rộng bao nhiêu? Để trả lời, hãy xem các dòng trong những chương trình điển hình rộng ra sao. Một lần nữa, ta xét bảy dự án. Hình 5-2 cho thấy phân bố độ dài dòng của cả bảy dự án. Sự đều đặn thật ấn tượng, nhất là quanh mốc 45 ký tự. Thật vậy, mỗi độ dài từ 20 đến 60 chiếm khoảng 1 phần trăm tổng số dòng. Cộng lại là 40 phần trăm! Có lẽ khoảng 30 phần trăm nữa có độ rộng dưới 10 ký tự. Nhớ rằng đây là thang log, nên đoạn sụt giảm trông có vẻ tuyến tính sau mốc 80 ký tự thực ra rất đáng kể. Lập trình viên rõ ràng thích dòng ngắn.



Hình 5-2
Phân bố độ rộng dòng trong Java

Điều này gợi ý rằng ta nên cố giữ dòng ngắn. Giới hạn 80 cột kiểu Hollerith ngày xưa hơi tùy tiện, và tôi không phản đối việc dòng dài tới 100, thậm chí 120 ký tự. Nhưng vượt xa hơn thì có lẽ chỉ là cẩu thả.

Tôi từng theo quy tắc rằng không bao giờ phải cuộn sang phải. Nhưng màn hình ngày nay quá rộng cho quy tắc đó, và các lập trình viên trẻ có thể thu nhỏ phông chữ đến mức nhét được 200 ký tự lên màn hình. Đừng làm vậy. Cá nhân tôi đặt giới hạn ở 120.

Độ thoáng và độ đặc theo chiều ngang

Chúng ta dùng khoảng trắng theo chiều ngang để liên kết những thứ liên quan chặt và tách rời những thứ ít liên quan hơn. Xét hàm sau:

```
private void measureLine(String line) {
  lineCount++;
  int lineSize = line.length();
  totalChars += lineSize;
  lineWidthHistogram.addLine(lineSize, lineCount);
  recordWidestLine(lineSize);
}
```

Tôi đặt khoảng trắng quanh các toán tử gán để làm chúng nổi bật. Câu lệnh gán có hai thành phần chính, tách biệt: vế trái và vế phải. Khoảng trắng làm rõ sự tách biệt đó.

Ngược lại, tôi không đặt khoảng trắng giữa tên hàm và dấu ngoặc mở. Lý do là hàm và các đối số của nó gắn bó chặt với nhau. Tách chúng ra sẽ khiến chúng trông rời rạc thay vì liền một khối. Tôi tách các đối số bên trong ngoặc bằng khoảng trắng để làm nổi bật dấu phẩy và cho thấy các đối số tách biệt nhau.

Một cách dùng khác của khoảng trắng là làm nổi bật độ ưu tiên của toán tử.

```
public class Quadratic {
  public static double root1(double a, double b, double c) {
    double determinant = determinant(a, b, c);
    return (-b + Math.sqrt(determinant)) / (2*a);
  }

  public static double root2(int a, int b, int c) {
    double determinant = determinant(a, b, c);
    return (-b - Math.sqrt(determinant)) / (2*a);
  }

  private static double determinant(double a, double b, double c) {
    return b*b - 4*a*c;
  }
}
```

Hãy để ý các phương trình đọc trôi chảy đến mức nào. Các thừa số không có khoảng trắng giữa chúng vì chúng có độ ưu tiên cao. Các số hạng được ngăn bằng khoảng trắng vì phép cộng và phép trừ có độ ưu tiên thấp hơn.

Đáng tiếc, phần lớn công cụ định dạng lại mã không nhận biết độ ưu tiên của toán tử và áp đặt một kiểu khoảng cách như nhau ở mọi chỗ. Vì thế những khoảng cách tinh tế như trên dễ bị mất đi sau khi bạn định dạng lại mã.

Căn lề ngang

Hồi còn viết hợp ngữ[3], tôi dùng căn lề ngang để làm nổi bật một số cấu trúc. Khi chuyển

3. Tôi đùa ai chứ? Tôi vẫn là một lập trình viên hợp ngữ. Có thể đưa cậu bé ra khỏi kim loại, nhưng không thể đưa kim loại ra khỏi cậu bé!

sang viết C, rồi C++, và cuối cùng là Java, tôi vẫn tiếp tục cố xếp thẳng tên các biến trong một nhóm khai báo, hoặc xếp thẳng các giá trị bên phải trong một loạt câu lệnh gán. Mã của tôi khi đó có lẽ trông như thế này:

```
public class FitNesseExpediter implements ResponseSender
{
  private Socket            socket;
  private InputStream       input;
  private OutputStream      output;
  private Request           request;
  private Response          response;
  private FitNesseContext   context;
  protected long            requestParsingTimeLimit;
  private long              requestProgress;
  private long              requestParsingDeadline;
  private boolean           hasError;

  public FitNesseExpediter(Socket          s,
                           FitNesseContext context) throws Exception
  {
    this.context =          context;
    socket =                s;
    input =                 s.getInputStream();
    output =                s.getOutputStream();
    requestParsingTimeLimit = 10000;
  }
```

Nhưng tôi nhận ra kiểu căn lề này không hữu ích. Việc căn lề dường như nhấn mạnh sai thứ và kéo mắt tôi rời khỏi ý định thật. Chẳng hạn, trong danh sách khai báo trên, bạn dễ bị cám dỗ đọc thẳng xuống danh sách tên biến mà không nhìn kiểu của chúng. Tương tự, trong danh sách câu lệnh gán, bạn dễ bị cám dỗ nhìn thẳng xuống danh sách giá trị bên phải mà không hề thấy toán tử gán. Tệ hơn, các công cụ định dạng tự động thường xóa bỏ kiểu căn lề này.

Vậy nên cuối cùng, tôi không làm kiểu đó nữa. Ngày nay tôi thích những khai báo và phép gán không căn lề, như dưới đây, vì chúng chỉ ra một khiếm khuyết quan trọng. Nếu tôi có những danh sách dài cần căn lề, vấn đề nằm ở độ dài của danh sách chứ không phải ở việc thiếu căn lề. Độ dài danh sách khai báo trong FitNesseExpediter dưới đây cho thấy lớp này nên được tách ra.

```
public class FitNesseExpediter implements ResponseSender
{
  private Socket socket;
  private InputStream input;
  private OutputStream output;
  private Request request;
  private Response response;
  private FitNesseContext context;
  protected long requestParsingTimeLimit;
  private long requestProgress;
  private long requestParsingDeadline;
  private boolean hasError;
```

```
  public FitNesseExpediter(Socket s, FitNesseContext context) throws
    Exception
  {
    this.context = context;
    socket = s;
    input = s.getInputStream();
    output = s.getOutputStream();
    requestParsingTimeLimit = 10000;
  }
```

Thụt lề

Một tệp nguồn là một hệ thứ bậc, gần giống một dàn ý. Có thông tin liên quan đến toàn bộ tệp, đến từng lớp trong tệp, đến các phương thức trong lớp, đến các khối trong phương thức, và đệ quy đến các khối trong khối. Mỗi mức của hệ thứ bậc này là một phạm vi, nơi có thể khai báo tên và nơi các khai báo cùng câu lệnh thực thi được diễn giải.

Để hệ thứ bậc các phạm vi này hiện ra trước mắt, ta thụt lề các dòng mã nguồn tương ứng với vị trí của chúng trong hệ thứ bậc. Những câu lệnh ở mức tệp, như phần lớn khai báo lớp, không thụt lề. Các phương thức trong một lớp thụt vào một mức so với lớp. Phần thân của những phương thức đó thụt vào một mức so với khai báo phương thức. Phần thân của các khối thụt vào một mức so với khối chứa chúng, và cứ thế.

Lập trình viên dựa rất nhiều vào cách thụt lề này. Họ nhìn theo mép trái để biết một dòng thuộc phạm vi nào. Nhờ đó họ có thể nhanh chóng nhảy qua những phạm vi không liên quan đến việc đang làm, chẳng hạn phần thân của câu lệnh if hay while. Họ quét mép trái để tìm khai báo phương thức mới, biến mới, thậm chí lớp mới. Không có thụt lề, con người gần như không thể đọc nổi chương trình.

Xét hai chương trình sau, giống hệt nhau về cú pháp và ngữ nghĩa:

```
public class FitNesseServer implements SocketServer { private FitNesseContext
context; public FitNesseServer(FitNesseContext context) { this.context =
context; } public void serve(Socket s) { serve(s, 10000); } public void
serve(Socket s, long requestTimeout) { try { FitNesseExpediter sender = new
FitNesseExpediter(s, context);
sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
catch(Exception e) { e.printStackTrace(); } } }

-----

public class FitNesseServer implements SocketServer {
  private FitNesseContext context;

  public FitNesseServer(FitNesseContext context) {
    this.context = context;
  }

  public void serve(Socket s) {
    serve(s, 10000);
  }
  public void serve(Socket s, long requestTimeout) {
    try {
      FitNesseExpediter sender = new FitNesseExpediter(s, context);
```

```
        sender.setRequestParsingTimeLimit(requestTimeout);
        sender.start();
      }
      catch (Exception e) {
        e.printStackTrace();
      }
    }
  }
```

Mắt bạn nhanh chóng nhận ra cấu trúc của tệp được thụt lề. Gần như ngay lập tức bạn thấy các biến, hàm khởi tạo, hàm truy cập và phương thức. Chỉ mất vài giây để nhận ra đây là một kiểu giao diện đơn giản cho socket, có cơ chế hết giờ. Còn phiên bản không thụt lề gần như không thể xuyên qua nếu không nghiên cứu kỹ.

Phá vỡ thụt lề

Đôi khi ta bị cám dỗ phá vỡ quy tắc thụt lề với những câu lệnh if ngắn, vòng lặp while ngắn hoặc hàm ngắn. Mỗi lần chiều theo cám dỗ ấy, tôi gần như luôn quay lại thêm thụt lề vào. Vì vậy tôi tránh thu gọn các phạm vi về một dòng như thế này:

```
public class CommentWidget extends TextWidget
{
  public static final String REGEXP = "^#[^\r\n]*(?:(?:\r\n)|\n|\r)?";
  public CommentWidget(ParentWidget parent, String text){super(parent,
    text);}
  public String render() throws Exception {return ""; }
}
```

Thay vào đó, tôi thích mở rộng và thụt lề các phạm vi, như thế này:

```
public class CommentWidget extends TextWidget {
  public static final String REGEXP = "^#[^\r\n]*(?:(?:\r\n)|\n|\r)?";

  public CommentWidget(ParentWidget parent, String text) {
    super(parent, text);
  }

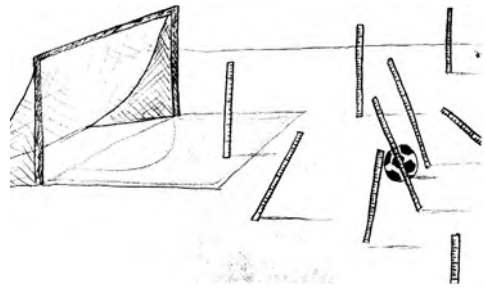
  public String render() throws Exception {
    return "";
  }
}
```

Thân rỗng

Đôi khi phần thân của câu lệnh while hoặc for là một thân rỗng, như dưới đây. Tôi không thích kiểu cấu trúc này và cố tránh chúng. Khi không tránh được, tôi bảo đảm thân rỗng được thụt lề đúng và bao trong ngoặc nhọn. Không thể kể hết số lần tôi bị lừa bởi một dấu chấm phẩy nằm im lìm ở cuối vòng lặp while, cùng dòng. Trừ khi bạn làm cho dấu chấm phẩy ấy hiện ra bằng cách thụt nó vào một dòng riêng, nó quá khó thấy.

```
while (dis.read(buf, 0, readBufferSize) != -1)
  ;
```

Quy tắc của nhóm



Tiêu đề mục này là một cách chơi chữ. Mỗi lập trình viên có bộ quy tắc định dạng ưa thích của riêng mình, nhưng nếu làm việc trong một nhóm thì quy tắc của nhóm là trên hết.

Một nhóm phát triển nên thống nhất một phong cách định dạng duy nhất, rồi mọi thành viên trong nhóm đều dùng phong cách đó. Chúng ta muốn phần mềm có một phong cách nhất quán. Chúng ta không muốn nó trông như được viết bởi một đám người mỗi người một ý.

Khi bắt đầu dự án FitNesse năm 2002, tôi ngồi lại với nhóm để bàn ra một phong cách lập trình. Việc đó mất khoảng 10 phút. Chúng tôi quyết định đặt ngoặc nhọn ở đâu, thụt lề mấy dấu cách, đặt tên lớp, biến và phương thức ra sao, vân vân. Rồi chúng tôi mã hóa những quy tắc ấy vào công cụ định dạng mã của IDE và giữ nguyên từ đó đến nay. Đó không phải những quy tắc tôi thích; đó là quy tắc do nhóm quyết định. Là một thành viên của nhóm, tôi tuân theo chúng khi viết mã trong dự án FitNesse.

Hãy nhớ, một hệ thống phần mềm tốt được tạo nên từ những tài liệu đọc trôi chảy. Chúng cần có một phong cách nhất quán và mượt mà. Người đọc cần có thể tin rằng những động tác định dạng đã thấy ở tệp nguồn này sẽ mang cùng ý nghĩa ở những tệp khác. Điều cuối cùng chúng ta muốn là làm tăng thêm độ phức tạp cho mã nguồn bằng cách viết nó trong một mớ phong cách cá nhân khác nhau.

Quy tắc định dạng của Uncle Bob

Những quy tắc cá nhân tôi dùng rất đơn giản và được minh họa bằng mã trong Đoạn mã 5-6. Hãy coi đây là một ví dụ cho thấy mã chính là tài liệu chuẩn lập trình tốt nhất.

Đoạn mã 5-6
CodeAnalyzer.java

```
public class CodeAnalyzer implements JavaFileAnalysis {
  private int lineCount;
  private int maxLineWidth;
  private int widestLineNumber;
  private LineWidthHistogram lineWidthHistogram;
  private int totalChars;

  public CodeAnalyzer() {
    lineWidthHistogram = new LineWidthHistogram();
  }
  public static List<File> findJavaFiles(File parentDirectory) {
    List<File> files = new ArrayList<File>();
    findJavaFiles(parentDirectory, files);
    return files;
  }

  private static void findJavaFiles(File parentDirectory, List<File> files) {
    for (File file : parentDirectory.listFiles()) {
      if (file.getName().endsWith(".java"))
        files.add(file);
      else if (file.isDirectory())
        findJavaFiles(file, files);
    }
  }

  public void analyzeFile(File javaFile) throws Exception {
    BufferedReader br = new BufferedReader(new FileReader(javaFile));
    String line;
    while ((line = br.readLine()) != null)
      measureLine(line);
  }

  private void measureLine(String line) {
    lineCount++;
    int lineSize = line.length();
    totalChars += lineSize;
    lineWidthHistogram.addLine(lineSize, lineCount);
    recordWidestLine(lineSize);
  }
  private void recordWidestLine(int lineSize) {
    if (lineSize > maxLineWidth) {
      maxLineWidth = lineSize;
      widestLineNumber = lineCount;
    }
  }

  public int getLineCount() {
    return lineCount;
  }

  public int getMaxLineWidth() {
    return maxLineWidth;
  }
```

Đoạn mã 5-6 (tiếp theo)
CodeAnalyzer.java

```
  public int getWidestLineNumber() {
    return widestLineNumber;
  }

  public LineWidthHistogram getLineWidthHistogram() {
    return lineWidthHistogram;
  }

  public double getMeanLineWidth() {
    return (double)totalChars/lineCount;
  }

  public int getMedianLineWidth() {
    Integer[] sortedWidths = getSortedWidths();
    int cumulativeLineCount = 0;
    for (int width : sortedWidths) {
      cumulativeLineCount += lineCountForWidth(width);
      if (cumulativeLineCount > lineCount/2)
        return width;
    }
    throw new Error("Cannot get here");
  }

  private int lineCountForWidth(int width) {
    return lineWidthHistogram.getLinesforWidth(width).size();
  }

  private Integer[] getSortedWidths() {
    Set<Integer> widths = lineWidthHistogram.getWidths();
    Integer[] sortedWidths = (widths.toArray(new Integer[0]));
    Arrays.sort(sortedWidths);
    return sortedWidths;
  }
}
```

6

Đối tượng và cấu trúc dữ liệu



Có lý do để chúng ta giữ các biến của mình ở chế độ riêng tư. Chúng ta không muốn ai khác phụ thuộc vào chúng. Chúng ta muốn giữ quyền tự do thay đổi kiểu hay cách hiện thực của chúng theo ý thích hoặc theo một phút bốc đồng. Vậy tại sao nhiều lập trình viên vẫn tự động thêm getter và setter cho đối tượng của mình, phơi bày các biến riêng tư như thể chúng là công khai?

Trừu tượng hóa dữ liệu

Hãy xét khác biệt giữa Đoạn mã 6-1 và Đoạn mã 6-2. Cả hai đều biểu diễn dữ liệu của một điểm trên mặt phẳng Descartes. Vậy mà một cái phơi bày cách hiện thực, cái còn lại che giấu hoàn toàn.

Đoạn mã 6-1
Điểm cụ thể

```
public class Point {
  public double x;
  public double y;
}
```

Đoạn mã 6-2
Điểm trừu tượng

```
public interface Point {
  double getX();
  double getY();
  void setCartesian(double x, double y);
  double getR();
  double getTheta();
  void setPolar(double r, double theta);
}
```

Điều tuyệt vời ở Đoạn mã 6-2 là bạn không thể nào biết được cách hiện thực dùng tọa độ vuông góc hay tọa độ cực. Có khi nó chẳng dùng cái nào cả! Vậy mà giao diện vẫn biểu diễn một cấu trúc dữ liệu không thể nhầm lẫn.

Nhưng nó biểu diễn nhiều hơn một cấu trúc dữ liệu. Các phương thức áp đặt một chính sách truy cập. Bạn có thể đọc từng tọa độ riêng lẻ, nhưng phải đặt các tọa độ cùng nhau như một thao tác nguyên tử.

Ngược lại, Đoạn mã 6-1 hiện thực rất rõ ràng bằng tọa độ vuông góc, và nó buộc chúng ta thao tác trên từng tọa độ một cách độc lập. Như thế là phơi bày cách hiện thực. Thật vậy, nó vẫn phơi bày cách hiện thực ngay cả khi các biến là riêng tư và chúng ta dùng getter, setter cho từng biến.

Che giấu cách hiện thực không chỉ là đặt một lớp hàm giữa các biến. Che giấu cách hiện thực là chuyện trừu tượng hóa! Một lớp không đơn thuần đẩy các biến của mình ra ngoài qua getter và setter. Nó phơi bày những giao diện trừu tượng cho phép người dùng thao tác trên bản chất của dữ liệu mà không cần biết cách hiện thực.

Hãy xét Đoạn mã 6-3 và Đoạn mã 6-4. Cái đầu dùng những thuật ngữ cụ thể để truyền đạt mức nhiên liệu của một xe, còn cái sau dùng trừu tượng phần trăm. Trong trường hợp cụ thể, bạn có thể khá chắc rằng đây chỉ là các accessor của biến. Trong trường hợp trừu tượng, bạn không có manh mối nào về hình thái của dữ liệu.

Đoạn mã 6-3
Xe cụ thể

```
public interface Vehicle {
  double getFuelTankCapacityInGallons();
  double getGallonsOfGasoline();
}
```

Đoạn mã 6-4
Xe trừu tượng

```
public interface Vehicle {
  double getPercentFuelRemaining();
}
```

Trong cả hai trường hợp trên, lựa chọn thứ hai đáng mong muốn hơn. Chúng ta không muốn phơi bày chi tiết dữ liệu của mình. Chúng ta muốn diễn đạt dữ liệu bằng những thuật ngữ trừu tượng. Điều này không chỉ đạt được bằng cách dùng giao diện và/hoặc getter, setter. Cần suy nghĩ nghiêm túc về cách tốt nhất để biểu diễn dữ liệu mà một đối tượng chứa. Lựa chọn tồi nhất là thản nhiên thêm getter và setter.

Đối tượng và cấu trúc dữ liệu: tính phản đối xứng

Hai ví dụ này cho thấy khác biệt giữa đối tượng và cấu trúc dữ liệu. Đối tượng che giấu dữ liệu của mình sau các lớp trừu tượng và phơi bày những hàm thao tác trên dữ liệu đó. Cấu trúc dữ liệu phơi bày dữ liệu và không có hàm nào có ý nghĩa. Hãy quay lại đọc lần nữa. Hãy để ý tính bổ sung cho nhau của hai định nghĩa. Chúng gần như đối lập hoàn toàn. Khác biệt này có vẻ nhỏ nhặt, nhưng hệ quả của nó rất xa.

Hãy xét ví dụ về hình học theo lối thủ tục trong Đoạn mã 6-5. Lớp Geometry thao tác trên ba lớp hình. Các lớp hình là những cấu trúc dữ liệu đơn giản, không có hành vi nào. Toàn bộ hành vi nằm trong lớp Geometry.

Đoạn mã 6-5
Hình học theo lối thủ tục

```
public class Square {
  public Point topLeft;
  public double side;
}
public class Rectangle {
  public Point topLeft;
  public double height;
  public double width;
}
```

```
public class Circle {
  public Point center;
  public double radius;
}

public class Geometry {
  public final double PI = 3.141592653589793;

  public double area(Object shape) throws NoSuchShapeException
  {
    if (shape instanceof Square) {
      Square s = (Square)shape;
      return s.side * s.side;
    }
    else if (shape instanceof Rectangle) {
      Rectangle r = (Rectangle)shape;
      return r.height * r.width;
    }
    else if (shape instanceof Circle) {
      Circle c = (Circle)shape;
      return PI * c.radius * c.radius;
    }
    throw new NoSuchShapeException();
  }
}
```

Những lập trình viên hướng đối tượng có thể sẽ nhăn mặt và phàn nàn rằng đây là lối thủ tục, và họ đúng. Nhưng cái nhếch mép đó có khi không có cơ sở. Hãy xét điều gì sẽ xảy ra nếu thêm một hàm perimeter() vào Geometry. Các lớp hình sẽ không bị ảnh hưởng gì! Mọi lớp khác phụ thuộc vào các hình cũng không bị ảnh hưởng! Mặt khác, nếu tôi thêm một hình mới, tôi phải sửa mọi hàm trong Geometry để xử lý nó. Lại đọc lại điều đó đi. Hãy để ý hai tình huống này đối lập nhau hoàn toàn.

Giờ hãy xét giải pháp hướng đối tượng trong Đoạn mã 6-6. Ở đây phương thức area() là đa hình. Không cần lớp Geometry. Vậy nếu tôi thêm một hình mới, không hàm hiện có nào bị ảnh hưởng, nhưng nếu tôi thêm một hàm mới thì mọi hình đều phải thay đổi![1]

Đoạn mã 6-6
Các hình đa hình

```
public class Square implements Shape {
  private Point topLeft;
  private double side;
  public double area() {
    return side*side;
  }
}
public class Rectangle implements Shape {
  private Point topLeft;
  private double height;
```

1. Có những cách lách điều này mà các nhà thiết kế hướng đối tượng giàu kinh nghiệm đều biết: VISITOR, hay dual-dispatch, chẳng hạn. Nhưng những kỹ thuật này có cái giá riêng và thường đưa cấu trúc trở lại thành một chương trình thủ tục.

```
    private double width;
    public double area() {
      return height * width;
    }
  }
```

Đoạn mã 6-6 (tiếp theo)
Các hình đa hình

```
public class Circle implements Shape {
  private Point center;
  private double radius;
  public final double PI = 3.141592653589793;
  public double area() {
    return PI * radius * radius;
  }
}
```

Lại một lần nữa, ta thấy tính bổ sung của hai định nghĩa này; chúng gần như đối lập hoàn toàn! Điều này phơi bày sự phân đôi nền tảng giữa đối tượng và cấu trúc dữ liệu:

> *Mã thủ tục (mã dùng cấu trúc dữ liệu) giúp dễ thêm hàm mới mà không phải thay đổi các cấu trúc dữ liệu hiện có. Ngược lại, mã OO giúp dễ thêm lớp mới mà không phải thay đổi các hàm hiện có.*

Điều ngược lại cũng đúng:

> *Mã thủ tục khiến việc thêm cấu trúc dữ liệu mới trở nên khó, vì mọi hàm đều phải thay đổi. Mã OO khiến việc thêm hàm mới trở nên khó, vì mọi lớp đều phải thay đổi.*

Vậy những gì khó với OO thì dễ với lối thủ tục, và những gì khó với lối thủ tục thì dễ với OO!

Trong bất kỳ hệ thống phức tạp nào cũng sẽ có lúc ta muốn thêm kiểu dữ liệu mới thay vì hàm mới. Với những trường hợp đó, đối tượng và OO là thích hợp nhất. Mặt khác, cũng sẽ có lúc ta muốn thêm hàm mới thay vì kiểu dữ liệu. Khi đó mã thủ tục và cấu trúc dữ liệu sẽ thích hợp hơn.

Những lập trình viên trưởng thành biết rằng ý tưởng mọi thứ đều là đối tượng chỉ là chuyện hoang đường. Đôi khi bạn thật sự muốn những cấu trúc dữ liệu đơn giản với các thủ tục thao tác trên chúng.

Định luật Demeter

Có một nguyên tắc kinh nghiệm nổi tiếng gọi là Định luật Demeter[2], nói rằng một module không nên biết về bên trong của những đối tượng mà nó thao tác. Như ta thấy ở mục trước, đối tượng che giấu dữ liệu và phơi bày các thao tác. Điều này có nghĩa là một đối tượng không nên phơi bày cấu trúc bên trong của mình qua accessor, vì làm thế là phơi bày chứ không phải che giấu cấu trúc bên trong của nó.

Chính xác hơn, Định luật Demeter nói rằng một phương thức f của lớp C chỉ nên gọi các phương thức của những thứ sau:

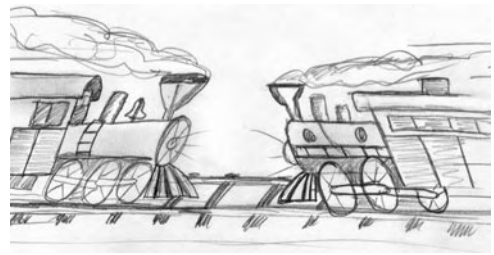
- C
- Một đối tượng được f tạo ra
- Một đối tượng được truyền làm đối số cho f
- Một đối tượng được giữ trong biến thực thể (instance variable) của C

Phương thức không nên gọi phương thức trên những đối tượng được trả về bởi bất kỳ hàm nào trong số các hàm được phép. Nói cách khác, hãy nói chuyện với bạn bè, không nói chuyện với người lạ.

Đoạn mã sau[3] có vẻ vi phạm Định luật Demeter (cùng với vài thứ khác) vì nó gọi hàm getScratchDir() trên giá trị trả về của getOptions(), rồi gọi getAbsolutePath() trên giá trị trả về của getScratchDir().

```
final String outputDir = ctxt.getOptions().getScratchDir().getAbsolutePath();
```

Tàu hỏa trật bánh



Kiểu mã này thường được gọi là tàu hỏa trật bánh vì nó trông như một đống toa tàu móc nối vào nhau. Những chuỗi gọi như thế thường bị coi là phong cách cẩu thả và nên tránh [G36]. Thường tốt nhất là tách chúng ra như sau:

```
Options opts = ctxt.getOptions();
File scratchDir = opts.getScratchDir();
final String outputDir = scratchDir.getAbsolutePath();
```

2. `http://en.wikipedia.org/wiki/Law_of_Demeter`

3. Tìm thấy đâu đó trong framework apache.

Hai đoạn mã này có vi phạm Định luật Demeter không? Chắc chắn module chứa chúng biết rằng đối tượng ctxt chứa options, options chứa một thư mục tạm (scratch directory), và thư mục đó có một đường dẫn tuyệt đối. Đó là khối lượng kiến thức quá lớn cho một hàm. Hàm gọi biết cách đi xuyên qua rất nhiều đối tượng khác nhau.

Việc này có vi phạm Demeter hay không phụ thuộc vào việc ctxt, Options và ScratchDir là đối tượng hay cấu trúc dữ liệu. Nếu chúng là đối tượng thì cấu trúc bên trong của chúng phải được che giấu chứ không phơi bày, nên việc biết về bên trong của chúng là vi phạm rõ ràng Định luật Demeter. Ngược lại, nếu ctxt, Options và ScratchDir chỉ là cấu trúc dữ liệu không có hành vi, thì đương nhiên chúng phơi bày cấu trúc bên trong, và Demeter không áp dụng.

Việc dùng các hàm accessor làm vấn đề rối lên. Nếu đoạn mã được viết như sau, có lẽ chúng ta đã không đặt câu hỏi về vi phạm Demeter.

```
final String outputDir = ctxt.options.scratchDir.absolutePath;
```

Vấn đề này sẽ bớt rối hơn nhiều nếu cấu trúc dữ liệu chỉ đơn giản có biến công khai và không có hàm, còn đối tượng có biến riêng tư và hàm công khai. Tuy nhiên, có những framework và chuẩn (ví dụ “beans”) đòi hỏi ngay cả cấu trúc dữ liệu đơn giản cũng phải có accessor và mutator.

Cấu trúc lai

Sự nhầm lẫn này đôi khi dẫn đến những cấu trúc lai đáng tiếc, nửa đối tượng nửa cấu trúc dữ liệu. Chúng có những hàm làm việc có ý nghĩa, đồng thời có hoặc biến công khai, hoặc accessor và mutator công khai mà trên thực tế biến các biến riêng tư thành công khai, dụ các hàm bên ngoài khác dùng những biến đó theo cách một chương trình thủ tục dùng cấu trúc dữ liệu.[4]

Những cấu trúc lai như vậy khiến việc thêm hàm mới khó, mà thêm cấu trúc dữ liệu mới cũng khó. Chúng là cái tệ nhất của cả hai thế giới. Hãy tránh tạo ra chúng. Chúng cho thấy một thiết kế lộn xộn mà tác giả không chắc, hoặc tệ hơn là không biết, mình cần được bảo vệ khỏi hàm hay khỏi kiểu.

Che giấu cấu trúc

Nếu ctxt, options và scratchDir là những đối tượng có hành vi thật thì sao? Khi đó, vì đối tượng phải che giấu cấu trúc bên trong, chúng ta không nên có khả năng đi xuyên qua chúng. Vậy làm sao lấy được đường dẫn tuyệt đối của thư mục tạm?

```
ctxt.getAbsolutePathOfScratchDirectoryOption();
```

hoặc

4. Đôi khi người ta gọi đây là Feature Envy, theo [Refactoring].

```
ctx.getScratchDirectoryOption().getAbsolutePath()
```

Lựa chọn đầu có thể dẫn đến bùng nổ phương thức trong đối tượng ctxt. Lựa chọn thứ hai giả định rằng getScratchDirectoryOption() trả về một cấu trúc dữ liệu, không phải một đối tượng. Cả hai lựa chọn đều không ổn.

Nếu ctxt là một đối tượng, ta nên bảo nó làm điều gì đó; không nên hỏi nó về bên trong của nó. Vậy tại sao ta lại muốn đường dẫn tuyệt đối của thư mục tạm? Ta định làm gì với nó? Hãy xét đoạn mã này, nằm (cách đó nhiều dòng) trong cùng module:

```
String outFile = outputDir + "/" + className.replace('.', '/') + ".class";
FileOutputStream fout = new FileOutputStream(outFile);
BufferedOutputStream bos = new BufferedOutputStream(fout);
```

Việc trộn lẫn các mức chi tiết khác nhau [G34][G6] hơi đáng lo. Dấu chấm, dấu gạch chéo, phần mở rộng tệp và đối tượng File không nên bị trộn với nhau một cách cẩu thả như vậy, rồi trộn luôn vào đoạn mã bao quanh. Bỏ qua điều đó, ta thấy ý định của việc lấy đường dẫn tuyệt đối của thư mục tạm là tạo một tệp tạm với tên cho trước.

Vậy nếu ta bảo đối tượng ctxt làm việc này thì sao?

```
BufferedOutputStream bos = ctxt.createScratchFileStream(classFileName);
```

Đó có vẻ là việc hợp lý để một đối tượng làm! Cách này cho phép ctxt che giấu bên trong và ngăn hàm hiện tại phải vi phạm Định luật Demeter bằng cách đi xuyên qua những đối tượng mà nó không nên biết.

Đối tượng truyền dữ liệu

Hình thái tiêu biểu nhất của cấu trúc dữ liệu là một lớp có biến công khai và không có hàm. Đôi khi người ta gọi nó là đối tượng truyền dữ liệu, hay DTO. DTO là cấu trúc rất hữu ích, nhất là khi giao tiếp với cơ sở dữ liệu hoặc phân tích thông điệp từ socket, v.v. Chúng thường trở thành chặng đầu tiên trong chuỗi các chặng chuyển đổi, biến dữ liệu thô trong cơ sở dữ liệu thành đối tượng trong mã ứng dụng.

Phổ biến hơn một chút là dạng "bean" trong Đoạn mã 6-7. Bean có các biến riêng tư được điều khiển qua getter và setter. Kiểu đóng gói nửa vời của bean có vẻ làm một số người theo chủ nghĩa OO thuần túy thấy dễ chịu hơn, nhưng thường không mang lại lợi ích nào khác.

Đoạn mã 6-7
address.java

```
public class Address {
  private String street;
  private String streetExtra;
  private String city;
  private String state;
```

```
    private String zip;

    public Address(String street, String streetExtra,
                   String city, String state, String zip) {
      this.street = street;
      this.streetExtra = streetExtra;
      this.city = city;
      this.state = state;
      this.zip = zip;
    }
    public String getStreet() {
      return street;
    }

    public String getStreetExtra() {
      return streetExtra;
    }
    public String getCity() {
      return city;
    }
    public String getState() {
      return state;
    }

    public String getZip() {
      return zip;
    }
  }
```

Active Record

Active Record là dạng đặc biệt của DTO. Chúng là cấu trúc dữ liệu với biến công khai (hoặc truy cập qua bean); nhưng chúng thường có các phương thức điều hướng như save và find. Thông thường, Active Record là bản dịch trực tiếp từ bảng cơ sở dữ liệu hoặc nguồn dữ liệu khác.

Đáng tiếc là ta thường thấy các nhà phát triển cố đối xử với những cấu trúc dữ liệu này như thể chúng là đối tượng, bằng cách đặt các phương thức quy tắc nghiệp vụ vào trong chúng. Điều này gượng gạo vì nó tạo ra một cấu trúc lai giữa cấu trúc dữ liệu và đối tượng.

Tất nhiên, giải pháp là đối xử với Active Record như một cấu trúc dữ liệu, và tạo những đối tượng riêng chứa các quy tắc nghiệp vụ, che giấu dữ liệu bên trong của chúng (mà có lẽ chỉ là các thực thể của Active Record).

Kết luận

Đối tượng phơi bày hành vi và che giấu dữ liệu. Điều này giúp dễ thêm những loại đối tượng mới mà không phải thay đổi hành vi hiện có. Nó cũng khiến việc thêm hành vi mới vào các đối tượng hiện có trở nên khó. Cấu trúc dữ liệu phơi bày dữ liệu và không có hành vi đáng kể. Điều này giúp dễ thêm hành vi mới vào các cấu trúc dữ liệu hiện có, nhưng khiến việc thêm cấu trúc dữ liệu mới vào các hàm hiện có trở nên khó.

Trong bất kỳ hệ thống nào, có lúc ta muốn sự linh hoạt để thêm kiểu dữ liệu mới, nên ở phần đó của hệ thống ta thích đối tượng hơn. Có lúc khác ta muốn sự linh hoạt để thêm hành vi mới, nên ở phần đó ta thích kiểu dữ liệu và thủ tục hơn. Những nhà phát triển phần mềm giỏi hiểu những vấn đề này mà không thành kiến và chọn cách tiếp cận tốt nhất cho công việc trước mắt.

Tài liệu tham khảo

[Refactoring]: Refactoring: Improving the Design of Existing Code, Martin Fowler et al., Addison-Wesley, 1999.

7

Xử lý lỗi

Tác giả: Michael Feathers



Có vẻ lạ khi một cuốn sách về mã sạch lại có một mục bàn về xử lý lỗi. Xử lý lỗi chỉ là một trong những việc ai cũng phải làm khi lập trình. Đầu vào có thể bất thường và thiết bị có thể hỏng. Nói ngắn gọn, mọi thứ đều có thể sai, và khi sai, chúng ta, những lập trình viên, phải chịu trách nhiệm bảo đảm mã của mình vẫn làm được việc nó cần làm.

Tuy vậy, mối liên hệ với mã sạch hẳn phải rõ. Nhiều code base bị xử lý lỗi chi phối hoàn toàn. Khi nói chi phối, tôi không có ý rằng chúng chỉ làm mỗi việc xử lý lỗi. Ý tôi là gần như không thể thấy code làm gì, vì xử lý lỗi nằm rải rác khắp nơi. Xử lý lỗi quan trọng, nhưng nếu nó che khuất logic thì là sai.

Trong chương này, tôi sẽ phác ra một số kỹ thuật và điều cần cân nhắc để bạn viết được thứ code vừa sạch vừa vững chắc, thứ code xử lý lỗi một cách thanh thoát và có phong độ.

Dùng ngoại lệ thay cho mã lỗi

Ngày trước, có nhiều ngôn ngữ không có ngoại lệ. Ở những ngôn ngữ đó, cách xử lý và báo lỗi rất hạn chế. Bạn hoặc đặt một cờ lỗi, hoặc trả về một mã lỗi để bên gọi kiểm tra. Đoạn mã 7-1 minh họa những cách làm này.

Đoạn mã 7-1
DeviceController.java

```
public class DeviceController {
    ...
    public void sendShutDown() {
        DeviceHandle handle = getHandle(DEV1);
        // Kiểm tra trạng thái của thiết bị
        if (handle != DeviceHandle.INVALID) {
            // Lưu trạng thái thiết bị vào trường record
            retrieveDeviceRecord(handle);
            // Nếu không bị tạm dừng thì tắt
            if (record.getStatus() != DEVICE_SUSPENDED) {
                pauseDevice(handle);
                clearDeviceWorkQueue(handle);
                closeDevice(handle);
            } else {
                logger.log("Device suspended. Unable to shut down");
            }
        } else {
            logger.log("Invalid handle for: " + DEV1.toString());
        }
    }
    ...
}
```

Vấn đề của những cách này là chúng làm rối bên gọi. Bên gọi phải kiểm tra lỗi ngay sau lời gọi. Đáng tiếc, quên thì rất dễ. Vì thế, tốt hơn là ném ngoại lệ khi gặp lỗi. Mã gọi sẽ sạch hơn. Logic của nó không bị xử lý lỗi che khuất.

Đoạn mã 7-2 cho thấy code sau khi chúng ta chọn ném ngoại lệ trong những phương thức có thể phát hiện lỗi.

Đoạn mã 7-2
DeviceController.java (có ngoại lệ)

```
public class DeviceController {
    ...

    public void sendShutDown() {
        try {
            tryToShutDown();
        } catch (DeviceShutDownError e) {
            logger.log(e);
        }
    }
```

Đoạn mã 7-2 (tiếp theo)
DeviceController.java (có ngoại lệ)

```
    private void tryToShutDown() throws DeviceShutDownError {
        DeviceHandle handle = getHandle(DEV1);
        DeviceRecord record = retrieveDeviceRecord(handle);

        pauseDevice(handle);
        clearDeviceWorkQueue(handle);
        closeDevice(handle);
    }

    private DeviceHandle getHandle(DeviceID id) {
        ...
        throw new DeviceShutDownError("Invalid handle for: " +
    id.toString());
        ...
    }

    ...
}
```

Hãy để ý nó sạch hơn bao nhiêu. Đây không chỉ là chuyện thẩm mỹ. Mã tốt hơn vì hai mối bận tâm từng quấn vào nhau, thuật toán tắt thiết bị và xử lý lỗi, giờ đã tách ra. Bạn có thể nhìn từng mối bận tâm và hiểu chúng độc lập.

Viết câu lệnh try-catch-finally trước

Một trong những điều thú vị nhất ở ngoại lệ là chúng định nghĩa một phạm vi trong chương trình của bạn. Khi chạy code trong phần try của câu lệnh try-catch-finally, bạn đang tuyên bố rằng việc thực thi có thể dừng ở bất kỳ điểm nào rồi tiếp tục ở catch.

Theo một nghĩa nào đó, khối try giống như giao dịch. Khối catch phải để chương trình ở trạng thái nhất quán, bất kể điều gì xảy ra trong try. Vì vậy, khi viết code có thể ném ngoại lệ, bạn nên bắt đầu bằng một câu lệnh try-catch-finally. Cách này giúp bạn định nghĩa điều người dùng code đó nên kỳ vọng, bất kể code chạy trong try gặp trục trặc gì.

Hãy xem một ví dụ. Chúng ta cần viết đoạn code truy cập một tệp và đọc vài đối tượng đã được tuần tự hóa.

Chúng ta bắt đầu bằng một bài kiểm thử đơn vị cho thấy sẽ nhận được ngoại lệ khi tệp không tồn tại:

```
@Test(expected = StorageException.class)
public void retrieveSectionShouldThrowOnInvalidFileName() {
    sectionStore.retrieveSection("invalid - file");
}
```

Bài kiểm thử dẫn chúng ta tới việc tạo đoạn khung này:

```
public List<RecordedGrip> retrieveSection(String sectionName) {
    // trả về tạm cho đến khi có cài đặt thật
    return new ArrayList<RecordedGrip>();
}
```

Bài kiểm thử thất bại vì code không ném ngoại lệ. Tiếp theo, chúng ta sửa phần cài đặt để nó thử truy cập một tệp không hợp lệ. Thao tác này ném ngoại lệ:

```
public List<RecordedGrip> retrieveSection(String sectionName) {
    try {
        FileInputStream stream = new FileInputStream(sectionName)
    } catch (Exception e) {
        throw new StorageException("retrieval error", e);
    }
    return new ArrayList<RecordedGrip>();
}
```

Giờ bài kiểm thử đã qua vì chúng ta đã bắt ngoại lệ. Đến đây, ta có thể tái cấu trúc. Ta có thể thu hẹp kiểu ngoại lệ được bắt cho khớp với kiểu thật sự được ném ra từ hàm dựng FileInputStream: FileNotFoundException.

```
public List<RecordedGrip> retrieveSection(String sectionName) {
    try {
        FileInputStream stream = new FileInputStream(sectionName);
        stream.close();
    } catch (FileNotFoundException e) {
        throw new StorageException("retrieval error", e);
    }
    return new ArrayList<RecordedGrip>();
}
```

Giờ đã định nghĩa phạm vi bằng cấu trúc try-catch, ta có thể dùng TDD để xây phần logic còn lại. Logic đó sẽ được thêm vào giữa lúc tạo FileInputStream và lúc close, và có thể giả định rằng không có gì sai.

Hãy thử viết những bài kiểm thử buộc ngoại lệ phải xảy ra, rồi thêm hành vi vào phần xử lý để làm hài lòng các bài kiểm thử đó. Việc này khiến bạn xây dựng phạm vi giao dịch của khối try trước, và giúp bạn giữ được tính giao dịch của phạm vi ấy.

Dùng ngoại lệ không kiểm tra

Cuộc tranh luận đã kết thúc. Nhiều năm qua, giới lập trình Java tranh cãi về lợi ích và cái giá của ngoại lệ có kiểm tra (checked exception). Khi mới xuất hiện trong phiên bản đầu tiên của Java, chúng có vẻ là một ý hay. Chữ ký của mỗi phương thức sẽ liệt kê mọi ngoại lệ mà phương thức có thể chuyển cho bên gọi. Hơn nữa, những ngoại lệ này là một phần trong kiểu của phương thức. Code của bạn đơn giản là không biên dịch được nếu chữ ký không khớp với những gì code có thể làm.

Vào thời đó, chúng tôi cho rằng ngoại lệ có kiểm tra là một ý hay; và đúng, chúng có thể mang lại vài lợi ích. Tuy vậy, giờ đã rõ là chúng không cần thiết để tạo ra phần mềm vững chắc. C# không có ngoại lệ có kiểm tra, và dù đã cố gắng hết sức, C++ cũng vậy. Python và Ruby cũng không. Thế mà vẫn có thể viết phần mềm vững chắc bằng tất cả những ngôn ngữ này. Vì thế, chúng ta phải quyết định, một cách thật sự, xem ngoại lệ có kiểm tra có đáng với cái giá của chúng không.

Cái giá nào? Cái giá của ngoại lệ có kiểm tra là sự vi phạm Nguyên tắc Open/Closed[1]. Nếu bạn ném một ngoại lệ có kiểm tra từ một phương thức trong code và khối catch nằm ở ba tầng phía trên, bạn phải khai báo ngoại lệ đó trong chữ ký của mọi phương thức nằm giữa bạn và catch. Nghĩa là một thay đổi ở tầng thấp của phần mềm có thể buộc nhiều tầng cao hơn phải đổi chữ ký. Những module bị đổi phải build lại và triển khai lại, dù chẳng có gì chúng quan tâm bị thay đổi.

Hãy xét thứ bậc lời gọi trong một hệ thống lớn. Các hàm ở trên cùng gọi các hàm bên dưới, những hàm này lại gọi tiếp các hàm bên dưới nữa, cứ thế đến vô tận. Giả sử một hàm ở tầng thấp nhất bị sửa theo cách buộc nó phải ném ngoại lệ. Nếu ngoại lệ đó là loại có kiểm tra, chữ ký hàm phải thêm mệnh đề throws. Nhưng điều này có nghĩa mọi hàm gọi hàm vừa sửa cũng phải sửa, hoặc để bắt ngoại lệ mới, hoặc thêm mệnh đề throws thích hợp vào chữ ký. Cứ thế đến vô tận. Kết quả ròng là một chuỗi thay đổi lan từ tầng thấp nhất lên tầng cao nhất! Tính đóng gói bị phá vỡ vì mọi hàm trên đường đi của một lần ném đều phải biết chi tiết của ngoại lệ tầng thấp đó. Mục đích của ngoại lệ là cho phép bạn xử lý lỗi từ xa, nên thật đáng tiếc khi ngoại lệ có kiểm tra lại phá vỡ tính đóng gói theo cách này.

Ngoại lệ có kiểm tra đôi khi hữu ích nếu bạn viết một thư viện quan trọng: bạn buộc phải bắt chúng. Nhưng trong phát triển ứng dụng thông thường, cái giá về sự phụ thuộc lớn hơn lợi ích.

Cung cấp ngữ cảnh qua ngoại lệ

Mỗi ngoại lệ bạn ném ra nên cung cấp đủ ngữ cảnh để xác định nguồn và vị trí của lỗi. Trong Java, bạn có thể lấy stack trace từ bất kỳ ngoại lệ nào; tuy vậy, stack trace không cho bạn biết ý định của thao tác đã thất bại.

Hãy tạo những thông báo lỗi giàu thông tin và chuyển chúng đi cùng ngoại lệ. Nêu rõ thao tác đã thất bại và kiểu thất bại. Nếu ứng dụng có ghi log, hãy chuyển đủ thông tin để có thể ghi log lỗi trong khối catch.

---

1. [Martin].

Định nghĩa lớp ngoại lệ theo nhu cầu của bên gọi

Có nhiều cách phân loại lỗi. Có thể phân loại theo nguồn: lỗi đến từ thành phần này hay thành phần khác? Hoặc theo kiểu: đó là hỏng thiết bị, hỏng mạng, hay lỗi lập trình? Tuy vậy, khi định nghĩa các lớp ngoại lệ trong một ứng dụng, mối bận tâm quan trọng nhất phải là chúng được bắt như thế nào.

Hãy xem một ví dụ về phân loại ngoại lệ kém. Đây là một câu lệnh try-catch-finally cho lời gọi tới thư viện của bên thứ ba. Nó bao quát mọi ngoại lệ mà các lời gọi có thể ném ra:

```
ACMEPort port = new ACMEPort(12);

try {
    port.open();
} catch (DeviceResponseException e) {
    reportPortError(e);
    logger.log("Device response exception", e);
} catch (ATM1212UnlockedException e) {
    reportPortError(e);
    logger.log("Unlock exception", e);
} catch (GMXError e) {
    reportPortError(e);
    logger.log("Device response exception");
} finally {
    …
}
```

Câu lệnh đó có rất nhiều trùng lặp, và điều này không có gì đáng ngạc nhiên. Trong hầu hết tình huống xử lý ngoại lệ, việc ta làm tương đối giống nhau bất kể nguyên nhân thật sự là gì. Ta phải ghi nhận một lỗi và bảo đảm mình có thể đi tiếp.

Trong trường hợp này, vì biết việc mình làm gần như giống nhau bất kể ngoại lệ nào, ta có thể đơn giản hóa code đáng kể bằng cách bọc API đang gọi và bảo đảm nó trả về một kiểu ngoại lệ chung:

```
LocalPort port = new LocalPort(12);
try {
    port.open();
} catch (PortDeviceFailure e) {
    reportError(e);
    logger.log(e.getMessage(), e);
} finally {
    …
}
```

Lớp LocalPort của chúng ta chỉ là một lớp bọc đơn giản, bắt và chuyển dịch các ngoại lệ do lớp ACMEPort ném ra:

```
public class LocalPort {
    private ACMEPort innerPort;

    public LocalPort(int portNumber) {
```

```
            innerPort = new ACMEPort(portNumber);
        }

        public void open() {
            try {
                innerPort.open();
            } catch (DeviceResponseException e) {
                throw new PortDeviceFailure(e);
            } catch (ATM1212UnlockedException e) {
                throw new PortDeviceFailure(e);
            } catch (GMXError e) {
                throw new PortDeviceFailure(e);
            }
        }
        …
    }
```

Những lớp bọc như lớp ta định nghĩa cho ACMEPort có thể rất hữu ích. Thật ra, bọc API của bên thứ ba là một cách làm tốt nhất. Khi bọc một API bên thứ ba, bạn giảm thiểu sự phụ thuộc vào nó: sau này bạn có thể chuyển sang thư viện khác mà không phải trả giá nhiều. Việc bọc cũng giúp bạn dễ giả lập các lời gọi bên thứ ba khi kiểm thử code của mình.

Một lợi ích cuối cùng của việc bọc là bạn không bị trói vào những lựa chọn thiết kế API của một nhà cung cấp cụ thể. Bạn có thể định nghĩa một API mà mình thấy thoải mái. Trong ví dụ trên, chúng ta định nghĩa một kiểu ngoại lệ duy nhất cho lỗi thiết bị cổng và thấy rằng mình có thể viết code sạch hơn nhiều.

Thường thì một lớp ngoại lệ duy nhất là đủ cho một vùng code cụ thể. Thông tin gửi kèm ngoại lệ có thể phân biệt các lỗi. Chỉ dùng các lớp khác nhau khi có lúc bạn muốn bắt một ngoại lệ và để ngoại lệ kia đi qua.

Định nghĩa luồng bình thường



Nếu làm theo lời khuyên ở các mục trên, bạn sẽ tách được khá rõ logic nghiệp vụ và phần xử lý lỗi. Phần lớn code của bạn sẽ bắt đầu giống một thuật toán sạch, không trang trí. Tuy vậy, quá trình này đẩy việc phát hiện lỗi ra rìa chương trình. Bạn bọc các API bên ngoài để có thể ném ngoại lệ của riêng mình, và định nghĩa một bộ xử lý phía trên code để

giải quyết mọi phép tính bị hủy giữa chừng. Hầu hết thời gian, đây là cách làm hay, nhưng có những lúc bạn có thể không muốn hủy.

Hãy xem một ví dụ. Đây là đoạn code khá vụng về để tính tổng chi phí trong một ứng dụng xuất hóa đơn:

```
try {
    MealExpenses expenses = expenseReportDAO.getMeals(employee.getID());
    m_total += expenses.getTotal();
} catch(MealExpensesNotFound e) {
    m_total += getMealPerDiem();
}
```

Trong nghiệp vụ này, nếu bữa ăn được tính vào chi phí thì chúng trở thành một phần của tổng. Nếu không, nhân viên nhận một khoản phụ cấp ăn trưa cho ngày hôm đó. Ngoại lệ làm rối logic. Chẳng phải tốt hơn sao nếu ta không phải xử lý trường hợp đặc biệt? Nếu không phải xử lý, code sẽ trông đơn giản hơn nhiều. Nó sẽ như thế này:

```
MealExpenses expenses = expenseReportDAO.getMeals(employee.getID());
m_total += expenses.getTotal();
```

Liệu ta có thể làm code đơn giản đến thế không? Hóa ra là có. Ta có thể sửa ExpenseReportDAO để nó luôn trả về một đối tượng MealExpense. Nếu không có chi phí bữa ăn nào, nó trả về một đối tượng MealExpense mà tổng của nó là khoản phụ cấp:

```
public class PerDiemMealExpenses implements MealExpenses {
    public int getTotal() {
        // trả về mức phụ cấp mặc định
    }
}
```

Đây được gọi là SPECIAL CASE PATTERN [Fowler]. Bạn tạo một lớp hoặc cấu hình một đối tượng để nó xử lý trường hợp đặc biệt thay bạn. Khi làm vậy, code phía client không phải xử lý hành vi ngoại lệ. Hành vi đó được đóng gói trong đối tượng trường hợp đặc biệt.

Đừng trả về null

Tôi nghĩ mọi cuộc thảo luận về xử lý lỗi đều nên nhắc đến những việc ta làm để rước lỗi vào mình. Đứng đầu danh sách là trả về null. Tôi không thể đếm nổi số ứng dụng mình từng thấy mà gần như dòng nào cũng có một phép kiểm tra null. Đây là một đoạn code ví dụ:

```
public void registerItem(Item item) {
    if (item != null) {
        ItemRegistry registry = peristentStore.getItemRegistry();
        if (registry != null) {
            Item existing = registry.getItem(item.getID());
            if (existing.getBillingPeriod().hasRetailOwner()) {
                existing.register(item);
```

```
            }
        }
    }
}
```

Nếu bạn làm việc trong một code base có kiểu code như thế này, nó có thể trông không đến nỗi tệ với bạn, nhưng nó tệ thật! Khi trả về null, thật ra ta đang tự tạo việc cho mình và đẩy rắc rối về phía bên gọi. Chỉ cần một phép kiểm tra null bị thiếu là đủ khiến ứng dụng quay cuồng mất kiểm soát.

Bạn có để ý rằng dòng thứ hai của câu lệnh if lồng nhau đó không có phép kiểm tra null không? Điều gì sẽ xảy ra lúc chạy nếu persistentStore là null? Chúng ta sẽ nhận một NullPointerException lúc chạy, và hoặc có ai đó bắt NullPointerException ở tầng cao nhất, hoặc không. Kiểu nào cũng tệ. Chính xác thì bạn nên làm gì khi nhận một NullPointerException ném ra từ sâu trong ứng dụng của mình?

Thật dễ nói rằng vấn đề của đoạn code trên là thiếu một phép kiểm tra null, nhưng thật ra vấn đề là nó có quá nhiều phép kiểm tra. Nếu bạn thấy muốn trả về null từ một phương thức, hãy cân nhắc ném ngoại lệ hoặc trả về một đối tượng SPECIAL CASE thay vào đó. Nếu bạn đang gọi một phương thức trả về null từ API của bên thứ ba, hãy cân nhắc bọc phương thức đó bằng một phương thức khác, phương thức này hoặc ném ngoại lệ, hoặc trả về một đối tượng trường hợp đặc biệt.

Trong nhiều trường hợp, đối tượng trường hợp đặc biệt là một cách khắc phục dễ dàng. Hãy tưởng tượng bạn có đoạn code như thế này:

```
List<Employee> employees = getEmployees();
if (employees != null) {
    for(Employee e : employees) {
        totalPay += e.getPay();
    }
}
```

Hiện tại, getEmployees có thể trả về null, nhưng nó có buộc phải thế không? Nếu sửa getEmployee để nó trả về một danh sách rỗng, ta có thể dọn gọn code:

```
List<Employee> employees = getEmployees();
for(Employee e : employees) {
    totalPay += e.getPay();
}
```

May thay, Java có Collections.emptyList(), và nó trả về một danh sách bất biến định sẵn mà ta có thể dùng cho mục đích này:

```
public List<Employee> getEmployees() {
    if( .. there are no employees .. )
        return Collections.emptyList();
}
```

Nếu viết code theo cách này, bạn sẽ giảm thiểu khả năng gặp NullPointerException và code

của bạn sẽ sạch hơn.

Đừng truyền null

Trả về null từ phương thức là xấu, nhưng truyền null vào phương thức còn tệ hơn. Trừ khi bạn làm việc với một API mong đợi bạn truyền null, bạn nên tránh truyền null trong code của mình bất cứ khi nào có thể.

Hãy xem một ví dụ để thấy vì sao. Đây là một phương thức đơn giản tính một đại lượng cho hai điểm:

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        return (p2.x - p1.x) * 1.5;
    }
    …
}
```

Điều gì xảy ra khi có người truyền null làm đối số?

```
calculator.xProjection(null, new Point(12, 13));
```

Tất nhiên, chúng ta sẽ nhận một NullPointerException.

Làm sao sửa được? Ta có thể tạo một kiểu ngoại lệ mới và ném nó:

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        if (p1 == null || p2 == null) {
            throw InvalidArgumentException(
                "Invalid argument for MetricsCalculator.xProjection");
        }
        return (p2.x - p1.x) * 1.5;
    }
}
```

Cách này có tốt hơn không? Nó có thể tốt hơn một chút so với null pointer exception, nhưng hãy nhớ, ta phải định nghĩa một bộ xử lý cho InvalidArgumentException. Bộ xử lý đó nên làm gì? Có hành động nào hợp lý để làm không?

Còn một lựa chọn khác. Ta có thể dùng một tập các phép khẳng định:

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        assert p1 != null : "p1 should not be null";
        assert p2 != null : "p2 should not be null";
        return (p2.x - p1.x) * 1.5;
    }
}
```

Đó là tài liệu tốt, nhưng nó không giải quyết vấn đề. Nếu có người truyền null, ta vẫn gặp lỗi lúc chạy.

Trong hầu hết ngôn ngữ lập trình, không có cách nào tốt để xử lý một giá trị null do bên gọi vô tình truyền vào. Vì thế, cách làm hợp lý là mặc định cấm truyền null. Khi làm vậy, bạn có thể viết code với hiểu biết rằng một null trong danh sách đối số là dấu hiệu của trục trặc, và rốt cuộc mắc ít lỗi bất cẩn hơn nhiều.

Kết luận

Mã sạch phải đọc được, nhưng cũng phải vững chắc. Hai mục tiêu này không xung đột. Ta có thể viết mã sạch vững chắc nếu xem xử lý lỗi là một mối bận tâm riêng, một thứ có thể nhìn độc lập với logic chính. Làm được đến đâu, ta có thể lý luận về nó độc lập đến đó, và tiến xa đến đó trong khả năng bảo trì code.

Tài liệu tham khảo

[Martin]: Agile Software Development: Principles, Patterns, and Practices, Robert C. Martin, Prentice Hall, 2002.

8

Ranh giới

Tác giả: James Grenning



Chúng ta hiếm khi kiểm soát được toàn bộ phần mềm trong hệ thống của mình. Đôi khi ta mua các gói phần mềm của bên thứ ba hoặc dùng phần mềm mã nguồn mở. Những lúc khác, ta phụ thuộc vào các nhóm trong chính công ty mình để tạo ra những thành phần hoặc hệ thống con. Bằng cách nào đó, ta phải tích hợp sạch sẽ đoạn mã xa lạ này với phần mềm

của mình. Trong chương này, chúng ta sẽ xem xét các cách làm và kỹ thuật giúp giữ cho ranh giới của phần mềm luôn sạch.

Dùng mã của bên thứ ba

Giữa bên cung cấp giao diện và bên sử dụng giao diện luôn có một sự căng thẳng tự nhiên. Nhà cung cấp các gói phần mềm và framework của bên thứ ba muốn sản phẩm có phạm vi ứng dụng rộng để chạy được trong nhiều môi trường và thu hút đông đảo người dùng. Ngược lại, người dùng muốn một giao diện tập trung vào nhu cầu riêng của mình. Sự căng thẳng này có thể gây ra vấn đề ở ranh giới hệ thống.

Hãy lấy `java.util.Map` làm ví dụ. Như bạn thấy khi xem Hình 8-1, `Map` có giao diện rất rộng với nhiều khả năng. Sức mạnh và tính linh hoạt đó chắc chắn hữu ích, nhưng cũng có thể trở thành gánh nặng. Chẳng hạn, ứng dụng của ta có thể tạo một `Map` rồi truyền nó đi khắp nơi. Ta có thể muốn không bên nhận nào xóa thứ gì trong `Map`. Nhưng ngay đầu danh sách đã có phương thức `clear()`. Bất kỳ ai dùng `Map` cũng có thể xóa sạch nó. Hoặc có thể quy ước thiết kế của ta chỉ cho phép lưu một số kiểu đối tượng nhất định trong `Map`, nhưng `Map` không giới hạn đáng tin cậy kiểu đối tượng được đặt vào đó. Người dùng nào quyết tâm cũng có thể thêm phần tử thuộc bất kỳ kiểu nào vào bất kỳ `Map` nào.

- `clear() void – Map`
- `containsKey(Object key) boolean – Map`
- `containsValue(Object value) boolean – Map`
- `entrySet() Set – Map`
- `equals(Object o) boolean – Map`
- `get(Object key) Object – Map`
- `getClass() Class<? extends Object> – Object`
- `hashCode() int – Map`
- `isEmpty() boolean – Map`
- `keySet() Set – Map`
- `notify() void – Object`
- `notifyAll() void – Object`
- `put(Object key, Object value) Object – Map`
- `putAll(Map t) void – Map`
- `remove(Object key) Object – Map`
- `size() int – Map`
- `toString() String – Object`
- `values() Collection – Map`
- `wait() void – Object`
- `wait(long timeout) void – Object`
- `wait(long timeout, int nanos) void – Object`

Hình 8-1
Các phương thức của `Map`

Nếu ứng dụng cần một `Map` chứa `Sensor`, bạn có thể thấy các cảm biến được thiết lập như sau:

```
Map sensors = new HashMap();
```

Sau đó, khi một phần khác của mã cần truy cập cảm biến, ta thấy đoạn mã này:

```
Sensor s = (Sensor)sensors.get(sensorId );
```

Ta không chỉ thấy cách làm này một lần, mà lặp đi lặp lại khắp nơi trong mã. Phía gọi đoạn mã phải chịu trách nhiệm lấy một `Object` từ `Map` rồi ép kiểu sang kiểu phù hợp. Cách này chạy được, nhưng không phải mã sạch. Nó cũng chưa kể câu chuyện rõ ràng như có thể. Có thể cải thiện đáng kể khả năng đọc của đoạn mã bằng cách dùng kiểu tổng quát, như dưới đây:

```
Map<Sensor> sensors = new HashMap<Sensor>();
...
Sensor s = sensors.get(sensorId );
```

Tuy nhiên, cách này không giải quyết được vấn đề `Map<Sensor>` cung cấp nhiều khả năng hơn điều ta cần hoặc muốn.

Truyền thoải mái một đối tượng `Map<Sensor>` khắp hệ thống đồng nghĩa với việc sẽ có nhiều nơi cần sửa nếu giao diện của `Map` thay đổi. Bạn có thể nghĩ thay đổi như vậy khó xảy ra, nhưng hãy nhớ rằng giao diện này đã thay đổi khi Java 5 bổ sung hỗ trợ kiểu tổng quát. Thật vậy, chúng tôi từng thấy những hệ thống không thể dùng kiểu tổng quát vì khối lượng thay đổi khổng lồ cần thiết để khắc phục việc dùng `Map` tràn lan.

Cách dùng `Map` sạch hơn có thể như sau. Người dùng `Sensors` sẽ chẳng bận tâm có dùng kiểu tổng quát hay không. Lựa chọn đó đã trở thành, và vốn dĩ luôn nên là, chi tiết triển khai.

```
public class Sensors {
  private Map sensors = new HashMap();

    public Sensor getById(String id) {
      return (Sensor) sensors.get(id);
    }

    // lược bớt
}
```

Giao diện ở ranh giới (`Map`) được che giấu. Nó có thể phát triển mà hầu như không ảnh hưởng đến phần còn lại của ứng dụng. Việc dùng kiểu tổng quát không còn là vấn đề lớn, vì lớp `Sensors` xử lý việc ép kiểu và quản lý kiểu.

Giao diện này cũng được điều chỉnh và giới hạn để đáp ứng nhu cầu của ứng dụng.

Nhờ đó, mã dễ hiểu hơn và khó bị dùng sai hơn. Lớp `Sensors` có thể thực thi các quy tắc thiết kế và nghiệp vụ.

Chúng tôi không khuyên bạn đóng gói mọi lần dùng `Map` theo cách này. Thay vào đó, chúng tôi khuyên bạn đừng truyền `Map` (hay bất kỳ giao diện ranh giới nào khác) khắp hệ thống. Nếu dùng một giao diện ranh giới như `Map`, hãy giữ nó bên trong lớp, hoặc một nhóm lớp liên quan gần gũi, nơi nó được sử dụng. Tránh trả nó về từ API công khai hoặc nhận nó làm đối số.

Khám phá và tìm hiểu ranh giới

Mã của bên thứ ba giúp chúng ta bổ sung nhiều chức năng hơn trong thời gian ngắn hơn. Ta nên bắt đầu từ đâu khi muốn dùng một gói phần mềm của bên thứ ba? Kiểm thử mã của bên thứ ba không phải việc của ta, nhưng viết kiểm thử cho đoạn mã bên thứ ba mà ta sử dụng có thể mang lại lợi ích cho chính ta.

Giả sử ta chưa rõ cách dùng thư viện của bên thứ ba. Ta có thể dành một hoặc hai ngày, thậm chí lâu hơn, để đọc tài liệu và quyết định cách sử dụng. Sau đó, ta có thể viết mã dùng thư viện ấy rồi xem nó có hoạt động như mình nghĩ không. Hẳn ta sẽ không ngạc nhiên nếu bị sa lầy vào những phiên gỡ lỗi kéo dài, cố tìm hiểu xem lỗi mình gặp nằm trong mã của ta hay của họ.

Tìm hiểu mã của bên thứ ba rất khó. Tích hợp mã đó cũng khó. Làm cả hai việc cùng lúc khó gấp đôi. Nếu thử một cách tiếp cận khác thì sao? Thay vì thử nghiệm trong mã sản phẩm, ta có thể viết một số kiểm thử để khám phá xem mình hiểu mã của bên thứ ba đến đâu. Jim Newkirk gọi những kiểm thử như vậy là kiểm thử tìm hiểu.[1]

Trong các kiểm thử tìm hiểu, ta gọi API của bên thứ ba theo cách dự định sử dụng trong ứng dụng. Về bản chất, ta đang làm các thí nghiệm có kiểm soát để kiểm tra hiểu biết của mình về API đó. Các kiểm thử tập trung vào những gì ta cần từ API.

Tìm hiểu log4j

Giả sử ta muốn dùng gói `apache log4j` thay cho trình ghi nhật ký tự xây dựng. Ta tải nó về và mở trang tài liệu giới thiệu. Chỉ đọc sơ qua, ta viết ca kiểm thử đầu tiên, mong nó ghi chữ “hello” ra bảng điều khiển.

```
@Test
public void testLogCreate() {
  Logger logger = Logger.getLogger("MyLogger");
  logger.info("hello");
}
```

Khi chạy, trình ghi nhật ký báo lỗi cho biết ta cần một thứ gọi là `Appender`. Đọc thêm một chút, ta tìm thấy `ConsoleAppender`. Vậy là ta tạo một `ConsoleAppender` để xem mình

1. [BeckTDD], tr. 136–137.

đã khám phá được bí quyết ghi ra bảng điều khiển chưa.

```
@Test
public void testLogAddAppender() {
  Logger logger = Logger.getLogger("MyLogger");
  ConsoleAppender appender = new ConsoleAppender();
  logger.addAppender(appender);
  logger.info("hello");
}
```

Lần này ta phát hiện `Appender` không có luồng xuất. Lạ thật, cứ tưởng nó phải có chứ. Tìm trên Google một chút, ta thử đoạn sau:

```
@Test
public void testLogAddAppender() {
  Logger logger = Logger.getLogger("MyLogger");
  logger.removeAllAppenders();
  logger.addAppender(new ConsoleAppender(
       new PatternLayout("%p %t %m%n"),
       ConsoleAppender.SYSTEM_OUT));
  logger.info("hello");
}
```

Cách này có tác dụng. Một thông báo ghi nhật ký có chữ "hello" xuất hiện trên bảng điều khiển! Thật kỳ lạ khi ta phải nói rõ với `ConsoleAppender` rằng nó ghi ra bảng điều khiển.

Điều thú vị là khi bỏ đối số `ConsoleAppender.SystemOut`, ta vẫn thấy "hello" được in ra. Nhưng nếu bỏ `PatternLayout`, nó lại phàn nàn không có luồng xuất. Hành vi này thật kỳ quặc.

Xem tài liệu kỹ hơn, ta thấy hàm khởi tạo mặc định của `ConsoleAppender` ở trạng thái "chưa cấu hình", điều này chẳng rõ ràng hay hữu ích chút nào. Cảm giác như đây là một lỗi, hoặc ít nhất là sự thiếu nhất quán trong `log4j`.

Tìm kiếm, đọc và kiểm thử thêm một chút, cuối cùng ta có được Đoạn mã 8-1. Ta đã khám phá ra khá nhiều về cách `log4j` hoạt động, đồng thời mã hóa hiểu biết đó thành một tập hợp kiểm thử đơn vị đơn giản.

Đoạn mã 8-1
LogTest.java

```
public class LogTest {
    private Logger logger;

     @Before
     public void initialize() {
         logger = Logger.getLogger("logger");
         logger.removeAllAppenders();
         Logger.getRootLogger().removeAllAppenders();
     }
     @Test
     public void basicLogger() {
         BasicConfigurator.configure();
```

```
            logger.info("basicLogger");
        }

        @Test
        public void addAppenderWithStream() {
         logger.addAppender(new ConsoleAppender(
              new PatternLayout("%p %t %m%n"),
              ConsoleAppender.SYSTEM_OUT));
            logger.info("addAppenderWithStream");
        }
```

Đoạn mã 8-1 (tiếp theo)
LogTest.java

```
        @Test
        public void addAppenderWithoutStream() {
         logger.addAppender(new ConsoleAppender(
              new PatternLayout("%p %t %m%n")));
            logger.info("addAppenderWithoutStream");
        }
    }
```

Giờ ta đã biết cách khởi tạo một trình ghi nhật ký đơn giản ra bảng điều khiển. Ta có thể đóng gói hiểu biết ấy vào lớp ghi nhật ký của riêng mình, để phần còn lại của ứng dụng được cách ly khỏi giao diện ranh giới `log4j`.

Kiểm thử tìm hiểu có lợi hơn miễn phí

Rốt cuộc, các kiểm thử tìm hiểu chẳng tốn gì. Dù sao ta cũng phải tìm hiểu API, và viết những kiểm thử đó là cách dễ dàng, tách biệt để tích lũy hiểu biết. Đó là những thí nghiệm chính xác giúp ta hiểu rõ hơn.

Kiểm thử tìm hiểu không chỉ miễn phí mà còn đem lại lợi ích. Khi gói phần mềm bên thứ ba có phiên bản mới, ta chạy lại các kiểm thử để xem có khác biệt về hành vi không.

Kiểm thử tìm hiểu xác minh rằng các gói phần mềm của bên thứ ba mà ta dùng hoạt động đúng như mong đợi. Sau khi tích hợp, không có gì bảo đảm mã của bên thứ ba sẽ luôn tương thích với nhu cầu của ta. Các tác giả ban đầu sẽ chịu áp lực phải thay đổi mã để đáp ứng nhu cầu mới của chính họ. Họ sẽ sửa lỗi và bổ sung khả năng mới. Mỗi phiên bản phát hành đều mang theo rủi ro mới. Nếu gói phần mềm thay đổi theo cách không tương thích với các kiểm thử của ta, ta sẽ phát hiện ngay.

Dù có cần phần hiểu biết mà kiểm thử tìm hiểu mang lại hay không, một ranh giới sạch vẫn cần được hỗ trợ bởi một tập hợp kiểm thử hướng ra ngoài, thực thi giao diện theo đúng cách mã sản phẩm sử dụng. Không có những kiểm thử ranh giới này để giúp việc chuyển đổi dễ dàng, ta có thể bị cám dỗ ở lại với phiên bản cũ lâu hơn mức nên làm.

Dùng mã chưa tồn tại

Có một kiểu ranh giới khác, ngăn cách điều đã biết với điều chưa biết. Trong mã thường có những chỗ mà kiến thức của ta dường như đột ngột chấm dứt. Đôi khi ta không thể biết được điều gì nằm bên kia ranh giới, ít nhất là vào lúc này. Đôi khi ta chọn không tìm hiểu xa hơn ranh giới.

Nhiều năm trước, tôi tham gia một nhóm phát triển phần mềm cho hệ thống liên lạc vô tuyến. Có một hệ thống con gọi là "Transmitter" mà chúng tôi biết rất ít, còn những người phụ trách hệ thống ấy chưa xác định được giao diện của nó. Chúng tôi không muốn bị chặn lại, nên bắt đầu công việc ở xa phần mã chưa biết đó.

Chúng tôi khá hiểu rõ thế giới của mình kết thúc ở đâu và thế giới mới bắt đầu từ đâu. Trong lúc làm việc, đôi khi chúng tôi chạm đến ranh giới ấy. Sương mù và mây mù của sự thiếu hiểu biết che khuất tầm nhìn về phía bên kia, nhưng công việc giúp chúng tôi nhận ra mình muốn giao diện ranh giới có dạng nào. Chúng tôi muốn có thể nói với bộ phát như sau:

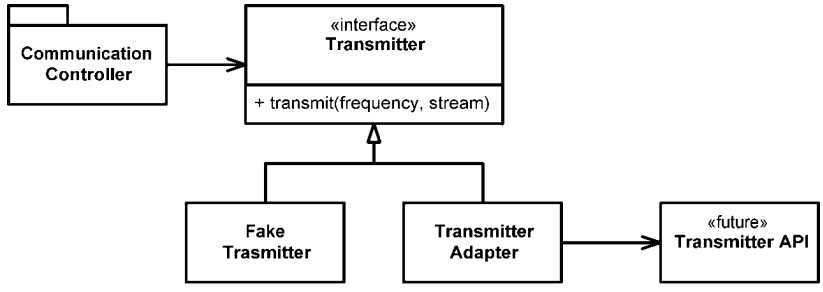
> *Bật bộ phát ở tần số đã cung cấp và phát ra tín hiệu tương tự biểu diễn dữ liệu đến từ luồng này.*

Chúng tôi không biết phải thực hiện việc đó thế nào vì API vẫn chưa được thiết kế. Vì vậy, chúng tôi quyết định sẽ xử lý chi tiết sau.

Để không bị chặn lại, chúng tôi tự định nghĩa giao diện của mình. Chúng tôi đặt cho nó một cái tên nghe bắt tai, chẳng hạn `Transmitter`. Chúng tôi tạo một phương thức tên `transmit`, nhận tần số và luồng dữ liệu. Đó là giao diện mà chúng tôi mong mình có.

Một điều hay khi viết giao diện mình mong muốn là ta kiểm soát được nó. Nhờ vậy, mã phía gọi dễ đọc hơn và tập trung hơn vào điều nó đang cố thực hiện.

Trong Hình 8-2, bạn có thể thấy chúng tôi đã cách ly các lớp `CommunicationsController` khỏi API của bộ phát, vốn nằm ngoài quyền kiểm soát và chưa được định nghĩa. Dùng một giao diện đặc thù cho ứng dụng do mình tạo ra, chúng tôi giữ cho mã `CommunicationsController` sạch sẽ và giàu tính biểu đạt. Khi API của bộ phát được định nghĩa, chúng tôi viết `TransmitterAdapter` để bắc cầu khoảng cách. `ADAPTER`[2] đóng gói việc tương tác với API và cung cấp một nơi duy nhất cần thay đổi khi API phát triển.



2. Xem mẫu Adapter trong [GOF].

Hình 8-2
Dự đoán giao diện bộ phát

Thiết kế này còn tạo cho ta một điểm nối[3] rất thuận tiện trong mã để kiểm thử. Dùng một `FakeTransmitter` phù hợp, ta có thể kiểm thử các lớp `CommunicationsController`. Ta cũng có thể viết kiểm thử ranh giới sau khi có `TransmitterAPI` để bảo đảm ta dùng API đúng cách.

Ranh giới sạch

Những điều thú vị xảy ra ở ranh giới. Thay đổi là một trong số đó. Thiết kế phần mềm tốt thích ứng với thay đổi mà không đòi hỏi đầu tư khổng lồ và làm lại nhiều. Khi dùng mã nằm ngoài quyền kiểm soát, ta phải đặc biệt cẩn trọng để bảo vệ khoản đầu tư của mình và bảo đảm việc thay đổi sau này không quá tốn kém.

Mã ở ranh giới cần được tách biệt rõ ràng và có các kiểm thử xác định kỳ vọng. Ta nên tránh để quá nhiều mã của mình biết đến những chi tiết riêng của bên thứ ba. Tốt hơn hết là phụ thuộc vào thứ mình kiểm soát thay vì thứ mình không kiểm soát, kẻo cuối cùng nó lại kiểm soát ta.

Ta quản lý ranh giới với bên thứ ba bằng cách chỉ để thật ít chỗ trong mã tham chiếu đến chúng. Ta có thể bọc chúng như đã làm với `Map`, hoặc dùng một `ADAPTER` để chuyển đổi từ giao diện hoàn hảo của mình sang giao diện được cung cấp. Dù chọn cách nào, mã của ta cũng diễn đạt rõ hơn, khuyến khích cách dùng nhất quán bên trong khi qua ranh giới, và có ít điểm cần bảo trì hơn khi mã của bên thứ ba thay đổi.

Tài liệu tham khảo

[BeckTDD]: Test Driven Development, Kent Beck, Addison-Wesley, 2003.

[GOF]: Design Patterns: Elements of Reusable Object Oriented Software, Gamma et al., Addison-Wesley, 1996.

[WELC]: Working Effectively with Legacy Code, Addison-Wesley, 2004.

3. Xem thêm về điểm nối trong [WELC].

9

Kiểm thử đơn vị



Nghề của chúng ta đã tiến một chặng đường dài trong mười năm qua. Năm 1997, chưa ai nghe nói đến Phát triển hướng kiểm thử (Test Driven Development). Với đại đa số chúng ta, kiểm thử đơn vị chỉ là những đoạn mã ngắn dùng một lần, được viết ra để bảo đảm chương trình "chạy được". Chúng ta cặm cụi viết các lớp và phương thức, rồi chế ra vài đoạn mã tùy tiện để kiểm tra chúng. Thường thì đó là một chương trình điều khiển đơn giản cho phép chúng ta tương tác thủ công với chương trình mình vừa viết.

Tôi nhớ đã viết một chương trình C++ cho hệ thống nhúng thời gian thực vào giữa thập niên 1990. Chương trình là một bộ định giờ đơn giản với khai báo sau:

```
void Timer::ScheduleCommand(Command* theCommand, int milliseconds)
```

Ý tưởng rất đơn giản: phương thức execute của Command sẽ chạy trong một luồng mới sau số mili giây đã chỉ định. Vấn đề là kiểm thử nó thế nào.

Tôi chắp vá một chương trình điều khiển đơn giản để lắng nghe bàn phím. Mỗi lần có ký tự được gõ, chương trình sẽ lên lịch để gõ lại chính ký tự đó sau năm giây. Rồi tôi gõ một giai điệu có nhịp điệu trên bàn phím và chờ giai điệu phát lại trên màn hình sau năm giây.

"Em . . . muốn . . . một-cô-gái . . . giống . . . cô-gái-đã-cưới . . . người . . . cha . . . già . . . thân-yêu."

Tôi thực sự vừa gõ phím "." vừa hát giai điệu đó, rồi lại hát lần nữa khi các dấu chấm hiện lên trên màn hình.

Đó là bài kiểm thử của tôi! Sau khi thấy nó chạy được và trình diễn cho đồng nghiệp, tôi vứt đoạn mã kiểm thử đi.

Như tôi đã nói, nghề của chúng ta đã tiến một chặng đường dài. Ngày nay, tôi sẽ viết một bài kiểm thử bảo đảm mọi ngóc ngách của đoạn mã hoạt động đúng như mong đợi. Tôi sẽ cô lập mã của mình khỏi hệ điều hành thay vì chỉ gọi các hàm định thời chuẩn. Tôi sẽ giả lập các hàm định thời đó để kiểm soát thời gian tuyệt đối. Tôi sẽ lên lịch các lệnh đặt những cờ boolean, rồi cho thời gian tiến lên từng bước, theo dõi các cờ và xác nhận chúng chuyển từ false sang true đúng lúc tôi đưa thời gian đến giá trị tương ứng.

Sau khi có một bộ kiểm thử chạy qua, tôi sẽ bảo đảm bất cứ ai cần làm việc với đoạn mã đều có thể chạy chúng dễ dàng. Tôi sẽ bảo đảm các bài kiểm thử và mã nguồn được đưa vào cùng một gói mã nguồn.

Đúng vậy, chúng ta đã tiến xa, nhưng vẫn còn cả chặng đường phía trước. Các phong trào Agile và TDD đã khuyến khích nhiều lập trình viên viết kiểm thử đơn vị tự động, và mỗi ngày lại có thêm người tham gia. Nhưng trong cuộc chạy đua điên cuồng để đưa kiểm thử vào quy trình làm việc, nhiều lập trình viên đã bỏ qua một số điểm tinh tế và quan trọng hơn trong việc viết kiểm thử tốt.

Ba quy luật của TDD

Đến nay, ai cũng biết TDD yêu cầu chúng ta viết kiểm thử đơn vị trước khi viết mã sản phẩm. Nhưng quy tắc đó chỉ là phần nổi của tảng băng. Hãy xem xét ba quy luật sau:[1]

Quy luật thứ nhất Bạn không được viết mã sản phẩm cho đến khi đã viết một kiểm thử đơn vị thất bại.

Quy luật thứ hai Bạn không được viết thêm kiểm thử đơn vị nào ngoài mức vừa đủ để nó thất bại, và không biên dịch được cũng được xem là thất bại.

Quy luật thứ ba Bạn không được viết thêm mã sản phẩm nào ngoài mức vừa đủ để kiểm thử đang thất bại hiện tại chạy qua.

`http://doi.ieeecomputersociety.org/10.1109/MS.2007.85`

Ba quy luật này buộc chúng ta vào một chu kỳ có lẽ chỉ dài khoảng ba mươi giây. Kiểm

1. Professionalism and Test-Driven Development, Robert C. Martin, Object Mentor, IEEE Software, May/June 2007 (Vol. 24, No. 3) pp. 32–36

thử và mã sản phẩm được viết cùng nhau, trong đó phần kiểm thử chỉ đi trước mã sản phẩm vài giây.

Nếu làm theo cách này, mỗi ngày chúng ta sẽ viết hàng chục bài kiểm thử, mỗi tháng hàng trăm bài, mỗi năm hàng nghìn bài. Khi đó, các bài kiểm thử sẽ bao phủ gần như toàn bộ mã sản phẩm. Số lượng khổng lồ của chúng, có thể sánh ngang với chính mã sản phẩm, sẽ tạo ra một vấn đề quản lý đáng ngại.

Giữ cho kiểm thử sạch

Vài năm trước, tôi được mời hướng dẫn một nhóm đã chủ động quyết định rằng mã kiểm thử không cần được duy trì theo cùng tiêu chuẩn chất lượng như mã sản phẩm. Họ cho phép nhau phá vỡ các quy tắc trong kiểm thử đơn vị. "Nhanh và bẩn" là phương châm của họ. Họ không cần đặt tên biến cho tử tế, các hàm kiểm thử không cần ngắn gọn và mô tả rõ ràng. Mã kiểm thử không cần được thiết kế tốt hay chia tách cẩn thận. Chỉ cần mã kiểm thử chạy được và bao phủ mã sản phẩm thì thế là đủ.

Một số bạn đọc đến đây có thể đồng tình với quyết định đó. Có lẽ ngày trước bạn từng viết những bài kiểm thử giống như bài tôi viết cho lớp Timer. Từ kiểu kiểm thử dùng một lần ấy đến một bộ kiểm thử đơn vị tự động là một bước tiến rất lớn. Vì thế, giống như nhóm tôi hướng dẫn, bạn có thể nghĩ rằng kiểm thử bẩn vẫn hơn là không có kiểm thử.

Điều nhóm này không nhận ra là có kiểm thử bẩn cũng chẳng khác gì, thậm chí còn tệ hơn, không có kiểm thử. Vấn đề nằm ở chỗ kiểm thử phải thay đổi khi mã sản phẩm phát triển. Kiểm thử càng bẩn thì càng khó sửa. Mã kiểm thử càng rối rắm, bạn càng có khả năng phải mất nhiều thời gian nhồi thêm kiểm thử mới vào bộ kiểm thử hơn là viết mã sản phẩm mới. Khi bạn sửa mã sản phẩm, các bài kiểm thử cũ bắt đầu thất bại, còn mớ hỗn độn trong mã kiểm thử khiến việc làm cho chúng chạy qua trở nên khó khăn. Dần dà, người ta xem các bài kiểm thử như một gánh nặng ngày càng lớn.

Qua từng bản phát hành, chi phí bảo trì bộ kiểm thử của nhóm tôi ngày càng tăng. Cuối cùng, đó trở thành điều khiến các nhà phát triển phàn nàn nhiều nhất. Khi quản lý hỏi tại sao ước lượng của họ cứ lớn mãi, các nhà phát triển đổ lỗi cho kiểm thử. Rốt cuộc, họ buộc phải loại bỏ hoàn toàn bộ kiểm thử.

Nhưng khi không còn bộ kiểm thử, họ mất khả năng bảo đảm các thay đổi trong kho mã hoạt động đúng như mong đợi. Họ không thể bảo đảm thay đổi ở một phần của hệ thống không làm hỏng những phần khác. Vì vậy, tỷ lệ lỗi bắt đầu tăng. Khi số lỗi ngoài ý muốn ngày càng nhiều, họ bắt đầu sợ thay đổi. Họ ngừng làm sạch mã sản phẩm vì sợ thay đổi sẽ gây hại nhiều hơn lợi. Mã sản phẩm bắt đầu mục ruỗng. Cuối cùng, họ chỉ còn lại một hệ thống không có kiểm thử, mã sản phẩm rối rắm đầy lỗi, khách hàng thất vọng và cảm giác rằng nỗ lực kiểm thử đã phụ lòng họ.

Theo một nghĩa nào đó, họ nói đúng. Nỗ lực kiểm thử đã phụ lòng họ. Nhưng chính quyết định cho phép kiểm thử trở nên bừa bộn đã gieo mầm cho thất bại đó. Nếu giữ cho kiểm thử sạch, nỗ lực kiểm thử của họ đã không thất bại. Tôi có thể nói điều này khá chắc chắn vì tôi từng tham gia và hướng dẫn nhiều nhóm thành công với kiểm thử đơn vị sạch.

Bài học rất đơn giản: mã kiểm thử cũng quan trọng như mã sản phẩm. Nó không phải

công dân hạng hai. Nó cần được suy nghĩ, thiết kế và chăm chút. Phải giữ cho nó sạch như mã sản phẩm.

Kiểm thử tạo điều kiện cho mọi "-ility"

Nếu không giữ cho kiểm thử sạch, bạn sẽ đánh mất chúng. Và không có chúng, bạn sẽ mất đi chính thứ giúp mã sản phẩm linh hoạt. Đúng vậy, bạn đọc không nhầm đâu. Chính kiểm thử đơn vị giúp mã của chúng ta linh hoạt, dễ bảo trì và tái sử dụng. Lý do rất đơn giản. Khi có kiểm thử, bạn không sợ thay đổi mã! Không có kiểm thử, mọi thay đổi đều có thể gây ra lỗi. Kiến trúc của bạn linh hoạt đến đâu, thiết kế được chia tách gọn gàng đến mức nào cũng vậy, nếu không có kiểm thử, bạn sẽ ngần ngại thay đổi vì sợ đưa vào những lỗi không được phát hiện.

Nhưng khi có kiểm thử, nỗi sợ đó gần như biến mất. Độ bao phủ kiểm thử càng cao, bạn càng ít sợ. Bạn có thể gần như yên tâm thay đổi cả những đoạn mã có kiến trúc chưa xuất sắc, thiết kế rối rắm và khó hiểu. Thậm chí, bạn có thể cải thiện kiến trúc và thiết kế mà không sợ hãi!

Vì vậy, có một bộ kiểm thử đơn vị tự động bao phủ mã sản phẩm là chìa khóa để giữ thiết kế và kiến trúc sạch nhất có thể. Kiểm thử tạo điều kiện cho mọi phẩm chất kết thúc bằng "-ility", bởi kiểm thử tạo điều kiện cho thay đổi.

Nếu kiểm thử bẩn, khả năng thay đổi mã của bạn sẽ bị cản trở, và bạn dần mất khả năng cải thiện cấu trúc mã. Kiểm thử càng bẩn, mã của bạn càng bẩn. Cuối cùng, bạn mất kiểm thử và mã của bạn mục ruỗng.

Kiểm thử sạch

Điều gì tạo nên một bài kiểm thử sạch? Ba thứ: khả năng đọc, khả năng đọc và khả năng đọc. Khả năng đọc trong kiểm thử đơn vị có lẽ còn quan trọng hơn trong mã sản phẩm. Điều gì làm cho kiểm thử dễ đọc? Cũng chính những yếu tố làm mọi đoạn mã dễ đọc: rõ ràng, đơn giản và diễn đạt cô đọng. Trong một bài kiểm thử, bạn muốn nói được nhiều nhất với số biểu thức ít nhất.

Hãy xem đoạn mã từ FitNesse trong Đoạn mã 9-1. Ba bài kiểm thử này khó hiểu và chắc chắn có thể cải thiện. Trước hết, các lời gọi addPage và assertSubString lặp lại gây trùng lặp mã [G5] rất nhiều. Quan trọng hơn, đoạn mã chứa đầy những chi tiết làm giảm sức diễn đạt của bài kiểm thử.

Đoạn mã 9-1
SerializedPageResponderTest.java

```
public void testGetPageHieratchyAsXml() throws Exception
{
  crawler.addPage(root, PathParser.parse("PageOne"));
  crawler.addPage(root, PathParser.parse("PageOne.ChildOne"));
  crawler.addPage(root, PathParser.parse("PageTwo"));
    request.setResource("root");
```

```
      request.addInput("type", "pages");
      Responder responder = new SerializedPageResponder();
      SimpleResponse response =
       (SimpleResponse) responder.makeResponse(
          new FitNesseContext(root), request);
      String xml = response.getContent();

      assertEquals("text/xml", response.getContentType());
      assertSubString("<name>PageOne</name>", xml);
      assertSubString("<name>PageTwo</name>", xml);
      assertSubString("<name>ChildOne</name>", xml);
  }

  public void testGetPageHieratchyAsXmlDoesntContainSymbolicLinks()
  throws Exception
  {
     WikiPage pageOne = crawler.addPage(root, PathParser.parse("PageOne"));
     crawler.addPage(root, PathParser.parse("PageOne.ChildOne"));
     crawler.addPage(root, PathParser.parse("PageTwo"));

      PageData data = pageOne.getData();
      WikiPageProperties properties = data.getProperties();
      WikiPageProperty symLinks = properties.set(SymbolicPage.PROPERTY_NAME);
      symLinks.set("SymPage", "PageTwo");
      pageOne.commit(data);

      request.setResource("root");
      request.addInput("type", "pages");
      Responder responder = new SerializedPageResponder();
      SimpleResponse response =
       (SimpleResponse) responder.makeResponse(
          new FitNesseContext(root), request);
      String xml = response.getContent();
      assertEquals("text/xml", response.getContentType());
      assertSubString("<name>PageOne</name>", xml);
      assertSubString("<name>PageTwo</name>", xml);
      assertSubString("<name>ChildOne</name>", xml);
      assertNotSubString("SymPage", xml);
  }

  public void testGetDataAsHtml() throws Exception
  {
    crawler.addPage(root, PathParser.parse("TestPageOne"), "test page");
      request.setResource("TestPageOne");
      request.addInput("type", "data");
```

Đoạn mã 9-1 (tiếp)
SerializedPageResponderTest.java

```
       Responder responder = new SerializedPageResponder();
       SimpleResponse response =
         (SimpleResponse) responder.makeResponse(
            new FitNesseContext(root), request);
       String xml = response.getContent();
       assertEquals("text/xml", response.getContentType());
       assertSubString("test page", xml);
```

```
        assertSubString("<Test", xml);
    }
```

Chẳng hạn, hãy xem các lời gọi PathParser. Chúng chuyển chuỗi thành các đối tượng PagePath mà crawler sử dụng. Việc chuyển đổi này hoàn toàn không liên quan đến bài kiểm thử đang xét, chỉ làm lu mờ ý định. Các chi tiết quanh việc tạo responder, lấy response và ép kiểu cũng chỉ là nhiễu. Rồi còn cách dựng URL của yêu cầu rất vụng về từ một resource và một đối số nữa. (Tôi đã góp phần viết đoạn mã này, nên cứ thoải mái phê bình thẳng tay.)

Rốt cuộc, đoạn mã này không được thiết kế để người khác đọc. Người đọc tội nghiệp bị nhấn chìm trong một loạt chi tiết phải hiểu trước khi các bài kiểm thử trở nên có ý nghĩa.

Giờ hãy xem các bài kiểm thử đã cải thiện trong Đoạn mã 9-2. Chúng làm đúng những việc như cũ, nhưng đã được tái cấu trúc thành dạng gọn gàng và dễ giải thích hơn nhiều.

Đoạn mã 9-2
SerializedPageResponderTest.java (đã tái cấu trúc)

```
public void testGetPageHierarchyAsXml() throws Exception {
  makePages("PageOne", "PageOne.ChildOne", "PageTwo");

    submitRequest("root", "type:pages");
    assertResponseIsXML();
    assertResponseContains(
       "<name>PageOne</name>", "<name>PageTwo</name>",
    "<name>ChildOne</name>"
    );
}

public void testSymbolicLinksAreNotInXmlPageHierarchy() throws Exception {
  WikiPage page = makePage("PageOne");
  makePages("PageOne.ChildOne", "PageTwo");

    addLinkTo(page, "PageTwo", "SymPage");

    submitRequest("root", "type:pages");
    assertResponseIsXML();
    assertResponseContains(
       "<name>PageOne</name>", "<name>PageTwo</name>",
    "<name>ChildOne</name>"
    );
    assertResponseDoesNotContain("SymPage");
}
```

Đoạn mã 9-2 (tiếp)
SerializedPageResponderTest.java (đã tái cấu trúc)

```
public void testGetDataAsXml() throws Exception {
  makePageWithContent("TestPageOne", "test page");

    submitRequest("TestPageOne", "type:data");
```

```
        assertResponseIsXML();
        assertResponseContains("test page", "<Test");
    }
```

Mẫu BUILD-OPERATE-CHECK[2] hiện lên rõ ràng qua cấu trúc của các bài kiểm thử này. Mỗi bài được chia thành ba phần rành mạch. Phần đầu dựng dữ liệu kiểm thử, phần thứ hai thao tác trên dữ liệu đó, phần thứ ba kiểm tra thao tác có tạo ra kết quả mong đợi hay không.

Hãy để ý rằng phần lớn những chi tiết gây phiền đã được loại bỏ. Các bài kiểm thử đi thẳng vào vấn đề, chỉ dùng những kiểu dữ liệu và hàm thực sự cần thiết. Ai đọc chúng cũng có thể nhanh chóng hiểu chúng làm gì, không bị chi tiết đánh lạc hướng hay nhấn chìm.

Ngôn ngữ kiểm thử chuyên biệt theo miền

Các bài kiểm thử trong Đoạn mã 9-2 minh họa kỹ thuật xây dựng một ngôn ngữ chuyên biệt theo miền cho kiểm thử. Thay vì dùng các API mà lập trình viên sử dụng để thao tác với hệ thống, chúng ta xây dựng một tập hợp hàm và tiện ích sử dụng những API đó, giúp viết kiểm thử thuận tiện hơn và đọc dễ hơn. Các hàm và tiện ích này trở thành một API chuyên biệt được các bài kiểm thử sử dụng. Chúng tạo thành ngôn ngữ kiểm thử để lập trình viên tự viết kiểm thử và giúp những người đọc chúng về sau.

API kiểm thử này không được thiết kế từ đầu. Nó phát triển dần qua quá trình tái cấu trúc liên tục mã kiểm thử đã bị nhiễm quá nhiều chi tiết che khuất ý định. Cũng như bạn vừa thấy tôi tái cấu trúc Đoạn mã 9-1 thành Đoạn mã 9-2, những nhà phát triển có kỷ luật cũng sẽ tái cấu trúc mã kiểm thử thành các dạng ngắn gọn và giàu sức diễn đạt hơn.

Hai tiêu chuẩn

Theo một nghĩa nào đó, nhóm tôi nhắc đến ở đầu chương đã đúng. Mã bên trong API kiểm thử có thể áp dụng một bộ tiêu chuẩn kỹ thuật khác với mã sản phẩm. Nó vẫn phải đơn giản, ngắn gọn và giàu sức diễn đạt, nhưng không nhất thiết hiệu quả như mã sản phẩm. Dù sao thì nó chạy trong môi trường kiểm thử chứ không phải môi trường sản phẩm, mà hai môi trường ấy có những yêu cầu rất khác nhau.

Hãy xem bài kiểm thử trong Đoạn mã 9-3. Tôi viết nó như một phần của hệ thống điều khiển môi trường mà mình đang làm thử nghiệm nguyên mẫu. Không cần biết chi tiết, bạn vẫn có thể thấy bài kiểm thử này xác nhận rằng báo động nhiệt độ thấp, bộ sưởi và quạt gió đều bật khi nhiệt độ "quá lạnh".

Đoạn mã 9-3
EnvironmentControllerTest.java

```
@Test
  public void turnOnLoTempAlarmAtThreashold() throws Exception {
    hw.setTemp(WAY_TOO_COLD);
```

2. http://fitnesse.org/FitNesse.AcceptanceTestPatterns

```
    controller.tic();
    assertTrue(hw.heaterState());
    assertTrue(hw.blowerState());
    assertFalse(hw.coolerState());
    assertFalse(hw.hiTempAlarm());
    assertTrue(hw.loTempAlarm());
  }
```

Dĩ nhiên ở đây có nhiều chi tiết. Chẳng hạn, hàm tic dùng để làm gì? Thực ra, tôi muốn bạn đừng bận tâm đến điều đó khi đọc bài kiểm thử này. Tôi chỉ muốn bạn xem trạng thái cuối cùng của hệ thống có nhất quán với mức nhiệt "quá lạnh" hay không.

Khi đọc bài kiểm thử, hãy để ý mắt bạn phải liên tục qua lại giữa tên trạng thái được kiểm tra và ý nghĩa của phép kiểm tra đó. Bạn thấy heaterState, rồi mắt lướt sang trái đến assertTrue. Bạn thấy coolerState, mắt lại phải lần sang trái đến assertFalse. Việc này vừa tẻ nhạt vừa dễ sai, khiến bài kiểm thử khó đọc.

Tôi cải thiện đáng kể khả năng đọc của bài kiểm thử này bằng cách chuyển nó thành Đoạn mã 9-4.

Đoạn mã 9-4
EnvironmentControllerTest.java (đã tái cấu trúc)

```
@Test
  public void turnOnLoTempAlarmAtThreshold() throws Exception {
    wayTooCold();
    assertEquals("HBchL", hw.getState());
  }
```

Dĩ nhiên, tôi đã che giấu chi tiết của hàm tic bằng cách tạo ra hàm wayTooCold. Nhưng điều đáng chú ý là chuỗi kỳ lạ trong assertEquals. Chữ hoa nghĩa là "bật", chữ thường nghĩa là "tắt", và các chữ cái luôn theo thứ tự sau: {heater, blower, cooler, hi-temp-alarm, lo-temp-alarm}.

Dù cách này gần như vi phạm quy tắc tránh buộc người đọc tự ánh xạ trong đầu,[3] nhưng trong trường hợp này có vẻ phù hợp. Khi đã biết ý nghĩa, bạn sẽ lướt mắt qua chuỗi đó và nhanh chóng hiểu kết quả. Đọc bài kiểm thử gần như trở thành niềm vui. Hãy xem Đoạn mã 9-5 để thấy các bài kiểm thử này dễ hiểu đến mức nào.

Đoạn mã 9-5
EnvironmentControllerTest.java (tuyển chọn lớn hơn)

```
@Test
  public void turnOnCoolerAndBlowerIfTooHot() throws Exception {
    tooHot();
    assertEquals("hBChl", hw.getState());
  }

  @Test
```

3. "Tránh ánh xạ trong đầu" ở trang 25.

```
  public void turnOnHeaterAndBlowerIfTooCold() throws Exception {
    tooCold();
    assertEquals("HBchl", hw.getState());
  }

  @Test
  public void turnOnHiTempAlarmAtThreshold() throws Exception {
    wayTooHot();
    assertEquals("hBCHl", hw.getState());
  }
  @Test
  public void turnOnLoTempAlarmAtThreshold() throws Exception {
    wayTooCold();
    assertEquals("HBchL", hw.getState());
  }
```

Hàm getState được trình bày trong Đoạn mã 9-6. Hãy chú ý, đây không phải đoạn mã hiệu quả lắm. Muốn nó hiệu quả hơn, có lẽ tôi nên dùng StringBuffer.

Đoạn mã 9-6
MockControlHardware.java

```
public String getState() {
    String state = "";
    state += heater ? "H" : "h";
    state += blower ? "B" : "b";
    state += cooler ? "C" : "c";
    state += hiTempAlarm ? "H" : "h";
    state += loTempAlarm ? "L" : "l";
    return state;
  }
```

StringBuffer hơi xấu. Ngay cả trong mã sản phẩm, nếu chi phí nhỏ, tôi cũng tránh dùng chúng; và bạn có thể lập luận rằng chi phí của đoạn mã trong Đoạn mã 9-6 rất thấp. Tuy nhiên, ứng dụng này rõ ràng là một hệ thống nhúng thời gian thực, và nhiều khả năng tài nguyên máy tính cũng như bộ nhớ rất hạn chế. Còn môi trường kiểm thử thì có lẽ chẳng bị giới hạn gì đáng kể.

Đó là bản chất của hai tiêu chuẩn. Có những điều bạn có thể chẳng bao giờ làm trong môi trường sản phẩm nhưng lại hoàn toàn ổn trong môi trường kiểm thử. Thường thì chúng liên quan đến hiệu quả sử dụng bộ nhớ hoặc CPU. Nhưng chúng không bao giờ liên quan đến độ sạch của mã.

Mỗi bài kiểm thử một assert

Có một trường phái tư tưởng[4] cho rằng mỗi hàm kiểm thử trong một bài kiểm thử JUnit chỉ nên có đúng một câu lệnh assert. Quy tắc này nghe có vẻ hà khắc, nhưng ưu điểm của nó

4. Xem bài đăng trên blog của Dave Astel: `http://www.artima.com/weblogs/viewpost.jsp?thread=35578`

thể hiện trong Đoạn mã 9-5. Mỗi bài kiểm thử đi đến một kết luận duy nhất, nhanh chóng và dễ hiểu.

Còn Đoạn mã 9-2 thì sao? Có vẻ khó mà dễ dàng gộp xác nhận đầu ra là XML với xác nhận đầu ra chứa những chuỗi con nhất định. Tuy nhiên, chúng ta có thể tách bài kiểm thử thành hai bài riêng, mỗi bài có một xác nhận cụ thể, như trong Đoạn mã 9-7.

Đoạn mã 9-7
SerializedPageResponderTest.java (một assert)

```
public void testGetPageHierarchyAsXml() throws Exception {
    givenPages("PageOne", "PageOne.ChildOne", "PageTwo");

      whenRequestIsIssued("root", "type:pages");
      thenResponseShouldBeXML();
  }

  public void testGetPageHierarchyHasRightTags() throws Exception {
    givenPages("PageOne", "PageOne.ChildOne", "PageTwo");
      whenRequestIsIssued("root", "type:pages");

      thenResponseShouldContain(
         "<name>PageOne</name>", "<name>PageTwo</name>",
    "<name>ChildOne</name>"
      );
  }
```

Hãy để ý tôi đã đổi tên các hàm theo quy ước given-when-then phổ biến.[5] Điều này giúp đọc kiểm thử dễ hơn nữa. Đáng tiếc là việc chia tách như trên tạo ra nhiều mã trùng lặp.

Chúng ta có thể loại bỏ phần trùng lặp bằng cách dùng mẫu TEMPLATE METHOD[6], đưa các phần given/when vào lớp cơ sở và đặt phần then trong các lớp dẫn xuất khác nhau. Hoặc ta có thể tạo hẳn một lớp kiểm thử riêng, đặt phần given và when trong hàm @Before, rồi đặt phần then trong từng hàm @Test. Nhưng với một vấn đề nhỏ như vậy, cách đó có vẻ tạo ra quá nhiều cơ chế. Cuối cùng, tôi thích dùng nhiều assert trong Đoạn mã 9-2 hơn.

Tôi nghĩ quy tắc một assert là một nguyên tắc hữu ích.[7] Thường thì tôi cố gắng tạo một ngôn ngữ kiểm thử chuyên biệt theo miền hỗ trợ quy tắc đó, như trong Đoạn mã 9-5. Nhưng tôi không ngại đặt nhiều hơn một assert trong một bài kiểm thử. Có lẽ cách diễn đạt tốt nhất là nên giảm thiểu số lượng assert trong mỗi bài kiểm thử.

Mỗi bài kiểm thử một khái niệm

Có lẽ quy tắc tốt hơn là mỗi hàm kiểm thử chỉ nên kiểm tra một khái niệm. Ta không muốn những hàm kiểm thử dài, lần lượt kiểm tra hết thứ linh tinh này đến thứ linh tinh khác. Đoạn mã 9-8 là ví dụ về một bài kiểm thử như vậy. Nên tách nó thành ba bài kiểm thử độc lập vì

5. [RSpec].

6. [GOF].

7. “Bám sát mã!”

nó kiểm tra ba việc riêng biệt. Gộp tất cả vào cùng một hàm buộc người đọc phải tìm hiểu lý do từng phần có mặt và mỗi phần đang kiểm tra điều gì.

Đoạn mã 9-8

```
    /**
     * Miscellaneous tests for the addMonths() method.
     */
    public void testAddMonths() {
        SerialDate d1 = SerialDate.createInstance(31, 5, 2004);

          SerialDate d2 = SerialDate.addMonths(1, d1);
          assertEquals(30, d2.getDayOfMonth());
          assertEquals(6, d2.getMonth());
          assertEquals(2004, d2.getYYYY());
          SerialDate d3 = SerialDate.addMonths(2, d1);
          assertEquals(31, d3.getDayOfMonth());
          assertEquals(7, d3.getMonth());
          assertEquals(2004, d3.getYYYY());

          SerialDate d4 = SerialDate.addMonths(1, SerialDate.addMonths(1,
d1));
          assertEquals(30, d4.getDayOfMonth());
          assertEquals(7, d4.getMonth());
          assertEquals(2004, d4.getYYYY());
    }
```

Có lẽ ba hàm kiểm thử nên được viết như sau:

- Cho ngày cuối cùng của một tháng có 31 ngày, chẳng hạn tháng Năm:
    1. Khi cộng thêm một tháng, mà tháng mới kết thúc vào ngày 30, chẳng hạn tháng Sáu, ngày phải là 30 chứ không phải 31.
    2. Khi cộng thêm hai tháng vào ngày đó, để tháng cuối cùng có 31 ngày, ngày phải là 31.
- Cho ngày cuối cùng của một tháng có 30 ngày, chẳng hạn tháng Sáu:
    1. Khi cộng thêm một tháng để tháng mới có ngày cuối cùng là ngày 31, ngày phải là 30 chứ không phải 31.

Diễn đạt như vậy, bạn sẽ thấy có một quy tắc chung ẩn trong các bài kiểm thử linh tinh này. Khi tăng tháng, ngày không thể lớn hơn ngày cuối cùng của tháng đó. Điều này hàm ý rằng tăng tháng từ ngày 28 tháng Hai sẽ cho ra ngày 28 tháng Ba. Bài kiểm thử đó còn thiếu và sẽ rất hữu ích nếu được viết thêm.

Vì vậy, vấn đề không nằm ở chỗ mỗi phần của Đoạn mã 9-8 có nhiều assert. Vấn đề là có nhiều khái niệm được kiểm tra. Quy tắc tốt nhất có lẽ là giảm số assert cho mỗi khái niệm và chỉ kiểm tra một khái niệm trong mỗi hàm kiểm thử.

F.I.R.S.T.[8]

Kiểm thử sạch tuân theo năm quy tắc khác, tạo thành từ viết tắt trên:

Nhanh (Fast) Kiểm thử phải nhanh. Chúng phải chạy nhanh chóng. Khi kiểm thử chạy chậm, bạn sẽ không muốn chạy thường xuyên. Nếu không chạy thường xuyên, bạn sẽ không phát hiện vấn đề đủ sớm để sửa dễ dàng. Bạn sẽ không còn thoải mái làm sạch mã. Cuối cùng, mã sẽ bắt đầu mục ruỗng.

Độc lập (Independent) Các bài kiểm thử không nên phụ thuộc lẫn nhau. Một bài không nên chuẩn bị điều kiện cho bài tiếp theo. Bạn phải có thể chạy từng bài độc lập và chạy theo bất cứ thứ tự nào mình muốn. Khi các bài kiểm thử phụ thuộc lẫn nhau, bài đầu tiên thất bại sẽ gây ra hàng loạt lỗi dây chuyền ở các bài sau, khiến việc chẩn đoán khó khăn và che giấu những lỗi tiếp theo.

Có thể lặp lại (Repeatable) Kiểm thử phải có thể lặp lại trong mọi môi trường. Bạn phải chạy được chúng trong môi trường sản phẩm, môi trường QA và trên máy tính xách tay khi đang đi tàu về nhà mà không có mạng. Nếu không thể lặp lại kiểm thử trong mọi môi trường, bạn sẽ luôn có cớ để giải thích tại sao chúng thất bại. Bạn cũng sẽ thấy mình không thể chạy kiểm thử khi môi trường không sẵn sàng.

Tự xác nhận (Self-Validating) Kiểm thử phải cho kết quả boolean. Hoặc đạt, hoặc thất bại. Bạn không nên phải đọc tệp log để biết kiểm thử có đạt hay không. Bạn không nên phải tự so sánh hai tệp văn bản khác nhau để biết kết quả. Nếu kiểm thử không tự xác nhận, việc thất bại có thể trở thành chuyện chủ quan và chạy kiểm thử có thể đòi hỏi một quá trình đánh giá thủ công dài dòng.

Đúng lúc (Timely) Kiểm thử phải được viết đúng lúc. Kiểm thử đơn vị nên được viết ngay trước mã sản phẩm giúp chúng chạy qua. Nếu viết kiểm thử sau mã sản phẩm, bạn có thể thấy mã sản phẩm khó kiểm thử. Bạn có thể quyết định rằng một số mã sản phẩm quá khó kiểm thử. Bạn có thể không thiết kế mã sản phẩm sao cho có thể kiểm thử được.

Kết luận

Chúng ta mới chỉ chạm nhẹ vào chủ đề này. Thật vậy, tôi nghĩ có thể viết hẳn một cuốn sách về kiểm thử sạch. Kiểm thử quan trọng đối với sức khỏe của một dự án chẳng kém mã sản phẩm. Có lẽ chúng còn quan trọng hơn, vì kiểm thử gìn giữ và tăng cường tính linh hoạt, khả năng bảo trì và khả năng tái sử dụng của mã sản phẩm. Vì vậy, hãy luôn giữ cho kiểm thử sạch. Hãy làm cho chúng giàu sức diễn đạt và ngắn gọn. Hãy tạo ra các API kiểm thử đóng vai trò như ngôn ngữ chuyên biệt theo miền, giúp bạn viết kiểm thử.

Nếu để kiểm thử mục ruỗng, mã của bạn cũng sẽ mục ruỗng. Hãy giữ cho kiểm thử sạch.

---

8. Tài liệu đào tạo của Object Mentor.

Tài liệu tham khảo

[RSpec]: RSpec: Behavior Driven Development for Ruby Programmers, Aslak Hellesøy, David Chelimsky, Pragmatic Bookshelf, 2008.

[GOF]: Design Patterns: Elements of Reusable Object Oriented Software, Gamma et al., Addison-Wesley, 1996.

10

Lớp

với Jeff Langr



Cho đến nay, trong cuốn sách này, chúng ta tập trung vào cách viết tốt từng dòng và từng khối mã. Chúng ta đã tìm hiểu kỹ cách tổ chức các hàm cho đúng và cách chúng tương tác với nhau. Nhưng dù đã quan tâm đến khả năng diễn đạt của các câu lệnh mã và những hàm cấu thành nên chúng đến đâu, mã vẫn chưa sạch nếu chúng ta chưa chú ý đến những cấp độ tổ chức mã cao hơn. Hãy nói về các lớp sạch.

Tổ chức lớp

Theo quy ước Java tiêu chuẩn, một lớp nên bắt đầu bằng danh sách biến. Nếu có hằng tĩnh công khai thì đặt chúng lên đầu. Tiếp theo là các biến tĩnh riêng tư, rồi đến các biến thực thể riêng tư. Hiếm khi có lý do chính đáng để khai báo biến công khai.

Các hàm công khai nên đứng sau danh sách biến. Chúng tôi thường đặt các hàm tiện ích riêng tư mà một hàm công khai gọi đến ngay sau chính hàm công khai đó. Cách này tuân theo quy tắc đi từng bước và giúp chương trình đọc như một bài báo.

Đóng gói

Chúng tôi muốn giữ biến và hàm tiện ích ở phạm vi riêng tư, nhưng không quá cứng nhắc. Đôi khi cần đặt một biến hoặc hàm tiện ích ở phạm vi protected để kiểm thử có thể truy cập. Với chúng tôi, kiểm thử là ưu tiên hàng đầu. Nếu một ca kiểm thử trong cùng gói cần gọi một hàm hoặc truy cập một biến, chúng tôi sẽ đặt nó ở phạm vi protected hoặc phạm vi gói. Tuy nhiên, trước tiên chúng tôi sẽ tìm cách duy trì tính riêng tư. Nới lỏng đóng gói luôn là lựa chọn cuối cùng.

Lớp phải nhỏ!

Quy tắc đầu tiên dành cho lớp là phải nhỏ. Quy tắc thứ hai là phải nhỏ hơn nữa. Không, chúng tôi sẽ không lặp lại nguyên văn chương Hàm. Nhưng cũng như với hàm, kích thước nhỏ là quy tắc hàng đầu khi thiết kế lớp. Cũng như với hàm, câu hỏi đầu tiên luôn là: "Nhỏ đến mức nào?"

Với hàm, chúng ta đo kích thước bằng cách đếm số dòng vật lý. Với lớp, chúng ta dùng một thước đo khác: đếm số trách nhiệm.[1]

Đoạn mã 10-1 phác họa lớp SuperDashboard, với khoảng 70 phương thức công khai. Hầu hết nhà phát triển đều đồng ý rằng nó hơi quá "siêu" về kích thước. Một số người có thể gọi SuperDashboard là "lớp Thần".

Đoạn mã 10-1
Quá nhiều trách nhiệm

```
public class SuperDashboard extends JFrame implements MetaDataUser
   public String getCustomizerLanguagePath()
   public void setSystemConfigPath(String systemConfigPath)
   public String getSystemConfigDocument()
   public void setSystemConfigDocument(String systemConfigDocument)
   public boolean getGuruState()
   public boolean getNoviceState()
   public boolean getOpenSourceState()
   public void showObject(MetaObject object)
   public void showProgress(String s)
```

1. [RDD].

Đoạn mã 10-1 (tiếp)
Quá nhiều trách nhiệm

```
public boolean isMetadataDirty()
public void setIsMetadataDirty(boolean isMetadataDirty)
public Component getLastFocusedComponent()
public void setLastFocused(Component lastFocused)
public void setMouseSelectState(boolean isMouseSelected)
public boolean isMouseSelected()
public LanguageManager getLanguageManager()
public Project getProject()
public Project getFirstProject()
public Project getLastProject()
public String getNewProjectName()
public void setComponentSizes(Dimension dim)
public String getCurrentDir()
public void setCurrentDir(String newDir)
public void updateStatus(int dotPos, int markPos)
public Class[] getDataBaseClasses()
public MetadataFeeder getMetadataFeeder()
public void addProject(Project project)
public boolean setCurrentProject(Project project)
public boolean removeProject(Project project)
public MetaProjectHeader getProgramMetadata()
public void resetDashboard()
public Project loadProject(String fileName, String projectName)
public void setCanSaveMetadata(boolean canSave)
public MetaObject getSelectedObject()
public void deselectObjects()
public void setProject(Project project)
public void editorAction(String actionName, ActionEvent event)
public void setMode(int mode)
public FileManager getFileManager()
public void setFileManager(FileManager fileManager)
public ConfigManager getConfigManager()
public void setConfigManager(ConfigManager configManager)
public ClassLoader getClassLoader()
public void setClassLoader(ClassLoader classLoader)
public Properties getProps()
public String getUserHome()
public String getBaseDir()
public int getMajorVersionNumber()
public int getMinorVersionNumber()
public int getBuildNumber()
public MetaObject pasting(
   MetaObject target, MetaObject pasted, MetaProject project)
public void processMenuItems(MetaObject metaObject)
public void processMenuSeparators(MetaObject metaObject)
public void processTabPages(MetaObject metaObject)
public void processPlacement(MetaObject object)
public void processCreateLayout(MetaObject object)
public void updateDisplayLayer(MetaObject object, int layerIndex)
public void propertyEditedRepaint(MetaObject object)
public void processDeleteObject(MetaObject object)
public boolean getAttachedToDesigner()
public void processProjectChangedState(boolean hasProjectChanged)
public void processObjectNameChanged(MetaObject object)
```

```
    public void runProject()
```

Đoạn mã 10-1 (tiếp)
Quá nhiều trách nhiệm

```
    public void setAçowDragging(boolean allowDragging)
    public boolean allowDragging()
    public boolean isCustomizing()
    public void setTitle(String title)
    public IdeMenuBar getIdeMenuBar()
    public void showHelper(MetaObject metaObject, String propertyName)
    // ... many non-public methods follow ...
}
```

Nhưng nếu SuperDashboard chỉ chứa các phương thức trong Đoạn mã 10-2 thì sao?

Đoạn mã 10-2
Đủ nhỏ chưa?

```
public class SuperDashboard extends JFrame implements MetaDataUser
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

Năm phương thức đâu có nhiều, phải không? Trong trường hợp này thì có, vì dù số phương thức ít, SuperDashboard vẫn có quá nhiều trách nhiệm.

Tên lớp nên mô tả những trách nhiệm mà lớp đảm nhận. Thực ra, đặt tên có lẽ là cách đầu tiên giúp xác định kích thước lớp. Nếu không thể nghĩ ra một cái tên ngắn gọn cho lớp thì có lẽ lớp đó quá lớn. Tên lớp càng mơ hồ, khả năng nó có quá nhiều trách nhiệm càng cao. Chẳng hạn, những tên lớp có các từ chung chung như Processor, Manager hay Super thường báo hiệu một sự gộp trách nhiệm không hợp lý.

Ta cũng nên có thể mô tả ngắn gọn lớp trong khoảng 25 từ, không dùng các từ "if", "and", "or" hay "but". Ta sẽ mô tả SuperDashboard thế nào? "SuperDashboard cung cấp quyền truy cập vào thành phần gần nhất giữ tiêu điểm, đồng thời cho phép theo dõi số phiên bản và số build." Từ "and" đầu tiên là dấu hiệu SuperDashboard đang gánh quá nhiều trách nhiệm.

Nguyên tắc trách nhiệm đơn nhất

Nguyên tắc trách nhiệm đơn nhất (Single Responsibility Principle, SRP)[2] nói rằng một lớp hay mô-đun chỉ nên có một lý do để thay đổi. Nguyên tắc này vừa định nghĩa trách nhiệm,

2. Bạn có thể đọc thêm nhiều hơn về nguyên tắc này trong [PPP].

vừa đưa ra nguyên tắc kinh nghiệm về kích thước lớp. Mỗi lớp nên có một trách nhiệm, tức một lý do để thay đổi.

Lớp SuperDashboard có vẻ nhỏ trong Đoạn mã 10-2 nhưng lại có hai lý do để thay đổi. Thứ nhất, nó theo dõi thông tin phiên bản, dường như cần cập nhật mỗi khi phát hành phần mềm. Thứ hai, nó quản lý các thành phần Java Swing (nó là lớp dẫn xuất của JFrame, biểu diễn của Swing cho cửa sổ GUI cấp cao nhất). Chắc chắn ta sẽ muốn cập nhật số phiên bản nếu thay đổi mã Swing, nhưng điều ngược lại không nhất thiết đúng: ta có thể thay đổi thông tin phiên bản do những thay đổi ở phần mã khác trong hệ thống.

Cố gắng xác định trách nhiệm, tức lý do để thay đổi, thường giúp chúng ta nhận ra và tạo ra những trừu tượng tốt hơn trong mã. Ta có thể dễ dàng tách cả ba phương thức của SuperDashboard liên quan đến thông tin phiên bản vào một lớp riêng tên là Version. (Xem Đoạn mã 10-3.) Lớp Version là một cấu trúc có tiềm năng được tái sử dụng cao trong các ứng dụng khác!

Đoạn mã 10-3
Lớp có một trách nhiệm

```
public class Version {
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

SRP là một trong những khái niệm quan trọng nhất trong thiết kế hướng đối tượng. Đây cũng là một trong những khái niệm đơn giản nhất để hiểu và tuân theo. Thế nhưng, lạ thay, SRP thường là nguyên tắc thiết kế lớp bị lạm dụng nhiều nhất. Chúng tôi thường xuyên gặp những lớp làm quá nhiều việc. Vì sao?

Làm cho phần mềm hoạt động và làm cho phần mềm sạch là hai hoạt động rất khác nhau. Hầu hết chúng ta có khả năng tập trung hữu hạn, nên thường chú trọng làm cho mã chạy được hơn là tổ chức và giữ mã sạch. Điều đó hoàn toàn hợp lý. Trong hoạt động lập trình cũng như trong chương trình, duy trì sự phân tách mối quan tâm quan trọng ngang nhau.

Vấn đề là quá nhiều người trong chúng ta nghĩ rằng công việc đã xong khi chương trình chạy được. Chúng ta không chuyển sang mối quan tâm còn lại là tổ chức và sự sạch sẽ. Thay vì quay lại chia những lớp phình to thành các đơn vị tách biệt, mỗi đơn vị chỉ có một trách nhiệm, chúng ta chuyển sang vấn đề tiếp theo.

Đồng thời, nhiều nhà phát triển lo rằng có nhiều lớp nhỏ, mỗi lớp làm một việc, sẽ khiến bức tranh tổng thể khó hiểu hơn. Họ ngại phải lần từ lớp này sang lớp khác mới tìm ra cách một phần công việc lớn được thực hiện.

Tuy nhiên, một hệ thống có nhiều lớp nhỏ không có nhiều bộ phận chuyển động hơn hệ thống có vài lớp lớn. Hệ thống chỉ có vài lớp lớn cũng cần học hỏi ngần ấy điều. Vậy câu hỏi là: Bạn muốn sắp xếp công cụ vào những hộp đựng có nhiều ngăn nhỏ, mỗi ngăn chứa một bộ phận được xác định và dán nhãn rõ ràng? Hay muốn có vài ngăn rồi quăng mọi thứ vào đó?

Hệ thống đủ lớn nào cũng chứa nhiều logic và độ phức tạp. Mục tiêu hàng đầu khi quản lý độ phức tạp ấy là tổ chức sao cho nhà phát triển biết tìm gì ở đâu và tại mỗi thời điểm chỉ cần hiểu phần phức tạp bị ảnh hưởng trực tiếp. Ngược lại, hệ thống có các lớp lớn đa dụng luôn cản trở chúng ta bằng cách buộc ta lội qua hàng đống thứ chưa cần biết lúc này.

Nhắc lại các ý trên để nhấn mạnh: chúng ta muốn hệ thống gồm nhiều lớp nhỏ, không phải vài lớp lớn. Mỗi lớp nhỏ đóng gói một trách nhiệm, có một lý do để thay đổi và cộng tác với một vài lớp khác để tạo ra các hành vi mong muốn của hệ thống.

Độ gắn kết

Lớp nên có ít biến thực thể. Mỗi phương thức của lớp nên thao tác trên một hoặc nhiều biến đó. Nói chung, một phương thức thao tác càng nhiều biến thì càng gắn kết với lớp của nó. Lớp mà mọi biến đều được mọi phương thức sử dụng đạt độ gắn kết tối đa.

Nhìn chung, tạo ra những lớp gắn kết tối đa vừa không khả thi vừa không nên làm. Tuy vậy, chúng ta muốn độ gắn kết cao. Độ gắn kết cao nghĩa là các phương thức và biến của lớp phụ thuộc lẫn nhau và kết hợp thành một chỉnh thể logic.

Hãy xem cách cài đặt Stack trong Đoạn mã 10-4. Đây là một lớp có độ gắn kết rất cao. Trong ba phương thức, chỉ có size() không dùng cả hai biến.

Đoạn mã 10-4
Stack.java, một lớp gắn kết

```
public class Stack {
  private int topOfStack = 0;
  List<Integer> elements = new LinkedList<Integer>();
    public int size() {
      return topOfStack;
    }

    public void push(int element) {
      topOfStack++;
      elements.add(element);
    }

    public int pop() throws PoppedWhenEmpty {
      if (topOfStack == 0)
        throw new PoppedWhenEmpty();
      int element = elements.get(--topOfStack);
      elements.remove(topOfStack);
      return element;
    }
}
```

Chiến lược giữ hàm nhỏ và danh sách tham số ngắn đôi khi dẫn đến việc có thêm nhiều biến thực thể chỉ được một nhóm phương thức sử dụng. Khi vậy, gần như luôn có ít nhất một lớp khác đang cố tách ra khỏi lớp lớn. Bạn nên thử tách các biến và phương thức thành hai hay nhiều lớp để các lớp mới có độ gắn kết cao hơn.

Duy trì độ gắn kết tạo ra nhiều lớp nhỏ

Chỉ riêng việc chia các hàm lớn thành hàm nhỏ hơn đã làm số lớp tăng lên. Hãy xét một hàm lớn có nhiều biến được khai báo bên trong. Giả sử bạn muốn tách một phần nhỏ của hàm đó thành hàm riêng. Tuy nhiên, đoạn mã muốn tách sử dụng bốn biến được khai báo trong hàm. Bạn có phải truyền cả bốn biến ấy vào hàm mới làm đối số không?

Không hề! Nếu đưa bốn biến đó thành biến thực thể của lớp, ta có thể tách đoạn mã mà không cần truyền biến nào. Việc chia hàm thành các phần nhỏ sẽ rất dễ dàng.

Đáng tiếc, điều này cũng làm lớp mất độ gắn kết vì nó tích lũy ngày càng nhiều biến thực thể chỉ tồn tại để một vài hàm dùng chung. Nhưng khoan đã! Nếu có vài hàm muốn dùng chung những biến nhất định thì chẳng phải chúng hợp thành một lớp riêng sao? Đúng vậy. Khi lớp mất độ gắn kết, hãy tách nó ra!

Vì thế, chia một hàm lớn thành nhiều hàm nhỏ thường tạo cơ hội tách thêm một số lớp nhỏ. Cách này giúp chương trình có tổ chức tốt hơn và cấu trúc minh bạch hơn.

Để minh họa ý mình, hãy dùng một ví dụ lâu đời trong cuốn sách tuyệt vời Literate Programming của Knuth.[3] Đoạn mã 10-5 trình bày phiên bản Java của chương trình PrintPrimes của Knuth. Công bằng mà nói, đây không phải chương trình do ông viết nguyên dạng, mà là bản do công cụ WEB của ông xuất ra. Tôi dùng nó vì đây là điểm khởi đầu tuyệt vời để chia một hàm lớn thành nhiều hàm và lớp nhỏ hơn.

Đoạn mã 10-5
PrintPrimes.java

```
package literatePrimes;

public class PrintPrimes {
  public static void main(String[] args) {
    final int M = 1000;
    final int RR = 50;
    final int CC = 4;
    final int WW = 10;
    final int ORDMAX = 30;
    int P[] = new int[M + 1];
    int PAGENUMBER;
    int PAGEOFFSET;
    int ROWOFFSET;
    int C;
    int J;
    int K;
    boolean JPRIME;
    int ORD;
    int SQUARE;
    int N;
    int MULT[] = new int[ORDMAX + 1];

    J = 1;
    K = 1;
    P[1] = 2;
```

3. [Knuth92].

```
      ORD = 2;
      SQUARE = 9;

      while (K < M) {
        do {
          J = J + 2;
          if (J == SQUARE) {
             ORD = ORD + 1;
             SQUARE = P[ORD] * P[ORD];
             MULT[ORD - 1] = J;
          }
          N = 2;
          JPRIME = true;
          while (N < ORD && JPRIME) {
             while (MULT[N] < J)
               MULT[N] = MULT[N] + P[N] + P[N];
             if (MULT[N] == J)
               JPRIME = false;
             N = N + 1;
          }
        } while (!JPRIME);
        K = K + 1;
        P[K] = J;
      }
      {
        PAGENUMBER = 1;
        PAGEOFFSET = 1;
        while (PAGEOFFSET <= M) {
          System.out.println("The First " + M +
                             " Prime Numbers --- Page " + PAGENUMBER);
          System.out.println("");
          for (ROWOFFSET = PAGEOFFSET; ROWOFFSET < PAGEOFFSET + RR;
     ROWOFFSET++){
             for (C = 0; C < CC;C++)
               if (ROWOFFSET + C * RR <= M)
                 System.out.format("%10d", P[ROWOFFSET + C * RR]);
             System.out.println("");
          }
          System.out.println("\f");
          PAGENUMBER = PAGENUMBER + 1;
          PAGEOFFSET = PAGEOFFSET + RR * CC;
        }
      }
    }
  }
```

Chương trình này, được viết thành một hàm duy nhất, là một mớ hỗn độn. Cấu trúc lồng sâu, vô số biến kỳ quặc và kết cấu phụ thuộc chặt chẽ. Ít nhất, nên chia hàm lớn duy nhất ấy thành vài hàm nhỏ hơn.

Đoạn mã 10-6 đến 10-8 cho thấy kết quả sau khi chia mã trong Đoạn mã 10-5 thành các lớp và hàm nhỏ hơn, đồng thời chọn tên có ý nghĩa cho những lớp, hàm và biến đó.

Đoạn mã 10-6
PrimePrinter.java (đã tái cấu trúc)

```
package literatePrimes;

public class PrimePrinter {
  public static void main(String[] args) {
    final int NUMBER_OF_PRIMES = 1000;
    int[] primes = PrimeGenerator.generate(NUMBER_OF_PRIMES);

        final int ROWS_PER_PAGE = 50;
        final int COLUMNS_PER_PAGE = 4;
        RowColumnPagePrinter tablePrinter =
          new RowColumnPagePrinter(ROWS_PER_PAGE,
                                   COLUMNS_PER_PAGE,
                                   "The First " + NUMBER_OF_PRIMES +
                                     " Prime Numbers");

        tablePrinter.print(primes);
    }

}
```

Đoạn mã 10-7
RowColumnPagePrinter.java

```
package literatePrimes;

import java.io.PrintStream;

public class RowColumnPagePrinter {
  private int rowsPerPage;
  private int columnsPerPage;
  private int numbersPerPage;
  private String pageHeader;
  private PrintStream printStream;

    public RowColumnPagePrinter(int rowsPerPage,
                                int columnsPerPage,
                                String pageHeader) {
      this.rowsPerPage = rowsPerPage;
      this.columnsPerPage = columnsPerPage;
      this.pageHeader = pageHeader;
      numbersPerPage = rowsPerPage * columnsPerPage;
      printStream = System.out;
    }
```

Đoạn mã 10-7 (tiếp)
RowColumnPagePrinter.java

```
    public void print(int data[]) {
      int pageNumber = 1;
      for (int firstIndexOnPage = 0;
           firstIndexOnPage < data.length;
           firstIndexOnPage += numbersPerPage) {
        int lastIndexOnPage =
```

```
            Math.min(firstIndexOnPage + numbersPerPage - 1,
                     data.length - 1);
          printPageHeader(pageHeader, pageNumber);
          printPage(firstIndexOnPage, lastIndexOnPage, data);
          printStream.println("\f");
          pageNumber++;
        }
      }
      private void printPage(int firstIndexOnPage,
                             int lastIndexOnPage,
                             int[] data) {
        int firstIndexOfLastRowOnPage =
          firstIndexOnPage + rowsPerPage - 1;
        for (int firstIndexInRow = firstIndexOnPage;
             firstIndexInRow <= firstIndexOfLastRowOnPage;
             firstIndexInRow++) {
          printRow(firstIndexInRow, lastIndexOnPage, data);
          printStream.println("");
        }
      }

      private void printRow(int firstIndexInRow,
                            int lastIndexOnPage,
                            int[] data) {
        for (int column = 0; column < columnsPerPage; column++) {
          int index = firstIndexInRow + column * rowsPerPage;
          if (index <= lastIndexOnPage)
            printStream.format("%10d", data[index]);
        }
      }

      private void printPageHeader(String pageHeader,
                                   int pageNumber) {
        printStream.println(pageHeader + " --- Page " + pageNumber);
        printStream.println("");
      }

      public void setOutput(PrintStream printStream) {
        this.printStream = printStream;
      }
    }
```

Đoạn mã 10-8
PrimeGenerator.java

```
package literatePrimes;

import java.util.ArrayList;

public class PrimeGenerator {
  private static int[] primes;
  private static ArrayList<Integer> multiplesOfPrimeFactors;

  protected static int[] generate(int n) {
    primes = new int[n];
    multiplesOfPrimeFactors = new ArrayList<Integer>();
```

```
    set2AsFirstPrime();
    checkOddNumbersForSubsequentPrimes();
    return primes;
  }

  private static void set2AsFirstPrime() {
    primes[0] = 2;
    multiplesOfPrimeFactors.add(2);
  }

  private static void checkOddNumbersForSubsequentPrimes() {
    int primeIndex = 1;
    for (int candidate = 3;
         primeIndex < primes.length;
         candidate += 2) {
      if (isPrime(candidate))
        primes[primeIndex++] = candidate;
    }
  }

  private static boolean isPrime(int candidate) {
    if (isLeastRelevantMultipleOfNextLargerPrimeFactor(candidate)) {
      multiplesOfPrimeFactors.add(candidate);
      return false;
    }
    return isNotMultipleOfAnyPreviousPrimeFactor(candidate);
  }

  private static boolean
  isLeastRelevantMultipleOfNextLargerPrimeFactor(int candidate) {
    int nextLargerPrimeFactor = primes[multiplesOfPrimeFactors.size()];
    int leastRelevantMultiple = nextLargerPrimeFactor *
    nextLargerPrimeFactor;
    return candidate == leastRelevantMultiple;
  }
  private static boolean
  isNotMultipleOfAnyPreviousPrimeFactor(int candidate) {
    for (int n = 1; n < multiplesOfPrimeFactors.size(); n++) {
      if (isMultipleOfNthPrimeFactor(candidate, n))
        return false;
    }
```

Đoạn mã 10-8 (tiếp)
PrimeGenerator.java

```
      return true;
    }

    private static boolean
    isMultipleOfNthPrimeFactor(int candidate, int n) {
      return
        candidate == smallestOddNthMultipleNotLessThanCandidate(candidate,
    n);
    }

    private static int
```

```
      smallestOddNthMultipleNotLessThanCandidate(int candidate, int n) {
        int multiple = multiplesOfPrimeFactors.get(n);
        while (multiple < candidate)
          multiple += 2 * primes[n];
        multiplesOfPrimeFactors.set(n, multiple);
        return multiple;
      }
  }
```

Điều đầu tiên bạn có thể nhận thấy là chương trình dài hơn hẳn. Nó tăng từ hơn một trang một chút lên gần ba trang. Có vài lý do cho sự gia tăng này. Thứ nhất, chương trình sau tái cấu trúc dùng tên biến dài hơn và mô tả rõ hơn. Thứ hai, chương trình dùng khai báo hàm và lớp để bổ sung lời giải thích cho mã. Thứ ba, chúng ta dùng khoảng trắng và kỹ thuật định dạng để giữ chương trình dễ đọc.

Hãy chú ý chương trình được chia thành ba trách nhiệm chính. Chương trình chính nằm riêng trong lớp PrimePrinter. Trách nhiệm của nó là xử lý môi trường thực thi. Lớp này sẽ thay đổi nếu cách gọi chương trình thay đổi. Chẳng hạn, nếu chuyển chương trình thành dịch vụ SOAP, đây là lớp sẽ bị ảnh hưởng.

RowColumnPagePrinter hiểu rõ cách định dạng danh sách số thành các trang có số hàng và cột nhất định. Nếu cần thay đổi định dạng đầu ra thì lớp này sẽ bị ảnh hưởng.

Lớp PrimeGenerator biết cách tạo danh sách số nguyên tố. Hãy lưu ý rằng lớp này không được thiết kế để khởi tạo thành đối tượng. Lớp chỉ là một phạm vi hữu ích để khai báo và ẩn các biến. Lớp này sẽ thay đổi nếu thuật toán tính số nguyên tố thay đổi.

Đây không phải là viết lại! Chúng ta không bắt đầu lại từ đầu để viết lại chương trình. Thật vậy, nếu xem kỹ hai chương trình, bạn sẽ thấy chúng dùng cùng một thuật toán và cơ chế để hoàn thành công việc.

Thay đổi được thực hiện bằng cách viết một bộ kiểm thử xác minh chính xác hành vi của chương trình đầu tiên. Sau đó, vô số thay đổi nhỏ được thực hiện, mỗi lần một thay đổi. Sau mỗi thay đổi, chương trình được chạy để bảo đảm hành vi không đổi. Từng bước nhỏ nối tiếp nhau, chương trình đầu tiên được làm sạch và chuyển thành chương trình thứ hai.

Tổ chức để thay đổi

Với hầu hết hệ thống, thay đổi diễn ra liên tục. Mỗi thay đổi đều khiến chúng ta đối mặt với nguy cơ phần còn lại của hệ thống không còn hoạt động như dự định. Trong một hệ thống sạch, chúng ta tổ chức các lớp sao cho giảm rủi ro khi thay đổi.

Lớp Sql trong Đoạn mã 10-9 dùng để tạo các chuỗi SQL đúng định dạng từ siêu dữ liệu phù hợp. Công việc vẫn đang tiếp diễn nên lớp này chưa hỗ trợ các chức năng SQL như câu lệnh update. Khi cần để lớp Sql hỗ trợ câu lệnh update, chúng ta sẽ phải "mở" lớp này để sửa đổi. Vấn đề của việc mở một lớp là nó tạo ra rủi ro. Bất kỳ sửa đổi nào cũng có thể làm hỏng phần mã khác trong lớp. Phải kiểm thử lại toàn bộ.

Đoạn mã 10-9
Lớp phải mở để thay đổi

```
public class Sql {
   public Sql(String table, Column[] columns)
   public String create()
   public String insert(Object[] fields)
   public String selectAll()
   public String findByKey(String keyColumn, String keyValue)
   public String select(Column column, String pattern)
   public String select(Criteria criteria)
   public String preparedInsert()
   private String columnList(Column[] columns)
   private String valuesList(Object[] fields, final Column[] columns)
   private String selectWithCriteria(String criteria)
   private String placeholderList(Column[] columns)
}
```

Lớp Sql phải thay đổi khi thêm một kiểu câu lệnh mới. Nó cũng phải thay đổi khi sửa chi tiết của một kiểu câu lệnh, chẳng hạn nếu cần sửa chức năng select để hỗ trợ truy vấn con. Hai lý do thay đổi này cho thấy lớp Sql vi phạm SRP.

Ta có thể nhận ra vi phạm SRP này chỉ bằng cách xem xét tổ chức. Dàn ý phương thức của Sql cho thấy có những phương thức riêng tư như selectWithCriteria dường như chỉ liên quan đến câu lệnh select.

Hành vi của phương thức riêng tư chỉ áp dụng cho một phần nhỏ của lớp có thể là một nguyên tắc kinh nghiệm hữu ích để nhận biết những khu vực tiềm năng cần cải thiện. Tuy nhiên, động lực chính để hành động nên đến từ chính những thay đổi của hệ thống. Nếu cho rằng lớp Sql đã hoàn chỉnh về mặt logic thì không cần lo chuyện tách trách nhiệm. Nếu trong tương lai gần chúng ta không cần chức năng update thì cứ để yên Sql. Nhưng ngay khi thấy mình phải mở một lớp, ta nên cân nhắc sửa lại thiết kế.

Nếu cân nhắc một giải pháp như trong Đoạn mã 10-10 thì sao? Mỗi phương thức giao diện công khai được định nghĩa trong Sql trước đó ở Đoạn mã 10-9 được tái cấu trúc thành một lớp dẫn xuất riêng của Sql. Lưu ý các phương thức riêng tư như valuesList được chuyển trực tiếp đến nơi cần dùng. Hành vi riêng tư dùng chung được tách vào hai lớp tiện ích là Where và ColumnList.

Đoạn mã 10-10
Một tập hợp các lớp khép kín

```
abstract public class Sql {
   public Sql(String table, Column[] columns)
   abstract public String generate();
}

public class CreateSql extends Sql {
   public CreateSql(String table, Column[] columns)
   @Override public String generate()
}
public class SelectSql extends Sql {
   public SelectSql(String table, Column[] columns)
   @Override public String generate()
}
public class InsertSql extends Sql {
   public InsertSql(String table, Column[] columns, Object[] fields)
```

```
      @Override public String generate()
      private String valuesList(Object[] fields, final Column[] columns)
   }

   public class SelectWithCriteriaSql extends Sql {
      public SelectWithCriteriaSql(
         String table, Column[] columns, Criteria criteria)
      @Override public String generate()
   }

   public class SelectWithMatchSql extends Sql {
      public SelectWithMatchSql(
         String table, Column[] columns, Column column, String pattern)
      @Override public String generate()
   }

   public class FindByKeySql extends Sql
      public FindByKeySql(
         String table, Column[] columns, String keyColumn, String keyValue)
      @Override public String generate()
   }

   public class PreparedInsertSql extends Sql {
      public PreparedInsertSql(String table, Column[] columns)
      @Override public String generate() {
      private String placeholderList(Column[] columns)
   }

   public class Where {
      public Where(String criteria)
      public String generate()
   }
```

Đoạn mã 10-10 (tiếp)
Một tập hợp các lớp khép kín

```
   public class ColumnList {
      public ColumnList(Column[] columns)
      public String generate()
   }
```

Mã trong mỗi lớp trở nên đơn giản đến mức không thể hơn. Thời gian cần thiết để hiểu bất kỳ lớp nào giảm xuống gần như bằng không. Rủi ro một hàm làm hỏng hàm khác nhỏ đến mức không đáng kể. Xét từ góc độ kiểm thử, việc chứng minh mọi phần logic trong giải pháp này đều đúng cũng dễ hơn, vì các lớp tách biệt với nhau.

Điều cũng quan trọng không kém là khi cần thêm các câu lệnh update, không lớp hiện có nào phải thay đổi! Chúng ta viết logic tạo câu lệnh update trong một lớp con mới của Sql tên là UpdateSql. Không đoạn mã nào khác trong hệ thống bị hỏng vì thay đổi này.

Logic Sql được tái cấu trúc của chúng ta đại diện cho giải pháp tốt nhất ở mọi phương diện. Nó tuân theo SRP. Nó cũng tuân theo một nguyên tắc thiết kế lớp hướng đối tượng

quan trọng khác, gọi là Nguyên tắc Mở-Đóng, hay OCP:[4] lớp nên mở để mở rộng nhưng đóng với sửa đổi. Lớp Sql sau tái cấu trúc mở để cho phép bổ sung chức năng mới qua kế thừa, nhưng ta có thể thực hiện thay đổi này mà vẫn giữ mọi lớp khác đóng. Chỉ cần đưa lớp UpdateSql vào.

Chúng ta muốn cấu trúc hệ thống sao cho phải động đến ít thứ nhất có thể khi cập nhật tính năng mới hoặc thay đổi tính năng hiện có. Trong một hệ thống lý tưởng, chúng ta bổ sung tính năng bằng cách mở rộng hệ thống, không phải bằng cách sửa đổi mã hiện có.

Cách ly khỏi thay đổi

Nhu cầu sẽ thay đổi, vì thế mã cũng sẽ thay đổi. Từ những bài học hướng đối tượng cơ bản, chúng ta biết có các lớp cụ thể, chứa chi tiết cài đặt (mã), và các lớp trừu tượng, chỉ biểu diễn khái niệm. Lớp khách phụ thuộc vào chi tiết cụ thể sẽ gặp rủi ro khi những chi tiết đó thay đổi. Ta có thể đưa vào giao diện và lớp trừu tượng để giúp cách ly tác động của những chi tiết ấy.

Sự phụ thuộc vào chi tiết cụ thể gây khó khăn cho việc kiểm thử hệ thống. Nếu đang xây dựng lớp Portfolio và lớp này phụ thuộc vào API TokyoStockExchange bên ngoài để tính giá trị danh mục đầu tư, các ca kiểm thử sẽ bị ảnh hưởng bởi tính biến động của việc tra cứu ấy. Thật khó viết kiểm thử khi cứ năm phút lại nhận được một kết quả khác!

Thay vì thiết kế Portfolio phụ thuộc trực tiếp vào TokyoStockExchange, ta tạo giao diện StockExchange khai báo một phương thức duy nhất:

```
public interface StockExchange {
   Money currentPrice(String symbol);
}
```

Ta thiết kế TokyoStockExchange để cài đặt giao diện này. Đồng thời, ta bảo đảm hàm khởi tạo của Portfolio nhận một tham chiếu StockExchange làm đối số:

```
public class Portfolio {
   private StockExchange exchange;
   public Portfolio(StockExchange exchange) {
      this.exchange = exchange;
   }
   // ...
}
```

Giờ đây, kiểm thử có thể tạo một cách cài đặt StockExchange có thể kiểm thử, mô phỏng TokyoStockExchange. Cách cài đặt kiểm thử này sẽ cố định giá trị hiện tại cho mọi mã cổ phiếu được dùng trong kiểm thử. Nếu ca kiểm thử mô phỏng việc mua năm cổ phiếu Microsoft cho danh mục đầu tư, ta lập trình cách cài đặt kiểm thử để luôn trả về 100 đô la cho mỗi cổ phiếu Microsoft. Cách cài đặt kiểm thử của giao diện StockExchange chỉ còn là một tra cứu bảng đơn giản. Sau đó, ta có thể viết một kiểm thử mong đợi tổng giá trị danh mục là 500 đô la.

---

4. [PPP].

```
public class PortfolioTest {
  private FixedStockExchangeStub exchange;
  private Portfolio portfolio;
    @Before
    protected void setUp() throws Exception {
      exchange = new FixedStockExchangeStub();
      exchange.fix("MSFT", 100);
      portfolio = new Portfolio(exchange);
    }

    @Test
    public void GivenFiveMSFTTotalShouldBe500() throws Exception {
      portfolio.add(5, "MSFT");
      Assert.assertEquals(500, portfolio.value());
    }
}
```

Nếu một hệ thống được tách rời đủ tốt để kiểm thử theo cách này, nó cũng sẽ linh hoạt hơn và khuyến khích tái sử dụng. Thiếu sự phụ thuộc có nghĩa là các thành phần của hệ thống được cách ly với nhau và với những thay đổi tốt hơn. Sự cách ly này giúp hiểu từng thành phần của hệ thống dễ hơn.

Bằng cách giảm thiểu sự phụ thuộc theo cách này, các lớp của chúng ta tuân theo một nguyên tắc thiết kế lớp khác gọi là Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle, DIP).[5] Về bản chất, DIP nói rằng các lớp nên phụ thuộc vào trừu tượng, không phải chi tiết cụ thể.

Thay vì phụ thuộc vào chi tiết cài đặt của lớp TokyoStockExchange, lớp Portfolio giờ đây phụ thuộc vào giao diện StockExchange. Giao diện StockExchange biểu diễn khái niệm trừu tượng về việc yêu cầu giá hiện tại của một mã cổ phiếu. Sự trừu tượng này cách ly mọi chi tiết cụ thể của việc lấy giá, bao gồm cả nguồn cung cấp giá đó.

Tài liệu tham khảo

[RDD]: Object Design: Roles, Responsibilities, and Collaborations, Rebecca Wirfs-Brock et al., Addison-Wesley, 2002.

[PPP]: Agile Software Development: Principles, Patterns, and Practices, Robert C. Martin, Prentice Hall, 2002.

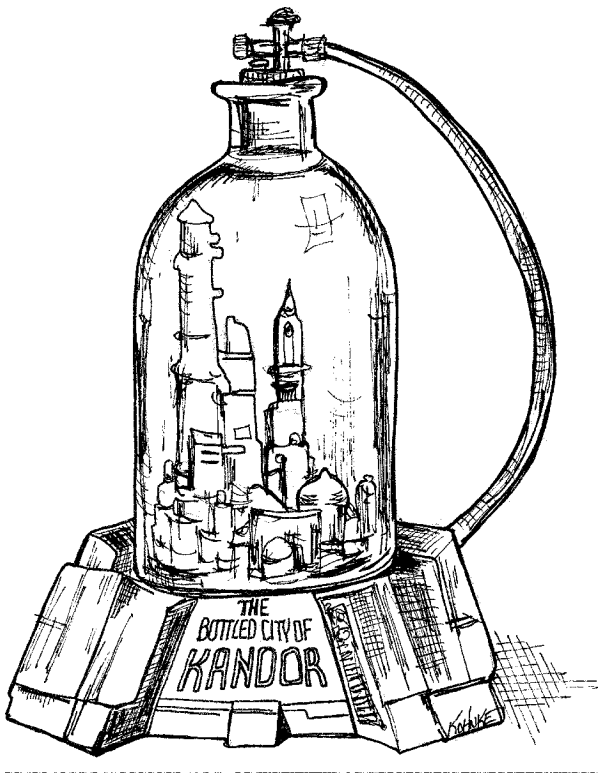
[Knuth92]: Literate Programming, Donald E. Knuth, Center for the Study of language and Information, Leland Stanford Junior University, 1992.

5. [PPP].

11

Hệ thống

Tác giả: TS. Kevin Dean Wampler



"Sự phức tạp giết chết mọi thứ. Nó hút cạn sức sống của lập trình viên, khiến việc lập kế hoạch, xây dựng và kiểm thử sản phẩm trở nên khó khăn."

—Ray Ozzie, CTO, Microsoft Corporation

Bạn sẽ xây dựng một thành phố như thế nào?

Bạn có thể tự quản lý mọi chi tiết không? Có lẽ là không. Ngay cả việc quản lý một thành phố hiện có cũng quá sức với một người. Thế nhưng các thành phố vẫn vận hành, hầu hết thời gian. Chúng vận hành vì có những nhóm người phụ trách từng phần cụ thể của thành phố: hệ thống nước, hệ thống điện, giao thông, thực thi pháp luật, quy chuẩn xây dựng, vân vân. Một số người chịu trách nhiệm về bức tranh tổng thể, số khác tập trung vào chi tiết.

Thành phố còn vận hành được vì chúng đã phát triển những mức độ trừu tượng và mô-đun hóa phù hợp, cho phép mỗi cá nhân và những "thành phần" họ quản lý làm việc hiệu quả, ngay cả khi họ không hiểu bức tranh tổng thể.

Các nhóm phần mềm thường cũng được tổ chức như vậy, nhưng hệ thống mà họ làm việc thường không có sự phân tách mối quan tâm và các mức độ trừu tượng tương tự. Mã sạch giúp chúng ta đạt được điều đó ở các mức trừu tượng thấp. Trong chương này, chúng ta sẽ xem xét cách giữ cho hệ thống sạch ở các mức trừu tượng cao hơn, tức cấp hệ thống.

Tách việc xây dựng hệ thống khỏi việc sử dụng hệ thống

Trước tiên, hãy nhớ rằng xây dựng là một quá trình rất khác với sử dụng. Khi tôi viết những dòng này, một khách sạn mới đang được xây dựng ngoài cửa sổ chỗ tôi ở tại Chicago. Hôm nay, nó chỉ là một khối bê tông trơ trụi, bên ngoài gắn cần cẩu và thang máy xây dựng. Những người bận rộn ở đó đều đội mũ bảo hộ và mặc quần áo lao động. Khoảng một năm nữa, khách sạn sẽ hoàn thành. Cần cẩu và thang máy sẽ biến mất. Tòa nhà sẽ sạch sẽ, được bao bọc bởi những vách kính và lớp sơn đẹp mắt. Những người làm việc và lưu trú ở đó cũng sẽ trông khác hẳn.

> *Hệ thống phần mềm nên tách quá trình khởi động, khi các đối tượng ứng dụng được tạo ra và các phụ thuộc được "kết nối" với nhau, khỏi logic lúc chạy tiếp quản sau khi khởi động.*

Quá trình khởi động là mối quan tâm mà mọi ứng dụng đều phải xử lý. Đây là mối quan tâm đầu tiên chúng ta xem xét trong chương này. Tách biệt mối quan tâm là một trong những kỹ thuật thiết kế lâu đời và quan trọng nhất trong nghề của chúng ta.

Đáng tiếc, hầu hết ứng dụng không tách biệt mối quan tâm này. Mã xử lý khởi động thường được viết tùy tiện và trộn lẫn với logic lúc chạy. Đây là một ví dụ điển hình:

```
public Service getService() {
  if (service == null)
    service = new MyServiceImpl(...); // Mặc định này đủ tốt cho hầu hết
    trường hợp chứ?
  return service;
}
```

Đây là thành ngữ KHỞI TẠO/ĐÁNH GIÁ LƯỜI, và nó có một số ưu điểm. Ta không phải chịu chi phí khởi tạo cho đến khi thực sự dùng đối tượng, nhờ đó thời gian khởi động có thể nhanh hơn. Ta cũng bảo đảm không bao giờ trả về `null`.

Tuy nhiên, giờ đây ta phụ thuộc cứng vào `MyServiceImpl` và mọi thứ mà hàm tạo của nó cần (tôi đã lược bỏ phần đó). Ta không thể biên dịch nếu chưa giải quyết các phụ thuộc này, ngay cả khi lúc chạy ta không bao giờ dùng đối tượng thuộc kiểu này!

Việc kiểm thử có thể trở thành vấn đề. Nếu `MyServiceImpl` là một đối tượng nặng, ta phải bảo đảm một ĐỐI TƯỢNG KIỂM THỬ THAY THẾ[1] hoặc ĐỐI TƯỢNG GIẢ được gán cho trường `service` trước khi phương thức này được gọi trong quá trình kiểm thử đơn vị. Vì logic tạo đối tượng bị trộn với xử lý thông thường lúc chạy, ta nên kiểm thử mọi nhánh thực thi, chẳng hạn phép kiểm tra `null` và khối lệnh của nó. Phương thức đang gánh cả hai trách nhiệm, nghĩa là nó làm nhiều hơn một việc và vi phạm Nguyên tắc Trách nhiệm Đơn nhất ở mức độ nhỏ.

Có lẽ tệ nhất là ta không biết `MyServiceImpl` có phải đối tượng phù hợp trong mọi trường hợp hay không. Tôi đã ngụ ý điều đó trong chú thích. Tại sao lớp có phương thức này lại phải biết ngữ cảnh toàn cục? Liệu ta có bao giờ biết chắc đối tượng nào là đúng để dùng ở đây không? Có thể nào một kiểu đối tượng phù hợp với mọi ngữ cảnh không?

Dĩ nhiên, một chỗ dùng KHỞI TẠO LƯỜI không phải vấn đề nghiêm trọng. Nhưng ứng dụng thường có nhiều thành ngữ thiết lập nhỏ như thế. Vì vậy, chiến lược thiết lập toàn cục (nếu có) bị rải rác khắp ứng dụng, ít mô-đun hóa và thường trùng lặp đáng kể.

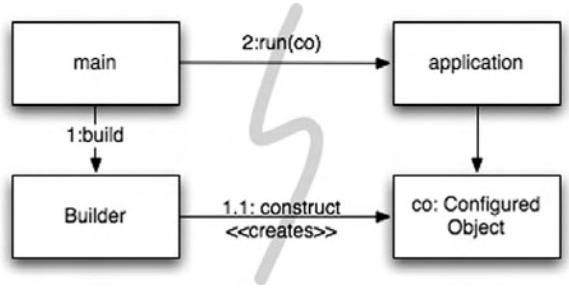
Nếu cẩn trọng khi xây dựng các hệ thống vững chắc, có cấu trúc tốt, ta không nên để những thành ngữ nhỏ tiện lợi làm suy yếu tính mô-đun. Việc tạo và kết nối đối tượng trong quá trình khởi động cũng không ngoại lệ. Ta nên mô-đun hóa riêng quá trình này khỏi logic thông thường lúc chạy, đồng thời bảo đảm có một chiến lược toàn cục, nhất quán để giải quyết các phụ thuộc chính.

Tách biệt hàm main

Một cách tách việc xây dựng khỏi việc sử dụng là chuyển mọi khía cạnh của việc xây dựng sang `main`, hoặc các mô-đun được `main` gọi, rồi thiết kế phần còn lại của hệ thống với giả định mọi đối tượng đã được tạo và kết nối phù hợp. (Xem Hình 11-1.)

Luồng điều khiển rất dễ theo dõi. Hàm `main` tạo các đối tượng cần thiết cho hệ thống rồi chuyển chúng cho ứng dụng, nơi đơn giản là sử dụng chúng. Hãy chú ý hướng của các mũi tên phụ thuộc cắt qua ranh giới giữa `main` và ứng dụng. Tất cả đều hướng ra khỏi `main`. Điều này có nghĩa ứng dụng không biết gì về `main` hay quá trình xây dựng. Ứng dụng chỉ mong mọi thứ đã được tạo đúng cách.

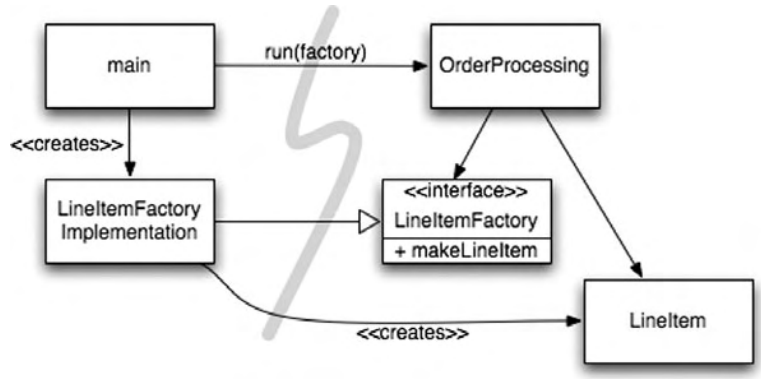
1. [Mezzaros07].



Hình 11-1
Tách việc xây dựng trong `main()`

Factory

Dĩ nhiên, đôi khi ta cần để ứng dụng quyết định thời điểm tạo đối tượng. Chẳng hạn, trong hệ thống xử lý đơn hàng, ứng dụng phải tạo các đối tượng `LineItem` để thêm vào một `Order`. Khi đó ta có thể dùng mẫu ABSTRACT FACTORY[2] để ứng dụng kiểm soát thời điểm tạo `LineItem`, nhưng vẫn tách chi tiết xây dựng khỏi mã ứng dụng. (Xem Hình 11-2.)



Hình 11-2
Tách biệt việc xây dựng bằng factory

Một lần nữa, hãy chú ý rằng mọi phụ thuộc đều hướng từ `main` về phía ứng dụng `OrderProcessing`. Điều này có nghĩa ứng dụng được tách rời khỏi chi tiết xây dựng `LineItem`. Khả năng đó nằm trong `LineItemFactoryImplementation`, ở phía `main` của ranh giới. Tuy vậy, ứng dụng vẫn hoàn toàn kiểm soát thời điểm tạo các đối tượng `LineItem`, thậm chí có thể cung cấp các đối số hàm tạo dành riêng cho ứng dụng.

2. [GOF].

Tiêm phụ thuộc

Một cơ chế mạnh để tách việc xây dựng khỏi việc sử dụng là Tiêm phụ thuộc (Dependency Injection, DI), tức áp dụng Đảo ngược điều khiển (Inversion of Control, IoC) vào việc quản lý phụ thuộc.[3] Đảo ngược điều khiển chuyển các trách nhiệm phụ từ một đối tượng sang những đối tượng khác được dành riêng cho mục đích đó, qua đó hỗ trợ Nguyên tắc Trách nhiệm Đơn nhất. Trong ngữ cảnh quản lý phụ thuộc, một đối tượng không nên tự chịu trách nhiệm khởi tạo các phụ thuộc của mình. Thay vào đó, nó nên giao trách nhiệm này cho một cơ chế "có thẩm quyền" khác, qua đó đảo ngược quyền điều khiển. Vì thiết lập là mối quan tâm toàn cục, cơ chế có thẩm quyền này thường là thủ tục `main` hoặc một bộ chứa chuyên dụng.

Các tra cứu JNDI là một cách triển khai DI "một phần", trong đó một đối tượng yêu cầu máy chủ thư mục cung cấp một "dịch vụ" khớp với tên cụ thể:

```
MyService myService = (MyService)(jndiContext.lookup("NameOfMyService"));
```

Đối tượng gọi không kiểm soát kiểu đối tượng thực sự được trả về (dĩ nhiên, miễn là đối tượng đó triển khai giao diện phù hợp), nhưng nó vẫn chủ động giải quyết phụ thuộc.

Tiêm phụ thuộc thực sự tiến thêm một bước. Lớp không thực hiện bước trực tiếp nào để giải quyết các phụ thuộc của nó, mà hoàn toàn thụ động. Thay vào đó, lớp cung cấp các phương thức setter hoặc đối số hàm tạo, hoặc cả hai, để tiêm các phụ thuộc vào. Trong quá trình xây dựng, bộ chứa DI khởi tạo các đối tượng cần thiết (thường theo yêu cầu) rồi dùng các đối số hàm tạo hoặc phương thức setter được cung cấp để kết nối các phụ thuộc. Những đối tượng phụ thuộc nào thực sự được dùng sẽ được chỉ định qua tệp cấu hình hoặc bằng mã trong một mô-đun xây dựng chuyên dụng.

Spring Framework cung cấp bộ chứa DI nổi tiếng nhất cho Java.[4] Bạn định nghĩa các đối tượng cần kết nối trong một tệp cấu hình XML, rồi yêu cầu các đối tượng cụ thể theo tên trong mã Java. Chúng ta sẽ xem ví dụ ngay sau đây.

Thế còn ưu điểm của KHỞI TẠO LƯỜI thì sao? Thành ngữ này đôi khi vẫn hữu ích khi dùng DI. Thứ nhất, hầu hết bộ chứa DI sẽ không tạo một đối tượng cho đến khi cần. Thứ hai, nhiều bộ chứa cung cấp cơ chế gọi factory hoặc tạo proxy, có thể dùng cho ĐÁNH GIÁ LƯỜI và các tối ưu hóa tương tự.[5]

Mở rộng quy mô

Thành phố phát triển từ thị trấn, thị trấn phát triển từ khu định cư. Ban đầu, đường sá nhỏ hẹp và gần như chưa tồn tại, rồi được trải nhựa, sau đó mở rộng dần theo thời gian. Những tòa nhà nhỏ và lô đất trống được lấp đầy bằng các tòa nhà lớn hơn, một số cuối cùng sẽ được thay bằng nhà chọc trời.

---

3. Ví dụ, xem [Fowler].

4. Xem [Spring]. Ngoài ra còn có framework Spring.NET.

5. Đừng quên rằng khởi tạo/đánh giá lười chỉ là một tối ưu hóa và có thể là tối ưu hóa quá sớm!

Ban đầu chưa có các dịch vụ như điện, nước, thoát nước và Internet (trời ơi!). Những dịch vụ này cũng được bổ sung khi dân số và mật độ xây dựng tăng lên.

Sự phát triển ấy không phải không có đau đớn. Đã bao lần bạn lái xe chậm rì, xe nối đuôi nhau qua một công trình "cải thiện" đường sá rồi tự hỏi: "Sao ngay từ đầu họ không xây đường rộng hơn nhỉ!?"

Nhưng không thể làm khác đi. Ai có thể biện minh cho chi phí xây một xa lộ sáu làn xuyên qua thị trấn nhỏ chỉ vì dự đoán nơi đó sẽ phát triển? Ai lại muốn có con đường như thế chạy qua thị trấn mình?

Thật hoang đường khi nghĩ rằng ta có thể làm cho hệ thống "đúng ngay từ đầu". Thay vào đó, ta nên chỉ triển khai các câu chuyện người dùng của hôm nay, rồi tái cấu trúc và mở rộng hệ thống để triển khai những câu chuyện mới vào ngày mai. Đó là cốt lõi của phát triển linh hoạt theo vòng lặp và tăng dần. Phát triển hướng kiểm thử, tái cấu trúc và mã sạch do chúng tạo ra giúp cách làm này hiệu quả ở cấp mã.

Còn ở cấp hệ thống thì sao? Chẳng phải kiến trúc hệ thống cần được hoạch định trước ư? Nó đâu thể phát triển tăng dần từ đơn giản đến phức tạp được, phải không?

> *Hệ thống phần mềm khác biệt với hệ thống vật lý. Kiến trúc của chúng có thể phát triển tăng dần nếu ta duy trì sự phân tách mối quan tâm thích hợp.*

Bản chất phù du của hệ thống phần mềm khiến điều này trở nên khả thi, như chúng ta sẽ thấy. Trước hết, hãy xem một ví dụ đối lập về kiến trúc không phân tách mối quan tâm đầy đủ.

Các kiến trúc EJB1 và EJB2 ban đầu không phân tách mối quan tâm phù hợp, do đó dựng lên những rào cản không cần thiết đối với sự phát triển tự nhiên. Hãy xem xét một Entity Bean cho lớp `Bank` có dữ liệu bền vững. Entity bean là biểu diễn trong bộ nhớ của dữ liệu quan hệ, nói cách khác là một hàng trong bảng.

Trước tiên, bạn phải định nghĩa một giao diện cục bộ (trong cùng tiến trình) hoặc từ xa (JVM riêng), để các phía client sử dụng. Đoạn mã 11-1 cho thấy một giao diện cục bộ có thể có:

Đoạn mã 11-1 Giao diện cục bộ EJB2 cho một EJB `Bank`

```
package com.example.banking;
import java.util.Collections;
import javax.ejb.*;

public interface BankLocal extends java.ejb.EJBLocalObject {
  String getStreetAddr1() throws EJBException;
  String getStreetAddr2() throws EJBException;
  String getCity() throws EJBException;
  String getState() throws EJBException;
  String getZipCode() throws EJBException;
  void setStreetAddr1(String street1) throws EJBException;
  void setStreetAddr2(String street2) throws EJBException;
  void setCity(String city) throws EJBException;
  void setState(String state) throws EJBException;
}
```

Đoạn mã 11-1 (tiếp) Giao diện cục bộ EJB2 cho một EJB `Bank`

```
void setZipCode(String zip) throws EJBException;
Collection getAccounts() throws EJBException;
void setAccounts(Collection accounts) throws EJBException;
void addAccount(AccountDTO accountDTO) throws EJBException;
}
```

Tôi đã trình bày một số thuộc tính địa chỉ của `Bank` và một tập hợp các tài khoản mà ngân hàng sở hữu. Mỗi tài khoản có dữ liệu được một EJB `Account` riêng xử lý. Đoạn mã 11-2 cho thấy lớp triển khai tương ứng cho bean `Bank`.

Đoạn mã 11-2 Lớp triển khai Entity Bean EJB2 tương ứng

```
package com.example.banking;
import java.util.Collections;
import javax.ejb.*;
public abstract class Bank implements javax.ejb.EntityBean {
  // Logic nghiệp vụ...
  public abstract String getStreetAddr1();
  public abstract String getStreetAddr2();
  public abstract String getCity();
  public abstract String getState();
  public abstract String getZipCode();
  public abstract void setStreetAddr1(String street1);
  public abstract void setStreetAddr2(String street2);
  public abstract void setCity(String city);
  public abstract void setState(String state);
  public abstract void setZipCode(String zip);
  public abstract Collection getAccounts();
  public abstract void setAccounts(Collection accounts);
  public void addAccount(AccountDTO accountDTO) {
    InitialContext context = new InitialContext();
    AccountHomeLocal accountHome = context.lookup("AccountHomeLocal");
    AccountLocal account = accountHome.create(accountDTO);
    Collection accounts = getAccounts();
    accounts.add(account);
  }
  // Logic của bộ chứa EJB
  public abstract void setId(Integer id);
  public abstract Integer getId();
  public Integer ejbCreate(Integer id) { ... }
  public void ejbPostCreate(Integer id) { ... }
  // Phần còn lại phải được triển khai nhưng thường để trống:
  public void setEntityContext(EntityContext ctx) {}
  public void unsetEntityContext() {}
  public void ejbActivate() {}
  public void ejbPassivate() {}
  public void ejbLoad() {}
  public void ejbStore() {}
  public void ejbRemove() {}
}
```

Tôi chưa trình bày giao diện `LocalHome` tương ứng, về cơ bản là một factory dùng để tạo đối tượng, cũng như bất kỳ phương thức tìm kiếm (truy vấn) `Bank` nào bạn có thể bổ sung.

Cuối cùng, bạn phải viết một hoặc nhiều bộ mô tả triển khai XML, chỉ định chi tiết ánh xạ đối tượng-quan hệ tới kho lưu trữ, hành vi giao dịch mong muốn, các ràng buộc bảo mật, vân vân.

Logic nghiệp vụ gắn chặt với "bộ chứa" ứng dụng EJB2. Bạn phải kế thừa các kiểu của bộ chứa và cung cấp nhiều phương thức vòng đời mà bộ chứa yêu cầu.

Vì gắn chặt với bộ chứa cồng kềnh này, việc kiểm thử đơn vị cô lập trở nên khó khăn. Ta phải giả lập bộ chứa, việc này rất khó, hoặc mất nhiều thời gian triển khai EJB và các kiểm thử lên máy chủ thật. Do phụ thuộc chặt chẽ, gần như không thể tái sử dụng bên ngoài kiến trúc EJB2.

Cuối cùng, ngay cả lập trình hướng đối tượng cũng bị suy yếu. Một bean không thể kế thừa từ bean khác. Hãy chú ý logic thêm một tài khoản mới. Các bean EJB2 thường định nghĩa "đối tượng truyền dữ liệu" (DTO), về cơ bản là các "struct" không có hành vi. Việc này thường dẫn đến các kiểu dữ liệu dư thừa chứa những dữ liệu gần như giống hệt nhau và buộc phải viết mã khuôn mẫu để sao chép dữ liệu từ đối tượng này sang đối tượng khác.

Mối quan tâm cắt ngang

Kiến trúc EJB2 ở một số khía cạnh đã tiến gần đến việc phân tách mối quan tâm thực sự. Chẳng hạn, các hành vi giao dịch, bảo mật và một số hành vi lưu trữ bền vững mong muốn được khai báo trong bộ mô tả triển khai, độc lập với mã nguồn.

Lưu ý rằng những mối quan tâm như lưu trữ bền vững thường cắt ngang ranh giới đối tượng tự nhiên của một miền. Ta muốn lưu trữ mọi đối tượng theo nhìn chung cùng một chiến lược, chẳng hạn dùng một DBMS[6] cụ thể thay vì tệp phẳng, tuân theo một số quy ước đặt tên cho bảng và cột, dùng ngữ nghĩa giao dịch nhất quán, vân vân.

Về nguyên tắc, ta có thể suy xét chiến lược lưu trữ bền vững theo cách mô-đun và đóng gói. Tuy vậy, trên thực tế, ta phải rải cùng một đoạn mã triển khai chiến lược lưu trữ bền vững khắp nhiều đối tượng. Ta gọi những mối quan tâm như vậy là mối quan tâm cắt ngang. Một lần nữa, framework lưu trữ bền vững có thể được mô-đun hóa, và logic miền của ta khi đứng riêng cũng có thể được mô-đun hóa. Vấn đề nằm ở sự giao cắt chi tiết giữa các miền này.

Thực tế, cách kiến trúc EJB xử lý việc lưu trữ bền vững, bảo mật và giao dịch đã "đi trước" lập trình hướng khía cạnh (AOP),[7] một cách tiếp cận đa dụng nhằm khôi phục tính mô-đun cho các mối quan tâm cắt ngang.

Trong AOP, các cấu trúc mô-đun gọi là khía cạnh chỉ định những điểm trong hệ thống cần được điều chỉnh hành vi theo cách nhất quán để hỗ trợ một mối quan tâm cụ thể. Việc chỉ định này được thực hiện bằng cơ chế khai báo hoặc lập trình súc tích.

Lấy lưu trữ bền vững làm ví dụ, bạn sẽ khai báo những đối tượng và thuộc tính nào (hoặc mẫu của chúng) cần được lưu trữ, rồi giao nhiệm vụ lưu trữ cho framework lưu trữ bền vững. Framework AOP thực hiện những thay đổi hành vi đó trên mã đích mà không

---

6. Hệ quản trị cơ sở dữ liệu.

7. Để biết thông tin tổng quát về khía cạnh, xem [AOSD]; để biết thông tin riêng về AspectJ, xem [AspectJ] và [Colyer].

xâm lấn.[8] Hãy xem ba khía cạnh hoặc cơ chế giống khía cạnh trong Java.

Proxy Java

Proxy Java phù hợp với các tình huống đơn giản, chẳng hạn bọc lời gọi phương thức trong từng đối tượng hoặc lớp. Tuy nhiên, các proxy động do JDK cung cấp chỉ hoạt động với giao diện. Muốn proxy lớp, bạn phải dùng thư viện thao tác bytecode như CGLIB, ASM hoặc Javassist.[9]

Đoạn mã 11-3 trình bày bộ khung proxy JDK để hỗ trợ lưu trữ bền vững cho ứng dụng `Bank`, chỉ bao gồm các phương thức lấy và thiết lập danh sách tài khoản.

Đoạn mã 11-3 Ví dụ proxy JDK

```
// Bank.java (lược bỏ tên gói...)
import java.utils.*;

// Trừu tượng hóa một ngân hàng.
public interface Bank {
  Collection<Account> getAccounts();
  void setAccounts(Collection<Account> accounts);
}
// BankImpl.java
import java.utils.*;
// “Plain Old Java Object” (POJO) triển khai phần trừu tượng.
public class BankImpl implements Bank {
  private List<Account> accounts;
    public Collection<Account> getAccounts() {
      return accounts;
    }
    public void setAccounts(Collection<Account> accounts) {
      this.accounts = new ArrayList<Account>();
      for (Account account: accounts) {
        this.accounts.add(account);
      }
    }
}

// BankProxyHandler.java
import java.lang.reflect.*;
import java.util.*;
```

Đoạn mã 11-3 (tiếp) Ví dụ proxy JDK

```
// “InvocationHandler” bắt buộc theo API proxy.
public class BankProxyHandler implements InvocationHandler {
  private Bank bank;

    public BankHandler (Bank bank) {
      this.bank = bank;
```

8. Nghĩa là không cần chỉnh sửa thủ công mã nguồn đích.

9. Xem [CGLIB], [ASM] và [Javassist].

```
    }

    // Phương thức được định nghĩa trong InvocationHandler
    public Object invoke(Object proxy, Method method, Object[] args)
        throws Throwable {
      String methodName = method.getName();
      if (methodName.equals("getAccounts")) {
        bank.setAccounts(getAccountsFromDatabase());
        return bank.getAccounts();
      } else if (methodName.equals("setAccounts")) {
        bank.setAccounts((Collection<Account>) args[0]);
        setAccountsToDatabase(bank.getAccounts());
        return null;
      } else {
        ...
      }
    }

    // Còn nhiều chi tiết khác:
    protected Collection<Account> getAccountsFromDatabase() { ... }
    protected void setAccountsToDatabase(Collection<Account> accounts) { ...
    }
}
// Ở nơi khác...

Bank bank = (Bank) Proxy.newProxyInstance(
  Bank.class.getClassLoader(),
  new Class[] { Bank.class },
  new BankProxyHandler(new BankImpl()));
```

Ta đã định nghĩa một giao diện `Bank`, sẽ được proxy bọc lại, và một Plain-Old Java Object (POJO) `BankImpl` triển khai logic nghiệp vụ. (Chúng ta sẽ quay lại POJO ngay sau đây.)

API Proxy yêu cầu một đối tượng `InvocationHandler`, đối tượng này được gọi để triển khai mọi lời gọi phương thức `Bank` thực hiện trên proxy. `BankProxyHandler` dùng API phản chiếu Java để ánh xạ các lời gọi phương thức tổng quát tới những phương thức tương ứng trong `BankImpl`, vân vân.

Có rất nhiều mã ở đây và nó tương đối phức tạp, ngay cả với trường hợp đơn giản này.[10] Dùng một trong các thư viện thao tác bytecode cũng khó tương tự. Khối lượng mã và độ phức tạp là hai nhược điểm của proxy. Chúng khiến việc viết mã sạch trở nên khó khăn! Ngoài ra, proxy không cung cấp cơ chế chỉ định các "điểm" thực thi quan tâm trên toàn hệ thống, điều cần có để tạo giải pháp AOP thực sự.[11]

---

10. Để xem ví dụ chi tiết hơn về API Proxy và cách sử dụng, chẳng hạn xem [Goetz].

11. Đôi khi người ta nhầm AOP với các kỹ thuật dùng để triển khai nó, chẳng hạn chặn phương thức và "bọc" bằng proxy. Giá trị thực sự của hệ thống AOP là khả năng chỉ định hành vi trên toàn hệ thống một cách súc tích và mô-đun.

Framework AOP thuần Java

May mắn thay, các đoạn mã khuôn mẫu của proxy phần lớn có thể được công cụ xử lý tự động. Proxy được dùng bên trong một số framework Java, chẳng hạn Spring AOP và JBoss AOP, để triển khai khía cạnh bằng Java thuần.[12] Trong Spring, bạn viết logic nghiệp vụ dưới dạng Plain-Old Java Object. POJO chỉ tập trung vào miền của nó. Chúng không phụ thuộc vào framework doanh nghiệp (hay bất kỳ miền nào khác). Vì thế, về mặt khái niệm, chúng đơn giản hơn và dễ phát triển theo hướng kiểm thử hơn. Sự đơn giản tương đối giúp bảo đảm dễ dàng hơn rằng bạn đang triển khai đúng các câu chuyện người dùng tương ứng, đồng thời dễ bảo trì và phát triển mã để đáp ứng những câu chuyện về sau.

Bạn đưa hạ tầng ứng dụng cần thiết vào, bao gồm các mối quan tâm cắt ngang như lưu trữ bền vững, giao dịch, bảo mật, bộ nhớ đệm, chuyển đổi dự phòng, vân vân, bằng các tệp cấu hình khai báo hoặc API. Trong nhiều trường hợp, thực chất bạn đang chỉ định các khía cạnh của thư viện Spring hoặc JBoss, còn framework xử lý cơ chế sử dụng proxy Java hoặc thư viện bytecode một cách trong suốt với người dùng. Các khai báo này điều khiển bộ chứa tiêm phụ thuộc (DI), bộ chứa khởi tạo các đối tượng chính và kết nối chúng theo yêu cầu.

Đoạn mã 11-4 cho thấy một phần điển hình của tệp cấu hình Spring V2.5, `app.xml`.[13]

Đoạn mã 11-4 Tệp cấu hình Spring 2.X

```
<beans>
  ...
  <bean id="appDataSource"
  class="org.apache.commons.dbcp.BasicDataSource"
  destroy-method="close"
  p:driverClassName="com.mysql.jdbc.Driver"
  p:url="jdbc:mysql://localhost:3306/mydb"
  p:username="me"/>
   <bean id="bankDataAccessObject"
   class="com.example.banking.persistence.BankDataAccessObject"
   p:dataSource-ref="appDataSource"/>

   <bean id="bank"
```

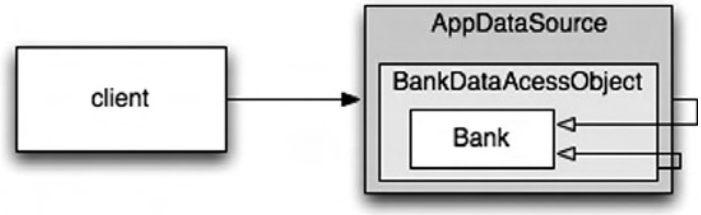
Đoạn mã 11-4 (tiếp) Tệp cấu hình Spring 2.X

```
  class="com.example.banking.model.Bank"
  p:dataAccessObject-ref="bankDataAccessObject"/>
  ...
</beans>
```

Mỗi "bean" giống như một phần của búp bê Nga lồng nhau, trong đó đối tượng miền `Bank` được một đối tượng truy cập dữ liệu (DAO) bọc (proxy), rồi chính DAO lại được một nguồn dữ liệu trình điều khiển JDBC bọc. (Xem Hình 11-3.)

12. Xem [Spring] và [JBoss]. "Java thuần" nghĩa là không dùng AspectJ.

13. Phỏng theo `http://www.theserverside.com/tt/articles/article.tss?l=IntrotoSpring25`



Hình 11-3
"Búp bê Nga" của các decorator

Phía client tưởng rằng nó đang gọi `getAccounts()` trên một đối tượng `Bank`, nhưng thực ra nó đang giao tiếp với lớp ngoài cùng trong một tập hợp các đối tượng DECORATOR[14] lồng nhau, mở rộng hành vi cơ bản của POJO `Bank`. Ta có thể thêm các decorator khác cho giao dịch, bộ nhớ đệm, vân vân.

Trong ứng dụng, chỉ cần vài dòng để yêu cầu bộ chứa DI cung cấp các đối tượng cấp cao nhất của hệ thống như đã chỉ định trong tệp XML:

```
XmlBeanFactory bf =
  new XmlBeanFactory(new ClassPathResource("app.xml", getClass()));
Bank bank = (Bank) bf.getBean("bank");
```

Vì chỉ cần rất ít dòng mã Java đặc thù của Spring, ứng dụng gần như tách biệt hoàn toàn khỏi Spring, loại bỏ mọi vấn đề phụ thuộc chặt chẽ của các hệ thống như EJB2.

Dù XML có thể dài dòng và khó đọc,[15] "chính sách" được chỉ định trong những tệp cấu hình này vẫn đơn giản hơn logic proxy và khía cạnh phức tạp bị ẩn khỏi tầm mắt và được tạo tự động. Kiến trúc này hấp dẫn đến mức các framework như Spring đã dẫn tới cuộc cải tổ toàn diện tiêu chuẩn EJB ở phiên bản 3. EJB3 phần lớn đi theo mô hình Spring, hỗ trợ khai báo các mối quan tâm cắt ngang bằng tệp cấu hình XML và/hoặc chú thích Java 5.

Đoạn mã 11-5 cho thấy đối tượng `Bank` được viết lại bằng EJB3.[16]

Đoạn mã 11-5 Một EJB `Bank` EJB3

```
package com.example.banking.model;
import javax.persistence.*;
import java.util.ArrayList;
import java.util.Collection;
@Entity
@Table(name = "BANKS")
public class Bank implements java.io.Serializable {
   @Id @GeneratedValue(strategy=GenerationType.AUTO)
   private int id;
```

14. [GOF].

15. Có thể đơn giản hóa ví dụ bằng các cơ chế tận dụng quy ước thay cho cấu hình và chú thích Java 5 để giảm lượng logic "kết nối" tường minh cần thiết.

16. Phỏng theo `http://www.onjava.com/pub/a/onjava/2006/05/17/standardizing-with-ejb3-java-persistence-api.html`

```
@Embeddable // Một đối tượng được “nhúng trực tiếp” trong hàng DB của
Bank
public class Address {
   protected String streetAddr1;
   protected String streetAddr2;
   protected String city;
   protected String state;
   protected String zipCode;
}

@Embedded
private Address address;

@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER,
           mappedBy="bank")
private Collection<Account> accounts = new ArrayList<Account>();

public int getId() {
   return id;
}
public void setId(int id) {
   this.id = id;
}

public void addAccount(Account account) {
   account.setBank(this);
   accounts.add(account);
}

public Collection<Account> getAccounts() {
   return accounts;
}
```

Đoạn mã 11-5 (tiếp) Một EJB `Bank` EJB3

```
public void setAccounts(Collection<Account> accounts) {
   this.accounts = accounts;
}
}
```

Mã này sạch hơn nhiều so với mã EJB2 ban đầu. Một số chi tiết thực thể vẫn còn đó, được chứa trong các chú thích. Tuy nhiên, vì không có thông tin nào nằm ngoài chú thích, mã vẫn sạch sẽ, rõ ràng và do đó dễ phát triển theo hướng kiểm thử, dễ bảo trì, vân vân.

Nếu muốn, có thể chuyển một phần hoặc toàn bộ thông tin lưu trữ bền vững trong chú thích sang bộ mô tả triển khai XML, để giữ lại một POJO thuần túy thực sự. Nếu chi tiết ánh xạ lưu trữ bền vững không thường xuyên thay đổi, nhiều nhóm có thể chọn giữ chú thích, nhưng cách này có ít nhược điểm tai hại hơn hẳn so với sự xâm lấn của EJB2.

Khía cạnh AspectJ

Cuối cùng, công cụ đầy đủ tính năng nhất để tách biệt mối quan tâm bằng khía cạnh là ngôn ngữ AspectJ,[17] một phần mở rộng của Java hỗ trợ "hạng nhất" cho khía cạnh như cấu trúc mô-đun hóa. Các cách tiếp cận thuần Java của Spring AOP và JBoss AOP đủ dùng cho 80–90 phần trăm trường hợp khía cạnh hữu ích nhất. Tuy nhiên, AspectJ cung cấp bộ công cụ rất phong phú và mạnh mẽ để tách biệt mối quan tâm. Nhược điểm của AspectJ là phải tiếp nhận một số công cụ mới và học các cấu trúc ngôn ngữ cùng thành ngữ sử dụng mới.

Những trở ngại khi áp dụng đã phần nào được giảm nhẹ nhờ "dạng chú thích" AspectJ mới được giới thiệu gần đây, trong đó chú thích Java 5 được dùng để định nghĩa khía cạnh bằng mã Java thuần. Ngoài ra, Spring Framework có một số tính năng giúp nhóm ít kinh nghiệm với AspectJ dễ tích hợp khía cạnh dựa trên chú thích hơn.

Bàn đầy đủ về AspectJ nằm ngoài phạm vi cuốn sách này. Xem [AspectJ], [Colyer] và [Spring] để biết thêm thông tin.

Phát triển kiến trúc hệ thống theo hướng kiểm thử

Không thể nói quá về sức mạnh của việc tách biệt mối quan tâm bằng các cách tiếp cận giống khía cạnh. Nếu bạn có thể viết logic miền của ứng dụng bằng POJO, tách khỏi mọi mối quan tâm kiến trúc ở cấp mã, thì bạn có thể thực sự phát triển kiến trúc theo hướng kiểm thử. Khi cần, bạn có thể phát triển nó từ đơn giản đến tinh vi bằng cách áp dụng công nghệ mới theo yêu cầu. Không cần thiết phải Thiết kế Lớn Ngay Từ Đầu[18] (BDUF). Thực tế, BDUF còn có hại vì nó cản trở khả năng thích ứng với thay đổi, một phần do con người ngại vứt bỏ công sức đã bỏ ra, một phần do các lựa chọn kiến trúc ảnh hưởng đến suy nghĩ tiếp theo về thiết kế.

Kiến trúc sư xây dựng phải áp dụng BDUF vì không khả thi nếu thay đổi triệt để kiến trúc của một công trình vật lý lớn khi việc xây dựng đã tiến triển đáng kể.[19] Dù phần mềm cũng có quy luật vật lý riêng,[20] việc thay đổi triệt để vẫn khả thi về mặt kinh tế nếu cấu trúc phần mềm phân tách các mối quan tâm hiệu quả.

Điều này có nghĩa ta có thể bắt đầu một dự án phần mềm với kiến trúc "đơn giản đến ngây thơ" nhưng được tách rời tốt, nhanh chóng bàn giao các câu chuyện người dùng hoạt động được, rồi bổ sung thêm hạ tầng khi mở rộng quy mô. Một số trang web lớn nhất thế giới đạt được khả năng sẵn sàng và hiệu năng rất cao nhờ bộ nhớ đệm dữ liệu, bảo mật, ảo hóa tinh vi, vân vân. Tất cả đều được thực hiện hiệu quả và linh hoạt vì các thiết kế ít phụ thuộc được giữ đơn giản phù hợp ở từng mức trừu tượng và phạm vi.

Dĩ nhiên, điều này không có nghĩa ta bước vào dự án mà "không định hướng". Ta vẫn

---

17. Xem [AspectJ] và [Colyer].

18. Đừng nhầm với thực hành tốt là thiết kế sơ bộ. BDUF là thực hành thiết kế mọi thứ từ đầu trước khi triển khai bất cứ thứ gì.

19. Ngay cả sau khi bắt đầu xây dựng, người ta vẫn cần nhiều vòng khám phá và thảo luận chi tiết.

20. Thuật ngữ "vật lý phần mềm" lần đầu được [Kolence] sử dụng.

có những kỳ vọng về phạm vi, mục tiêu và tiến độ chung của dự án, cũng như cấu trúc tổng quát của hệ thống sẽ tạo ra. Tuy nhiên, ta phải duy trì khả năng đổi hướng để ứng phó với hoàn cảnh luôn thay đổi.

Kiến trúc EJB ban đầu chỉ là một trong nhiều API nổi tiếng được thiết kế quá mức và làm tổn hại sự phân tách mối quan tâm. Ngay cả API được thiết kế tốt cũng có thể quá mức cần thiết khi thực sự không cần đến. Phần lớn thời gian, một API tốt nên gần như biến mất khỏi tầm mắt để nhóm dành phần lớn nỗ lực sáng tạo tập trung vào các câu chuyện người dùng đang triển khai. Nếu không, các ràng buộc kiến trúc sẽ cản trở việc bàn giao giá trị tối ưu cho khách hàng một cách hiệu quả.

Tóm lại cuộc thảo luận dài này:

> *Kiến trúc hệ thống tối ưu gồm các miền mối quan tâm được mô-đun hóa, mỗi miền được triển khai bằng các đối tượng Plain Old Java (hoặc ngôn ngữ khác). Các miền khác nhau được tích hợp bằng khía cạnh hoặc công cụ giống khía cạnh, ít xâm lấn nhất có thể. Kiến trúc này có thể được phát triển theo hướng kiểm thử, giống như mã.*

Tối ưu việc ra quyết định

Tính mô-đun và sự phân tách mối quan tâm giúp có thể quản lý và ra quyết định phi tập trung. Trong một hệ thống đủ lớn, dù là thành phố hay dự án phần mềm, không một người nào có thể đưa ra mọi quyết định.

Ai cũng biết tốt nhất là giao trách nhiệm cho những người đủ năng lực nhất. Ta thường quên rằng cũng tốt nhất nếu trì hoãn quyết định đến thời điểm muộn nhất có thể. Đây không phải lười biếng hay vô trách nhiệm, mà giúp ta đưa ra lựa chọn có hiểu biết dựa trên thông tin tốt nhất có thể. Một quyết định quá sớm là quyết định được đưa ra khi hiểu biết chưa tối ưu. Nếu quyết định quá sớm, ta sẽ có ít phản hồi của khách hàng hơn, ít thời gian suy ngẫm về dự án hơn và ít kinh nghiệm hơn với những lựa chọn triển khai của mình.

> *Tính linh hoạt mà hệ thống POJO với các mối quan tâm được mô-đun hóa đem lại cho phép chúng ta đưa ra quyết định tối ưu, đúng lúc, dựa trên hiểu biết mới nhất. Độ phức tạp của những quyết định này cũng giảm đi.*

Dùng tiêu chuẩn một cách khôn ngoan, khi chúng đem lại giá trị rõ ràng

Xây dựng công trình là điều kỳ diệu để quan sát, vì tốc độ dựng nên những tòa nhà mới (ngay cả giữa mùa đông khắc nghiệt) và vì những thiết kế phi thường mà công nghệ ngày nay cho phép. Xây dựng là một ngành trưởng thành với các bộ phận, phương pháp và tiêu chuẩn được tối ưu hóa cao, hình thành qua áp lực suốt nhiều thế kỷ.

Nhiều nhóm đã dùng kiến trúc EJB2 chỉ vì nó là tiêu chuẩn, ngay cả khi những thiết kế nhẹ hơn và đơn giản hơn đã đủ dùng. Tôi từng thấy các nhóm bị ám ảnh bởi nhiều tiêu chuẩn được quảng bá rầm rộ và mất tập trung vào việc tạo ra giá trị cho khách hàng.

> *Tiêu chuẩn giúp tái sử dụng ý tưởng và thành phần dễ dàng hơn, tuyển dụng người có kinh nghiệm phù hợp, đóng gói các ý tưởng tốt và kết nối các thành phần. Tuy nhiên, quá trình tạo ra tiêu chuẩn đôi khi mất quá nhiều thời gian để ngành công nghiệp chờ đợi, và một số tiêu chuẩn không còn đáp ứng đúng nhu cầu thực tế của những người mà chúng nhắm đến phục vụ.*

Hệ thống cần ngôn ngữ chuyên biệt cho miền

Xây dựng công trình, cũng như hầu hết các miền khác, đã phát triển một ngôn ngữ phong phú với từ vựng, thành ngữ và mẫu thiết kế[21] truyền tải thông tin thiết yếu một cách rõ ràng, súc tích. Gần đây, người ta lại quan tâm đến việc tạo ra Ngôn ngữ chuyên biệt cho miền (Domain-Specific Language, DSL)[22] trong phần mềm. Đó là những ngôn ngữ kịch bản nhỏ, tách biệt, hoặc các API trong ngôn ngữ tiêu chuẩn, cho phép viết mã dưới dạng văn xuôi có cấu trúc giống như cách chuyên gia miền có thể diễn đạt.

Một DSL tốt thu hẹp "khoảng cách giao tiếp" giữa khái niệm miền và mã triển khai khái niệm đó, cũng như các thực hành Agile tối ưu hóa giao tiếp trong nhóm và với các bên liên quan của dự án. Nếu bạn triển khai logic miền bằng cùng ngôn ngữ mà chuyên gia miền sử dụng, nguy cơ dịch sai miền sang phần triển khai sẽ thấp hơn.

Khi được sử dụng hiệu quả, DSL nâng mức trừu tượng lên cao hơn các thành ngữ mã và mẫu thiết kế. Chúng cho phép nhà phát triển thể hiện ý định của mã ở mức trừu tượng thích hợp.

> *Ngôn ngữ chuyên biệt cho miền cho phép diễn đạt mọi mức trừu tượng và mọi miền trong ứng dụng dưới dạng POJO, từ chính sách cấp cao đến chi tiết cấp thấp.*

---

21. Công trình của [Alexander] có ảnh hưởng đặc biệt lớn đến cộng đồng phần mềm.

22. Ví dụ, xem [DSL]. [JMock] là một ví dụ hay về API Java tạo ra DSL.

Kết luận

Hệ thống cũng phải sạch. Một kiến trúc xâm lấn lấn át logic miền và làm suy giảm tính linh hoạt. Khi logic miền bị che khuất, chất lượng sẽ giảm vì lỗi dễ ẩn náu hơn và việc triển khai các câu chuyện người dùng trở nên khó khăn hơn. Nếu tính linh hoạt bị tổn hại, năng suất giảm và lợi ích của TDD mất đi.

Ở mọi mức trừu tượng, ý định phải rõ ràng. Điều này chỉ đạt được nếu bạn viết POJO và dùng các cơ chế giống khía cạnh để tích hợp những mối quan tâm triển khai khác mà không xâm lấn.

Dù thiết kế hệ thống hay từng mô-đun riêng lẻ, đừng bao giờ quên dùng cách đơn giản nhất có thể để giải quyết vấn đề.

Tài liệu tham khảo

[Alexander]: Christopher Alexander, A Timeless Way of Building, Oxford University Press, New York, 1979.

[AOSD]: Aspect-Oriented Software Development port, `http://aosd.net`

[ASM]: ASM Home Page, `http://asm.objectweb.org/`

[AspectJ]: `http://eclipse.org/aspectj`

[CGLIB]: Code Generation Library, `http://cglib.sourceforge.net/`

[Colyer]: Adrian Colyer, Andy Clement, George Hurley, Mathew Webster, Eclipse AspectJ, Person Education, Inc., Upper Saddle River, NJ, 2005.

[DSL]: Ngôn ngữ lập trình chuyên biệt cho miền, `http://en.wikipedia.org/wiki/Domain-specific_programming_language`

[Fowler]: Inversion of Control Containers and the Dependency Injection pattern, `http://martinfowler.com/articles/injection.html`

[Goetz]: Brian Goetz, Java Theory and Practice: Decorating with Dynamic Proxies, `http://www.ibm.com/developerworks/java/library/j-jtp08305.html`

[Javassist]: Javassist Home Page, `http://www.csg.is.titech.ac.jp/~chiba/javassist/`

[JBoss]: JBoss Home Page, `http://jboss.org`

[JMock]: JMock, thư viện đối tượng giả nhẹ cho Java, `http://jmock.org`

[Kolence]: Kenneth W. Kolence, Software physics and computer performance measurements, Proceedings of the ACM annual conference, Volume 2, Boston, Massachusetts, pp. 1024–1040, 1972.

[Spring]: The Spring Framework, `http://www.springframework.org`

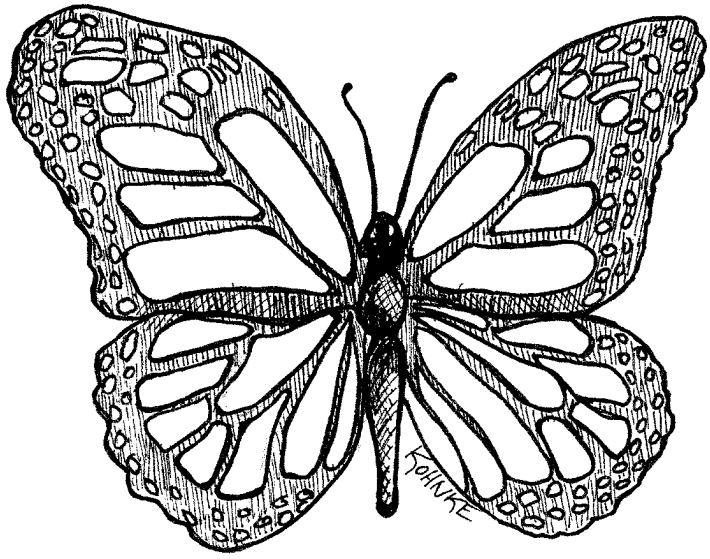
[Mezzaros07]: XUnit Patterns, Gerard Mezzaros, Addison-Wesley, 2007.

[GOF]: Design Patterns: Elements of Reusable Object Oriented Software, Gamma et al., Addison-Wesley, 1996.

12

Thiết kế nổi lên

Tác giả: Jeff Langr



Thiết kế sạch nhờ thiết kế nổi lên

Nếu có bốn quy tắc đơn giản mà bạn có thể làm theo để tạo ra những thiết kế tốt trong quá trình làm việc thì sao? Nếu nhờ làm theo các quy tắc đó, bạn hiểu sâu hơn về cấu trúc và thiết kế của mã, từ đó dễ áp dụng những nguyên tắc như SRP và DIP hơn thì sao? Nếu bốn quy tắc này giúp những thiết kế tốt tự hình thành thì sao?

Nhiều người trong chúng ta thấy bốn quy tắc Thiết kế Đơn giản của Kent Beck[1] rất hữu ích để tạo ra phần mềm có thiết kế tốt.

1. [XPE].

Theo Kent, một thiết kế là "đơn giản" nếu tuân theo các quy tắc sau:

- Chạy qua tất cả các kiểm thử
- Không có trùng lặp
- Thể hiện ý định của lập trình viên
- Giảm thiểu số lượng lớp và phương thức

Các quy tắc được liệt kê theo thứ tự quan trọng.

Quy tắc Thiết kế Đơn giản 1: Chạy qua tất cả các kiểm thử

Trước hết, thiết kế phải tạo ra một hệ thống hoạt động đúng như dự định. Trên giấy tờ, hệ thống có thể có thiết kế hoàn hảo. Nhưng nếu không có cách đơn giản nào để xác minh hệ thống thực sự hoạt động như dự định thì mọi công sức trên giấy đều đáng ngờ.

Một hệ thống được kiểm thử toàn diện và luôn vượt qua tất cả các kiểm thử là một hệ thống có thể kiểm thử được. Điều đó nghe hiển nhiên, nhưng rất quan trọng. Hệ thống không thể kiểm thử thì không thể xác minh. Có thể nói, một hệ thống không thể xác minh thì không bao giờ nên được triển khai.

May mắn thay, việc làm cho hệ thống có thể kiểm thử được sẽ thúc đẩy chúng ta hướng tới thiết kế có các lớp nhỏ, mỗi lớp chỉ đảm nhận một mục đích. Kiểm thử các lớp tuân theo SRP dễ hơn nhiều. Càng viết nhiều kiểm thử, chúng ta càng hướng tới những thứ dễ kiểm thử hơn. Vì vậy, bảo đảm hệ thống có thể được kiểm thử đầy đủ giúp chúng ta tạo ra thiết kế tốt hơn.

Sự phụ thuộc chặt chẽ khiến việc viết kiểm thử trở nên khó khăn. Do đó, tương tự như trên, càng viết nhiều kiểm thử, chúng ta càng áp dụng những nguyên tắc như DIP và các công cụ như tiêm phụ thuộc, giao diện và trừu tượng hóa để giảm thiểu sự phụ thuộc. Nhờ vậy, thiết kế của chúng ta càng được cải thiện.

Thật đáng kinh ngạc, chỉ cần làm theo một quy tắc đơn giản, hiển nhiên rằng chúng ta cần có kiểm thử và chạy chúng liên tục, hệ thống sẽ tuân thủ tốt hơn các mục tiêu OO cốt lõi: sự phụ thuộc thấp và độ gắn kết cao. Viết kiểm thử dẫn đến thiết kế tốt hơn.

Quy tắc Thiết kế Đơn giản 2–4: Tái cấu trúc

Khi đã có kiểm thử, chúng ta có thể giữ cho mã và các lớp luôn sạch. Ta làm điều đó bằng cách tái cấu trúc mã từng bước. Cứ viết thêm vài dòng mã, ta lại dừng và suy ngẫm về thiết kế mới. Ta vừa làm thiết kế xuống cấp ư? Nếu có, ta dọn dẹp rồi chạy kiểm thử để chứng minh rằng mình chưa làm hỏng gì. Có sẵn những kiểm thử này sẽ xua tan nỗi lo rằng việc dọn dẹp mã sẽ làm hỏng nó!

Trong bước tái cấu trúc này, chúng ta có thể áp dụng mọi kiến thức về thiết kế phần mềm tốt. Ta có thể tăng độ gắn kết, giảm sự phụ thuộc, tách biệt các mối quan tâm, mô-đun hóa những mối quan tâm của hệ thống, thu nhỏ hàm và lớp, chọn tên tốt hơn, vân vân. Đây cũng là lúc áp dụng ba quy tắc cuối của thiết kế đơn giản: loại bỏ trùng lặp, bảo đảm tính biểu đạt và giảm thiểu số lượng lớp cùng phương thức.

Không trùng lặp

Trùng lặp là kẻ thù hàng đầu của một hệ thống được thiết kế tốt. Nó tạo thêm công việc, thêm rủi ro và thêm độ phức tạp không cần thiết. Trùng lặp xuất hiện dưới nhiều hình thức. Dĩ nhiên, những dòng mã trông giống hệt nhau là trùng lặp. Những dòng mã tương tự nhau thường có thể được chỉnh sửa để trông giống nhau hơn, nhờ đó dễ tái cấu trúc hơn. Trùng lặp cũng có thể tồn tại dưới những dạng khác, chẳng hạn trùng lặp trong cách triển khai. Ví dụ, trong một lớp tập hợp, ta có thể có hai phương thức:

```
int size() {}
boolean isEmpty() {}
```

Ta có thể triển khai riêng từng phương thức. Phương thức `isEmpty` có thể theo dõi một giá trị boolean, còn `size` theo dõi một bộ đếm. Hoặc ta có thể loại bỏ phần trùng lặp bằng cách định nghĩa `isEmpty` dựa trên `size`:

```
boolean isEmpty() {
   return 0 == size();
}
```

Muốn tạo ra một hệ thống sạch, ta cần quyết tâm loại bỏ trùng lặp, dù chỉ là vài dòng mã. Chẳng hạn, hãy xem đoạn mã sau:

```
public void scaleToOneDimension(
     float desiredDimension, float imageDimension) {
  if (Math.abs(desiredDimension - imageDimension) < errorThreshold)
     return;
  float scalingFactor = desiredDimension / imageDimension;
  scalingFactor = (float)(Math.floor(scalingFactor * 100) * 0.01f);

   RenderedOp newImage = ImageUtilities.getScaledImage(
      image, scalingFactor, scalingFactor);
   image.dispose();
   System.gc();
   image = newImage;
}
public synchronized void rotate(int degrees) {
  RenderedOp newImage = ImageUtilities.getRotatedImage(
     image, degrees);
  image.dispose();
  System.gc();
  image = newImage;
}
```

Để giữ hệ thống sạch, ta nên loại bỏ phần trùng lặp nhỏ giữa các phương thức `scaleToOneDimension` và `rotate`:

```
public void scaleToOneDimension(
     float desiredDimension, float imageDimension) {
  if (Math.abs(desiredDimension - imageDimension) < errorThreshold)
     return;
```

```
    float scalingFactor = desiredDimension / imageDimension;
    scalingFactor = (float)(Math.floor(scalingFactor * 100) * 0.01f);

    replaceImage(ImageUtilities.getScaledImage(
       image, scalingFactor, scalingFactor));
  }

  public synchronized void rotate(int degrees) {
     replaceImage(ImageUtilities.getRotatedImage(image, degrees));
  }
  private void replaceImage(RenderedOp newImage) {
     image.dispose();
     System.gc();
     image = newImage;
  }
```

Khi tách ra điểm chung ở mức rất nhỏ như vậy, ta bắt đầu nhận ra những vi phạm SRP. Vì thế, có thể ta sẽ chuyển phương thức vừa tách ra sang một lớp khác. Việc đó khiến phương thức dễ được nhìn thấy hơn. Một thành viên khác trong nhóm có thể nhận ra cơ hội trừu tượng hóa thêm phương thức mới và tái sử dụng nó trong một ngữ cảnh khác. Kiểu "tái sử dụng ở quy mô nhỏ" này có thể giảm đáng kể độ phức tạp của hệ thống. Hiểu cách đạt được sự tái sử dụng ở quy mô nhỏ là điều thiết yếu để đạt được sự tái sử dụng ở quy mô lớn.

Mẫu TEMPLATE METHOD[2] là một kỹ thuật phổ biến để loại bỏ trùng lặp ở cấp độ cao hơn. Ví dụ:

```
public class VacationPolicy {
   public void accrueUSDivisionVacation() {
      // code to calculate vacation based on hours worked to date
      // ...
      // code to ensure vacation meets US minimums
      // ...
      // code to apply vaction to payroll record
      // ...
   }

   public void accrueEUDivisionVacation() {
      // code to calculate vacation based on hours worked to date
      // ...
      // code to ensure vacation meets EU minimums
      // ...
      // code to apply vaction to payroll record
      // ...
   }
}
```

Mã trong `accrueUSDivisionVacation` và `accrueEuropeanDivisionVacation` phần lớn giống nhau, ngoại trừ phần tính các mức tối thiểu theo luật. Phần đó của thuật toán thay đổi tùy theo loại nhân viên.

Ta có thể loại bỏ phần trùng lặp rõ ràng bằng cách áp dụng mẫu TEMPLATE METHOD.

---

2. [GOF].

```
abstract public class VacationPolicy {
   public void accrueVacation() {
      calculateBaseVacationHours();

      alterForLegalMinimums();
      applyToPayroll();
   }

   private void calculateBaseVacationHours() { /* ... */ };
   abstract protected void alterForLegalMinimums();
   private void applyToPayroll() { /* ... */ };
}

public class USVacationPolicy extends VacationPolicy {
   @Override protected void alterForLegalMinimums() {
      // US specific logic
   }
}
public class EUVacationPolicy extends VacationPolicy {
   @Override protected void alterForLegalMinimums() {
      // EU specific logic
   }
}
```

Các lớp con lấp vào "chỗ trống" trong thuật toán `accrueVacation`, cung cấp những phần thông tin duy nhất không bị trùng lặp.

Biểu đạt

Hầu hết chúng ta từng trải qua cảm giác phải làm việc với mã rối rắm. Nhiều người trong chúng ta cũng từng viết ra những đoạn mã rối rắm như thế. Viết mã mà ta hiểu thì dễ, bởi lúc viết, ta đang hiểu sâu vấn đề mình tìm cách giải quyết. Những người bảo trì mã sau này sẽ không hiểu sâu đến vậy.

Phần lớn chi phí của một dự án phần mềm phát sinh trong giai đoạn bảo trì lâu dài. Để giảm thiểu nguy cơ phát sinh lỗi khi thay đổi hệ thống, chúng ta cần hiểu hệ thống làm gì. Hệ thống càng phức tạp thì nhà phát triển càng mất nhiều thời gian để hiểu, và nguy cơ hiểu sai cũng càng lớn. Vì vậy, mã phải thể hiện rõ ý định của người viết. Người viết làm cho mã càng rõ ràng thì người khác càng mất ít thời gian để hiểu. Nhờ đó, lỗi sẽ giảm và chi phí bảo trì cũng thấp hơn.

Bạn có thể thể hiện suy nghĩ của mình bằng cách chọn tên tốt. Ta muốn nghe tên một lớp hay một hàm mà không thấy bất ngờ khi phát hiện ra trách nhiệm của nó.

Bạn cũng có thể thể hiện suy nghĩ của mình bằng cách giữ cho hàm và lớp nhỏ gọn. Các lớp và hàm nhỏ thường dễ đặt tên, dễ viết và dễ hiểu.

Bạn còn có thể thể hiện suy nghĩ của mình bằng cách dùng thuật ngữ chuẩn. Chẳng hạn, các mẫu thiết kế phần lớn nhằm mục đích giao tiếp và biểu đạt. Khi dùng tên mẫu chuẩn như COMMAND hay VISITOR trong tên các lớp triển khai những mẫu đó, bạn có thể mô tả ngắn gọn thiết kế của mình với các nhà phát triển khác.

Các kiểm thử đơn vị được viết tốt cũng có tính biểu đạt. Một mục tiêu chính của kiểm

thử là làm tài liệu thông qua ví dụ. Người đọc kiểm thử của chúng ta phải nhanh chóng hiểu được một lớp làm những gì.

Nhưng cách quan trọng nhất để biểu đạt là cố gắng làm điều đó. Quá thường xuyên, chúng ta làm cho mã chạy được rồi chuyển sang vấn đề tiếp theo mà không suy nghĩ đủ để giúp người kế tiếp dễ đọc mã. Hãy nhớ, người có khả năng đọc mã tiếp theo nhất chính là bạn.

Vì vậy, hãy tự hào một chút về tay nghề của mình. Dành chút thời gian cho từng hàm và lớp. Chọn tên tốt hơn, tách hàm lớn thành các hàm nhỏ hơn, và nói chung hãy chăm chút cho những gì mình tạo ra. Sự chăm chút là nguồn lực quý giá.

Tối thiểu hóa số lớp và phương thức

Ngay cả những khái niệm nền tảng như loại bỏ trùng lặp, tính biểu đạt của mã và SRP cũng có thể bị đẩy đi quá xa. Trong nỗ lực làm cho lớp và phương thức nhỏ gọn, ta có thể tạo ra quá nhiều lớp và phương thức vụn vặt. Vì thế, quy tắc này cũng khuyên ta giữ số lượng hàm và lớp ở mức thấp.

Số lượng lớp và phương thức lớn đôi khi là kết quả của chủ nghĩa giáo điều vô ích. Chẳng hạn, hãy nghĩ đến một tiêu chuẩn lập trình yêu cầu tạo giao diện cho mọi lớp. Hoặc nghĩ đến những nhà phát triển khăng khăng rằng trường dữ liệu và hành vi luôn phải tách riêng thành các lớp dữ liệu và lớp hành vi. Ta nên chống lại giáo điều như vậy và chọn cách tiếp cận thực tế hơn.

Mục tiêu của chúng ta là giữ cho toàn bộ hệ thống nhỏ gọn, đồng thời giữ cho các hàm và lớp cũng nhỏ gọn. Tuy nhiên, hãy nhớ đây là quy tắc có mức ưu tiên thấp nhất trong bốn quy tắc Thiết kế Đơn giản. Vì vậy, dù việc giữ số lượng lớp và hàm ở mức thấp là quan trọng, có kiểm thử, loại bỏ trùng lặp và thể hiện suy nghĩ của mình còn quan trọng hơn.

Kết luận

Có tập hợp những thực hành đơn giản nào có thể thay thế kinh nghiệm không? Rõ ràng là không. Mặt khác, những thực hành được mô tả trong chương này và trong cuốn sách này là kết tinh từ nhiều thập kỷ kinh nghiệm của các tác giả. Thực hành thiết kế đơn giản có thể và thực sự khuyến khích, giúp các nhà phát triển tuân theo những nguyên tắc và mẫu tốt mà nếu không thì phải mất nhiều năm mới học được.

Tài liệu tham khảo

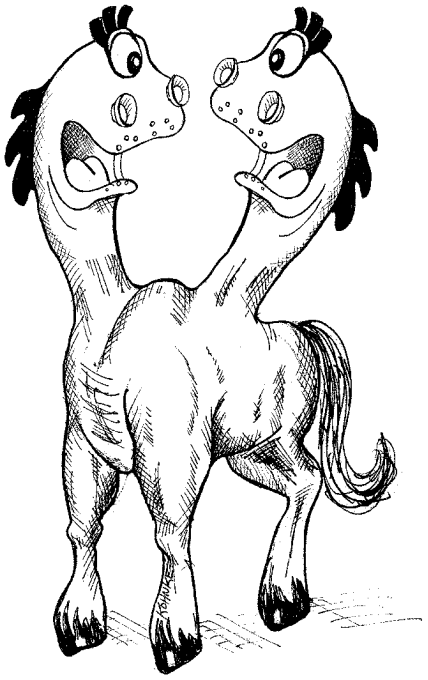
[XPE]: *Extreme Programming Explained: Embrace Change*, Kent Beck, Addison-Wesley, 1999.

[GOF]: *Design Patterns: Elements of Reusable Object Oriented Software*, Gamma et al., Addison-Wesley, 1996.

13

Xử lý đồng thời

Tác giả: Brett L. Schuchert



"Objects are abstractions of processing. Threads are abstractions of schedule."
—James O. Coplien[1]

Viết chương trình xử lý đồng thời sạch rất khó, cực kỳ khó. Viết mã chạy trên một luồng

1. Trao đổi riêng.

dễ hơn nhiều. Cũng rất dễ viết mã đa luồng trông có vẻ ổn trên bề mặt nhưng lại sai ở tầng sâu hơn. Mã như vậy vẫn chạy tốt cho đến khi hệ thống chịu tải.

Trong chương này, chúng ta bàn về nhu cầu xử lý đồng thời và những khó khăn mà nó đặt ra. Sau đó, chúng ta đưa ra một số khuyến nghị để giải quyết các khó khăn ấy và viết mã đồng thời sạch. Cuối cùng, chúng ta kết luận bằng những vấn đề liên quan đến kiểm thử mã đồng thời.

Xử lý đồng thời sạch là một chủ đề phức tạp, xứng đáng có riêng một cuốn sách. Trong cuốn sách này, chúng tôi sẽ trình bày tổng quan ở đây và cung cấp hướng dẫn chi tiết hơn trong “Concurrency II” ở trang 317. Nếu bạn chỉ tò mò về xử lý đồng thời, chương này hiện tại là đủ. Nếu cần hiểu sâu hơn, bạn cũng nên đọc phần hướng dẫn đó.

Vì sao cần xử lý đồng thời?

Xử lý đồng thời là một chiến lược tách biệt việc gì được làm khỏi thời điểm nó được làm. Trong các ứng dụng đơn luồng, việc gì và khi nào được thực hiện gắn kết chặt đến mức ta thường có thể xác định trạng thái của toàn bộ ứng dụng bằng cách xem dấu vết ngăn xếp. Lập trình viên gỡ lỗi một hệ thống như vậy có thể đặt một điểm dừng hoặc một chuỗi điểm dừng, rồi biết trạng thái hệ thống dựa vào những điểm dừng nào được chạm tới.

Tách việc gì khỏi thời điểm thực hiện có thể cải thiện đáng kể cả thông lượng lẫn cấu trúc của ứng dụng. Xét về cấu trúc, ứng dụng trông giống nhiều máy tính nhỏ phối hợp với nhau hơn là một vòng lặp chính khổng lồ. Nhờ đó hệ thống dễ hiểu hơn và ta có một số cách mạnh mẽ để tách biệt các mối quan tâm.

Chẳng hạn, hãy xem mô hình “Servlet” tiêu chuẩn của các ứng dụng Web. Những hệ thống này chạy trong một bộ chứa Web hoặc EJB, bộ chứa đó quản lý một phần việc xử lý đồng thời thay bạn. Các servlet được thực thi bất đồng bộ mỗi khi có yêu cầu Web đến. Lập trình viên servlet không phải quản lý tất cả yêu cầu đến. Về nguyên tắc, mỗi lần thực thi servlet sống trong thế giới nhỏ riêng của nó và tách biệt với mọi lần thực thi servlet khác.

Dĩ nhiên, nếu mọi chuyện dễ dàng đến thế thì chương này đã không cần thiết. Trên thực tế, mức độ tách biệt mà các bộ chứa Web cung cấp còn lâu mới hoàn hảo. Lập trình viên servlet phải hiểu rõ và hết sức cẩn thận để bảo đảm chương trình đồng thời của mình đúng. Dù vậy, lợi ích về cấu trúc của mô hình servlet vẫn đáng kể.

Nhưng cấu trúc không phải động lực duy nhất để áp dụng xử lý đồng thời. Một số hệ thống có yêu cầu về thời gian phản hồi và thông lượng, buộc ta phải tự viết giải pháp đồng thời. Ví dụ, hãy xét một bộ tổng hợp thông tin đơn luồng lấy thông tin từ nhiều trang Web khác nhau rồi gộp lại thành bản tóm tắt hằng ngày. Vì hệ thống này chỉ có một luồng, nó lần lượt truy cập từng trang Web, luôn hoàn thành trang này trước khi bắt đầu trang tiếp theo. Lượt chạy hằng ngày phải hoàn tất trong chưa đầy 24 giờ. Tuy nhiên, khi ngày càng có nhiều trang Web được bổ sung, thời gian chạy tăng lên đến mức cần hơn 24 giờ mới thu thập hết dữ liệu. Luồng đơn phải chờ rất nhiều ở các socket Web cho đến khi thao tác I/O hoàn tất. Ta có thể cải thiện hiệu năng bằng thuật toán đa luồng, truy cập nhiều trang Web cùng lúc.

Hoặc hãy xét một hệ thống chỉ xử lý mỗi lần một người dùng và cần đúng một giây cho

mỗi người. Hệ thống phản hồi khá nhanh khi chỉ có vài người dùng, nhưng khi số người dùng tăng, thời gian phản hồi cũng tăng. Không ai muốn xếp hàng sau 150 người khác! Ta có thể cải thiện thời gian phản hồi của hệ thống bằng cách xử lý đồng thời nhiều người dùng.

Hoặc xét hệ thống diễn giải các tập dữ liệu lớn nhưng chỉ đưa ra được lời giải hoàn chỉnh sau khi xử lý tất cả. Có thể xử lý mỗi tập dữ liệu trên một máy tính riêng, để nhiều tập được xử lý song song.

Những lầm tưởng và quan niệm sai

Vậy là có những lý do thuyết phục để áp dụng xử lý đồng thời. Tuy nhiên, như đã nói, xử lý đồng thời rất khó. Nếu không cẩn thận, bạn có thể tạo ra những tình huống hết sức tai hại. Hãy xem những lầm tưởng và quan niệm sai thường gặp sau:

- Xử lý đồng thời luôn cải thiện hiệu năng.
  Đôi khi xử lý đồng thời có thể cải thiện hiệu năng, nhưng chỉ khi có nhiều thời gian chờ để chia sẻ giữa nhiều luồng hoặc nhiều bộ xử lý. Cả hai tình huống này đều không đơn giản.
- Thiết kế không thay đổi khi viết chương trình đồng thời.
  Trên thực tế, thiết kế thuật toán đồng thời có thể khác đáng kể với thiết kế hệ thống đơn luồng. Việc tách biệt điều gì được làm khỏi thời điểm thực hiện thường ảnh hưởng rất lớn đến cấu trúc hệ thống.
- Khi làm việc với bộ chứa như Web hoặc EJB, không cần hiểu các vấn đề về xử lý đồng thời.
  Thực ra, bạn cần biết chính xác bộ chứa đang làm gì và cách bảo vệ chương trình trước các vấn đề cập nhật đồng thời và bế tắc được mô tả ở phần sau của chương này.

Sau đây là một số nhận định cân bằng hơn về việc viết phần mềm đồng thời:

- Xử lý đồng thời phát sinh chi phí phụ, cả về hiệu năng lẫn lượng mã phải viết thêm.
- Xử lý đồng thời đúng rất phức tạp, kể cả với những vấn đề đơn giản.
- Lỗi đồng thời thường không tái hiện được, vì thế người ta hay xem chúng như sự cố nhất thời[2] thay vì lỗi thực sự.
- Xử lý đồng thời thường đòi hỏi thay đổi căn bản trong chiến lược thiết kế.

Thách thức

Điều gì khiến lập trình đồng thời khó đến vậy? Hãy xem lớp đơn giản sau:

```
public class X {
   private int lastIdUsed;

    public int getNextId() {
```

2. Tia vũ trụ, trục trặc và những thứ tương tự.

```
            return ++lastIdUsed;
        }
    }
```

Giả sử ta tạo một thể hiện của `X`, đặt trường `lastIdUsed` thành 42 rồi chia sẻ thể hiện đó cho hai luồng. Bây giờ, giả sử cả hai luồng đều gọi phương thức `getNextId()`. Có ba kết quả có thể xảy ra:

- Luồng một nhận giá trị 43, luồng hai nhận giá trị 44, `lastIdUsed` là 44.
- Luồng một nhận giá trị 44, luồng hai nhận giá trị 43, `lastIdUsed` là 44.
- Luồng một nhận giá trị 43, luồng hai nhận giá trị 43, `lastIdUsed` là 43.

Kết quả thứ ba đáng ngạc nhiên[3] xảy ra khi hai luồng giẫm chân lên nhau. Điều này xảy ra vì có nhiều đường thực thi khả dĩ mà hai luồng có thể đi qua dòng mã Java duy nhất đó, và một số đường dẫn đến kết quả sai. Có bao nhiêu đường khác nhau? Để trả lời chính xác, ta cần hiểu trình biên dịch Just-In-Time làm gì với mã byte được sinh ra, cũng như mô hình bộ nhớ Java xem thao tác nào là nguyên tử.

Câu trả lời nhanh, chỉ xét mã byte đã sinh, là có 12.870 đường thực thi khả dĩ[4] cho hai luồng cùng thực thi trong phương thức `getNextId`. Nếu đổi kiểu của `lastIdUsed` từ `int` sang `long`, số đường khả dĩ tăng lên 2.704.156. Dĩ nhiên, phần lớn các đường đó cho kết quả hợp lệ. Vấn đề là một số đường thì không.

Các nguyên tắc phòng vệ khi xử lý đồng thời

Sau đây là một loạt nguyên tắc và kỹ thuật giúp bảo vệ hệ thống khỏi các vấn đề do mã đồng thời gây ra.

---

3. Xem “Digging Deeper” *ở trang 323*.

4. Xem “Possible Paths of Execution” *ở trang 321*.

Nguyên tắc trách nhiệm đơn

SRP[5] quy định rằng một phương thức, lớp hoặc thành phần nhất định chỉ nên có một lý do để thay đổi. Thiết kế xử lý đồng thời đủ phức tạp để tự nó trở thành một lý do thay đổi, vì vậy cần được tách khỏi phần còn lại của mã. Đáng tiếc, chi tiết triển khai xử lý đồng thời thường bị nhúng thẳng vào mã sản phẩm. Hãy cân nhắc một số điều sau:

- Mã liên quan đến xử lý đồng thời có vòng đời phát triển, thay đổi và tinh chỉnh riêng.
- Mã liên quan đến xử lý đồng thời có những thách thức riêng, khác với mã không liên quan, và thường khó hơn.
- Số cách mã đồng thời viết sai có thể thất bại đã đủ gây khó khăn, chưa kể gánh nặng từ mã ứng dụng xung quanh.

Khuyến nghị: Hãy tách mã xử lý đồng thời khỏi phần mã còn lại.[6]

Hệ quả: Giới hạn phạm vi dữ liệu

Như đã thấy, hai luồng cùng sửa một trường trong đối tượng dùng chung có thể cản trở lẫn nhau và gây ra hành vi bất ngờ. Một giải pháp là dùng từ khóa `synchronized` để bảo vệ vùng tới hạn trong mã sử dụng đối tượng dùng chung. Điều quan trọng là hạn chế số lượng vùng tới hạn như vậy. Càng có nhiều nơi có thể cập nhật dữ liệu dùng chung thì càng có khả năng:

- Bạn quên bảo vệ một hoặc nhiều nơi, khiến toàn bộ mã sửa dữ liệu dùng chung đó mất an toàn.
- Phải lặp lại công sức để bảo đảm mọi nơi đều được bảo vệ hiệu quả, vi phạm DRY[7].
- Khó xác định nguồn gốc lỗi, vốn đã đủ khó tìm.

Khuyến nghị: Hãy coi trọng việc đóng gói dữ liệu; hạn chế nghiêm ngặt quyền truy cập vào mọi dữ liệu có thể được chia sẻ.

Hệ quả: Dùng bản sao dữ liệu

Một cách hay để tránh dữ liệu dùng chung là ngay từ đầu đừng chia sẻ dữ liệu. Trong một số tình huống, ta có thể sao chép đối tượng rồi coi chúng là chỉ đọc. Trường hợp khác, ta có thể sao chép đối tượng, thu thập kết quả từ nhiều luồng trong các bản sao này, rồi gộp kết quả bằng một luồng duy nhất.

Nếu có cách dễ dàng để tránh chia sẻ đối tượng, mã tạo ra sẽ ít có khả năng phát sinh vấn đề hơn nhiều. Có thể bạn lo ngại chi phí tạo thêm đối tượng. Hãy thử nghiệm để xem liệu đây có thực sự là vấn đề hay không. Tuy nhiên, nếu dùng bản sao đối tượng giúp mã tránh phải đồng bộ hóa, phần tiết kiệm được nhờ không phải dùng khóa nội tại có thể bù cho chi phí tạo đối tượng và thu gom rác bổ sung.

---

5. [PPP]
6. Xem “Client/Server Example” ở trang 317.
7. [PRAG].

Hệ quả: Các luồng nên độc lập tối đa

Hãy viết mã đa luồng sao cho mỗi luồng tồn tại trong thế giới riêng, không chia sẻ dữ liệu với luồng nào khác. Mỗi luồng xử lý một yêu cầu của máy khách, nhận toàn bộ dữ liệu cần thiết từ nguồn không dùng chung và lưu vào biến cục bộ. Như vậy, mỗi luồng hoạt động như thể nó là luồng duy nhất trên đời và không có yêu cầu đồng bộ hóa nào.

Ví dụ, các lớp kế thừa `HttpServlet` nhận toàn bộ thông tin dưới dạng tham số truyền vào các phương thức `doGet` và `doPost`. Điều này khiến mỗi Servlet hoạt động như thể nó có máy riêng. Chừng nào mã trong Servlet chỉ dùng biến cục bộ, Servlet sẽ không thể gây ra vấn đề đồng bộ hóa. Dĩ nhiên, phần lớn ứng dụng dùng Servlet rồi cũng phải làm việc với tài nguyên dùng chung như kết nối cơ sở dữ liệu.

Khuyến nghị: Hãy thử phân vùng dữ liệu thành các tập con độc lập để các luồng độc lập, có thể chạy trên những bộ xử lý khác nhau, xử lý chúng.

Hiểu rõ thư viện của bạn

Java 5 có nhiều cải tiến hỗ trợ phát triển đồng thời so với các phiên bản trước. Khi viết mã đa luồng trong Java 5, cần cân nhắc một số điều:

- Dùng các collection an toàn luồng được cung cấp sẵn.
- Dùng framework executor để thực thi các tác vụ không liên quan với nhau.
- Khi có thể, hãy dùng giải pháp không chặn.
- Một số lớp thư viện không an toàn luồng.

Collection an toàn luồng

Khi Java còn non trẻ, Doug Lea đã viết cuốn sách có ảnh hưởng lớn[8] *Concurrent Programming in Java*. Cùng với cuốn sách, ông phát triển một số collection an toàn luồng; sau này chúng trở thành một phần của JDK trong gói `java.util.concurrent`. Các collection trong gói này an toàn trong môi trường đa luồng và hoạt động hiệu quả. Trên thực tế, cách triển khai `ConcurrentHashMap` hoạt động tốt hơn `HashMap` trong gần như mọi tình huống. Nó cũng cho phép đọc và ghi đồng thời cùng lúc, đồng thời có các phương thức hỗ trợ những thao tác kết hợp phổ biến vốn không an toàn luồng. Nếu môi trường triển khai là Java 5, hãy bắt đầu với `ConcurrentHashMap`.

Một số lớp khác cũng được bổ sung để hỗ trợ thiết kế xử lý đồng thời nâng cao. Sau đây là vài ví dụ:

Lớp	Mô tả
`ReentrantLock`	Một khóa có thể được lấy trong phương thức này và giải phóng trong phương thức khác.
`Semaphore`	Cách triển khai semaphore kinh điển, một khóa có bộ đếm.
`CountDownLatch`	Một khóa chờ một số sự kiện xảy ra rồi mới giải phóng tất cả các luồng đang chờ. Nhờ vậy mọi luồng đều có cơ hội công bằng để bắt đầu gần như cùng lúc.

8. [Lea99].

Khuyến nghị: Hãy xem lại các lớp hiện có. Với Java, hãy làm quen với `java.util.concurrent`, `java.util.concurrent.atomic`, `java.util.concurrent.locks`.

Hiểu rõ các mô hình thực thi

Có nhiều cách khác nhau để phân chia hành vi trong ứng dụng đồng thời. Để bàn về chúng, ta cần hiểu một số định nghĩa cơ bản.

Thuật ngữ	Định nghĩa
Tài nguyên có giới hạn	Tài nguyên có kích thước hoặc số lượng cố định được dùng trong môi trường đồng thời. Ví dụ gồm kết nối cơ sở dữ liệu và bộ đệm đọc/ghi kích thước cố định.
Loại trừ tương hỗ	Tại một thời điểm, chỉ một luồng có thể truy cập dữ liệu hoặc tài nguyên dùng chung.
Nghẽn tài nguyên	Một luồng hoặc một nhóm luồng không thể tiếp tục trong thời gian quá lâu hoặc mãi mãi. Ví dụ, nếu luôn cho các luồng chạy nhanh đi trước mà chúng không bao giờ hết, các luồng chạy chậm hơn có thể bị bỏ đói.
Bế tắc (deadlock)	Hai hoặc nhiều luồng chờ nhau hoàn tất. Mỗi luồng giữ một tài nguyên mà luồng kia cần, và không luồng nào có thể hoàn tất cho đến khi nhận được tài nguyên kia.
Bế tắc sống (livelock)	Các luồng đi cùng nhịp, mỗi luồng cố làm việc nhưng lại gặp luồng khác "cản đường". Do cộng hưởng, các luồng tiếp tục cố tiến lên nhưng không thể tiến triển trong thời gian quá lâu, hoặc mãi mãi.

Dựa trên các định nghĩa này, giờ ta có thể bàn về những mô hình thực thi khác nhau được dùng trong lập trình đồng thời.

Nhà sản xuất-Người tiêu thụ[9]

Một hoặc nhiều luồng nhà sản xuất tạo ra công việc rồi đặt vào bộ đệm hoặc hàng đợi. Một hoặc nhiều luồng người tiêu thụ lấy công việc khỏi hàng đợi và hoàn thành nó. Hàng đợi giữa nhà sản xuất và người tiêu thụ là tài nguyên có giới hạn. Điều này có nghĩa nhà sản xuất phải chờ hàng đợi có chỗ trống trước khi ghi, còn người tiêu thụ phải chờ đến khi có thứ để tiêu thụ. Việc phối hợp thông qua hàng đợi đòi hỏi nhà sản xuất và người tiêu thụ báo hiệu cho nhau. Nhà sản xuất ghi vào hàng đợi rồi báo rằng hàng đợi không còn trống. Người tiêu thụ đọc khỏi hàng đợi rồi báo rằng hàng đợi không còn đầy. Cả hai phía có thể phải chờ được thông báo mới tiếp tục được.

Người đọc-Người ghi[10]

Khi có một tài nguyên dùng chung chủ yếu cung cấp thông tin cho người đọc nhưng đôi lúc được người ghi cập nhật, thông lượng là vấn đề cần quan tâm. Ưu tiên thông lượng có thể gây nghẽn tài nguyên và tích tụ thông tin cũ. Cho phép cập nhật có thể ảnh hưởng đến thông lượng. Việc điều phối người đọc để họ không đọc khi người ghi đang cập nhật, và ngược lại, là một bài toán cân bằng khó. Người ghi thường chặn nhiều người đọc trong thời

9. `http://en.wikipedia.org/wiki/Producer-consumer`

10. `http://en.wikipedia.org/wiki/Readers-writers_problem`

gian dài, gây vấn đề về thông lượng.

Thách thức là cân bằng nhu cầu của cả người đọc lẫn người ghi để bảo đảm hoạt động đúng, đạt thông lượng hợp lý và tránh nghẽn tài nguyên. Một chiến lược đơn giản buộc người ghi chờ đến khi không còn người đọc nào rồi mới cho phép cập nhật. Tuy nhiên, nếu người đọc liên tục xuất hiện, người ghi sẽ bị bỏ đói. Ngược lại, nếu người ghi thường xuyên xuất hiện và được ưu tiên, thông lượng sẽ giảm. Bài toán này nhằm tìm sự cân bằng đó và tránh các vấn đề cập nhật đồng thời.

Các triết gia ăn tối[11]

Hãy tưởng tượng một số triết gia ngồi quanh một chiếc bàn tròn. Bên trái mỗi người có một chiếc nĩa. Giữa bàn là một tô mì spaghetti lớn. Các triết gia dành thời gian suy nghĩ, trừ khi họ đói. Khi đói, họ nhặt chiếc nĩa ở hai bên mình lên để ăn. Một triết gia không thể ăn nếu không cầm cả hai chiếc nĩa. Nếu triết gia bên phải hoặc bên trái đang dùng một chiếc nĩa mà người đó cần, họ phải chờ đến khi triết gia kia ăn xong và đặt nĩa xuống. Ăn xong, mỗi triết gia đặt cả hai chiếc nĩa xuống bàn rồi chờ đến khi đói lại.

Thay triết gia bằng luồng, thay nĩa bằng tài nguyên, ta có một vấn đề tương tự nhiều ứng dụng doanh nghiệp, nơi các tiến trình cạnh tranh tài nguyên. Nếu không được thiết kế cẩn thận, hệ thống cạnh tranh theo cách này có thể gặp bế tắc, bế tắc sống, suy giảm thông lượng và hiệu quả.

Phần lớn vấn đề đồng thời bạn gặp có thể là một biến thể của ba vấn đề này. Hãy nghiên cứu các thuật toán ấy và tự viết giải pháp để khi gặp vấn đề đồng thời, bạn sẵn sàng xử lý hơn.

Khuyến nghị: Hãy học các thuật toán cơ bản này và hiểu những giải pháp của chúng.

Cẩn thận với phụ thuộc giữa các phương thức đồng bộ

Phụ thuộc giữa các phương thức đồng bộ gây ra lỗi khó phát hiện trong mã đồng thời. Ngôn ngữ Java có khái niệm `synchronized` để bảo vệ một phương thức riêng lẻ. Tuy nhiên, nếu cùng một lớp dùng chung có nhiều hơn một phương thức đồng bộ, hệ thống của bạn có thể được viết sai.[12]

Khuyến nghị: Tránh dùng nhiều hơn một phương thức trên một đối tượng dùng chung.

Sẽ có lúc bạn buộc phải dùng nhiều phương thức trên một đối tượng dùng chung. Khi đó, có ba cách để làm cho mã đúng:

- Khóa phía máy khách: Máy khách khóa máy chủ trước khi gọi phương thức đầu tiên và bảo đảm khóa được giữ trong suốt đoạn mã gọi phương thức cuối cùng.
- Khóa phía máy chủ: Trong máy chủ, tạo một phương thức khóa máy chủ, gọi tất cả các phương thức rồi mở khóa. Máy khách gọi phương thức mới này.
- Máy chủ thích nghi: Tạo một thành phần trung gian thực hiện việc khóa. Đây là ví dụ

11. `http://en.wikipedia.org/wiki/Dining_philosophers_problem`

12. Xem “Dependencies Between Methods Can Break Concurrent Code” ở trang 329.

về khóa phía máy chủ trong trường hợp không thể thay đổi máy chủ ban đầu.

Giữ các vùng đồng bộ nhỏ

Từ khóa `synchronized` tạo ra một khóa. Mọi đoạn mã được bảo vệ bằng cùng một khóa đều được bảo đảm chỉ có một luồng thực thi bên trong tại bất kỳ thời điểm nào. Khóa tốn kém vì gây chậm trễ và phát sinh chi phí phụ. Vì vậy, ta không muốn rải các câu lệnh `synchronized` khắp mã. Mặt khác, các vùng tới hạn[13] phải được bảo vệ. Do đó, ta muốn thiết kế mã sao cho có ít vùng tới hạn nhất có thể.

Một số lập trình viên thiếu kinh nghiệm cố đạt mục tiêu này bằng cách làm cho vùng tới hạn thật lớn. Tuy nhiên, kéo dài việc đồng bộ hóa vượt quá vùng tới hạn tối thiểu làm tăng tranh chấp và giảm hiệu năng.[14]

Khuyến nghị: Hãy giữ các vùng đồng bộ nhỏ nhất có thể.

Viết mã tắt hệ thống đúng cách rất khó

Viết một hệ thống được thiết kế để luôn chạy và tồn tại mãi mãi khác với viết thứ gì đó hoạt động một thời gian rồi tắt một cách êm đẹp.

Tắt hệ thống êm đẹp cho đúng có thể khó. Các vấn đề thường gặp liên quan đến bế tắc,[15] khi luồng chờ một tín hiệu để tiếp tục nhưng tín hiệu đó không bao giờ đến.

Ví dụ, hãy hình dung một hệ thống có luồng cha tạo ra một số luồng con rồi chờ tất cả chúng kết thúc trước khi giải phóng tài nguyên và tắt. Nếu một luồng được tạo ra rơi vào bế tắc thì sao? Luồng cha sẽ chờ mãi, và hệ thống không bao giờ tắt.

Hoặc xét một hệ thống tương tự đã nhận lệnh tắt. Luồng cha bảo tất cả luồng con bỏ tác vụ và kết thúc. Nhưng nếu hai luồng con đang hoạt động theo cặp nhà sản xuất/người tiêu thụ thì sao? Giả sử nhà sản xuất nhận tín hiệu từ luồng cha rồi nhanh chóng tắt. Người tiêu thụ có thể đang chờ thông điệp từ nhà sản xuất và bị chặn ở trạng thái không thể nhận tín hiệu tắt. Nó có thể mắc kẹt chờ nhà sản xuất và không bao giờ kết thúc, khiến luồng cha cũng không thể kết thúc.

Những tình huống như thế này không hề hiếm. Vì vậy, nếu phải viết mã đồng thời có yêu cầu tắt êm đẹp, hãy chuẩn bị dành phần lớn thời gian để làm cho việc tắt diễn ra đúng cách.

Khuyến nghị: Hãy nghĩ đến việc tắt hệ thống từ sớm và triển khai sớm. Việc này sẽ mất nhiều thời gian hơn bạn dự tính. Hãy xem lại các thuật toán hiện có, vì vấn đề này có lẽ khó hơn bạn nghĩ.

---

13. Vùng tới hạn là bất kỳ đoạn mã nào phải được bảo vệ khỏi việc sử dụng đồng thời để chương trình hoạt động đúng.

14. Xem “Increasing Throughput” *ở trang 333*.

15. Xem “Deadlock” *ở trang 335*.

Kiểm thử mã đa luồng

Không thực tế nếu muốn chứng minh mã là đúng. Kiểm thử không bảo đảm tính đúng đắn. Tuy nhiên, kiểm thử tốt có thể giảm thiểu rủi ro. Tất cả điều này đều đúng với giải pháp đơn luồng. Ngay khi có từ hai luồng trở lên dùng cùng mã và làm việc với dữ liệu dùng chung, mọi thứ trở nên phức tạp hơn đáng kể.

Khuyến nghị: Hãy viết các kiểm thử có khả năng bộc lộ vấn đề rồi chạy thường xuyên với các cấu hình chương trình, cấu hình hệ thống và mức tải khác nhau. Nếu kiểm thử thất bại, hãy truy tìm nguyên nhân. Đừng bỏ qua một lần thất bại chỉ vì lần chạy sau kiểm thử lại thành công.

Có rất nhiều điều cần cân nhắc. Sau đây là thêm một số khuyến nghị cụ thể:

- Xem các lỗi chập chờn là dấu hiệu có thể liên quan đến luồng.
- Trước hết, hãy làm cho mã không đa luồng hoạt động đúng.
- Làm cho mã đa luồng có thể thay thế linh hoạt.
- Làm cho mã đa luồng có thể điều chỉnh.
- Chạy với số luồng nhiều hơn số bộ xử lý.
- Chạy trên các nền tảng khác nhau.
- Gắn thiết bị đo đạc vào mã để cố ép lỗi xuất hiện.

Xem các lỗi chập chờn là dấu hiệu có thể liên quan đến luồng

Mã đa luồng khiến những thứ "đơn giản là không thể lỗi" lại thất bại. Hầu hết nhà phát triển không có trực giác tốt về cách luồng tương tác với mã khác, kể cả các tác giả. Lỗi trong mã đa luồng có thể chỉ biểu hiện một lần trong một nghìn, thậm chí một triệu lần thực thi. Việc cố tái hiện lỗi hệ thống có thể khiến người ta nản lòng. Điều này thường khiến nhà phát triển gạt lỗi sang một bên, cho rằng đó là tia vũ trụ, trục trặc phần cứng hay một kiểu sự cố "chỉ xảy ra một lần" nào khác. Tốt nhất hãy giả định rằng không có sự cố nào chỉ xảy ra một lần. Càng phớt lờ những lỗi "một lần" như vậy, người ta càng xây thêm mã dựa trên một cách tiếp cận có thể đã sai.

Khuyến nghị: Đừng xem lỗi hệ thống là sự cố chỉ xảy ra một lần.

Trước hết, hãy làm cho mã không đa luồng hoạt động đúng

Điều này có vẻ hiển nhiên, nhưng nhắc lại cũng chẳng hại gì. Hãy bảo đảm mã hoạt động bên ngoài môi trường luồng. Nói chung, điều đó có nghĩa là tạo các POJO để các luồng gọi đến. POJO không nhận biết luồng, vì vậy có thể được kiểm thử bên ngoài môi trường đa luồng. Càng đưa được nhiều phần trong hệ thống vào các POJO như vậy thì càng tốt.

Khuyến nghị: Đừng cố xử lý lỗi không liên quan đến luồng và lỗi liên quan đến luồng cùng lúc. Hãy bảo đảm mã hoạt động bên ngoài các luồng.

Làm cho mã đa luồng có thể thay thế linh hoạt

Hãy viết mã hỗ trợ xử lý đồng thời sao cho có thể chạy với nhiều cấu hình:

- Một luồng, nhiều luồng, hoặc thay đổi số lượng trong khi chạy.
- Mã đa luồng tương tác với thành phần có thể là thật hoặc đối tượng giả dùng trong kiểm thử.
- Thực thi với các đối tượng giả chạy nhanh, chậm hoặc với tốc độ thay đổi.
- Cấu hình để kiểm thử có thể chạy qua một số vòng lặp nhất định.

Khuyến nghị: Hãy làm cho mã dựa trên luồng đặc biệt dễ thay thế để có thể chạy với nhiều cấu hình khác nhau.

Làm cho mã đa luồng có thể điều chỉnh

Tìm được số luồng cân bằng phù hợp thường đòi hỏi thử và sai. Ngay từ đầu, hãy tìm cách đo hiệu năng hệ thống với các cấu hình khác nhau. Cho phép dễ dàng điều chỉnh số luồng. Cân nhắc cho phép thay đổi số lượng này khi hệ thống đang chạy. Cân nhắc cho phép hệ thống tự điều chỉnh dựa trên thông lượng và mức sử dụng hệ thống.

Chạy với số luồng nhiều hơn số bộ xử lý

Khi hệ thống chuyển đổi giữa các tác vụ, nhiều chuyện có thể xảy ra. Để khuyến khích việc chuyển đổi tác vụ, hãy chạy với số luồng nhiều hơn số bộ xử lý hoặc số lõi. Tác vụ càng chuyển đổi thường xuyên, càng có khả năng bạn phát hiện mã thiếu vùng tới hạn hoặc gây bế tắc.

Chạy trên các nền tảng khác nhau

Giữa năm 2007, chúng tôi xây dựng một khóa học về lập trình đồng thời. Việc xây dựng khóa học chủ yếu diễn ra trên OS X. Khóa học được giảng dạy bằng Windows XP chạy trong máy ảo. Các kiểm thử được viết để minh họa điều kiện thất bại không thất bại thường xuyên trên môi trường XP như khi chạy trên OS X.

Trong mọi trường hợp, mã được kiểm thử đều đã biết là sai. Điều này càng củng cố sự thật rằng các hệ điều hành khác nhau có chính sách xử lý luồng khác nhau, và mỗi chính sách ảnh hưởng đến quá trình thực thi mã. Mã đa luồng hoạt động khác nhau trong các môi trường khác nhau.[16] Bạn nên chạy kiểm thử trong mọi môi trường có thể triển khai.

Khuyến nghị: Hãy chạy mã đa luồng trên tất cả nền tảng đích từ sớm và thường xuyên.

---

16. Bạn có biết mô hình luồng trong Java không bảo đảm lập lịch luồng có tính chiếm quyền? Các hệ điều hành hiện đại hỗ trợ lập lịch chiếm quyền, nên ta có được điều đó "miễn phí". Dù vậy, JVM không bảo đảm điều này.

Gắn thiết bị đo đạc vào mã để cố ép lỗi xuất hiện

Lỗi trong mã đồng thời thường ẩn mình. Các kiểm thử đơn giản thường không bộc lộ chúng. Thực tế, chúng thường ẩn trong quá trình xử lý bình thường. Lỗi có thể chỉ xuất hiện vài giờ, vài ngày, thậm chí vài tuần một lần!

Lý do lỗi luồng có thể hiếm gặp, thất thường và khó tái hiện là trong hàng nghìn đường đi có thể qua một vùng dễ có lỗi, chỉ một vài đường thực sự thất bại. Vì vậy, xác suất đi vào đường gây lỗi có thể thấp đến đáng ngạc nhiên. Điều này khiến việc phát hiện và gỡ lỗi rất khó.

Làm thế nào để tăng cơ hội bắt gặp những lỗi hiếm như vậy? Bạn có thể gắn thiết bị đo đạc vào mã và buộc mã chạy theo thứ tự khác nhau bằng cách thêm lời gọi đến các phương thức như `Object.wait()`, `Object.sleep()`, `Object.yield()` và `Object.priority()`.

Mỗi phương thức này có thể ảnh hưởng đến thứ tự thực thi, qua đó tăng khả năng phát hiện lỗi. Tốt hơn hết là mã sai thất bại sớm và thường xuyên nhất có thể.

Có hai lựa chọn để gắn thiết bị đo đạc vào mã:

- Viết thủ công.
- Tự động hóa.

Viết thủ công

Bạn có thể tự thêm lời gọi `wait()`, `sleep()`, `yield()` và `priority()` vào mã. Cách này có thể phù hợp khi kiểm thử một đoạn mã đặc biệt khó nhằn.

Sau đây là một ví dụ:

```
public synchronized String nextUrlOrNull() {
    if(hasNext()) {
        String url = urlGenerator.next();
        Thread.yield(); // được chèn vào để kiểm thử.
        updateHasNext();
        return url;
    }
    return null;
}
```

Lời gọi `yield()` được chèn vào sẽ thay đổi các đường thực thi mà mã đi qua và có thể khiến mã thất bại trong khi trước đó không thất bại. Nếu mã hỏng, không phải vì bạn thêm lời gọi `yield()`.[17] Mã vốn đã sai, và việc này chỉ làm lỗi lộ ra.

Cách tiếp cận này có nhiều vấn đề:

- Bạn phải tự tìm các vị trí thích hợp để chèn.
- Làm sao biết nên đặt lời gọi ở đâu và dùng loại lời gọi nào?
- Nếu để mã như vậy trong môi trường sản phẩm, mã sẽ chậm đi không cần thiết.

17. Điều này không hoàn toàn đúng. Vì JVM không bảo đảm lập lịch luồng có tính chiếm quyền, một thuật toán cụ thể có thể luôn hoạt động trên hệ điều hành không chiếm quyền điều khiển luồng. Điều ngược lại cũng có thể xảy ra, nhưng vì những lý do khác.

- Đây là cách thử đại trà. Bạn có thể tìm ra lỗi, cũng có thể không. Thực tế, khả năng không đứng về phía bạn.

Điều ta cần là cách thực hiện việc này khi kiểm thử nhưng không phải khi chạy trong môi trường sản phẩm. Ta cũng cần dễ dàng thay đổi cấu hình giữa các lần chạy khác nhau, nhờ đó tăng cơ hội phát hiện lỗi khi tổng hợp kết quả.

Rõ ràng, nếu chia hệ thống thành các POJO không biết gì về luồng và các lớp điều khiển luồng, việc tìm vị trí thích hợp để gắn thiết bị đo đạc sẽ dễ hơn. Hơn nữa, ta có thể tạo nhiều bộ dụng cụ kiểm thử khác nhau để gọi các POJO trong những chế độ khác nhau, với lời gọi `sleep`, `yield` và những thứ tương tự.

Tự động hóa

Bạn có thể dùng các công cụ như framework hướng khía cạnh, CGLIB hoặc ASM để tự động gắn thiết bị đo đạc vào mã. Chẳng hạn, bạn có thể dùng một lớp chỉ có một phương thức:

```
public class ThreadJigglePoint {
    public static void jiggle() {
    }
}
```

Bạn có thể thêm lời gọi đến phương thức này ở nhiều nơi trong mã:

```
public synchronized String nextUrlOrNull() {
    if(hasNext()) {
        ThreadJiglePoint.jiggle();
        String url = urlGenerator.next();
        ThreadJiglePoint.jiggle();
        updateHasNext();
        ThreadJiglePoint.jiggle();
        return url;
    }
    return null;
}
```

Giờ bạn dùng một aspect đơn giản chọn ngẫu nhiên giữa không làm gì, ngủ hoặc nhường luồng.

Hoặc hãy hình dung lớp `ThreadJigglePoint` có hai cách triển khai. Cách thứ nhất triển khai `jiggle` sao cho không làm gì và được dùng trong môi trường sản phẩm. Cách thứ hai tạo ra một số ngẫu nhiên để chọn giữa ngủ, nhường luồng hoặc đi tiếp. Nếu chạy kiểm thử một nghìn lần với thao tác lắc ngẫu nhiên, bạn có thể tìm ra một số lỗi. Nếu kiểm thử thành công, ít nhất bạn có thể nói rằng mình đã làm tròn trách nhiệm. Cách này hơi đơn giản, nhưng có thể là một lựa chọn hợp lý khi chưa có công cụ tinh vi hơn.

Có một công cụ tên là ConTest[18] do IBM phát triển, có chức năng tương tự nhưng tinh vi hơn khá nhiều.

18. `http://www.alphaworks.ibm.com/tech/contest`

Mục đích là làm cho mã bị xáo trộn để các luồng chạy theo thứ tự khác nhau ở những thời điểm khác nhau. Kết hợp các kiểm thử được viết tốt với thao tác xáo trộn có thể tăng đáng kể cơ hội phát hiện lỗi.

Khuyến nghị: Hãy dùng các chiến lược xáo trộn để tìm lỗi.

Kết luận

Viết mã đồng thời cho đúng rất khó. Mã vốn dễ theo dõi có thể trở thành ác mộng khi nhiều luồng và dữ liệu dùng chung xuất hiện. Nếu phải viết mã đồng thời, bạn cần viết mã sạch một cách nghiêm túc, nếu không sẽ phải đối mặt với những lỗi khó phát hiện và hiếm gặp.

Trước hết, hãy tuân theo Nguyên tắc trách nhiệm đơn. Chia hệ thống thành các POJO để tách mã nhận biết luồng khỏi mã không nhận biết luồng. Khi kiểm thử mã nhận biết luồng, hãy bảo đảm bạn chỉ kiểm thử phần đó, không kiểm thử thứ gì khác. Điều này cho thấy mã nhận biết luồng nên nhỏ và tập trung.

Hãy biết các nguồn gây ra vấn đề đồng thời: nhiều luồng thao tác trên dữ liệu dùng chung hoặc dùng chung một nhóm tài nguyên. Các tình huống ở ranh giới, chẳng hạn tắt hệ thống gọn gàng hoặc hoàn thành một vòng lặp, có thể đặc biệt khó xử lý.

Hãy học thư viện và nắm các thuật toán cơ bản. Hiểu cách một số tính năng của thư viện hỗ trợ giải quyết những vấn đề tương tự các thuật toán cơ bản.

Hãy học cách tìm các vùng mã cần khóa rồi khóa chúng. Đừng khóa những vùng không cần khóa. Tránh gọi một vùng đã khóa từ một vùng đã khóa khác. Việc này đòi hỏi hiểu sâu điều gì được chia sẻ và điều gì không. Hãy giữ số lượng đối tượng dùng chung cũng như phạm vi chia sẻ ở mức hẹp nhất có thể. Thay đổi thiết kế của các đối tượng chứa dữ liệu dùng chung để đáp ứng nhu cầu của máy khách, thay vì buộc máy khách phải quản lý trạng thái dùng chung.

Vấn đề sẽ phát sinh. Những vấn đề không xuất hiện sớm thường bị gạt đi như thể chỉ xảy ra một lần. Những "lỗi một lần" này thường chỉ xảy ra khi tải cao hoặc vào những thời điểm tưởng như ngẫu nhiên. Vì vậy, bạn cần có khả năng chạy đi chạy lại mã liên quan đến luồng trong nhiều cấu hình và trên nhiều nền tảng. Khả năng kiểm thử, vốn đến một cách tự nhiên khi tuân theo Ba Quy luật TDD, đòi hỏi một mức độ linh hoạt nhất định, nhờ đó có thể chạy mã với nhiều cấu hình hơn.

Bạn sẽ cải thiện đáng kể cơ hội tìm ra mã sai nếu dành thời gian gắn thiết bị đo đạc vào mã. Bạn có thể làm thủ công hoặc dùng một loại công nghệ tự động nào đó. Hãy đầu tư vào việc này từ sớm. Bạn nên chạy mã dựa trên luồng càng lâu càng tốt trước khi đưa vào môi trường sản phẩm.

Nếu tiếp cận sạch sẽ, cơ hội làm đúng của bạn sẽ tăng lên đáng kể.

Tài liệu tham khảo

[Lea99]: *Concurrent Programming in Java: Design Principles and Patterns*, ấn bản thứ 2, Doug Lea, Prentice Hall, 1999.

[PPP]: *Agile Software Development: Principles, Patterns, and Practices*, Robert C. Martin, Prentice Hall, 2002.

[PRAG]: *The Pragmatic Programmer*, Andrew Hunt, Dave Thomas, Addison-Wesley, 2000.

14

Tinh chỉnh từng bước



Nghiên cứu tình huống về bộ phân tích đối số dòng lệnh

Chương này là một nghiên cứu tình huống về quá trình tinh chỉnh từng bước. Bạn sẽ thấy một mô-đun khởi đầu tốt nhưng không mở rộng được. Sau đó, bạn sẽ thấy mô-đun được tái cấu trúc và làm sạch ra sao.

Hẳn ai trong chúng ta cũng từng phải phân tích đối số dòng lệnh. Nếu không có sẵn tiện ích phù hợp, ta chỉ việc duyệt mảng chuỗi được truyền vào hàm `main`. Có nhiều tiện

ích tốt từ nhiều nguồn khác nhau, nhưng không tiện ích nào làm đúng mọi thứ tôi muốn. Vậy nên, dĩ nhiên tôi quyết định tự viết một cái. Tôi gọi nó là `Args`.

`Args` rất dễ dùng. Bạn chỉ cần khởi tạo lớp `Args` với các đối số đầu vào và một chuỗi định dạng, rồi truy vấn đối tượng `Args` để lấy giá trị của các đối số. Hãy xem ví dụ đơn giản sau:

Đoạn mã 14-1
Cách dùng Args đơn giản

```
public static void main(String[] args) {
  try {
    Args arg = new Args("l,p#,d*", args);
    boolean logging = arg.getBoolean('l');
    int port = arg.getInt('p');
    String directory = arg.getString('d');
    executeApplication(logging, port, directory);
  } catch (ArgsException e) {
    System.out.printf("Argument error: %s\n", e.errorMessage());
  }
}
```

Bạn thấy cách dùng đơn giản thế nào. Ta chỉ cần tạo một thể hiện của lớp `Args` với hai tham số. Tham số đầu tiên là chuỗi định dạng, hay lược đồ: `"l,p#,d*"`. Chuỗi này định nghĩa ba đối số dòng lệnh. Đối số đầu tiên, `-l`, là kiểu boolean. Đối số thứ hai, `-p`, là kiểu số nguyên. Đối số thứ ba, `-d`, là kiểu chuỗi. Tham số thứ hai của hàm dựng `Args` đơn giản là mảng đối số dòng lệnh được truyền vào `main`.

Nếu hàm dựng kết thúc mà không ném `ArgsException`, các đối số dòng lệnh đầu vào đã được phân tích và đối tượng `Args` sẵn sàng để truy vấn. Những phương thức như `getBoolean`, `getInteger` và `getString` cho phép ta truy cập giá trị của các đối số theo tên.

Nếu có vấn đề trong chuỗi định dạng hoặc trong chính các đối số dòng lệnh, một `ArgsException` sẽ được ném ra. Bạn có thể gọi phương thức `errorMessage` của ngoại lệ để lấy mô tả tiện dụng về lỗi đã xảy ra.

Cài đặt Args

Đoạn mã 14-2 là phần cài đặt lớp `Args`. Hãy đọc thật kỹ. Tôi đã dụng công chăm chút phong cách và cấu trúc, hy vọng bạn thấy đáng để noi theo.

Đoạn mã 14-2
Args.java

```
package com.objectmentor.utilities.args;

import static com.objectmentor.utilities.args.ArgsException.ErrorCode.*;
import java.util.*;
```

```
public class Args {
  private Map<Character, ArgumentMarshaler> marshalers;
  private Set<Character> argsFound;
  private ListIterator<String> currentArgument;

  public Args(String schema, String[] args) throws ArgsException {
    marshalers = new HashMap<Character, ArgumentMarshaler>();
    argsFound = new HashSet<Character>();

    parseSchema(schema);
    parseArgumentStrings(Arrays.asList(args));
  }
  private void parseSchema(String schema) throws ArgsException {
    for (String element : schema.split(","))
      if (element.length() > 0)
        parseSchemaElement(element.trim());
  }

  private void parseSchemaElement(String element) throws ArgsException {
    char elementId = element.charAt(0);
    String elementTail = element.substring(1);
    validateSchemaElementId(elementId);
    if (elementTail.length() == 0)
      marshalers.put(elementId, new BooleanArgumentMarshaler());
    else if (elementTail.equals("*"))
      marshalers.put(elementId, new StringArgumentMarshaler());
    else if (elementTail.equals("#"))
      marshalers.put(elementId, new IntegerArgumentMarshaler());
    else if (elementTail.equals("##"))
      marshalers.put(elementId, new DoubleArgumentMarshaler());
    else if (elementTail.equals("[*]"))
      marshalers.put(elementId, new StringArrayArgumentMarshaler());
    else
      throw new ArgsException(INVALID_ARGUMENT_FORMAT, elementId,
    elementTail);
  }

  private void validateSchemaElementId(char elementId) throws ArgsException {
    if (!Character.isLetter(elementId))
      throw new ArgsException(INVALID_ARGUMENT_NAME, elementId, null);
  }

  private void parseArgumentStrings(List<String> argsList) throws
    ArgsException
  {
    for (currentArgument = argsList.listIterator();
    currentArgument.hasNext();)
    {
      String argString = currentArgument.next();
      if (argString.startsWith("-")) {
        parseArgumentCharacters(argString.substring(1));
      } else {
        currentArgument.previous();
        break;
      }
    }
  }
```

```
    private void parseArgumentCharacters(String argChars) throws ArgsException
      {
      for (int i = 0; i < argChars.length(); i++)
        parseArgumentCharacter(argChars.charAt(i));
    }

    private void parseArgumentCharacter(char argChar) throws ArgsException {
      ArgumentMarshaler m = marshalers.get(argChar);
      if (m == null) {
        throw new ArgsException(UNEXPECTED_ARGUMENT, argChar, null);
      } else {
        argsFound.add(argChar);
        try {
          m.set(currentArgument);
        } catch (ArgsException e) {
          e.setErrorArgumentId(argChar);
          throw e;
        }
      }
    }

    public boolean has(char arg) {
      return argsFound.contains(arg);
    }

    public int nextArgument() {
      return currentArgument.nextIndex();
    }

    public boolean getBoolean(char arg) {
      return BooleanArgumentMarshaler.getValue(marshalers.get(arg));
    }

    public String getString(char arg) {
      return StringArgumentMarshaler.getValue(marshalers.get(arg));
    }
    public int getInt(char arg) {
      return IntegerArgumentMarshaler.getValue(marshalers.get(arg));
    }

    public double getDouble(char arg) {
      return DoubleArgumentMarshaler.getValue(marshalers.get(arg));
    }

    public String[] getStringArray(char arg) {
      return StringArrayArgumentMarshaler.getValue(marshalers.get(arg));
    }
  }
```

Bạn có thể đọc đoạn mã này từ trên xuống dưới mà không phải nhảy qua lại hay nhìn trước nhiều. Có lẽ bạn chỉ cần nhìn trước để tìm định nghĩa `ArgumentMarshaler`, phần tôi cố ý bỏ đi. Đọc kỹ đoạn mã này xong, hẳn bạn đã hiểu giao diện `ArgumentMarshaler` và các lớp phái sinh của nó làm gì. Giờ tôi sẽ cho bạn xem một vài lớp đó, từ Đoạn mã 14-3 đến Đoạn mã 14-6.

Đoạn mã 14-3
ArgumentMarshaler.java

```
public interface ArgumentMarshaler {
  void set(Iterator<String> currentArgument) throws ArgsException;
}
```

Đoạn mã 14-4
BooleanArgumentMarshaler.java

```
public class BooleanArgumentMarshaler implements ArgumentMarshaler {
  private boolean booleanValue = false;

  public void set(Iterator<String> currentArgument) throws ArgsException {
    booleanValue = true;
  }

  public static boolean getValue(ArgumentMarshaler am) {
    if (am != null && am instanceof BooleanArgumentMarshaler)
      return ((BooleanArgumentMarshaler) am).booleanValue;
    else
      return false;
  }
}
```

Đoạn mã 14-5
StringArgumentMarshaler.java

```
import static com.objectmentor.utilities.args.ArgsException.ErrorCode.*;

public class StringArgumentMarshaler implements ArgumentMarshaler {
  private String stringValue = "";
  public void set(Iterator<String> currentArgument) throws ArgsException {
    try {
      stringValue = currentArgument.next();
    } catch (NoSuchElementException e) {
      throw new ArgsException(MISSING_STRING);
    }
  }

  public static String getValue(ArgumentMarshaler am) {
    if (am != null && am instanceof StringArgumentMarshaler)
      return ((StringArgumentMarshaler) am).stringValue;
    else
      return "";
  }
}
```

Đoạn mã 14-6
IntegerArgumentMarshaler.java

```
import static com.objectmentor.utilities.args.ArgsException.ErrorCode.*;

public class IntegerArgumentMarshaler implements ArgumentMarshaler {
  private int intValue = 0;

  public void set(Iterator<String> currentArgument) throws ArgsException {
    String parameter = null;
    try {
      parameter = currentArgument.next();
      intValue = Integer.parseInt(parameter);
    } catch (NoSuchElementException e) {
      throw new ArgsException(MISSING_INTEGER);
    } catch (NumberFormatException e) {
      throw new ArgsException(INVALID_INTEGER, parameter);
    }
  }

  public static int getValue(ArgumentMarshaler am) {
    if (am != null && am instanceof IntegerArgumentMarshaler)
      return ((IntegerArgumentMarshaler) am).intValue;
    else
      return 0;
  }
}
```

Các lớp phái sinh `ArgumentMarshaler` còn lại chỉ lặp lại mô thức này cho số thực dấu phẩy động và mảng chuỗi; đưa chúng vào đây sẽ khiến chương này rườm rà. Tôi để phần đó làm bài tập cho bạn.

Có thể còn một thông tin khác khiến bạn băn khoăn: định nghĩa các hằng mã lỗi. Chúng nằm trong lớp `ArgsException` (Đoạn mã 14-7).

Đoạn mã 14-7
ArgsException.java

```
import static com.objectmentor.utilities.args.ArgsException.ErrorCode.*;

public class ArgsException extends Exception {
  private char errorArgumentId = '\0';
  private String errorParameter = null;
  private ErrorCode errorCode = OK;

  public ArgsException() {}

  public ArgsException(String message) {super(message);}

  public ArgsException(ErrorCode errorCode) {
    this.errorCode = errorCode;
  }
  public ArgsException(ErrorCode errorCode, String errorParameter) {
    this.errorCode = errorCode;
```

```
    this.errorParameter = errorParameter;
  }
  public ArgsException(ErrorCode errorCode,
                       char errorArgumentId, String errorParameter) {
    this.errorCode = errorCode;
    this.errorParameter = errorParameter;
    this.errorArgumentId = errorArgumentId;
  }

  public char getErrorArgumentId() {
    return errorArgumentId;
  }
  public void setErrorArgumentId(char errorArgumentId) {
    this.errorArgumentId = errorArgumentId;
  }
  public String getErrorParameter() {
    return errorParameter;
  }

  public void setErrorParameter(String errorParameter) {
    this.errorParameter = errorParameter;
  }

  public ErrorCode getErrorCode() {
    return errorCode;
  }

  public void setErrorCode(ErrorCode errorCode) {
    this.errorCode = errorCode;
  }

  public String errorMessage() {
    switch (errorCode) {
      case OK:
        return "TILT: Should not get here.";
      case UNEXPECTED_ARGUMENT:
        return String.format("Argument -%c unexpected.", errorArgumentId);
      case MISSING_STRING:
        return String.format("Could not find string parameter for -%c.",
                             errorArgumentId);
      case INVALID_INTEGER:
        return String.format("Argument -%c expects an integer but was '%s'.",
                             errorArgumentId, errorParameter);
      case MISSING_INTEGER:
        return String.format("Could not find integer parameter for -%c.",
                             errorArgumentId);
      case INVALID_DOUBLE:
        return String.format("Argument -%c expects a double but was '%s'.",
                             errorArgumentId, errorParameter);
      case MISSING_DOUBLE:
        return String.format("Could not find double parameter for -%c.",
                             errorArgumentId);
      case INVALID_ARGUMENT_NAME:
        return String.format("'%c' is not a valid argument name.",
                             errorArgumentId);
      case INVALID_ARGUMENT_FORMAT:
        return String.format("'%s' is not a valid argument format.",
                             errorParameter);
```

```
      }
      return "";
    }

    public enum ErrorCode {
      OK, INVALID_ARGUMENT_FORMAT, UNEXPECTED_ARGUMENT, INVALID_ARGUMENT_NAME,
      MISSING_STRING,
      MISSING_INTEGER, INVALID_INTEGER,
      MISSING_DOUBLE, INVALID_DOUBLE}
  }
```

Thật đáng kinh ngạc là một ý tưởng đơn giản như vậy lại cần đến ngần ấy mã để triển khai đầy đủ chi tiết. Một lý do là Java khá rườm rà. Là ngôn ngữ định kiểu tĩnh, Java đòi hỏi nhiều từ ngữ để thỏa mãn hệ thống kiểu. Trong Ruby, Python hay Smalltalk, chương trình này ngắn hơn nhiều.[1]

Hãy đọc lại đoạn mã một lần nữa. Chú ý cách đặt tên, kích thước các hàm và định dạng mã. Nếu là lập trình viên giàu kinh nghiệm, có thể bạn sẽ bắt bẻ đôi chỗ về phong cách hay cấu trúc. Dù vậy, tôi hy vọng nhìn chung bạn sẽ thấy chương trình được viết gọn gàng, có cấu trúc sạch sẽ.

Chẳng hạn, hẳn bạn thấy rõ cách bổ sung một kiểu đối số mới, như ngày tháng hay số phức, và việc đó chỉ tốn chút công sức. Tóm lại, chỉ cần tạo một lớp phái sinh mới của `ArgumentMarshaler`, thêm một hàm `getXXX` và một nhánh `case` trong hàm `parseSchemaElement`. Có lẽ cũng cần thêm một `ArgsException.ErrorCode` mới cùng thông báo lỗi tương ứng.

Tôi đã làm việc này như thế nào?

Để bạn yên tâm, tôi không viết chương trình này từ đầu đến cuối theo đúng hình thức hiện tại. Quan trọng hơn, tôi không mong bạn viết được chương trình sạch sẽ, tao nhã chỉ trong một lượt. Nếu vài thập kỷ qua dạy cho chúng ta điều gì, thì đó là lập trình thiên về tay nghề hơn khoa học. Muốn viết mã sạch, trước hết bạn phải viết mã bẩn, rồi làm sạch nó.

Điều này hẳn không làm bạn ngạc nhiên. Ta đã học được sự thật đó từ hồi tiểu học, khi thầy cô cố (thường là vô ích) khuyên ta viết bản nháp cho các bài luận. Họ bảo rằng ta nên viết bản nháp đầu, rồi bản thứ hai, sau đó viết thêm vài bản nữa cho đến khi có bản cuối cùng. Họ muốn ta hiểu rằng viết một bài luận sạch sẽ là quá trình tinh chỉnh từng bước.

Hầu hết lập trình viên mới vào nghề, cũng như hầu hết học sinh tiểu học, không làm theo lời khuyên ấy. Họ tin rằng mục tiêu chính là làm cho chương trình chạy được. Khi chương trình đã "chạy được", họ chuyển sang việc tiếp theo, để mặc nó trong tình trạng mà họ đã khiến nó "chạy được". Hầu hết lập trình viên dày dạn đều biết đó là tự sát nghề nghiệp.

---

1. Gần đây tôi viết lại mô-đun này bằng Ruby. Bản đó chỉ bằng một phần bảy kích thước và có cấu trúc nhỉnh hơn đôi chút.

Args: bản nháp đầu tiên

Đoạn mã 14-8 cho thấy một phiên bản cũ hơn của lớp `Args`. Nó "chạy được". Và nó bừa bộn.

Đoạn mã 14-8
Args.java (bản nháp đầu tiên)

```
import java.text.ParseException;
import java.util.*;

public class Args {
  private String schema;
  private String[] args;
  private boolean valid = true;
  private Set<Character> unexpectedArguments = new TreeSet<Character>();
  private Map<Character, Boolean> booleanArgs =
    new HashMap<Character, Boolean>();
  private Map<Character, String> stringArgs = new HashMap<Character,
    String>();
  private Map<Character, Integer> intArgs = new HashMap<Character,
    Integer>();
  private Set<Character> argsFound = new HashSet<Character>();
  private int currentArgument;
  private char errorArgumentId = '\0';
  private String errorParameter = "TILT";
  private ErrorCode errorCode = ErrorCode.OK;

  private enum ErrorCode {
    OK, MISSING_STRING, MISSING_INTEGER, INVALID_INTEGER,
    UNEXPECTED_ARGUMENT}
  public Args(String schema, String[] args) throws ParseException {
    this.schema = schema;
    this.args = args;
    valid = parse();
  }

  private boolean parse() throws ParseException {
    if (schema.length() == 0 && args.length == 0)
      return true;
    parseSchema();
    try {
      parseArguments();
    } catch (ArgsException e) {
    }
    return valid;
  }
  private boolean parseSchema() throws ParseException {
    for (String element : schema.split(",")) {
      if (element.length() > 0) {
        String trimmedElement = element.trim();
        parseSchemaElement(trimmedElement);
      }
    }
    return true;
  }
```

```
private void parseSchemaElement(String element) throws ParseException {
  char elementId = element.charAt(0);
  String elementTail = element.substring(1);
  validateSchemaElementId(elementId);
  if (isBooleanSchemaElement(elementTail))
    parseBooleanSchemaElement(elementId);
  else if (isStringSchemaElement(elementTail))
    parseStringSchemaElement(elementId);
  else if (isIntegerSchemaElement(elementTail)) {
    parseIntegerSchemaElement(elementId);
  } else {
    throw new ParseException(
      String.format("Argument: %c has invalid format: %s.",
                    elementId, elementTail), 0);
  }
}
private void validateSchemaElementId(char elementId) throws ParseException
  {
  if (!Character.isLetter(elementId)) {
    throw new ParseException(
      "Bad character:" + elementId + "in Args format: " + schema, 0);
  }
}

private void parseBooleanSchemaElement(char elementId) {
  booleanArgs.put(elementId, false);
}
private void parseIntegerSchemaElement(char elementId) {
  intArgs.put(elementId, 0);
}

private void parseStringSchemaElement(char elementId) {
  stringArgs.put(elementId, "");
}

private boolean isStringSchemaElement(String elementTail) {
  return elementTail.equals("*");
}

private boolean isBooleanSchemaElement(String elementTail) {
  return elementTail.length() == 0;
}

private boolean isIntegerSchemaElement(String elementTail) {
  return elementTail.equals("#");
}
private boolean parseArguments() throws ArgsException {
  for (currentArgument = 0; currentArgument < args.length;
  currentArgument++)
  {
    String arg = args[currentArgument];
    parseArgument(arg);
  }
  return true;
}

private void parseArgument(String arg) throws ArgsException {
```

```
    if (arg.startsWith("-"))
      parseElements(arg);
  }

  private void parseElements(String arg) throws ArgsException {
    for (int i = 1; i < arg.length(); i++)
      parseElement(arg.charAt(i));
  }

  private void parseElement(char argChar) throws ArgsException {
    if (setArgument(argChar))
      argsFound.add(argChar);
    else {
      unexpectedArguments.add(argChar);
      errorCode = ErrorCode.UNEXPECTED_ARGUMENT;
      valid = false;
    }
  }
  private boolean setArgument(char argChar) throws ArgsException {
    if (isBooleanArg(argChar))
      setBooleanArg(argChar, true);
    else if (isStringArg(argChar))
      setStringArg(argChar);
    else if (isIntArg(argChar))
      setIntArg(argChar);
    else
      return false;

    return true;
  }
  private boolean isIntArg(char argChar) {return
    intArgs.containsKey(argChar);}

  private void setIntArg(char argChar) throws ArgsException {
    currentArgument++;
    String parameter = null;
    try {
      parameter = args[currentArgument];
      intArgs.put(argChar, new Integer(parameter));
    } catch (ArrayIndexOutOfBoundsException e) {
      valid = false;
      errorArgumentId = argChar;
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (NumberFormatException e) {
      valid = false;
      errorArgumentId = argChar;
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw new ArgsException();
    }
  }
  private void setStringArg(char argChar) throws ArgsException {
    currentArgument++;
    try {
      stringArgs.put(argChar, args[currentArgument]);
    } catch (ArrayIndexOutOfBoundsException e) {
      valid = false;
```

```
        errorArgumentId = argChar;
        errorCode = ErrorCode.MISSING_STRING;
        throw new ArgsException();
      }
    }
    private boolean isStringArg(char argChar) {
      return stringArgs.containsKey(argChar);
    }

    private void setBooleanArg(char argChar, boolean value) {
      booleanArgs.put(argChar, value);
    }
    private boolean isBooleanArg(char argChar) {
      return booleanArgs.containsKey(argChar);
    }

    public int cardinality() {
      return argsFound.size();
    }

    public String usage() {
      if (schema.length() > 0)
        return "-[" + schema + "]";
      else
        return "";
    }

    public String errorMessage() throws Exception {
      switch (errorCode) {
        case OK:
          throw new Exception("TILT: Should not get here.");
        case UNEXPECTED_ARGUMENT:
          return unexpectedArgumentMessage();
        case MISSING_STRING:
          return String.format("Could not find string parameter for -%c.",
                               errorArgumentId);
        case INVALID_INTEGER:
          return String.format("Argument -%c expects an integer but was '%s'.",
                               errorArgumentId, errorParameter);
        case MISSING_INTEGER:
          return String.format("Could not find integer parameter for -%c.",
                               errorArgumentId);
      }
      return "";
    }
    private String unexpectedArgumentMessage() {
      StringBuffer message = new StringBuffer("Argument(s) -");
      for (char c : unexpectedArguments) {
        message.append(c);
      }
      message.append(" unexpected.");

      return message.toString();
    }

    private boolean falseIfNull(Boolean b) {
      return b != null && b;
    }
```

```
  private int zeroIfNull(Integer i) {
    return i == null ? 0 : i;
  }

  private String blankIfNull(String s) {
    return s == null ? "" : s;
  }

  public String getString(char arg) {
    return blankIfNull(stringArgs.get(arg));
  }
  public int getInt(char arg) {
    return zeroIfNull(intArgs.get(arg));
  }

  public boolean getBoolean(char arg) {
    return falseIfNull(booleanArgs.get(arg));
  }

  public boolean has(char arg) {
    return argsFound.contains(arg);
  }

  public boolean isValid() {
    return valid;
  }

  private class ArgsException extends Exception {
  }
}
```

Tôi hy vọng phản ứng ban đầu của bạn trước đống mã này là: “May mà anh ấy không để nó nguyên như vậy!” Nếu đúng thế, hãy nhớ người khác cũng sẽ nghĩ vậy về đoạn mã nháp bạn để lại.

Nói “bản nháp đầu tiên” có lẽ là cách gọi tử tế nhất cho đoạn mã này. Rõ ràng nó còn đang dang dở. Chỉ riêng số lượng biến thể hiện đã đủ ngợp. Những chuỗi kỳ quặc như `"TILT"`, các `HashSet` và `TreeSet`, cùng những khối try-catch-catch, tất cả hợp lại thành một đống đang thối rữa.

Tôi không định viết ra một đống thối rữa như vậy. Thật ra, tôi đã cố giữ mọi thứ tương đối ngăn nắp. Bạn có thể nhận ra điều đó qua cách tôi đặt tên hàm và biến, cũng như cấu trúc sơ sài của chương trình. Nhưng rõ ràng tôi đã để vấn đề vượt khỏi tầm kiểm soát.

Mớ hỗn độn ấy hình thành dần dần. Các phiên bản trước đó không hề tệ đến thế. Chẳng hạn, Đoạn mã 14-9 cho thấy phiên bản cũ hơn, khi mới chỉ xử lý được đối số boolean.

Đoạn mã 14-9
Args.java (chỉ có boolean)

```
package com.objectmentor.utilities.getopts;

import java.util.*;
public class Args {
```

```
private String schema;
private String[] args;
private boolean valid;
private Set<Character> unexpectedArguments = new TreeSet<Character>();
private Map<Character, Boolean> booleanArgs =
  new HashMap<Character, Boolean>();
private int numberOfArguments = 0;

public Args(String schema, String[] args) {
  this.schema = schema;
  this.args = args;
  valid = parse();
}
public boolean isValid() {
  return valid;
}

private boolean parse() {
  if (schema.length() == 0 && args.length == 0)
    return true;
  parseSchema();
  parseArguments();
  return unexpectedArguments.size() == 0;
}

private boolean parseSchema() {
  for (String element : schema.split(",")) {
    parseSchemaElement(element);
  }
  return true;
}

private void parseSchemaElement(String element) {
  if (element.length() == 1) {
    parseBooleanSchemaElement(element);
  }
}

private void parseBooleanSchemaElement(String element) {
  char c = element.charAt(0);
  if (Character.isLetter(c)) {
    booleanArgs.put(c, false);
  }
}

private boolean parseArguments() {
  for (String arg : args)
    parseArgument(arg);
  return true;
}
private void parseArgument(String arg) {
  if (arg.startsWith("-"))
    parseElements(arg);
}

private void parseElements(String arg) {
  for (int i = 1; i < arg.length(); i++)
    parseElement(arg.charAt(i));
```

```
    }

    private void parseElement(char argChar) {
      if (isBoolean(argChar)) {
        numberOfArguments++;
        setBooleanArg(argChar, true);
      } else
        unexpectedArguments.add(argChar);
    }

    private void setBooleanArg(char argChar, boolean value) {
      booleanArgs.put(argChar, value);
    }

    private boolean isBoolean(char argChar) {
      return booleanArgs.containsKey(argChar);
    }

    public int cardinality() {
      return numberOfArguments;
    }

    public String usage() {
      if (schema.length() > 0)
        return "-["+schema+"]";
      else
        return "";
    }

    public String errorMessage() {
      if (unexpectedArguments.size() > 0) {
        return unexpectedArgumentMessage();
      } else
        return "";
    }
    private String unexpectedArgumentMessage() {
      StringBuffer message = new StringBuffer("Argument(s) -");
      for (char c : unexpectedArguments) {
        message.append(c);
      }
      message.append(" unexpected.");
      return message.toString();
    }

    public boolean getBoolean(char arg) {
      return booleanArgs.get(arg);
    }
  }
```

Dù có thể bắt bẻ nhiều điểm trong đoạn mã này, nó thực ra cũng không tệ lắm. Nó gọn, đơn giản và dễ hiểu. Tuy vậy, ngay trong đoạn mã này đã lộ ra những mầm mống của đống thối rữa về sau. Có thể thấy rõ nó đã phát triển thành mớ hỗn độn kia như thế nào.

Lưu ý rằng mớ hỗn độn về sau chỉ có thêm hai kiểu đối số so với phiên bản này: `String` và số nguyên. Chỉ thêm hai kiểu đối số mà tác động tiêu cực lên mã lại rất lớn. Từ một thứ có thể bảo trì tương đối dễ dàng, nó biến thành thứ mà tôi nghĩ sẽ đầy lỗi và khiếm khuyết.

Tôi bổ sung hai kiểu đối số đó từng bước. Trước tiên, tôi thêm đối số `String`, tạo ra phiên bản sau:

Đoạn mã 14-10
Args.java (boolean và String)

```
package com.objectmentor.utilities.getopts;

import java.text.ParseException;
import java.util.*;
public class Args {
  private String schema;
  private String[] args;
  private boolean valid = true;
  private Set<Character> unexpectedArguments = new TreeSet<Character>();
  private Map<Character, Boolean> booleanArgs =
    new HashMap<Character, Boolean>();
  private Map<Character, String> stringArgs =
    new HashMap<Character, String>();
  private Set<Character> argsFound = new HashSet<Character>();
  private int currentArgument;
  private char errorArgument = '\0';
  enum ErrorCode {
    OK, MISSING_STRING}

  private ErrorCode errorCode = ErrorCode.OK;
  public Args(String schema, String[] args) throws ParseException {
    this.schema = schema;
    this.args = args;
    valid = parse();
  }

  private boolean parse() throws ParseException {
    if (schema.length() == 0 && args.length == 0)
      return true;
    parseSchema();
    parseArguments();
    return valid;
  }
  private boolean parseSchema() throws ParseException {
    for (String element : schema.split(",")) {
      if (element.length() > 0) {
        String trimmedElement = element.trim();
        parseSchemaElement(trimmedElement);
      }
    }
    return true;
  }

  private void parseSchemaElement(String element) throws ParseException {
    char elementId = element.charAt(0);
    String elementTail = element.substring(1);
    validateSchemaElementId(elementId);
    if (isBooleanSchemaElement(elementTail))
      parseBooleanSchemaElement(elementId);
    else if (isStringSchemaElement(elementTail))
```

```
    parseStringSchemaElement(elementId);
}

private void validateSchemaElementId(char elementId) throws ParseException
  {
  if (!Character.isLetter(elementId)) {
    throw new ParseException(
      "Bad character:" + elementId + "in Args format: " + schema, 0);
  }
}

private void parseStringSchemaElement(char elementId) {
  stringArgs.put(elementId, "");
}

private boolean isStringSchemaElement(String elementTail) {
  return elementTail.equals("*");
}

private boolean isBooleanSchemaElement(String elementTail) {
  return elementTail.length() == 0;
}

private void parseBooleanSchemaElement(char elementId) {
  booleanArgs.put(elementId, false);
}

private boolean parseArguments() {
  for (currentArgument = 0; currentArgument < args.length;
  currentArgument++)
  {
    String arg = args[currentArgument];
    parseArgument(arg);
  }
  return true;
}

private void parseArgument(String arg) {
  if (arg.startsWith("-"))
    parseElements(arg);
}

private void parseElements(String arg) {
  for (int i = 1; i < arg.length(); i++)
    parseElement(arg.charAt(i));
}

private void parseElement(char argChar) {
  if (setArgument(argChar))
    argsFound.add(argChar);
  else {
    unexpectedArguments.add(argChar);
    valid = false;
  }
}

private boolean setArgument(char argChar) {
  boolean set = true;
```

```
    if (isBoolean(argChar))
      setBooleanArg(argChar, true);
    else if (isString(argChar))
      setStringArg(argChar, "");
    else
      set = false;

    return set;
  }

  private void setStringArg(char argChar, String s) {
    currentArgument++;
    try {
      stringArgs.put(argChar, args[currentArgument]);
    } catch (ArrayIndexOutOfBoundsException e) {
      valid = false;
      errorArgument = argChar;
      errorCode = ErrorCode.MISSING_STRING;
    }
  }

  private boolean isString(char argChar) {
    return stringArgs.containsKey(argChar);
  }

  private void setBooleanArg(char argChar, boolean value) {
    booleanArgs.put(argChar, value);
  }

  private boolean isBoolean(char argChar) {
    return booleanArgs.containsKey(argChar);
  }

  public int cardinality() {
    return argsFound.size();
  }

  public String usage() {
    if (schema.length() > 0)
      return "-[" + schema + "]";
    else
      return "";
  }

  public String errorMessage() throws Exception {
    if (unexpectedArguments.size() > 0) {
      return unexpectedArgumentMessage();
    } else
      switch (errorCode) {
        case MISSING_STRING:
          return String.format("Could not find string parameter for -%c.",
                               errorArgument);
        case OK:
          throw new Exception("TILT: Should not get here.");
      }
    return "";
  }
```

```
    private String unexpectedArgumentMessage() {
      StringBuffer message = new StringBuffer("Argument(s) -");
      for (char c : unexpectedArguments) {
        message.append(c);
      }
      message.append(" unexpected.");

      return message.toString();
    }

    public boolean getBoolean(char arg) {
      return falseIfNull(booleanArgs.get(arg));
    }

    private boolean falseIfNull(Boolean b) {
      return b == null ? false : b;
    }

    public String getString(char arg) {
      return blankIfNull(stringArgs.get(arg));
    }

    private String blankIfNull(String s) {
      return s == null ? "" : s;
    }

    public boolean has(char arg) {
      return argsFound.contains(arg);
    }

    public boolean isValid() {
      return valid;
    }
  }
```

Bạn thấy mọi thứ bắt đầu vượt tầm kiểm soát. Vẫn chưa đến nỗi quá tệ, nhưng mớ hỗn độn rõ ràng đang lớn dần. Nó đã thành một đống bừa bộn, dù chưa thối rữa hẳn. Phải đến khi thêm kiểu đối số số nguyên, đống ấy mới thực sự lên men và bốc mùi.

Thế là tôi dừng lại

Tôi còn ít nhất hai kiểu đối số nữa cần bổ sung, và có thể đoán chúng sẽ khiến mọi chuyện tệ hơn nhiều. Nếu cứ húc đầu tiến lên, có lẽ tôi vẫn làm cho chúng chạy được, nhưng sẽ để lại một đống hỗn độn quá lớn để sửa. Nếu muốn cấu trúc mã này còn bảo trì được, thì phải sửa ngay lúc này.

Vì vậy, tôi ngừng thêm tính năng và bắt đầu tái cấu trúc. Vừa mới bổ sung đối số `String` và số nguyên, tôi nhận ra mỗi kiểu đối số đòi hỏi mã mới ở ba nơi chính. Trước hết, mỗi kiểu cần có cách phân tích phần tử trong lược đồ để chọn `HashMap` tương ứng. Kế đến, cần phân tích các chuỗi dòng lệnh và chuyển chúng sang kiểu thực. Cuối cùng, mỗi kiểu cần một phương thức `getXXX` để trả lại giá trị đúng kiểu cho bên gọi.

Nhiều kiểu khác nhau nhưng có các phương thức tương tự, nghe giống một lớp. Và thế là ý tưởng `ArgumentMarshaler` ra đời.

Từng bước một

Một trong những cách tốt nhất để phá hỏng chương trình là thay đổi lớn cấu trúc của nó dưới danh nghĩa cải tiến. Có chương trình không bao giờ hồi phục sau những "cải tiến" như thế. Vấn đề là rất khó làm cho chương trình hoạt động lại đúng như trước khi được "cải tiến".

Để tránh chuyện đó, tôi dùng kỷ luật Phát triển dựa trên kiểm thử (TDD). Một trong những nguyên lý trung tâm của cách làm này là luôn giữ cho hệ thống chạy được. Nói cách khác, khi dùng TDD, tôi không được phép tạo ra thay đổi khiến hệ thống hỏng. Mọi thay đổi tôi thực hiện đều phải giữ cho hệ thống hoạt động như trước.

Muốn làm được vậy, tôi cần một bộ kiểm thử tự động có thể chạy bất cứ lúc nào để xác nhận hành vi của hệ thống không đổi. Với lớp `Args`, tôi đã tạo một bộ kiểm thử đơn vị và kiểm thử chấp nhận trong lúc xây nên đống thối rữa ấy. Các kiểm thử đơn vị được viết bằng Java và chạy bằng JUnit. Các kiểm thử chấp nhận được viết thành những trang wiki trong FitNesse. Tôi có thể chạy chúng bất cứ khi nào, và nếu chúng đạt, tôi tự tin rằng hệ thống hoạt động đúng như đã đặc tả.

Sau đó, tôi tiến hành rất nhiều thay đổi nhỏ. Mỗi thay đổi đưa cấu trúc hệ thống đến gần hơn với ý tưởng `ArgumentMarshaler`, đồng thời vẫn giữ cho hệ thống hoạt động. Thay đổi đầu tiên là thêm bộ khung `ArgumentMarshaller` vào cuối đống mã thối rữa (Đoạn mã 14-11).

Đoạn mã 14-11
ArgumentMarshaller được thêm vào Args.java

```
private class ArgumentMarshaler {
    private boolean booleanValue = false;

    public void setBoolean(boolean value) {
      booleanValue = value;
    }

    public boolean getBoolean() {return booleanValue;}
  }

  private class BooleanArgumentMarshaler extends ArgumentMarshaler {
  }

  private class StringArgumentMarshaler extends ArgumentMarshaler {
  }
  private class IntegerArgumentMarshaler extends ArgumentMarshaler {
  }
}
```

Rõ ràng việc này không thể làm hỏng gì. Tiếp đó, tôi thực hiện thay đổi đơn giản nhất có thể, sao cho gây ảnh hưởng ít nhất. Tôi đổi `HashMap` dành cho các đối số boolean để nó lưu `ArgumentMarshaler`.

```
private Map<Character, ArgumentMarshaler> booleanArgs =
  new HashMap<Character, ArgumentMarshaler>();
```

Điều này làm hỏng vài câu lệnh, tôi nhanh chóng sửa chúng.

```
...
private void parseBooleanSchemaElement(char elementId) {
  booleanArgs.put(elementId, new BooleanArgumentMarshaler());
}
..
```

```
private void setBooleanArg(char argChar, boolean value) {
  booleanArgs.get(argChar).setBoolean(value);
}
...
public boolean getBoolean(char arg) {
  return falseIfNull(booleanArgs.get(arg).getBoolean());
}
```

Bạn có thể thấy các thay đổi này nằm đúng ở những nơi tôi vừa nói: phân tích, thiết lập và lấy giá trị của kiểu đối số. Đáng tiếc là dù thay đổi nhỏ, một số kiểm thử bắt đầu thất bại. Nếu xem kỹ `getBoolean`, bạn sẽ thấy rằng khi gọi hàm với `'y'` mà không có đối số `y`, `booleanArgs.get('y')` sẽ trả về `null` và hàm ném `NullPointerException`. Hàm `falseIfNull` vốn được dùng để xử lý trường hợp này, nhưng thay đổi của tôi khiến nó không còn phù hợp.

Tinh thần phát triển tăng dần đòi hỏi tôi phải sửa lỗi này ngay trước khi làm bất cứ thay đổi nào khác. Thật ra, cách sửa không khó. Tôi chỉ cần chuyển chỗ kiểm tra `null`. Thứ cần kiểm tra không còn là giá trị boolean bị `null`, mà là `ArgumentMarshaller`.

Trước tiên, tôi bỏ lời gọi `falseIfNull` khỏi hàm `getBoolean`. Giờ nó vô ích, nên tôi cũng xóa luôn hàm đó. Kiểm thử vẫn thất bại theo cách cũ, vì vậy tôi tin rằng mình chưa gây ra lỗi mới.

```
public boolean getBoolean(char arg) {
  return booleanArgs.get(arg).getBoolean();
}
```

Tiếp theo, tôi tách hàm thành hai dòng và đưa `ArgumentMarshaler` vào biến riêng tên là `argumentMarshaller`. Tôi không thích cái tên dài dòng này; nó thừa thãi và làm hàm thêm rối. Vì vậy tôi rút gọn thành `am` [N5].

```
public boolean getBoolean(char arg) {
  Args.ArgumentMarshaler am = booleanArgs.get(arg);
  return am.getBoolean();
}
```

Rồi tôi thêm phần phát hiện `null`.

```
public boolean getBoolean(char arg) {
  Args.ArgumentMarshaler am = booleanArgs.get(arg);
  return am != null && am.getBoolean();
}
```

Đối số kiểu chuỗi

Việc bổ sung các đối số kiểu chuỗi rất giống với việc bổ sung đối số boolean. Tôi phải đổi `HashMap` rồi làm cho các hàm phân tích, thiết lập và lấy giá trị hoạt động. Phần tiếp theo hẳn sẽ không có gì bất ngờ, ngoại trừ có lẽ việc tôi dường như đang đưa toàn bộ phần cài đặt marshaling vào lớp cơ sở `ArgumentMarshaler` thay vì phân tán nó xuống các lớp dẫn xuất.

```
private Map<Character, ArgumentMarshaler> stringArgs =
    new HashMap<Character, ArgumentMarshaler>();
...
private void parseStringSchemaElement(char elementId) {
  stringArgs.put(elementId, new StringArgumentMarshaler());
}
...
private void setStringArg(char argChar) throws ArgsException {
  currentArgument++;
  try {
    stringArgs.get(argChar).setString(args[currentArgument]);
  } catch (ArrayIndexOutOfBoundsException e) {
    valid = false;
    errorArgumentId = argChar;
    errorCode = ErrorCode.MISSING_STRING;
    throw new ArgsException();
  }
}
...
public String getString(char arg) {
  Args.ArgumentMarshaler am = stringArgs.get(arg);
  return am == null ? "" : am.getString();
}
...
private class ArgumentMarshaler {
  private boolean booleanValue = false;
  private String stringValue;

  public void setBoolean(boolean value) {
    booleanValue = value;
  }
  public boolean getBoolean() {
    return booleanValue;
  }

  public void setString(String s) {
    stringValue = s;
  }
  public String getString() {
    return stringValue == null ? "" : stringValue;
  }
```

```
  }
```

Một lần nữa, tôi thực hiện các thay đổi từng bước một để kiểm thử vẫn chạy được, dù chưa nhất thiết phải đạt. Khi một kiểm thử hỏng, tôi bảo đảm nó chạy đạt trở lại rồi mới tiếp tục thay đổi.

Đến đây chắc bạn đã thấy ý định của tôi. Sau khi chuyển toàn bộ hành vi marshaling hiện có vào lớp cơ sở `ArgumentMarshaler`, tôi sẽ bắt đầu đẩy hành vi đó xuống các lớp dẫn xuất. Nhờ vậy, tôi có thể giữ cho mọi thứ tiếp tục hoạt động trong khi dần thay đổi cấu trúc chương trình.

Bước tiếp theo hiển nhiên là chuyển chức năng xử lý đối số số nguyên vào `ArgumentMarshaler`. Lần này cũng không có gì bất ngờ.

```
  private Map<Character, ArgumentMarshaler> intArgs =
       new HashMap<Character, ArgumentMarshaler>();
  ...
  private void parseIntegerSchemaElement(char elementId) {
    intArgs.put(elementId, new IntegerArgumentMarshaler());
  }
  ...
  private void setIntArg(char argChar) throws ArgsException {
    currentArgument++;
    String parameter = null;
    try {
      parameter = args[currentArgument];
      intArgs.get(argChar).setInteger(Integer.parseInt(parameter));
    } catch (ArrayIndexOutOfBoundsException e) {
      valid = false;
      errorArgumentId = argChar;
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (NumberFormatException e) {
      valid = false;
      errorArgumentId = argChar;
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw new ArgsException();
    }
  }
  ...
  public int getInt(char arg) {
    Args.ArgumentMarshaler am = intArgs.get(arg);
    return am == null ? 0 : am.getInteger();
  }
  ...
  private class ArgumentMarshaler {
    private boolean booleanValue = false;
    private String stringValue;
    private int integerValue;

    public void setBoolean(boolean value) {
      booleanValue = value;
    }

    public boolean getBoolean() {
```

```
      return booleanValue;
    }
    public void setString(String s) {
      stringValue = s;
    }
    public String getString() {
      return stringValue == null ? "" : stringValue;
    }

    public void setInteger(int i) {
      integerValue = i;
    }

    public int getInteger() {
      return integerValue;
    }
  }
```

Sau khi chuyển toàn bộ phần marshaling vào `ArgumentMarshaler`, tôi bắt đầu đẩy chức năng xuống các lớp dẫn xuất. Bước đầu tiên là chuyển hàm `setBoolean` vào `BooleanArgumentMarshaller` và bảo đảm nó được gọi đúng. Vì thế, tôi tạo một phương thức `set` trừu tượng.

```
  private abstract class ArgumentMarshaler {
    protected boolean booleanValue = false;
    private String stringValue;
    private int integerValue;

    public void setBoolean(boolean value) {
      booleanValue = value;
    }

    public boolean getBoolean() {
      return booleanValue;
    }

    public void setString(String s) {
      stringValue = s;
    }

    public String getString() {
      return stringValue == null ? "" : stringValue;
    }

    public void setInteger(int i) {
      integerValue = i;
    }
    public int getInteger() {
      return integerValue;
    }
    public abstract void set(String s);
  }
```

Sau đó tôi cài đặt phương thức `set` trong `BooleanArgumentMarshaller`.

```
private class BooleanArgumentMarshaler extends ArgumentMarshaler {
  public void set(String s) {
    booleanValue = true;
  }
}
```

Cuối cùng, tôi thay lời gọi `setBoolean` bằng lời gọi `set`.

```
private void setBooleanArg(char argChar, boolean value) {
  booleanArgs.get(argChar).set("true");
}
```

Tất cả kiểm thử vẫn đạt. Vì thay đổi này đưa `set` xuống `BooleanArgumentMarshaler`, tôi xóa phương thức `setBoolean` khỏi lớp cơ sở `ArgumentMarshaler`.

Lưu ý rằng phương thức trừu tượng `set` nhận đối số kiểu `String`, nhưng phần cài đặt trong `BooleanArgumentMarshaller` không dùng đến nó. Tôi đưa đối số đó vào vì biết rằng `StringArgumentMarshaller` và `IntegerArgumentMarshaller` sẽ dùng.

Tiếp theo, tôi muốn chuyển phương thức `get` xuống `BooleanArgumentMarshaler`. Việc chuyển các hàm `get` luôn xấu xí, vì kiểu trả về phải là `Object`, và trong trường hợp này còn phải ép kiểu thành `Boolean`.

```
public boolean getBoolean(char arg) {
  Args.ArgumentMarshaler am = booleanArgs.get(arg);
  return am != null && (Boolean)am.get();
}
```

Chỉ để mã biên dịch được, tôi thêm hàm `get` vào `ArgumentMarshaler`.

```
private abstract class ArgumentMarshaler {
  ...

  public Object get() {
    return null;
  }
}
```

Mã biên dịch được, nhưng đương nhiên kiểm thử thất bại. Để kiểm thử đạt trở lại, chỉ cần khai báo `get` là trừu tượng và cài đặt nó trong `BooleanAgumentMarshaler`.

```
private abstract class ArgumentMarshaler {
  protected boolean booleanValue = false;
  ...
  public abstract Object get();
}

private class BooleanArgumentMarshaler extends ArgumentMarshaler {
  public void set(String s) {
    booleanValue = true;
  }
  public Object get() {
```

```
      return booleanValue;
    }
  }
```

Một lần nữa, kiểm thử đều đạt. Vậy cả `get` lẫn `set` đều đã được chuyển xuống `BooleanArgumentMarshaler`! Nhờ đó, tôi có thể xóa hàm `getBoolean` cũ khỏi `ArgumentMarshaler`, chuyển biến `booleanValue` kiểu `protected` xuống `BooleanArgumentMarshaler` và đặt nó thành `private`.

Tôi làm tương tự với kiểu chuỗi. Tôi chuyển cả `set` lẫn `get` xuống lớp con, xóa các hàm không dùng và chuyển các biến.

```
  private void setStringArg(char argChar) throws ArgsException {
    currentArgument++;
    try {
      stringArgs.get(argChar).set(args[currentArgument]);
    } catch (ArrayIndexOutOfBoundsException e) {
      valid = false;
      errorArgumentId = argChar;
      errorCode = ErrorCode.MISSING_STRING;
      throw new ArgsException();
    }
  }
  ...
  public String getString(char arg) {
    Args.ArgumentMarshaler am = stringArgs.get(arg);
    return am == null ? "" : (String) am.get();
  }
  ...
  private abstract class ArgumentMarshaler {
    private int integerValue;

    public void setInteger(int i) {
      integerValue = i;
    }

    public int getInteger() {
      return integerValue;
    }

    public abstract void set(String s);

    public abstract Object get();
  }
  private class BooleanArgumentMarshaler extends ArgumentMarshaler {
    private boolean booleanValue = false;
    public void set(String s) {
      booleanValue = true;
    }
    public Object get() {
      return booleanValue;
    }
  }

  private class StringArgumentMarshaler extends ArgumentMarshaler {
    private String stringValue = "";
```

```
    public void set(String s) {
      stringValue = s;
    }

    public Object get() {
      return stringValue;
    }
  }
  private class IntegerArgumentMarshaler extends ArgumentMarshaler {

    public void set(String s) {

    }

    public Object get() {
      return null;
    }
  }
  }
```

Cuối cùng, tôi lặp lại quá trình này với số nguyên. Việc đó phức tạp hơn đôi chút vì số nguyên cần được phân tích cú pháp, và thao tác phân tích có thể ném ngoại lệ. Nhưng kết quả tốt hơn, vì toàn bộ khái niệm `NumberFormatException` được giấu bên trong `IntegerArgumentMarshaler`.

```
  private boolean isIntArg(char argChar) {return intArgs.containsKey(argChar);}

  private void setIntArg(char argChar) throws ArgsException {
    currentArgument++;
    String parameter = null;
    try {
      parameter = args[currentArgument];
      intArgs.get(argChar).set(parameter);
    } catch (ArrayIndexOutOfBoundsException e) {
      valid = false;
      errorArgumentId = argChar;
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (ArgsException e) {
      valid = false;
      errorArgumentId = argChar;
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw e;
    }
  }
  ...
  private void setBooleanArg(char argChar) {
    try {
      booleanArgs.get(argChar).set("true");
    } catch (ArgsException e) {
    }
  }
  ...
  public int getInt(char arg) {
    Args.ArgumentMarshaler am = intArgs.get(arg);
```

```
    return am == null ? 0 : (Integer) am.get();
  }
  ...
  private abstract class ArgumentMarshaler {
    public abstract void set(String s) throws ArgsException;
    public abstract Object get();
  }
  ...
  private class IntegerArgumentMarshaler extends ArgumentMarshaler {
    private int intValue = 0;
    public void set(String s) throws ArgsException {
      try {
        intValue = Integer.parseInt(s);
      } catch (NumberFormatException e) {
        throw new ArgsException();
      }
    }

    public Object get() {
      return intValue;
    }
  }
```

Dĩ nhiên, kiểm thử vẫn tiếp tục đạt. Tiếp theo, tôi loại bỏ ba map riêng biệt ở đầu thuật toán. Điều này khiến toàn hệ thống trở nên tổng quát hơn. Tuy nhiên, tôi không thể xóa chúng ngay vì như vậy hệ thống sẽ hỏng. Thay vào đó, tôi thêm một `Map` mới dành cho `ArgumentMarshaler`, rồi lần lượt đổi các phương thức để dùng map này thay cho ba map ban đầu.

```
  public class Args {
  ...
    private Map<Character, ArgumentMarshaler> booleanArgs =
      new HashMap<Character, ArgumentMarshaler>();
    private Map<Character, ArgumentMarshaler> stringArgs =
      new HashMap<Character, ArgumentMarshaler>();
    private Map<Character, ArgumentMarshaler> intArgs =
      new HashMap<Character, ArgumentMarshaler>();
    private Map<Character, ArgumentMarshaler> marshalers =
      new HashMap<Character, ArgumentMarshaler>();
  ...
    private void parseBooleanSchemaElement(char elementId) {
      ArgumentMarshaler m = new BooleanArgumentMarshaler();
      booleanArgs.put(elementId, m);
      marshalers.put(elementId, m);
    }

    private void parseIntegerSchemaElement(char elementId) {
      ArgumentMarshaler m = new IntegerArgumentMarshaler();
      intArgs.put(elementId, m);
      marshalers.put(elementId, m);
    }

    private void parseStringSchemaElement(char elementId) {
      ArgumentMarshaler m = new StringArgumentMarshaler();
      stringArgs.put(elementId, m);
      marshalers.put(elementId, m);
```

```
    }
```

Tất nhiên, mọi kiểm thử vẫn đạt. Tiếp theo, tôi đổi `isBooleanArg` từ dạng này:

```
private boolean isBooleanArg(char argChar) {
  return booleanArgs.containsKey(argChar);
}
```

sang dạng này:

```
private boolean isBooleanArg(char argChar) {
  ArgumentMarshaler m = marshalers.get(argChar);
  return m instanceof BooleanArgumentMarshaler;
}
```

Kiểm thử vẫn đạt. Vì vậy, tôi thực hiện thay đổi tương tự với `isIntArg` và `isStringArg`.

```
private boolean isIntArg(char argChar) {
  ArgumentMarshaler m = marshalers.get(argChar);
  return m instanceof IntegerArgumentMarshaler;
}
private boolean isStringArg(char argChar) {
  ArgumentMarshaler m = marshalers.get(argChar);
  return m instanceof StringArgumentMarshaler;
}
```

Kiểm thử vẫn đạt. Vì vậy, tôi loại bỏ các lần gọi `marshalers.get` bị lặp như sau:

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  if (isBooleanArg(m))
    setBooleanArg(argChar);
  else if (isStringArg(m))
    setStringArg(argChar);
  else if (isIntArg(m))
    setIntArg(argChar);
  else
    return false;
  return true;
}

private boolean isIntArg(ArgumentMarshaler m) {
  return m instanceof IntegerArgumentMarshaler;
}
private boolean isStringArg(ArgumentMarshaler m) {
  return m instanceof StringArgumentMarshaler;
}
private boolean isBooleanArg(ArgumentMarshaler m) {
  return m instanceof BooleanArgumentMarshaler;
}
```

Ba hàm `isxxxArg` giờ chẳng còn lý do tồn tại. Vì vậy, tôi đưa nội dung của chúng vào ngay chỗ gọi.

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  if (m instanceof BooleanArgumentMarshaler)
    setBooleanArg(argChar);
  else if (m instanceof StringArgumentMarshaler)
    setStringArg(argChar);
  else if (m instanceof IntegerArgumentMarshaler)
    setIntArg(argChar);
  else
    return false;

  return true;
}
```

Tiếp theo, tôi bắt đầu dùng map `marshalers` trong các hàm `set`, bỏ dần việc dùng ba map kia. Tôi bắt đầu với boolean.

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  if (m instanceof BooleanArgumentMarshaler)
    setBooleanArg(m);
  else if (m instanceof StringArgumentMarshaler)
    setStringArg(argChar);
  else if (m instanceof IntegerArgumentMarshaler)
    setIntArg(argChar);
  else
    return false;

  return true;
}
...
private void setBooleanArg(ArgumentMarshaler m) {
  try {
    m.set("true"); // was: booleanArgs.get(argChar).set("true");
  } catch (ArgsException e) {
  }
}
```

Kiểm thử vẫn đạt, nên tôi làm tương tự với chuỗi và số nguyên. Nhờ vậy, tôi có thể gộp một phần mã xử lý ngoại lệ xấu xí vào hàm `setArgument`.

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  try {
    if (m instanceof BooleanArgumentMarshaler)
      setBooleanArg(m);
    else if (m instanceof StringArgumentMarshaler)
      setStringArg(m);
    else if (m instanceof IntegerArgumentMarshaler)
      setIntArg(m);
    else
      return false;
  } catch (ArgsException e) {
    valid = false;
    errorArgumentId = argChar;
```

```
      throw e;
    }
    return true;
  }
  private void setIntArg(ArgumentMarshaler m) throws ArgsException {
    currentArgument++;
    String parameter = null;
    try {
      parameter = args[currentArgument];
      m.set(parameter);
    } catch (ArrayIndexOutOfBoundsException e) {
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (ArgsException e) {
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw e;
    }
  }

  private void setStringArg(ArgumentMarshaler m) throws ArgsException {
    currentArgument++;
    try {
      m.set(args[currentArgument]);
    } catch (ArrayIndexOutOfBoundsException e) {
      errorCode = ErrorCode.MISSING_STRING;
      throw new ArgsException();
    }
  }
```

Tôi sắp có thể loại bỏ ba map cũ. Trước tiên, tôi cần đổi hàm `getBoolean` từ dạng này:

```
  public boolean getBoolean(char arg) {
    Args.ArgumentMarshaler am = booleanArgs.get(arg);
    return am != null && (Boolean) am.get();
  }
```

sang dạng này:

```
  public boolean getBoolean(char arg) {
    Args.ArgumentMarshaler am = marshalers.get(arg);
    boolean b = false;
    try {
      b = am != null && (Boolean) am.get();
    } catch (ClassCastException e) {
      b = false;
    }
    return b;
  }
```

Thay đổi cuối cùng này có thể khiến bạn ngạc nhiên. Vì sao đột nhiên tôi lại xử lý `ClassCastException`? Bởi vì tôi có một bộ kiểm thử đơn vị và một bộ kiểm thử chấp nhận riêng, viết bằng FitNesse. Hóa ra các kiểm thử FitNesse bảo đảm rằng nếu gọi `getBoolean` với một đối số không phải boolean thì kết quả là `false`. Bộ kiểm thử đơn vị thì không kiểm

tra điều đó. Cho đến lúc này, tôi chỉ chạy kiểm thử đơn vị.[2]

Thay đổi này cho phép tôi bỏ thêm một chỗ dùng map boolean:

```
private void parseBooleanSchemaElement(char elementId) {
  ArgumentMarshaler m = new BooleanArgumentMarshaler();
  booleanArgs.put(elementId, m);
  marshalers.put(elementId, m);
}
```

Giờ ta có thể xóa map boolean.

```
public class Args {
...
  private Map<Character, ArgumentMarshaler> booleanArgs =
  new HashMap<Character, ArgumentMarshaler>();
  private Map<Character, ArgumentMarshaler> stringArgs =
  new HashMap<Character, ArgumentMarshaler>();
  private Map<Character, ArgumentMarshaler> intArgs =
  new HashMap<Character, ArgumentMarshaler>();
  private Map<Character, ArgumentMarshaler> marshalers =
  new HashMap<Character, ArgumentMarshaler>();
...
```

Tiếp theo, tôi chuyển các đối số kiểu chuỗi và số nguyên theo cách tương tự, đồng thời chỉnh sửa đôi chút phần boolean.

```
private void parseBooleanSchemaElement(char elementId) {
  marshalers.put(elementId, new BooleanArgumentMarshaler());
}
```

```
private void parseIntegerSchemaElement(char elementId) {
  marshalers.put(elementId, new IntegerArgumentMarshaler());
}

private void parseStringSchemaElement(char elementId) {
  marshalers.put(elementId, new StringArgumentMarshaler());
}
...
public String getString(char arg) {
  Args.ArgumentMarshaler am = marshalers.get(arg);
  try {
    return am == null ? "" : (String) am.get();
  } catch (ClassCastException e) {
    return "";
  }
}

public int getInt(char arg) {
  Args.ArgumentMarshaler am = marshalers.get(arg);
  try {
    return am == null ? 0 : (Integer) am.get();
```

2. Để tránh gặp thêm bất ngờ kiểu này, tôi bổ sung một kiểm thử đơn vị mới gọi toàn bộ kiểm thử FitNesse.

```
    } catch (Exception e) {
      return 0;
    }
  }
  ...
  public class Args {
  ...
    private Map<Character, ArgumentMarshaler> stringArgs =
    new HashMap<Character, ArgumentMarshaler>();
    private Map<Character, ArgumentMarshaler> intArgs =
    new HashMap<Character, ArgumentMarshaler>();
    private Map<Character, ArgumentMarshaler> marshalers =
    new HashMap<Character, ArgumentMarshaler>();
  ...
```

Tiếp theo, tôi gộp ba hàm phân tích vào một, vì giờ chúng chẳng còn làm được bao nhiêu:

```
  private void parseSchemaElement(String element) throws ParseException {
    char elementId = element.charAt(0);
    String elementTail = element.substring(1);
    validateSchemaElementId(elementId);
    if (isBooleanSchemaElement(elementTail))
      marshalers.put(elementId, new BooleanArgumentMarshaler());
    else if (isStringSchemaElement(elementTail))
      marshalers.put(elementId, new StringArgumentMarshaler());
    else if (isIntegerSchemaElement(elementTail)) {
      marshalers.put(elementId, new IntegerArgumentMarshaler());
    } else {
      throw new ParseException(String.format(
      "Argument: %c has invalid format: %s.", elementId, elementTail), 0);
    }
  }
```

Được rồi, giờ hãy nhìn lại toàn bộ cấu trúc. Đoạn mã 14-12 trình bày hình thức hiện tại của lớp `Args`.

Đoạn mã 14-12
Args.java (Sau lần tái cấu trúc đầu tiên)

```
package com.objectmentor.utilities.getopts;

import java.text.ParseException;
import java.util.*;

public class Args {
  private String schema;
  private String[] args;
  private boolean valid = true;
  private Set<Character> unexpectedArguments = new TreeSet<Character>();
  private Map<Character, ArgumentMarshaler> marshalers =
  new HashMap<Character, ArgumentMarshaler>();
  private Set<Character> argsFound = new HashSet<Character>();
  private int currentArgument;
  private char errorArgumentId = '\0';
  private String errorParameter = "TILT";
```

```
  private ErrorCode errorCode = ErrorCode.OK;
  private enum ErrorCode {
    OK, MISSING_STRING, MISSING_INTEGER, INVALID_INTEGER,
    UNEXPECTED_ARGUMENT}

  public Args(String schema, String[] args) throws ParseException {
    this.schema = schema;
    this.args = args;
    valid = parse();
  }

  private boolean parse() throws ParseException {
    if (schema.length() == 0 && args.length == 0)
      return true;
    parseSchema();
    try {
      parseArguments();
    } catch (ArgsException e) {
    }
    return valid;
  }

  private boolean parseSchema() throws ParseException {
    for (String element : schema.split(",")) {
      if (element.length() > 0) {
        String trimmedElement = element.trim();
        parseSchemaElement(trimmedElement);
      }
    }
    return true;
  }

  private void parseSchemaElement(String element) throws ParseException {
    char elementId = element.charAt(0);
    String elementTail = element.substring(1);
    validateSchemaElementId(elementId);
    if (isBooleanSchemaElement(elementTail))
      marshalers.put(elementId, new BooleanArgumentMarshaler());
    else if (isStringSchemaElement(elementTail))
      marshalers.put(elementId, new StringArgumentMarshaler());
    else if (isIntegerSchemaElement(elementTail)) {
      marshalers.put(elementId, new IntegerArgumentMarshaler());
    } else {
      throw new ParseException(String.format(
      "Argument: %c has invalid format: %s.", elementId, elementTail), 0);
    }
  }

  private void validateSchemaElementId(char elementId) throws ParseException
    {
    if (!Character.isLetter(elementId)) {
      throw new ParseException(
      "Bad character:" + elementId + "in Args format: " + schema, 0);
    }
  }
  private boolean isStringSchemaElement(String elementTail) {
    return elementTail.equals("*");
  }
```

```
  private boolean isBooleanSchemaElement(String elementTail) {
    return elementTail.length() == 0;
  }

  private boolean isIntegerSchemaElement(String elementTail) {
    return elementTail.equals("#");
  }

  private boolean parseArguments() throws ArgsException {
    for (currentArgument=0; currentArgument<args.length; currentArgument++) {
      String arg = args[currentArgument];
      parseArgument(arg);
    }
    return true;
  }

  private void parseArgument(String arg) throws ArgsException {
    if (arg.startsWith("-"))
      parseElements(arg);
  }

  private void parseElements(String arg) throws ArgsException {
    for (int i = 1; i < arg.length(); i++)
      parseElement(arg.charAt(i));
  }

  private void parseElement(char argChar) throws ArgsException {
    if (setArgument(argChar))
      argsFound.add(argChar);
    else {
      unexpectedArguments.add(argChar);
      errorCode = ErrorCode.UNEXPECTED_ARGUMENT;
      valid = false;
    }
  }
```

Đoạn mã 14-12 (tiếp)
Args.java (Sau lần tái cấu trúc đầu tiên)

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  try {
    if (m instanceof BooleanArgumentMarshaler)
      setBooleanArg(m);
    else if (m instanceof StringArgumentMarshaler)
      setStringArg(m);
    else if (m instanceof IntegerArgumentMarshaler)
      setIntArg(m);
    else
      return false;
  } catch (ArgsException e) {
    valid = false;
    errorArgumentId = argChar;
    throw e;
  }
```

```
    return true;
  }

  private void setIntArg(ArgumentMarshaler m) throws ArgsException {
    currentArgument++;
    String parameter = null;
    try {
      parameter = args[currentArgument];
      m.set(parameter);
    } catch (ArrayIndexOutOfBoundsException e) {
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (ArgsException e) {
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw e;
    }
  }

  private void setStringArg(ArgumentMarshaler m) throws ArgsException {
    currentArgument++;
    try {
      m.set(args[currentArgument]);
    } catch (ArrayIndexOutOfBoundsException e) {
      errorCode = ErrorCode.MISSING_STRING;
      throw new ArgsException();
    }
  }

  private void setBooleanArg(ArgumentMarshaler m) {
    try {
      m.set("true");
    } catch (ArgsException e) {
    }
  }

  public int cardinality() {
    return argsFound.size();
  }
```

Đoạn mã 14-12 (tiếp)
Args.java (Sau lần tái cấu trúc đầu tiên)

```
  public String usage() {
    if (schema.length() > 0)
      return "-[" + schema + "]";
    else
      return "";
  }

  public String errorMessage() throws Exception {
    switch (errorCode) {
      case OK:
        throw new Exception("TILT: Should not get here.");
      case UNEXPECTED_ARGUMENT:
        return unexpectedArgumentMessage();
```

```
      case MISSING_STRING:
        return String.format("Could not find string parameter for -%c.",
                             errorArgumentId);
      case INVALID_INTEGER:
        return String.format("Argument -%c expects an integer but was '%s'.",
                             errorArgumentId, errorParameter);
      case MISSING_INTEGER:
        return String.format("Could not find integer parameter for -%c.",
                             errorArgumentId);
    }
    return "";
  }

  private String unexpectedArgumentMessage() {
    StringBuffer message = new StringBuffer("Argument(s) -");
    for (char c : unexpectedArguments) {
      message.append(c);
    }
    message.append(" unexpected.");

    return message.toString();
  }
  public boolean getBoolean(char arg) {
    Args.ArgumentMarshaler am = marshalers.get(arg);
    boolean b = false;
    try {
      b = am != null && (Boolean) am.get();
    } catch (ClassCastException e) {
      b = false;
    }
    return b;
  }

  public String getString(char arg) {
    Args.ArgumentMarshaler am = marshalers.get(arg);
    try {
      return am == null ? "" : (String) am.get();
    } catch (ClassCastException e) {
      return "";
    }
  }
```

Đoạn mã 14-12 (tiếp)
Args.java (Sau lần tái cấu trúc đầu tiên)

```
  public int getInt(char arg) {
    Args.ArgumentMarshaler am = marshalers.get(arg);
    try {
      return am == null ? 0 : (Integer) am.get();
    } catch (Exception e) {
      return 0;
    }
  }

  public boolean has(char arg) {
    return argsFound.contains(arg);
```

```
  }

  public boolean isValid() {
    return valid;
  }

  private class ArgsException extends Exception {
  }

  private abstract class ArgumentMarshaler {
    public abstract void set(String s) throws ArgsException;
    public abstract Object get();
  }
  private class BooleanArgumentMarshaler extends ArgumentMarshaler {
    private boolean booleanValue = false;

    public void set(String s) {
      booleanValue = true;
    }

    public Object get() {
      return booleanValue;
    }
  }
  private class StringArgumentMarshaler extends ArgumentMarshaler {
    private String stringValue = "";

    public void set(String s) {
      stringValue = s;
    }

    public Object get() {
      return stringValue;
    }
  }

  private class IntegerArgumentMarshaler extends ArgumentMarshaler {
    private int intValue = 0;

    public void set(String s) throws ArgsException {
      try {
        intValue = Integer.parseInt(s);
      } catch (NumberFormatException e) {
        throw new ArgsException();
      }
    }

    public Object get() {
      return intValue;
    }
  }
  }
```

Sau tất cả công việc ấy, kết quả hơi đáng thất vọng. Cấu trúc có khá hơn đôi chút, nhưng tất cả biến vẫn nằm ở đầu lớp; trong `setArgument` vẫn còn một nhánh rẽ kiểu dữ liệu thật kinh khủng; các hàm `set` đều rất xấu. Chưa kể đến toàn bộ phần xử lý lỗi. Chúng ta còn nhiều việc phải làm.

Tôi rất muốn loại bỏ nhánh rẽ kiểu dữ liệu trong `setArgument` [G23]. Điều tôi muốn ở `setArgument` là một lời gọi duy nhất đến `ArgumentMarshaler.set`. Như vậy, tôi cần chuyển `setIntArg`, `setStringArg` và `setBooleanArg` xuống các lớp dẫn xuất `ArgumentMarshaler` tương ứng. Nhưng có một vấn đề.

Nếu xem kỹ `setIntArg`, bạn sẽ thấy nó dùng hai biến thực thể: `args` và `currentArg`. Muốn chuyển `setIntArg` xuống `BooleanArgumentMarshaler`, tôi phải truyền cả `args` lẫn `currentArgs` làm đối số cho hàm. Làm vậy không ổn [F1]. Tôi muốn chỉ truyền một đối số thay vì hai. May thay, có một giải pháp đơn giản. Ta có thể chuyển mảng `args` thành danh sách rồi truyền một `Iterator` xuống các hàm `set`. Tôi đã thực hiện việc này qua mười bước, sau mỗi bước đều chạy được tất cả kiểm thử. Nhưng tôi sẽ chỉ trình bày kết quả. Có lẽ bạn sẽ hình dung được phần lớn các bước nhỏ đó.

```
public class Args {
  private String schema;
  private String[] args;
  private boolean valid = true;
  private Set<Character> unexpectedArguments = new TreeSet<Character>();
  private Map<Character, ArgumentMarshaler> marshalers =
  new HashMap<Character, ArgumentMarshaler>();
  private Set<Character> argsFound = new HashSet<Character>();
  private Iterator<String> currentArgument;
  private char errorArgumentId = '\0';
  private String errorParameter = "TILT";
  private ErrorCode errorCode = ErrorCode.OK;
  private List<String> argsList;

  private enum ErrorCode {
    OK, MISSING_STRING, MISSING_INTEGER, INVALID_INTEGER,
    UNEXPECTED_ARGUMENT}

  public Args(String schema, String[] args) throws ParseException {
    this.schema = schema;
    argsList = Arrays.asList(args);
    valid = parse();
  }

  private boolean parse() throws ParseException {
    if (schema.length() == 0 && argsList.size() == 0)
      return true;
    parseSchema();
    try {
      parseArguments();
    } catch (ArgsException e) {
    }
    return valid;
  }
  ---
  private boolean parseArguments() throws ArgsException {
    for (currentArgument = argsList.iterator(); currentArgument.hasNext();) {
      String arg = currentArgument.next();
      parseArgument(arg);
    }

    return true;
```

```
  }
  ---
  private void setIntArg(ArgumentMarshaler m) throws ArgsException {
    String parameter = null;
    try {
      parameter = currentArgument.next();
      m.set(parameter);
    } catch (NoSuchElementException e) {
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (ArgsException e) {
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw e;
    }
  }

  private void setStringArg(ArgumentMarshaler m) throws ArgsException {
    try {
      m.set(currentArgument.next());
    } catch (NoSuchElementException e) {
      errorCode = ErrorCode.MISSING_STRING;
      throw new ArgsException();
    }
  }
```

Đây là những thay đổi đơn giản, sau mỗi thay đổi tất cả kiểm thử vẫn chạy đạt. Giờ ta có thể bắt đầu chuyển các hàm `set` xuống các lớp dẫn xuất tương ứng. Trước tiên, tôi cần thay đổi `setArgument` như sau:

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  if (m == null)
    return false;
  try {
    if (m instanceof BooleanArgumentMarshaler)
      setBooleanArg(m);
    else if (m instanceof StringArgumentMarshaler)
      setStringArg(m);
    else if (m instanceof IntegerArgumentMarshaler)
      setIntArg(m);
    else
      return false;
  } catch (ArgsException e) {
    valid = false;
    errorArgumentId = argChar;
    throw e;
  }
  return true;
}
```

Thay đổi này quan trọng vì ta muốn loại bỏ hoàn toàn chuỗi `if-else`. Do đó, trước hết cần đưa điều kiện báo lỗi ra khỏi chuỗi đó.

Giờ ta có thể bắt đầu chuyển các hàm `set`. Hàm `setBooleanArg` rất đơn giản, nên ta chuẩn bị nó trước. Mục tiêu là đổi hàm `setBooleanArg` để nó chỉ chuyển tiếp lời gọi đến

`BooleanArgumentMarshaler`.

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  if (m == null)
    return false;
  try {
    if (m instanceof BooleanArgumentMarshaler)
      setBooleanArg(m, currentArgument);
    else if (m instanceof StringArgumentMarshaler)
      setStringArg(m);
    else if (m instanceof IntegerArgumentMarshaler)
      setIntArg(m);
  } catch (ArgsException e) {
    valid = false;
    errorArgumentId = argChar;
    throw e;
  }
  return true;
}
---
private void setBooleanArg(ArgumentMarshaler m,
                           Iterator<String> currentArgument)
                           throws ArgsException {
  try {
    m.set("true");
  } catch (ArgsException e) {
  }
}
```

Chẳng phải ta vừa mới đưa phần xử lý ngoại lệ vào đó sao? Trong quá trình tái cấu trúc, việc đưa một thứ vào để rồi lại lấy nó ra là chuyện thường gặp. Các bước nhỏ và yêu cầu giữ cho kiểm thử luôn chạy khiến ta phải di chuyển mọi thứ nhiều lần. Tái cấu trúc cũng giống như giải khối Rubik. Cần rất nhiều bước nhỏ để đạt mục tiêu lớn. Mỗi bước mở đường cho bước tiếp theo.

Vì sao ta truyền iterator vào khi `setBooleanArg` chắc chắn không cần đến nó? Vì `setIntArg` và `setStringArg` sẽ cần! Và vì tôi muốn chuyển cả ba hàm này thông qua một phương thức trừu tượng trong `ArgumentMarshaller`, nên phải truyền iterator cho `setBooleanArg`.

`setBooleanArg` trở nên thừa. Nếu trong `ArgumentMarshaler` có phương thức `set`, ta có thể gọi trực tiếp. Vậy đã đến lúc tạo phương thức đó. Bước đầu tiên là thêm phương thức trừu tượng mới vào `ArgumentMarshaler`.

```
private abstract class ArgumentMarshaler {
  public abstract void set(Iterator<String> currentArgument)
                       throws ArgsException;
  public abstract void set(String s) throws ArgsException;
  public abstract Object get();
}
```

Dĩ nhiên, việc này làm hỏng tất cả lớp dẫn xuất. Vậy hãy cài đặt phương thức mới trong từng lớp.

```
  private class BooleanArgumentMarshaler extends ArgumentMarshaler {
    private boolean booleanValue = false;

    public void set(Iterator<String> currentArgument) throws ArgsException {
      booleanValue = true;
    }

    public void set(String s) {
      booleanValue = true;
    }

    public Object get() {
      return booleanValue;
    }
  }

  private class StringArgumentMarshaler extends ArgumentMarshaler {
    private String stringValue = "";

    public void set(Iterator<String> currentArgument) throws ArgsException {
    }

    public void set(String s) {
      stringValue = s;
    }

    public Object get() {
      return stringValue;
    }
  }

  private class IntegerArgumentMarshaler extends ArgumentMarshaler {
    private int intValue = 0;

    public void set(Iterator<String> currentArgument) throws ArgsException {
    }
    public void set(String s) throws ArgsException {
      try {
        intValue = Integer.parseInt(s);
      } catch (NumberFormatException e) {
        throw new ArgsException();
      }
    }

    public Object get() {
      return intValue;
    }
  }
```

Giờ ta có thể xóa `setBooleanArg`!

```
  private boolean setArgument(char argChar) throws ArgsException {
      ArgumentMarshaler m = marshalers.get(argChar);
      if (m == null)
        return false;
      try {
        if (m instanceof BooleanArgumentMarshaler)
```

```
        m.set(currentArgument);
      else if (m instanceof StringArgumentMarshaler)
        setStringArg(m);
      else if (m instanceof IntegerArgumentMarshaler)
        setIntArg(m);
      } catch (ArgsException e) {
        valid = false;
        errorArgumentId = argChar;
        throw e;
      }
      return true;
}
```

Tất cả kiểm thử đều qua, và phương thức `set` đã được chuyển xuống `BooleanArgumentMarshaler`!

Giờ ta làm tương tự với chuỗi và số nguyên.

```
private boolean setArgument(char argChar) throws ArgsException {
  ArgumentMarshaler m = marshalers.get(argChar);
  if (m == null)
    return false;
  try {
    if (m instanceof BooleanArgumentMarshaler)
      m.set(currentArgument);
    else if (m instanceof StringArgumentMarshaler)
      m.set(currentArgument);
    else if (m instanceof IntegerArgumentMarshaler)
      m.set(currentArgument);
  } catch (ArgsException e) {
    valid = false;
    errorArgumentId = argChar;
    throw e;
  }
  return true;
}
---
private class StringArgumentMarshaler extends ArgumentMarshaler {
  private String stringValue = "";

  public void set(Iterator<String> currentArgument) throws ArgsException {
    try {
      stringValue = currentArgument.next();
    } catch (NoSuchElementException e) {
      errorCode = ErrorCode.MISSING_STRING;
      throw new ArgsException();
    }
  }

  public void set(String s) {
  }

  public Object get() {
    return stringValue;
  }
}
```

```
  private class IntegerArgumentMarshaler extends ArgumentMarshaler {
    private int intValue = 0;
  public void set(Iterator<String> currentArgument) throws ArgsException {
    String parameter = null;
    try {
      parameter = currentArgument.next();
      set(parameter);
    } catch (NoSuchElementException e) {
      errorCode = ErrorCode.MISSING_INTEGER;
      throw new ArgsException();
    } catch (ArgsException e) {
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_INTEGER;
      throw e;
    }
  }
    public void set(String s) throws ArgsException {
      try {
        intValue = Integer.parseInt(s);
      } catch (NumberFormatException e) {
        throw new ArgsException();
      }
    }

    public Object get() {
      return intValue;
    }
  }
```

Và thế là có thể tung đòn kết liễu: bỏ hẳn đoạn rẽ nhánh theo kiểu dữ liệu! Chạm đích!

```
  private boolean setArgument(char argChar) throws ArgsException {
    ArgumentMarshaler m = marshalers.get(argChar);
    if (m == null)
      return false;
    try {
      m.set(currentArgument);
      return true;
    } catch (ArgsException e) {
      valid = false;
      errorArgumentId = argChar;
      throw e;
    }
  }
```

Giờ ta có thể bỏ một số hàm cũ kỹ trong `IntegerArgumentMarshaler` và dọn nó gọn hơn.

```
  private class IntegerArgumentMarshaler extends ArgumentMarshaler {
    private int intValue = 0

    public void set(Iterator<String> currentArgument) throws ArgsException {
      String parameter = null;
      try {
        parameter = currentArgument.next();
        intValue = Integer.parseInt(parameter);
```

```
      } catch (NoSuchElementException e) {
        errorCode = ErrorCode.MISSING_INTEGER;
        throw new ArgsException();
      } catch (NumberFormatException e) {
        errorParameter = parameter;
        errorCode = ErrorCode.INVALID_INTEGER;
        throw new ArgsException();
      }
    }
    public Object get() {
      return intValue;
    }
  }
```

Ta cũng có thể đổi `ArgumentMarshaler` thành một interface.

```
  private interface ArgumentMarshaler {
    void set(Iterator<String> currentArgument) throws ArgsException;
    Object get();
  }
```

Giờ hãy xem việc thêm kiểu đối số mới vào cấu trúc này dễ đến đâu. Ta chỉ cần rất ít thay đổi, và các thay đổi đó nên được cô lập. Trước tiên, ta thêm một ca kiểm thử để xác nhận đối số kiểu double hoạt động đúng.

```
  public void testSimpleDoublePresent() throws Exception {
    Args args = new Args("x##", new String[] {"-x","42.3"});
    assertTrue(args.isValid());
    assertEquals(1, args.cardinality());
    assertTrue(args.has('x'));
    assertEquals(42.3, args.getDouble('x'), .001);
  }
```

Tiếp theo, ta dọn mã phân tích schema và thêm phần nhận diện ## cho kiểu đối số double.

```
  private void parseSchemaElement(String element) throws ParseException {
    char elementId = element.charAt(0);
    String elementTail = element.substring(1);
    validateSchemaElementId(elementId);
    if (elementTail.length() == 0)
      marshalers.put(elementId, new BooleanArgumentMarshaler());
    else if (elementTail.equals("*"))
      marshalers.put(elementId, new StringArgumentMarshaler());
    else if (elementTail.equals("#"))
      marshalers.put(elementId, new IntegerArgumentMarshaler());
      else if (elementTail.equals("##"))
        marshalers.put(elementId, new DoubleArgumentMarshaler());
      else
        throw new ParseException(String.format(
           "Argument: %c has invalid format: %s.", elementId, elementTail), 0);
  }
```

Tiếp theo, ta viết lớp `DoubleArgumentMarshaler`.

```
private class DoubleArgumentMarshaler implements ArgumentMarshaler {
  private double doubleValue = 0;

  public void set(Iterator<String> currentArgument) throws ArgsException {
    String parameter = null;
    try {
      parameter = currentArgument.next();
      doubleValue = Double.parseDouble(parameter);
    } catch (NoSuchElementException e) {
      errorCode = ErrorCode.MISSING_DOUBLE;
      throw new ArgsException();
    } catch (NumberFormatException e) {
      errorParameter = parameter;
      errorCode = ErrorCode.INVALID_DOUBLE;
      throw new ArgsException();
    }
  }

  public Object get() {
    return doubleValue;
  }
}
```

Việc này buộc ta phải thêm một `ErrorCode` mới.

```
private enum ErrorCode {
    OK, MISSING_STRING, MISSING_INTEGER, INVALID_INTEGER,
    UNEXPECTED_ARGUMENT,
    MISSING_DOUBLE, INVALID_DOUBLE}
```

Và cần thêm hàm `getDouble`.

```
public double getDouble(char arg) {
  Args.ArgumentMarshaler am = marshalers.get(arg);
  try {
    return am == null ? 0 : (Double) am.get();
  } catch (Exception e) {
    return 0.0;
  }
}
```

Tất cả kiểm thử đều qua! Việc này khá nhẹ nhàng. Giờ hãy kiểm tra xem phần xử lý lỗi có hoạt động đúng không. Ca kiểm thử tiếp theo xác nhận rằng lỗi được báo khi truyền một chuỗi không thể phân tích vào đối số ##.

```
public void testInvalidDouble() throws Exception {
  Args args = new Args("x##", new String[] {"-x","Forty two"});
  assertFalse(args.isValid());
  assertEquals(0, args.cardinality());
  assertFalse(args.has('x'));
  assertEquals(0, args.getInt('x'));
  assertEquals("Argument -x expects a double but was 'Forty two'.",
               args.errorMessage());
}
```

```
---
public String errorMessage() throws Exception {
   switch (errorCode) {
     case OK:
       throw new Exception("TILT: Should not get here.");
     case UNEXPECTED_ARGUMENT:
       return unexpectedArgumentMessage();
     case MISSING_STRING:
       return String.format("Could not find string parameter for -%c.",
                            errorArgumentId);
     case INVALID_INTEGER:
       return String.format("Argument -%c expects an integer but was '%s'.",
                            errorArgumentId, errorParameter);
     case MISSING_INTEGER:
       return String.format("Could not find integer parameter for -%c.",
                            errorArgumentId);
     case INVALID_DOUBLE:
       return String.format("Argument -%c expects a double but was '%s'.",
                            errorArgumentId, errorParameter);
     case MISSING_DOUBLE:
       return String.format("Could not find double parameter for -%c.",
                            errorArgumentId);
   }
   return "";
}
```

Ca này qua đúng như dự kiến. Ta viết nó chỉ để đầy đủ.

Mã xử lý ngoại lệ khá xấu và không thực sự thuộc về lớp `Args`. Ta cũng đang ném `ParseException`, vốn chẳng thuộc về ta. Vậy hãy gộp tất cả ngoại lệ vào một lớp `ArgsException` rồi chuyển lớp đó sang module riêng.

```
public class ArgsException extends Exception {
  private char errorArgumentId = '\0';
  private String errorParameter = "TILT";
  private ErrorCode errorCode = ErrorCode.OK;
  public ArgsException() {}

  public ArgsException(String message) {super(message);}

  public enum ErrorCode {
    OK, MISSING_STRING, MISSING_INTEGER, INVALID_INTEGER,
    UNEXPECTED_ARGUMENT,
    MISSING_DOUBLE, INVALID_DOUBLE}
}
---
public class Args {
  ...
  private char errorArgumentId = '\0';
  private String errorParameter = "TILT";
  private ArgsException.ErrorCode errorCode = ArgsException.ErrorCode.OK;
  private List<String> argsList;

  public Args(String schema, String[] args) throws ArgsException {
    this.schema = schema;
    argsList = Arrays.asList(args);
    valid = parse();
```

```
  }

  private boolean parse() throws ArgsException {
    if (schema.length() == 0 && argsList.size() == 0)
      return true;
    parseSchema();
    try {
      parseArguments();
    } catch (ArgsException e) {
    }
    return valid;
  }

  private boolean parseSchema() throws ArgsException {
    ...
  }

  private void parseSchemaElement(String element) throws ArgsException {
    ...
    else
      throw new ArgsException(
         String.format("Argument: %c has invalid format: %s.",
                       elementId,elementTail));
  }
  private void validateSchemaElementId(char elementId) throws ArgsException {
    if (!Character.isLetter(elementId)) {
      throw new ArgsException(
        "Bad character:" + elementId + "in Args format: " + schema);
    }
  }
  ...

  private void parseElement(char argChar) throws ArgsException {
    if (setArgument(argChar))
      argsFound.add(argChar);
    else {
      unexpectedArguments.add(argChar);
      errorCode = ArgsException.ErrorCode.UNEXPECTED_ARGUMENT;
      valid = false;
    }
  }
  ...
```

```
private class StringArgumentMarshaler implements ArgumentMarshaler {
  private String stringValue = "";

  public void set(Iterator<String> currentArgument) throws ArgsException {
    try {
      stringValue = currentArgument.next();
    } catch (NoSuchElementException e) {
      errorCode = ArgsException.ErrorCode.MISSING_STRING;
      throw new ArgsException();
    }
  }

  public Object get() {
    return stringValue;
```

```
    }
  }

  private class IntegerArgumentMarshaler implements ArgumentMarshaler {
    private int intValue = 0;

    public void set(Iterator<String> currentArgument) throws ArgsException {
      String parameter = null;
      try {
        parameter = currentArgument.next();
        intValue = Integer.parseInt(parameter);
      } catch (NoSuchElementException e) {
        errorCode = ArgsException.ErrorCode.MISSING_INTEGER;
        throw new ArgsException();
      } catch (NumberFormatException e) {
        errorParameter = parameter;
        errorCode = ArgsException.ErrorCode.INVALID_INTEGER;
        throw new ArgsException();
      }
    }

    public Object get() {
      return intValue;
    }
  }

  private class DoubleArgumentMarshaler implements ArgumentMarshaler {
    private double doubleValue = 0;
    public void set(Iterator<String> currentArgument) throws ArgsException {
      String parameter = null;
      try {
        parameter = currentArgument.next();
        doubleValue = Double.parseDouble(parameter);
      } catch (NoSuchElementException e) {
        errorCode = ArgsException.ErrorCode.MISSING_DOUBLE;
        throw new ArgsException();
      } catch (NumberFormatException e) {
        errorParameter = parameter;
        errorCode = ArgsException.ErrorCode.INVALID_DOUBLE;
        throw new ArgsException();
      }
    }

    public Object get() {
      return doubleValue;
    }
  }
```

Thế là tốt. Ngoại lệ duy nhất `Args` ném ra giờ là `ArgsException`. Chuyển `ArgsException` sang module riêng cũng giúp ta chuyển nhiều mã hỗ trợ xử lý lỗi ra khỏi module `Args` và đưa vào module đó. Đây là nơi tự nhiên, hợp lý để đặt chúng, nhờ vậy việc dọn module `Args` sau này sẽ dễ hơn nhiều.

Như vậy, ngoại lệ và mã lỗi đã được tách hoàn toàn khỏi module `Args`. (Xem Đoạn mã 14-13 đến Đoạn mã 14-16.) Việc này được thực hiện qua khoảng 30 bước nhỏ, giữ cho các kiểm thử qua sau từng bước.

Đoạn mã 14-13
ArgsTest.java

```
package com.objectmentor.utilities.args;

import junit.framework.TestCase;
public class ArgsTest extends TestCase {
  public void testCreateWithNoSchemaOrArguments() throws Exception {
    Args args = new Args("", new String[0]);
    assertEquals(0, args.cardinality());
  }

    public void testWithNoSchemaButWithOneArgument() throws Exception {
      try {
        new Args("", new String[]{"-x"});
        fail();
      } catch (ArgsException e) {
        assertEquals(ArgsException.ErrorCode.UNEXPECTED_ARGUMENT,
                     e.getErrorCode());
        assertEquals('x', e.getErrorArgumentId());
      }
    }
    public void testWithNoSchemaButWithMultipleArguments() throws Exception {
      try {
        new Args("", new String[]{"-x", "-y"});
        fail();
      } catch (ArgsException e) {
        assertEquals(ArgsException.ErrorCode.UNEXPECTED_ARGUMENT,
                     e.getErrorCode());
        assertEquals('x', e.getErrorArgumentId());
      }

    }
    public void testNonLetterSchema() throws Exception {
      try {
        new Args("*", new String[]{});
        fail("Args constructor should have thrown exception");
      } catch (ArgsException e) {
        assertEquals(ArgsException.ErrorCode.INVALID_ARGUMENT_NAME,
                     e.getErrorCode());
        assertEquals('*', e.getErrorArgumentId());
      }
    }
    public void testInvalidArgumentFormat() throws Exception {
      try {
        new Args("f~", new String[]{});
        fail("Args constructor should have throws exception");
      } catch (ArgsException e) {
        assertEquals(ArgsException.ErrorCode.INVALID_FORMAT,
    e.getErrorCode());
        assertEquals('f', e.getErrorArgumentId());
      }
    }

    public void testSimpleBooleanPresent() throws Exception {
      Args args = new Args("x", new String[]{"-x"});
      assertEquals(1, args.cardinality());
```

```
    assertEquals(true, args.getBoolean('x'));
  }
  public void testSimpleStringPresent() throws Exception {
    Args args = new Args("x*", new String[]{"-x", "param"});
    assertEquals(1, args.cardinality());
    assertTrue(args.has('x'));
    assertEquals("param", args.getString('x'));
  }
  public void testMissingStringArgument() throws Exception {
    try {
      new Args("x*", new String[]{"-x"});
      fail();
    } catch (ArgsException e) {
      assertEquals(ArgsException.ErrorCode.MISSING_STRING,
e.getErrorCode());
      assertEquals('x', e.getErrorArgumentId());
    }
  }

  public void testSpacesInFormat() throws Exception {
    Args args = new Args("x, y", new String[]{"-xy"});
    assertEquals(2, args.cardinality());
    assertTrue(args.has('x'));
    assertTrue(args.has('y'));
  }
  public void testSimpleIntPresent() throws Exception {
    Args args = new Args("x#", new String[]{"-x", "42"});
    assertEquals(1, args.cardinality());
    assertTrue(args.has('x'));
    assertEquals(42, args.getInt('x'));
  }

  public void testInvalidInteger() throws Exception {
    try {
      new Args("x#", new String[]{"-x", "Forty two"});
```

Đoạn mã 14-13 (tiếp)
ArgsTest.java

```
      fail();
    } catch (ArgsException e) {
      assertEquals(ArgsException.ErrorCode.INVALID_INTEGER,
e.getErrorCode());
      assertEquals('x', e.getErrorArgumentId());
      assertEquals("Forty two", e.getErrorParameter());
    }

  }

  public void testMissingInteger() throws Exception {
    try {
      new Args("x#", new String[]{"-x"});
      fail();
    } catch (ArgsException e) {
      assertEquals(ArgsException.ErrorCode.MISSING_INTEGER,
e.getErrorCode());
```

```
      assertEquals('x', e.getErrorArgumentId());
    }
  }

  public void testSimpleDoublePresent() throws Exception {
    Args args = new Args("x##", new String[]{"-x", "42.3"});
    assertEquals(1, args.cardinality());
    assertTrue(args.has('x'));
    assertEquals(42.3, args.getDouble('x'), .001);
  }

  public void testInvalidDouble() throws Exception {
    try {
      new Args("x##", new String[]{"-x", "Forty two"});
      fail();
    } catch (ArgsException e) {
      assertEquals(ArgsException.ErrorCode.INVALID_DOUBLE,
e.getErrorCode());
      assertEquals('x', e.getErrorArgumentId());
      assertEquals("Forty two", e.getErrorParameter());
    }
  }
  public void testMissingDouble() throws Exception {
    try {
      new Args("x##", new String[]{"-x"});
      fail();
    } catch (ArgsException e) {
      assertEquals(ArgsException.ErrorCode.MISSING_DOUBLE,
e.getErrorCode());
      assertEquals('x', e.getErrorArgumentId());
    }
  }
}
```

Đoạn mã 14-14
ArgsExceptionTest.java

```
public class ArgsExceptionTest extends TestCase {
  public void testUnexpectedMessage() throws Exception {
    ArgsException e =
```

Đoạn mã 14-14 (tiếp)
ArgsExceptionTest.java

```
      new ArgsException(ArgsException.ErrorCode.UNEXPECTED_ARGUMENT,
                        'x', null);
    assertEquals("Argument -x unexpected.", e.errorMessage());
  }

  public void testMissingStringMessage() throws Exception {
    ArgsException e = new
    ArgsException(ArgsException.ErrorCode.MISSING_STRING,
                                        'x', null);
```

```
    assertEquals("Could not find string parameter for -x.",
    e.errorMessage());
  }
  public void testInvalidIntegerMessage() throws Exception {
    ArgsException e =
      new ArgsException(ArgsException.ErrorCode.INVALID_INTEGER,
                        'x', "Forty two");
    assertEquals("Argument -x expects an integer but was 'Forty two'.",
                 e.errorMessage());
  }

  public void testMissingIntegerMessage() throws Exception {
    ArgsException e =
      new ArgsException(ArgsException.ErrorCode.MISSING_INTEGER, 'x', null);
    assertEquals("Could not find integer parameter for -x.",
    e.errorMessage());
  }
  public void testInvalidDoubleMessage() throws Exception {
    ArgsException e = new
    ArgsException(ArgsException.ErrorCode.INVALID_DOUBLE,
                                        'x', "Forty two");
    assertEquals("Argument -x expects a double but was 'Forty two'.",
                 e.errorMessage());
  }

  public void testMissingDoubleMessage() throws Exception {
    ArgsException e = new
    ArgsException(ArgsException.ErrorCode.MISSING_DOUBLE,
                                        'x', null);
    assertEquals("Could not find double parameter for -x.",
    e.errorMessage());
  }
}
```

Đoạn mã 14-15
ArgsException.java

```
public class ArgsException extends Exception {
  private char errorArgumentId = '\0';
  private String errorParameter = "TILT";
  private ErrorCode errorCode = ErrorCode.OK;
  public ArgsException() {}

  public ArgsException(String message) {super(message);}

  public ArgsException(ErrorCode errorCode) {
    this.errorCode = errorCode;
  }
```

Đoạn mã 14-15 (tiếp)
ArgsException.java

```
  public ArgsException(ErrorCode errorCode, String errorParameter) {
```

```
    this.errorCode = errorCode;
    this.errorParameter = errorParameter;
  }

  public ArgsException(ErrorCode errorCode, char errorArgumentId,
                       String errorParameter) {
    this.errorCode = errorCode;
    this.errorParameter = errorParameter;
    this.errorArgumentId = errorArgumentId;
  }

  public char getErrorArgumentId() {
    return errorArgumentId;
  }

  public void setErrorArgumentId(char errorArgumentId) {
    this.errorArgumentId = errorArgumentId;
  }

  public String getErrorParameter() {
    return errorParameter;
  }

  public void setErrorParameter(String errorParameter) {
    this.errorParameter = errorParameter;
  }

  public ErrorCode getErrorCode() {
    return errorCode;
  }

  public void setErrorCode(ErrorCode errorCode) {
    this.errorCode = errorCode;
  }
  public String errorMessage() throws Exception {
    switch (errorCode) {
      case OK:
        throw new Exception("TILT: Should not get here.");
      case UNEXPECTED_ARGUMENT:
        return String.format("Argument -%c unexpected.", errorArgumentId);
      case MISSING_STRING:
        return String.format("Could not find string parameter for -%c.",
                             errorArgumentId);
      case INVALID_INTEGER:
        return String.format("Argument -%c expects an integer but was '%s'.",
                             errorArgumentId, errorParameter);
      case MISSING_INTEGER:
        return String.format("Could not find integer parameter for -%c.",
                             errorArgumentId);
      case INVALID_DOUBLE:
        return String.format("Argument -%c expects a double but was '%s'.",
                             errorArgumentId, errorParameter);
      case MISSING_DOUBLE:
        return String.format("Could not find double parameter for -%c.",
                             errorArgumentId);
    }
    return "";
  }
```

```
  public enum ErrorCode {
    OK, INVALID_FORMAT, UNEXPECTED_ARGUMENT, INVALID_ARGUMENT_NAME,
    MISSING_STRING,
    MISSING_INTEGER, INVALID_INTEGER,
    MISSING_DOUBLE, INVALID_DOUBLE}
}
```

Đoạn mã 14-16
Args.java

```
public class Args {
  private String schema;
  private Map<Character, ArgumentMarshaler> marshalers =
    new HashMap<Character, ArgumentMarshaler>();
  private Set<Character> argsFound = new HashSet<Character>();
  private Iterator<String> currentArgument;
  private List<String> argsList;

  public Args(String schema, String[] args) throws ArgsException {
    this.schema = schema;
    argsList = Arrays.asList(args);
    parse();
  }

  private void parse() throws ArgsException {
    parseSchema();
    parseArguments();
  }
  private boolean parseSchema() throws ArgsException {
    for (String element : schema.split(",")) {
      if (element.length() > 0) {
        parseSchemaElement(element.trim());
      }
    }
    return true;
  }

  private void parseSchemaElement(String element) throws ArgsException {
    char elementId = element.charAt(0);
    String elementTail = element.substring(1);
    validateSchemaElementId(elementId);
    if (elementTail.length() == 0)
      marshalers.put(elementId, new BooleanArgumentMarshaler());
    else if (elementTail.equals("*"))
      marshalers.put(elementId, new StringArgumentMarshaler());
    else if (elementTail.equals("#"))
      marshalers.put(elementId, new IntegerArgumentMarshaler());
    else if (elementTail.equals("##"))
      marshalers.put(elementId, new DoubleArgumentMarshaler());
    else
      throw new ArgsException(ArgsException.ErrorCode.INVALID_FORMAT,
                              elementId, elementTail);
  }

  private void validateSchemaElementId(char elementId) throws ArgsException {
```

```
      if (!Character.isLetter(elementId)) {
        throw new ArgsException(ArgsException.ErrorCode.INVALID_ARGUMENT_NAME,
                                elementId, null);
      }
    }

    private void parseArguments() throws ArgsException {
      for (currentArgument = argsList.iterator(); currentArgument.hasNext();) {
        String arg = currentArgument.next();
        parseArgument(arg);
      }
    }

    private void parseArgument(String arg) throws ArgsException {
      if (arg.startsWith("-"))
        parseElements(arg);
    }

    private void parseElements(String arg) throws ArgsException {
      for (int i = 1; i < arg.length(); i++)
        parseElement(arg.charAt(i));
    }

    private void parseElement(char argChar) throws ArgsException {
      if (setArgument(argChar))
        argsFound.add(argChar);
      else {
        throw new ArgsException(ArgsException.ErrorCode.UNEXPECTED_ARGUMENT,
                                argChar, null);
      }
    }
    private boolean setArgument(char argChar) throws ArgsException {
      ArgumentMarshaler m = marshalers.get(argChar);
      if (m == null)
        return false;
      try {
        m.set(currentArgument);
        return true;
      } catch (ArgsException e) {
        e.setErrorArgumentId(argChar);
        throw e;
      }
    }
```

```
  public int cardinality() {
    return argsFound.size();
  }

  public String usage() {
    if (schema.length() > 0)
      return "-[" + schema + "]";
    else
      return "";
  }
  public boolean getBoolean(char arg) {
    ArgumentMarshaler am = marshalers.get(arg);
    boolean b = false;
```

```
    try {
      b = am != null && (Boolean) am.get();
    } catch (ClassCastException e) {
      b = false;
    }
    return b;
  }
  public String getString(char arg) {
    ArgumentMarshaler am = marshalers.get(arg);
    try {
      return am == null ? "" : (String) am.get();
    } catch (ClassCastException e) {
      return "";
    }
  }

  public int getInt(char arg) {
    ArgumentMarshaler am = marshalers.get(arg);
    try {
      return am == null ? 0 : (Integer) am.get();
    } catch (Exception e) {
      return 0;
    }
  }

  public double getDouble(char arg) {
    ArgumentMarshaler am = marshalers.get(arg);
    try {
      return am == null ? 0 : (Double) am.get();
    } catch (Exception e) {
      return 0.0;
    }
  }
  public boolean has(char arg) {
    return argsFound.contains(arg);
  }
```

Phần lớn thay đổi trong lớp `Args` là xóa mã. Nhiều đoạn mã chỉ được chuyển từ `Args` sang `ArgsException`. Tốt. Chúng tôi cũng chuyển toàn bộ `ArgumentMarshaler` sang các tệp riêng. Còn tốt hơn.

Phần lớn thiết kế phần mềm tốt chỉ là phân chia hợp lý, tạo chỗ thích hợp cho từng loại mã. Tách biệt các mối quan tâm giúp mã dễ hiểu và dễ bảo trì hơn.

Điều đáng chú ý là phương thức `errorMessage` của `ArgsException`. Rõ ràng, đưa phần định dạng thông báo lỗi vào `Args` là vi phạm SRP. `Args` nên tập trung xử lý đối số, không phải định dạng thông báo lỗi. Nhưng liệu đưa phần định dạng thông báo lỗi vào `ArgsException` có thật sự hợp lý?

Thành thật mà nói, đây là một sự thỏa hiệp. Người dùng không thích các thông báo lỗi do `ArgsException` cung cấp sẽ phải tự viết thông báo riêng. Nhưng việc có sẵn các thông báo lỗi tiện dùng cũng có giá trị đáng kể.

Đến đây hẳn đã rõ rằng ta chỉ còn cách giải pháp cuối cùng ở đầu chương một khoảng ngắn. Tôi để phần biến đổi cuối cùng lại làm bài tập cho bạn.

Kết luận

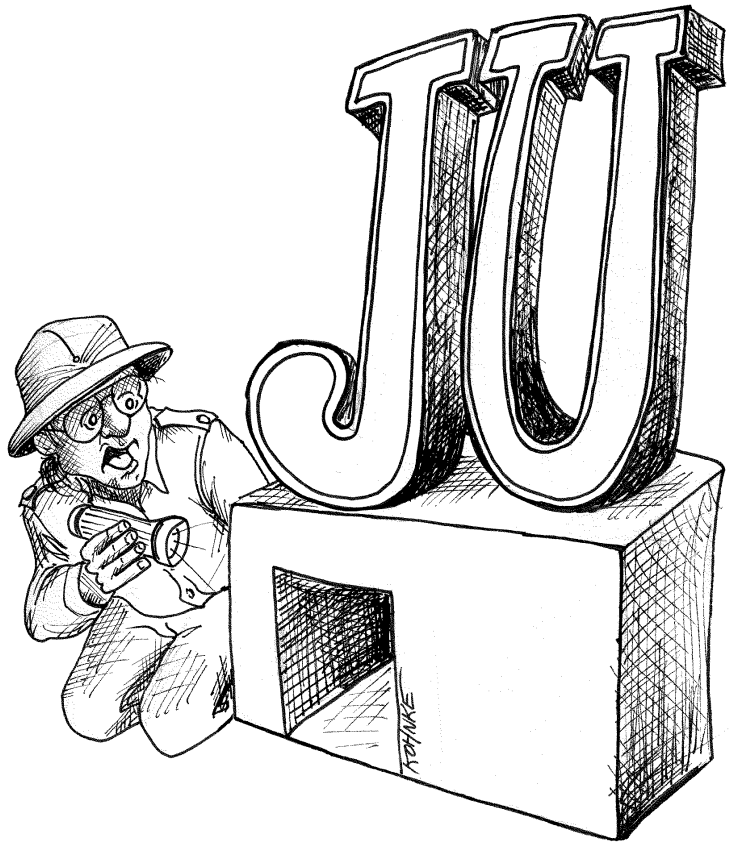
Mã chạy được thôi thì chưa đủ. Mã chạy được thường vẫn hỏng nặng. Lập trình viên chỉ hài lòng với việc mã chạy được đang hành xử thiếu chuyên nghiệp. Họ có thể lo rằng mình không đủ thời gian cải thiện cấu trúc và thiết kế mã, nhưng tôi không đồng ý. Không gì gây tác động xấu lâu dài đến một dự án phát triển bằng mã tồi. Tiến độ tồi có thể sửa lại, yêu cầu tồi có thể xác định lại, động lực nhóm tồi có thể khôi phục. Nhưng mã tồi sẽ mục ruỗng và lên men, trở thành gánh nặng không thể tránh khỏi kéo cả nhóm xuống. Tôi đã nhiều lần chứng kiến các nhóm gần như đình trệ vì trong lúc vội vã, họ tạo ra một mớ mã độc hại rồi mãi mãi bị nó chi phối.

Dĩ nhiên, có thể dọn mã tồi, nhưng cái giá rất đắt. Khi mã mục ruỗng, các module len lỏi vào nhau, tạo ra vô số phụ thuộc ngầm và rối rắm. Tìm rồi phá vỡ những phụ thuộc cũ là việc dài hơi, gian nan. Trong khi đó, giữ mã sạch lại khá dễ. Nếu sáng bạn làm một module rối tung, chiều vẫn có thể dọn nó. Tốt hơn nữa, nếu bạn vừa làm nó rối cách đây năm phút, hãy dọn ngay bây giờ, việc đó rất dễ.

Vậy nên, hãy liên tục giữ cho mã sạch và đơn giản hết mức có thể. Đừng bao giờ để nó bắt đầu mục ruỗng.

15

Bên trong JUnit



JUnit là một trong những framework Java nổi tiếng nhất. Xét theo các framework, nó đơn giản trong ý tưởng, chính xác trong định nghĩa và tao nhã trong cách triển khai. Nhưng

mã nguồn trông như thế nào? Trong chương này, chúng ta sẽ phê bình một ví dụ lấy từ framework JUnit.

Framework JUnit

JUnit có nhiều tác giả, nhưng nó bắt đầu từ Kent Beck và Eric Gamma trên một chuyến bay đến Atlanta. Kent muốn học Java, còn Eric muốn tìm hiểu framework kiểm thử Smalltalk của Kent. "Còn gì tự nhiên hơn với hai kẻ mê máy tính trong không gian chật hẹp là lấy máy tính xách tay ra và bắt đầu lập trình?"[1] Sau ba giờ làm việc ở độ cao lớn, họ đã viết xong những phần cơ bản của JUnit.

Mô-đun chúng ta sẽ xem xét là đoạn mã thông minh giúp xác định lỗi khi so sánh chuỗi. Mô-đun này có tên ComparisonCompactor. Khi được cho hai chuỗi khác nhau, chẳng hạn ABCDE và ABXDE, nó sẽ làm lộ ra điểm khác biệt bằng cách tạo chuỗi như <…B[X]D…>.

Tôi có thể giải thích thêm, nhưng các ca kiểm thử làm việc đó tốt hơn. Vậy hãy xem Đoạn mã 15-1, bạn sẽ hiểu sâu các yêu cầu của mô-đun này. Nhân tiện, hãy phê bình cấu trúc của các bài kiểm thử. Chúng có thể đơn giản hơn hoặc dễ hiểu hơn không?

Đoạn mã 15-1
ComparisonCompactorTest.java

```
package junit.tests.framework;

import junit.framework.ComparisonCompactor;
import junit.framework.TestCase;
public class ComparisonCompactorTest extends TestCase {

   public void testMessage() {
     String failure= new ComparisonCompactor(0, "b", "c").compact("a");
     assertTrue("a expected:<[b]> but was:<[c]>".equals(failure));
   }
   public void testStartSame() {
     String failure= new ComparisonCompactor(1, "ba", "bc").compact(null);
     assertEquals("expected:<b[a]> but was:<b[c]>", failure);
   }

   public void testEndSame() {
     String failure= new ComparisonCompactor(1, "ab", "cb").compact(null);
     assertEquals("expected:<[a]b> but was:<[c]b>", failure);
   }

   public void testSame() {
     String failure= new ComparisonCompactor(1, "ab", "ab").compact(null);
     assertEquals("expected:<ab> but was:<ab>", failure);
   }

   public void testNoContextStartAndEndSame() {
     String failure= new ComparisonCompactor(0, "abc", "adc").compact(null);
     assertEquals("expected:<...[b]...> but was:<...[d]...>", failure);
```

1. *JUnit Pocket Guide*, Kent Beck, O'Reilly, 2004, tr. 43.

```
    }

    public void testStartAndEndContext() {
      String failure= new ComparisonCompactor(1, "abc", "adc").compact(null);
      assertEquals("expected:<a[b]c> but was:<a[d]c>", failure);
    }

    public void testStartAndEndContextWithEllipses() {
      String failure=
         new ComparisonCompactor(1, "abcde", "abfde").compact(null);
      assertEquals("expected:<...b[c]d...> but was:<...b[f]d...>", failure);
    }
    public void testComparisonErrorStartSameComplete() {
      String failure= new ComparisonCompactor(2, "ab", "abc").compact(null);
      assertEquals("expected:<ab[]> but was:<ab[c]>", failure);
    }

    public void testComparisonErrorEndSameComplete() {
      String failure= new ComparisonCompactor(0, "bc", "abc").compact(null);
      assertEquals("expected:<[]...> but was:<[a]...>", failure);
    }

    public void testComparisonErrorEndSameCompleteContext() {
      String failure= new ComparisonCompactor(2, "bc", "abc").compact(null);
      assertEquals("expected:<[]bc> but was:<[a]bc>", failure);
    }

    public void testComparisonErrorOverlapingMatches() {
      String failure= new ComparisonCompactor(0, "abc", "abbc").compact(null);
      assertEquals("expected:<...[]...> but was:<...[b]...>", failure);
    }

    public void testComparisonErrorOverlapingMatchesContext() {
      String failure= new ComparisonCompactor(2, "abc", "abbc").compact(null);
      assertEquals("expected:<ab[]c> but was:<ab[b]c>", failure);
    }
    public void testComparisonErrorOverlapingMatches2() {
       String failure= new ComparisonCompactor(0, "abcdde",
  "abcde").compact(null);
       assertEquals("expected:<...[d]...> but was:<...[]...>", failure);
    }
    public void testComparisonErrorOverlapingMatches2Context() {
      String failure=
         new ComparisonCompactor(2, "abcdde", "abcde").compact(null);
      assertEquals("expected:<...cd[d]e> but was:<...cd[]e>", failure);
    }

    public void testComparisonErrorWithActualNull() {
      String failure= new ComparisonCompactor(0, "a", null).compact(null);
      assertEquals("expected:<a> but was:<null>", failure);
    }

    public void testComparisonErrorWithActualNullContext() {
      String failure= new ComparisonCompactor(2, "a", null).compact(null);
      assertEquals("expected:<a> but was:<null>", failure);
    }

    public void testComparisonErrorWithExpectedNull() {
```

```
      String failure= new ComparisonCompactor(0, null, "a").compact(null);
      assertEquals("expected:<null> but was:<a>", failure);
    }

    public void testComparisonErrorWithExpectedNullContext() {
      String failure= new ComparisonCompactor(2, null, "a").compact(null);
      assertEquals("expected:<null> but was:<a>", failure);
    }

    public void testBug609972() {
      String failure= new ComparisonCompactor(10, "S&P500",
     "0").compact(null);
      assertEquals("expected:<[S&P50]0> but was:<[]0>", failure);
    }
  }
```

Tôi đã chạy phân tích độ bao phủ mã trên ComparisonCompactor bằng các bài kiểm thử này. Mã được bao phủ 100 phần trăm. Mọi dòng mã, mọi câu lệnh if và vòng lặp for đều được các bài kiểm thử thực thi. Điều này khiến tôi rất tin tưởng rằng mã hoạt động đúng và rất kính trọng tay nghề của các tác giả.

Mã của ComparisonCompactor nằm trong Đoạn mã 15-2. Hãy dành chút thời gian xem qua mã này. Tôi nghĩ bạn sẽ thấy nó được phân chia khá hợp lý, diễn đạt tương đối rõ ràng và có cấu trúc đơn giản. Khi xem xong, chúng ta sẽ cùng bắt bẻ vài điểm nhỏ.

Đoạn mã 15-2
ComparisonCompactor.java (Bản gốc)

```
package junit.framework;
public class ComparisonCompactor {

    private static final String ELLIPSIS = "...";
    private static final String DELTA_END = "]";
    private static final String DELTA_START = "[";

    private int fContextLength;
    private String fExpected;
    private String fActual;
    private int fPrefix;
    private int fSuffix;
    public ComparisonCompactor(int contextLength,
                               String expected,
                                 String actual) {
      fContextLength = contextLength;
      fExpected = expected;
      fActual = actual;
    }
    public String compact(String message) {
      if (fExpected == null || fActual == null || areStringsEqual())
        return Assert.format(message, fExpected, fActual);

        findCommonPrefix();
        findCommonSuffix();
        String expected = compactString(fExpected);
        String actual = compactString(fActual);
```

```
        return Assert.format(message, expected, actual);
    }
    private String compactString(String source) {
      String result = DELTA_START +
                      source.substring(fPrefix, source.length() -
                        fSuffix + 1) + DELTA_END;
      if (fPrefix > 0)
        result = computeCommonPrefix() + result;
      if (fSuffix > 0)
        result = result + computeCommonSuffix();
      return result;
    }
    private void findCommonPrefix() {
      fPrefix = 0;
      int end = Math.min(fExpected.length(), fActual.length());
      for (; fPrefix < end; fPrefix++) {
        if (fExpected.charAt(fPrefix) != fActual.charAt(fPrefix))
          break;
      }
    }

    private void findCommonSuffix() {
      int expectedSuffix = fExpected.length() - 1;
      int actualSuffix = fActual.length() - 1;
      for (;
           actualSuffix >= fPrefix && expectedSuffix >= fPrefix;
            actualSuffix--, expectedSuffix--) {
        if (fExpected.charAt(expectedSuffix) != fActual.charAt(actualSuffix))
          break;
      }
      fSuffix = fExpected.length() - expectedSuffix;
    }

    private String computeCommonPrefix() {
      return (fPrefix > fContextLength ? ELLIPSIS : "") +
               fExpected.substring(Math.max(0, fPrefix - fContextLength),
                                   fPrefix);
    }

    private String computeCommonSuffix() {
      int end = Math.min(fExpected.length() - fSuffix + 1 + fContextLength,
                         fExpected.length());
      return fExpected.substring(fExpected.length() - fSuffix + 1, end) +
             (fExpected.length() - fSuffix + 1 < fExpected.length() -
              fContextLength ? ELLIPSIS : "");
    }
    private boolean areStringsEqual() {
      return fExpected.equals(fActual);
    }
  }
```

Có thể bạn có vài điều phàn nàn về mô-đun này. Có một số biểu thức dài và vài chỗ +1 kỳ lạ, vân vân. Nhưng nhìn chung mô-đun này khá tốt. Dù sao thì nó cũng có thể trông như Đoạn mã 15-3.

Đoạn mã 15-3
ComparisonCompator.java (đã làm xấu mã)

```
package junit.framework;

public class ComparisonCompactor {
  private int ctxt;
  private String s1;
  private String s2;
  private int pfx;
  private int sfx;
    public ComparisonCompactor(int ctxt, String s1, String s2) {
      this.ctxt = ctxt;
      this.s1 = s1;
      this.s2 = s2;
    }

    public String compact(String msg) {
      if (s1 == null || s2 == null || s1.equals(s2))
        return Assert.format(msg, s1, s2);
        pfx = 0;
        for (; pfx < Math.min(s1.length(), s2.length()); pfx++) {
          if (s1.charAt(pfx) != s2.charAt(pfx))
            break;
        }
        int sfx1 = s1.length() - 1;
        int sfx2 = s2.length() - 1;
        for (; sfx2 >= pfx && sfx1 >= pfx; sfx2--, sfx1--) {
          if (s1.charAt(sfx1) != s2.charAt(sfx2))
            break;
        }
        sfx = s1.length() - sfx1;
        String cmp1 = compactString(s1);
        String cmp2 = compactString(s2);
        return Assert.format(msg, cmp1, cmp2);
    }

    private String compactString(String s) {
      String result =
        "[" + s.substring(pfx, s.length() - sfx + 1) + "]";
      if (pfx > 0)
        result = (pfx > ctxt ? "..." : "") +
          s1.substring(Math.max(0, pfx - ctxt), pfx) + result;
      if (sfx > 0) {
        int end = Math.min(s1.length() - sfx + 1 + ctxt, s1.length());
        result = result + (s1.substring(s1.length() - sfx + 1, end) +
          (s1.length() - sfx + 1 < s1.length() - ctxt ? "..." : ""));
      }
      return result;
    }

}
```

Dù các tác giả đã để mô-đun này ở trạng thái rất tốt, Quy tắc Hướng đạo sinh[2] nhắc chúng ta rằng nên để lại mọi thứ sạch hơn lúc tìm thấy. Vậy làm sao chúng ta có thể cải thiện mã ban đầu trong Đoạn mã 15-2?

Điều đầu tiên tôi không thích là tiền tố f ở các biến thành viên [N6]. Các môi trường ngày nay khiến kiểu mã hóa phạm vi này trở nên dư thừa. Vậy hãy loại bỏ mọi chữ f.

```
private int contextLength;
private String expected;
private String actual;
private int prefix;
private int suffix;
```

Tiếp theo, ở đầu hàm compact có một điều kiện chưa được đóng gói [G28].

```
public String compact(String message) {
  if (expected == null || actual == null || areStringsEqual())
    return Assert.format(message, expected, actual);
    findCommonPrefix();
    findCommonSuffix();
    String expected = compactString(this.expected);
    String actual = compactString(this.actual);
    return Assert.format(message, expected, actual);
}
```

Điều kiện này nên được đóng gói để làm rõ ý định của chúng ta. Vậy hãy trích xuất một phương thức giải thích điều kiện đó.

```
public String compact(String message) {
  if (shouldNotCompact())
    return Assert.format(message, expected, actual);

    findCommonPrefix();
    findCommonSuffix();
    String expected = compactString(this.expected);
    String actual = compactString(this.actual);
    return Assert.format(message, expected, actual);
}

private boolean shouldNotCompact() {
  return expected == null || actual == null || areStringsEqual();
}
```

Tôi không thích cách viết this.expected và this.actual trong hàm compact. Điều này xảy ra khi chúng ta đổi tên fExpected thành expected. Tại sao hàm này lại có các biến trùng tên với biến thành viên? Chúng không biểu thị thứ gì khác sao [N4]? Chúng ta nên đặt tên sao cho không nhập nhằng.

```
String compactExpected = compactString(expected);
String compactActual = compactString(actual);
```

2. Xem “Quy tắc Hướng đạo sinh” ở trang 14.

Các ý phủ định hơi khó hiểu hơn ý khẳng định [G29]. Vậy hãy đảo ngược câu lệnh if và đảo nghĩa điều kiện.

```
public String compact(String message) {
  if (canBeCompacted()) {
    findCommonPrefix();
    findCommonSuffix();
    String compactExpected = compactString(expected);
    String compactActual = compactString(actual);
    return Assert.format(message, compactExpected, compactActual);
  } else {
    return Assert.format(message, expected, actual);
  }
}
private boolean canBeCompacted() {
  return expected != null && actual != null && !areStringsEqual();
}
```

Tên hàm thật kỳ lạ [N7]. Dù nó có thực hiện việc rút gọn các chuỗi, trên thực tế hàm có thể không rút gọn chúng nếu canBeCompacted trả về false. Vì vậy, tên hàm compact che giấu tác dụng phụ của việc kiểm tra lỗi. Cũng cần lưu ý rằng hàm trả về một thông điệp đã định dạng, chứ không chỉ các chuỗi đã rút gọn. Vậy tên hàm thực sự nên là formatCompactedComparison. Như vậy khi đọc cùng đối số của hàm, câu lệnh sẽ dễ hiểu hơn nhiều:

```
public String formatCompactedComparison(String message) {
```

Phần thân câu lệnh if là nơi thực sự rút gọn các chuỗi expected và actual. Chúng ta nên trích xuất phần này thành một phương thức tên là compactExpectedAndActual. Tuy nhiên, ta muốn hàm formatCompactedComparison thực hiện toàn bộ việc định dạng. Hàm compact… chỉ nên làm việc rút gọn [G30]. Vậy hãy tách như sau:

```
...
private String compactExpected;
private String compactActual;

...

public String formatCompactedComparison(String message) {
  if (canBeCompacted()) {
    compactExpectedAndActual();
    return Assert.format(message, compactExpected, compactActual);
  } else {
    return Assert.format(message, expected, actual);
  }
}

private void compactExpectedAndActual() {
  findCommonPrefix();
  findCommonSuffix();
  compactExpected = compactString(expected);
  compactActual = compactString(actual);
```

```
  }
```

Hãy chú ý rằng việc này buộc chúng ta phải nâng compactExpected và compactActual thành các biến thành viên. Tôi không thích việc hai dòng cuối của hàm mới trả về các biến, còn hai dòng đầu thì không. Chúng không tuân theo quy ước nhất quán [G11]. Vậy chúng ta nên đổi findCommonPrefix và findCommonSuffix để trả về các giá trị tiền tố và hậu tố.

```
  private void compactExpectedAndActual() {
    prefixIndex = findCommonPrefix();
    suffixIndex = findCommonSuffix();
    compactExpected = compactString(expected);
    compactActual = compactString(actual);
  }

  private int findCommonPrefix() {
    int prefixIndex = 0;
    int end = Math.min(expected.length(), actual.length());
    for (; prefixIndex < end; prefixIndex++) {
      if (expected.charAt(prefixIndex) != actual.charAt(prefixIndex))
        break;
    }
    return prefixIndex;
  }

  private int findCommonSuffix() {
    int expectedSuffix = expected.length() - 1;
    int actualSuffix = actual.length() - 1;
    for (; actualSuffix >= prefixIndex && expectedSuffix >= prefixIndex;
         actualSuffix--, expectedSuffix--) {
      if (expected.charAt(expectedSuffix) != actual.charAt(actualSuffix))
        break;
    }
    return expected.length() - expectedSuffix;
  }
```

Chúng ta cũng nên đổi tên các biến thành viên cho chính xác hơn một chút [N1]; xét cho cùng, cả hai đều là chỉ số.

Quan sát kỹ findCommonSuffix cho thấy một sự phụ thuộc thời điểm bị ẩn [G31]; nó phụ thuộc vào việc prefixIndex được tính bởi findCommonPrefix. Nếu gọi hai hàm này sai thứ tự, việc gỡ lỗi sẽ rất khó khăn. Vậy để làm lộ sự phụ thuộc thời điểm này, hãy cho findCommonSuffix nhận prefixIndex làm đối số.

```
  private void compactExpectedAndActual() {
    prefixIndex = findCommonPrefix();
    suffixIndex = findCommonSuffix(prefixIndex);
    compactExpected = compactString(expected);
    compactActual = compactString(actual);
  }

  private int findCommonSuffix(int prefixIndex) {
    int expectedSuffix = expected.length() - 1;
    int actualSuffix = actual.length() - 1;
    for (; actualSuffix >= prefixIndex && expectedSuffix >= prefixIndex;
```

```
        actualSuffix--, expectedSuffix--) {
      if (expected.charAt(expectedSuffix) != actual.charAt(actualSuffix))
        break;
    }
    return expected.length() - expectedSuffix;
  }
```

Tôi không thực sự hài lòng với cách này. Việc truyền đối số prefixIndex hơi tùy tiện [G32]. Nó giúp xác lập thứ tự nhưng chẳng giải thích gì về sự cần thiết của thứ tự đó. Một lập trình viên khác có thể đảo ngược thay đổi của chúng ta vì không có dấu hiệu nào cho thấy tham số này thực sự cần thiết. Vậy hãy thử một hướng khác.

```
  private void compactExpectedAndActual() {
    findCommonPrefixAndSuffix();
    compactExpected = compactString(expected);
    compactActual = compactString(actual);
  }

  private void findCommonPrefixAndSuffix() {
    findCommonPrefix();
    int expectedSuffix = expected.length() - 1;
    int actualSuffix = actual.length() - 1;
    for (;
         actualSuffix >= prefixIndex && expectedSuffix >= prefixIndex;
         actualSuffix--, expectedSuffix--
      ) {
      if (expected.charAt(expectedSuffix) != actual.charAt(actualSuffix))
        break;
    }
    suffixIndex = expected.length() - expectedSuffix;
  }

  private void findCommonPrefix() {
    prefixIndex = 0;
    int end = Math.min(expected.length(), actual.length());
    for (; prefixIndex < end; prefixIndex++)
      if (expected.charAt(prefixIndex) != actual.charAt(prefixIndex))
        break;
  }
```

Chúng ta đã đưa findCommonPrefix và findCommonSuffix trở lại như trước, đổi tên findCommonSuffix thành findCommonPrefixAndSuffix và cho nó gọi findCommonPrefix trước khi làm bất cứ việc gì khác. Cách này thể hiện rõ tính phụ thuộc thời điểm của hai hàm hơn hẳn giải pháp trước. Nó cũng chỉ ra findCommonPrefixAndSuffix xấu xí đến mức nào. Giờ hãy làm sạch nó.

```
  private void findCommonPrefixAndSuffix() {
    findCommonPrefix();
    int suffixLength = 1;
    for (; !suffixOverlapsPrefix(suffixLength); suffixLength++) {
      if (charFromEnd(expected, suffixLength) !=
          charFromEnd(actual, suffixLength))
        break;
```

```
  }
  suffixIndex = suffixLength;
}
private char charFromEnd(String s, int i) {
  return s.charAt(s.length()-i);}
private boolean suffixOverlapsPrefix(int suffixLength) {
  return actual.length() - suffixLength < prefixLength ||
    expected.length() - suffixLength < prefixLength;
}
```

Cách này tốt hơn nhiều. Nó cho thấy suffixIndex thực ra là độ dài của hậu tố và tên hiện tại không phù hợp. Điều tương tự cũng đúng với prefixIndex, dù trong trường hợp đó "chỉ số" và "độ dài" đồng nghĩa. Dù vậy, dùng "độ dài" sẽ nhất quán hơn. Vấn đề là biến suffixIndex không bắt đầu từ 0 mà từ 1, nên nó không phải độ dài thực sự. Đây cũng là lý do có nhiều chỗ +1 trong computeCommonSuffix [G33]. Vậy hãy sửa điều đó. Kết quả nằm trong Đoạn mã 15-4.

Đoạn mã 15-4
ComparisonCompactor.java (bản trung gian)

```
public class ComparisonCompactor {
...
  private int suffixLength;
...
  private void findCommonPrefixAndSuffix() {
    findCommonPrefix();
    suffixLength = 0;
    for (; !suffixOverlapsPrefix(suffixLength); suffixLength++) {
      if (charFromEnd(expected, suffixLength) !=
          charFromEnd(actual, suffixLength))
        break;
    }
  }
  private char charFromEnd(String s, int i) {
    return s.charAt(s.length() - i - 1);
  }

  private boolean suffixOverlapsPrefix(int suffixLength) {
    return actual.length() - suffixLength <= prefixLength ||
      expected.length() - suffixLength <= prefixLength;
  }

...
  private String compactString(String source) {
    String result =
      DELTA_START +
      source.substring(prefixLength, source.length() - suffixLength) +
      DELTA_END;
    if (prefixLength > 0)
      result = computeCommonPrefix() + result;
    if (suffixLength > 0)
      result = result + computeCommonSuffix();
    return result;
  }
```

```
...
  private String computeCommonSuffix() {
    int end = Math.min(expected.length() - suffixLength +
       contextLength, expected.length()
    );
    return
       expected.substring(expected.length() - suffixLength, end) +
       (expected.length() - suffixLength <
         expected.length() - contextLength ?
         ELLIPSIS : "");
  }
```

Chúng ta đã thay +1 trong computeCommonSuffix bằng -1 ở charFromEnd, nơi nó hoàn toàn hợp lý, và thay bằng hai toán tử <= trong suffixOverlapsPrefix, cũng rất hợp lý. Nhờ đó, chúng ta có thể đổi tên suffixIndex thành suffixLength, cải thiện đáng kể khả năng đọc của mã.

Tuy nhiên, có một vấn đề. Khi loại bỏ các chỗ +1, tôi nhận thấy dòng sau trong compactString:

```
if (suffixLength > 0)
```

Hãy xem dòng này trong Đoạn mã 15-4. Theo lý, vì giờ suffixLength nhỏ hơn trước một đơn vị, tôi nên đổi toán tử > thành >=. Nhưng điều đó chẳng có nghĩa gì. Bây giờ thì lại có nghĩa! Điều này cho thấy trước đây nó không có nghĩa và có lẽ là một lỗi.

À, không hẳn là lỗi. Phân tích sâu hơn, ta thấy câu lệnh if giờ ngăn không cho nối một hậu tố có độ dài bằng không. Trước khi thay đổi, câu lệnh if không có tác dụng vì suffixIndex không bao giờ nhỏ hơn một!

Điều này khiến chúng ta phải xem xét cả hai câu lệnh if trong compactString! Có vẻ như có thể loại bỏ cả hai. Vậy hãy chú thích chúng lại rồi chạy các bài kiểm thử. Chúng đều đạt! Vậy hãy cấu trúc lại compactString để loại bỏ các câu lệnh if thừa và làm hàm đơn giản hơn nhiều [G9].

```
private String compactString(String source) {
    return
      computeCommonPrefix() +
      DELTA_START +
      source.substring(prefixLength, source.length() - suffixLength) +
      DELTA_END +
      computeCommonSuffix();
  }
```

Cách này tốt hơn nhiều! Giờ ta thấy hàm compactString chỉ đơn thuần ghép các đoạn lại với nhau. Có lẽ chúng ta có thể làm cho nó rõ ràng hơn nữa. Thực vậy, còn nhiều chỗ nhỏ có thể dọn dẹp. Nhưng thay vì dẫn bạn qua toàn bộ những thay đổi còn lại, tôi sẽ chỉ cho bạn kết quả trong Đoạn mã 15-5.

Đoạn mã 15-5
ComparisonCompactor.java (bản cuối)

```
package junit.framework;

public class ComparisonCompactor {
  private static final String ELLIPSIS = "...";
  private static final String DELTA_END = "]";
  private static final String DELTA_START = "[";

  private int contextLength;
  private String expected;
  private String actual;
  private int prefixLength;
  private int suffixLength;

  public ComparisonCompactor(
    int contextLength, String expected, String actual
  ) {
    this.contextLength = contextLength;
    this.expected = expected;
    this.actual = actual;
  }
  public String formatCompactedComparison(String message) {
    String compactExpected = expected;
    String compactActual = actual;
    if (shouldBeCompacted()) {
      findCommonPrefixAndSuffix();
      compactExpected = compact(expected);
      compactActual = compact(actual);
    }
    return Assert.format(message, compactExpected, compactActual);
  }
  private boolean shouldBeCompacted() {
    return !shouldNotBeCompacted();
  }

  private boolean shouldNotBeCompacted() {
    return expected == null ||
           actual == null ||
           expected.equals(actual);
  }

  private void findCommonPrefixAndSuffix() {
    findCommonPrefix();
    suffixLength = 0;
    for (; !suffixOverlapsPrefix(); suffixLength++) {
      if (charFromEnd(expected, suffixLength) !=
          charFromEnd(actual, suffixLength)
      )
        break;
    }

    private char charFromEnd(String s, int i) {
      return s.charAt(s.length() - i - 1);
    }
```

```
        private boolean suffixOverlapsPrefix() {
          return actual.length() - suffixLength <= prefixLength ||
            expected.length() - suffixLength <= prefixLength;
        }

        private void findCommonPrefix() {
          prefixLength = 0;
          int end = Math.min(expected.length(), actual.length());
          for (; prefixLength < end; prefixLength++)
            if (expected.charAt(prefixLength) != actual.charAt(prefixLength))
              break;
        }

        private String compact(String s) {
          return new StringBuilder()
            .append(startingEllipsis())
            .append(startingContext())
            .append(DELTA_START)
            .append(delta(s))
            .append(DELTA_END)
            .append(endingContext())
            .append(endingEllipsis())
            .toString();
        }

        private String startingEllipsis() {
          return prefixLength > contextLength ? ELLIPSIS : "";
        }
        private String startingContext() {
          int contextStart = Math.max(0, prefixLength - contextLength);
          int contextEnd = prefixLength;
          return expected.substring(contextStart, contextEnd);
        }

        private String delta(String s) {
          int deltaStart = prefixLength;
          int deltaEnd = s.length() - suffixLength;
          return s.substring(deltaStart, deltaEnd);
        }

        private String endingContext() {
          int contextStart = expected.length() - suffixLength;
          int contextEnd =
            Math.min(contextStart + contextLength, expected.length());
          return expected.substring(contextStart, contextEnd);
        }

        private String endingEllipsis() {
          return (suffixLength > contextLength ? ELLIPSIS : "");
        }
    }
```

Thực ra, cách này khá đẹp. Mô-đun được chia thành một nhóm hàm phân tích và một nhóm hàm tổng hợp. Chúng được sắp xếp theo cấu trúc tô pô, sao cho định nghĩa của mỗi hàm xuất hiện ngay sau chỗ nó được dùng. Tất cả các hàm phân tích xuất hiện trước, còn các hàm tổng hợp xuất hiện sau cùng.

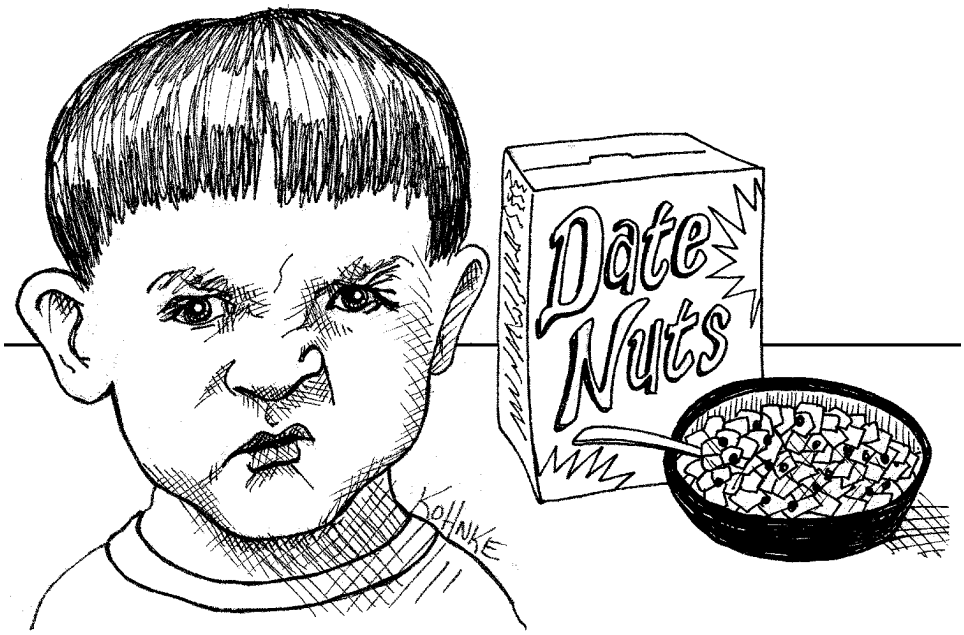
Nếu quan sát kỹ, bạn sẽ nhận thấy tôi đã đảo ngược vài quyết định đưa ra trước đó trong chương này. Chẳng hạn, tôi đã đưa một số phương thức được trích xuất trở lại vào formatCompactedComparison và thay đổi chiều hướng của biểu thức shouldNotBeCompacted. Đây là điều điển hình. Thường thì một lần tái cấu trúc dẫn đến một lần khác, rồi lần đó lại khiến ta hoàn tác lần đầu. Tái cấu trúc là một quá trình lặp lại, đầy thử nghiệm và sai sót, tất yếu hội tụ về một thứ mà chúng ta cho là xứng đáng với một chuyên gia.

Kết luận

Như vậy, chúng ta đã thực hiện Quy tắc Hướng đạo sinh. Chúng ta để lại mô-đun này sạch hơn đôi chút so với lúc tìm thấy. Không phải trước đó nó không sạch. Các tác giả đã làm rất tốt. Nhưng không mô-đun nào miễn nhiễm với việc cải thiện, và mỗi người chúng ta có trách nhiệm để lại mã tốt hơn một chút so với lúc mình tìm thấy.

16

Tái cấu trúc SerialDate



Nếu bạn truy cập `http://www.jfree.org/jcommon/index.php`, bạn sẽ tìm thấy thư viện JCommon. Sâu bên trong thư viện đó có một gói tên là org.jfree.date. Trong gói này có một lớp tên là SerialDate. Chúng ta sẽ cùng tìm hiểu lớp đó.

Tác giả của SerialDate là David Gilbert. David rõ ràng là một lập trình viên giàu kinh nghiệm và có năng lực. Như ta sẽ thấy, mã của ông thể hiện tính chuyên nghiệp và kỷ luật đáng kể. Xét trên mọi phương diện, đây là "mã tốt". Vậy mà tôi sắp mổ xẻ nó.

Đây không phải hành động ác ý. Tôi cũng không cho rằng mình giỏi hơn David đến mức có quyền phán xét mã của ông. Thật vậy, nếu bạn tìm thấy một đoạn mã nào đó của tôi, chắc chắn bạn cũng sẽ tìm ra khối điều đáng chê.

Không, đây không phải hành động xấu tính hay ngạo mạn. Điều tôi sắp làm chỉ đơn

thuần là một cuộc review chuyên nghiệp. Đây là việc tất cả chúng ta nên thấy thoải mái khi thực hiện, và cũng nên đón nhận khi người khác làm với mình. Chỉ qua những lời phê bình như thế này chúng ta mới học được. Bác sĩ làm vậy. Phi công làm vậy. Luật sư làm vậy. Và lập trình viên chúng ta cũng cần học cách làm như thế.

Còn một điều nữa về David Gilbert: David không chỉ là một lập trình viên giỏi. David có đủ can đảm và thiện chí để tặng mã của mình miễn phí cho cộng đồng rộng lớn. Ông công khai mã để mọi người cùng xem, mời mọi người sử dụng và đánh giá. Ông đã làm rất tốt!

SerialDate (Đoạn mã B-1, trang 349) là một lớp biểu diễn ngày tháng trong Java. Tại sao lại cần lớp biểu diễn ngày tháng khi Java đã có java.util.Date, java.util.Calendar và những lớp khác? Tác giả viết lớp này vì một vấn đề mà chính tôi cũng thường gặp. Chú thích trong Javadoc mở đầu của ông (dòng 67) giải thích rất rõ. Ta có thể tranh luận về ý định của ông, nhưng bản thân tôi chắc chắn từng phải xử lý vấn đề này và hoan nghênh một lớp chuyên về ngày tháng thay vì thời gian.

Trước hết, làm cho nó hoạt động

Có một số kiểm thử đơn vị trong lớp tên SerialDateTests (Đoạn mã B-2, trang 366). Tất cả đều chạy qua. Đáng tiếc, xem nhanh các kiểm thử cho thấy chúng không kiểm thử mọi thứ [T1]. Chẳng hạn, tìm kiếm "Find Usages" với phương thức MonthCodeToQuarter (dòng 334) cho thấy phương thức này không được dùng [F4]. Vì vậy, kiểm thử đơn vị không kiểm thử nó.

Tôi khởi chạy Clover để xem kiểm thử đơn vị bao phủ những gì và bỏ sót những gì. Clover báo rằng kiểm thử đơn vị chỉ thực thi 91 trong số 185 câu lệnh có thể thực thi trong SerialDate, khoảng 50 phần trăm [T2]. Bản đồ độ bao phủ trông như một tấm chăn chắp vá, với những mảng mã lớn chưa được thực thi rải khắp lớp.

Mục tiêu của tôi là hiểu hoàn toàn lớp này, đồng thời tái cấu trúc nó. Tôi không thể làm vậy nếu không tăng đáng kể độ bao phủ kiểm thử. Vì thế, tôi viết bộ kiểm thử đơn vị hoàn toàn độc lập của riêng mình (Đoạn mã B-4, trang 374).

Khi xem các kiểm thử này, bạn sẽ nhận thấy nhiều kiểm thử đã bị chú thích. Chúng không chạy qua. Chúng thể hiện hành vi mà tôi nghĩ SerialDate nên có. Vì vậy, khi tái cấu trúc SerialDate, tôi cũng sẽ tìm cách làm cho các kiểm thử này chạy qua.

Ngay cả khi một số kiểm thử vẫn bị chú thích, Clover báo rằng bộ kiểm thử đơn vị mới thực thi 170 trong số 185 câu lệnh có thể thực thi, đạt 92 phần trăm. Kết quả này khá tốt, và tôi nghĩ chúng ta còn có thể nâng con số đó lên nữa.

Một vài kiểm thử đầu tiên bị chú thích (dòng 23-63) phần nào là do tôi tự phụ. Chương trình không được thiết kế để chạy qua các kiểm thử này, nhưng với tôi, hành vi đó có vẻ hiển nhiên [G2]. Tôi không rõ tại sao phương thức testWeekdayCodeToString lại được viết ngay từ đầu, nhưng vì nó đã tồn tại nên rõ ràng nó không nên phân biệt chữ hoa chữ thường. Viết các kiểm thử này rất đơn giản [T3]. Làm cho chúng chạy qua còn dễ hơn: tôi chỉ sửa dòng 259 và 263 để dùng equalsIgnoreCase.

Tôi để các kiểm thử ở dòng 32 và dòng 45 dưới dạng chú thích vì tôi không chắc có

nên hỗ trợ các dạng viết tắt “tues” và “thurs” hay không.

Các kiểm thử ở dòng 153 và dòng 154 không chạy qua. Rõ ràng chúng phải chạy qua [G2]. Ta có thể dễ dàng sửa chúng, cùng với các kiểm thử từ dòng 163 đến dòng 213, bằng cách thay đổi hàm stringToMonthCode như sau.

```
457          if ((result < 1) || (result > 12)) {
                 result = -1;
458              for (int i = 0; i < monthNames.length; i++) {
459                  if (s.equalsIgnoreCase(shortMonthNames[i])) {
460                      result = i + 1;
461                      break;
462                  }
463                  if (s.equalsIgnoreCase(monthNames[i])) {
464                      result = i + 1;
465                      break;
466                  }
467              }
468          }
```

Kiểm thử bị chú thích ở dòng 318 làm lộ ra lỗi trong phương thức getFollowingDayOfWeek (dòng 672). Ngày 25 tháng 12 năm 2004 là thứ Bảy. Thứ Bảy tiếp theo là ngày 1 tháng 1 năm 2005. Thế nhưng khi chạy kiểm thử, ta thấy getFollowingDayOfWeek trả về ngày 25 tháng 12 như thể đó là thứ Bảy tiếp theo sau ngày 25 tháng 12. Rõ ràng điều này sai [G3],[T1]. Vấn đề nằm ở dòng 685. Đây là lỗi điều kiện biên điển hình [T5]. Dòng đó phải là:

```
685          if (baseDOW >= targetWeekday) {
```

Điều đáng chú ý là hàm này từng được sửa trước đó. Lịch sử thay đổi (dòng 43) cho thấy các “lỗi” đã được sửa trong getPreviousDayOfWeek, getFollowingDayOfWeek và getNearestDayOfWeek [T6].

Kiểm thử đơn vị testGetNearestDayOfWeek (dòng 329), kiểm thử phương thức getNearestDayOfWeek (dòng 705), ban đầu không dài và đầy đủ như bây giờ. Tôi đã thêm nhiều ca kiểm thử vì các ca ban đầu không phải tất cả đều chạy qua [T6]. Bạn có thể nhận ra kiểu lỗi bằng cách xem những ca kiểm thử nào bị chú thích. Kiểu đó cho ta biết nhiều điều [T7]. Nó cho thấy thuật toán thất bại nếu ngày gần nhất nằm trong tương lai. Rõ ràng có một dạng lỗi điều kiện biên nào đó [T5].

Kiểu độ bao phủ kiểm thử mà Clover báo cáo cũng đáng chú ý [T8]. Dòng 719 không bao giờ được thực thi! Điều đó có nghĩa câu lệnh if ở dòng 718 luôn sai. Quả đúng như vậy, xem mã thì thấy điều này là tất yếu. Biến adjust luôn âm, nên không thể lớn hơn hoặc bằng 4. Vậy thuật toán này đơn giản là sai.

Thuật toán đúng như sau:

```
int delta = targetDOW - base.getDayOfWeek();
int positiveDelta = delta + 7;
int adjust = positiveDelta % 7;
if (adjust > 3)
  adjust -= 7;
```

```
        return SerialDate.addDays(adjust, base);
```

Cuối cùng, có thể làm cho các kiểm thử ở dòng 417 và dòng 429 chạy qua chỉ bằng cách ném IllegalArgumentException thay vì trả về chuỗi lỗi từ weekInMonthToString và relativeToString.

Với những thay đổi này, tất cả kiểm thử đơn vị đều chạy qua, và tôi tin SerialDate hiện đã hoạt động. Vậy giờ là lúc làm cho nó "đúng đắn".

Sau đó, làm cho nó đúng đắn

Chúng ta sẽ đi từ đầu đến cuối SerialDate, cải thiện nó trên đường đi. Dù bạn không thấy điều này trong phần thảo luận, sau mỗi thay đổi tôi đều chạy toàn bộ kiểm thử đơn vị của JCommon, bao gồm cả kiểm thử đơn vị SerialDate đã cải tiến. Vì vậy, bạn có thể yên tâm rằng mọi thay đổi trình bày ở đây đều hoạt động với toàn bộ JCommon.

Bắt đầu từ dòng 1, ta thấy một loạt chú thích về giấy phép, bản quyền, tác giả và lịch sử thay đổi. Tôi thừa nhận có những vấn đề pháp lý cần giải quyết, nên thông tin bản quyền và giấy phép phải được giữ lại. Mặt khác, lịch sử thay đổi là tàn dư từ thập niên 1960. Giờ đây chúng ta có các công cụ quản lý mã nguồn để làm việc đó. Nên xóa lịch sử này [C1].

Có thể rút gọn danh sách import bắt đầu ở dòng 61 bằng cách dùng java.text.* và java.util.*. [J1]

Tôi nhăn mặt trước cách định dạng HTML trong Javadoc (dòng 67). Tôi không thích một tệp mã nguồn chứa nhiều hơn một ngôn ngữ. Chú thích này có bốn ngôn ngữ: Java, tiếng Anh, Javadoc và HTML [G1]. Dùng nhiều ngôn ngữ như vậy khiến việc giữ mọi thứ ngăn nắp trở nên khó khăn. Chẳng hạn, cách căn chỉnh đẹp ở dòng 71 và dòng 72 bị mất khi tạo Javadoc, nhưng ai lại muốn thấy <ul> và <li> trong mã nguồn? Một cách tốt hơn có thể là đặt toàn bộ chú thích bên trong <pre> để giữ nguyên định dạng thể hiện trong mã nguồn khi hiển thị trong Javadoc.[1]

Dòng 86 là khai báo lớp. Tại sao lớp này tên là SerialDate? Từ "serial" có ý nghĩa gì? Có phải vì lớp này được kế thừa từ Serializable không? Có vẻ không phải.

Tôi sẽ không để bạn phải đoán. Tôi biết lý do (hoặc ít nhất tôi nghĩ mình biết lý do) dùng từ "serial". Manh mối nằm ở các hằng SERIAL_LOWER_BOUND và SERIAL_UPPER_BOUND tại dòng 98 và dòng 101. Manh mối rõ hơn nữa nằm trong chú thích bắt đầu ở dòng 830. Lớp này tên SerialDate vì nó được triển khai bằng một "số sê-ri", tức số ngày tính từ ngày 30 tháng 12 năm 1899.

Tôi có hai vấn đề với cách gọi này. Thứ nhất, thuật ngữ "số sê-ri" không thực sự chính xác. Có thể đây chỉ là chuyện bắt bẻ, nhưng cách biểu diễn này giống độ lệch tương đối hơn là số sê-ri. "Số sê-ri" thường liên quan đến mã nhận diện sản phẩm hơn là ngày tháng. Vì thế, tôi không thấy tên này mô tả rõ ràng [N1]. Một từ mô tả phù hợp hơn có thể là "ordinal".

Vấn đề thứ hai đáng kể hơn. Tên SerialDate hàm ý một cách triển khai. Đây là lớp

1. Một giải pháp còn tốt hơn là để Javadoc trình bày mọi chú thích dưới dạng văn bản định dạng sẵn, để chú thích hiển thị giống nhau trong mã nguồn và tài liệu.

trừu tượng. Không cần hàm ý gì về cách triển khai cả. Thật ra, có lý do chính đáng để che giấu cách triển khai! Vì vậy, tôi cho rằng tên này không đúng mức trừu tượng [N2]. Theo tôi, tên lớp này đơn giản nên là Date.

Đáng tiếc là trong thư viện Java đã có quá nhiều lớp tên Date, nên có lẽ đây không phải lựa chọn tốt nhất. Vì lớp này nói về ngày, chứ không phải thời gian, tôi cân nhắc đặt tên Day, nhưng tên đó cũng được dùng nhiều ở nơi khác. Cuối cùng, tôi chọn DayDate làm phương án dung hòa tốt nhất.

Từ giờ trở đi trong phần thảo luận này, tôi sẽ dùng tên DayDate. Bạn cần nhớ rằng các đoạn mã đang xem vẫn dùng tên SerialDate.

Tôi hiểu vì sao DayDate kế thừa Comparable và Serializable. Nhưng tại sao nó lại kế thừa MonthConstants? Lớp MonthConstants (Đoạn mã B-3, trang 372) chỉ là một nhóm các hằng static final xác định các tháng. Kế thừa lớp chứa hằng là một mẹo cũ mà lập trình viên Java dùng để khỏi phải viết những biểu thức như MonthConstants.January, nhưng đó là ý tưởng tồi [J2]. MonthConstants thực sự nên là một enum.

```
public abstract class DayDate implements Comparable,
                                         Serializable {
  public static enum Month {
    JANUARY(1),
    FEBRUARY(2),
    MARCH(3),
    APRIL(4),
    MAY(5),
    JUNE(6),
    JULY(7),
    AUGUST(8),
    SEPTEMBER(9),
    OCTOBER(10),
    NOVEMBER(11),
    DECEMBER(12);

     Month(int index) {
       this.index = index;
     }

         public static Month make(int monthIndex) {
           for (Month m : Month.values()) {
             if (m.index == monthIndex)
               return m;
           }
           throw new IllegalArgumentException("Invalid month index " +
monthIndex);
         }
         public final int index;
     }
```

Đổi MonthConstants thành enum này buộc phải sửa khá nhiều chỗ trong lớp DayDate và mọi nơi dùng nó. Tôi mất một giờ để hoàn tất. Tuy nhiên, giờ đây mọi hàm trước kia nhận số nguyên chỉ tháng đều nhận một giá trị Month. Điều đó giúp ta bỏ được phương thức isValidMonthCode (dòng 326) và mọi thao tác kiểm tra mã tháng như trong monthCodeToQuarter (dòng 356) [G5].

Tiếp theo là serialVersionUID ở dòng 91. Biến này dùng để kiểm soát việc tuần tự hóa. Nếu ta thay đổi nó, mọi DayDate được ghi bằng phiên bản phần mềm cũ sẽ không còn đọc được và gây ra InvalidClassException. Nếu không khai báo biến serialVersionUID, trình biên dịch sẽ tự động tạo một giá trị, và giá trị đó sẽ khác mỗi lần bạn thay đổi mô-đun. Tôi biết mọi tài liệu đều khuyên nên tự kiểm soát biến này, nhưng theo tôi, tự động kiểm soát việc tuần tự hóa an toàn hơn nhiều [G4]. Dù sao, tôi thà gỡ lỗi InvalidClassException còn hơn gặp hành vi kỳ quặc xảy ra nếu quên đổi serialVersionUID. Vì vậy, tôi sẽ xóa biến này, ít nhất là lúc này.[2]

Tôi thấy chú thích ở dòng 93 thừa. Chú thích thừa chỉ là nơi tích tụ lời dối trá và thông tin sai lệch [C2]. Vì thế, tôi sẽ xóa nó cùng những chú thích tương tự.

Các chú thích ở dòng 97 và dòng 100 nói về số sê-ri, vấn đề tôi vừa bàn ở trên [C1]. Những biến này mô tả ngày sớm nhất và muộn nhất mà DayDate có thể biểu diễn. Ta có thể làm rõ hơn một chút [N1].

```
public static final int EARLIEST_DATE_ORDINAL = 2;     // 1/1/1900
public static final int LATEST_DATE_ORDINAL = 2958465; // 12/31/9999
```

Tôi không rõ vì sao EARLIEST_DATE_ORDINAL là 2 chứ không phải 0. Chú thích ở dòng 829 hé lộ rằng chuyện này có liên quan đến cách Microsoft Excel biểu diễn ngày tháng. Có thêm một giải thích sâu hơn trong lớp dẫn xuất DayDate tên SpreadsheetDate (Đoạn mã B-5, trang 382). Chú thích ở dòng 71 mô tả vấn đề rất rõ.

Vấn đề của tôi là chuyện này dường như liên quan đến cách triển khai SpreadsheetDate, không liên quan gì đến DayDate. Tôi kết luận rằng EARLIEST_DATE_ORDINAL và LATEST_DATE_ORDINAL thực sự không thuộc về DayDate, mà nên được chuyển sang SpreadsheetDate [G6].

Quả thật, tìm trong mã cho thấy những biến này chỉ được dùng bên trong SpreadsheetDate. Không có gì trong DayDate hay bất kỳ lớp nào khác của framework JCommon dùng chúng. Vì vậy, tôi sẽ chuyển chúng xuống SpreadsheetDate.

Các biến tiếp theo, MINIMUM_YEAR_SUPPORTED và MAXIMUM_YEAR_SUPPORTED (dòng 104 và dòng 107), đặt ra một tình thế khó xử. Nếu DayDate là lớp trừu tượng không hé lộ gì về cách triển khai, thì rõ ràng lớp này không nên cho biết năm tối thiểu hay tối đa. Một lần nữa, tôi muốn chuyển các biến này xuống SpreadsheetDate [G6]. Tuy nhiên, tìm nhanh nơi sử dụng cho thấy còn một lớp khác dùng chúng: RelativeDayOfWeekRule (Đoạn mã B-6, trang 390). Ta thấy cách sử dụng ở dòng 177 và dòng 178 trong hàm getDate, nơi chúng được dùng để kiểm tra năm truyền vào getDate có hợp lệ hay không. Tình thế khó xử ở đây là một lớp sử dụng lớp trừu tượng lại cần biết thông tin về cách triển khai của lớp đó.

---

2. Một số người phản biện văn bản này không đồng tình với quyết định đó. Họ cho rằng trong một framework mã nguồn mở, tốt hơn hết là tự kiểm soát ID tuần tự để những thay đổi nhỏ trong phần mềm không làm ngày tháng đã tuần tự hóa từ trước mất hiệu lực. Đây là ý kiến xác đáng. Tuy nhiên, ít nhất thì dù bất tiện, lỗi này có nguyên nhân rõ ràng. Ngược lại, nếu tác giả lớp quên cập nhật ID, kiểu lỗi sẽ không xác định và có thể hoàn toàn im lặng. Theo tôi, bài học thực sự là không nên kỳ vọng giải tuần tự hóa dữ liệu giữa các phiên bản.

Ta cần cung cấp thông tin này mà không làm ô nhiễm chính DayDate. Thông thường, ta sẽ lấy thông tin về cách triển khai từ một thực thể của lớp dẫn xuất. Nhưng hàm getDate không nhận một thực thể DayDate làm đối số. Tuy vậy, nó trả về một thực thể như thế, nghĩa là ở đâu đó nó phải tạo thực thể ấy. Các dòng từ 187 đến 205 gợi ý câu trả lời. Thực thể DayDate được tạo bởi một trong ba hàm getPreviousDayOfWeek, getNearestDayOfWeek hoặc getFollowingDayOfWeek. Nhìn lại đoạn mã DayDate, ta thấy các hàm này (dòng 638–724) đều trả về một ngày được tạo bởi addDays (dòng 571), hàm này gọi createInstance (dòng 808), và hàm đó tạo SpreadsheetDate! [G7]

Thông thường, lớp cơ sở không nên biết về các lớp dẫn xuất của nó. Để khắc phục điều này, ta nên dùng mẫu ABSTRACT FACTORY[3] và tạo DayDateFactory. Factory này sẽ tạo các thực thể DayDate cần thiết, đồng thời có thể trả lời những câu hỏi về cách triển khai, chẳng hạn ngày tối đa và tối thiểu.

```
public abstract class DayDateFactory {
  private static DayDateFactory factory = new SpreadsheetDateFactory();
  public static void setInstance(DayDateFactory factory) {
    DayDateFactory.factory = factory;
  }

   protected abstract DayDate _makeDate(int ordinal);
   protected abstract DayDate _makeDate(int day, DayDate.Month month, int
year);
   protected abstract DayDate _makeDate(int day, int month, int year);
   protected abstract DayDate _makeDate(java.util.Date date);
   protected abstract int _getMinimumYear();
   protected abstract int _getMaximumYear();

   public static DayDate makeDate(int ordinal) {
     return factory._makeDate(ordinal);
   }

       public static DayDate makeDate(int day, DayDate.Month month, int
year) {
         return factory._makeDate(day, month, year);
       }

       public static DayDate makeDate(int day, int month, int year) {
         return factory._makeDate(day, month, year);
       }
       public static DayDate makeDate(java.util.Date date) {
         return factory._makeDate(date);
       }

       public static int getMinimumYear() {
         return factory._getMinimumYear();
       }

       public static int getMaximumYear() {
         return factory._getMaximumYear();
       }
  }
```

3. [GOF].

Lớp factory này thay các phương thức createInstance bằng phương thức makeDate, cải thiện tên gọi đáng kể [N1]. Mặc định nó dùng SpreadsheetDateFactory, nhưng có thể đổi sang factory khác bất cứ lúc nào. Các phương thức static ủy quyền cho phương thức trừu tượng sử dụng kết hợp các mẫu SINGLETON,[4] DECORATOR,[5] và ABSTRACT FACTORY mà tôi thấy hữu ích.

SpreadsheetDateFactory trông như sau.

```
        public class SpreadsheetDateFactory extends DayDateFactory {
          public DayDate _makeDate(int ordinal) {
            return new SpreadsheetDate(ordinal);
          }
             public DayDate _makeDate(int day, DayDate.Month month, int year) {
               return new SpreadsheetDate(day, month, year);
             }
             public DayDate _makeDate(int day, int month, int year) {
               return new SpreadsheetDate(day, month, year);
             }

             public DayDate _makeDate(Date date) {
               final GregorianCalendar calendar = new GregorianCalendar();
               calendar.setTime(date);
               return new SpreadsheetDate(
                 calendar.get(Calendar.DATE),
                 DayDate.Month.make(calendar.get(Calendar.MONTH) + 1),
                 calendar.get(Calendar.YEAR));
             }

          protected int _getMinimumYear() {
            return SpreadsheetDate.MINIMUM_YEAR_SUPPORTED;
          }

          protected int _getMaximumYear() {
            return SpreadsheetDate.MAXIMUM_YEAR_SUPPORTED;
          }
      }
```

Như bạn thấy, tôi đã chuyển các biến MINIMUM_YEAR_SUPPORTED và MAXIMUM_YEAR_SUPPORTED vào SpreadsheetDate, nơi chúng thuộc về [G6].

Vấn đề tiếp theo trong DayDate là các hằng ngày bắt đầu từ dòng 109. Thực ra, chúng nên là một enum khác [J3]. Ta đã thấy kiểu này trước đây, nên tôi sẽ không nhắc lại. Bạn sẽ thấy nó trong các đoạn mã cuối.

Tiếp theo là một loạt bảng, bắt đầu với LAST_DAY_OF_MONTH ở dòng 140. Vấn đề đầu tiên của tôi với các bảng này là những chú thích mô tả chúng bị thừa [C3]. Tên đã đủ rõ. Vì vậy, tôi sẽ xóa các chú thích.

Không có lý do chính đáng nào để bảng này không phải private [G8], vì đã có hàm static lastDayOfMonth cung cấp cùng dữ liệu.

Bảng tiếp theo, AGGREGATE_DAYS_TO_END_OF_MONTH, khó hiểu hơn một

4. Như trên.

5. Như trên.

chút vì không được dùng ở đâu trong framework JCommon [G9]. Vì vậy, tôi đã xóa nó.

LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_MONTH cũng vậy.

Bảng tiếp theo, AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH, chỉ được dùng trong SpreadsheetDate (dòng 434 và dòng 473). Điều này đặt ra câu hỏi liệu có nên chuyển nó sang SpreadsheetDate hay không. Lý do không chuyển là bảng này không đặc thù cho một cách triển khai nào [G6]. Mặt khác, ngoài SpreadsheetDate ra thì chẳng có cách triển khai nào khác tồn tại, nên cần chuyển bảng đến gần nơi sử dụng nó hơn [G10].

Điều khiển tôi quyết định là để nhất quán [G11], ta nên đặt bảng thành private và cung cấp nó qua một hàm như julianDateOfLastDayOfMonth. Dường như không ai cần hàm như vậy. Hơn nữa, nếu một cách triển khai DayDate mới cần đến bảng này, ta có thể dễ dàng chuyển nó trở lại DayDate. Vì vậy, tôi đã chuyển nó đi.

Bảng LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_MONTH cũng tương tự.

Tiếp theo, ta thấy ba nhóm hằng có thể chuyển thành enum (dòng 162–205). Nhóm đầu tiên chọn một tuần trong tháng. Tôi đổi nó thành enum tên WeekInMonth.

```
      public enum WeekInMonth {
          FIRST(1), SECOND(2), THIRD(3), FOURTH(4), LAST(0);
          public final int index;

              WeekInMonth(int index) {
                this.index = index;
              }
          }
```

Nhóm hằng thứ hai (dòng 177–187) khó hiểu hơn một chút. Các hằng INCLUDE_NONE, INCLUDE_FIRST, INCLUDE_SECOND và INCLUDE_BOTH dùng để mô tả các ngày ở hai đầu mút xác định một phạm vi có được tính vào phạm vi đó hay không. Trong toán học, người ta mô tả điều này bằng các thuật ngữ “khoảng mở”, “khoảng nửa mở” và “khoảng đóng”. Tôi nghĩ dùng thuật ngữ toán học sẽ rõ hơn [N3], nên đổi chúng thành enum tên DateInterval với các giá trị CLOSED, CLOSED_LEFT, CLOSED_RIGHT và OPEN.

Nhóm hằng thứ ba (dòng 18–205) mô tả việc tìm một ngày trong tuần cụ thể sẽ trả về lần xuất hiện trước, sau hay gần nhất. Tìm tên phù hợp thật không dễ. Cuối cùng, tôi chọn WeekdayRange với các giá trị LAST, NEXT và NEAREST.

Có thể bạn không đồng ý với những tên tôi chọn. Chúng hợp lý với tôi, nhưng có thể không hợp với bạn. Điểm quan trọng là giờ đây chúng ở dạng dễ thay đổi [J3]. Chúng không còn được truyền dưới dạng số nguyên mà là các ký hiệu. Tôi có thể dùng chức năng “change name” trong IDE để đổi tên hoặc kiểu mà không lo bỏ sót một số -1 hay 2 nào đó trong mã, hoặc để lại khai báo đối số int được mô tả kém.

Trường description ở dòng 208 dường như không ai dùng. Tôi đã xóa nó cùng phương thức truy cập và thay đổi [G9].

Tôi cũng xóa hàm khởi tạo mặc định rỗng ở dòng 213 [G12]. Trình biên dịch sẽ tạo nó giúp ta.

Có thể bỏ qua phương thức isValidWeekdayCode (dòng 216–238) vì ta đã xóa nó khi tạo enum Day.

Tiếp theo là phương thức stringToWeekdayCode (dòng 242–270). Javadoc không bổ

sung gì nhiều cho chữ ký phương thức chỉ làm mã thêm rối [C3],[G12]. Giá trị duy nhất Javadoc này cung cấp là mô tả giá trị trả về -1. Tuy nhiên, vì ta đã chuyển sang enum Day, chú thích đó thực ra sai [C2]. Giờ phương thức ném IllegalArgumentException. Vì vậy, tôi đã xóa Javadoc.

Tôi cũng xóa mọi từ khóa final trong đối số và khai báo biến. Theo những gì tôi thấy, chúng không đem lại giá trị thực sự nào mà chỉ làm mã rối hơn [G12]. Bỏ final đi ngược một số quan niệm thông thường. Chẳng hạn, Robert Simmons[6] khuyên chúng ta mạnh mẽ rằng ". . . hãy rải final khắp mã nguồn." Rõ ràng tôi không đồng ý. Tôi nghĩ final có một vài cách dùng tốt, chẳng hạn đôi khi dùng cho hằng, nhưng ngoài ra từ khóa này ít giá trị và làm mã rối đáng kể. Có lẽ tôi nghĩ vậy vì những lỗi mà final có thể bắt được vốn đã được các kiểm thử đơn vị tôi viết phát hiện.

Tôi không thích các câu lệnh if trùng lặp [G5] bên trong vòng lặp for (dòng 259 và dòng 263), nên nối chúng thành một câu lệnh if duy nhất bằng toán tử ||. Tôi cũng dùng enum Day để điều khiển vòng lặp for và thực hiện thêm một số thay đổi nhỏ khác.

Tôi nhận ra phương thức này thực sự không thuộc về DayDate. Nó đúng ra là hàm parse của Day. Vì vậy, tôi chuyển nó vào enum Day. Tuy nhiên, enum Day trở nên khá lớn. Vì khái niệm Day không phụ thuộc vào DayDate, tôi chuyển enum Day ra khỏi lớp DayDate, sang tệp mã nguồn riêng [G13].

Tôi cũng chuyển hàm tiếp theo, weekdayCodeToString (dòng 272–286), vào enum Day và gọi nó là toString.

```
       public enum Day {
         MONDAY(Calendar.MONDAY),
         TUESDAY(Calendar.TUESDAY),
         WEDNESDAY(Calendar.WEDNESDAY),s
         THURSDAY(Calendar.THURSDAY),
         FRIDAY(Calendar.FRIDAY),
         SATURDAY(Calendar.SATURDAY),
         SUNDAY(Calendar.SUNDAY);

           public final int index;
           private static DateFormatSymbols dateSymbols = new
       DateFormatSymbols();

           Day(int day) {
             index = day;
           }

           public static Day make(int index) throws IllegalArgumentException {
             for (Day d : Day.values())
               if (d.index == index)
                 return d;
             throw new IllegalArgumentException(
               String.format("Illegal day index: %d.", index));
           }

           public static Day parse(String s) throws IllegalArgumentException {
             String[] shortWeekdayNames =
```

6. [Simmons04], trang 73.

```
            dateSymbols.getShortWeekdays();
          String[] weekDayNames =
            dateSymbols.getWeekdays();
            s = s.trim();
            for (Day day : Day.values()) {
              if (s.equalsIgnoreCase(shortWeekdayNames[day.index]) ||
                  s.equalsIgnoreCase(weekDayNames[day.index])) {
                return day;
              }
            }
            throw new IllegalArgumentException(
              String.format("%s is not a valid weekday string", s));
        }
        public String toString() {
          return dateSymbols.getWeekdays()[index];
        }
      }
```

Có hai hàm getMonths (dòng 288–316). Hàm đầu gọi hàm thứ hai. Không ai gọi hàm thứ hai ngoài hàm đầu. Vì vậy, tôi gộp hai hàm thành một và đơn giản hóa đáng kể [G9],[G12],[F4]. Cuối cùng, tôi đổi tên cho tự mô tả hơn [N1].

```
        public static String[] getMonthNames() {
          return dateFormatSymbols.getMonths();
        }
```

Hàm isValidMonthCode (dòng 326–346) trở nên không cần thiết vì enum Month, nên tôi xóa nó [G9].

Hàm monthCodeToQuarter có mùi FEATURE ENVY[7] [G14] và có lẽ thuộc về enum Month dưới dạng phương thức tên quarter. Vì vậy, tôi thay nó bằng phương thức này.

```
            public int quarter() {
              return 1 + (index-1)/3;
            }
```

Enum Month giờ đủ lớn để nằm trong lớp riêng. Vì nhất quán với enum Day, tôi chuyển nó ra khỏi DayDate [G11],[G13].

Hai phương thức tiếp theo tên là monthCodeToString (dòng 377–426). Một lần nữa, ta thấy kiểu một phương thức gọi phương thức song sinh bằng một cờ. Truyền cờ làm đối số cho hàm thường là ý tưởng tồi, nhất là khi cờ đó chỉ chọn định dạng đầu ra [G15]. Tôi đổi tên, đơn giản hóa, sắp xếp lại các hàm này và chuyển chúng vào enum Month [N1],[N3],[C3],[G14].

```
        public String toString() {
          return dateFormatSymbols.getMonths()[index - 1];
        }

        public String toShortString() {
          return dateFormatSymbols.getShortMonths()[index - 1];
```

7. [Refactoring].

```
  }
```

Phương thức tiếp theo là stringToMonthCode (dòng 428–472). Tôi đổi tên, chuyển nó vào enum Month và đơn giản hóa [N1],[N3],[C3],[G14],[G12].

```
  public static Month parse(String s) {
    s = s.trim();
    for (Month m : Month.values())
      if (m.matches(s))
        return m;

      try {
        return make(Integer.parseInt(s));
      }
      catch (NumberFormatException e) {}
      throw new IllegalArgumentException("Invalid month " + s);
  }

  private boolean matches(String s) {
    return s.equalsIgnoreCase(toString()) ||
           s.equalsIgnoreCase(toShortString());
  }
```

Phương thức isLeapYear (dòng 495–517) có thể được viết biểu đạt hơn một chút [G16].

```
  public static boolean isLeapYear(int year) {
    boolean fourth = year % 4 == 0;
    boolean hundredth = year % 100 == 0;
    boolean fourHundredth = year % 400 == 0;
    return fourth && (!hundredth || fourHundredth);
  }
```

Hàm tiếp theo, leapYearCount (dòng 519–536), thực sự không thuộc về DayDate. Không ai gọi nó ngoài hai phương thức trong SpreadsheetDate. Vì vậy, tôi chuyển nó xuống đó [G6].

Hàm lastDayOfMonth (dòng 538–560) dùng mảng LAST_DAY_OF_MONTH. Mảng này thực sự thuộc về enum Month [G17], nên tôi chuyển nó vào đó. Tôi cũng đơn giản hóa hàm và làm nó dễ hiểu hơn [G16].

```
  public static int lastDayOfMonth(Month month, int year) {
    if (month == Month.FEBRUARY && isLeapYear(year))
      return month.lastDay() + 1;
     else
      return month.lastDay();
  }
```

Giờ mọi thứ bắt đầu thú vị hơn một chút. Hàm tiếp theo là addDays (dòng 562–576). Trước hết, vì hàm này thao tác trên các biến của DayDate nên không nên là static [G18]. Vì vậy, tôi đổi nó thành phương thức thực thể. Thứ hai, nó gọi hàm toSerial. Nên đổi tên hàm này thành toOrdinal [N1]. Cuối cùng, có thể đơn giản hóa phương thức.

```
    public DayDate addDays(int days) {
      return DayDateFactory.makeDate(toOrdinal() + days);
    }
```

addMonths (dòng 578–602) cũng vậy. Nó nên là phương thức thực thể [G18]. Thuật toán hơi phức tạp, nên tôi dùng các BIẾN TẠM GIẢI THÍCH[8] [G19] để làm rõ hơn. Tôi cũng đổi tên phương thức getYYY thành getYear [N1].

```
    public DayDate addMonths(int months) {
      int thisMonthAsOrdinal = 12 * getYear() + getMonth().index - 1;
      int resultMonthAsOrdinal = thisMonthAsOrdinal + months;
      int resultYear = resultMonthAsOrdinal / 12;
      Month resultMonth = Month.make(resultMonthAsOrdinal % 12 + 1);

        int lastDayOfResultMonth = lastDayOfMonth(resultMonth,
resultYear);
        int resultDay = Math.min(getDayOfMonth(), lastDayOfResultMonth);
        return DayDateFactory.makeDate(resultDay, resultMonth,
resultYear);
    }
```

Hàm addYears (dòng 604–626) không có gì bất ngờ so với các hàm khác.

```
    public DayDate plusYears(int years) {
      int resultYear = getYear() + years;
      int lastDayOfMonthInResultYear = lastDayOfMonth(getMonth(),
resultYear);
      int resultDay = Math.min(getDayOfMonth(),
lastDayOfMonthInResultYear);
      return DayDateFactory.makeDate(resultDay, getMonth(), resultYear);
    }
```

Trong đầu tôi có một điều cứ vướng mắc về việc đổi các phương thức này từ static sang phương thức thực thể. Biểu thức date.addDays(5) có nói rõ rằng đối tượng ngày không thay đổi và một thực thể DayDate mới được trả về hay không? Hay nó khiến người đọc hiểu nhầm rằng ta đang cộng năm ngày vào đối tượng ngày? Có thể bạn không nghĩ đó là vấn đề lớn, nhưng đoạn mã như sau rất dễ gây hiểu nhầm [G20].

```
DayDate date = DateFactory.makeDate(5, Month.DECEMBER, 1952);
date.addDays(7); // tăng ngày thêm một tuần.
```

Người đọc đoạn mã này rất có thể sẽ cho rằng addDays thay đổi đối tượng ngày. Vì vậy, ta cần một tên phá vỡ sự mơ hồ này [N4]. Tôi đổi tên thành plusDays và plusMonths. Theo tôi, ý định của phương thức được thể hiện rõ trong cách viết này:

```
DayDate date = oldDate.plusDays(5);
```

8. [Beck97].

Còn cách viết sau không trôi chảy đủ để người đọc mặc nhiên hiểu rằng đối tượng ngày bị thay đổi:

```
        date.plusDays(5);
```

Các thuật toán tiếp tục trở nên thú vị hơn. getPreviousDayOfWeek (dòng 628–660) hoạt động nhưng hơi phức tạp. Sau khi suy nghĩ về điều thực sự đang diễn ra [G21], tôi đơn giản hóa được nó và dùng BIẾN TẠM GIẢI THÍCH [G19] để làm rõ. Tôi cũng đổi nó từ phương thức static sang phương thức thực thể [G18], đồng thời bỏ phương thức thực thể trùng lặp [G5] (dòng 997–1008).

```
          public DayDate getPreviousDayOfWeek(Day targetDayOfWeek) {
            int offsetToTarget = targetDayOfWeek.index - getDayOfWeek().index;
            if (offsetToTarget >= 0)
              offsetToTarget -= 7;
            return plusDays(offsetToTarget);
          }
```

getFollowingDayOfWeek (dòng 662–693) được phân tích và cho kết quả y hệt.

```
        public DayDate getFollowingDayOfWeek(Day targetDayOfWeek) {
            int offsetToTarget = targetDayOfWeek.index - getDayOfWeek().index;
            if (offsetToTarget <= 0)
               offsetToTarget += 7;
             return plusDays(offsetToTarget);
         }
```

Hàm tiếp theo là getNearestDayOfWeek (dòng 695–726), hàm mà ta đã sửa ở trang 270. Nhưng những thay đổi tôi thực hiện khi đó không nhất quán với kiểu hiện tại của hai hàm trước [G11]. Vì vậy, tôi làm cho nó nhất quán, đồng thời dùng một số BIẾN TẠM GIẢI THÍCH [G19] để làm rõ thuật toán.

```
       public DayDate getNearestDayOfWeek(final Day targetDay) {
           int offsetToThisWeeksTarget = targetDay.index - getDayOfWeek().index;
           int offsetToFutureTarget = (offsetToThisWeeksTarget + 7) % 7;
           int offsetToPreviousTarget = offsetToFutureTarget - 7;

              if (offsetToFutureTarget > 3)
                return plusDays(offsetToPreviousTarget);
              else
                return plusDays(offsetToFutureTarget);
         }
```

Phương thức getEndOfCurrentMonth (dòng 728–740) hơi kỳ lạ vì nó là một phương thức thực thể nhưng lại có mùi [G14] lớp của chính nó khi nhận đối số DayDate. Tôi biến nó thành phương thức thực thể đúng nghĩa và làm rõ vài tên gọi.

```
    public DayDate getEndOfMonth() {
        Month month = getMonth();
        int year = getYear();
        int lastDay = lastDayOfMonth(month, year);
        return DayDateFactory.makeDate(lastDay, month, year);
      }
```

Tái cấu trúc weekInMonthToString (dòng 742–761) quả thực rất thú vị. Trước tiên, tôi dùng công cụ tái cấu trúc trong IDE để chuyển phương thức vào enum WeekInMonth mà tôi đã tạo ở trang 275. Sau đó, tôi đổi tên phương thức thành toString. Tiếp theo, tôi đổi nó từ phương thức static thành phương thức thực thể. Tất cả kiểm thử vẫn chạy qua. (Bạn đoán được tôi định làm gì chưa?)

Tiếp đó, tôi xóa hẳn phương thức này! Năm lệnh assert thất bại (dòng 411–415, Đoạn mã B-4, trang 374). Tôi sửa những dòng đó để dùng tên các giá trị enum (FIRST, SECOND, . . .). Tất cả kiểm thử đều chạy qua. Bạn hiểu vì sao không? Bạn cũng hiểu vì sao cần từng bước này không? Công cụ tái cấu trúc đảm bảo mọi nơi gọi weekInMonthToString trước đó giờ đều gọi toString trên giá trị weekInMonth, vì mọi giá trị enum đều triển khai toString để đơn giản trả về tên của chúng. . . .

Đáng tiếc là tôi đã tỏ ra khôn khéo quá mức. Chuỗi tái cấu trúc tuyệt vời ấy có thanh nhã đến đâu, cuối cùng tôi cũng nhận ra rằng những người duy nhất dùng hàm này là các kiểm thử mà tôi vừa sửa, nên tôi đã xóa các kiểm thử đó.

Lừa tôi lần đầu là lỗi của bạn. Lừa tôi lần thứ hai là lỗi của tôi! Sau khi xác định không ai ngoài các kiểm thử gọi relativeToString (dòng 765–781), tôi chỉ việc xóa hàm và các kiểm thử của nó.

Cuối cùng, ta cũng đến các phương thức trừu tượng của lớp trừu tượng này. Phương thức đầu tiên rất phù hợp: toSerial (dòng 838–844). Ở trang 279 tôi đã đổi tên nó thành toOrdinal. Xem xét nó trong ngữ cảnh này, tôi quyết định nên đổi tên thành getOrdinalDay.

Phương thức trừu tượng tiếp theo là toDate (dòng 838–844). Nó chuyển DayDate thành java.util.Date. Tại sao phương thức này lại trừu tượng? Nếu xem phần triển khai trong SpreadsheetDate (dòng 198–207, Đoạn mã B-5, trang 382), ta thấy nó không phụ thuộc vào điều gì trong cách triển khai của lớp đó [G6]. Vì vậy, tôi chuyển nó lên lớp cha.

Các phương thức getYYYY, getMonth và getDayOfMonth được trừu tượng hóa hợp lý. Tuy nhiên, getDayOfWeek cũng là phương thức nên được chuyển lên từ SpreadSheetDate vì nó không phụ thuộc vào thứ gì không thể tìm thấy trong DayDate [G6]. Hay là có?

Nếu xem kỹ (dòng 247, Đoạn mã B-5, trang 382), bạn sẽ thấy thuật toán ngầm phụ thuộc vào điểm gốc của ngày ordinal, nói cách khác là thứ trong tuần của ngày 0. Vì vậy, dù hàm này không có phụ thuộc vật lý nào không thể chuyển sang DayDate, nó vẫn có phụ thuộc logic.

Những phụ thuộc logic như vậy khiến tôi băn khoăn [G22]. Nếu thứ gì đó về mặt logic phụ thuộc vào cách triển khai thì về mặt vật lý cũng nên như vậy. Hơn nữa, theo tôi, bản thân thuật toán có thể dùng chung, chỉ một phần nhỏ hơn nhiều phụ thuộc vào cách triển khai [G6].

Vì thế, tôi tạo một phương thức trừu tượng trong DayDate tên getDayOfWeekForOrdinalZero và triển khai nó trong SpreadsheetDate để trả về Day.SATURDAY. Sau đó, tôi

chuyển phương thức getDayOfWeek lên DayDate và sửa nó để gọi getOrdinalDay cùng getDayOfWeekForOrdinalZero.

```
  public Day getDayOfWeek() {
      Day startingDay = getDayOfWeekForOrdinalZero();
      int startingOffset = startingDay.index - Day.SUNDAY.index;
      return Day.make((getOrdinalDay() + startingOffset) % 7 + 1);
    }
```

Nhân tiện, hãy xem kỹ chú thích từ dòng 895 đến dòng 899. Có thật cần lặp lại như vậy không? Như thường lệ, tôi xóa chú thích này cùng tất cả các chú thích khác.

Phương thức tiếp theo là compare (dòng 902–913). Một lần nữa, phương thức này bị trừu tượng hóa không đúng chỗ [G6], nên tôi chuyển phần triển khai lên DayDate. Tên gọi cũng chưa truyền đạt đủ [N1]. Phương thức này thực ra trả về số ngày chênh lệch so với đối số. Vì vậy, tôi đổi tên thành daysSince. Tôi cũng nhận thấy chưa có kiểm thử nào cho phương thức này, nên đã viết kiểm thử.

Sáu hàm tiếp theo (dòng 915–980) đều là phương thức trừu tượng cần được triển khai trong DayDate. Vì vậy, tôi chuyển tất cả chúng lên từ SpreadsheetDate.

Hàm cuối cùng, isInRange (dòng 982–995), cũng cần được chuyển lên và tái cấu trúc. Câu lệnh switch hơi xấu [G23], có thể thay bằng cách chuyển các trường hợp vào enum DateInterval.

```
public enum DateInterval {
    OPEN {
       public boolean isIn(int d, int left, int right) {
         return d > left && d < right;
       }
    },
    CLOSED_LEFT {
       public boolean isIn(int d, int left, int right) {
         return d >= left && d < right;
       }
    },
    CLOSED_RIGHT {
       public boolean isIn(int d, int left, int right) {
         return d > left && d <= right;
       }
    },
    CLOSED {
       public boolean isIn(int d, int left, int right) {
         return d >= left && d <= right;
       }
    };
      public abstract boolean isIn(int d, int left, int right);
  }

public boolean isInRange(DayDate d1, DayDate d2, DateInterval interval) {
    int left = Math.min(d1.getOrdinalDay(), d2.getOrdinalDay());
    int right = Math.max(d1.getOrdinalDay(), d2.getOrdinalDay());
    return interval.isIn(getOrdinalDay(), left, right);
  }
```

Vậy là kết thúc DayDate. Giờ chúng ta sẽ xem lại toàn bộ lớp thêm một lượt để xem nó trôi chảy đến đâu.

Trước hết, chú thích mở đầu đã quá lỗi thời, nên tôi rút ngắn và cải thiện nó [C2].

Tiếp theo, tôi chuyển mọi enum còn lại sang tệp riêng [G12].

Sau đó, tôi chuyển biến static (dateFormatSymbols) và ba phương thức static (getMonthNames, isLeapYear, lastDayOfMonth) vào lớp mới tên DateUtil [G6].

Tôi chuyển các phương thức trừu tượng lên đầu lớp, nơi chúng thuộc về [G24].

Tôi đổi Month.make thành Month.fromInt [N1] và làm tương tự với mọi enum khác. Tôi cũng tạo phương thức truy cập toInt() cho mọi enum và đặt trường index thành private.

Có một số đoạn trùng lặp thú vị [G5] trong plusYears và plusMonths mà tôi loại bỏ được bằng cách tách một phương thức mới tên correctLastDayOfMonth, giúp cả ba phương thức rõ ràng hơn nhiều.

Tôi bỏ số ma thuật 1 [G25], thay bằng Month.JANUARY.toInt() hoặc Day.SUNDAY.toInt() tùy trường hợp. Tôi dành một chút thời gian dọn dẹp SpreadsheetDate và cải thiện đôi chút các thuật toán. Kết quả cuối cùng nằm trong Đoạn mã B-7, trang 394, đến Đoạn mã B-16, trang 405.

Điều thú vị là độ bao phủ mã trong DayDate giảm xuống còn 84,9 phần trăm! Không phải vì có ít chức năng được kiểm thử hơn, mà vì lớp đã thu nhỏ đáng kể, khiến một vài dòng chưa được bao phủ chiếm tỷ trọng lớn hơn. Hiện DayDate có 45 trong số 53 câu lệnh có thể thực thi được kiểm thử bao phủ. Những dòng chưa được bao phủ quá đơn giản, không đáng để kiểm thử.

Kết luận

Một lần nữa, chúng ta đã làm theo Quy tắc Hướng đạo sinh. Ta trả mã về kho trong tình trạng sạch hơn một chút so với lúc lấy ra. Việc này mất chút thời gian, nhưng đáng công. Độ bao phủ kiểm thử tăng lên, một số lỗi được sửa, mã trở nên rõ ràng và gọn hơn. Hy vọng người tiếp theo xem đoạn mã này sẽ thấy dễ xử lý hơn chúng ta. Người đó có lẽ còn có thể dọn dẹp nó thêm một chút nữa.

Tài liệu tham khảo

[GOF]: *Design Patterns: Elements of Reusable Object Oriented Software*, Gamma et al., Addison-Wesley, 1996.

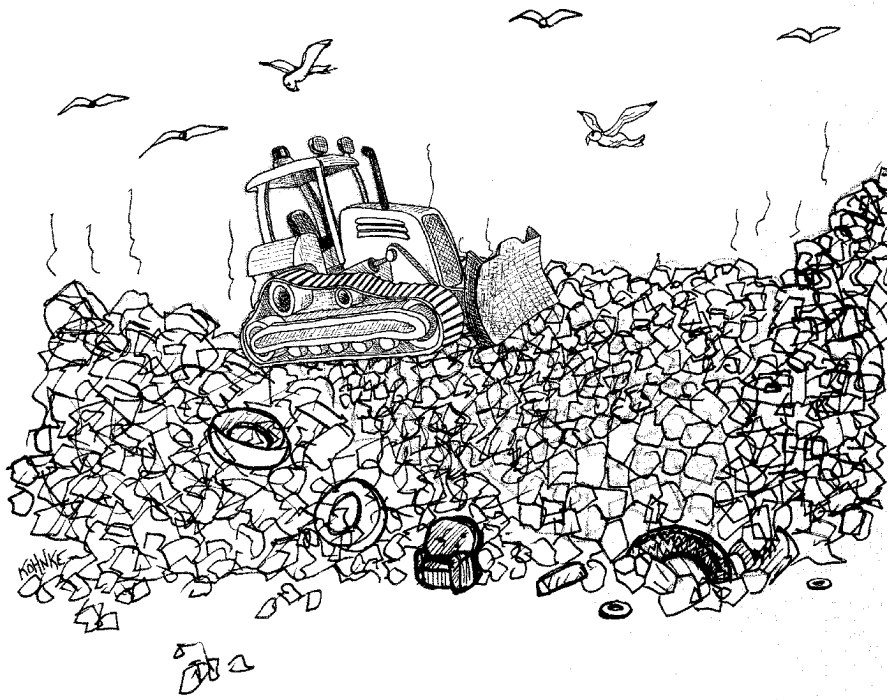
[Simmons04]: *Hardcore Java*, Robert Simmons, Jr., O'Reilly, 2004.

[Refactoring]: *Refactoring: Improving the Design of Existing Code*, Martin Fowler et al., Addison-Wesley, 1999.

[Beck97]: *Smalltalk Best Practice Patterns*, Kent Beck, Prentice Hall, 1997.

17

Mùi code và nguyên tắc kinh nghiệm



Trong cuốn sách tuyệt vời *Refactoring*,[1] Martin Fowler đã nhận diện nhiều "mùi code". Danh sách dưới đây gồm nhiều mùi code của Martin, đồng thời bổ sung thêm nhiều mùi do

1. [Refactoring].

tôi tự nhận ra. Danh sách cũng có những điều đúc kết và nguyên tắc kinh nghiệm khác mà tôi dùng trong nghề.

Tôi biên soạn danh sách này bằng cách xem qua một số chương trình khác nhau và tái cấu trúc chúng. Mỗi khi thực hiện một thay đổi, tôi tự hỏi vì sao mình làm thế rồi ghi lại lý do ở đây. Kết quả là một danh sách khá dài những điều khiến tôi thấy code có mùi khi đọc.

Danh sách này được sắp xếp để đọc từ trên xuống, đồng thời cũng dùng làm tài liệu tra cứu. Mỗi nguyên tắc kinh nghiệm đều có dẫn chiếu chéo, cho biết nó được nhắc đến ở đâu trong phần còn lại của sách, tại “Phụ lục C” ở trang 409.

Chú thích

C1: Thông tin không phù hợp

Chú thích không nên chứa những thông tin phù hợp hơn với một hệ thống khác, chẳng hạn hệ thống quản lý mã nguồn, hệ thống theo dõi vấn đề hay bất kỳ hệ thống lưu trữ hồ sơ nào khác. Ví dụ, lịch sử thay đổi chỉ khiến tệp mã nguồn rối thêm bởi những khối văn bản dài dòng, chẳng mấy thú vị. Nói chung, siêu dữ liệu như tác giả, ngày sửa đổi gần nhất, mã SPR, v.v. không nên xuất hiện trong chú thích. Chú thích nên dành cho các ghi chú kỹ thuật về mã và thiết kế.

C2: Chú thích lỗi thời

Một chú thích đã cũ, không còn liên quan và sai lệch là chú thích lỗi thời. Chú thích nhanh chóng trở nên cũ. Tốt nhất đừng viết chú thích sẽ lỗi thời. Nếu phát hiện chú thích lỗi thời, hãy cập nhật hoặc xóa nó càng sớm càng tốt. Chú thích lỗi thời thường tách rời khỏi đoạn mã mà ban đầu nó mô tả, rồi biến thành những hòn đảo lơ lửng, chẳng còn liên quan và chỉ dẫn sai trong mã.

C3: Chú thích thừa

Một chú thích là thừa nếu nó mô tả điều vốn đã tự nói lên đầy đủ. Ví dụ:

```
i++; // tăng i
```

Một ví dụ khác là Javadoc không nói gì hơn (thậm chí còn ít hơn) chữ ký hàm:

```
/**
 * @param sellRequest
 * @return
 * @throws ManagedComponentException
 */
public SellResponse beginSellItem(SellRequest sellRequest)
  throws ManagedComponentException
```

Chú thích nên nói những điều mà bản thân mã không thể nói.

C4: Chú thích viết kém

Chú thích đáng để viết thì cũng đáng được viết cho tốt. Nếu định viết chú thích, hãy dành thời gian để viết câu tốt nhất có thể. Chọn từ cẩn thận. Dùng ngữ pháp và dấu câu chính xác. Đừng lan man. Đừng nói điều hiển nhiên. Hãy ngắn gọn.

C5: Mã bị chú thích vô hiệu hóa

Tôi phát điên khi thấy những đoạn mã bị chú thích vô hiệu hóa. Ai biết chúng đã cũ đến mức nào? Ai biết chúng còn ý nghĩa hay không? Vậy mà chẳng ai xóa, vì ai cũng cho rằng người khác cần đến chúng hoặc đang có kế hoạch dùng chúng.

Đoạn mã ấy cứ nằm đó mục ruỗng, ngày qua ngày càng ít liên quan hơn. Nó gọi những hàm không còn tồn tại. Nó dùng những biến đã đổi tên. Nó tuân theo những quy ước lỗi thời từ lâu. Nó làm ô nhiễm các mô-đun chứa nó và khiến người đọc mất tập trung. Mã bị chú thích vô hiệu hóa là một tai họa.

Thấy mã bị chú thích vô hiệu hóa thì xóa đi! Đừng lo, hệ thống quản lý mã nguồn vẫn lưu nó. Nếu ai đó thật sự cần, họ có thể quay lại lấy phiên bản cũ. Đừng để mã bị chú thích vô hiệu hóa sống sót.

Môi trường

E1: Build cần nhiều hơn một bước

Build một dự án phải là thao tác đơn giản, chỉ cần một bước. Bạn không nên phải lấy nhiều mảnh nhỏ từ hệ thống quản lý mã nguồn. Bạn cũng không nên cần một chuỗi lệnh bí hiểm hay các script phụ thuộc vào ngữ cảnh để build từng thành phần. Bạn không nên phải tìm khắp nơi những JAR, tệp XML và tạo tác phụ nhỏ mà hệ thống cần. Bạn cần có thể lấy mã hệ thống bằng một lệnh đơn giản, rồi chạy một lệnh đơn giản khác để build nó.

```
svn get mySystem
cd mySystem
ant all
```

E2: Kiểm thử cần nhiều hơn một bước

Bạn cần có thể chạy toàn bộ kiểm thử đơn vị chỉ bằng một lệnh. Tốt nhất là chạy tất cả kiểm thử bằng cách bấm một nút trong IDE. Tệ nhất thì bạn cũng phải chạy được chúng bằng một lệnh đơn giản trong shell. Chạy được toàn bộ kiểm thử là điều cơ bản và quan trọng đến mức thao tác này phải nhanh, dễ và hiển nhiên.

Hàm

F1: Quá nhiều đối số

Hàm nên có ít đối số. Tốt nhất là không có đối số, kế đến là một, hai rồi ba. Nhiều hơn ba là rất đáng ngờ và nên kiên quyết tránh. (Xem “Đối số của hàm” ở trang 40.)

F2: Đối số đầu ra

Đối số đầu ra trái với trực giác. Người đọc mong đợi đối số là đầu vào, không phải đầu ra. Nếu hàm phải thay đổi trạng thái của thứ gì đó, hãy để nó thay đổi trạng thái của đối tượng mà nó được gọi trên đó. (Xem “Đối số đầu ra” ở trang 45.)

F3: Đối số cờ

Đối số Boolean nói rõ rằng hàm làm nhiều hơn một việc. Chúng gây khó hiểu và nên bị loại bỏ. (Xem “Đối số cờ” ở trang 41.)

F4: Hàm chết

Các phương thức không bao giờ được gọi nên bị loại bỏ. Giữ mã chết chỉ gây lãng phí. Đừng ngại xóa hàm. Hãy nhớ hệ thống quản lý mã nguồn vẫn lưu nó.

Tổng quát

G1: Nhiều ngôn ngữ trong cùng một tệp mã nguồn

Các môi trường lập trình hiện đại ngày nay cho phép đặt nhiều ngôn ngữ khác nhau vào cùng một tệp mã nguồn. Chẳng hạn, một tệp Java có thể chứa các đoạn XML, HTML, YAML, JavaDoc, tiếng Anh, JavaScript, v.v. Một ví dụ khác, ngoài HTML, tệp JSP có thể chứa Java, cú pháp thư viện thẻ, chú thích tiếng Anh, Javadoc, XML, JavaScript, v.v. Trường hợp tốt nhất thì việc này gây khó hiểu, tệ nhất là cẩu thả.

Lý tưởng nhất là một tệp mã nguồn chỉ chứa đúng một ngôn ngữ. Trên thực tế, có lẽ ta sẽ phải dùng nhiều hơn một. Nhưng nên cố gắng giảm cả số lượng lẫn phạm vi của các ngôn ngữ bổ sung trong tệp mã nguồn.

G2: Hành vi hiển nhiên chưa được triển khai

Theo “Nguyên tắc ít gây ngạc nhiên nhất”,[2] mọi hàm hay lớp nên triển khai những hành vi mà một lập trình viên khác có thể hợp lý mong đợi. Ví dụ, hãy xét một hàm chuyển tên

2. Còn gọi là “Nguyên tắc ít gây kinh ngạc nhất”: `http://en.wikipedia.org/wiki/Principle_of_least_astonishment`

ngày trong tuần thành enum biểu diễn ngày đó.

```
Day day = DayDate.StringToDay(String dayName);
```

Ta mong chuỗi `"Monday"` được chuyển thành `Day.MONDAY`. Ta cũng mong các dạng viết tắt thông dụng được chuyển đổi và hàm không phân biệt chữ hoa chữ thường.

Khi một hành vi hiển nhiên không được triển khai, người đọc và người dùng mã không còn có thể dựa vào trực giác về tên hàm. Họ mất lòng tin vào tác giả ban đầu và phải quay lại đọc chi tiết mã.

G3: Hành vi sai ở các ranh giới

Nói rằng mã phải hoạt động chính xác nghe có vẻ hiển nhiên. Vấn đề là chúng ta hiếm khi nhận ra hành vi chính xác phức tạp đến mức nào. Lập trình viên thường viết hàm mà họ nghĩ sẽ chạy đúng, rồi tin vào trực giác thay vì bỏ công chứng minh mã hoạt động trong mọi trường hợp góc và ranh giới.

Không gì thay thế được sự cẩn trọng cần thiết. Mỗi điều kiện biên, mỗi trường hợp góc, mỗi tình huống bất thường và ngoại lệ đều có thể phá hỏng một thuật toán tưởng như tao nhã, hợp lý. Đừng dựa vào trực giác. Hãy tìm từng điều kiện biên và viết kiểm thử cho nó.

G4: Vô hiệu hóa các cơ chế an toàn

Thảm họa Chernobyl xảy ra vì người quản lý nhà máy lần lượt vô hiệu hóa từng cơ chế an toàn. Các cơ chế đó gây bất tiện khi tiến hành một thí nghiệm. Kết quả là thí nghiệm không được thực hiện, và thế giới chứng kiến thảm họa hạt nhân dân sự lớn đầu tiên.

Vô hiệu hóa cơ chế an toàn rất rủi ro. Có thể cần kiểm soát thủ công `serialVersionUID`, nhưng việc đó luôn nguy hiểm. Tắt một số cảnh báo của trình biên dịch (hoặc tắt hết!) có thể giúp build thành công, nhưng đổi lại là nguy cơ phải ngồi gỡ lỗi triền miên. Tắt các kiểm thử đang thất bại rồi tự nhủ sẽ làm chúng chạy qua sau cũng tệ như nghĩ thẻ tín dụng là tiền miễn phí.

G5: Trùng lặp

Đây là một trong những quy tắc quan trọng nhất trong sách, bạn nên xem nó thật nghiêm túc. Hầu như tác giả nào viết về thiết kế phần mềm cũng nhắc đến quy tắc này. Dave Thomas và Andy Hunt gọi đó là nguyên tắc DRY[3] (Đừng lặp lại chính mình). Kent Beck đưa nó thành một trong những nguyên tắc cốt lõi của Extreme Programming và gọi là: “Một lần, và chỉ một lần.” Ron Jeffries xếp quy tắc này ở vị trí thứ hai, chỉ sau việc làm cho mọi kiểm thử đều chạy qua.

Mỗi lần thấy mã bị trùng lặp, đó là cơ hội trừu tượng hóa đã bị bỏ lỡ. Phần mã lặp có lẽ có thể trở thành một hàm con hoặc thậm chí một lớp riêng. Gom phần trùng lặp vào một

3. [PRAG].

cấu trúc trừu tượng như vậy sẽ làm giàu vốn từ vựng trong thiết kế của bạn. Các lập trình viên khác có thể dùng những cấu trúc trừu tượng bạn tạo ra. Việc viết mã sẽ nhanh hơn và ít lỗi hơn vì bạn đã nâng mức trừu tượng.

Dạng trùng lặp dễ thấy nhất là những khối mã giống hệt nhau, trông như thể ai đó rê chuột rồi dán cùng một đoạn mã hết lần này đến lần khác. Nên thay chúng bằng các phương thức đơn giản.

Dạng kín đáo hơn là chuỗi switch/case hoặc if/else xuất hiện lặp đi lặp lại ở nhiều mô-đun, lần nào cũng kiểm tra cùng một tập điều kiện. Nên thay chúng bằng đa hình.

Kín đáo hơn nữa là những mô-đun có thuật toán tương tự nhau nhưng không dùng chung các dòng mã giống nhau. Đó vẫn là trùng lặp và cần được xử lý bằng mẫu TEMPLATE METHOD[4] hoặc STRATEGY.[5]

Thật vậy, hầu hết mẫu thiết kế xuất hiện trong mười lăm năm qua chỉ là những cách quen thuộc để loại bỏ trùng lặp. Các dạng chuẩn Codd cũng là một chiến lược loại bỏ trùng lặp trong lược đồ cơ sở dữ liệu. Bản thân lập trình hướng đối tượng cũng là một chiến lược tổ chức mô-đun và loại bỏ trùng lặp. Lập trình cấu trúc cũng vậy, chẳng có gì đáng ngạc nhiên.

Tôi nghĩ đã nói đủ. Hãy tìm và loại bỏ trùng lặp ở bất cứ đâu có thể.

G6: Mã ở sai mức trừu tượng

Điều quan trọng là tạo ra các cấu trúc trừu tượng để tách những khái niệm tổng quát ở mức cao khỏi các chi tiết ở mức thấp. Đôi khi ta làm vậy bằng cách tạo lớp trừu tượng chứa các khái niệm mức cao và các lớp dẫn xuất chứa khái niệm mức thấp. Khi đó, cần bảo đảm sự phân tách hoàn chỉnh: mọi khái niệm mức thấp nằm trong các lớp dẫn xuất, còn mọi khái niệm mức cao nằm trong lớp cơ sở.

Chẳng hạn, các hằng, biến hay hàm tiện ích chỉ liên quan đến phần triển khai chi tiết không nên có mặt trong lớp cơ sở. Lớp cơ sở không nên biết gì về chúng.

Quy tắc này cũng áp dụng cho tệp mã nguồn, thành phần và mô-đun. Thiết kế phần mềm tốt đòi hỏi ta tách các khái niệm ở những mức khác nhau và đặt chúng vào những nơi chứa khác nhau. Những nơi chứa đó đôi khi là lớp cơ sở hay lớp dẫn xuất, đôi khi là tệp mã nguồn, mô-đun hoặc thành phần. Dù dùng loại nào, sự phân tách phải hoàn chỉnh. Ta không muốn trộn lẫn khái niệm mức thấp với mức cao.

Hãy xem đoạn mã sau:

```
public interface Stack {
  Object pop() throws EmptyException;
  void push(Object o) throws FullException;
  double percentFull();

  class EmptyException extends Exception {}
  class FullException extends Exception {}
}
```

4. [GOF].

5. [GOF].

Hàm `percentFull` nằm sai mức trừu tượng. Nhiều cách triển khai Stack có thể xem khái niệm độ đầy là hợp lý, nhưng cũng có những cách triển khai không thể biết mình đầy đến đâu. Vì vậy, nên đặt hàm này trong một giao diện dẫn xuất như `BoundedStack`.

Có thể bạn nghĩ rằng khi ngăn xếp không giới hạn, cách triển khai chỉ cần trả về 0. Vấn đề là không có ngăn xếp nào thực sự không giới hạn. Kiểm tra

```
stack.percentFull() < 50.0
```

cũng chẳng thể ngăn được `OutOfMemoryException`. Cài đặt hàm để nó trả về 0 tức là nói dối.

Bạn không thể nói dối hay giả vờ để chữa một cấu trúc trừu tượng đặt sai chỗ. Tách biệt các cấu trúc trừu tượng là một trong những việc khó nhất mà nhà phát triển phần mềm phải làm, và khi làm sai thì chẳng có cách sửa nhanh nào.

G7: Lớp cơ sở phụ thuộc vào lớp dẫn xuất

Lý do phổ biến nhất để chia các khái niệm thành lớp cơ sở và lớp dẫn xuất là giúp các khái niệm mức cao trong lớp cơ sở độc lập với các khái niệm mức thấp trong lớp dẫn xuất. Vì thế, khi thấy lớp cơ sở nhắc đến tên các lớp dẫn xuất, ta nên nghi ngờ có vấn đề. Nói chung, lớp cơ sở không nên biết gì về các lớp dẫn xuất.

Dĩ nhiên, quy tắc này có ngoại lệ. Đôi khi số lớp dẫn xuất được cố định chặt chẽ, và lớp cơ sở có mã để chọn giữa chúng. Ta thường thấy điều này trong các cách triển khai máy trạng thái hữu hạn. Tuy nhiên, trong trường hợp ấy, lớp dẫn xuất và lớp cơ sở phụ thuộc chặt chẽ vào nhau, đồng thời luôn được triển khai cùng nhau trong một tệp JAR. Nói chung, ta muốn có thể triển khai lớp dẫn xuất và lớp cơ sở trong các tệp JAR riêng.

Việc triển khai lớp dẫn xuất và lớp cơ sở trong các tệp JAR riêng, đồng thời bảo đảm các tệp JAR chứa lớp cơ sở không biết gì về nội dung của các tệp JAR chứa lớp dẫn xuất, cho phép ta triển khai hệ thống thành những thành phần riêng biệt, độc lập. Khi sửa đổi các thành phần đó, ta có thể triển khai lại chúng mà không phải triển khai lại các thành phần cơ sở. Nhờ vậy, tác động của thay đổi giảm đáng kể và việc bảo trì hệ thống đang vận hành cũng đơn giản hơn nhiều.

G8: Quá nhiều thông tin

Mô-đun được định nghĩa tốt có giao diện rất nhỏ, cho phép làm được nhiều việc với ít thao tác. Mô-đun được định nghĩa kém có giao diện rộng và phức tạp, buộc bạn phải dùng nhiều thao tác khác nhau để làm những việc đơn giản. Giao diện được định nghĩa tốt không cung cấp nhiều hàm để phụ thuộc vào, nên sự phụ thuộc thấp. Giao diện được định nghĩa kém có nhiều hàm bạn phải gọi, nên sự phụ thuộc cao.

Nhà phát triển phần mềm giỏi học cách giới hạn những gì họ để lộ qua giao diện của lớp và mô-đun. Một lớp càng có ít phương thức càng tốt. Một hàm càng biết ít biến càng tốt. Một lớp càng có ít biến thực thể càng tốt.

Hãy ẩn dữ liệu. Ẩn các hàm tiện ích. Ẩn các hằng và biến tạm. Đừng tạo lớp có quá nhiều phương thức hay biến thực thể. Đừng tạo nhiều biến và hàm `protected` cho các lớp con. Hãy tập trung giữ giao diện thật gọn và nhỏ. Giới hạn thông tin để giúp giảm sự phụ thuộc.

G9: Mã chết

Mã chết là mã không được thực thi. Bạn có thể thấy nó trong thân một câu lệnh if kiểm tra điều kiện không thể xảy ra. Bạn cũng có thể thấy nó trong khối catch của một try không bao giờ ném ngoại lệ, trong những hàm tiện ích nhỏ không bao giờ được gọi, hoặc trong các nhánh switch/case không bao giờ xảy ra.

Vấn đề là mã chết bắt đầu có mùi sau một thời gian. Càng cũ, mùi càng nồng và chua. Đó là vì khi thiết kế thay đổi, mã chết không được cập nhật đầy đủ. Nó vẫn biên dịch được nhưng không theo những quy ước hay quy tắc mới hơn. Nó được viết khi hệ thống còn khác. Khi tìm thấy mã chết, hãy làm điều đúng đắn: chôn cất tử tế. Xóa nó khỏi hệ thống.

G10: Tách biệt theo chiều dọc

Biến và hàm nên được định nghĩa gần nơi chúng được dùng. Biến cục bộ nên được khai báo ngay trước lần sử dụng đầu tiên và có phạm vi theo chiều dọc ngắn. Ta không muốn biến cục bộ được khai báo cách chỗ sử dụng hàng trăm dòng.

Hàm riêng nên được định nghĩa ngay dưới chỗ sử dụng đầu tiên. Hàm riêng thuộc phạm vi của cả lớp, nhưng ta vẫn muốn giới hạn khoảng cách theo chiều dọc giữa chỗ gọi và chỗ định nghĩa. Tìm một hàm riêng chỉ nên cần rà xuống dưới từ lần sử dụng đầu tiên.

G11: Không nhất quán

Nếu làm một việc theo cách nào đó, hãy làm mọi việc tương tự theo cùng cách. Điều này bắt nguồn từ nguyên tắc ít gây ngạc nhiên nhất. Hãy cẩn thận khi chọn quy ước và một khi đã chọn thì cần tiếp tục tuân theo.

Nếu trong một hàm bạn dùng biến tên `response` để chứa một `HttpServletResponse`, hãy dùng nhất quán cùng tên biến trong những hàm khác có dùng đối tượng `HttpServletResponse`. Nếu bạn đặt tên phương thức là `processVerificationRequest`, hãy dùng tên tương tự như `processDeletionRequest` cho các phương thức xử lý những loại yêu cầu khác.

Sự nhất quán đơn giản như vậy, nếu được áp dụng đều đặn, có thể giúp mã dễ đọc và sửa đổi hơn nhiều.

G12: Rác

Một hàm khởi tạo mặc định không có phần triển khai thì có ích gì? Nó chỉ khiến mã thêm lộn xộn với những thứ vô nghĩa. Biến không được dùng, hàm không bao giờ được gọi, chú thích không cung cấp thông tin, v.v. Tất cả đều là rác và nên bị loại bỏ. Hãy giữ tệp mã nguồn sạch sẽ, ngăn nắp, không có rác.

G13: Tạo sự phụ thuộc nhân tạo

Những thứ không phụ thuộc vào nhau thì không nên bị buộc phụ thuộc một cách nhân tạo. Chẳng hạn, không nên đặt enum tổng quát bên trong các lớp cụ thể hơn, vì như thế cả ứng dụng phải biết đến những lớp cụ thể ấy. Điều tương tự cũng đúng với các hàm tĩnh đa dụng được khai báo trong lớp chuyên biệt.

Nói chung, sự phụ thuộc nhân tạo là sự phụ thuộc giữa hai mô-đun không phục vụ mục đích trực tiếp nào. Nó xuất hiện khi ta đặt biến, hằng hay hàm vào một vị trí tạm thời tiện lợi nhưng không phù hợp. Đó là lười biếng và cẩu thả.

Hãy dành thời gian xác định nên khai báo hàm, hằng và biến ở đâu. Đừng tiện tay ném chúng vào chỗ thuận tiện nhất rồi để nguyên ở đó.

G14: Hàm mê dữ liệu của lớp khác

Đây là một trong những mùi code của Martin Fowler.[6] Các phương thức của một lớp nên quan tâm đến biến và hàm của chính lớp đó, chứ không phải biến và hàm của lớp khác. Khi một phương thức dùng các hàm truy cập và hàm sửa đổi của một đối tượng khác để thao tác dữ liệu bên trong đối tượng đó, nó đang thèm muốn phạm vi của lớp chứa đối tượng kia. Nó ước mình nằm trong lớp kia để có thể truy cập trực tiếp các biến mà nó đang thao tác. Ví dụ:

---

6. [Refactoring].

```
public class HourlyPayCalculator {
  public Money calculateWeeklyPay(HourlyEmployee e) {
    int tenthRate = e.getTenthRate().getPennies();
    int tenthsWorked = e.getTenthsWorked();
    int straightTime = Math.min(400, tenthsWorked);
    int overTime = Math.max(0, tenthsWorked - straightTime);
    int straightPay = straightTime * tenthRate;
    int overtimePay = (int)Math.round(overTime*tenthRate*1.5);
    return new Money(straightPay + overtimePay);
  }
}
```

Phương thức `calculateWeeklyPay` truy cập sâu vào đối tượng `HourlyEmployee` để lấy dữ liệu mà nó xử lý. Phương thức này thèm muốn phạm vi của `HourlyEmployee`. Nó "ước" mình có thể nằm bên trong `HourlyEmployee`.

Nếu mọi yếu tố khác như nhau, ta muốn loại bỏ kiểu mê dữ liệu này vì nó để lộ phần bên trong của một lớp cho lớp khác. Tuy vậy, đôi khi đây là điều bất đắc dĩ. Hãy xét ví dụ sau:

```
public class HourlyEmployeeReport {
  private HourlyEmployee employee;

  public HourlyEmployeeReport(HourlyEmployee e) {
    this.employee = e;
  }

  String reportHours() {
    return String.format(
      "Name: %s\tHours:%d.%1d\n",
      employee.getName(),
      employee.getTenthsWorked()/10,
      employee.getTenthsWorked()%10);
  }
}
```

Rõ ràng phương thức `reportHours` thèm muốn lớp `HourlyEmployee`. Mặt khác, ta không muốn `HourlyEmployee` phải biết định dạng báo cáo. Chuyển chuỗi định dạng vào lớp `HourlyEmployee` sẽ vi phạm một số nguyên tắc thiết kế hướng đối tượng.[7] Làm vậy sẽ khiến `HourlyEmployee` phụ thuộc vào định dạng báo cáo, làm lớp này chịu ảnh hưởng mỗi khi định dạng thay đổi.

G15: Đối số chọn hành vi

Hầu như chẳng có gì tệ hơn một đối số `false` lủng lẳng ở cuối lời gọi hàm. Nó có nghĩa gì? Nếu là `true` thì điều gì sẽ thay đổi? Mục đích của đối số chọn hành vi vốn đã khó nhớ, và mỗi đối số như vậy lại gộp nhiều hàm thành một. Đây chỉ là cách lười biếng để khỏi chia một hàm lớn thành vài hàm nhỏ. Hãy xem:

---

7. Cụ thể là Nguyên tắc trách nhiệm đơn lẻ, Nguyên tắc đóng mở và Nguyên tắc đóng chung. Xem [PPP].

```
public int calculateWeeklyPay(boolean overtime) {
  int tenthRate = getTenthRate();
  int tenthsWorked = getTenthsWorked();
  int straightTime = Math.min(400, tenthsWorked);
  int overTime = Math.max(0, tenthsWorked - straightTime);
  int straightPay = straightTime * tenthRate;
  double overtimeRate = overtime ? 1.5 : 1.0 * tenthRate;
  int overtimePay = (int)Math.round(overTime*overtimeRate);
  return straightPay + overtimePay;
}
```

Bạn gọi hàm này với `true` nếu giờ làm thêm được trả gấp rưỡi, và với `false` nếu giờ làm thêm được trả theo mức thông thường. Mỗi khi tình cờ gặp `calculateWeeklyPay(false)`, bạn đã phải nhớ xem lời gọi ấy có nghĩa gì. Nhưng điều đáng tiếc thật sự ở một hàm như thế này là tác giả đã bỏ lỡ cơ hội viết:

```
public int straightPay() {
  return getTenthsWorked() * getTenthRate();
}

public int overTimePay() {
  int overTimeTenths = Math.max(0, getTenthsWorked() - 400);
  int overTimePay = overTimeBonus(overTimeTenths);
  return straightPay() + overTimePay;
}

private int overTimeBonus(int overTimeTenths) {
  double bonus = 0.5 * getTenthRate() * overTimeTenths;
  return (int) Math.round(bonus);
}
```

Đối số chọn hành vi không nhất thiết phải là Boolean. Nó có thể là enum, số nguyên hoặc bất kỳ kiểu đối số nào dùng để chọn hành vi của hàm. Nói chung, có nhiều hàm vẫn tốt hơn là truyền một thứ gì đó vào hàm để chọn hành vi.

G16: Ý định bị che khuất

Ta muốn mã biểu đạt rõ nhất có thể. Biểu thức nối dài, ký pháp Hungary và số ma thuật đều che khuất ý định của tác giả. Ví dụ, hàm `overTimePay` có thể được viết như sau:

```
public int m_otCalc() {
  return iThsWkd * iThsRte +
    (int) Math.round(0.5 * iThsRte *
      Math.max(0, iThsWkd - 400)
    );
}
```

Đoạn mã có vẻ ngắn gọn, nhưng gần như không thể hiểu nổi. Hãy dành thời gian để làm rõ ý định trong mã cho người đọc.

G17: Trách nhiệm đặt sai chỗ

Một trong những quyết định quan trọng nhất mà nhà phát triển phần mềm phải đưa ra là đặt mã ở đâu. Chẳng hạn, hằng PI nên nằm ở đâu? Trong lớp `Math` chăng? Có lẽ nó thuộc lớp `Trigonometry`? Hay có thể là lớp `Circle`?

Ở đây cần áp dụng nguyên tắc ít gây ngạc nhiên nhất. Mã nên được đặt ở nơi người đọc tự nhiên mong đợi. Hằng PI nên nằm ở nơi khai báo các hàm lượng giác. Hằng `OVERTIME_RATE` nên được khai báo trong lớp `HourlyPayCalculator`.

Đôi khi ta tỏ ra "khôn lỏi" khi quyết định đặt một chức năng ở đâu. Ta đặt nó vào một hàm tiện cho mình nhưng chưa chắc hợp với trực giác của người đọc. Ví dụ, có thể ta cần in báo cáo tổng số giờ một nhân viên đã làm. Ta có thể cộng số giờ trong đoạn mã in báo cáo, hoặc duy trì một tổng lũy kế trong phần mã tiếp nhận thẻ chấm công.

Một cách đưa ra quyết định này là xem tên hàm. Giả sử mô-đun báo cáo có hàm tên `getTotalHours`. Giả sử mô-đun tiếp nhận thẻ chấm công có hàm `saveTimeCard`. Nhìn vào tên, hàm nào ngụ ý rằng nó tính tổng số giờ? Câu trả lời hẳn rất rõ.

Dĩ nhiên, đôi khi có lý do về hiệu năng để tính tổng số giờ khi tiếp nhận thẻ chấm công thay vì lúc in báo cáo. Không sao, nhưng tên hàm cần thể hiện điều đó. Ví dụ, trong mô-đun thẻ chấm công nên có hàm `computeRunningTotalOfHours`.

G18: Dùng static không phù hợp

`Math.max(double a, double b)` là một phương thức tĩnh tốt. Nó không thao tác trên một thực thể cụ thể nào; quả thật, sẽ thật ngớ ngẩn nếu phải viết `new Math().max(a,b)` hay thậm chí `a.max(b)`. Toàn bộ dữ liệu `max` sử dụng đều đến từ hai đối số, không phải từ một đối tượng "sở hữu" nào. Quan trọng hơn, gần như không có khả năng ta muốn `Math.max` hỗ trợ đa hình.

Tuy nhiên, đôi khi ta viết những hàm tĩnh lẽ ra không nên là hàm tĩnh. Ví dụ, hãy xét:

```
HourlyPayCalculator.calculatePay(employee, overtimeRate).
```

Hàm tĩnh này có vẻ hợp lý. Nó không hoạt động trên một đối tượng cụ thể nào và nhận toàn bộ dữ liệu từ các đối số. Tuy vậy, khả năng ta muốn hàm này hỗ trợ đa hình là có thật. Chẳng hạn, ta có thể muốn triển khai nhiều thuật toán tính lương theo giờ, như `OvertimeHourlyPayCalculator` và `StraightTimeHourlyPayCalculator`. Trong trường hợp này, hàm không nên là hàm tĩnh mà nên là một phương thức thành viên không tĩnh của `Employee`.

Nói chung, nên ưu tiên phương thức không tĩnh hơn phương thức tĩnh. Khi chưa chắc, hãy để hàm là phương thức không tĩnh. Nếu thật sự muốn hàm tĩnh, hãy chắc chắn rằng không có khả năng bạn sẽ muốn nó hoạt động đa hình.

G19: Dùng biến diễn giải

Kent Beck đã viết về điều này trong cuốn sách tuyệt vời *Smalltalk Best Practice Patterns*[8] và gần đây hơn trong cuốn cũng tuyệt vời không kém *Implementation Patterns*.[9] Một trong những cách hiệu quả để làm chương trình dễ đọc là chia phép tính thành các giá trị trung gian, lưu trong những biến có tên rõ nghĩa.

Hãy xem ví dụ sau từ FitNesse:

```
Matcher match = headerPattern.matcher(line);
if(match.find())
{
  String key = match.group(1);
  String value = match.group(2);
  headers.put(key.toLowerCase(), value);
}
```

Chỉ cần dùng các biến diễn giải đơn giản, ta đã thấy rõ nhóm khớp thứ nhất là khóa, nhóm thứ hai là giá trị.

Rất khó dùng quá tay cách này. Thường thì có nhiều biến diễn giải vẫn tốt hơn ít biến. Thật đáng kinh ngạc khi một mô-đun khó hiểu bỗng trở nên trong sáng chỉ bằng cách chia phép tính thành những giá trị trung gian có tên rõ ràng.

G20: Tên hàm phải nói rõ hàm làm gì

Hãy xem đoạn mã này:

```
Date newDate = date.add(5);
```

Bạn nghĩ hàm này cộng năm ngày vào ngày đó chăng? Hay là tuần, hay giờ? Đối tượng ngày có bị thay đổi không, hay hàm chỉ trả về một `Date` mới mà không đổi đối tượng cũ? Chỉ nhìn lời gọi thì không thể biết hàm làm gì.

Nếu hàm cộng năm ngày vào ngày và làm thay đổi ngày đó, hãy đặt tên là `addDaysTo` hoặc `increaseByDays`. Còn nếu hàm trả về một ngày mới, muộn hơn năm ngày nhưng không thay đổi thực thể ngày, hãy đặt tên là `daysLater` hoặc `daysSince`.

Nếu phải xem phần triển khai (hoặc tài liệu) của hàm mới biết nó làm gì, bạn nên tìm tên tốt hơn hoặc sắp xếp lại chức năng để có thể đặt nó vào những hàm mang tên rõ ràng hơn.

G21: Hiểu thuật toán

Nhiều đoạn mã rất buồn cười được viết ra vì người ta không dành thời gian hiểu thuật toán. Họ khiến thứ gì đó chạy được bằng cách chèn đủ câu lệnh if và cờ, mà không thật sự dừng lại suy nghĩ xem chuyện gì đang diễn ra.

---

8. [Beck97], trang 108.

9. [Beck07].

Lập trình thường là một quá trình khám phá. Bạn nghĩ mình biết thuật toán phù hợp cho một việc, nhưng rồi phải vọc vạch, thăm dò, thử tới thử lui cho đến khi khiến nó "chạy được". Làm sao biết nó "chạy được"? Vì nó vượt qua những ca kiểm thử mà bạn nghĩ ra.

Cách tiếp cận này chẳng có gì sai. Quả thật, thường đó là cách duy nhất để khiến hàm làm được điều bạn mong muốn. Tuy nhiên, không thể cứ giữ mãi dấu ngoặc kép quanh chữ "chạy được".

Trước khi cho rằng mình đã hoàn thành một hàm, hãy chắc rằng bạn hiểu nó hoạt động ra sao. Việc mọi kiểm thử đều chạy qua chưa đủ. Bạn phải biết[10] lời giải là đúng.

Thường thì cách tốt nhất để có được hiểu biết ấy là tái cấu trúc hàm thành thứ sạch sẽ, biểu đạt rõ đến mức cách nó hoạt động trở nên hiển nhiên.

G22: Biến sự phụ thuộc logic thành phụ thuộc cụ thể

Nếu một mô-đun phụ thuộc vào mô-đun khác, sự phụ thuộc đó phải được thể hiện cụ thể chứ không chỉ tồn tại về mặt logic. Mô-đun phụ thuộc không nên tự giả định (nói cách khác, tạo ra sự phụ thuộc logic) về mô-đun mà nó dựa vào. Thay vào đó, nó phải hỏi tường minh mô-đun ấy mọi thông tin mà nó cần.

Ví dụ, hãy tưởng tượng bạn đang viết một hàm in báo cáo văn bản thuần về số giờ nhân viên đã làm. Một lớp tên `HourlyReporter` thu thập toàn bộ dữ liệu thành dạng tiện dụng rồi chuyển cho `HourlyReportFormatter` để in. (Xem Đoạn mã 17-1.)

Đoạn mã 17-1
HourlyReporter.java

```
public class HourlyReporter {
  private HourlyReportFormatter formatter;
  private List<LineItem> page;
  private final int PAGE_SIZE = 55;

  public HourlyReporter(HourlyReportFormatter formatter) {
    this.formatter = formatter;
    page = new ArrayList<LineItem>();
  }

  public void generateReport(List<HourlyEmployee> employees) {
    for (HourlyEmployee e : employees) {
      addLineItemToPage(e);
      if (page.size() == PAGE_SIZE)
        printAndClearItemList();
    }
    if (page.size() > 0)
      printAndClearItemList();
  }

  private void printAndClearItemList() {
```

10. Biết cách mã hoạt động khác với biết liệu thuật toán có thực hiện được công việc cần thiết hay không. Không chắc thuật toán có phù hợp hay không thường là chuyện bình thường. Không biết mã của mình làm gì thì chỉ là lười biếng.

```
      formatter.format(page);
      page.clear();
    }

    private void addLineItemToPage(HourlyEmployee e) {
      LineItem item = new LineItem();
      item.name = e.getName();
      item.hours = e.getTenthsWorked() / 10;
```

Đoạn mã 17-1 (tiếp theo)
HourlyReporter.java

```
      item.tenths = e.getTenthsWorked() % 10;
      page.add(item);
    }

    public class LineItem {
      public String name;
      public int hours;
      public int tenths;
    }
  }
```

Trong mã này có một sự phụ thuộc logic chưa được thể hiện cụ thể. Bạn thấy đó là gì không? Đó là hằng `PAGE_SIZE`. Vì sao `HourlyReporter` phải biết kích thước trang? Kích thước trang nên thuộc trách nhiệm của `HourlyReportFormatter`.

Việc khai báo `PAGE_SIZE` trong `HourlyReporter` thể hiện trách nhiệm đặt sai chỗ [G17], khiến `HourlyReporter` cho rằng nó biết kích thước trang nên là bao nhiêu. Giả định ấy tạo ra sự phụ thuộc logic. `HourlyReporter` phụ thuộc vào việc `HourlyReportFormatter` có thể xử lý các trang cỡ 55. Nếu một cách triển khai nào đó của `HourlyReportFormatter` không xử lý được kích thước như vậy, lỗi sẽ xảy ra.

Ta có thể thể hiện cụ thể sự phụ thuộc này bằng cách tạo một phương thức mới tên `getMaxPageSize()` trong `HourlyReportFormatter`. Sau đó, `HourlyReporter` sẽ gọi hàm này thay vì dùng hằng `PAGE_SIZE`.

G23: Ưu tiên đa hình thay cho if/else hoặc switch/case

Gợi ý này có thể nghe lạ nếu xét chủ đề của Chương 6. Dù sao, trong chương đó tôi lập luận rằng câu lệnh switch có lẽ phù hợp ở những phần của hệ thống có khả năng cần thêm hàm cao hơn khả năng cần thêm kiểu mới.

Thứ nhất, đa số mọi người dùng câu lệnh switch vì đó là giải pháp vét cạn hiển nhiên, chứ không phải vì nó phù hợp với tình huống. Vì thế, nguyên tắc kinh nghiệm này nhắc ta cân nhắc đa hình trước khi dùng switch.

Thứ hai, những trường hợp hàm thay đổi thường xuyên hơn kiểu dữ liệu tương đối hiếm. Vì vậy, mọi câu lệnh switch đều đáng ngờ.

Tôi dùng quy tắc "MỘT SWITCH" như sau: Với mỗi kiểu lựa chọn, không được có quá một câu lệnh switch. Các nhánh trong câu lệnh switch ấy phải tạo ra những đối tượng đa hình thay thế cho các câu lệnh switch khác trong phần còn lại của hệ thống.

G24: Tuân theo quy ước chuẩn

Mỗi nhóm nên tuân theo một chuẩn viết mã dựa trên những thông lệ chung của ngành. Chuẩn này cần quy định những điều như khai báo biến thực thể ở đâu, đặt tên lớp, phương thức và biến thế nào, đặt dấu ngoặc nhọn ở đâu, v.v. Nhóm không cần tài liệu mô tả những quy ước ấy, vì chính mã của họ đã nêu ví dụ.

Mọi người trong nhóm đều nên tuân theo các quy ước này. Điều đó có nghĩa mỗi thành viên phải đủ chín chắn để hiểu rằng đặt dấu ngoặc nhọn ở đâu chẳng quan trọng chút nào, miễn là cả nhóm thống nhất vị trí đặt chúng.

Nếu muốn biết tôi tuân theo quy ước nào, bạn có thể xem mã đã tái cấu trúc trong Đoạn mã B-7 ở trang 394 đến Đoạn mã B-14.

G25: Thay số ma thuật bằng hằng có tên

Đây có lẽ là một trong những quy tắc lâu đời nhất trong phát triển phần mềm. Tôi nhớ đã đọc về nó từ cuối thập niên sáu mươi trong các tài liệu nhập môn COBOL, FORTRAN và PL/1. Nói chung, để số trần trong mã là một ý tưởng tồi. Bạn nên giấu chúng sau các hằng có tên rõ nghĩa.

Ví dụ, số 86.400 nên được giấu sau hằng `SECONDS_PER_DAY`. Nếu mỗi trang in 55 dòng, hằng 55 nên được giấu sau hằng `LINES_PER_PAGE`.

Một số hằng dễ nhận ra đến mức không phải lúc nào cũng cần đặt sau một hằng có tên, miễn là chúng được dùng cùng đoạn mã tự giải thích rõ nghĩa. Ví dụ:

```
double milesWalked = feetWalked/5280.0;
int dailyPay = hourlyRate * 8;
double circumference = radius * Math.PI * 2;
```

Trong các ví dụ trên, ta có thật sự cần các hằng `FEET_PER_MILE`, `WORK_HOURS_PER_DAY` và `TWO` không? Rõ ràng, trường hợp cuối thật vô lý. Có những công thức viết hằng trực tiếp thành số sẽ tốt hơn. Bạn có thể tranh luận về trường hợp `WORK_HOURS_PER_DAY` vì luật lệ hay quy ước có thể thay đổi. Mặt khác, công thức có số 8 đọc rất tự nhiên, nên tôi

ngần ngại bắt người đọc gánh thêm 17 ký tự. Còn trong trường hợp `FEET_PER_MILE`, số 5280 nổi tiếng và đặc thù đến mức người đọc vẫn nhận ra nó ngay cả khi đứng một mình trên trang giấy, không có ngữ cảnh nào xung quanh.

Những hằng như 3.141592653589793 cũng rất nổi tiếng và dễ nhận ra. Tuy nhiên, khả năng mắc lỗi quá cao, không nên để chúng ở dạng số trần. Mỗi khi thấy 3.1415927535890793, người ta biết đó là π nên không xem xét kỹ. (Bạn có phát hiện lỗi một chữ số không?) Ta cũng không muốn mọi người dùng 3.14, 3.14159, 3.142, v.v. Vì thế, thật tốt khi `Math.PI` đã được định nghĩa sẵn.

Thuật ngữ "số ma thuật" không chỉ nói về số. Nó áp dụng cho mọi token có giá trị không tự nói rõ ý nghĩa. Ví dụ:

```
assertEquals(7777, Employee.find("John Doe").employeeNumber());
```

Trong câu khẳng định này có hai số ma thuật. Số đầu tiên hiển nhiên là 7777, dù ý nghĩa của nó không rõ. Số ma thuật thứ hai là `"John Doe"`, và ý định ở đây cũng không rõ.

Hóa ra `"John Doe"` là tên của nhân viên mã số 7777 trong cơ sở dữ liệu kiểm thử nổi tiếng do nhóm chúng tôi tạo. Mọi người trong nhóm đều biết khi kết nối với cơ sở dữ liệu này, sẽ có sẵn một số nhân viên cùng các giá trị và thuộc tính quen thuộc. Hóa ra `"John Doe"` là nhân viên hưởng lương theo giờ duy nhất trong cơ sở dữ liệu kiểm thử đó. Vì vậy, kiểm thử này thực ra nên viết như sau:

```
assertEquals(
  HOURLY_EMPLOYEE_ID,
  Employee.find(HOURLY_EMPLOYEE_NAME).employeeNumber());
```

G26: Chính xác

Cho rằng kết quả đầu tiên khớp truy vấn cũng là kết quả duy nhất có lẽ là ngây thơ. Dùng số thực dấu phẩy động để biểu diễn tiền tệ gần như là tội ác. Tránh khóa hoặc quản lý giao dịch vì nghĩ rằng cập nhật đồng thời ít có khả năng xảy ra, cùng lắm cũng là lười biếng. Khai báo biến kiểu `ArrayList` trong khi `List` đã đủ dùng là tự giới hạn quá mức. Mặc định đặt tất cả biến ở mức `protected` lại là giới hạn chưa đủ.

Khi đưa ra quyết định trong mã, hãy xác định thật chính xác. Hãy biết vì sao bạn quyết định như vậy và sẽ xử lý ngoại lệ ra sao. Đừng lười trong việc xác định độ chính xác cho các quyết định. Nếu quyết định gọi một hàm có thể trả về `null`, hãy kiểm tra `null`. Nếu truy vấn thứ bạn nghĩ là bản ghi duy nhất trong cơ sở dữ liệu, hãy bảo đảm mã kiểm tra xem còn bản ghi nào khác không. Nếu phải xử lý tiền tệ, hãy dùng số nguyên[11] và xử lý việc làm tròn cho phù hợp. Nếu có khả năng cập nhật đồng thời, hãy triển khai một cơ chế khóa nào đó.

Sự mơ hồ và thiếu chính xác trong mã xuất phát từ bất đồng hoặc lười biếng. Dù lý do nào, cũng cần loại bỏ chúng.

---

11. Hoặc tốt hơn nữa, dùng một lớp `Money` lưu giá trị bằng số nguyên.

G27: Cấu trúc thay cho quy ước

Hãy áp đặt các quyết định thiết kế bằng cấu trúc thay vì quy ước. Quy ước đặt tên rất tốt, nhưng không bằng những cấu trúc buộc mọi người phải tuân thủ. Ví dụ, switch/case với enum được đặt tên rõ ràng vẫn kém hơn lớp cơ sở có các phương thức trừu tượng. Không ai bị buộc phải viết câu lệnh switch/case theo cùng một cách mỗi lần, còn lớp cơ sở thì buộc các lớp cụ thể phải triển khai mọi phương thức trừu tượng.

G28: Đóng gói điều kiện

Logic Boolean vốn đã khó hiểu, càng khó hơn khi phải đọc trong ngữ cảnh câu lệnh if hay while. Hãy tách thành các hàm giải thích ý định của điều kiện.

Ví dụ:

```
if (shouldBeDeleted(timer))
```

tốt hơn:

```
if (timer.hasExpired() && !timer.isRecurrent())
```

G29: Tránh điều kiện phủ định

Điều kiện phủ định khó hiểu hơn một chút so với điều kiện khẳng định. Vì vậy, khi có thể, hãy viết điều kiện theo hướng khẳng định. Ví dụ:

```
if (buffer.shouldCompact())
```

tốt hơn:

```
if (!buffer.shouldNotCompact())
```

G30: Mỗi hàm chỉ nên làm một việc

Ta thường dễ bị cám dỗ viết những hàm gồm nhiều phần, mỗi phần thực hiện một loạt thao tác. Loại hàm này làm nhiều việc, nên cần được chuyển thành nhiều hàm nhỏ hơn, mỗi hàm chỉ làm một việc.

Ví dụ:

```
public void pay() {
  for (Employee e : employees) {
    if (e.isPayday()) {
      Money pay = e.calculatePay();
      e.deliverPay(pay);
    }
  }
}
```

Đoạn mã này làm ba việc. Nó lặp qua mọi nhân viên, kiểm tra xem từng người có đến ngày nhận lương không, rồi trả lương cho nhân viên ấy. Nên viết lại như sau:

```
public void pay() {
  for (Employee e : employees)
    payIfNecessary(e);
}

private void payIfNecessary(Employee e) {
  if (e.isPayday())
    calculateAndDeliverPay(e);
}

private void calculateAndDeliverPay(Employee e) {
  Money pay = e.calculatePay();
  e.deliverPay(pay);
}
```

Mỗi hàm trong số này chỉ làm một việc. (Xem “Làm một việc” ở trang 35.)

G31: Sự phụ thuộc thời điểm bị che giấu

Sự phụ thuộc thời điểm thường là cần thiết, nhưng đừng che giấu nó. Hãy sắp xếp các đối số của hàm sao cho thứ tự cần gọi các hàm trở nên hiển nhiên. Xét đoạn mã sau:

```
public class MoogDiver {
  Gradient gradient;
  List<Spline> splines;

  public void dive(String reason) {
    saturateGradient();
    reticulateSplines();
    diveForMoog(reason);
  }
  ...
}
```

Thứ tự của ba hàm rất quan trọng. Phải làm bão hòa gradient trước rồi mới có thể nối các spline, sau đó mới được lặn tìm moog. Đáng tiếc là mã không áp đặt thứ tự phụ thuộc thời điểm này. Một lập trình viên khác có thể gọi `reticulateSplines` trước khi gọi `saturateGradient`, dẫn đến `UnsaturatedGradientException`. Cách tốt hơn là:

```
public class MoogDiver {
  Gradient gradient;
  List<Spline> splines;

  public void dive(String reason) {
    Gradient gradient = saturateGradient();
    List<Spline> splines = reticulateSplines(gradient);
    diveForMoog(splines, reason);
  }
  ...
```

```
  }
```

Cách này làm lộ sự phụ thuộc thời điểm bằng cách tạo thành một dây chuyền. Mỗi hàm tạo ra kết quả mà hàm tiếp theo cần, vì thế không có cách hợp lý nào để gọi chúng sai thứ tự.

Có thể bạn sẽ phàn nàn rằng cách này làm các hàm phức tạp hơn, và bạn nói đúng. Nhưng phần phức tạp cú pháp thêm vào phơi bày độ phức tạp thực sự về thời điểm của tình huống.

Lưu ý rằng tôi vẫn giữ nguyên các biến thực thể. Tôi cho rằng các phương thức riêng trong lớp cần chúng. Dù vậy, tôi vẫn muốn để các đối số ở đó để thể hiện rõ sự phụ thuộc thời điểm.

G32: Đừng tùy tiện

Hãy có lý do cho cách cấu trúc mã và bảo đảm cấu trúc thể hiện được lý do đó. Nếu cấu trúc trông tùy tiện, người khác sẽ thấy mình có thể thay đổi nó. Nếu cấu trúc nhất quán trong toàn hệ thống, người khác sẽ dùng và giữ theo quy ước. Chẳng hạn, gần đây khi gộp các thay đổi vào FitNesse, tôi phát hiện một người đóng góp đã viết như sau:

```
public class AliasLinkWidget extends ParentWidget
{
  public static class VariableExpandingWidgetRoot {
    ...
    ...
  }
}
```

Vấn đề là `VariableExpandingWidgetRoot` không cần nằm trong phạm vi của `AliasLinkWidget`. Hơn nữa, một số lớp không liên quan khác lại dùng `AliasLinkWidget.VariableExpandingWidgetRoot`. Những lớp đó không cần biết đến `AliasLinkWidget`.

Có lẽ lập trình viên đưa `VariableExpandingWidgetRoot` vào `AliasWidget` cho tiện, hoặc nghĩ rằng nó thật sự cần được đặt trong phạm vi của `AliasWidget`. Dù lý do là gì, kết quả vẫn tùy tiện. Các lớp công khai không phải tiện ích của một lớp khác thì không nên được đặt trong lớp đó. Theo quy ước, chúng nên được khai báo công khai ở cấp cao nhất của gói.

G33: Đóng gói điều kiện biên

Điều kiện biên rất khó theo dõi. Hãy đặt phần xử lý chúng ở một nơi. Đừng để chúng rò rỉ khắp mã. Ta không muốn các phép cộng trừ 1 rải rác lung tung. Hãy xem ví dụ đơn giản sau trong FIT:

```
if(level + 1 < tags.length)
{
  parts = new Parse(body, tags, level + 1, offset + endTag);
  body = null;
```

```
  }
```

Bạn thấy `level+1` xuất hiện hai lần. Đây là một điều kiện biên nên được đóng gói trong biến tên chẳng hạn `nextLevel`.

```
int nextLevel = level + 1;
if(nextLevel < tags.length)
{
  parts = new Parse(body, tags, nextLevel, offset + endTag);
  body = null;
}
```

G34: Hàm chỉ nên đi xuống một mức trừu tượng

Các câu lệnh bên trong một hàm nên được viết ở cùng một mức trừu tượng, thấp hơn một mức so với thao tác được mô tả bởi tên hàm. Đây có thể là nguyên tắc kinh nghiệm khó diễn giải và tuân theo nhất. Ý tưởng nghe có vẻ đơn giản, nhưng con người quá giỏi trong việc trộn lẫn các mức trừu tượng một cách trơn tru. Chẳng hạn, hãy xem đoạn mã sau lấy từ FitNesse:

```
public String render() throws Exception
{
  StringBuffer html = new StringBuffer("<hr");
  if(size > 0)
    html.append(" size=\"").append(size + 1).append("\"");
  html.append(">");

  return html.toString();
}
```

Chỉ cần xem qua một lúc là bạn hiểu chuyện gì xảy ra. Hàm này dựng thẻ HTML để vẽ một đường kẻ ngang trên trang. Chiều cao của đường kẻ được quy định trong biến `size`.

Giờ hãy xem lại. Phương thức này trộn ít nhất hai mức trừu tượng. Mức thứ nhất là khái niệm đường kẻ ngang có kích thước. Mức thứ hai là cú pháp của thẻ HR. Đoạn mã này thuộc mô-đun `HruleWidget` trong FitNesse. Mô-đun này phát hiện một dãy gồm bốn dấu gạch ngang trở lên rồi chuyển nó thành thẻ HR tương ứng. Càng nhiều dấu gạch ngang, kích thước càng lớn.

Tôi tái cấu trúc đoạn mã như sau. Lưu ý rằng tôi đổi tên trường `size` để phản ánh mục đích thực sự của nó. Trường này lưu số dấu gạch ngang thừa.

```
public String render() throws Exception
{
  HtmlTag hr = new HtmlTag("hr");
  if (extraDashes > 0)
    hr.addAttribute("size", hrSize(extraDashes));
  return hr.html();
}

private String hrSize(int height)
{
```

```
    int hrSize = height + 1;
    return String.format("%d", hrSize);
  }
```

Thay đổi này tách riêng hai mức trừu tượng một cách gọn gàng. Hàm `render` chỉ dựng thẻ HR mà không cần biết gì về cú pháp HTML của thẻ đó. Mô-đun `HtmlTag` đảm nhiệm những chi tiết cú pháp rắc rối.

Thật ra, nhờ thay đổi này tôi đã phát hiện một lỗi khó thấy. Mã ban đầu không đặt dấu gạch chéo đóng vào thẻ HR theo yêu cầu của chuẩn XHTML. (Nói cách khác, nó xuất `<hr>` thay vì `<hr/>`.) Mô-đun `HtmlTag` đã được thay đổi để tuân theo XHTML từ lâu.

Tách các mức trừu tượng là một trong những chức năng quan trọng nhất của tái cấu trúc, đồng thời cũng là một trong những việc khó làm tốt nhất. Ví dụ, hãy xem đoạn mã dưới đây. Đây là lần đầu tôi thử tách các mức trừu tượng trong phương thức `HruleWidget.render`.

```
  public String render() throws Exception
  {
    HtmlTag hr = new HtmlTag("hr");
    if (size > 0) {
      hr.addAttribute("size", ""+(size+1));
    }
    return hr.html();
  }
```

Lúc đó, mục tiêu của tôi là tạo sự phân tách cần thiết và làm cho các kiểm thử chạy qua. Tôi dễ dàng đạt mục tiêu, nhưng kết quả vẫn là một hàm trộn lẫn các mức trừu tượng. Trong trường hợp này, những mức trừu tượng bị trộn là việc dựng thẻ HR và diễn giải, định dạng biến `size`. Điều đó cho thấy khi chia một hàm theo các mức trừu tượng, ta thường phát hiện thêm những ranh giới trừu tượng mới vốn bị cấu trúc trước đó che khuất.

G35: Giữ dữ liệu cấu hình ở mức cao

Nếu có một hằng như giá trị mặc định hay cấu hình vốn được biết đến và được mong đợi ở mức trừu tượng cao, đừng chôn nó trong một hàm mức thấp. Hãy đưa nó ra làm đối số cho hàm mức thấp được gọi từ hàm mức cao. Hãy xem đoạn mã sau trong FitNesse:

```
  public static void main(String[] args) throws Exception
  {
    Arguments arguments = parseCommandLine(args);
    ...
  }

  public class Arguments
  {
    public static final String DEFAULT_PATH = ".";
    public static final String DEFAULT_ROOT = "FitNesseRoot";
    public static final int DEFAULT_PORT = 80;
    public static final int DEFAULT_VERSION_DAYS = 14;
    ...
```

```
}
```

Các đối số dòng lệnh được phân tích ngay dòng lệnh thực thi đầu tiên của FitNesse. Các giá trị mặc định của chúng được quy định ở đầu lớp `Arguments`. Bạn không phải mò xuống các mức thấp của hệ thống để tìm những câu lệnh như sau:

```
if (arguments.port == 0) // mặc định dùng cổng 80
```

Các hằng cấu hình nằm ở mức rất cao và dễ thay đổi. Chúng được truyền xuống phần còn lại của ứng dụng. Các mức thấp hơn của ứng dụng không sở hữu giá trị của những hằng này.

G36: Tránh điều hướng bắc cầu

Nói chung, ta không muốn một mô-đun biết quá nhiều về các đối tác của nó. Cụ thể hơn, nếu A cộng tác với B, còn B cộng tác với C, ta không muốn các mô-đun dùng A biết về C. (Chẳng hạn, ta không muốn `a.getB().getC().doSomething()`.)

Cách làm này đôi khi được gọi là Luật Demeter. Các Pragmatic Programmers gọi nó là "Viết mã kín đáo".[12] Dù gọi thế nào, ý chính là bảo đảm các mô-đun chỉ biết những đối tác trực tiếp của mình, chứ không biết sơ đồ điều hướng của toàn hệ thống.

Nếu nhiều mô-đun dùng dạng câu lệnh `a.getB().getC()`, ta sẽ khó thay đổi thiết kế và kiến trúc để chèn Q vào giữa B và C. Khi đó, ta phải tìm từng chỗ dùng `a.getB().getC()` và đổi thành `a.getB().getQ().getC()`. Đó là cách kiến trúc trở nên cứng nhắc. Quá nhiều mô-đun biết quá nhiều về kiến trúc.

Thay vào đó, ta muốn các đối tác trực tiếp cung cấp mọi dịch vụ cần thiết. Ta không nên phải lần theo đồ thị đối tượng của hệ thống để tìm phương thức cần gọi. Ta chỉ cần viết:

```
myCollaborator.doSomething().
```

Chú thích

1. [PRAG], trang 138.

Java

J1: Tránh danh sách import dài bằng cách dùng ký tự đại diện

Nếu dùng từ hai lớp trở lên trong một gói, hãy import cả gói:

```
import package.*;
```

Danh sách import dài khiến người đọc nản lòng. Ta không muốn làm đầu mô-đun thêm

12. [PRAG], tr. 138.

lộn xộn với 80 dòng import. Thay vào đó, muốn các import nói gọn gàng những gói mà mô-đun cộng tác.

Import cụ thể tạo ra phụ thuộc cứng, còn import ký tự đại diện thì không. Nếu import một lớp cụ thể, lớp đó phải tồn tại. Nhưng nếu import cả gói bằng ký tự đại diện, không cần lớp cụ thể nào phải tồn tại. Câu lệnh import chỉ thêm gói vào đường dẫn tìm kiếm khi tra tên. Vì thế, các import này không tạo ra phụ thuộc thật sự và giúp mô-đun ít phụ thuộc lẫn nhau hơn.

Đôi khi danh sách dài các import cụ thể có ích. Chẳng hạn, nếu đang xử lý mã cũ và muốn biết cần tạo mock, stub cho những lớp nào, bạn có thể xem danh sách import cụ thể để tìm tên đầy đủ của tất cả các lớp, rồi tạo những stub phù hợp. Tuy nhiên, trường hợp cần import cụ thể như vậy rất hiếm. Hơn nữa, hầu hết IDE hiện đại cho phép chuyển import ký tự đại diện thành danh sách import cụ thể chỉ bằng một lệnh. Vì vậy, kể cả với mã cũ, import ký tự đại diện vẫn tốt hơn.

Import ký tự đại diện đôi khi có thể gây xung đột và mơ hồ về tên. Hai lớp cùng tên nhưng thuộc các gói khác nhau cần được import riêng, hoặc ít nhất phải ghi rõ đầy đủ tên gói khi dùng. Điều đó có thể gây phiền, nhưng hiếm khi xảy ra, nên nhìn chung import ký tự đại diện vẫn tốt hơn import cụ thể.

J2: Đừng kế thừa hằng

Tôi đã thấy cách làm này nhiều lần và lần nào cũng nhăn mặt. Lập trình viên đặt một số hằng trong giao diện rồi lấy chúng bằng cách kế thừa giao diện ấy. Hãy xem đoạn mã sau:

```
public class HourlyEmployee extends Employee {
  private int tenthsWorked;
  private double hourlyRate;

  public Money calculatePay() {
    int straightTime = Math.min(tenthsWorked, TENTHS_PER_WEEK);
    int overTime = tenthsWorked - straightTime;
    return new Money(
      hourlyRate * (tenthsWorked + OVERTIME_RATE * overTime)
    );
  }
  ...
}
```

Các hằng `TENTHS_PER_WEEK` và `OVERTIME_RATE` từ đâu ra? Có thể chúng đến từ lớp `Employee`, hãy xem thử:

```
public abstract class Employee implements PayrollConstants {
  public abstract boolean isPayday();
  public abstract Money calculatePay();
  public abstract void deliverPay(Money pay);
}
```

Không phải ở đó. Vậy chúng ở đâu? Hãy nhìn kỹ lớp `Employee`. Nó cài đặt `PayrollConstants`.

```
public interface PayrollConstants {
  public static final int TENTHS_PER_WEEK = 400;
  public static final double OVERTIME_RATE = 1.5;
}
```

Đây là cách làm thật xấu xí! Các hằng bị giấu ở đầu hệ thống phân cấp kế thừa. Eo ôi! Đừng dùng kế thừa để lách quy tắc phạm vi của ngôn ngữ. Thay vào đó, hãy dùng import static.

```
import static PayrollConstants.*;

public class HourlyEmployee extends Employee {
  private int tenthsWorked;
  private double hourlyRate;

  public Money calculatePay() {
    int straightTime = Math.min(tenthsWorked, TENTHS_PER_WEEK);
    int overTime = tenthsWorked - straightTime;
    return new Money(
      hourlyRate * (tenthsWorked + OVERTIME_RATE * overTime)
    );
  }
  ...
}
```

J3: Hằng hay enum

Giờ enum đã được thêm vào ngôn ngữ (Java 5), hãy dùng chúng! Đừng tiếp tục dùng mẹo cũ với các số nguyên `public static final`. Ý nghĩa của số nguyên có thể bị quên mất. Ý nghĩa của enum thì không, vì chúng thuộc về một kiểu liệt kê có tên.

Hơn nữa, hãy nghiên cứu kỹ cú pháp enum. Chúng có thể có phương thức và trường. Nhờ vậy, enum là công cụ rất mạnh, biểu đạt linh hoạt hơn nhiều so với số nguyên. Xét biến thể sau của mã tính lương:

```
public class HourlyEmployee extends Employee {
  private int tenthsWorked;
  HourlyPayGrade grade;

  public Money calculatePay() {
    int straightTime = Math.min(tenthsWorked, TENTHS_PER_WEEK);
    int overTime = tenthsWorked - straightTime;
    return new Money(
      grade.rate() * (tenthsWorked + OVERTIME_RATE * overTime)
    );
  }
  ...
}

public enum HourlyPayGrade {
  APPRENTICE {
    public double rate() {
      return 1.0;
```

```
      }
    },
    LEUTENANT_JOURNEYMAN {
      public double rate() {
        return 1.2;
      }
    },
    JOURNEYMAN {
      public double rate() {
        return 1.5;
      }
    },
    MASTER {
      public double rate() {
        return 2.0;
      }
    };
    public abstract double rate();
  }
```

Tên

N1: Chọn tên có tính mô tả

Đừng vội chọn tên. Hãy bảo đảm tên có tính mô tả. Ý nghĩa thường thay đổi khi phần mềm phát triển, vì thế hãy thường xuyên đánh giá lại xem những tên bạn chọn còn phù hợp hay không.

Đây không chỉ là lời khuyên để nghe cho vui. Tên trong phần mềm quyết định 90 phần trăm khả năng đọc của mã. Bạn cần dành thời gian chọn tên cho khôn ngoan và giữ chúng phù hợp. Tên quan trọng đến mức không thể xem nhẹ.

Xét đoạn mã dưới đây. Nó làm gì? Nếu tôi cho bạn xem mã với những cái tên được chọn kỹ, bạn sẽ hiểu ngay. Còn như thế này, nó chỉ là một mớ ký hiệu và số ma thuật.

```
  public int x() {
    int q = 0;
    int z = 0;
    for (int kk = 0; kk < 10; kk++) {
      if (l[z] == 10)
      {
        q += 10 + (l[z + 1] + l[z + 2]);
        z += 1;
      }
      else if (l[z] + l[z + 1] == 10)
      {
        q += 10 + l[z + 2];
        z += 2;
      } else {
        q += l[z] + l[z + 1];
        z += 2;
      }
    }
```

```
    return q;
  }
```

Đây là đoạn mã nên được viết. Thực ra, đoạn này còn ít hoàn chỉnh hơn đoạn trên. Dù vậy, bạn có thể hiểu ngay nó đang cố làm gì và rất có thể tự viết được các hàm còn thiếu dựa trên ý nghĩa suy ra. Các số ma thuật không còn là ma thuật, còn cấu trúc thuật toán mô tả rõ ràng điều nó làm.

```
  public int score() {
    int score = 0;
    int frame = 0;
    for (int frameNumber = 0; frameNumber < 10; frameNumber++) {
      if (isStrike(frame)) {
        score += 10 + nextTwoBallsForStrike(frame);
        frame += 1;
      } else if (isSpare(frame)) {
        score += 10 + nextBallForSpare(frame);
        frame += 2;
      } else {
        score += twoBallsInFrame(frame);
        frame += 2;
      }
    }
    return score;
  }
```

Tên được chọn cẩn thận có sức mạnh làm cho cấu trúc mã đồng thời truyền tải mô tả. Mô tả đó định hình kỳ vọng của người đọc về chức năng của các hàm khác trong mô-đun. Nhìn đoạn mã trên, bạn có thể suy ra phần cài đặt của `isStrike()`. Khi đọc phương thức `isStrike`, bạn sẽ thấy nó "gần như đúng những gì mình mong đợi".[13]

```
  private boolean isStrike(int frame) {
    return rolls[frame] == 10;
  }
```

N2: Chọn tên phù hợp với mức trừu tượng

Đừng chọn tên mô tả phần cài đặt; hãy chọn tên thể hiện mức trừu tượng của lớp hoặc hàm bạn đang làm việc. Việc này không dễ. Một lần nữa, con người quá giỏi trộn lẫn các mức trừu tượng. Mỗi lần xem lại mã, có lẽ bạn sẽ tìm thấy một biến được đặt tên ở mức quá thấp. Hãy đổi tên ngay khi phát hiện. Làm cho mã dễ đọc đòi hỏi sự kiên trì cải thiện liên tục. Xét giao diện `Modem` sau:

```
  public interface Modem {
    boolean dial(String phoneNumber);
    boolean disconnect();
    boolean send(char c);
    char recv();
```

13. Xem lời trích dẫn của Ward Cunningham ở trang 11.

```
    String getConnectedPhoneNumber();
}
```

Thoạt nhìn, giao diện này có vẻ ổn. Các hàm đều hợp lý. Quả thật, chúng phù hợp với nhiều ứng dụng. Nhưng hãy xét ứng dụng dùng một số modem không kết nối bằng cách quay số. Thay vào đó, chúng được nối cố định với nhau bằng dây, chẳng hạn modem cáp cung cấp kết nối Internet cho phần lớn hộ gia đình ngày nay. Có lẽ một số modem kết nối bằng cách gửi số cổng đến bộ chuyển mạch qua USB. Rõ ràng, khái niệm số điện thoại nằm ở sai mức trừu tượng. Cách đặt tên tốt hơn cho tình huống này có thể là:

```
public interface Modem {
  boolean connect(String connectionLocator);
  boolean disconnect();
  boolean send(char c);
  char recv();
  String getConnectedLocator();
}
```

Những tên này không giả định kết nối phải dùng số điện thoại. Chúng vẫn dùng được với số điện thoại, hoặc bất kỳ chiến lược kết nối nào khác.

N3: Dùng thuật ngữ chuẩn khi có thể

Tên dễ hiểu hơn nếu dựa trên quy ước hoặc cách dùng sẵn có. Chẳng hạn, nếu dùng mẫu DECORATOR, bạn nên đưa từ Decorator vào tên các lớp trang trí. Ví dụ, `AutoHangupModemDecorator` có thể là tên lớp bổ sung khả năng tự ngắt modem khi phiên kết nối kết thúc.

Mẫu thiết kế chỉ là một loại quy ước. Chẳng hạn, trong Java, các hàm chuyển đối tượng thành dạng biểu diễn chuỗi thường được đặt tên là `toString`. Làm theo những quy ước như vậy tốt hơn là tự nghĩ ra tên riêng.

Các nhóm thường tự tạo một hệ thống tên chuẩn cho dự án. Eric Evans gọi đó là ngôn ngữ phổ quát của dự án.[14] Mã của bạn nên dùng rộng rãi các thuật ngữ trong ngôn ngữ đó. Tóm lại, càng dùng được nhiều tên mang những ý nghĩa đặc thù phù hợp với dự án, người đọc càng dễ hiểu mã đang nói về điều gì.

N4: Tên không mơ hồ

Hãy chọn tên giúp người đọc hiểu rõ hàm hoặc biến hoạt động ra sao. Xét ví dụ sau từ FitNesse:

```
private String doRename() throws Exception
{
  if(refactorReferences)
    renameReferences();
  renamePage();
```

14. [DDD].

```
    pathToRename.removeNameFromEnd();
    pathToRename.addNameToEnd(newName);
    return PathParser.render(pathToRename);
  }
```

Tên hàm này chỉ nói rất chung chung và mơ hồ về việc hàm làm gì. Điều đó càng rõ khi bên trong hàm tên `doRename` lại có một hàm tên `renamePage`! Tên gọi cho ta biết gì về sự khác nhau giữa hai hàm? Chẳng biết gì cả.

Tên tốt hơn cho hàm đó là `renamePageAndOptionallyAllReferences`. Tên này có vẻ dài, đúng là dài, nhưng hàm chỉ được gọi ở một nơi trong mô-đun, nên giá trị giải thích của nó quan trọng hơn độ dài.

N5: Dùng tên dài cho phạm vi dài

Độ dài tên nên tương ứng với độ dài phạm vi. Trong phạm vi rất nhỏ, bạn có thể dùng tên biến ngắn; trong phạm vi lớn, nên dùng tên dài hơn.

Tên biến như `i` và `j` hoàn toàn ổn nếu phạm vi của chúng chỉ dài năm dòng. Xét đoạn mã từ ví dụ "Bowling Game" kinh điển cũ:

```
  private void rollMany(int n, int pins)
  {
    for (int i=0; i<n; i++)
      g.roll(pins);
  }
```

Đoạn này hoàn toàn rõ ràng. Nếu thay `i` bằng cái tên phiền phức như `rollCount`, mã sẽ khó hiểu hơn. Ngược lại, biến và hàm có tên ngắn sẽ mất ý nghĩa khi phạm vi trải dài. Phạm vi tên càng rộng, tên càng cần dài và chính xác.

N6: Tránh mã hóa tên

Không nên mã hóa thông tin về kiểu hay phạm vi vào tên. Các tiền tố như `m_` hoặc `f` chẳng có ích trong môi trường ngày nay. Tương tự, các tiền tố mã hóa dự án và/hoặc hệ thống con như `vis_` (viết tắt của visual imaging system, hệ thống tạo ảnh trực quan) gây rối và thừa. Một lần nữa, môi trường hiện đại đã cung cấp mọi thông tin đó, không cần làm biến dạng tên. Hãy giữ tên trong sạch, tránh nhiễm ký pháp Hungary.

N7: Tên nên mô tả tác dụng phụ

Tên nên mô tả mọi điều mà hàm, biến hoặc lớp là hoặc làm. Đừng che giấu tác dụng phụ trong tên. Đừng dùng một động từ đơn giản để mô tả hàm làm nhiều hơn hành động đơn giản đó. Chẳng hạn, hãy xét đoạn mã sau từ TestNG:

```
public ObjectOutputStream getOos() throws IOException {
  if (m_oos == null) {
    m_oos = new ObjectOutputStream(m_socket.getOutputStream());
  }
  return m_oos;
}
```

Hàm này làm nhiều hơn là lấy một `oos`: nó tạo `oos` nếu chưa được tạo. Vì vậy, tên tốt hơn có thể là `createOrReturnOos`.

Kiểm thử

T1: Kiểm thử chưa đầy đủ

Bộ kiểm thử nên có bao nhiêu ca? Đáng tiếc, thước đo nhiều lập trình viên dùng là "Có vẻ đủ rồi". Bộ kiểm thử nên kiểm tra mọi thứ có thể bị hỏng. Chừng nào còn điều kiện chưa được kiểm tra hoặc phép tính chưa được xác thực, kiểm thử vẫn chưa đầy đủ.

T2: Dùng công cụ đo độ bao phủ!

Công cụ đo độ bao phủ chỉ ra những lỗ hổng trong chiến lược kiểm thử. Chúng giúp dễ dàng tìm ra các mô-đun, lớp và hàm chưa được kiểm thử đầy đủ. Hầu hết IDE đều biểu thị trực quan: những dòng đã được thực thi có màu xanh lá, còn dòng chưa được thực thi có màu đỏ. Nhờ vậy, có thể nhanh chóng tìm ra các câu lệnh if hoặc catch mà phần thân chưa được kiểm tra.

T3: Đừng bỏ qua kiểm thử đơn giản

Chúng dễ viết, còn giá trị tài liệu của chúng lớn hơn chi phí tạo ra.

T4: Kiểm thử bị bỏ qua là câu hỏi về một điểm mơ hồ

Đôi khi ta không chắc một chi tiết hành vi vì yêu cầu chưa rõ. Ta có thể diễn đạt câu hỏi về yêu cầu bằng một kiểm thử được chú thích để vô hiệu hóa, hoặc một kiểm thử được gắn `@Ignore`. Chọn cách nào tùy thuộc vào việc điểm mơ hồ có liên quan đến thứ khiến mã biên dịch được hay không.

T5: Kiểm thử điều kiện ranh giới

Hãy đặc biệt cẩn thận khi kiểm thử điều kiện ranh giới. Ta thường xử lý đúng phần giữa của thuật toán nhưng đánh giá sai các ranh giới.

T6: Kiểm thử kỹ quanh lỗi

Lỗi thường tụ lại với nhau. Khi phát hiện lỗi trong một hàm, nên kiểm thử hàm đó thật kỹ. Có lẽ lỗi ấy không đứng một mình.

T7: Mẫu lỗi tiết lộ nhiều điều

Đôi khi có thể chẩn đoán vấn đề bằng cách tìm mẫu trong cách các ca kiểm thử thất bại. Đây là thêm một lý do để viết các ca kiểm thử đầy đủ nhất có thể. Các ca kiểm thử đầy đủ, được sắp xếp hợp lý, sẽ làm lộ ra các mẫu.

Ví dụ đơn giản: giả sử bạn nhận thấy mọi kiểm thử có đầu vào dài hơn năm ký tự đều thất bại. Hoặc mọi kiểm thử truyền số âm vào đối số thứ hai của một hàm đều thất bại thì sao? Đôi khi chỉ nhìn mẫu đỏ xanh trong báo cáo kiểm thử cũng đủ gợi ra tiếng "À!" dẫn đến lời giải. Xem lại trang 267 để thấy một ví dụ thú vị về điều này trong ví dụ SerialDate.

T8: Mẫu độ bao phủ kiểm thử có thể tiết lộ nhiều điều

Xem những đoạn mã nào được hoặc không được thực thi bởi các kiểm thử đạt có thể giúp tìm ra lý do những kiểm thử thất bại.

T9: Kiểm thử phải nhanh

Kiểm thử chậm là kiểm thử không được chạy. Khi tiến độ căng, những kiểm thử chậm sẽ bị bỏ khỏi bộ kiểm thử. Vì vậy, hãy làm mọi việc cần thiết để giữ cho kiểm thử chạy nhanh.

Kết luận

Khó có thể nói danh sách các nguyên tắc kinh nghiệm và mùi code này đã đầy đủ. Thật ra, tôi không chắc một danh sách như vậy có bao giờ đầy đủ được không. Nhưng có lẽ đầy đủ không phải là mục tiêu, vì danh sách này thể hiện một hệ giá trị.

Quả thật, hệ giá trị ấy là mục tiêu và chủ đề của cuốn sách này. Mã sạch không được viết bằng cách tuân theo một bộ quy tắc. Học thuộc một danh sách các nguyên tắc kinh nghiệm không biến bạn thành người thợ phần mềm lành nghề. Tính chuyên nghiệp và tay nghề đến từ những giá trị dẫn dắt kỷ luật.

Tài liệu tham khảo

[Refactoring]: *Refactoring: Improving the Design of Existing Code*, Martin Fowler và cộng sự, Addison-Wesley, 1999.

[PRAG]: *The Pragmatic Programmer*, Andrew Hunt, Dave Thomas, Addison-Wesley, 2000.

[GOF]: *Design Patterns: Elements of Reusable Object Oriented Software*, Gamma và cộng sự, Addison-Wesley, 1996.

[Beck97]: *Smalltalk Best Practice Patterns*, Kent Beck, Prentice Hall, 1997.

[Beck07]: *Implementation Patterns*, Kent Beck, Addison-Wesley, 2008.

[PPP]: *Agile Software Development: Principles, Patterns, and Practices*, Robert C. Martin, Prentice Hall, 2002.

[DDD]: *Domain Driven Design*, Eric Evans, Addison-Wesley, 2003.

Phụ lục A

Xử lý đồng thời II

Tác giả: Brett L. Schuchert

Phụ lục này bổ sung và mở rộng chương Xử lý đồng thời ở trang 177. Nội dung được chia thành các chủ đề độc lập, nhìn chung bạn có thể đọc theo thứ tự bất kỳ. Một số phần có nội dung trùng lặp để phù hợp với cách đọc đó.

Ví dụ client/server

Hãy hình dung một ứng dụng client/server đơn giản. Server chờ kết nối của client trên một socket. Client kết nối rồi gửi yêu cầu.

Server

Đây là phiên bản rút gọn của một ứng dụng server. Toàn bộ mã nguồn của ví dụ này có từ trang 343, mục Client/server không dùng luồng.

```
ServerSocket serverSocket = new ServerSocket(8009);
while (keepProcessing) {
    try {
        Socket socket = serverSocket.accept();
        process(socket);
    } catch (Exception e) {
        handle(e);
    }
}
```

Ứng dụng đơn giản này chờ kết nối, xử lý thông điệp đến rồi tiếp tục chờ yêu cầu tiếp theo

từ client. Đây là mã phía client kết nối với server:

```
private void connectSendReceive(int i) {
    try {
        Socket socket = new Socket("localhost", PORT);
        MessageUtils.sendMessage(socket, Integer.toString(i));
        MessageUtils.getMessage(socket);
        socket.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Cặp client/server này hoạt động tốt đến đâu? Làm thế nào mô tả chính xác hiệu năng của nó? Đây là một kiểm thử khẳng định hiệu năng ở mức "chấp nhận được":

```
@Test(timeout = 10000)
public void shouldRunInUnder10Seconds() throws Exception {
    Thread[] threads = createThreads();
    startAllThreadsw(threads);
    waitForAllThreadsToFinish(threads);
}
```

Phần thiết lập được lược đi để ví dụ đơn giản hơn (xem "ClientTest.java" ở trang 344). Kiểm thử này khẳng định chương trình phải hoàn tất trong vòng 10.000 mili giây.

Đây là ví dụ kinh điển về kiểm tra thông lượng của một hệ thống. Hệ thống phải xử lý xong một loạt yêu cầu client trong mười giây. Chỉ cần server xử lý kịp từng yêu cầu riêng lẻ thì kiểm thử sẽ đạt.

Nếu kiểm thử thất bại thì sao? Nếu không xây dựng một vòng lặp thăm dò sự kiện, trong một luồng đơn lẻ ta không thể làm gì nhiều để tăng tốc đoạn mã này. Dùng nhiều luồng có giải quyết được vấn đề không? Có thể, nhưng trước hết ta cần biết thời gian đang tiêu tốn vào đâu. Có hai khả năng:

- I/O: dùng socket, kết nối cơ sở dữ liệu, chờ bộ nhớ ảo hoán đổi, v.v.
- Bộ xử lý: tính toán số học, xử lý biểu thức chính quy, thu gom rác, v.v.

Hệ thống thường có cả hai loại công việc, nhưng với mỗi thao tác thường sẽ có một loại chiếm ưu thế. Nếu mã bị giới hạn bởi bộ xử lý, bổ sung phần cứng xử lý có thể tăng thông lượng và giúp kiểm thử đạt. Nhưng số chu kỳ CPU luôn có giới hạn, nên thêm luồng vào một vấn đề bị giới hạn bởi bộ xử lý sẽ không làm nó nhanh hơn.

Ngược lại, nếu tiến trình bị giới hạn bởi I/O thì xử lý đồng thời có thể tăng hiệu quả. Khi một phần của hệ thống chờ I/O, phần khác có thể tận dụng thời gian chờ đó để xử lý việc khác, nhờ vậy sử dụng CPU hiệu quả hơn.

Thêm luồng

Tạm giả sử kiểm thử hiệu năng thất bại. Ta có thể tăng thông lượng để kiểm thử đạt bằng cách nào? Nếu phương thức `process` của server bị giới hạn bởi I/O, có thể cho server dùng luồng như sau (chỉ cần thay đổi `processMessage`):

```
void process(final Socket socket) {
    if (socket == null)
        return;
    Runnable clientHandler = new Runnable() {
        public void run() {
            try {
                String message = MessageUtils.getMessage(socket);
                MessageUtils.sendMessage(socket, "Processed: " + message);
                closeIgnoringException(socket);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    };

    Thread clientConnection = new Thread(clientHandler);
    clientConnection.start();
}
```

Giả sử thay đổi này giúp kiểm thử đạt.[1] Mã đã hoàn chỉnh, đúng chưa?

Nhận xét về server

Server đã cập nhật hoàn tất kiểm thử chỉ trong hơn một giây. Tiếc là giải pháp này hơi ngây thơ và nảy sinh một số vấn đề mới.

Server có thể tạo bao nhiêu luồng? Mã không đặt giới hạn, nên ta có thể chạm đến giới hạn của Máy ảo Java (JVM). Với nhiều hệ thống đơn giản, như vậy có thể đủ dùng. Nhưng nếu hệ thống phải hỗ trợ nhiều người dùng trên mạng công cộng thì sao? Quá nhiều người kết nối cùng lúc có thể khiến hệ thống đình trệ.

Tạm gác vấn đề hành vi đó sang một bên. Giải pháp trên còn có vấn đề về độ sạch và cấu trúc. Mã server đang đảm nhận bao nhiêu trách nhiệm?

- Quản lý kết nối socket
- Xử lý client
- Chính sách về luồng
- Chính sách tắt server

Đáng tiếc là tất cả trách nhiệm này đều nằm trong hàm `process`. Ngoài ra, đoạn mã còn đan xen nhiều cấp độ trừu tượng. Vì vậy, dù hàm `process` nhỏ, ta vẫn cần chia lại nó.

Server có nhiều lý do để thay đổi, vì thế nó vi phạm Nguyên tắc Trách nhiệm Đơn lẻ. Để giữ cho hệ thống đồng thời sạch sẽ, việc quản lý luồng nên được gom vào một vài nơi có

1. Bạn có thể tự kiểm chứng bằng cách chạy thử mã trước và sau khi thay đổi. Xem mã không dùng luồng bắt đầu ở trang 343, và mã có dùng luồng bắt đầu ở trang 346.

kiểm soát chặt chẽ. Hơn nữa, mọi mã quản lý luồng chỉ nên làm đúng việc quản lý luồng. Vì sao? Chỉ riêng việc lần tìm lỗi xử lý đồng thời đã đủ khó, không cần phải đồng thời tháo gỡ những vấn đề không liên quan khác.

Nếu tạo một lớp riêng cho từng trách nhiệm liệt kê ở trên, kể cả trách nhiệm quản lý luồng, thì khi thay đổi chiến lược quản lý luồng, phạm vi mã bị ảnh hưởng sẽ nhỏ hơn và không làm nhiễm bẩn các trách nhiệm khác. Cách này cũng giúp kiểm thử những trách nhiệm còn lại dễ hơn nhiều mà không phải bận tâm đến luồng. Đây là phiên bản cập nhật thực hiện điều đó:

```
public void run() {
    while (keepProcessing) {
        try {
            ClientConnection clientConnection =
    connectionManager.awaitClient();
            ClientRequestProcessor requestProcessor
                = new ClientRequestProcessor(clientConnection);
            clientScheduler.schedule(requestProcessor);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    connectionManager.shutdown();
}
```

Giờ đây, mọi việc liên quan đến luồng đều tập trung ở một nơi: `clientScheduler`. Nếu có vấn đề về xử lý đồng thời, ta chỉ cần tìm ở một chỗ:

```
public interface ClientScheduler {
    void schedule(ClientRequestProcessor requestProcessor);
}
```

Chính sách hiện tại rất dễ triển khai:

```
public class ThreadPerRequestScheduler implements ClientScheduler {
    public void schedule(final ClientRequestProcessor requestProcessor) {
        Runnable runnable = new Runnable() {
            public void run() {
                requestProcessor.process();
            }
        };

        Thread thread = new Thread(runnable);
        thread.start();
    }
}
```

Khi đã tách toàn bộ việc quản lý luồng vào một nơi, việc thay đổi cách kiểm soát luồng trở nên dễ dàng hơn nhiều. Chẳng hạn, chuyển sang framework Executor của Java 5 chỉ cần viết một lớp mới rồi gắn nó vào (Đoạn mã A-1).

Đoạn mã A-1
ExecutorClientScheduler.java

```
import java.util.concurrent.Executor;
import java.util.concurrent.Executors;

public class ExecutorClientScheduler implements ClientScheduler {
    Executor executor;

    public ExecutorClientScheduler(int availableThreads) {
        executor = Executors.newFixedThreadPool(availableThreads);
    }
    public void schedule(final ClientRequestProcessor requestProcessor) {
        Runnable runnable = new Runnable() {
            public void run() {
                requestProcessor.process();
            }
        };
        executor.execute(runnable);
    }
}
```

Kết luận

Đưa xử lý đồng thời vào ví dụ cụ thể này cho thấy một cách tăng thông lượng hệ thống và một cách kiểm tra thông lượng đó bằng framework kiểm thử. Gom toàn bộ mã xử lý đồng thời vào một số ít lớp là một ví dụ áp dụng Nguyên tắc Trách nhiệm Đơn lẻ. Điều này đặc biệt quan trọng trong lập trình đồng thời vì độ phức tạp của nó.

Các đường thực thi khả dĩ

Xem phương thức `incrementValue`, một phương thức Java chỉ có một dòng, không có vòng lặp hay rẽ nhánh:

```
public class IdGenerator {
    int lastIdUsed;

    public int incrementValue() {
        return ++lastIdUsed;
    }
}
```

Bỏ qua tràn số nguyên và giả sử chỉ có một luồng truy cập một thực thể `IdGenerator`. Khi đó chỉ có một đường thực thi và một kết quả được đảm bảo:

- Giá trị trả về bằng với giá trị của `lastIdUsed`; cả hai đều lớn hơn một đơn vị so với ngay trước khi gọi phương thức.

Nếu dùng hai luồng mà không đổi phương thức thì sao? Mỗi luồng gọi `incrementValue` một lần, những kết quả nào có thể xảy ra? Có bao nhiêu đường thực thi khả dĩ? Trước hết

là các kết quả (giả sử ban đầu `lastIdUsed` có giá trị 93):

- Luồng 1 nhận giá trị 94, luồng 2 nhận giá trị 95, và `lastIdUsed` lúc này là 95.
- Luồng 1 nhận giá trị 95, luồng 2 nhận giá trị 94, và `lastIdUsed` lúc này là 95.
- Luồng 1 nhận giá trị 94, luồng 2 cũng nhận giá trị 94, và `lastIdUsed` lúc này là 94.

Kết quả cuối cùng tuy bất ngờ nhưng hoàn toàn có thể xảy ra. Để hiểu vì sao có những kết quả khác nhau, ta cần biết số lượng đường thực thi khả dĩ và cách Máy ảo Java thực thi chúng.

Số lượng đường thực thi

Để tính số đường thực thi khả dĩ, ta bắt đầu từ mã bytecode được tạo ra. Một dòng Java (`return ++lastIdUsed;`) trở thành tám chỉ thị bytecode. Hai luồng có thể đan xen việc thực thi tám chỉ thị này như người chia bài đan các lá bài khi xáo một bộ bài.[2] Ngay cả khi mỗi tay chỉ có tám lá, số kết quả xáo trộn vẫn rất lớn.

Với trường hợp đơn giản gồm N chỉ thị theo một chuỗi, không có vòng lặp hay điều kiện, và T luồng, tổng số đường thực thi khả dĩ bằng:

$$\frac{(NT)!}{(N!)^T}$$

Tính số thứ tự khả dĩ

Phần giải thích này trích từ email của Uncle Bob gửi Brett:

> *Với N bước và T luồng, tổng cộng có T * N bước. Trước mỗi bước, một lần chuyển ngữ cảnh sẽ chọn một trong T luồng. Vì thế, mỗi đường đi có thể biểu diễn bằng một chuỗi chữ số cho biết các lần chuyển ngữ cảnh. Với các bước A và B cùng hai luồng 1 và 2, sáu đường đi khả dĩ là 1122, 1212, 1221, 2112, 2121 và 2211. Nếu biểu diễn theo bước thì lần lượt là A1B1A2B2, A1A2B1B2, A1A2B2B1, A2A1B1B2, A2A1B2B1 và A2B2A1B1. Với ba luồng, chuỗi có thể là 112233, 112323, 113223, 113232, 112233, 121233, 121323, 121332, 123132, 123123, ...*
>
> *Một đặc điểm của các chuỗi này là luôn phải có N lần xuất hiện của mỗi T. Vì vậy chuỗi 111111 không hợp lệ, vì nó có sáu số 1 và không có số 2 hay 3 nào.*

Tính số thứ tự khả dĩ (tiếp theo)

Ta cần tìm các hoán vị của N số 1, N số 2, . . . và N số T. Thực chất đây là các hoán vị của N * T phần tử lấy đủ N * T phần tử, tức (N * T)!, nhưng phải loại bỏ mọi phần tử trùng lặp. Do đó, ta cần đếm số trường hợp trùng rồi chia (N * T)! cho số đó.

Với hai bước và hai luồng, có bao nhiêu trường hợp trùng? Mỗi chuỗi bốn chữ số có hai số 1 và hai số 2. Có thể đổi chỗ từng cặp mà không làm thay đổi ý nghĩa chuỗi. Ta có

---

2. Đây là một sự đơn giản hóa. Tuy nhiên, để phục vụ phần thảo luận này, ta có thể dùng mô hình đơn giản hóa đó.

thể đổi chỗ các số 1, các số 2, cả hai cặp hoặc không đổi cặp nào. Vì thế mỗi chuỗi có bốn dạng tương đương, tức ba dạng trùng. Ba trong bốn lựa chọn là trùng; nói cách khác, một phần tư số hoán vị KHÔNG trùng. 4! * .25 = 6. Có vẻ cách tính này đúng.

Có bao nhiêu trường hợp trùng? Khi N = 2 và T = 2, ta có thể đổi chỗ các số 1, các số 2 hoặc cả hai. Khi N = 2 và T = 3, ta có thể đổi chỗ các số 1, các số 2, các số 3, các số 1 và 2, các số 1 và 3, hoặc các số 2 và 3. Việc đổi chỗ chính là các hoán vị của N. Giả sử có P hoán vị của N. Số cách khác nhau để sắp xếp các hoán vị đó là P^T.

Vậy số dạng tương đương là $N!^T$. Số đường đi là $(T*N)!/(N!^T)$. Với trường hợp T = 2, N = 2, ta lại được 6 (24/4).

Với N = 2 và T = 3, ta có 720/8 = 90.

Với N = 3 và T = 3, ta có $9!/6^3 = 1680$.

Trong trường hợp đơn giản của một dòng mã Java, tương đương tám dòng bytecode, và hai luồng, tổng số đường thực thi khả dĩ là 12.870. Nếu kiểu của `lastIdUsed` là `long`, mỗi lần đọc/ghi trở thành hai thao tác thay vì một, và số thứ tự khả dĩ tăng thành 2.704.156.

Điều gì xảy ra nếu ta thay đổi phương thức này một chút?

```
public synchronized void incrementValue() {
    ++lastIdUsed;
}
```

Với hai luồng, số đường thực thi khả dĩ giảm còn hai; trong trường hợp tổng quát, con số đó là N!.

Tìm hiểu sâu hơn

Còn kết quả đáng ngạc nhiên rằng hai luồng có thể gọi phương thức một lần (trước khi thêm `synchronized`) và nhận cùng một kết quả số thì sao? Điều đó có thể xảy ra như thế nào? Trước hết, hãy bắt đầu từ cơ bản.

Thao tác nguyên tử là gì? Có thể định nghĩa thao tác nguyên tử là thao tác không thể bị ngắt quãng. Chẳng hạn, trong đoạn mã sau, dòng 5 gán 0 cho `lastId` là thao tác nguyên tử vì theo mô hình bộ nhớ Java, phép gán giá trị 32 bit không thể bị ngắt quãng.

```
01: public class Example {
02:    int lastId;
03:
04:    public void resetId() {
05:        value = 0;
06:    }
07:
08:    public int getNextId() {
09:        ++value;
10:    }
11:}
```

Nếu đổi kiểu của `lastId` từ `int` sang `long` thì sao? Dòng 5 còn là thao tác nguyên tử không? Theo đặc tả JVM thì không. Trên một bộ xử lý cụ thể, thao tác đó có thể nguyên tử, nhưng đặc tả JVM quy định phép gán mọi giá trị 64 bit cần hai phép gán 32 bit. Điều

này có nghĩa là giữa lần gán 32 bit thứ nhất và thứ hai, một luồng khác có thể chen vào và thay đổi một trong các giá trị.

Còn toán tử tăng trước ++ ở dòng 9 thì sao? Toán tử tăng trước có thể bị ngắt quãng nên không nguyên tử. Để hiểu điều này, hãy xem chi tiết bytecode của cả hai phương thức.

Trước khi tiếp tục, đây là ba định nghĩa quan trọng:

- Khung (frame): Mỗi lần gọi phương thức cần một khung. Khung chứa địa chỉ trả về, các tham số được truyền vào phương thức và các biến cục bộ được định nghĩa trong phương thức. Đây là kỹ thuật tiêu chuẩn để mô tả ngăn xếp lời gọi, được các ngôn ngữ hiện đại dùng cho việc gọi hàm/phương thức cơ bản và gọi đệ quy.
- Biến cục bộ: Mọi biến được định nghĩa trong phạm vi của phương thức. Tất cả phương thức không tĩnh đều có ít nhất một biến là `this`, đại diện cho đối tượng hiện tại, tức đối tượng nhận thông điệp gần nhất (trong luồng hiện tại) khiến phương thức được gọi.
- Ngăn xếp toán hạng: Nhiều chỉ thị trong Máy ảo Java nhận tham số. Các tham số đó được đặt trên ngăn xếp toán hạng. Ngăn xếp là cấu trúc dữ liệu vào sau ra trước (LIFO) tiêu chuẩn.

Đây là bytecode được tạo ra cho `resetId()`:

Mã lệnh	Mô tả	Ngăn xếp toán hạng sau lệnh
ALOAD 0	Nạp biến thứ 0 lên ngăn xếp toán hạng. Biến thứ 0 là `this`, đối tượng hiện tại. Khi phương thức được gọi, đối tượng nhận thông điệp, một thực thể của `Example`, được đưa vào mảng biến cục bộ của khung tạo ra cho lần gọi phương thức. Đây luôn là biến đầu tiên được đưa vào mọi phương thức thực thể.	this
ICONST_0	Đưa giá trị hằng 0 lên ngăn xếp toán hạng.	this, 0
PUTFIELD lastId	Lưu giá trị trên cùng của ngăn xếp (0) vào trường `value` của đối tượng được tham chiếu bởi tham chiếu đối tượng nằm ngay dưới cùng, tức `this`.	<empty>

Ba chỉ thị này được đảm bảo là nguyên tử vì dù luồng đang thực thi có thể bị ngắt sau bất kỳ chỉ thị nào, một luồng khác không thể chạm tới thông tin của chỉ thị `PUTFIELD` (giá trị hằng 0 trên đỉnh ngăn xếp, tham chiếu `this` ngay bên dưới, cùng giá trị trường). Vì vậy, khi phép gán diễn ra, ta được đảm bảo giá trị 0 sẽ được lưu vào trường. Thao tác này là nguyên tử. Các toán hạng đều xử lý thông tin cục bộ của phương thức nên nhiều luồng không can thiệp lẫn nhau.

Nếu mười luồng thực thi ba chỉ thị này thì có 4.38679733629e+24 thứ tự khả dĩ. Tuy nhiên, chỉ có một kết quả có thể xảy ra, nên các thứ tự khác nhau không quan trọng. Trong trường hợp này, kết quả tương tự cũng được đảm bảo với `long`. Vì sao? Cả mười luồng đều gán một giá trị hằng. Dù chúng đan xen với nhau, kết quả cuối cùng vẫn như nhau.

Với phép toán ++ trong phương thức `getNextId`, vấn đề sẽ nảy sinh. Giả sử lúc bắt đầu phương thức, `lastId` có giá trị 42. Đây là bytecode của phương thức mới:

Mã lệnh	Mô tả	Ngăn xếp toán hạng sau lệnh
ALOAD 0	Đưa `this` lên ngăn xếp toán hạng.	this
DUP	Sao chép phần tử trên cùng của ngăn xếp. Lúc này trên ngăn xếp có hai bản sao của `this`.	this, this
GETFIELD lastId	Lấy giá trị trường `lastId` từ đối tượng được phần tử trên cùng (`this`) trỏ tới rồi đưa giá trị đó trở lại ngăn xếp.	this, 42
ICONST_1	Đưa hằng số nguyên 1 lên ngăn xếp.	this, 42, 1
IADD	Cộng hai giá trị trên cùng của ngăn xếp toán hạng rồi đưa kết quả trở lại ngăn xếp.	this, 43
DUP_X1	Sao chép giá trị 43 và đặt nó trước `this`.	43, this, 43
PUTFIELD value	Lưu giá trị 43 trên cùng của ngăn xếp vào trường `value` của đối tượng hiện tại, được đại diện bởi giá trị kế trên ngăn xếp là `this`.	43
IRETURN	Trả về phần tử duy nhất trên cùng của ngăn xếp.	<empty>

Hãy hình dung luồng thứ nhất thực hiện xong ba chỉ thị đầu, đến hết `GETFIELD`, rồi bị ngắt. Luồng thứ hai tiếp quản và thực hiện toàn bộ phương thức, tăng `lastId` thêm một đơn vị; nó nhận về 43. Sau đó luồng thứ nhất tiếp tục từ chỗ dừng. Trên ngăn xếp toán hạng vẫn là 42, vì đó là giá trị của `lastId` khi nó thực thi `GETFIELD`. Luồng này cộng thêm một thành 43 rồi lưu kết quả. Luồng thứ nhất cũng nhận về 43. Một lần tăng bị mất vì luồng thứ nhất ghi đè kết quả của luồng thứ hai sau khi luồng thứ hai đã chen ngang và hoàn tất.

Đặt phương thức `getNexId()` thành `synchronized` sẽ khắc phục vấn đề này.

Kết luận

Không cần hiểu sâu bytecode mới biết các luồng có thể giẫm lên việc của nhau như thế nào. Nếu hiểu được ví dụ này, bạn sẽ thấy các luồng có thể can thiệp lẫn nhau, và chỉ cần biết đến khả năng đó là đủ.

Dù vậy, ví dụ đơn giản này cho thấy ta cần hiểu mô hình bộ nhớ đủ để biết điều gì an toàn, điều gì không. Nhiều người lầm tưởng toán tử `++` (tăng trước hoặc tăng sau) là nguyên tử, nhưng rõ ràng không phải vậy. Vì thế, bạn cần biết:

- Những đối tượng/giá trị nào được chia sẻ
- Đoạn mã nào có thể gây ra vấn đề đọc/cập nhật đồng thời
- Cách ngăn những vấn đề đồng thời đó xảy ra

Hiểu thư viện của bạn

Framework Executor

Như đã minh họa trong `ExecutorClientScheduler.java` ở trang 321, framework Executor được giới thiệu trong Java 5 cho phép thực thi linh hoạt bằng nhóm luồng. Đây là một lớp trong gói `java.util.concurrent`.

Nếu bạn đang tạo luồng mà không dùng nhóm luồng, hoặc tự viết một nhóm luồng, hãy cân nhắc dùng Executor. Mã sẽ sạch hơn, dễ theo dõi hơn và ngắn hơn.

Framework Executor gom luồng vào nhóm, tự động điều chỉnh kích thước và tạo lại luồng khi cần. Nó cũng hỗ trợ future, một cấu trúc lập trình đồng thời phổ biến. Framework Executor hoạt động với các lớp cài đặt `Runnable` và cả những lớp cài đặt giao diện `Callable`. `Callable` trông giống `Runnable`, nhưng có thể trả về kết quả, một nhu cầu thường gặp trong các giải pháp đa luồng.

Future rất hữu ích khi mã cần thực hiện nhiều thao tác độc lập rồi chờ tất cả hoàn tất:

```
public String processRequest(String message) throws Exception {
    Callable<String> makeExternalCall = new Callable<String>() {
        public String call() throws Exception {
            String result = "";
            // thực hiện yêu cầu bên ngoài
            return result;
        }
    };

    Future<String> result = executorService.submit(makeExternalCall);
    String partialResult = doSomeLocalProcessing();
    return result.get() + partialResult;
}
```

Trong ví dụ này, phương thức bắt đầu thực thi đối tượng `makeExternalCall` rồi tiếp tục xử lý việc khác. Dòng cuối gọi `result.get()`, lệnh này chặn cho đến khi future hoàn tất.

Giải pháp không chặn

Máy ảo Java 5 tận dụng thiết kế bộ xử lý hiện đại, hỗ trợ cập nhật không chặn đáng tin cậy. Chẳng hạn, hãy xét một lớp dùng đồng bộ hóa (và do đó có chặn) để cập nhật giá trị an toàn với luồng:

```
public class ObjectWithValue {
    private int value;
    public void synchronized incrementValue() { ++value; }
    public int getValue() { return value; }
}
```

Java 5 có một loạt lớp mới cho những tình huống như vậy, chẳng hạn `AtomicBoolean`, `AtomicInteger` và `AtomicReference`, cùng nhiều lớp khác. Ta có thể viết lại đoạn mã trên theo cách không chặn như sau:

```
public class ObjectWithValue {
    private AtomicInteger value = new AtomicInteger(0);

    public void incrementValue() {
        value.incrementAndGet();
    }
    public int getValue() {
        return value.get();
    }
}
```

Dù phiên bản này dùng một đối tượng thay vì kiểu nguyên thủy và gọi phương thức như `incrementAndGet()` thay cho ++, hiệu năng của lớp hầu như luôn tốt hơn phiên bản trước. Trong một số trường hợp nó chỉ nhanh hơn đôi chút, còn trường hợp chậm hơn gần như không có.

Điều này có thể xảy ra thế nào? Bộ xử lý hiện đại thường có một thao tác gọi là Compare and Swap (CAS). Thao tác này tương tự khóa lạc quan trong cơ sở dữ liệu, còn phiên bản `synchronized` tương tự khóa bi quan.

Từ khóa `synchronized` luôn lấy khóa, kể cả khi không có luồng thứ hai nào định cập nhật cùng giá trị. Dù hiệu năng của khóa nội tại đã được cải thiện qua từng phiên bản, chúng vẫn tốn kém.

Phiên bản không chặn giả định rằng nhiều luồng thường không sửa cùng một giá trị đủ thường xuyên để gây ra vấn đề. Thay vào đó, nó phát hiện hiệu quả xem tình huống đó có xảy ra hay không và thử lại cho đến khi cập nhật thành công. Chi phí phát hiện này hầu như luôn thấp hơn chi phí lấy khóa, kể cả khi mức tranh chấp từ vừa đến cao.

Máy ảo thực hiện việc này như thế nào? Thao tác CAS là nguyên tử. Xét về logic, CAS có thể hình dung như sau:

```
int variableBeingSet;

void simulateNonBlockingSet(int newValue) {
    int currentValue;
    do {
        currentValue = variableBeingSet
    } while(currentValue != compareAndSwap(currentValue, newValue));
}
int synchronized compareAndSwap(int currentValue, int newValue) {
    if(variableBeingSet == currentValue) {
        variableBeingSet = newValue;
        return currentValue;
    }
    return variableBeingSet;
}
```

Khi một phương thức định cập nhật biến được chia sẻ, thao tác CAS xác minh biến sắp được gán vẫn có giá trị đã biết gần nhất hay không. Nếu đúng, biến sẽ được thay đổi. Nếu không, biến không được gán vì một luồng khác đã chen vào. Phương thức đang thử cập nhật bằng CAS nhận ra thay đổi chưa diễn ra và thử lại.

Các lớp không an toàn với luồng

Có một số lớp vốn không an toàn với luồng. Sau đây là vài ví dụ:

- `SimpleDateFormat`
- Kết nối cơ sở dữ liệu
- Các lớp chứa trong `java.util`
- Servlet

Lưu ý rằng một số lớp collection có những phương thức riêng lẻ an toàn với luồng. Tuy nhiên, thao tác gọi nhiều hơn một phương thức thì không. Ví dụ, nếu không muốn thay thế một phần tử trong `HashTable` khi phần tử đó đã có sẵn, ta có thể viết:

```
if(!hashTable.containsKey(someKey)) {
    hashTable.put(someKey, new SomeValue());
}
```

Mỗi phương thức riêng lẻ đều an toàn với luồng. Tuy nhiên, một luồng khác có thể thêm giá trị vào giữa lần gọi `containsKey` và `put`. Có một số cách khắc phục:

- Khóa `HashTable` trước và đảm bảo mọi nơi dùng nó đều làm như vậy, tức khóa phía client:

```
synchronized(map) {
    if(!map.conainsKey(key))
        map.put(key,value);
}
```

- Bọc `HashTable` trong đối tượng riêng và dùng API khác, tức khóa phía server bằng một ADAPTER:

```
public class WrappedHashtable<K, V> {
    private Map<K, V> map = new Hashtable<K, V>();

    public synchronized void putIfAbsent(K key, V value) {
        if (map.containsKey(key))
            map.put(key, value);
    }
}
```

- Dùng các collection an toàn với luồng:

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<Integer,
String>();
map.putIfAbsent(key, value);
```

Các collection trong `java.util.concurrent` có những thao tác như `putIfAbsent()` để hỗ trợ loại thao tác này.

Phụ thuộc giữa các phương thức có thể làm hỏng mã đồng thời

Đây là ví dụ đơn giản về cách tạo ra sự phụ thuộc giữa các phương thức:

```
public class IntegerIterator implements Iterator<Integer>
    private Integer nextValue = 0;

    public synchronized boolean hasNext() {
        return nextValue < 100000;
    }
    public synchronized Integer next() {
        if (nextValue == 100000)
            throw new IteratorPastEndException();
        return nextValue++;
    }
    public synchronized Integer getNextValue() {
        return nextValue;
    }
}
```

Đây là đoạn mã dùng `IntegerIterator`:

```
IntegerIterator iterator = new IntegerIterator();
while(iterator.hasNext()) {
    int nextValue = iterator.next();
    // xử lý nextValue
}
```

Nếu chỉ có một luồng thực thi đoạn mã này thì không có vấn đề. Nhưng điều gì xảy ra nếu hai luồng cùng dùng một thực thể `IngeterIterator`, mỗi luồng xử lý các giá trị mình nhận được và mỗi phần tử trong danh sách chỉ được xử lý một lần? Hầu hết thời gian, chẳng có gì xấu xảy ra; hai luồng vui vẻ chia sẻ danh sách, xử lý các phần tử iterator đưa ra rồi dừng khi iterator chạy hết. Tuy nhiên, có một xác suất nhỏ là ở cuối vòng lặp, hai luồng can thiệp lẫn nhau, khiến một luồng vượt quá cuối iterator và ném ngoại lệ.

Vấn đề nằm ở đây: Luồng 1 gọi `hasNext()`, nhận về `true`. Luồng 1 bị tạm dừng, rồi Luồng 2 cũng gọi câu hỏi đó và vẫn nhận về `true`. Luồng 2 gọi `next()`, nhận được một giá trị như dự kiến, nhưng đồng thời làm cho `hasNext()` trả về `false`. Luồng 1 chạy tiếp, vẫn nghĩ `hasNext()` là `true`, rồi gọi `next()`. Dù từng phương thức riêng lẻ đều được đồng bộ hóa, phía client lại gọi hai phương thức.

Đây là vấn đề có thật, cũng là kiểu vấn đề thường xuất hiện trong mã đồng thời. Trong tình huống này, nó đặc biệt tinh vi vì chỉ gây lỗi khi xảy ra ở vòng lặp cuối cùng. Nếu các luồng ngắt quãng đúng vào lúc đó, một luồng có thể vượt quá cuối iterator. Đây là kiểu lỗi chỉ xuất hiện rất lâu sau khi hệ thống đã đưa vào vận hành, và rất khó lần ra.

Bạn có ba lựa chọn:

- Chấp nhận lỗi.
- Thay đổi client để giải quyết vấn đề: khóa phía client.
- Thay đổi server, đồng thời cũng thay đổi client để giải quyết vấn đề: khóa phía server.

Chấp nhận lỗi

Đôi khi có thể sắp đặt mọi thứ sao cho lỗi xảy ra mà không gây hại. Chẳng hạn, client ở trên có thể bắt ngoại lệ rồi dọn dẹp. Thành thật mà nói, cách này hơi cẩu thả, chẳng khác nào xử lý rò rỉ bộ nhớ bằng cách khởi động lại máy vào lúc nửa đêm.

Khóa phía máy khách

Để `IntegerIterator` hoạt động đúng với nhiều luồng, hãy sửa máy khách này (và mọi máy khách khác) như sau:

```
IntegerIterator iterator = new IntegerIterator();

while (true) {
    int nextValue;

    synchronized (iterator) {
        if (!iterator.hasNext())
            break;
        nextValue = iterator.next();
    }
    doSometingWith(nextValue);
}
```

Mỗi máy khách tự khóa bằng từ khóa `synchronized`. Việc lặp lại này vi phạm nguyên tắc DRY, nhưng có thể cần thiết nếu mã sử dụng các công cụ bên thứ ba không an toàn với luồng.

Cách này rủi ro vì mọi lập trình viên dùng máy chủ đều phải nhớ khóa nó trước khi sử dụng và mở khóa khi xong. Nhiều, rất nhiều năm trước, tôi từng làm việc trên một hệ thống dùng kiểu khóa phía máy khách cho một tài nguyên dùng chung. Tài nguyên đó được sử dụng ở hàng trăm chỗ khác nhau trong mã. Một lập trình viên đáng thương đã quên khóa tài nguyên ở một trong những chỗ ấy.

Hệ thống là một máy tính chia sẻ thời gian, nhiều thiết bị đầu cuối, chạy phần mềm kế toán cho Local 705 thuộc công đoàn tài xế xe tải. Máy tính đặt trong phòng có sàn nâng, điều hòa nhiệt độ, cách trụ sở Local 705 khoảng 50 dặm về phía bắc. Tại trụ sở, hàng chục nhân viên nhập liệu gõ các khoản đóng công đoàn vào thiết bị đầu cuối. Các thiết bị kết nối với máy tính qua đường dây điện thoại riêng và modem bán song công 600 bps. (Chuyện này đã lâu lắm rồi.)

Khoảng mỗi ngày một lần, một thiết bị đầu cuối lại "đứng máy". Chẳng có quy luật gì cả. Tình trạng này không ưu tiên thiết bị nào hay thời điểm nào. Cứ như có ai đó gieo xúc xắc để chọn thời điểm và thiết bị sẽ đứng máy. Đôi khi nhiều thiết bị cùng đứng. Có khi vài ngày liền chẳng có chiếc nào bị.

Ban đầu, giải pháp duy nhất là khởi động lại. Nhưng việc khởi động lại rất khó phối hợp. Chúng tôi phải gọi cho trụ sở, yêu cầu mọi người dừng công việc đang làm trên tất cả thiết bị đầu cuối. Sau đó mới có thể tắt rồi khởi động lại máy. Nếu ai đó đang làm việc quan trọng cần một, hai giờ, thiết bị bị treo đành phải nằm đó.

Sau vài tuần gỡ lỗi, chúng tôi phát hiện nguyên nhân là bộ đếm của một bộ đệm vòng

bị lệch so với con trỏ. Bộ đệm này điều khiển đầu ra tới thiết bị đầu cuối. Giá trị con trỏ cho biết bộ đệm trống, nhưng bộ đếm lại nói nó đã đầy. Vì trống nên chẳng có gì để hiển thị, còn vì cũng đang đầy nên không thể thêm gì vào bộ đệm để hiện lên màn hình.

Vậy là chúng tôi biết vì sao các thiết bị bị treo, nhưng chưa biết vì sao bộ đệm vòng lại lệch trạng thái. Chúng tôi bèn thêm một đoạn vá để né vấn đề. Máy tính có thể đọc được các công tắc trên bảng điều khiển phía trước. (Chuyện này còn lâu hơn nữa.) Chúng tôi viết một hàm bẫy nhỏ, phát hiện khi một công tắc bị gạt rồi tìm bộ đệm vòng nào vừa trống vừa đầy. Nếu tìm thấy, hàm đặt lại bộ đệm về trạng thái trống. Thế là các thiết bị đầu cuối bị treo lại hiển thị được.

Giờ thì chúng tôi không cần khởi động lại hệ thống khi thiết bị đầu cuối bị treo. Người của Local chỉ cần gọi báo, rồi chúng tôi đi vào phòng máy tính và gạt một công tắc.

Dĩ nhiên đôi khi họ làm việc vào cuối tuần, còn chúng tôi thì không. Vì vậy, chúng tôi bổ sung một hàm vào bộ lập lịch để kiểm tra tất cả bộ đệm vòng mỗi phút và đặt lại mọi bộ đệm vừa trống vừa đầy. Màn hình được khơi thông trước cả khi Local kịp gọi điện.

Chúng tôi mất thêm vài tuần dò từng trang mã hợp ngữ nguyên khối mới tìm ra thủ phạm. Chúng tôi tính toán và nhận thấy tần suất bị treo phù hợp với việc có một lần dùng bộ đệm vòng mà không khóa. Chỉ cần tìm ra chỗ dùng sai đó. Tiếc là chuyện xảy ra từ quá lâu rồi, chúng tôi không có công cụ tìm kiếm, tham chiếu chéo hay bất kỳ trợ giúp tự động nào khác. Chúng tôi chỉ còn cách dò từng danh sách mã.

Tôi đã học được một bài học quan trọng trong mùa đông lạnh giá ở Chicago năm 1971. Khóa phía máy khách thật tệ.

Khóa phía máy chủ

Có thể bỏ phần mã lặp bằng cách thay đổi `IntegerIterator` như sau:

```
public class IntegerIteratorServerLocked {
    private Integer nextValue = 0;
    public synchronized Integer getNextOrNull() {
        if (nextValue < 100000)
            return nextValue++;
        else
            return null;
    }
}
```

Mã máy khách cũng thay đổi:

```
while (true) {
    Integer nextValue = iterator.getNextOrNull();
    if (next == null)
        break;
    // làm gì đó với nextValue
}
```

Trong trường hợp này, chúng ta thực sự thay đổi API của lớp để hỗ trợ xử lý đồng thời.[3] Máy khách cần kiểm tra `null` thay vì gọi `hasNext()`.

Nói chung, nên ưu tiên khóa phía máy chủ vì những lý do sau:

- Giảm mã lặp: khóa phía máy khách buộc mỗi máy khách phải khóa máy chủ đúng cách. Khi đưa mã khóa vào máy chủ, máy khách có thể dùng đối tượng mà không phải viết thêm mã khóa.
- Cho hiệu năng tốt hơn: khi triển khai đơn luồng, bạn có thể thay máy chủ an toàn với luồng bằng máy chủ không an toàn với luồng để tránh mọi chi phí phụ trội.
- Giảm khả năng xảy ra lỗi: chỉ cần một lập trình viên quên khóa đúng cách là đủ.
- Áp dụng một chính sách duy nhất: chính sách nằm ở một nơi, trong máy chủ, thay vì rải rác ở nhiều nơi, trong từng máy khách.
- Thu hẹp phạm vi biến dùng chung: máy khách không biết đến chúng hay cách khóa chúng. Tất cả đều được giấu trong máy chủ. Khi có sự cố, số chỗ cần kiểm tra cũng ít hơn.

Nếu bạn không sở hữu mã máy chủ thì sao?

- Dùng một ADAPTER để đổi API và thêm khóa:

```
public class ThreadSafeIntegerIterator {
    private IntegerIterator iterator = new IntegerIterator();

    public synchronized Integer getNextOrNull() {
        if(iterator.hasNext())
            return iterator.next();
        return null;
    }
}
```

- Hoặc tốt hơn, dùng các collection an toàn với luồng có giao diện mở rộng.

Tăng thông lượng

Giả sử chúng ta muốn truy cập mạng và đọc nội dung một nhóm trang từ một danh sách URL. Mỗi khi đọc xong một trang, ta sẽ phân tích trang đó để tích lũy một số thống kê. Sau khi đọc hết các trang, ta sẽ in báo cáo tổng hợp.

Lớp sau trả về nội dung của một trang khi nhận URL:

```
public class PageReader {
    //...
    public String getPageFor(String url) {
        HttpMethod method = new GetMethod(url);
        try {
            httpClient.executeMethod(method);
```

3. Thực ra, giao diện `Iterator` vốn không an toàn với luồng. Nó chưa bao giờ được thiết kế để nhiều luồng cùng sử dụng, nên điều này chẳng có gì đáng ngạc nhiên.

```
            String response = method.getResponseBodyAsString();
            return response;
        } catch (Exception e) {
            handle(e);
        } finally {
            method.releaseConnection();
        }
    }
}
```

Lớp tiếp theo là bộ lặp cung cấp nội dung các trang dựa trên một bộ lặp URL:

```
public class PageIterator {
    private PageReader reader;
    private URLIterator urls;

    public PageIterator(PageReader reader, URLIterator urls) {
        this.urls = urls;
        this.reader = reader;
    }

    public synchronized String getNextPageOrNull() {
        if (urls.hasNext())
            getPageFor(urls.next());
        else
            return null;
    }

    public String getPageFor(String url) {
        return reader.getPageFor(url);
    }
}
```

Có thể chia sẻ một thể hiện của `PageIterator` giữa nhiều luồng khác nhau, mỗi luồng dùng thể hiện `PageReader` riêng để đọc và phân tích các trang nhận được từ bộ lặp.

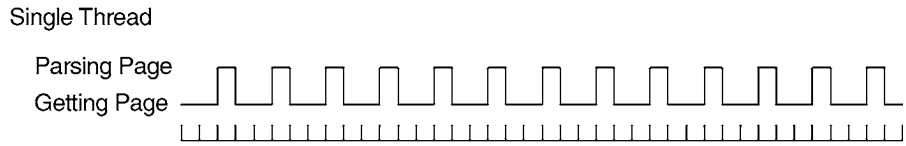
Hãy để ý khối `synchronized` rất nhỏ. Nó chỉ chứa đoạn then chốt bên trong `PageIterator`. Luôn nên đồng bộ hóa ít nhất có thể thay vì nhiều nhất có thể.

Tính thông lượng khi tính toán bằng một luồng

Giờ ta thử tính toán đơn giản. Để tiện lập luận, giả sử:

- Thời gian I/O để lấy một trang (trung bình): 1 giây
- Thời gian xử lý để phân tích trang (trung bình): 0,5 giây
- I/O cần 0% CPU, còn xử lý cần 100% CPU.

Với N trang được xử lý bởi một luồng, tổng thời gian thực thi là 1,5 giây * N. Hình A-1 cho thấy ảnh chụp 13 trang, tương đương khoảng 19,5 giây.

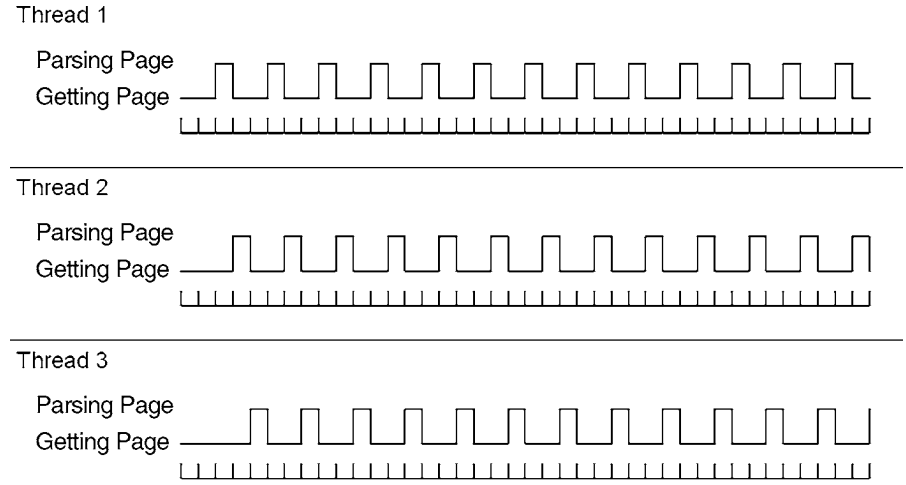


Hình A-1
Một luồng

Tính thông lượng khi dùng nhiều luồng

Nếu có thể lấy các trang theo thứ tự bất kỳ và xử lý độc lập, ta có thể dùng nhiều luồng để tăng thông lượng. Điều gì xảy ra nếu dùng ba luồng? Trong cùng khoảng thời gian, ta lấy được bao nhiêu trang?

Như Hình A-2 cho thấy, giải pháp đa luồng cho phép việc phân tích bị giới hạn bởi xử lý chồng lấp với việc đọc trang bị giới hạn bởi I/O. Trong một thế giới lý tưởng, bộ xử lý được tận dụng tối đa. Mỗi lần đọc trang kéo dài một giây được chồng lấp với hai lần phân tích. Như vậy, ta xử lý được hai trang mỗi giây, gấp ba lần thông lượng của giải pháp đơn luồng.



Hình A-2
Ba luồng chạy đồng thời

Bế tắc

Hãy hình dung một ứng dụng Web có hai nhóm tài nguyên dùng chung, mỗi nhóm có số lượng hữu hạn:

- Nhóm kết nối cơ sở dữ liệu để lưu trữ cục bộ công việc đang xử lý
- Nhóm kết nối MQ tới kho lưu trữ chính

Giả sử ứng dụng có hai thao tác: tạo mới và cập nhật.

- Tạo mới: lấy kết nối tới kho lưu trữ chính và cơ sở dữ liệu. Giao tiếp với dịch vụ kho lưu trữ chính, sau đó lưu công việc vào cơ sở dữ liệu xử lý cục bộ.
- Cập nhật: lấy kết nối cơ sở dữ liệu rồi tới kho lưu trữ chính. Đọc từ cơ sở dữ liệu xử lý công việc rồi gửi tới kho lưu trữ chính.

Điều gì xảy ra khi số người dùng nhiều hơn số kết nối trong nhóm? Giả sử mỗi nhóm có mười kết nối.

- Mười người dùng thực hiện thao tác tạo mới, lấy hết mười kết nối cơ sở dữ liệu. Mỗi luồng bị ngắt sau khi lấy kết nối cơ sở dữ liệu nhưng trước khi lấy kết nối tới kho lưu trữ chính.
- Mười người dùng thực hiện thao tác cập nhật, lấy hết mười kết nối tới kho lưu trữ chính. Mỗi luồng bị ngắt sau khi lấy kết nối tới kho lưu trữ chính nhưng trước khi lấy kết nối cơ sở dữ liệu.
- Giờ mười luồng "tạo mới" phải chờ kết nối tới kho lưu trữ chính, còn mười luồng "cập nhật" phải chờ kết nối cơ sở dữ liệu.
- Bế tắc. Hệ thống không bao giờ tự phục hồi.

Tình huống này nghe có vẻ khó xảy ra, nhưng ai muốn một hệ thống cứ cách tuần lại đóng băng một lần? Ai muốn gỡ lỗi một hệ thống có triệu chứng khó tái hiện đến thế? Đây là kiểu vấn đề xuất hiện trong thực tế rồi mất hàng tuần mới giải quyết được.

Một "giải pháp" thường thấy là thêm các câu lệnh gỡ lỗi để tìm hiểu chuyện gì đang xảy ra. Dĩ nhiên, các câu lệnh gỡ lỗi làm thay đổi mã đủ nhiều để bế tắc xảy ra trong tình huống khác và phải mất hàng tháng mới tái diễn.[4]

Muốn thực sự giải quyết bế tắc, ta cần hiểu nguyên nhân gây ra nó. Bế tắc cần đủ bốn điều kiện sau:

- Loại trừ tương hỗ
- Khóa và chờ
- Không thể thu hồi

4. Chẳng hạn, có người thêm đầu ra gỡ lỗi rồi vấn đề "biến mất". Mã gỡ lỗi đã "sửa" vấn đề, khiến nó vẫn còn trong hệ thống.

- Chờ vòng tròn

Loại trừ tương hỗ

Loại trừ tương hỗ xảy ra khi nhiều luồng cần dùng cùng một tài nguyên, nhưng tài nguyên đó:

- Không thể được nhiều luồng sử dụng cùng lúc.
- Có số lượng giới hạn.

Ví dụ thường gặp về loại tài nguyên này là kết nối cơ sở dữ liệu, tệp đang mở để ghi, khóa bản ghi hoặc semaphore.

Khóa và chờ

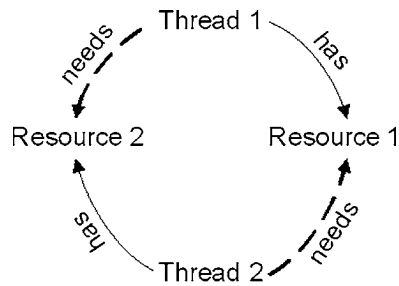
Sau khi lấy được một tài nguyên, luồng sẽ không giải phóng nó cho đến khi đã lấy được mọi tài nguyên khác cần dùng và hoàn thành công việc.

Không thể thu hồi

Một luồng không thể lấy tài nguyên khỏi luồng khác. Khi một luồng đang giữ tài nguyên, cách duy nhất để luồng khác lấy được nó là chờ luồng đang giữ giải phóng.

Chờ vòng tròn

Hiện tượng này còn được gọi là vòng ôm chết người. Hãy hình dung hai luồng T1 và T2 cùng hai tài nguyên R1 và R2. T1 giữ R1, T2 giữ R2. T1 cũng cần R2, còn T2 cũng cần R1. Tình huống sẽ giống Hình A-3:



Hình A-3

Cả bốn điều kiện này phải đồng thời tồn tại thì mới có thể xảy ra bế tắc. Phá vỡ bất kỳ điều kiện nào cũng khiến bế tắc không thể xảy ra.

Phá vỡ điều kiện loại trừ tương hỗ

Một cách tránh bế tắc là né điều kiện loại trừ tương hỗ. Bạn có thể làm vậy bằng cách:

- Dùng tài nguyên cho phép nhiều bên sử dụng đồng thời, chẳng hạn `AtomicInteger`.
- Tăng số lượng tài nguyên để bằng hoặc vượt số luồng tranh dùng.
- Kiểm tra tất cả tài nguyên đều rảnh trước khi chiếm lấy bất kỳ tài nguyên nào.

Tiếc là hầu hết tài nguyên đều có số lượng giới hạn và không cho phép dùng đồng thời. Cũng không hiếm trường hợp danh tính của tài nguyên thứ hai phụ thuộc vào kết quả thao tác trên tài nguyên thứ nhất. Nhưng đừng nản, vẫn còn ba điều kiện nữa.

Phá vỡ điều kiện khóa và chờ

Bạn cũng có thể loại bỏ bế tắc bằng cách không chờ. Hãy kiểm tra từng tài nguyên trước khi chiếm lấy nó; nếu gặp tài nguyên đang bận, giải phóng mọi tài nguyên rồi bắt đầu lại.

Cách này có thể dẫn đến một số vấn đề:

- Đói tài nguyên: một luồng liên tục không lấy được tài nguyên cần thiết (có thể do nó cần một tổ hợp tài nguyên riêng hiếm khi cùng rảnh).
- Livelock: nhiều luồng có thể rơi vào cùng nhịp, mỗi luồng lấy một tài nguyên rồi lại nhả ra, lặp đi lặp lại. Điều này đặc biệt dễ xảy ra với các thuật toán lập lịch CPU đơn giản (hãy nghĩ đến thiết bị nhúng hoặc thuật toán cân bằng luồng tự viết đơn giản).

Cả hai vấn đề đều có thể làm thông lượng kém. Vấn đề thứ nhất khiến CPU ít được sử dụng, còn vấn đề thứ hai khiến CPU hoạt động nhiều nhưng vô ích.

Cách này kém hiệu quả thật, nhưng vẫn hơn không có gì. Ưu điểm là hầu như lúc nào cũng có thể triển khai nếu mọi cách khác đều thất bại.

Phá vỡ điều kiện không thể thu hồi

Một cách khác để tránh bế tắc là cho phép các luồng lấy tài nguyên khỏi luồng khác. Việc này thường thực hiện qua một cơ chế yêu cầu đơn giản. Khi phát hiện tài nguyên đang bận, một luồng yêu cầu chủ sở hữu giải phóng nó. Nếu chủ sở hữu cũng đang chờ tài nguyên khác, nó giải phóng tất cả rồi bắt đầu lại.

Cách này tương tự cách trước nhưng có ưu điểm là luồng vẫn được phép chờ tài nguyên. Nhờ đó, số lần phải bắt đầu lại giảm xuống. Tuy nhiên, hãy lưu ý rằng quản lý tất cả các yêu cầu ấy có thể rất phức tạp.

Phá vỡ điều kiện chờ vòng tròn

Đây là cách phổ biến nhất để ngăn bế tắc. Với hầu hết hệ thống, chỉ cần mọi bên thống nhất một quy ước đơn giản.

Trong ví dụ trên, Luồng 1 cần cả Tài nguyên 1 và Tài nguyên 2, còn Luồng 2 cần Tài nguyên 2 rồi Tài nguyên 1. Chỉ cần buộc cả hai luồng cấp phát tài nguyên theo cùng thứ tự là không thể xảy ra chờ vòng tròn.

Tổng quát hơn, nếu mọi luồng thống nhất được thứ tự toàn cục của các tài nguyên và đều cấp phát theo thứ tự ấy thì bế tắc không thể xảy ra. Cũng như các chiến lược khác, cách này có thể gây ra vấn đề:

- Thứ tự lấy tài nguyên có thể không khớp với thứ tự sử dụng, nên tài nguyên lấy từ đầu có thể đến cuối mới được dùng. Khi ấy tài nguyên có thể bị khóa lâu hơn mức cần thiết.
- Đôi khi không thể áp đặt thứ tự lấy tài nguyên. Nếu mã định danh của tài nguyên thứ hai được xác định bởi thao tác thực hiện trên tài nguyên thứ nhất thì không thể sắp thứ tự.

Có nhiều cách tránh bế tắc. Một số dẫn đến đói tài nguyên, số khác dùng CPU nhiều và làm giảm khả năng phản hồi. TANSTAAFL![5]

Tách riêng phần liên quan đến luồng trong giải pháp để có thể điều chỉnh và thử nghiệm là cách hữu hiệu giúp hiểu rõ hơn, từ đó xác định chiến lược phù hợp nhất.

Kiểm thử mã đa luồng

Làm thế nào viết một kiểm thử để chứng minh đoạn mã sau bị lỗi?

Đoạn mã A-2

```
01: public class ClassWithThreadingProblem {
02:     int nextId;
03:
04:     public int takeNextId() {
05:         return nextId++;
06:     }
07:}
```

Sau đây là mô tả một kiểm thử sẽ chứng minh mã bị lỗi:

- Ghi nhớ giá trị hiện tại của `nextId`.
- Tạo hai luồng, cả hai đều gọi `takeNextId()` một lần.
- Xác minh `nextId` lớn hơn giá trị ban đầu hai đơn vị.
- Chạy cho đến khi chứng minh được `nextId` chỉ tăng một thay vì hai.

Đoạn mã A-2 cho thấy một kiểm thử như vậy:

5. Không có bữa trưa nào miễn phí.

Đoạn mã A-2
ClassWithThreadingProblemTest.java

```
01: package example;
02:
03: import static org.junit.Assert.fail;
04:
05: import org.junit.Test;
06:
07: public class ClassWithThreadingProblemTest {
08:     @Test
09:     public void twoThreadsShouldFailEventually() throws Exception {
10:         final ClassWithThreadingProblem classWithThreadingProblem
                    = new ClassWithThreadingProblem();
11:
12:         Runnable runnable = new Runnable() {
13:             public void run() {
14:                 classWithThreadingProblem.takeNextId();
15:             }
16:         };
17:
18:         for (int i = 0; i < 50000; ++i) {
19:             int startingId = classWithThreadingProblem.lastId;
20:             int expectedResult = 2 + startingId;
21:
22:             Thread t1 = new Thread(runnable);
23:             Thread t2 = new Thread(runnable);
24:             t1.start();
25:             t2.start();
26:             t1.join();
27:             t2.join();
28:
29:             int endingId = classWithThreadingProblem.lastId;
30:
31:             if (endingId != expectedResult)
32:                 return;
33:         }
34:
35:         fail("Should have exposed a threading issue but it did not.");
36:     }
37: }
```

Dòng	Mô tả
10	Tạo một thể hiện duy nhất của `ClassWithThreadingProblem`. Lưu ý, ta phải dùng từ khóa `final` vì sẽ sử dụng nó bên dưới trong một lớp bên trong vô danh.
12–16	Tạo một lớp bên trong vô danh sử dụng thể hiện duy nhất của `ClassWithThreadingProblem`.
18	Chạy đoạn mã "đủ nhiều" lần để chứng minh mã thất bại, nhưng không nhiều đến mức kiểm thử "chạy quá lâu". Đây là một phép cân bằng; ta không muốn chờ quá lâu mới thấy lỗi. Chọn con số này rất khó, dù sau này ta sẽ thấy có thể giảm đáng kể con số đó.
19	Ghi nhớ giá trị ban đầu. Kiểm thử này nhằm chứng minh mã trong `ClassWithThreadingProblem` bị lỗi. Nếu kiểm thử vượt qua, nó đã chứng minh mã bị lỗi. Nếu kiểm thử thất bại, nó chưa chứng minh được mã bị lỗi.
20	Ta kỳ vọng giá trị cuối cùng lớn hơn giá trị hiện tại hai đơn vị.
22–23	Tạo hai luồng, cả hai đều dùng đối tượng ta tạo ở các dòng 12–16. Như vậy, hai luồng có thể cùng cố dùng một thể hiện `ClassWithThreadingProblem` và gây nhiễu lẫn nhau.
24–25	Cho phép hai luồng có thể chạy.
26–27	Chờ cả hai luồng chạy xong rồi mới kiểm tra kết quả.
29	Ghi lại giá trị cuối cùng thực tế.
31–32	`endingId` có khác giá trị ta mong đợi không? Nếu có, kết thúc kiểm thử, ta đã chứng minh mã bị lỗi. Nếu không, thử lại.
35	Nếu chạy đến đây, kiểm thử chưa chứng minh được mã sản phẩm bị lỗi trong khoảng thời gian "hợp lý"; kiểm thử đã thất bại. Hoặc mã không lỗi, hoặc ta chưa chạy đủ số vòng lặp để điều kiện lỗi xảy ra.

Kiểm thử này chắc chắn tạo ra các điều kiện để xảy ra vấn đề cập nhật đồng thời. Tuy nhiên, vấn đề hiếm khi xảy ra nên phần lớn lần chạy, kiểm thử sẽ không phát hiện được.

Thật vậy, để phát hiện vấn đề, ta cần đặt số vòng lặp vượt quá một triệu. Ngay cả vậy, trong mười lần chạy với một triệu vòng lặp, vấn đề chỉ xuất hiện một lần. Điều đó có nghĩa là có lẽ ta phải đặt số vòng lặp cao hơn nhiều so với một trăm triệu để có thể tin cậy phát hiện lỗi. Ta sẵn sàng chờ bao lâu?

Ngay cả khi điều chỉnh kiểm thử để phát hiện lỗi ổn định trên một máy, có lẽ ta sẽ phải điều chỉnh lại với các giá trị khác để chứng minh lỗi trên máy, hệ điều hành hoặc phiên bản JVM khác.

Đây là một vấn đề đơn giản. Nếu không dễ chứng minh mã lỗi với vấn đề này, làm sao ta phát hiện được những vấn đề thực sự phức tạp?

Vậy có thể dùng cách nào để chứng minh lỗi đơn giản này? Quan trọng hơn, làm thế nào viết kiểm thử chứng minh được lỗi trong mã phức tạp hơn? Làm sao phát hiện lỗi trong mã khi không biết phải tìm ở đâu?

Sau đây là vài ý tưởng:

- Kiểm thử Monte Carlo. Viết kiểm thử linh hoạt để có thể điều chỉnh. Sau đó chạy đi chạy lại, chẳng hạn trên máy chủ kiểm thử, đồng thời thay đổi ngẫu nhiên các giá trị điều chỉnh. Nếu kiểm thử thất bại lần nào thì mã có lỗi. Hãy bắt đầu viết những kiểm thử này sớm để máy chủ tích hợp liên tục sớm bắt đầu chạy chúng. Nhân tiện, hãy ghi nhật ký cẩn thận các điều kiện khiến kiểm thử thất bại.
- Chạy kiểm thử trên từng nền tảng triển khai mục tiêu. Lặp đi lặp lại. Liên tục. Kiểm thử chạy càng lâu mà không thất bại thì càng có khả năng:
  - Mã sản phẩm đúng, hoặc

  - Kiểm thử chưa đủ để phát hiện vấn đề.
- Chạy kiểm thử trên máy có tải thay đổi. Nếu có thể mô phỏng tải gần với môi trường sản xuất thì hãy làm.

Tuy vậy, ngay cả khi làm tất cả những việc này, khả năng phát hiện vấn đề luồng trong mã vẫn không cao. Những vấn đề khó chịu nhất là loại có xác suất cực thấp, chỉ xảy ra một lần trong một tỷ cơ hội. Chúng là nỗi kinh hoàng của các hệ thống phức tạp.

Công cụ hỗ trợ kiểm thử mã dựa trên luồng

IBM đã tạo ra một công cụ tên là ConTest.[6] Công cụ này bổ sung mã đo đạc vào các lớp để tăng khả năng mã không an toàn với luồng bộc lộ lỗi.

Chúng tôi không có quan hệ trực tiếp nào với IBM hay nhóm phát triển ConTest. Một đồng nghiệp đã giới thiệu công cụ này. Chúng tôi nhận thấy khả năng tìm ra vấn đề luồng cải thiện rõ rệt chỉ sau vài phút sử dụng.

Sau đây là các bước sử dụng ConTest:

- Viết kiểm thử và mã sản phẩm, bảo đảm có những kiểm thử được thiết kế riêng để mô phỏng nhiều người dùng dưới tải thay đổi, như đã nói ở trên.
- Bổ sung mã đo đạc cho mã kiểm thử và mã sản phẩm bằng ConTest.
- Chạy kiểm thử.

Khi bổ sung mã đo đạc bằng ConTest, tỷ lệ phát hiện lỗi của chúng tôi tăng từ khoảng một lần trong mười triệu vòng lặp lên khoảng một lần trong ba mươi vòng lặp. Sau đây là số vòng lặp của vài lần chạy kiểm thử sau khi bổ sung mã đo đạc: 13, 23, 0, 54, 16, 14, 6, 69, 107, 49, 2. Rõ ràng, các lớp đã được bổ sung mã đo đạc thất bại sớm hơn nhiều và đáng tin cậy hơn hẳn.

Kết luận

Chương này chỉ là một chuyến đi rất ngắn qua vùng đất rộng lớn và hiểm trở của lập trình đồng thời. Chúng tôi mới chỉ chạm đến bề mặt. Ở đây, chúng tôi tập trung vào các nguyên tắc giúp giữ mã xử lý đồng thời sạch sẽ, nhưng nếu định viết các hệ thống xử lý đồng thời thì bạn còn nhiều điều phải học. Chúng tôi khuyên bạn bắt đầu với cuốn sách tuyệt vời của Doug Lea, *Concurrent Programming in Java: Design Principles and Patterns*.[7]

Trong chương này, chúng tôi nói về cập nhật đồng thời, cùng các nguyên tắc đồng bộ hóa và khóa sạch giúp ngăn chặn vấn đề đó. Chúng tôi nói về cách luồng có thể tăng thông lượng của hệ thống bị giới hạn bởi I/O, đồng thời trình bày những kỹ thuật sạch để đạt được cải thiện ấy. Chúng tôi nói về bế tắc và các nguyên tắc ngăn chặn nó một cách sạch sẽ. Cuối cùng, chúng tôi nói về các chiến lược phát hiện vấn đề đồng thời bằng cách bổ sung mã đo

6. `http://www.haifa.ibm.com/projects/verification/contest/index.html`

7. Xem [Lea99], trang 191.

đạc vào mã của bạn.

Hướng dẫn: Ví dụ mã đầy đủ

Máy khách/máy chủ không dùng luồng

Đoạn mã A-3
Server.java

```
package com.objectmentor.clientserver.nonthreaded;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketException;

import common.MessageUtils;
public class Server implements Runnable {
    ServerSocket serverSocket;
    volatile boolean keepProcessing = true;

    public Server(int port, int millisecondsTimeout) throws IOException {
        serverSocket = new ServerSocket(port);
        serverSocket.setSoTimeout(millisecondsTimeout);
    }

    public void run() {
        System.out.printf("Server Starting\n");
        while (keepProcessing) {
            try {
                System.out.printf("accepting client\n");
                Socket socket = serverSocket.accept();
                System.out.printf("got client\n");
                process(socket);
            } catch (Exception e) {
                handle(e);
            }
        }
    }
    private void handle(Exception e) {
        if (!(e instanceof SocketException)) {
            e.printStackTrace();
        }
    }

    public void stopProcessing() {
        keepProcessing = false;
        closeIgnoringException(serverSocket);
    }
```

Đoạn mã A-3 (tiếp theo)
Server.java

```
    void process(Socket socket) {
        if (socket == null)
            return;

        try {
            System.out.printf("Server: getting message\n");
            String message = MessageUtils.getMessage(socket);
            System.out.printf("Server: got message: %s\n", message);
            Thread.sleep(1000);
            System.out.printf("Server: sending reply: %s\n", message);
            MessageUtils.sendMessage(socket, "Processed: " + message);
            System.out.printf("Server: sent\n");
            closeIgnoringException(socket);
        } catch (Exception e) {
            e.printStackTrace();
        }

    }
    private void closeIgnoringException(Socket socket) {
        if (socket != null)
            try {
                socket.close();
            } catch (IOException ignore) {
            }
    }

    private void closeIgnoringException(ServerSocket serverSocket) {
        if (serverSocket != null)
            try {
                serverSocket.close();
            } catch (IOException ignore) {
            }
    }
}
```

Đoạn mã A-4
ClientTest.java

```
package com.objectmentor.clientserver.nonthreaded;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketException;

import common.MessageUtils;

public class Server implements Runnable {
    ServerSocket serverSocket;
    volatile boolean keepProcessing = true;
```

Đoạn mã A-4 (tiếp theo)
ClientTest.java

```
    public Server(int port, int millisecondsTimeout) throws IOException {
        serverSocket = new ServerSocket(port);
        serverSocket.setSoTimeout(millisecondsTimeout);
    }

    public void run() {
        System.out.printf("Server Starting\n");

        while (keepProcessing) {
            try {
                System.out.printf("accepting client\n");
                Socket socket = serverSocket.accept();
                System.out.printf("got client\n");
                process(socket);
            } catch (Exception e) {
                handle(e);
            }
        }
    }

    private void handle(Exception e) {
        if (!(e instanceof SocketException)) {
            e.printStackTrace();
        }
    }

    public void stopProcessing() {
        keepProcessing = false;
        closeIgnoringException(serverSocket);
    }

    void process(Socket socket) {
        if (socket == null)
            return;

        try {
            System.out.printf("Server: getting message\n");
            String message = MessageUtils.getMessage(socket);
            System.out.printf("Server: got message: %s\n", message);
            Thread.sleep(1000);
            System.out.printf("Server: sending reply: %s\n", message);
            MessageUtils.sendMessage(socket, "Processed: " + message);
            System.out.printf("Server: sent\n");
            closeIgnoringException(socket);
        } catch (Exception e) {
            e.printStackTrace();
        }

    }

    private void closeIgnoringException(Socket socket) {
        if (socket != null)
            try {
                socket.close();
```

```
            } catch (IOException ignore) {
            }
    }
    private void closeIgnoringException(ServerSocket serverSocket) {
        if (serverSocket != null)
            try {
                serverSocket.close();
            } catch (IOException ignore) {
            }
    }
}
```

Đoạn mã A-5
MessageUtils.java

```
package common;

import java.io.IOException;
import java.io.InputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.OutputStream;
import java.net.Socket;
public class MessageUtils {
    public static void sendMessage(Socket socket, String message)
            throws IOException {
        OutputStream stream = socket.getOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(stream);
        oos.writeUTF(message);
        oos.flush();
    }

    public static String getMessage(Socket socket) throws IOException {
        InputStream stream = socket.getInputStream();
        ObjectInputStream ois = new ObjectInputStream(stream);
        return ois.readUTF();
    }
}
```

Máy khách/máy chủ dùng luồng

Chuyển server sang dùng luồng chỉ cần thay đổi phương thức xử lý thông điệp (các dòng mới được in đậm để dễ nhận ra):

```
void process(final Socket socket) {
    if (socket == null)
        return;

    Runnable clientHandler = new Runnable() {
        public void run() {
            try {
                System.out.printf("Server: getting message\n");
                String message = MessageUtils.getMessage(socket);
```

```
                    System.out.printf("Server: got message: %s\n", message);
                    Thread.sleep(1000);
                    System.out.printf("Server: sending reply: %s\n", message);
                    MessageUtils.sendMessage(socket, "Processed: " + message);
                    System.out.printf("Server: sent\n");
                    closeIgnoringException(socket);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        };

        Thread clientConnection = new Thread(clientHandler);
        clientConnection.start();
    }
```

Phụ lục B

org.jfree.date.SerialDate

Đoạn mã B-1
SerialDate.Java

```
 1 /*
   ===========================================================================
 2 * JCommon : a free general purpose class library for the Java(tm)
   platform
 3 *
   ===========================================================================
 4 *
 5 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
 6 *
 7 * Project Info: http://www.jfree.org/jcommon/index.html
 8 *
 9 * This library is free software; you can redistribute it and/or modify
   it
10 * under the terms of the GNU Lesser General Public License as published
   by
11 * the Free Software Foundation; either version 2.1 of the License, or
12 * (at your option) any later version.
13 *
14 * This library is distributed in the hope that it will be useful, but
15 * WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY
16 * or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public
17 * License for more details.
18 *
19 * You should have received a copy of the GNU Lesser General Public
20 * License along with this library; if not, write to the Free Software
21 * Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
   02110-1301,
22 * USA.
23 *
24 * [Java is a trademark or registered trademark of Sun Microsystems, Inc.
25 * in the United States and other countries.]
```

```
26 *
27 * ---------------
28 * SerialDate.java
29 * ---------------
30 * (C) Copyright 2001-2005, by Object Refinery Limited.
31 *
32 * Original Author: David Gilbert (for Object Refinery Limited);
33 * Contributor(s): -;
34 *
35 * $Id: SerialDate.java,v 1.7 2005/11/03 09:25:17 mungady Exp $
36 *
37 * Changes (from 11-Oct-2001)
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
38 * --------------------------
39 * 11-Oct-2001 : Re-organised the class and moved it to new package
40 *               com.jrefinery.date (DG);
41 * 05-Nov-2001 : Added a getDescription() method, and eliminated
  NotableDate
42 *               class (DG);
43 * 12-Nov-2001 : IBD requires setDescription() method, now that
  NotableDate
44 *               class is gone (DG); Changed getPreviousDayOfWeek(),
45 *               getFollowingDayOfWeek() and getNearestDayOfWeek() to
  correct
46 *               bugs (DG);
47 * 05-Dec-2001 : Fixed bug in SpreadsheetDate class (DG);
48 * 29-May-2002 : Moved the month constants into a separate interface
49 *               (MonthConstants) (DG);
50 * 27-Aug-2002 : Fixed bug in addMonths() method, thanks to N???levka
  Petr (DG);
51 * 03-Oct-2002 : Fixed errors reported by Checkstyle (DG);
52 * 13-Mar-2003 : Implemented Serializable (DG);
53 * 29-May-2003 : Fixed bug in addMonths method (DG);
54 * 04-Sep-2003 : Implemented Comparable. Updated the isInRange javadocs
  (DG);
55 * 05-Jan-2005 : Fixed bug in addYears() method (1096282) (DG);
56 *
57 */
58
59 package org.jfree.date;
60
61 import java.io.Serializable;
62 import java.text.DateFormatSymbols;
63 import java.text.SimpleDateFormat;
64 import java.util.Calendar;
65 import java.util.GregorianCalendar;
66
67 /**
68 * An abstract class that defines our requirements for manipulating
  dates,
69 * without tying down a particular implementation.
70 * <P>
71 * Requirement 1 : match at least what Excel does for dates;
```

```
 72 * Requirement 2 : class is immutable;
 73 * <P>
 74 * Why not just use java.util.Date? We will, when it makes sense. At
   times,
 75 * java.util.Date can be *too* precise - it represents an instant in
   time,
 76 * accurate to 1/1000th of a second (with the date itself depending on
   the
 77 * time-zone). Sometimes we just want to represent a particular day (e.g.
   21
 78 * January 2015) without concerning ourselves about the time of day, or
   the
 79 * time-zone, or anything else. That's what we've defined SerialDate for.
 80 * <P>
 81 * You can call getInstance() to get a concrete subclass of SerialDate,
 82 * without worrying about the exact implementation.
 83 *
 84 * @author David Gilbert
 85 */
 86 public abstract class SerialDate implements Comparable,
 87                                              Serializable,
 88                                              MonthConstants {
 89
 90     /** For serialization. */
 91     private static final long serialVersionUID = -293716040467423637L;
 92
 93     /** Date format symbols. */
 94     public static final DateFormatSymbols
 95         DATE_FORMAT_SYMBOLS = new
   SimpleDateFormat().getDateFormatSymbols();
 96
 97     /** The serial number for 1 January 1900. */
 98     public static final int SERIAL_LOWER_BOUND = 2;
 99
100     /** The serial number for 31 December 9999. */
101     public static final int SERIAL_UPPER_BOUND = 2958465;
102
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
103      /** The lowest year value supported by this date format. */
104      public static final int MINIMUM_YEAR_SUPPORTED = 1900;
105
106      /** The highest year value supported by this date format. */
107      public static final int MAXIMUM_YEAR_SUPPORTED = 9999;
108
109      /** Useful constant for Monday. Equivalent to
   java.util.Calendar.MONDAY. */
110      public static final int MONDAY = Calendar.MONDAY;
111
112      /**
113       * Useful constant for Tuesday. Equivalent to
   java.util.Calendar.TUESDAY.
114       */
115      public static final int TUESDAY = Calendar.TUESDAY;
```

```
116
117      /**
118       * Useful constant for Wednesday. Equivalent to
119       * java.util.Calendar.WEDNESDAY.
120       */
121      public static final int WEDNESDAY = Calendar.WEDNESDAY;
122
123      /**
124       * Useful constant for Thrusday. Equivalent to
   java.util.Calendar.THURSDAY.
125       */
126      public static final int THURSDAY = Calendar.THURSDAY;
127
128      /** Useful constant for Friday. Equivalent to
   java.util.Calendar.FRIDAY. */
129      public static final int FRIDAY = Calendar.FRIDAY;
130
131      /**
132       * Useful constant for Saturday. Equivalent to
   java.util.Calendar.SATURDAY.
133       */
134      public static final int SATURDAY = Calendar.SATURDAY;
135
136      /** Useful constant for Sunday. Equivalent to
   java.util.Calendar.SUNDAY. */
137      public static final int SUNDAY = Calendar.SUNDAY;
138
139      /** The number of days in each month in non leap years. */
140      static final int[] LAST_DAY_OF_MONTH =
141          {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
142
143      /** The number of days in a (non-leap) year up to the end of each
   month. */
144      static final int[] AGGREGATE_DAYS_TO_END_OF_MONTH =
145          {0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365};
146
147      /** The number of days in a year up to the end of the preceding
   month. */
148      static final int[] AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH =
149          {0, 0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334,
   365};
150
151      /** The number of days in a leap year up to the end of each month.
   */
152      static final int[] LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_MONTH =
153          {0, 31, 60, 91, 121, 152, 182, 213, 244, 274, 305, 335, 366};
154
155      /**
156       * The number of days in a leap year up to the end of the preceding
   month.
157       */
158      static final int[]
159          LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH =
160              {0, 0, 31, 60, 91, 121, 152, 182, 213, 244, 274, 305, 335,
   366};
161
162      /** A useful constant for referring to the first week in a month.
   */
```

```
163      public static final int FIRST_WEEK_IN_MONTH = 1;
164
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
165      /** A useful constant for referring to the second week in a month.
   */
166      public static final int SECOND_WEEK_IN_MONTH = 2;
167
168      /** A useful constant for referring to the third week in a month.
   */
169      public static final int THIRD_WEEK_IN_MONTH = 3;
170
171      /** A useful constant for referring to the fourth week in a month.
   */
172      public static final int FOURTH_WEEK_IN_MONTH = 4;
173
174      /** A useful constant for referring to the last week in a month. */
175      public static final int LAST_WEEK_IN_MONTH = 0;
176
177      /** Useful range constant. */
178      public static final int INCLUDE_NONE = 0;
179
180      /** Useful range constant. */
181      public static final int INCLUDE_FIRST = 1;
182
183      /** Useful range constant. */
184      public static final int INCLUDE_SECOND = 2;
185
186      /** Useful range constant. */
187      public static final int INCLUDE_BOTH = 3;
188
189      /**
190       * Useful constant for specifying a day of the week relative to a
   fixed
191       * date.
192       */
193      public static final int PRECEDING = -1;
194
195      /**
196       * Useful constant for specifying a day of the week relative to a
   fixed
197       * date.
198       */
199      public static final int NEAREST = 0;
200
201      /**
202       * Useful constant for specifying a day of the week relative to a
   fixed
203       * date.
204       */
205      public static final int FOLLOWING = 1;
206
207      /** A description for the date. */
208      private String description;
```

```
209
210      /**
211        * Default constructor.
212        */
213      protected SerialDate() {
214      }
215
216      /**
217       * Returns <code>true</code> if the supplied integer code
   represents a
218       * valid day-of-the-week, and <code>false</code> otherwise.
219       *
220       * @param code the code being checked for validity.
221       *
222       * @return <code>true</code> if the supplied integer code
   represents a
223       *         valid day-of-the-week, and <code>false</code> otherwise.
224       */
225      public static boolean isValidWeekdayCode(final int code) {
226
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
227            switch(code) {
228                case SUNDAY:
229                case MONDAY:
230                case TUESDAY:
231                case WEDNESDAY:
232                case THURSDAY:
233                case FRIDAY:
234                case SATURDAY:
235                    return true;
236                default:
237                    return false;
238            }
239
240      }
241
242      /**
243       * Converts the supplied string to a day of the week.
244       *
245       * @param s a string representing the day of the week.
246       *
247       * @return <code>-1</code> if the string is not convertable, the
   day of
248       *         the week otherwise.
249       */
250      public static int stringToWeekdayCode(String s) {
251
252            final String[] shortWeekdayNames
253                = DATE_FORMAT_SYMBOLS.getShortWeekdays();
254            final String[] weekDayNames =
   DATE_FORMAT_SYMBOLS.getWeekdays();
255
256            int result = -1;
```

```
257             s = s.trim();
258             for (int i = 0; i < weekDayNames.length; i++) {
259                 if (s.equals(shortWeekdayNames[i])) {
260                     result = i;
261                     break;
262                 }
263                 if (s.equals(weekDayNames[i])) {
264                     result = i;
265                     break;
266                 }
267             }
268             return result;
269
270       }
271
272       /**
273        * Returns a string representing the supplied day-of-the-week.
274        * <P>
275        * Need to find a better approach.
276        *
277        * @param weekday the day of the week.
278        *
279        * @return a string representing the supplied day-of-the-week.
280        */
281       public static String weekdayCodeToString(final int weekday) {
282
283             final String[] weekdays = DATE_FORMAT_SYMBOLS.getWeekdays();
284             return weekdays[weekday];
285
286       }
287
288       /**
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
289        * Returns an array of month names.
290        *
291        * @return an array of month names.
292        */
293       public static String[] getMonths() {
294
295           return getMonths(false);
296
297       }
298
299       /**
300        * Returns an array of month names.
301        *
302        * @param shortened a flag indicating that shortened month names
   should
303        *                  be returned.
304        *
305        * @return an array of month names.
306        */
307       public static String[] getMonths(final boolean shortened) {
```

```
308
309          if (shortened) {
310              return DATE_FORMAT_SYMBOLS.getShortMonths();
311          }
312          else {
313              return DATE_FORMAT_SYMBOLS.getMonths();
314          }
315
316      }
317
318      /**
319       * Returns true if the supplied integer code represents a valid
   month.
320       *
321       * @param code the code being checked for validity.
322       *
323       * @return <code>true</code> if the supplied integer code
   represents a
324       *         valid month.
325       */
326      public static boolean isValidMonthCode(final int code) {
327
328          switch(code) {
329              case JANUARY:
330              case FEBRUARY:
331              case MARCH:
332              case APRIL:
333              case MAY:
334              case JUNE:
335              case JULY:
336              case AUGUST:
337              case SEPTEMBER:
338              case OCTOBER:
339              case NOVEMBER:
340              case DECEMBER:
341                  return true;
342              default:
343                  return false;
344          }
345
346      }
347
348      /**
349       * Returns the quarter for the specified month.
350       *
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
351       * @param code the month code (1-12).
352       *
353       * @return the quarter that the month belongs to.
354       * @throws java.lang.IllegalArgumentException
355       */
356      public static int monthCodeToQuarter(final int code) {
357
```

```
358          switch(code) {
359              case JANUARY:
360              case FEBRUARY:
361              case MARCH: return 1;
362              case APRIL:
363              case MAY:
364              case JUNE: return 2;
365              case JULY:
366              case AUGUST:
367              case SEPTEMBER: return 3;
368              case OCTOBER:
369              case NOVEMBER:
370              case DECEMBER: return 4;
371              default: throw new IllegalArgumentException(
372                  "SerialDate.monthCodeToQuarter: invalid month code.");
373          }
374
375      }
376
377      /**
378       * Returns a string representing the supplied month.
379       * <P>
380       * The string returned is the long form of the month name taken
    from the
381       * default locale.
382       *
383       * @param month the month.
384       *
385       * @return a string representing the supplied month.
386       */
387      public static String monthCodeToString(final int month) {
388
389          return monthCodeToString(month, false);
390
391      }
392
393      /**
394       * Returns a string representing the supplied month.
395       * <P>
396       * The string returned is the long or short form of the month name
    taken
397       * from the default locale.
398       *
399       * @param month the month.
400       * @param shortened if <code>true</code> return the abbreviation of
    the
401       *                  month.
402       *
403       * @return a string representing the supplied month.
404       * @throws java.lang.IllegalArgumentException
405       */
406      public static String monthCodeToString(final int month,
407                                             final boolean shortened) {
408
409          // check arguments...
410          if (!isValidMonthCode(month)) {
411              throw new IllegalArgumentException(
```

```
412                 "SerialDate.monthCodeToString: month outside valid
   range.");
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
413           }
414
415           final String[] months;
416
417           if (shortened) {
418               months = DATE_FORMAT_SYMBOLS.getShortMonths();
419           }
420           else {
421               months = DATE_FORMAT_SYMBOLS.getMonths();
422           }
423
424           return months[month - 1];
425
426       }
427
428       /**
429        * Converts a string to a month code.
430        * <P>
431        * This method will return one of the constants JANUARY, FEBRUARY,
   ...,
432        * DECEMBER that corresponds to the string. If the string is not
433        * recognised, this method returns -1.
434        *
435        * @param s the string to parse.
436        *
437        * @return <code>-1</code> if the string is not parseable, the
   month of the
438        *         year otherwise.
439        */
440       public static int stringToMonthCode(String s) {
441
442           final String[] shortMonthNames =
   DATE_FORMAT_SYMBOLS.getShortMonths();
443           final String[] monthNames = DATE_FORMAT_SYMBOLS.getMonths();
444
445           int result = -1;
446           s = s.trim();
447
448           // first try parsing the string as an integer (1-12)...
449           try {
450               result = Integer.parseInt(s);
451           }
452           catch (NumberFormatException e) {
453               // suppress
454           }
455
456           // now search through the month names...
457           if ((result < 1) || (result > 12)) {
458               for (int i = 0; i < monthNames.length; i++) {
459                   if (s.equals(shortMonthNames[i])) {
```

```
460                         result = i + 1;
461                         break;
462                     }
463                     if (s.equals(monthNames[i])) {
464                         result = i + 1;
465                         break;
466                     }
467                 }
468             }
469
470             return result;
471
472         }
473
474         /**
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
475          * Returns true if the supplied integer code represents a valid
476          * week-in-the-month, and false otherwise.
477          *
478          * @param code the code being checked for validity.
479          * @return <code>true</code> if the supplied integer code
   represents a
480          *         valid week-in-the-month.
481          */
482         public static boolean isValidWeekInMonthCode(final int code) {
483
484             switch(code) {
485                 case FIRST_WEEK_IN_MONTH:
486                 case SECOND_WEEK_IN_MONTH:
487                 case THIRD_WEEK_IN_MONTH:
488                 case FOURTH_WEEK_IN_MONTH:
489                 case LAST_WEEK_IN_MONTH: return true;
490                 default: return false;
491             }
492
493         }
494
495         /**
496          * Determines whether or not the specified year is a leap year.
497          *
498          * @param yyyy the year (in the range 1900 to 9999).
499          *
500          * @return <code>true</code> if the specified year is a leap year.
501          */
502         public static boolean isLeapYear(final int yyyy) {
503
504             if ((yyyy % 4) != 0) {
505                 return false;
506             }
507             else if ((yyyy % 400) == 0) {
508                 return true;
509             }
510             else if ((yyyy % 100) == 0) {
```

```
511             return false;
512         }
513         else {
514             return true;
515         }
516
517     }
518
519     /**
520      * Returns the number of leap years from 1900 to the specified year
521      * INCLUSIVE.
522      * <P>
523      * Note that 1900 is not a leap year.
524      *
525      * @param yyyy the year (in the range 1900 to 9999).
526      *
527      * @return the number of leap years from 1900 to the specified
   year.
528      */
529     public static int leapYearCount(final int yyyy) {
530
531         final int leap4 = (yyyy - 1896) / 4;
532         final int leap100 = (yyyy - 1800) / 100;
533         final int leap400 = (yyyy - 1600) / 400;
534         return leap4 - leap100 + leap400;
535
536     }
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
537
538     /**
539      * Returns the number of the last day of the month, taking into
   account
540      * leap years.
541      *
542      * @param month the month.
543      * @param yyyy the year (in the range 1900 to 9999).
544      *
545      * @return the number of the last day of the month.
546      */
547     public static int lastDayOfMonth(final int month, final int yyyy) {
548
549         final int result = LAST_DAY_OF_MONTH[month];
550         if (month != FEBRUARY) {
551             return result;
552         }
553         else if (isLeapYear(yyyy)) {
554             return result + 1;
555         }
556         else {
557             return result;
558         }
559
560     }
```

```
561
562      /**
563       * Creates a new date by adding the specified number of days to the
   base
564       * date.
565       *
566       * @param days the number of days to add (can be negative).
567       * @param base the base date.
568       *
569       * @return a new date.
570       */
571      public static SerialDate addDays(final int days, final SerialDate
   base) {
572
573          final int serialDayNumber = base.toSerial() + days;
574          return SerialDate.createInstance(serialDayNumber);
575
576      }
577
578      /**
579       * Creates a new date by adding the specified number of months to
   the base
580       * date.
581       * <P>
582       * If the base date is close to the end of the month, the day on
   the result
583       * may be adjusted slightly: 31 May + 1 month = 30 June.
584       *
585       * @param months the number of months to add (can be negative).
586       * @param base the base date.
587       *
588       * @return a new date.
589       */
590      public static SerialDate addMonths(final int months,
591                                         final SerialDate base) {
592
593          final int yy = (12 * base.getYYYY() + base.getMonth() + months
   - 1)
594                         / 12;
595          final int mm = (12 * base.getYYYY() + base.getMonth() + months
   - 1)
596                         % 12 + 1;
597          final int dd = Math.min(
598              base.getDayOfMonth(), SerialDate.lastDayOfMonth(mm, yy)
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
599          );
600          return SerialDate.createInstance(dd, mm, yy);
601
602      }
603
604      /**
605       * Creates a new date by adding the specified number of years to
   the base
```

```
606       * date.
607       *
608       * @param years the number of years to add (can be negative).
609       * @param base the base date.
610       *
611       * @return A new date.
612       */
613      public static SerialDate addYears(final int years, final SerialDate
   base) {
614
615          final int baseY = base.getYYYY();
616          final int baseM = base.getMonth();
617          final int baseD = base.getDayOfMonth();
618
619          final int targetY = baseY + years;
620          final int targetD = Math.min(
621              baseD, SerialDate.lastDayOfMonth(baseM, targetY)
622          );
623
624          return SerialDate.createInstance(targetD, baseM, targetY);
625
626      }
627
628      /**
629       * Returns the latest date that falls on the specified
   day-of-the-week and
630       * is BEFORE the base date.
631       *
632       * @param targetWeekday a code for the target day-of-the-week.
633       * @param base the base date.
634       *
635       * @return the latest date that falls on the specified
   day-of-the-week and
636       *         is BEFORE the base date.
637       */
638      public static SerialDate getPreviousDayOfWeek(final int
   targetWeekday,
639                                                     final SerialDate
   base) {
640
641          // check arguments...
642          if (!SerialDate.isValidWeekdayCode(targetWeekday)) {
643              throw new IllegalArgumentException(
644                  "Invalid day-of-the-week code."
645              );
646          }
647
648          // find the date...
649          final int adjust;
650          final int baseDOW = base.getDayOfWeek();
651          if (baseDOW > targetWeekday) {
652              adjust = Math.min(0, targetWeekday - baseDOW);
653          }
654          else {
655              adjust = -7 + Math.max(0, targetWeekday - baseDOW);
656          }
657
658          return SerialDate.addDays(adjust, base);
```

```
659
660     }
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
661
662     /**
663      * Returns the earliest date that falls on the specified
   day-of-the-week
664      * and is AFTER the base date.
665      *
666      * @param targetWeekday a code for the target day-of-the-week.
667      * @param base the base date.
668      *
669      * @return the earliest date that falls on the specified
   day-of-the-week
670      *         and is AFTER the base date.
671      */
672     public static SerialDate getFollowingDayOfWeek(final int
   targetWeekday,
673                                                    final SerialDate
   base) {
674
675         // check arguments...
676         if (!SerialDate.isValidWeekdayCode(targetWeekday)) {
677             throw new IllegalArgumentException(
678                 "Invalid day-of-the-week code."
679             );
680         }
681
682         // find the date...
683         final int adjust;
684         final int baseDOW = base.getDayOfWeek();
685         if (baseDOW > targetWeekday) {
686             adjust = 7 + Math.min(0, targetWeekday - baseDOW);
687         }
688         else {
689             adjust = Math.max(0, targetWeekday - baseDOW);
690         }
691
692         return SerialDate.addDays(adjust, base);
693     }
694
695     /**
696      * Returns the date that falls on the specified day-of-the-week and
   is
697      * CLOSEST to the base date.
698      *
699      * @param targetDOW a code for the target day-of-the-week.
700      * @param base the base date.
701      *
702      * @return the date that falls on the specified day-of-the-week and
   is
703      *         CLOSEST to the base date.
704      */
```

```
705      public static SerialDate getNearestDayOfWeek(final int targetDOW,
706                                                    final SerialDate base)
   {
707
708          // check arguments...
709          if (!SerialDate.isValidWeekdayCode(targetDOW)) {
710              throw new IllegalArgumentException(
711                  "Invalid day-of-the-week code."
712              );
713          }
714
715          // find the date...
716          final int baseDOW = base.getDayOfWeek();
717          int adjust = -Math.abs(targetDOW - baseDOW);
718          if (adjust >= 4) {
719              adjust = 7 - adjust;
720          }
721          if (adjust <= -4) {
722              adjust = 7 + adjust;
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
723          }
724          return SerialDate.addDays(adjust, base);
725
726      }
727
728      /**
729        * Rolls the date forward to the last day of the month.
730        *
731        * @param base the base date.
732        *
733        * @return a new serial date.
734        */
735      public SerialDate getEndOfCurrentMonth(final SerialDate base) {
736           final int last = SerialDate.lastDayOfMonth(
737                base.getMonth(), base.getYYYY()
738           );
739           return SerialDate.createInstance(last, base.getMonth(),
   base.getYYYY());
740      }
741
742      /**
743       * Returns a string corresponding to the week-in-the-month code.
744       * <P>
745       * Need to find a better approach.
746       *
747       * @param count an integer code representing the week-in-the-month.
748       *
749       * @return a string corresponding to the week-in-the-month code.
750       */
751      public static String weekInMonthToString(final int count) {
752
753          switch (count) {
754              case SerialDate.FIRST_WEEK_IN_MONTH : return "First";
```

```
755             case SerialDate.SECOND_WEEK_IN_MONTH : return "Second";
756             case SerialDate.THIRD_WEEK_IN_MONTH : return "Third";
757             case SerialDate.FOURTH_WEEK_IN_MONTH : return "Fourth";
758             case SerialDate.LAST_WEEK_IN_MONTH : return "Last";
759             default :
760                 return "SerialDate.weekInMonthToString(): invalid
   code.";
761         }
762
763     }
764
765     /**
766      * Returns a string representing the supplied 'relative'.
767      * <P>
768      * Need to find a better approach.
769      *
770      * @param relative a constant representing the 'relative'.
771      *
772      * @return a string representing the supplied 'relative'.
773      */
774     public static String relativeToString(final int relative) {
775
776         switch (relative) {
777             case SerialDate.PRECEDING : return "Preceding";
778             case SerialDate.NEAREST : return "Nearest";
779             case SerialDate.FOLLOWING : return "Following";
780             default : return "ERROR : Relative To String";
781         }
782
783     }
784
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
785     /**
786       * Factory method that returns an instance of some concrete
   subclass of
787       * {@link SerialDate}.
788       *
789       * @param day the day (1-31).
790       * @param month the month (1-12).
791       * @param yyyy the year (in the range 1900 to 9999).
792       *
793       * @return An instance of {@link SerialDate}.
794       */
795     public static SerialDate createInstance(final int day, final int
   month,
796                                              final int yyyy) {
797         return new SpreadsheetDate(day, month, yyyy);
798     }
799
800     /**
801       * Factory method that returns an instance of some concrete
   subclass of
802       * {@link SerialDate}.
```

```
803         *
804         * @param serial the serial number for the day (1 January 1900 =
    2).
805         *
806         * @return a instance of SerialDate.
807         */
808       public static SerialDate createInstance(final int serial) {
809           return new SpreadsheetDate(serial);
810       }
811
812       /**
813        * Factory method that returns an instance of a subclass of
    SerialDate.
814        *
815        * @param date A Java date object.
816        *
817        * @return a instance of SerialDate.
818        */
819       public static SerialDate createInstance(final java.util.Date date)
    {
820
821            final GregorianCalendar calendar = new GregorianCalendar();
822            calendar.setTime(date);
823            return new SpreadsheetDate(calendar.get(Calendar.DATE),
824                                       calendar.get(Calendar.MONTH) + 1,
825                                       calendar.get(Calendar.YEAR));
826
827       }
828
829       /**
830        * Returns the serial number for the date, where 1 January 1900 = 2
    (this
831        * corresponds, almost, to the numbering system used in Microsoft
    Excel for
832        * Windows and Lotus 1-2-3).
833        *
834        * @return the serial number for the date.
835        */
836       public abstract int toSerial();
837
838       /**
839        * Returns a java.util.Date. Since java.util.Date has more
    precision than
840        * SerialDate, we need to define a convention for the 'time of
    day'.
841        *
842        * @return this as <code>java.util.Date</code>.
843        */
844       public abstract java.util.Date toDate();
845
846       /**
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
847        * Returns a description of the date.
```

```
848         *
849         * @return a description of the date.
850         */
851       public String getDescription() {
852           return this.description;
853       }
854
855       /**
856         * Sets the description for the date.
857         *
858         * @param description the new description for the date.
859         */
860       public void setDescription(final String description) {
861           this.description = description;
862       }
863
864       /**
865         * Converts the date to a string.
866         *
867         * @return a string representation of the date.
868         */
869       public String toString() {
870           return getDayOfMonth() + "-" +
   SerialDate.monthCodeToString(getMonth())
871                                   + "-" + getYYYY();
872       }
873
874       /**
875        * Returns the year (assume a valid range of 1900 to 9999).
876        *
877        * @return the year.
878        */
879       public abstract int getYYYY();
880
881       /**
882        * Returns the month (January = 1, February = 2, March = 3).
883        *
884        * @return the month of the year.
885        */
886       public abstract int getMonth();
887
888       /**
889        * Returns the day of the month.
890        *
891        * @return the day of the month.
892        */
893       public abstract int getDayOfMonth();
894
895       /**
896        * Returns the day of the week.
897        *
898        * @return the day of the week.
899        */
900       public abstract int getDayOfWeek();
901
902       /**
903        * Returns the difference (in days) between this date and the
   specified
```

```
904       * 'other' date.
905       * <P>
906       * The result is positive if this date is after the 'other' date
   and
907       * negative if it is before the 'other' date.
908       *
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
909       * @param other the date being compared to.
910       *
911       * @return the difference between this and the other date.
912       */
913      public abstract int compare(SerialDate other);
914
915      /**
916       * Returns true if this SerialDate represents the same date as the
917       * specified SerialDate.
918       *
919       * @param other the date being compared to.
920       *
921       * @return <code>true</code> if this SerialDate represents the same
   date as
922       *         the specified SerialDate.
923       */
924      public abstract boolean isOn(SerialDate other);
925
926      /**
927       * Returns true if this SerialDate represents an earlier date
   compared to
928       * the specified SerialDate.
929       *
930       * @param other The date being compared to.
931       *
932       * @return <code>true</code> if this SerialDate represents an
   earlier date
933       *         compared to the specified SerialDate.
934       */
935      public abstract boolean isBefore(SerialDate other);
936
937      /**
938       * Returns true if this SerialDate represents the same date as the
939       * specified SerialDate.
940       *
941       * @param other the date being compared to.
942       *
943       * @return <code>true<code> if this SerialDate represents the same
   date
944       *         as the specified SerialDate.
945       */
946      public abstract boolean isOnOrBefore(SerialDate other);
947
948      /**
949       * Returns true if this SerialDate represents the same date as the
950       * specified SerialDate.
```

```
951       *
952       * @param other the date being compared to.
953       *
954       * @return <code>true</code> if this SerialDate represents the same
   date
955       *         as the specified SerialDate.
956       */
957      public abstract boolean isAfter(SerialDate other);
958
959      /**
960       * Returns true if this SerialDate represents the same date as the
961       * specified SerialDate.
962       *
963       * @param other the date being compared to.
964       *
965       * @return <code>true</code> if this SerialDate represents the same
   date
966       *         as the specified SerialDate.
967       */
968      public abstract boolean isOnOrAfter(SerialDate other);
969
970      /**
971       * Returns <code>true</code> if this {@link SerialDate} is within
   the
```

Đoạn mã B-1 (tiếp theo)
SerialDate.Java

```
972       * specified range (INCLUSIVE). The date order of d1 and d2 is not
973       * important.
974       *
975       * @param d1 a boundary date for the range.
976       * @param d2 the other boundary date for the range.
977       *
978       * @return A boolean.
979       */
980      public abstract boolean isInRange(SerialDate d1, SerialDate d2);
981
982      /**
983       * Returns <code>true</code> if this {@link SerialDate} is within
   the
984       * specified range (caller specifies whether or not the end-points
   are
985       * included). The date order of d1 and d2 is not important.
986       *
987       * @param d1 a boundary date for the range.
988       * @param d2 the other boundary date for the range.
989       * @param include a code that controls whether or not the start and
   end
990       *                dates are included in the range.
991       *
992       * @return A boolean.
993       */
994      public abstract boolean isInRange(SerialDate d1, SerialDate d2,
995                                        int include);
996
```

```
 997     /**
 998       * Returns the latest date that falls on the specified
    day-of-the-week and
 999       * is BEFORE this date.
1000       *
1001       * @param targetDOW a code for the target day-of-the-week.
1002       *
1003       * @return the latest date that falls on the specified
    day-of-the-week and
1004       *         is BEFORE this date.
1005       */
1006     public SerialDate getPreviousDayOfWeek(final int targetDOW) {
1007          return getPreviousDayOfWeek(targetDOW, this);
1008     }
1009
1010     /**
1011       * Returns the earliest date that falls on the specified
    day-of-the-week
1012       * and is AFTER this date.
1013       *
1014       * @param targetDOW a code for the target day-of-the-week.
1015       *
1016       * @return the earliest date that falls on the specified
    day-of-the-week
1017       *         and is AFTER this date.
1018       */
1019     public SerialDate getFollowingDayOfWeek(final int targetDOW) {
1020          return getFollowingDayOfWeek(targetDOW, this);
1021     }
1022
1023     /**
1024       * Returns the nearest date that falls on the specified
    day-of-the-week.
1025       *
1026       * @param targetDOW a code for the target day-of-the-week.
1027       *
1028       * @return the nearest date that falls on the specified
    day-of-the-week.
1029       */
1030     public SerialDate getNearestDayOfWeek(final int targetDOW) {
1031          return getNearestDayOfWeek(targetDOW, this);
1032     }
1033
1034 }
```

Đoạn mã B-2
SerialDateTest.java

```
1 /*
 ========================================================================
2 * JCommon : a free general purpose class library for the Java(tm)
 platform
3 *
 ========================================================================
4 *
5 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
```

```
 6 *
 7 * Project Info: http://www.jfree.org/jcommon/index.html
 8 *
 9 * This library is free software; you can redistribute it and/or modify
   it
10 * under the terms of the GNU Lesser General Public License as published
   by
11 * the Free Software Foundation; either version 2.1 of the License, or
12 * (at your option) any later version.
13 *
14 * This library is distributed in the hope that it will be useful, but
15 * WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY
16 * or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public
17 * License for more details.
18 *
19 * You should have received a copy of the GNU Lesser General Public
20 * License along with this library; if not, write to the Free Software
21 * Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
   02110-1301,
22 * USA.
23 *
24 * [Java is a trademark or registered trademark of Sun Microsystems, Inc.
25 * in the United States and other countries.]
26 *
27 * --------------------
28 * SerialDateTests.java
29 * --------------------
30 * (C) Copyright 2001-2005, by Object Refinery Limited.
31 *
32 * Original Author: David Gilbert (for Object Refinery Limited);
33 * Contributor(s): -;
34 *
35 * $Id: SerialDateTests.java,v 1.6 2005/11/16 15:58:40 taqua Exp $
36 *
37 * Changes
38 * -------
39 * 15-Nov-2001 : Version 1 (DG);
40 * 25-Jun-2002 : Removed unnecessary import (DG);
41 * 24-Oct-2002 : Fixed errors reported by Checkstyle (DG);
42 * 13-Mar-2003 : Added serialization test (DG);
43 * 05-Jan-2005 : Added test for bug report 1096282 (DG);
44 *
45 */
46
47 package org.jfree.date.junit;
48
49 import java.io.ByteArrayInputStream;
50 import java.io.ByteArrayOutputStream;
51 import java.io.ObjectInput;
52 import java.io.ObjectInputStream;
53 import java.io.ObjectOutput;
54 import java.io.ObjectOutputStream;
55
56 import junit.framework.Test;
57 import junit.framework.TestCase;
58 import junit.framework.TestSuite;
59
```

```
60 import org.jfree.date.MonthConstants;
61 import org.jfree.date.SerialDate;
62
```

Đoạn mã B-2 (tiếp theo)
SerialDateTest.java

```
63 /**
64 * Some JUnit tests for the {@link SerialDate} class.
65 */
66 public class SerialDateTests extends TestCase {
67
68     /** Date representing November 9. */
69     private SerialDate nov9Y2001;
70
71     /**
72       * Creates a new test case.
73       *
74       * @param name the name.
75       */
76     public SerialDateTests(final String name) {
77          super(name);
78     }
79
80     /**
81       * Returns a test suite for the JUnit test runner.
82       *
83       * @return The test suite.
84       */
85     public static Test suite() {
86          return new TestSuite(SerialDateTests.class);
87     }
88
89     /**
90       * Problem set up.
91       */
92     protected void setUp() {
93          this.nov9Y2001 = SerialDate.createInstance(9,
   MonthConstants.NOVEMBER, 2001);
94     }
95
96     /**
97       * 9 Nov 2001 plus two months should be 9 Jan 2002.
98       */
99     public void testAddMonthsTo9Nov2001() {
100          final SerialDate jan9Y2002 = SerialDate.addMonths(2,
   this.nov9Y2001);
101          final SerialDate answer = SerialDate.createInstance(9, 1,
   2002);
102          assertEquals(answer, jan9Y2002);
103     }
104
105     /**
106       * A test case for a reported bug, now fixed.
107       */
108     public void testAddMonthsTo5Oct2003() {
```

```
109          final SerialDate d1 = SerialDate.createInstance(5,
   MonthConstants.OCTOBER, 2003);
110          final SerialDate d2 = SerialDate.addMonths(2, d1);
111          assertEquals(d2, SerialDate.createInstance(5,
   MonthConstants.DECEMBER, 2003));
112      }
113
114      /**
115        * A test case for a reported bug, now fixed.
116        */
117      public void testAddMonthsTo1Jan2003() {
118          final SerialDate d1 = SerialDate.createInstance(1,
   MonthConstants.JANUARY, 2003);
119          final SerialDate d2 = SerialDate.addMonths(0, d1);
120          assertEquals(d2, d1);
121      }
122
123      /**
124        * Monday preceding Friday 9 November 2001 should be 5 November.
```

Đoạn mã B-2 (tiếp theo)
SerialDateTest.java

```
125        */
126      public void testMondayPrecedingFriday9Nov2001() {
127          SerialDate mondayBefore = SerialDate.getPreviousDayOfWeek(
128              SerialDate.MONDAY, this.nov9Y2001
129          );
130          assertEquals(5, mondayBefore.getDayOfMonth());
131      }
132
133      /**
134        * Monday following Friday 9 November 2001 should be 12 November.
135        */
136      public void testMondayFollowingFriday9Nov2001() {
137          SerialDate mondayAfter = SerialDate.getFollowingDayOfWeek(
138              SerialDate.MONDAY, this.nov9Y2001
139          );
140          assertEquals(12, mondayAfter.getDayOfMonth());
141      }
142
143      /**
144        * Monday nearest Friday 9 November 2001 should be 12 November.
145        */
146      public void testMondayNearestFriday9Nov2001() {
147          SerialDate mondayNearest = SerialDate.getNearestDayOfWeek(
148              SerialDate.MONDAY, this.nov9Y2001
149          );
150          assertEquals(12, mondayNearest.getDayOfMonth());
151      }
152
153      /**
154        * The Monday nearest to 22nd January 1970 falls on the 19th.
155        */
156      public void testMondayNearest22Jan1970() {
```

```
157          SerialDate jan22Y1970 = SerialDate.createInstance(22,
   MonthConstants.JANUARY, 1970);
158          SerialDate
   mondayNearest=SerialDate.getNearestDayOfWeek(SerialDate.MONDAY,
   jan22Y1970);
159          assertEquals(19, mondayNearest.getDayOfMonth());
160      }
161
162      /**
163       * Problem that the conversion of days to strings returns the right
   result. Actually, this
164       * result depends on the Locale so this test needs to be modified.
165       */
166      public void testWeekdayCodeToString() {
167
168          final String test =
   SerialDate.weekdayCodeToString(SerialDate.SATURDAY);
169          assertEquals("Saturday", test);
170
171      }
172
173      /**
174       * Test the conversion of a string to a weekday. Note that this test
   will fail if the
175       * default locale doesn't use English weekday names...devise a
   better test!
176       */
177      public void testStringToWeekday() {
178
179          int weekday = SerialDate.stringToWeekdayCode("Wednesday");
180          assertEquals(SerialDate.WEDNESDAY, weekday);
181
182          weekday = SerialDate.stringToWeekdayCode(" Wednesday ");
183          assertEquals(SerialDate.WEDNESDAY, weekday);
184
```

Đoạn mã B-2 (tiếp theo)
SerialDateTest.java

```
185          weekday = SerialDate.stringToWeekdayCode("Wed");
186          assertEquals(SerialDate.WEDNESDAY, weekday);
187
188      }
189
190      /**
191       * Test the conversion of a string to a month. Note that this test
   will fail if the
192       * default locale doesn't use English month names...devise a better
   test!
193       */
194      public void testStringToMonthCode() {
195
196          int m = SerialDate.stringToMonthCode("January");
197          assertEquals(MonthConstants.JANUARY, m);
198
199          m = SerialDate.stringToMonthCode(" January ");
```

```
200         assertEquals(MonthConstants.JANUARY, m);
201
202         m = SerialDate.stringToMonthCode("Jan");
203         assertEquals(MonthConstants.JANUARY, m);
204
205     }
206
207     /**
208      * Tests the conversion of a month code to a string.
209      */
210     public void testMonthCodeToStringCode() {
211
212         final String test =
    SerialDate.monthCodeToString(MonthConstants.DECEMBER);
213         assertEquals("December", test);
214
215     }
216
217     /**
218       * 1900 is not a leap year.
219       */
220     public void testIsNotLeapYear1900() {
221          assertTrue(!SerialDate.isLeapYear(1900));
222     }
223
224     /**
225       * 2000 is a leap year.
226       */
227     public void testIsLeapYear2000() {
228          assertTrue(SerialDate.isLeapYear(2000));
229     }
230
231     /**
232       * The number of leap years from 1900 up-to-and-including 1899 is
    0.
233       */
234     public void testLeapYearCount1899() {
235          assertEquals(SerialDate.leapYearCount(1899), 0);
236     }
237
238     /**
239       * The number of leap years from 1900 up-to-and-including 1903 is
    0.
240       */
241     public void testLeapYearCount1903() {
242          assertEquals(SerialDate.leapYearCount(1903), 0);
243     }
244
245     /**
246      * The number of leap years from 1900 up-to-and-including 1904 is 1.
247      */
```

Đoạn mã B-2 (tiếp theo)
SerialDateTest.java

```
248     public void testLeapYearCount1904() {
```

```
249         assertEquals(SerialDate.leapYearCount(1904), 1);
250     }
251
252     /**
253       * The number of leap years from 1900 up-to-and-including 1999 is
   24.
254       */
255     public void testLeapYearCount1999() {
256          assertEquals(SerialDate.leapYearCount(1999), 24);
257     }
258
259     /**
260       * The number of leap years from 1900 up-to-and-including 2000 is
   25.
261       */
262     public void testLeapYearCount2000() {
263          assertEquals(SerialDate.leapYearCount(2000), 25);
264     }
265
266     /**
267      * Serialize an instance, restore it, and check for equality.
268      */
269     public void testSerialization() {
270
271         SerialDate d1 = SerialDate.createInstance(15, 4, 2000);
272         SerialDate d2 = null;
273
274         try {
275             ByteArrayOutputStream buffer = new ByteArrayOutputStream();
276             ObjectOutput out = new ObjectOutputStream(buffer);
277             out.writeObject(d1);
278             out.close();
279
280             ObjectInput in = new ObjectInputStream( new
   ByteArrayInputStream(buffer.toByteArray()));
281             d2 = (SerialDate) in.readObject();
282             in.close();
283         }
284         catch (Exception e) {
285             System.out.println(e.toString());
286         }
287         assertEquals(d1, d2);
288
289     }
290
291     /**
292       * A test for bug report 1096282 (now fixed).
293       */
294     public void test1096282() {
295          SerialDate d = SerialDate.createInstance(29, 2, 2004);
296          d = SerialDate.addYears(1, d);
297          SerialDate expected = SerialDate.createInstance(28, 2, 2005);
298          assertTrue(d.isOn(expected));
299     }
300
301     /**
302      * Miscellaneous tests for the addMonths() method.
303      */
```

```
304     public void testAddMonths() {
305         SerialDate d1 = SerialDate.createInstance(31, 5, 2004);
306
```

Đoạn mã B-2 (tiếp theo)
SerialDateTest.java

```
307         SerialDate d2 = SerialDate.addMonths(1, d1);
308         assertEquals(30, d2.getDayOfMonth());
309         assertEquals(6, d2.getMonth());
310         assertEquals(2004, d2.getYYYY());
311
312         SerialDate d3 = SerialDate.addMonths(2, d1);
313         assertEquals(31, d3.getDayOfMonth());
314         assertEquals(7, d3.getMonth());
315         assertEquals(2004, d3.getYYYY());
316
317         SerialDate d4 = SerialDate.addMonths(1, SerialDate.addMonths(1,
    d1));
318         assertEquals(30, d4.getDayOfMonth());
319         assertEquals(7, d4.getMonth());
320         assertEquals(2004, d4.getYYYY());
321     }
322 }
```

Đoạn mã B-3
MonthConstants.java

```
 1 /*
   ========================================================================
 2 * JCommon : a free general purpose class library for the Java(tm)
   platform
 3 *
   ========================================================================
 4 *
 5 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
 6 *
 7 * Project Info: http://www.jfree.org/jcommon/index.html
 8 *
 9 * This library is free software; you can redistribute it and/or modify
   it
10 * under the terms of the GNU Lesser General Public License as published
   by
11 * the Free Software Foundation; either version 2.1 of the License, or
12 * (at your option) any later version.
13 *
14 * This library is distributed in the hope that it will be useful, but
15 * WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY
16 * or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public
17 * License for more details.
18 *
19 * You should have received a copy of the GNU Lesser General Public
```

```
20 * License along with this library; if not, write to the Free Software
21 * Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
   02110-1301,
22 * USA.
23 *
24 * [Java is a trademark or registered trademark of Sun Microsystems, Inc.
25 * in the United States and other countries.]
26 *
27 * -------------------
28 * MonthConstants.java
29 * -------------------
30 * (C) Copyright 2002, 2003, by Object Refinery Limited.
31 *
32 * Original Author: David Gilbert (for Object Refinery Limited);
33 * Contributor(s): -;
34 *
35 * $Id: MonthConstants.java,v 1.4 2005/11/16 15:58:40 taqua Exp $
36 *
37 * Changes
38 * -------
39 * 29-May-2002 : Version 1 (code moved from SerialDate class) (DG);
40 *
41 */
42
43 package org.jfree.date;
44
45 /**
46 * Useful constants for months. Note that these are NOT equivalent to the
47 * constants defined by java.util.Calendar (where JANUARY=0 and
   DECEMBER=11).
48 * <P>
49 * Used by the SerialDate and RegularTimePeriod classes.
50 *
51 * @author David Gilbert
52 */
53 public interface MonthConstants {
54
55     /** Constant for January. */
56     public static final int JANUARY = 1;
57
58     /** Constant for February. */
59     public static final int FEBRUARY = 2;
60
```

Đoạn mã B-3 (tiếp theo)
MonthConstants.java

```
61     /** Constant for March. */
62     public static final int MARCH = 3;
63
64     /** Constant for April. */
65     public static final int APRIL = 4;
66
67     /** Constant for May. */
68     public static final int MAY = 5;
69
```

```
70     /** Constant for June. */
71     public static final int JUNE = 6;
72
73     /** Constant for July. */
74     public static final int JULY = 7;
75
76     /** Constant for August. */
77     public static final int AUGUST = 8;
78
79     /** Constant for September. */
80     public static final int SEPTEMBER = 9;
81
82     /** Constant for October. */
83     public static final int OCTOBER = 10;
84
85     /** Constant for November. */
86     public static final int NOVEMBER = 11;
87
88     /** Constant for December. */
89     public static final int DECEMBER = 12;
90
91 }
```

Đoạn mã B-4
BobsSerialDateTest.java

```
 1 package org.jfree.date.junit;
 2
 3 import junit.framework.TestCase;
 4 import org.jfree.date.*;
 5 import static org.jfree.date.SerialDate.*;
 6
 7 import java.util.*;
 8
 9 public class BobsSerialDateTest extends TestCase {
10
11 public void testIsValidWeekdayCode() throws Exception {
12     for (int day = 1; day <= 7; day++)
13        assertTrue(isValidWeekdayCode(day));
14     assertFalse(isValidWeekdayCode(0));
15     assertFalse(isValidWeekdayCode(8));
16 }
17
18 public void testStringToWeekdayCode() throws Exception {
19
20     assertEquals(-1, stringToWeekdayCode("Hello"));
21     assertEquals(MONDAY, stringToWeekdayCode("Monday"));
22     assertEquals(MONDAY, stringToWeekdayCode("Mon"));
23 //todo     assertEquals(MONDAY,stringToWeekdayCode("monday"));
24 //     assertEquals(MONDAY,stringToWeekdayCode("MONDAY"));
25 //     assertEquals(MONDAY, stringToWeekdayCode("mon"));
26
27     assertEquals(TUESDAY, stringToWeekdayCode("Tuesday"));
28     assertEquals(TUESDAY, stringToWeekdayCode("Tue"));
29 //     assertEquals(TUESDAY,stringToWeekdayCode("tuesday"));
30 //     assertEquals(TUESDAY,stringToWeekdayCode("TUESDAY"));
```

```
31 //     assertEquals(TUESDAY, stringToWeekdayCode("tue"));
32 //     assertEquals(TUESDAY, stringToWeekdayCode("tues"));
33
34     assertEquals(WEDNESDAY, stringToWeekdayCode("Wednesday"));
35     assertEquals(WEDNESDAY, stringToWeekdayCode("Wed"));
36 //     assertEquals(WEDNESDAY,stringToWeekdayCode("wednesday"));
37 //     assertEquals(WEDNESDAY,stringToWeekdayCode("WEDNESDAY"));
38 //     assertEquals(WEDNESDAY, stringToWeekdayCode("wed"));
39
40     assertEquals(THURSDAY, stringToWeekdayCode("Thursday"));
41     assertEquals(THURSDAY, stringToWeekdayCode("Thu"));
42 //     assertEquals(THURSDAY,stringToWeekdayCode("thursday"));
43 //     assertEquals(THURSDAY,stringToWeekdayCode("THURSDAY"));
44 //     assertEquals(THURSDAY, stringToWeekdayCode("thu"));
45 //     assertEquals(THURSDAY, stringToWeekdayCode("thurs"));
46
47     assertEquals(FRIDAY, stringToWeekdayCode("Friday"));
48     assertEquals(FRIDAY, stringToWeekdayCode("Fri"));
49 //     assertEquals(FRIDAY,stringToWeekdayCode("friday"));
50 //     assertEquals(FRIDAY,stringToWeekdayCode("FRIDAY"));
51 //     assertEquals(FRIDAY, stringToWeekdayCode("fri"));
52
53     assertEquals(SATURDAY, stringToWeekdayCode("Saturday"));
54     assertEquals(SATURDAY, stringToWeekdayCode("Sat"));
55 //     assertEquals(SATURDAY,stringToWeekdayCode("saturday"));
56 //     assertEquals(SATURDAY,stringToWeekdayCode("SATURDAY"));
57 //     assertEquals(SATURDAY, stringToWeekdayCode("sat"));
58
59     assertEquals(SUNDAY, stringToWeekdayCode("Sunday"));
60     assertEquals(SUNDAY, stringToWeekdayCode("Sun"));
61 //     assertEquals(SUNDAY,stringToWeekdayCode("sunday"));
62 //     assertEquals(SUNDAY,stringToWeekdayCode("SUNDAY"));
63 //     assertEquals(SUNDAY, stringToWeekdayCode("sun"));
64 }
65
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
66    public void testWeekdayCodeToString() throws Exception {
67      assertEquals("Sunday", weekdayCodeToString(SUNDAY));
68      assertEquals("Monday", weekdayCodeToString(MONDAY));
69      assertEquals("Tuesday", weekdayCodeToString(TUESDAY));
70      assertEquals("Wednesday", weekdayCodeToString(WEDNESDAY));
71      assertEquals("Thursday", weekdayCodeToString(THURSDAY));
72      assertEquals("Friday", weekdayCodeToString(FRIDAY));
73      assertEquals("Saturday", weekdayCodeToString(SATURDAY));
74    }
75
76    public void testIsValidMonthCode() throws Exception {
77      for (int i = 1; i <= 12; i++)
78        assertTrue(isValidMonthCode(i));
79      assertFalse(isValidMonthCode(0));
80      assertFalse(isValidMonthCode(13));
81    }
82
```

```
 83    public void testMonthToQuarter() throws Exception {
 84      assertEquals(1, monthCodeToQuarter(JANUARY));
 85      assertEquals(1, monthCodeToQuarter(FEBRUARY));
 86      assertEquals(1, monthCodeToQuarter(MARCH));
 87      assertEquals(2, monthCodeToQuarter(APRIL));
 88      assertEquals(2, monthCodeToQuarter(MAY));
 89      assertEquals(2, monthCodeToQuarter(JUNE));
 90      assertEquals(3, monthCodeToQuarter(JULY));
 91      assertEquals(3, monthCodeToQuarter(AUGUST));
 92      assertEquals(3, monthCodeToQuarter(SEPTEMBER));
 93      assertEquals(4, monthCodeToQuarter(OCTOBER));
 94      assertEquals(4, monthCodeToQuarter(NOVEMBER));
 95      assertEquals(4, monthCodeToQuarter(DECEMBER));
 96
 97        try {
 98          monthCodeToQuarter(-1);
 99          fail("Invalid Month Code should throw exception");
100        } catch (IllegalArgumentException e) {
101        }
102    }
103
104    public void testMonthCodeToString() throws Exception {
105      assertEquals("January", monthCodeToString(JANUARY));
106      assertEquals("February", monthCodeToString(FEBRUARY));
107      assertEquals("March", monthCodeToString(MARCH));
108      assertEquals("April", monthCodeToString(APRIL));
109      assertEquals("May", monthCodeToString(MAY));
110      assertEquals("June", monthCodeToString(JUNE));
111      assertEquals("July", monthCodeToString(JULY));
112      assertEquals("August", monthCodeToString(AUGUST));
113      assertEquals("September", monthCodeToString(SEPTEMBER));
114      assertEquals("October", monthCodeToString(OCTOBER));
115      assertEquals("November", monthCodeToString(NOVEMBER));
116      assertEquals("December", monthCodeToString(DECEMBER));
117
118        assertEquals("Jan", monthCodeToString(JANUARY, true));
119        assertEquals("Feb", monthCodeToString(FEBRUARY, true));
120        assertEquals("Mar", monthCodeToString(MARCH, true));
121        assertEquals("Apr", monthCodeToString(APRIL, true));
122        assertEquals("May", monthCodeToString(MAY, true));
123        assertEquals("Jun", monthCodeToString(JUNE, true));
124        assertEquals("Jul", monthCodeToString(JULY, true));
125        assertEquals("Aug", monthCodeToString(AUGUST, true));
126        assertEquals("Sep", monthCodeToString(SEPTEMBER, true));
127        assertEquals("Oct", monthCodeToString(OCTOBER, true));
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
128     assertEquals("Nov", monthCodeToString(NOVEMBER, true));
129     assertEquals("Dec", monthCodeToString(DECEMBER, true));
130
131     try {
132        monthCodeToString(-1);
133        fail("Invalid month code should throw exception");
134     } catch (IllegalArgumentException e) {
```

```
135     }
136
137 }
138
139 public void testStringToMonthCode() throws Exception {
140     assertEquals(JANUARY,stringToMonthCode("1"));
141     assertEquals(FEBRUARY,stringToMonthCode("2"));
142     assertEquals(MARCH,stringToMonthCode("3"));
143     assertEquals(APRIL,stringToMonthCode("4"));
144     assertEquals(MAY,stringToMonthCode("5"));
145     assertEquals(JUNE,stringToMonthCode("6"));
146     assertEquals(JULY,stringToMonthCode("7"));
147     assertEquals(AUGUST,stringToMonthCode("8"));
148     assertEquals(SEPTEMBER,stringToMonthCode("9"));
149     assertEquals(OCTOBER,stringToMonthCode("10"));
150     assertEquals(NOVEMBER, stringToMonthCode("11"));
151     assertEquals(DECEMBER,stringToMonthCode("12"));
152
153 //todo     assertEquals(-1, stringToMonthCode("0"));
154 //     assertEquals(-1, stringToMonthCode("13"));
155
156     assertEquals(-1,stringToMonthCode("Hello"));
157
158     for (int m = 1; m <= 12; m++) {
159        assertEquals(m, stringToMonthCode(monthCodeToString(m, false)));
160        assertEquals(m, stringToMonthCode(monthCodeToString(m, true)));
161     }
162
163 //     assertEquals(1,stringToMonthCode("jan"));
164 //     assertEquals(2,stringToMonthCode("feb"));
165 //     assertEquals(3,stringToMonthCode("mar"));
166 //     assertEquals(4,stringToMonthCode("apr"));
167 //     assertEquals(5,stringToMonthCode("may"));
168 //     assertEquals(6,stringToMonthCode("jun"));
169 //     assertEquals(7,stringToMonthCode("jul"));
170 //     assertEquals(8,stringToMonthCode("aug"));
171 //     assertEquals(9,stringToMonthCode("sep"));
172 //     assertEquals(10,stringToMonthCode("oct"));
173 //     assertEquals(11,stringToMonthCode("nov"));
174 //     assertEquals(12,stringToMonthCode("dec"));
175
176 //     assertEquals(1,stringToMonthCode("JAN"));
177 //     assertEquals(2,stringToMonthCode("FEB"));
178 //     assertEquals(3,stringToMonthCode("MAR"));
179 //     assertEquals(4,stringToMonthCode("APR"));
180 //     assertEquals(5,stringToMonthCode("MAY"));
181 //     assertEquals(6,stringToMonthCode("JUN"));
182 //     assertEquals(7,stringToMonthCode("JUL"));
183 //     assertEquals(8,stringToMonthCode("AUG"));
184 //     assertEquals(9,stringToMonthCode("SEP"));
185 //     assertEquals(10,stringToMonthCode("OCT"));
186 //     assertEquals(11,stringToMonthCode("NOV"));
187 //     assertEquals(12,stringToMonthCode("DEC"));
188
189 //     assertEquals(1,stringToMonthCode("january"));
190 //     assertEquals(2,stringToMonthCode("february"));
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
191 //   assertEquals(3,stringToMonthCode("march"));
192 //   assertEquals(4,stringToMonthCode("april"));
193 //   assertEquals(5,stringToMonthCode("may"));
194 //   assertEquals(6,stringToMonthCode("june"));
195 //   assertEquals(7,stringToMonthCode("july"));
196 //   assertEquals(8,stringToMonthCode("august"));
197 //   assertEquals(9,stringToMonthCode("september"));
198 //   assertEquals(10,stringToMonthCode("october"));
199 //   assertEquals(11,stringToMonthCode("november"));
200 //   assertEquals(12,stringToMonthCode("december"));
201
202 //   assertEquals(1,stringToMonthCode("JANUARY"));
203 //   assertEquals(2,stringToMonthCode("FEBRUARY"));
204 //   assertEquals(3,stringToMonthCode("MAR"));
205 //   assertEquals(4,stringToMonthCode("APRIL"));
206 //   assertEquals(5,stringToMonthCode("MAY"));
207 //   assertEquals(6,stringToMonthCode("JUNE"));
208 //   assertEquals(7,stringToMonthCode("JULY"));
209 //   assertEquals(8,stringToMonthCode("AUGUST"));
210 //   assertEquals(9,stringToMonthCode("SEPTEMBER"));
211 //   assertEquals(10,stringToMonthCode("OCTOBER"));
212 //   assertEquals(11,stringToMonthCode("NOVEMBER"));
213 //   assertEquals(12,stringToMonthCode("DECEMBER"));
214 }
215
216 public void testIsValidWeekInMonthCode() throws Exception {
217    for (int w = 0; w <= 4; w++) {
218      assertTrue(isValidWeekInMonthCode(w));
219    }
220    assertFalse(isValidWeekInMonthCode(5));
221 }
222
223 public void testIsLeapYear() throws Exception {
224    assertFalse(isLeapYear(1900));
225    assertFalse(isLeapYear(1901));
226    assertFalse(isLeapYear(1902));
227    assertFalse(isLeapYear(1903));
228    assertTrue(isLeapYear(1904));
229    assertTrue(isLeapYear(1908));
230    assertFalse(isLeapYear(1955));
231    assertTrue(isLeapYear(1964));
232    assertTrue(isLeapYear(1980));
233    assertTrue(isLeapYear(2000));
234    assertFalse(isLeapYear(2001));
235    assertFalse(isLeapYear(2100));
236 }
237
238 public void testLeapYearCount() throws Exception {
239    assertEquals(0, leapYearCount(1900));
240    assertEquals(0, leapYearCount(1901));
241    assertEquals(0, leapYearCount(1902));
242    assertEquals(0, leapYearCount(1903));
243    assertEquals(1, leapYearCount(1904));
244    assertEquals(1, leapYearCount(1905));
```

```
245    assertEquals(1, leapYearCount(1906));
246    assertEquals(1, leapYearCount(1907));
247    assertEquals(2, leapYearCount(1908));
248    assertEquals(24, leapYearCount(1999));
249    assertEquals(25, leapYearCount(2001));
250    assertEquals(49, leapYearCount(2101));
251    assertEquals(73, leapYearCount(2201));
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
252     assertEquals(97, leapYearCount(2301));
253     assertEquals(122, leapYearCount(2401));
254 }
255
256 public void testLastDayOfMonth() throws Exception {
257     assertEquals(31, lastDayOfMonth(JANUARY, 1901));
258     assertEquals(28, lastDayOfMonth(FEBRUARY, 1901));
259     assertEquals(31, lastDayOfMonth(MARCH, 1901));
260     assertEquals(30, lastDayOfMonth(APRIL, 1901));
261     assertEquals(31, lastDayOfMonth(MAY, 1901));
262     assertEquals(30, lastDayOfMonth(JUNE, 1901));
263     assertEquals(31, lastDayOfMonth(JULY, 1901));
264     assertEquals(31, lastDayOfMonth(AUGUST, 1901));
265     assertEquals(30, lastDayOfMonth(SEPTEMBER, 1901));
266     assertEquals(31, lastDayOfMonth(OCTOBER, 1901));
267     assertEquals(30, lastDayOfMonth(NOVEMBER, 1901));
268     assertEquals(31, lastDayOfMonth(DECEMBER, 1901));
269     assertEquals(29, lastDayOfMonth(FEBRUARY, 1904));
270 }
271
272 public void testAddDays() throws Exception {
273     SerialDate newYears = d(1, JANUARY, 1900);
274     assertEquals(d(2, JANUARY, 1900), addDays(1, newYears));
275     assertEquals(d(1, FEBRUARY, 1900), addDays(31, newYears));
276     assertEquals(d(1, JANUARY, 1901), addDays(365, newYears));
277     assertEquals(d(31, DECEMBER, 1904), addDays(5 * 365, newYears));
278 }
279
280 private static SpreadsheetDate d(int day, int month, int year) {return
    new SpreadsheetDate(day, month, year);}
281
282 public void testAddMonths() throws Exception {
283     assertEquals(d(1, FEBRUARY, 1900), addMonths(1, d(1, JANUARY,
    1900)));
284     assertEquals(d(28, FEBRUARY, 1900), addMonths(1, d(31, JANUARY,
    1900)));
285     assertEquals(d(28, FEBRUARY, 1900), addMonths(1, d(30, JANUARY,
    1900)));
286     assertEquals(d(28, FEBRUARY, 1900), addMonths(1, d(29, JANUARY,
    1900)));
287     assertEquals(d(28, FEBRUARY, 1900), addMonths(1, d(28, JANUARY,
    1900)));
288     assertEquals(d(27, FEBRUARY, 1900), addMonths(1, d(27, JANUARY,
    1900)));
289
```

```
290     assertEquals(d(30, JUNE, 1900), addMonths(5, d(31, JANUARY, 1900)));
291     assertEquals(d(30, JUNE, 1901), addMonths(17, d(31, JANUARY,
   1900)));
292
293     assertEquals(d(29, FEBRUARY, 1904), addMonths(49, d(31, JANUARY,
   1900)));
294
295 }
296
297 public void testAddYears() throws Exception {
298     assertEquals(d(1, JANUARY, 1901), addYears(1, d(1, JANUARY, 1900)));
299     assertEquals(d(28, FEBRUARY, 1905), addYears(1, d(29, FEBRUARY,
   1904)));
300     assertEquals(d(28, FEBRUARY, 1905), addYears(1, d(28, FEBRUARY,
   1904)));
301     assertEquals(d(28, FEBRUARY, 1904), addYears(1, d(28, FEBRUARY,
   1903)));
302 }
303
304 public void testGetPreviousDayOfWeek() throws Exception {
305     assertEquals(d(24, FEBRUARY, 2006), getPreviousDayOfWeek(FRIDAY,
   d(1, MARCH, 2006)));
306     assertEquals(d(22, FEBRUARY, 2006), getPreviousDayOfWeek(WEDNESDAY,
   d(1, MARCH, 2006)));
307     assertEquals(d(29, FEBRUARY, 2004), getPreviousDayOfWeek(SUNDAY,
   d(3, MARCH, 2004)));
308     assertEquals(d(29, DECEMBER, 2004), getPreviousDayOfWeek(WEDNESDAY,
   d(5, JANUARY, 2005)));
309
310     try {
311       getPreviousDayOfWeek(-1, d(1, JANUARY, 2006));
312       fail("Invalid day of week code should throw exception");
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
313     } catch (IllegalArgumentException e) {
314     }
315 }
316
317 public void testGetFollowingDayOfWeek() throws Exception {
318 //     assertEquals(d(1, JANUARY, 2005),getFollowingDayOfWeek(SATURDAY,
   d(25, DECEMBER, 2004)));
319     assertEquals(d(1, JANUARY, 2005), getFollowingDayOfWeek(SATURDAY,
   d(26, DECEMBER, 2004)));
320     assertEquals(d(3, MARCH, 2004), getFollowingDayOfWeek(WEDNESDAY,
   d(28, FEBRUARY, 2004)));
321
322     try {
323        getFollowingDayOfWeek(-1, d(1, JANUARY, 2006));
324        fail("Invalid day of week code should throw exception");
325     } catch (IllegalArgumentException e) {
326     }
327 }
328
329 public void testGetNearestDayOfWeek() throws Exception {
```

```
330     assertEquals(d(16, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(16,
   APRIL, 2006)));
331     assertEquals(d(16, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(17,
   APRIL, 2006)));
332     assertEquals(d(16, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(18,
   APRIL, 2006)));
333     assertEquals(d(16, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(19,
   APRIL, 2006)));
334     assertEquals(d(23, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(20,
   APRIL, 2006)));
335     assertEquals(d(23, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(21,
   APRIL, 2006)));
336     assertEquals(d(23, APRIL, 2006), getNearestDayOfWeek(SUNDAY, d(22,
   APRIL, 2006)));
337
338 //todo     assertEquals(d(17, APRIL, 2006), getNearestDayOfWeek(MONDAY,
   d(16, APRIL, 2006)));
339     assertEquals(d(17, APRIL, 2006), getNearestDayOfWeek(MONDAY, d(17,
   APRIL, 2006)));
340     assertEquals(d(17, APRIL, 2006), getNearestDayOfWeek(MONDAY, d(18,
   APRIL, 2006)));
341     assertEquals(d(17, APRIL, 2006), getNearestDayOfWeek(MONDAY, d(19,
   APRIL, 2006)));
342     assertEquals(d(17, APRIL, 2006), getNearestDayOfWeek(MONDAY, d(20,
   APRIL, 2006)));
343     assertEquals(d(24, APRIL, 2006), getNearestDayOfWeek(MONDAY, d(21,
   APRIL, 2006)));
344     assertEquals(d(24, APRIL, 2006), getNearestDayOfWeek(MONDAY, d(22,
   APRIL, 2006)));
345
346 //     assertEquals(d(18, APRIL, 2006), getNearestDayOfWeek(TUESDAY,
   d(16, APRIL, 2006)));
347 //     assertEquals(d(18, APRIL, 2006), getNearestDayOfWeek(TUESDAY,
   d(17, APRIL, 2006)));
348     assertEquals(d(18, APRIL, 2006), getNearestDayOfWeek(TUESDAY, d(18,
   APRIL, 2006)));
349     assertEquals(d(18, APRIL, 2006), getNearestDayOfWeek(TUESDAY, d(19,
   APRIL, 2006)));
350     assertEquals(d(18, APRIL, 2006), getNearestDayOfWeek(TUESDAY, d(20,
   APRIL, 2006)));
351     assertEquals(d(18, APRIL, 2006), getNearestDayOfWeek(TUESDAY, d(21,
   APRIL, 2006)));
352     assertEquals(d(25, APRIL, 2006), getNearestDayOfWeek(TUESDAY, d(22,
   APRIL, 2006)));
353
354 //     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(16, APRIL, 2006)));
355 //     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(17, APRIL, 2006)));
356 //     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(18, APRIL, 2006)));
357     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(19, APRIL, 2006)));
358     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(20, APRIL, 2006)));
359     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(21, APRIL, 2006)));
```

```
360     assertEquals(d(19, APRIL, 2006), getNearestDayOfWeek(WEDNESDAY,
   d(22, APRIL, 2006)));
361
362 //     assertEquals(d(13, APRIL, 2006), getNearestDayOfWeek(THURSDAY,
   d(16, APRIL, 2006)));
363 //     assertEquals(d(20, APRIL, 2006), getNearestDayOfWeek(THURSDAY,
   d(17, APRIL, 2006)));
364 //     assertEquals(d(20, APRIL, 2006), getNearestDayOfWeek(THURSDAY,
   d(18, APRIL, 2006)));
365 //     assertEquals(d(20, APRIL, 2006), getNearestDayOfWeek(THURSDAY,
   d(19, APRIL, 2006)));
366     assertEquals(d(20, APRIL, 2006), getNearestDayOfWeek(THURSDAY, d(20,
   APRIL, 2006)));
367     assertEquals(d(20, APRIL, 2006), getNearestDayOfWeek(THURSDAY, d(21,
   APRIL, 2006)));
368     assertEquals(d(20, APRIL, 2006), getNearestDayOfWeek(THURSDAY, d(22,
   APRIL, 2006)));
369
370 //     assertEquals(d(14, APRIL, 2006), getNearestDayOfWeek(FRIDAY,
   d(16, APRIL, 2006)));
371 //     assertEquals(d(14, APRIL, 2006), getNearestDayOfWeek(FRIDAY,
   d(17, APRIL, 2006)));
372 //     assertEquals(d(21, APRIL, 2006), getNearestDayOfWeek(FRIDAY,
   d(18, APRIL, 2006)));
373 //     assertEquals(d(21, APRIL, 2006), getNearestDayOfWeek(FRIDAY,
   d(19, APRIL, 2006)));
374 //     assertEquals(d(21, APRIL, 2006), getNearestDayOfWeek(FRIDAY,
   d(20, APRIL, 2006)));
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
375     assertEquals(d(21, APRIL, 2006), getNearestDayOfWeek(FRIDAY, d(21,
   APRIL, 2006)));
376     assertEquals(d(21, APRIL, 2006), getNearestDayOfWeek(FRIDAY, d(22,
   APRIL, 2006)));
377
378 //     assertEquals(d(15, APRIL, 2006), getNearestDayOfWeek(SATURDAY,
   d(16, APRIL, 2006)));
379 //     assertEquals(d(15, APRIL, 2006), getNearestDayOfWeek(SATURDAY,
   d(17, APRIL, 2006)));
380 //     assertEquals(d(15, APRIL, 2006), getNearestDayOfWeek(SATURDAY,
   d(18, APRIL, 2006)));
381 //     assertEquals(d(22, APRIL, 2006), getNearestDayOfWeek(SATURDAY,
   d(19, APRIL, 2006)));
382 //     assertEquals(d(22, APRIL, 2006), getNearestDayOfWeek(SATURDAY,
   d(20, APRIL, 2006)));
383 //     assertEquals(d(22, APRIL, 2006), getNearestDayOfWeek(SATURDAY,
   d(21, APRIL, 2006)));
384     assertEquals(d(22, APRIL, 2006), getNearestDayOfWeek(SATURDAY, d(22,
   APRIL, 2006)));
385
386     try {
387        getNearestDayOfWeek(-1, d(1, JANUARY, 2006));
388        fail("Invalid day of week code should throw exception");
```

```
389     } catch (IllegalArgumentException e) {
390     }
391 }
392
393 public void testEndOfCurrentMonth() throws Exception {
394     SerialDate d = SerialDate.createInstance(2);
395     assertEquals(d(31, JANUARY, 2006), d.getEndOfCurrentMonth(d(1,
    JANUARY, 2006)));
396     assertEquals(d(28, FEBRUARY, 2006), d.getEndOfCurrentMonth(d(1,
    FEBRUARY, 2006)));
397     assertEquals(d(31, MARCH, 2006), d.getEndOfCurrentMonth(d(1, MARCH,
    2006)));
398     assertEquals(d(30, APRIL, 2006), d.getEndOfCurrentMonth(d(1, APRIL,
    2006)));
399     assertEquals(d(31, MAY, 2006), d.getEndOfCurrentMonth(d(1, MAY,
    2006)));
400     assertEquals(d(30, JUNE, 2006), d.getEndOfCurrentMonth(d(1, JUNE,
    2006)));
401     assertEquals(d(31, JULY, 2006), d.getEndOfCurrentMonth(d(1, JULY,
    2006)));
402     assertEquals(d(31, AUGUST, 2006), d.getEndOfCurrentMonth(d(1,
    AUGUST, 2006)));
403     assertEquals(d(30, SEPTEMBER, 2006), d.getEndOfCurrentMonth(d(1,
    SEPTEMBER, 2006)));
404     assertEquals(d(31, OCTOBER, 2006), d.getEndOfCurrentMonth(d(1,
    OCTOBER, 2006)));
405     assertEquals(d(30, NOVEMBER, 2006), d.getEndOfCurrentMonth(d(1,
    NOVEMBER, 2006)));
406     assertEquals(d(31, DECEMBER, 2006), d.getEndOfCurrentMonth(d(1,
    DECEMBER, 2006)));
407     assertEquals(d(29, FEBRUARY, 2008), d.getEndOfCurrentMonth(d(1,
    FEBRUARY, 2008)));
408 }
409
410 public void testWeekInMonthToString() throws Exception {
411     assertEquals("First",weekInMonthToString(FIRST_WEEK_IN_MONTH));
412     assertEquals("Second",weekInMonthToString(SECOND_WEEK_IN_MONTH));
413     assertEquals("Third",weekInMonthToString(THIRD_WEEK_IN_MONTH));
414     assertEquals("Fourth",weekInMonthToString(FOURTH_WEEK_IN_MONTH));
415     assertEquals("Last",weekInMonthToString(LAST_WEEK_IN_MONTH));
416
417 //todo     try {
418 //       weekInMonthToString(-1);
419 //       fail("Invalid week code should throw exception");
420 //     } catch (IllegalArgumentException e) {
421 //     }
422 }
423
424 public void testRelativeToString() throws Exception {
425     assertEquals("Preceding",relativeToString(PRECEDING));
426     assertEquals("Nearest",relativeToString(NEAREST));
427     assertEquals("Following",relativeToString(FOLLOWING));
428
429 //todo     try {
430 //       relativeToString(-1000);
431 //       fail("Invalid relative code should throw exception");
432 //     } catch (IllegalArgumentException e) {
433 //     }
```

```
434 }
435
```

Đoạn mã B-4 (tiếp theo)
BobsSerialDateTest.java

```
436    public void testCreateInstanceFromDDMMYYY() throws Exception {
437      SerialDate date = createInstance(1, JANUARY, 1900);
438      assertEquals(1,date.getDayOfMonth());
439      assertEquals(JANUARY,date.getMonth());
440      assertEquals(1900,date.getYYYY());
441      assertEquals(2,date.toSerial());
442    }
443
444    public void testCreateInstanceFromSerial() throws Exception {
445      assertEquals(d(1, JANUARY, 1900),createInstance(2));
446      assertEquals(d(1, JANUARY, 1901), createInstance(367));
447    }
448
449    public void testCreateInstanceFromJavaDate() throws Exception {
450      assertEquals(d(1, JANUARY, 1900), createInstance(new
   GregorianCalendar(1900,0,1).getTime()));
451      assertEquals(d(1, JANUARY, 2006), createInstance(new
   GregorianCalendar(2006,0,1).getTime()));
452    }
453
454 public static void main(String[] args) {
455   junit.textui.TestRunner.run(BobsSerialDateTest.class);
456 }
457 }
```

Đoạn mã B-5
SpreadsheetDate.java

```
 1 /*
   ============================================================================
 2 * JCommon : a free general purpose class library for the Java(tm)
   platform
 3 *
   ============================================================================
 4 *
 5 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
 6 *
 7 * Project Info: http://www.jfree.org/jcommon/index.html
 8 *
 9 * This library is free software; you can redistribute it and/or modify
   it
10 * under the terms of the GNU Lesser General Public License as published
   by
11 * the Free Software Foundation; either version 2.1 of the License, or
12 * (at your option) any later version.
13 *
14 * This library is distributed in the hope that it will be useful, but
```

```
15 * WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY
16 * or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public
17 * License for more details.
18 *
19 * You should have received a copy of the GNU Lesser General Public
20 * License along with this library; if not, write to the Free Software
21 * Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
   02110-1301,
22 * USA.
23 *
24 * [Java is a trademark or registered trademark of Sun Microsystems, Inc.
25 * in the United States and other countries.]
26 *
27 * --------------------
28 * SpreadsheetDate.java
29 * --------------------
30 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
31 *
32 * Original Author: David Gilbert (for Object Refinery Limited);
33 * Contributor(s): -;
34 *
35 * $Id: SpreadsheetDate.java,v 1.8 2005/11/03 09:25:39 mungady Exp $
36 *
37 * Changes
38 * -------
39 * 11-Oct-2001 : Version 1 (DG);
40 * 05-Nov-2001 : Added getDescription() and setDescription() methods
   (DG);
41 * 12-Nov-2001 : Changed name from ExcelDate.java to SpreadsheetDate.java
   (DG);
42 *                Fixed a bug in calculating day, month and year from
   serial
43 *                number (DG);
44 * 24-Jan-2002 : Fixed a bug in calculating the serial number from the
   day,
45 *                month and year. Thanks to Trevor Hills for the report
   (DG);
46 * 29-May-2002 : Added equals(Object) method (SourceForge ID 558850)
   (DG);
47 * 03-Oct-2002 : Fixed errors reported by Checkstyle (DG);
48 * 13-Mar-2003 : Implemented Serializable (DG);
49 * 04-Sep-2003 : Completed isInRange() methods (DG);
50 * 05-Sep-2003 : Implemented Comparable (DG);
51 * 21-Oct-2003 : Added hashCode() method (DG);
52 *
53 */
54
55 package org.jfree.date;
56
57 import java.util.Calendar;
58 import java.util.Date;
59
60 /**
61 * Represents a date using an integer, in a similar fashion to the
62 * implementation in Microsoft Excel. The range of dates supported is
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
 63 * 1-Jan-1900 to 31-Dec-9999.
 64 * <P>
 65 * Be aware that there is a deliberate bug in Excel that recognises the
   year
 66 * 1900 as a leap year when in fact it is not a leap year. You can find
   more
 67 * information on the Microsoft website in article Q181370:
 68 * <P>
 69 * http://support.microsoft.com/support/kb/articles/Q181/3/70.asp
 70 * <P>
 71 * Excel uses the convention that 1-Jan-1900 = 1. This class uses the
 72 * convention 1-Jan-1900 = 2.
 73 * The result is that the day number in this class will be different to
   the
 74 * Excel figure for January and February 1900...but then Excel adds in an
   extra
 75 * day (29-Feb-1900 which does not actually exist!) and from that point
   forward
 76 * the day numbers will match.
 77 *
 78 * @author David Gilbert
 79 */
 80 public class SpreadsheetDate extends SerialDate {
 81
 82     /** For serialization. */
 83     private static final long serialVersionUID = -2039586705374454461L;
 84
 85     /**
 86      * The day number (1-Jan-1900 = 2, 2-Jan-1900 = 3, ..., 31-Dec-9999
   =
 87      * 2958465).
 88      */
 89     private int serial;
 90
 91     /** The day of the month (1 to 28, 29, 30 or 31 depending on the
   month). */
 92     private int day;
 93
 94     /** The month of the year (1 to 12). */
 95     private int month;
 96
 97     /** The year (1900 to 9999). */
 98     private int year;
 99
100     /** An optional description for the date. */
101     private String description;
102
103     /**
104      * Creates a new date instance.
105      *
106      * @param day the day (in the range 1 to 28/29/30/31).
107      * @param month the month (in the range 1 to 12).
108      * @param year the year (in the range 1900 to 9999).
```

```
109      */
110     public SpreadsheetDate(final int day, final int month, final int
   year) {
111
112         if ((year >= 1900) && (year <= 9999)) {
113             this.year = year;
114         }
115         else {
116             throw new IllegalArgumentException(
117                  "The 'year' argument must be in range 1900 to 9999."
118             );
119         }
120
121         if ((month >= MonthConstants.JANUARY)
122                  && (month <= MonthConstants.DECEMBER)) {
123             this.month = month;
124         }
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
125         else {
126             throw new IllegalArgumentException(
127                 "The 'month' argument must be in the range 1 to 12."
128             );
129         }
130
131         if ((day >= 1) && (day <= SerialDate.lastDayOfMonth(month,
   year))) {
132             this.day = day;
133         }
134         else {
135             throw new IllegalArgumentException("Invalid 'day'
   argument.");
136         }
137
138         // the serial number needs to be synchronised with the
   day-month-year...
139         this.serial = calcSerial(day, month, year);
140
141         this.description = null;
142
143     }
144
145     /**
146      * Standard constructor - creates a new date object representing the
147      * specified day number (which should be in the range 2 to 2958465.
148      *
149      * @param serial the serial number for the day (range: 2 to
   2958465).
150      */
151     public SpreadsheetDate(final int serial) {
152
153         if ((serial >= SERIAL_LOWER_BOUND) && (serial <=
   SERIAL_UPPER_BOUND)) {
154             this.serial = serial;
```

```
155         }
156         else {
157             throw new IllegalArgumentException(
158                 "SpreadsheetDate: Serial must be in range 2 to
   2958465.");
159         }
160
161         // the day-month-year needs to be synchronised with the serial
   number...
162         calcDayMonthYear();
163
164     }
165
166     /**
167       * Returns the description that is attached to the date. It is not
168       * required that a date have a description, but for some
   applications it
169       * is useful.
170       *
171       * @return The description that is attached to the date.
172       */
173     public String getDescription() {
174          return this.description;
175     }
176
177     /**
178       * Sets the description for the date.
179       *
180       * @param description the description for this date
   (<code>null</code>
181       *                    permitted).
182       */
183     public void setDescription(final String description) {
184          this.description = description;
185     }
186
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
187     /**
188       * Returns the serial number for the date, where 1 January 1900 = 2
189       * (this corresponds, almost, to the numbering system used in
   Microsoft
190       * Excel for Windows and Lotus 1-2-3).
191       *
192       * @return The serial number of this date.
193       */
194     public int toSerial() {
195          return this.serial;
196     }
197
198     /**
199       * Returns a <code>java.util.Date</code> equivalent to this date.
200       *
201       * @return The date.
```

```
202       */
203     public Date toDate() {
204          final Calendar calendar = Calendar.getInstance();
205          calendar.set(getYYYY(), getMonth() - 1, getDayOfMonth(), 0, 0,
   0);
206          return calendar.getTime();
207     }
208
209     /**
210       * Returns the year (assume a valid range of 1900 to 9999).
211       *
212       * @return The year.
213       */
214     public int getYYYY() {
215          return this.year;
216     }
217
218     /**
219       * Returns the month (January = 1, February = 2, March = 3).
220       *
221       * @return The month of the year.
222       */
223     public int getMonth() {
224          return this.month;
225     }
226
227     /**
228       * Returns the day of the month.
229       *
230       * @return The day of the month.
231       */
232     public int getDayOfMonth() {
233          return this.day;
234     }
235
236     /**
237       * Returns a code representing the day of the week.
238       * <P>
239       * The codes are defined in the {@link SerialDate} class as:
240       * <code>SUNDAY</code>, <code>MONDAY</code>, <code>TUESDAY</code>,
241       * <code>WEDNESDAY</code>, <code>THURSDAY</code>,
   <code>FRIDAY</code>, and
242       * <code>SATURDAY</code>.
243       *
244       * @return A code representing the day of the week.
245       */
246     public int getDayOfWeek() {
247          return (this.serial + 6) % 7 + 1;
248     }
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
249
250     /**
251      * Tests the equality of this date with an arbitrary object.
```

```
252      * <P>
253      * This method will return true ONLY if the object is an instance of
   the
254      * {@link SerialDate} base class, and it represents the same day as
   this
255      * {@link SpreadsheetDate}.
256      *
257      * @param object the object to compare (<code>null</code>
   permitted).
258      *
259      * @return A boolean.
260      */
261     public boolean equals(final Object object) {
262
263         if (object instanceof SerialDate) {
264             final SerialDate s = (SerialDate) object;
265             return (s.toSerial() == this.toSerial());
266         }
267         else {
268             return false;
269         }
270
271     }
272
273     /**
274       * Returns a hash code for this object instance.
275       *
276       * @return A hash code.
277       */
278     public int hashCode() {
279          return toSerial();
280     }
281
282     /**
283       * Returns the difference (in days) between this date and the
   specified
284       * 'other' date.
285       *
286       * @param other the date being compared to.
287       *
288       * @return The difference (in days) between this date and the
   specified
289       *         'other' date.
290       */
291     public int compare(final SerialDate other) {
292          return this.serial - other.toSerial();
293     }
294
295     /**
296       * Implements the method required by the Comparable interface.
297       *
298       * @param other the other object (usually another SerialDate).
299       *
300       * @return A negative integer, zero, or a positive integer as this
   object
301       *         is less than, equal to, or greater than the specified
   object.
302       */
```

```
303     public int compareTo(final Object other) {
304          return compare((SerialDate) other);
305     }
306
307     /**
308      * Returns true if this SerialDate represents the same date as the
309      * specified SerialDate.
310      *
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
311       * @param other the date being compared to.
312       *
313       * @return <code>true</code> if this SerialDate represents the same
   date as
314       *         the specified SerialDate.
315       */
316     public boolean isOn(final SerialDate other) {
317          return (this.serial == other.toSerial());
318     }
319
320     /**
321       * Returns true if this SerialDate represents an earlier date
   compared to
322       * the specified SerialDate.
323       *
324       * @param other the date being compared to.
325       *
326       * @return <code>true</code> if this SerialDate represents an
   earlier date
327       *         compared to the specified SerialDate.
328       */
329     public boolean isBefore(final SerialDate other) {
330          return (this.serial < other.toSerial());
331     }
332
333     /**
334       * Returns true if this SerialDate represents the same date as the
335       * specified SerialDate.
336       *
337       * @param other the date being compared to.
338       *
339       * @return <code>true</code> if this SerialDate represents the same
   date
340       *         as the specified SerialDate.
341       */
342     public boolean isOnOrBefore(final SerialDate other) {
343          return (this.serial <= other.toSerial());
344     }
345
346     /**
347       * Returns true if this SerialDate represents the same date as the
348       * specified SerialDate.
349       *
350       * @param other the date being compared to.
```

```
351       *
352       * @return <code>true</code> if this SerialDate represents the same
   date
353       *         as the specified SerialDate.
354       */
355     public boolean isAfter(final SerialDate other) {
356          return (this.serial > other.toSerial());
357     }
358
359     /**
360       * Returns true if this SerialDate represents the same date as the
361       * specified SerialDate.
362       *
363       * @param other the date being compared to.
364       *
365       * @return <code>true</code> if this SerialDate represents the same
   date as
366       *         the specified SerialDate.
367       */
368     public boolean isOnOrAfter(final SerialDate other) {
369          return (this.serial >= other.toSerial());
370     }
371
372     /**
373      * Returns <code>true</code> if this {@link SerialDate} is within
   the
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
374       * specified range (INCLUSIVE). The date order of d1 and d2 is not
375       * important.
376       *
377       * @param d1 a boundary date for the range.
378       * @param d2 the other boundary date for the range.
379       *
380       * @return A boolean.
381       */
382     public boolean isInRange(final SerialDate d1, final SerialDate d2) {
383          return isInRange(d1, d2, SerialDate.INCLUDE_BOTH);
384     }
385
386     /**
387      * Returns true if this SerialDate is within the specified range
   (caller
388      * specifies whether or not the end-points are included). The order
   of d1
389      * and d2 is not important.
390      *
391      * @param d1 one boundary date for the range.
392      * @param d2 a second boundary date for the range.
393      * @param include a code that controls whether or not the start and
   end
394      *                dates are included in the range.
395      *
```

```
396      * @return <code>true</code> if this SerialDate is within the
   specified
397      *         range.
398      */
399     public boolean isInRange(final SerialDate d1, final SerialDate d2,
400                              final int include) {
401         final int s1 = d1.toSerial();
402         final int s2 = d2.toSerial();
403         final int start = Math.min(s1, s2);
404         final int end = Math.max(s1, s2);
405
406         final int s = toSerial();
407         if (include == SerialDate.INCLUDE_BOTH) {
408             return (s >= start && s <= end);
409         }
410         else if (include == SerialDate.INCLUDE_FIRST) {
411             return (s >= start && s < end);
412         }
413         else if (include == SerialDate.INCLUDE_SECOND) {
414             return (s > start && s <= end);
415         }
416         else {
417             return (s > start && s < end);
418         }
419     }
420
421     /**
422      * Calculate the serial number from the day, month and year.
423      * <P>
424      * 1-Jan-1900 = 2.
425      *
426      * @param d the day.
427      * @param m the month.
428      * @param y the year.
429      *
430      * @return the serial number from the day, month and year.
431      */
432     private int calcSerial(final int d, final int m, final int y) {
433         final int yy = ((y - 1900) * 365) + SerialDate.leapYearCount(y -
   1);
434         int mm = SerialDate.AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH[m];
435         if (m > MonthConstants.FEBRUARY) {
```

Đoạn mã B-5 (tiếp theo)
SpreadsheetDate.java

```
436             if (SerialDate.isLeapYear(y)) {
437                 mm = mm + 1;
438             }
439         }
440         final int dd = d;
441         return yy + mm + dd + 1;
442     }
443
444     /**
445      * Calculate the day, month and year from the serial number.
```

```
446      */
447     private void calcDayMonthYear() {
448
449         // get the year from the serial date
450         final int days = this.serial - SERIAL_LOWER_BOUND;
451         // overestimated because we ignored leap days
452         final int overestimatedYYYY = 1900 + (days / 365);
453         final int leaps = SerialDate.leapYearCount(overestimatedYYYY);
454         final int nonleapdays = days - leaps;
455         // underestimated because we overestimated years
456         int underestimatedYYYY = 1900 + (nonleapdays / 365);
457
458         if (underestimatedYYYY == overestimatedYYYY) {
459             this.year = underestimatedYYYY;
460         }
461         else {
462             int ss1 = calcSerial(1, 1, underestimatedYYYY);
463             while (ss1 <= this.serial) {
464                 underestimatedYYYY = underestimatedYYYY + 1;
465                 ss1 = calcSerial(1, 1, underestimatedYYYY);
466             }
467             this.year = underestimatedYYYY - 1;
468         }
469
470         final int ss2 = calcSerial(1, 1, this.year);
471
472         int[] daysToEndOfPrecedingMonth
473             = AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH;
474
475         if (isLeapYear(this.year)) {
476             daysToEndOfPrecedingMonth
477                 = LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH;
478         }
479
480         // get the month from the serial date
481         int mm = 1;
482         int sss = ss2 + daysToEndOfPrecedingMonth[mm] - 1;
483         while (sss < this.serial) {
484             mm = mm + 1;
485             sss = ss2 + daysToEndOfPrecedingMonth[mm] - 1;
486         }
487         this.month = mm - 1;
488
489         // what's left is d(+1);
490         this.day = this.serial - ss2
491                    - daysToEndOfPrecedingMonth[this.month] + 1;
492
493     }
494
495 }
```

Đoạn mã B-6
RelativeDayOfWeekRule.java

```
1 /*
 ===========================================================================
```

```
 2 * JCommon : a free general purpose class library for the Java(tm)
  platform
 3 *
  ===========================================================================
 4 *
 5 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
 6 *
 7 * Project Info: http://www.jfree.org/jcommon/index.html
 8 *
 9 * This library is free software; you can redistribute it and/or modify
  it
10 * under the terms of the GNU Lesser General Public License as published
  by
11 * the Free Software Foundation; either version 2.1 of the License, or
12 * (at your option) any later version.
13 *
14 * This library is distributed in the hope that it will be useful, but
15 * WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY
16 * or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public
17 * License for more details.
18 *
19 * You should have received a copy of the GNU Lesser General Public
20 * License along with this library; if not, write to the Free Software
21 * Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
  02110-1301,
22 * USA.
23 *
24 * [Java is a trademark or registered trademark of Sun Microsystems, Inc.
25 * in the United States and other countries.]
26 *
27 * --------------------------
28 * RelativeDayOfWeekRule.java
29 * --------------------------
30 * (C) Copyright 2000-2003, by Object Refinery Limited and Contributors.
31 *
32 * Original Author: David Gilbert (for Object Refinery Limited);
33 * Contributor(s): -;
34 *
35 * $Id: RelativeDayOfWeekRule.java,v 1.6 2005/11/16 15:58:40 taqua Exp $
36 *
37 * Changes (from 26-Oct-2001)
38 * --------------------------
39 * 26-Oct-2001 : Changed package to com.jrefinery.date.*;
40 * 03-Oct-2002 : Fixed errors reported by Checkstyle (DG);
41 *
42 */
43
44 package org.jfree.date;
45
46 /**
47 * An annual date rule that returns a date for each year based on (a) a
48 * reference rule; (b) a day of the week; and (c) a selection parameter
49 * (SerialDate.PRECEDING, SerialDate.NEAREST, SerialDate.FOLLOWING).
50 * <P>
51 * For example, Good Friday can be specified as 'the Friday PRECEDING
  Easter
52 * Sunday'.
```

```
53 *
54 * @author David Gilbert
55 */
56 public class RelativeDayOfWeekRule extends AnnualDateRule {
57
58     /** A reference to the annual date rule on which this rule is based.
  */
59     private AnnualDateRule subrule;
60
61     /**
62      * The day of the week (SerialDate.MONDAY, SerialDate.TUESDAY, and
  so on).
```

Đoạn mã B-6 (tiếp theo)
RelativeDayOfWeekRule.java

```
63      */
64     private int dayOfWeek;
65
66     /** Specifies which day of the week (PRECEDING, NEAREST or
  FOLLOWING). */
67     private int relative;
68
69     /**
70       * Default constructor - builds a rule for the Monday following 1
  January.
71       */
72     public RelativeDayOfWeekRule() {
73          this(new DayAndMonthRule(), SerialDate.MONDAY,
  SerialDate.FOLLOWING);
74     }
75
76     /**
77       * Standard constructor - builds rule based on the supplied
  sub-rule.
78       *
79       * @param subrule the rule that determines the reference date.
80       * @param dayOfWeek the day-of-the-week relative to the reference
  date.
81       * @param relative indicates *which* day-of-the-week (preceding,
  nearest
82       *                  or following).
83       */
84     public RelativeDayOfWeekRule(final AnnualDateRule subrule,
85              final int dayOfWeek, final int relative) {
86          this.subrule = subrule;
87          this.dayOfWeek = dayOfWeek;
88          this.relative = relative;
89     }
90
91     /**
92       * Returns the sub-rule (also called the reference rule).
93       *
94       * @return The annual date rule that determines the reference date
  for this
95       *         rule.
```

```
 96       */
 97     public AnnualDateRule getSubrule() {
 98          return this.subrule;
 99     }
100
101     /**
102       * Sets the sub-rule.
103       *
104       * @param subrule the annual date rule that determines the
   reference date
105       *                for this rule.
106       */
107     public void setSubrule(final AnnualDateRule subrule) {
108          this.subrule = subrule;
109     }
110
111     /**
112       * Returns the day-of-the-week for this rule.
113       *
114       * @return the day-of-the-week for this rule.
115       */
116     public int getDayOfWeek() {
117          return this.dayOfWeek;
118     }
119
120     /**
121      * Sets the day-of-the-week for this rule.
122      *
123      * @param dayOfWeek the day-of-the-week (SerialDate.MONDAY,
124      *                   SerialDate.TUESDAY, and so on).
```

Đoạn mã B-6 (tiếp theo)
RelativeDayOfWeekRule.java

```
125       */
126     public void setDayOfWeek(final int dayOfWeek) {
127          this.dayOfWeek = dayOfWeek;
128     }
129
130     /**
131       * Returns the 'relative' attribute, that determines *which*
132       * day-of-the-week we are interested in (SerialDate.PRECEDING,
133       * SerialDate.NEAREST or SerialDate.FOLLOWING).
134       *
135       * @return The 'relative' attribute.
136       */
137     public int getRelative() {
138          return this.relative;
139     }
140
141     /**
142       * Sets the 'relative' attribute (SerialDate.PRECEDING,
   SerialDate.NEAREST,
143       * SerialDate.FOLLOWING).
144       *
```

```
145       * @param relative determines *which* day-of-the-week is selected
   by this
146       *                  rule.
147       */
148     public void setRelative(final int relative) {
149          this.relative = relative;
150     }
151
152     /**
153       * Creates a clone of this rule.
154       *
155       * @return a clone of this rule.
156       *
157       * @throws CloneNotSupportedException this should never happen.
158       */
159     public Object clone() throws CloneNotSupportedException {
160          final RelativeDayOfWeekRule duplicate
161              = (RelativeDayOfWeekRule) super.clone();
162          duplicate.subrule = (AnnualDateRule)
   duplicate.getSubrule().clone();
163          return duplicate;
164     }
165
166     /**
167      * Returns the date generated by this rule, for the specified year.
168      *
169      * @param year the year (1900 &lt;= year &lt;= 9999).
170      *
171      * @return The date generated by the rule for the given year
   (possibly
172      *         <code>null</code>).
173      */
174     public SerialDate getDate(final int year) {
175
176         // check argument...
177         if ((year < SerialDate.MINIMUM_YEAR_SUPPORTED)
178             || (year > SerialDate.MAXIMUM_YEAR_SUPPORTED)) {
179             throw new IllegalArgumentException(
180                 "RelativeDayOfWeekRule.getDate(): year outside valid
   range.");
181         }
182
183         // calculate the date...
184         SerialDate result = null;
185         final SerialDate base = this.subrule.getDate(year);
186
```

Đoạn mã B-6 (tiếp theo)
RelativeDayOfWeekRule.java

```
187         if (base != null) {
188             switch (this.relative) {
189                 case(SerialDate.PRECEDING):
190                     result =
   SerialDate.getPreviousDayOfWeek(this.dayOfWeek,
191                             base);
```

```
192                     break;
193                 case(SerialDate.NEAREST):
194                     result =
   SerialDate.getNearestDayOfWeek(this.dayOfWeek,
195                             base);
196                     break;
197                 case(SerialDate.FOLLOWING):
198                     result =
   SerialDate.getFollowingDayOfWeek(this.dayOfWeek,
199                             base);
200                     break;
201                 default:
202                     break;
203             }
204         }
205         return result;
206
207     }
208
209 }
```

Đoạn mã B-7
DayDate.java (Final)

```
  1 /*
   ===========================================================================
  2 * JCommon : a free general purpose class library for the Java(tm)
   platform
  3 *
   ===========================================================================
  4 *
  5 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
...
 36 */
 37 package org.jfree.date;
 38
 39 import java.io.Serializable;
 40 import java.util.*;
 41
 42 /**
 43 * An abstract class that represents immutable dates with a precision of
 44 * one day. The implementation will map each date to an integer that
 45 * represents an ordinal number of days from some fixed origin.
 46 *
 47 * Why not just use java.util.Date? We will, when it makes sense. At
    times,
 48 * java.util.Date can be *too* precise - it represents an instant in
    time,
 49 * accurate to 1/1000th of a second (with the date itself depending on
    the
 50 * time-zone). Sometimes we just want to represent a particular day (e.g.
    21
 51 * January 2015) without concerning ourselves about the time of day, or
    the
 52 * time-zone, or anything else. That's what we've defined DayDate for.
 53 *
```

```
 54 * Use DayDateFactory.makeDate to create an instance.
 55 *
 56 * @author David Gilbert
 57 * @author Robert C. Martin did a lot of refactoring.
 58 */
 59
 60 public abstract class DayDate implements Comparable, Serializable {
 61 public abstract int getOrdinalDay();
 62 public abstract int getYear();
 63 public abstract Month getMonth();
 64 public abstract int getDayOfMonth();
 65
 66 protected abstract Day getDayOfWeekForOrdinalZero();
 67
 68 public DayDate plusDays(int days) {
 69     return DayDateFactory.makeDate(getOrdinalDay() + days);
 70 }
 71
 72 public DayDate plusMonths(int months) {
 73     int thisMonthAsOrdinal = getMonth().toInt() - Month.JANUARY.toInt();
 74     int thisMonthAndYearAsOrdinal = 12 * getYear() + thisMonthAsOrdinal;
 75     int resultMonthAndYearAsOrdinal = thisMonthAndYearAsOrdinal +
   months;
 76     int resultYear = resultMonthAndYearAsOrdinal / 12;
 77     int resultMonthAsOrdinal = resultMonthAndYearAsOrdinal % 12 +
   Month.JANUARY.toInt();
 78     Month resultMonth = Month.fromInt(resultMonthAsOrdinal);
 79     int resultDay = correctLastDayOfMonth(getDayOfMonth(), resultMonth,
   resultYear);
 80     return DayDateFactory.makeDate(resultDay, resultMonth, resultYear);
 81 }
 82
 83 public DayDate plusYears(int years) {
 84     int resultYear = getYear() + years;
 85     int resultDay = correctLastDayOfMonth(getDayOfMonth(), getMonth(),
   resultYear);
 86     return DayDateFactory.makeDate(resultDay, getMonth(), resultYear);
 87 }
 88
 89 private int correctLastDayOfMonth(int day, Month month, int year) {
 90     int lastDayOfMonth = DateUtil.lastDayOfMonth(month, year);
 91     if (day > lastDayOfMonth)
```

Đoạn mã B-7 (tiếp theo)
DayDate.java (Final)

```
 92         day = lastDayOfMonth;
 93       return day;
 94    }
 95
 96    public DayDate getPreviousDayOfWeek(Day targetDayOfWeek) {
 97      int offsetToTarget = targetDayOfWeek.toInt() -
   getDayOfWeek().toInt();
 98      if (offsetToTarget >= 0)
 99        offsetToTarget -= 7;
100      return plusDays(offsetToTarget);
```

```
101    }
102
103    public DayDate getFollowingDayOfWeek(Day targetDayOfWeek) {
104      int offsetToTarget = targetDayOfWeek.toInt() -
   getDayOfWeek().toInt();
105      if (offsetToTarget <= 0)
106        offsetToTarget += 7;
107      return plusDays(offsetToTarget);
108    }
109
110    public DayDate getNearestDayOfWeek(Day targetDayOfWeek) {
111      int offsetToThisWeeksTarget = targetDayOfWeek.toInt() -
   getDayOfWeek().toInt();
112      int offsetToFutureTarget = (offsetToThisWeeksTarget + 7) % 7;
113      int offsetToPreviousTarget = offsetToFutureTarget - 7;
114
115        if (offsetToFutureTarget > 3)
116          return plusDays(offsetToPreviousTarget);
117        else
118          return plusDays(offsetToFutureTarget);
119    }
120
121    public DayDate getEndOfMonth() {
122      Month month = getMonth();
123      int year = getYear();
124      int lastDay = DateUtil.lastDayOfMonth(month, year);
125      return DayDateFactory.makeDate(lastDay, month, year);
126    }
127
128    public Date toDate() {
129      final Calendar calendar = Calendar.getInstance();
130      int ordinalMonth = getMonth().toInt() - Month.JANUARY.toInt();
131      calendar.set(getYear(), ordinalMonth, getDayOfMonth(), 0, 0, 0);
132      return calendar.getTime();
133    }
134
135    public String toString() {
136      return String.format("%02d-%s-%d", getDayOfMonth(), getMonth(),
   getYear());
137    }
138
139    public Day getDayOfWeek() {
140      Day startingDay = getDayOfWeekForOrdinalZero();
141      int startingOffset = startingDay.toInt() - Day.SUNDAY.toInt();
142      int ordinalOfDayOfWeek = (getOrdinalDay() + startingOffset) % 7;
143      return Day.fromInt(ordinalOfDayOfWeek + Day.SUNDAY.toInt());
144    }
145
146    public int daysSince(DayDate date) {
147      return getOrdinalDay() - date.getOrdinalDay();
148    }
149
150    public boolean isOn(DayDate other) {
151      return getOrdinalDay() == other.getOrdinalDay();
152    }
153
```

Đoạn mã B-7 (tiếp theo)
DayDate.java (Final)

```
154 public boolean isBefore(DayDate other) {
155   return getOrdinalDay() < other.getOrdinalDay();
156 }
157
158 public boolean isOnOrBefore(DayDate other) {
159   return getOrdinalDay() <= other.getOrdinalDay();
160 }
161
162 public boolean isAfter(DayDate other) {
163   return getOrdinalDay() > other.getOrdinalDay();
164 }
165
166 public boolean isOnOrAfter(DayDate other) {
167   return getOrdinalDay() >= other.getOrdinalDay();
168 }
169
170 public boolean isInRange(DayDate d1, DayDate d2) {
171   return isInRange(d1, d2, DateInterval.CLOSED);
172 }
173
174 public boolean isInRange(DayDate d1, DayDate d2, DateInterval interval)
    {
175   int left = Math.min(d1.getOrdinalDay(), d2.getOrdinalDay());
176   int right = Math.max(d1.getOrdinalDay(), d2.getOrdinalDay());
177   return interval.isIn(getOrdinalDay(), left, right);
178 }
179 }
```

Đoạn mã B-8
Month.java (Final)

```
  1 package org.jfree.date;
  2
  3 import java.text.DateFormatSymbols;
  4
  5 public enum Month {
  6 JANUARY(1), FEBRUARY(2), MARCH(3),
  7 APRIL(4), MAY(5),          JUNE(6),
  8 JULY(7),      AUGUST(8), SEPTEMBER(9),
  9 OCTOBER(10),NOVEMBER(11),DECEMBER(12);
 10 private static DateFormatSymbols dateFormatSymbols = new
    DateFormatSymbols();
 11 private static final int[] LAST_DAY_OF_MONTH =
 12     {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
 13
 14 private int index;
 15
 16 Month(int index) {
 17     this.index = index;
 18 }
 19
```

```
20 public static Month fromInt(int monthIndex) {
21     for (Month m : Month.values()) {
22       if (m.index == monthIndex)
23         return m;
24     }
25     throw new IllegalArgumentException("Invalid month index " +
  monthIndex);
26 }
27
28 public int lastDay() {
29     return LAST_DAY_OF_MONTH[index];
30 }
31
32 public int quarter() {
33     return 1 + (index - 1) / 3;
34 }
35
36 public String toString() {
37     return dateFormatSymbols.getMonths()[index - 1];
38 }
39
40 public String toShortString() {
41     return dateFormatSymbols.getShortMonths()[index - 1];
42 }
43
44 public static Month parse(String s) {
45     s = s.trim();
46     for (Month m : Month.values())
47       if (m.matches(s))
48         return m;
49
50     try {
51       return fromInt(Integer.parseInt(s));
52     }
53     catch (NumberFormatException e) {}
54     throw new IllegalArgumentException("Invalid month " + s);
55 }
56
57 private boolean matches(String s) {
58     return s.equalsIgnoreCase(toString()) ||
59            s.equalsIgnoreCase(toShortString());
60 }
61
62 public int toInt() {
63     return index;
64 }
65 }
```

Đoạn mã B-9
Day.java (Final)

```
 1 package org.jfree.date;
 2
 3 import java.util.Calendar;
 4 import java.text.DateFormatSymbols;
 5
```

```
 6 public enum Day {
 7 MONDAY(Calendar.MONDAY),
 8 TUESDAY(Calendar.TUESDAY),
 9 WEDNESDAY(Calendar.WEDNESDAY),
10 THURSDAY(Calendar.THURSDAY),
11 FRIDAY(Calendar.FRIDAY),
12 SATURDAY(Calendar.SATURDAY),
13 SUNDAY(Calendar.SUNDAY);
14
15 private final int index;
16 private static DateFormatSymbols dateSymbols = new DateFormatSymbols();
17
18 Day(int day) {
19     index = day;
20 }
21
22 public static Day fromInt(int index) throws IllegalArgumentException {
23     for (Day d : Day.values())
24       if (d.index == index)
25         return d;
26     throw new IllegalArgumentException(
27       String.format("Illegal day index: %d.", index));
28 }
29
30 public static Day parse(String s) throws IllegalArgumentException {
31     String[] shortWeekdayNames =
32       dateSymbols.getShortWeekdays();
33     String[] weekDayNames =
34       dateSymbols.getWeekdays();
35
36     s = s.trim();
37     for (Day day : Day.values()) {
38       if (s.equalsIgnoreCase(shortWeekdayNames[day.index]) ||
39           s.equalsIgnoreCase(weekDayNames[day.index])) {
40         return day;
41       }
42     }
43     throw new IllegalArgumentException(
44       String.format("%s is not a valid weekday string", s));
45 }
46
47 public String toString() {
48     return dateSymbols.getWeekdays()[index];
49 }
50
51 public int toInt() {
52     return index;
53 }
54 }
```

Đoạn mã B-10
DateInterval.java (Final)

```
 1 package org.jfree.date;
 2
 3 public enum DateInterval {
```

```
 4 OPEN {
 5     public boolean isIn(int d, int left, int right) {
 6       return d > left && d < right;
 7     }
 8 },
 9 CLOSED_LEFT {
10     public boolean isIn(int d, int left, int right) {
11       return d >= left && d < right;
12     }
13 },
14 CLOSED_RIGHT {
15     public boolean isIn(int d, int left, int right) {
16       return d > left && d <= right;
17     }
18 },
19 CLOSED {
20     public boolean isIn(int d, int left, int right) {
21       return d >= left && d <= right;
22     }
23 };
24
25 public abstract boolean isIn(int d, int left, int right);
26 }
```

Đoạn mã B-11
WeekInMonth.java (Final)

```
 1 package org.jfree.date;
 2
 3 public enum WeekInMonth {
 4 FIRST(1), SECOND(2), THIRD(3), FOURTH(4), LAST(0);
 5 private final int index;
 6
 7 WeekInMonth(int index) {
 8     this.index = index;
 9 }
10
11 public int toInt() {
12     return index;
13 }
14 }
```

Đoạn mã B-12
WeekdayRange.java (Final)

```
1 package org.jfree.date;
2
3 public enum WeekdayRange {
4 LAST, NEAREST, NEXT
5 }
```

Đoạn mã B-13
DateUtil.java (Final)

```
 1 package org.jfree.date;
 2
 3 import java.text.DateFormatSymbols;
 4
 5 public class DateUtil {
 6 private static DateFormatSymbols dateFormatSymbols = new
  DateFormatSymbols();
 7
 8 public static String[] getMonthNames() {
 9     return dateFormatSymbols.getMonths();
10 }
11
12 public static boolean isLeapYear(int year) {
13     boolean fourth = year % 4 == 0;
14     boolean hundredth = year % 100 == 0;
15     boolean fourHundredth = year % 400 == 0;
16     return fourth && (!hundredth || fourHundredth);
17 }
18
19 public static int lastDayOfMonth(Month month, int year) {
20     if (month == Month.FEBRUARY && isLeapYear(year))
21       return month.lastDay() + 1;
22     else
23       return month.lastDay();
24 }
25
26 public static int leapYearCount(int year) {
27     int leap4 = (year - 1896) / 4;
28     int leap100 = (year - 1800) / 100;
29     int leap400 = (year - 1600) / 400;
30     return leap4 - leap100 + leap400;
31 }
32 }
```

Đoạn mã B-14
DayDateFactory.java (Final)

```
 1 package org.jfree.date;
 2
 3 public abstract class DayDateFactory {
 4 private static DayDateFactory factory = new SpreadsheetDateFactory();
 5 public static void setInstance(DayDateFactory factory) {
 6     DayDateFactory.factory = factory;
 7 }
 8
 9 protected abstract DayDate _makeDate(int ordinal);
10 protected abstract DayDate _makeDate(int day, Month month, int year);
11 protected abstract DayDate _makeDate(int day, int month, int year);
12 protected abstract DayDate _makeDate(java.util.Date date);
13 protected abstract int _getMinimumYear();
14 protected abstract int _getMaximumYear();
```

```
15
16 public static DayDate makeDate(int ordinal) {
17     return factory._makeDate(ordinal);
18 }
19
20 public static DayDate makeDate(int day, Month month, int year) {
21     return factory._makeDate(day, month, year);
22 }
23
24 public static DayDate makeDate(int day, int month, int year) {
25     return factory._makeDate(day, month, year);
26 }
27
28 public static DayDate makeDate(java.util.Date date) {
29     return factory._makeDate(date);
30 }
31
32 public static int getMinimumYear() {
33     return factory._getMinimumYear();
34 }
35
36 public static int getMaximumYear() {
37     return factory._getMaximumYear();
38 }
39 }
```

Đoạn mã B-15
SpreadsheetDateFactory.java (Final)

```
 1 package org.jfree.date;
 2
 3 import java.util.*;
 4
 5 public class SpreadsheetDateFactory extends DayDateFactory {
 6 public DayDate _makeDate(int ordinal) {
 7     return new SpreadsheetDate(ordinal);
 8 }
 9
10 public DayDate _makeDate(int day, Month month, int year) {
11     return new SpreadsheetDate(day, month, year);
12 }
13
14 public DayDate _makeDate(int day, int month, int year) {
15     return new SpreadsheetDate(day, month, year);
16 }
17
18 public DayDate _makeDate(Date date) {
19     final GregorianCalendar calendar = new GregorianCalendar();
20     calendar.setTime(date);
21     return new SpreadsheetDate(
22       calendar.get(Calendar.DATE),
23       Month.fromInt(calendar.get(Calendar.MONTH) + 1),
24       calendar.get(Calendar.YEAR));
25 }
26
27 protected int _getMinimumYear() {
```

```
    return SpreadsheetDate.MINIMUM_YEAR_SUPPORTED;
}

protected int _getMaximumYear() {
    return SpreadsheetDate.MAXIMUM_YEAR_SUPPORTED;
}
}
```

Đoạn mã B-16
SpreadsheetDate.java (Final)

```
/*
 ===========================================================================
 * JCommon : a free general purpose class library for the Java(tm)
  platform
 *
 ===========================================================================
 *
 * (C) Copyright 2000-2005, by Object Refinery Limited and Contributors.
 *
...
 *
 */

package org.jfree.date;

import static org.jfree.date.Month.FEBRUARY;

import java.util.*;

/**
 * Represents a date using an integer, in a similar fashion to the
 * implementation in Microsoft Excel. The range of dates supported is
 * 1-Jan-1900 to 31-Dec-9999.
 * <p/>
 * Be aware that there is a deliberate bug in Excel that recognises the
  year
 * 1900 as a leap year when in fact it is not a leap year. You can find
  more
 * information on the Microsoft website in article Q181370:
 * <p/>
 * http://support.microsoft.com/support/kb/articles/Q181/3/70.asp
 * <p/>
 * Excel uses the convention that 1-Jan-1900 = 1. This class uses the
 * convention 1-Jan-1900 = 2.
 * The result is that the day number in this class will be different to
  the
 * Excel figure for January and February 1900...but then Excel adds in an
  extra
 * day (29-Feb-1900 which does not actually exist!) and from that point
  forward
 * the day numbers will match.
 *
 * @author David Gilbert
 */
public class SpreadsheetDate extends DayDate {
```

```
 82 public static final int EARLIEST_DATE_ORDINAL = 2;       // 1/1/1900
 83 public static final int LATEST_DATE_ORDINAL = 2958465; // 12/31/9999
 84 public static final int MINIMUM_YEAR_SUPPORTED = 1900;
 85 public static final int MAXIMUM_YEAR_SUPPORTED = 9999;
 86 static final int[] AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH =
 87     {0, 0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365};
 88 static final int[] LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH =
 89     {0, 0, 31, 60, 91, 121, 152, 182, 213, 244, 274, 305, 335, 366};
 90
 91 private int ordinalDay;
 92 private int day;
 93 private Month month;
 94 private int year;
 95
 96 public SpreadsheetDate(int day, Month month, int year) {
 97     if (year < MINIMUM_YEAR_SUPPORTED || year > MAXIMUM_YEAR_SUPPORTED)
 98       throw new IllegalArgumentException(
 99         "The 'year' argument must be in range " +
100         MINIMUM_YEAR_SUPPORTED + " to " + MAXIMUM_YEAR_SUPPORTED + ".");
101     if (day < 1 || day > DateUtil.lastDayOfMonth(month, year))
102       throw new IllegalArgumentException("Invalid 'day' argument.");
103
104     this.year = year;
105     this.month = month;
```

Đoạn mã B-16 (tiếp theo)
SpreadsheetDate.java (Final)

```
106       this.day = day;
107       ordinalDay = calcOrdinal(day, month, year);
108   }
109
110   public SpreadsheetDate(int day, int month, int year) {
111     this(day, Month.fromInt(month), year);
112   }
113
114   public SpreadsheetDate(int serial) {
115     if (serial < EARLIEST_DATE_ORDINAL || serial > LATEST_DATE_ORDINAL)
116       throw new IllegalArgumentException(
117         "SpreadsheetDate: Serial must be in range 2 to 2958465.");
118
119       ordinalDay = serial;
120       calcDayMonthYear();
121   }
122
123   public int getOrdinalDay() {
124     return ordinalDay;
125   }
126
127   public int getYear() {
128     return year;
129   }
130
131   public Month getMonth() {
132     return month;
133   }
```

```
134
135    public int getDayOfMonth() {
136      return day;
137    }
138
139    protected Day getDayOfWeekForOrdinalZero() {return Day.SATURDAY;}
140
141    public boolean equals(Object object) {
142      if (!(object instanceof DayDate))
143        return false;
144
145        DayDate date = (DayDate) object;
146        return date.getOrdinalDay() == getOrdinalDay();
147    }
148
149    public int hashCode() {
150      return getOrdinalDay();
151    }
152
153    public int compareTo(Object other) {
154      return daysSince((DayDate) other);
155    }
156
157    private int calcOrdinal(int day, Month month, int year) {
158      int leapDaysForYear = DateUtil.leapYearCount(year - 1);
159      int daysUpToYear = (year - MINIMUM_YEAR_SUPPORTED) * 365 +
   leapDaysForYear;
160      int daysUpToMonth =
   AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH[month.toInt()];
161      if (DateUtil.isLeapYear(year) && month.toInt() > FEBRUARY.toInt())
162        daysUpToMonth++;
163      int daysInMonth = day - 1;
164      return daysUpToYear + daysUpToMonth + daysInMonth +
   EARLIEST_DATE_ORDINAL;
165    }
166
```

Đoạn mã B-16 (tiếp theo)
SpreadsheetDate.java (Final)

```
167 private void calcDayMonthYear() {
168   int days = ordinalDay - EARLIEST_DATE_ORDINAL;
169   int overestimatedYear = MINIMUM_YEAR_SUPPORTED + days / 365;
170   int nonleapdays = days - DateUtil.leapYearCount(overestimatedYear);
171   int underestimatedYear = MINIMUM_YEAR_SUPPORTED + nonleapdays / 365;
172
173   year = huntForYearContaining(ordinalDay, underestimatedYear);
174   int firstOrdinalOfYear = firstOrdinalOfYear(year);
175   month = huntForMonthContaining(ordinalDay, firstOrdinalOfYear);
176   day = ordinalDay - firstOrdinalOfYear -
   daysBeforeThisMonth(month.toInt());
177 }
178
179 private Month huntForMonthContaining(int anOrdinal, int
   firstOrdinalOfYear) {
180   int daysIntoThisYear = anOrdinal - firstOrdinalOfYear;
```

```
  int aMonth = 1;
  while (daysBeforeThisMonth(aMonth) < daysIntoThisYear)
    aMonth++;

  return Month.fromInt(aMonth - 1);
}

private int daysBeforeThisMonth(int aMonth) {
  if (DateUtil.isLeapYear(year))
    return LEAP_YEAR_AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH[aMonth] -
  1;
  else
    return AGGREGATE_DAYS_TO_END_OF_PRECEDING_MONTH[aMonth] - 1;
}

private int huntForYearContaining(int anOrdinalDay, int startingYear) {
  int aYear = startingYear;
  while (firstOrdinalOfYear(aYear) <= anOrdinalDay)
    aYear++;

  return aYear - 1;
}

private int firstOrdinalOfYear(int year) {
  return calcOrdinal(1, Month.JANUARY, year);
}

public static DayDate createInstance(Date date) {
  GregorianCalendar calendar = new GregorianCalendar();
  calendar.setTime(date);
  return new SpreadsheetDate(calendar.get(Calendar.DATE),
                             Month.fromInt(calendar.get(Calendar.MONTH)
  + 1),
                             calendar.get(Calendar.YEAR));

}
}
```

Phụ lục C

Tham chiếu chéo các nguyên tắc

Tham chiếu chéo giữa các mùi code và nguyên tắc kinh nghiệm. Mọi tham chiếu chéo khác đều có thể bỏ.

C1 ..... 16-276, 16-279, 17-292
C2 ..... 16-279, 16-285, 16-295, 17-292
C3 ..... 16-283, 16-285, 16-288, 17-293
C4 17-293
C5 17-293
E1 17-294
E2 17-294
F1 ..... 14-239, 17-295
F2 17-295
F3 17-295
F4 ..... 14-289, 16-273, 16-285, 16-287, 16-288, 17-295
G1 ..... 16-276, 17-295
G2 ..... 16-273, 16-274, 17-296
G3 ..... 16-274, 17-296
G4 ..... 9-31, 16-279, 16-286, 16-291, 17-297
G5 ..... 9-31, 16-279, 16-286, 16-291, 16-296, 17-297
G6 ..... 6-106, 16-280, 16-283, 16-284, 16-289, 16-293, 16-294, 16-296, 17-299
G7 ..... 16-281, 16-283, 17-300
G8 ..... 16-283, 17-301
G9 ..... 16-283, 16-285, 16-286, 16-287, 17-302
G10 ..... 5-86, 15-264, 16-276, 16-284, 17-302
G11 ..... 15-264, 16-284, 16-288, 16-292, 17-302
G12 ..... 16-284, 16-285, 16-286, 16-287, 16-288, 16-295, 17-303

G13 ..... 16-286, 16-288, 17-303
G14 ..... 16-288, 16-292, 17-304
G15 ..... 16-288, 17-305
G16 ..... 16-289, 17-306
G17 ..... 16-289, 17-307, 17-312
G18 ..... 16-289, 16-290, 16-291, 17-308
G19 ..... 16-290, 16-291, 16-292, 17-309
G20 ..... 16-290, 17-309
G21 ..... 16-291, 17-310
G22 ..... 16-294, 17-322
G23 ..... ??-44, 14-239, 16-295, 17-313
G24 ..... 16-296, 17-313
G25 ..... 16-296, 17-314
G26 17-316
G27 17-316
G28 ..... 15-262, 17-317
G29 ..... 15-262, 17-317
G30 ..... 15-263, 17-317
G31 ..... 15-264, 17-318
G32 ..... 15-265, 17-319
G33 ..... 15-265, 15-266, 17-320
G34 ..... 1-40, 6-106, 17-321
G35 ..... 5-90, 17-323
G36 ..... 6-103, 17-324
J1 ..... 16-276, 17-325
J2 ..... 16-278, 16-285, 17-326
J3 ..... 16-283, 16-285, 17-327
N1 ..... 15-264, 16-277, 16-279, 16-282, 16-287, 16-288, 16-289, 16-290, 16-294, 16-296, 17-328
N2 ..... 16-277, 17-330
N3 ..... 16-284, 16-288, 17-331
N4 ..... 15-263, 16-291, 17-332
N5 ..... 2-26, 14-221, 15-262, 17-332
N6 ..... 15-261, 17-333
N7 ..... 15-263, 17-333
T1 ..... 16-273, 16-274, 17-334
T2 ..... 16-273, 17-334
T3 ..... 16-274, 17-334
T4 17-334
T5 ..... 16-274, 16-275, 17-335
T6 ..... 16-275, 17-335

T7 ..... 16-275, 17-335
T8 ..... 16-275, 17-335
T9 17-336

Lời kết

Năm 2005, khi dự hội nghị Agile ở Denver, Elisabeth Hedrickson[1] đưa tôi một chiếc vòng đeo tay màu xanh lá, kiểu mà Lance Armstrong đã làm cho phổ biến. Trên vòng có dòng chữ "Test Obsessed". Tôi vui vẻ đeo vào và đeo với vẻ tự hào. Từ khi học TDD từ Kent Beck năm 1999, quả thật tôi đã trở nên ám ảnh với phát triển hướng kiểm thử.

Nhưng rồi một chuyện lạ xảy ra. Tôi nhận ra mình không thể tháo chiếc vòng ra. Không phải vì nó bị kẹt về mặt vật lý, mà vì nó bị kẹt về mặt đạo đức. Chiếc vòng đưa ra một tuyên bố công khai về đạo đức nghề nghiệp của tôi. Nó là dấu hiệu hữu hình cho cam kết của tôi về việc viết ra thứ mã tốt nhất mình có thể viết. Tháo nó ra có vẻ như phản bội lại những đạo đức đó và cam kết đó.

Thế nên nó vẫn ở trên cổ tay tôi. Khi viết code, tôi thấy nó ở đó trong tầm mắt ngoại vi. Nó là lời nhắc thường trực về lời hứa tôi đã tự đặt cho mình là viết mã sạch.



1. http://www.qualitytree.com/

Về bản dịch

Bản dịch này dựa trên ấn bản *Clean Code: A Handbook of Agile Software Craftsmanship* của Robert C. Martin, Pearson Education xuất bản năm 2009. Toàn bộ nội dung chính được dịch trọn vẹn: lời tựa, mười bảy chương, ba phụ lục và lời kết. Không có chương nào bị tóm tắt hay lược bớt.

Mã nguồn trong sách được giữ nguyên. Tên lớp, tên phương thức, tên biến, tên tệp, URL và tên riêng không bị dịch hay đổi dạng. Chú thích bên trong mã được dịch sang tiếng Việt để người đọc theo dõi mạch giải thích, trừ Phụ lục B: phần này in nguyên bản mã nguồn của `org.jfree.date.SerialDate` nên chú thích trong mã giữ nguyên tiếng Anh như bản gốc. Một vài lỗi chính tả và lỗi mã có sẵn trong bản in cũng được giữ lại, vì chúng là đối tượng được phân tích trong chính cuốn sách.

Các câu dẫn như "ở trang 177" hay "Đoạn mã B-1, trang 349" là số trang của ấn bản tiếng Anh. Bản dịch này được dàn trang lại nên số trang không còn trùng với ấn bản gốc. Bảng chỉ mục cuối sách cũng được lược bỏ, vì mục này chỉ có ích khi đi kèm đúng cách đánh số trang của bản tiếng Anh.

Một số thuật ngữ được dùng thống nhất trong toàn bộ bản dịch: *clean code* là "mã sạch", *code smell* là "mùi code", *heuristic* là "nguyên tắc kinh nghiệm", *refactoring* là "tái cấu trúc", *design pattern* là "mẫu thiết kế".

Bản dịch chỉ nhằm phục vụ mục đích học tập cá nhân. Bản quyền nội dung gốc thuộc Pearson Education và các tác giả.